
DATA SCIENCE

Série 1 — Volume II

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume II : Modélisation statistique

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	2
Vérifier son installation	3
Plan du cahier	3
Chapitre 1 : Régression linéaire — exercices et applications	4
Préparation	4
Applications	5
Application 1.1 — Retrouver les moindres carrés par la descente de gradient (section 1.1.3)	5
Application 1.2 — Le panier selon l'âge et le canal, pas à pas (sections 1.1.7 à 1.1.9)	5
Application 1.3 — Reconstruire les tableaux de <code>statsmodels</code> (sections 1.2.1 à 1.2.4)	8
Application 1.4 — Vérifier la théorie par simulation : couverture des intervalles et bootstrap (sections 1.2.5 et 1.2.6)	10
Application 1.5 — Le diagnostic complet de <code>m2</code> (section 1.3)	12
Application 1.6 — Le surajustement, puis huit modèles en concurrence (sections 1.4.1 à 1.4.4)	17
Application 1.7 — La sélection automatique fabrique des découvertes (section 1.4.5)	21
Application 1.8 — Ridge et Lasso à la main (sections 1.5.1 et 1.5.2)	23
Application 1.9 — Beaucoup de variables, peu de clients (section 1.5.4)	25
Application 1.10 — Huber à la main et données contaminées (sections 1.6.1 à 1.6.4)	27
Application 1.11 — La régression quantile (section 1.6.5)	30
Application 1.12 — Points relais : un modèle mixte de bout en bout (section 1.7)	31
Exercices	36
Exercice 1.1 ★ — Moindres carrés à la main (section 1.1)	37
Exercice 1.2 ★ — Lire un modèle en log (section 1.1)	37
Exercice 1.3 ★★ — Régression simple (section 1.1)	37
Exercice 1.4 ★★ — Test et intervalle à la main (section 1.2)	37
Exercice 1.5 ★★ — Test F (section 1.2)	37
Exercice 1.6 ★★ — Levier (section 1.3)	37
Exercice 1.7 ★★ — PRESS (section 1.4)	37
Exercice 1.8 ★★ — Multicolinéarité (section 1.3)	38
Exercice 1.9 ★★ — Prédire une moyenne en euros (section 1.1)	38
Exercice 1.10 ★★★ — Régression sur les ventes mensuelles (section 1.3)	38
Exercice 1.11 ★★★ — Ridge, cas orthonormal (section 1.5)	38
Exercice 1.12 ★★ — Modèle mixte, calcul à la main (section 1.7)	38

Exercice 1.13 ★★ — Robustesse, simulation (section 1.6)	38
Corrigés	39
Corrigé 1.1	39
Corrigé 1.2	39
Corrigé 1.3	40
Corrigé 1.4	40
Corrigé 1.5	40
Corrigé 1.6	41
Corrigé 1.7	41
Corrigé 1.8	41
Corrigé 1.9	42
Corrigé 1.10	43
Corrigé 1.11	44
Corrigé 1.12	44
Corrigé 1.13	44
Chapitre 2 : Modèles linéaires généralisés — exercices et applications	46
Préparation	46
Applications	47
Application 2.1 — L'algorithme IRLS écrit à la main	47
Application 2.2 — L'offre de bienvenue : du tableau croisé aux effets marginaux	49
Application 2.3 — Évaluer un modèle de classement	52
Application 2.4 — Comptages : Poisson, exposition, surdispersion	54
Application 2.5 — Montants : régression Gamma contre régression sur le logarithme	57
Application 2.6 — Vérifier un modèle : déviance, résidus, calibration, influence	59
Application 2.7 — Un GAM pour la durée des sessions	64
Application 2.8 — Zéros en excès, Tweedie et modèle à deux parties	66
Application 2.9 — La vérité dévoilée	70
Exercices	72
Exercice 2.1 ★ — Cotes et logit (section 2.2.1–2.2.2 du livre)	72
Exercice 2.2 ★ — Rapport de cotes, risque relatif (section 2.2.3 du livre)	72
Exercice 2.3 ★ — Régression de Poisson (section 2.3.1 du livre)	72
Exercice 2.4 ★★ — IRLS à la main (section 2.1.5 du livre)	72
Exercice 2.5 ★★ — Déviance et rapport de vraisemblance (section 2.4.1–2.4.2 du livre)	72
Exercice 2.6 ★★ — Décalage (section 2.3.2 du livre)	72
Exercice 2.7 ★★ — Surdispersion (section 2.3.3 du livre)	73
Exercice 2.8 ★★ — Régression sur $\log y$ (section 2.3.4 du livre)	73
Exercice 2.9 ★★ — Comparer des modèles (section 2.4.5 du livre)	73
Exercice 2.10 ★★ — Sur les données : la ville (section 2.4.2 du livre)	73
Exercice 2.11 ★★ — Choisir un seuil (section 2.2.8 du livre)	73
Exercice 2.12 ★★ — GAM : choisir la souplesse (section 2.5.2 du livre)	73
Exercice 2.13 ★★ — Zéros en excès (section 2.6.1 du livre)	73
Exercice 2.14 ★★ — Démonstration et Tweedie (section 2.1.3, 2.6.3 du livre)	73
Corrigés	74
Corrigé 2.1	74
Corrigé 2.2	74
Corrigé 2.3	74
Corrigé 2.4	75
Corrigé 2.5	75
Corrigé 2.6	76
Corrigé 2.7	76

Corrigé 2.8	77
Corrigé 2.9	77
Corrigé 2.10	77
Corrigé 2.11	78
Corrigé 2.12	78
Corrigé 2.13	79
Corrigé 2.14	79
Chapitre 3 : Analyse multivariée — exercices et applications	81
Applications	81
Application 3.1 — Un indice de satisfaction pour chaque répondante	81
Application 3.2 — Deux échelles de mesure et leur lien avec le comportement	82
Application 3.3 — Segmenter les clientes : k-means écrit à la main, choix de k, stabilité	83
Application 3.4 — Cartographier des associations : correspondances et ACM	86
Application 3.5 — Prédire le rachat avec l'analyse discriminante	88
Exercices	90
Exercice 3.1 ★ — ACP à la main (section 3.1 du livre)	90
Exercice 3.2 ★ — Lire un éboulis (section 3.1 du livre)	90
Exercice 3.3 ★ — Les unités (section 3.1 du livre)	90
Exercice 3.4 ★★ — Propriétés des scores (section 3.1 du livre)	90
Exercice 3.5 ★★ — Reconstruction (section 3.1 du livre)	90
Exercice 3.6 ★★ — Modèle factoriel à un facteur (section 3.2 du livre)	90
Exercice 3.7 ★★ — Invariance par rotation (section 3.2 du livre)	91
Exercice 3.8 ★★ — Alpha de Cronbach (section 3.2 du livre)	91
Exercice 3.9 ★★ — K-means : le piège du minimum local (section 3.3 du livre)	91
Exercice 3.10 ★★ — Silhouette (section 3.3 du livre)	91
Exercice 3.11 ★★ — Critères d'agrégation (section 3.3 du livre)	91
Exercice 3.12 ★★ — Une segmentation, ou pas ? (section 3.3 du livre)	91
Exercice 3.13 ★★ — Analyse discriminante (section 3.5 du livre)	92
Exercice 3.14 ★★ — Analyse des correspondances (section 3.4 du livre)	92
Corrigés	92
Corrigé 3.1	92
Corrigé 3.2	93
Corrigé 3.3	93
Corrigé 3.4	94
Corrigé 3.5	94
Corrigé 3.6	95
Corrigé 3.7	95
Corrigé 3.8	96
Corrigé 3.9	96
Corrigé 3.10	96
Corrigé 3.11	97
Corrigé 3.12	97
Corrigé 3.13	98
Corrigé 3.14	99
Chapitre 4 : Séries temporelles — exercices et applications	100
Applications	100
Application 4.1 — Les ventes de 2026	101
Application 4.2 — Le filtre de Kalman en numpy	102
Application 4.3 — Un modèle « à la Prophet » écrit à la main	104
Application 4.4 — La validation à origine glissante	105

Exercices	106
Exercice 4.1 ★ — Autocorrélation à la main (section 4.1.4 du livre)	106
Exercice 4.2 ★ — Stationnaire ou non ? (section 4.1.3 du livre)	107
Exercice 4.5 ★★ — Ljung-Box à la main (section 4.1.5 du livre)	107
Exercice 4.6 ★★ — Du logarithme aux euros (section 4.3.2 du livre)	107
Exercice 4.7 ★★ — MASE et MAPE (section 4.3.4 du livre)	107
Exercice 4.8 ★★ — Identifier un modèle (section 4.2.1 et 4.2.4 du livre)	107
Exercice 4.9 ★★ ★ — La sur-différenciation, en théorie (section 4.1.6 du livre)	108
Exercice 4.10 ★★ ★ — Filtre de Kalman à la main (section 4.5.2 du livre)	108
Exercice 4.11 ★★ ★ — Régression fallacieuse et remède (section 4.4.2 du livre)	108
Exercice 4.12 ★★ — GARCH à la main (section 4.4.3 du livre)	108
Exercice 4.13 ★★ ★ — Le concours complet sur une autre série (section 4.3 du livre)	108
Corrigés	109
Corrigé 4.1	109
Corrigé 4.2	109
Corrigé 4.3	110
Corrigé 4.4	110
Corrigé 4.5	111
Corrigé 4.6	111
Corrigé 4.7	111
Corrigé 4.8	112
Corrigé 4.9	113
Corrigé 4.10	113
Corrigé 4.11	114
Corrigé 4.12	114
Corrigé 4.13	114
Chapitre 5 : Analyse de survie — exercices et applications	116
Préparation	116
Applications	117
Application 5.1 — L'expérience de la censure	117
Application 5.2 — Kaplan-Meier écrit à la main	118
Application 5.3 — Comparer des groupes, et le piège de l'entrée tardive	120
Application 5.4 — Le modèle de Cox à la main	122
Application 5.5 — Modèles paramétriques et valeur vie client	124
Application 5.6 — Risques concurrents	126
Exercices	128
Exercice 5.1 ★ — Censure et Kaplan-Meier à la main (sections 5.1, 5.2 du livre)	128
Exercice 5.2 ★ — Relations entre les fonctions (section 5.1 du livre)	128
Exercice 5.3 ★ — Taux constant (section 5.1 du livre)	128
Exercice 5.4 ★★ — Greenwood (section 5.2 du livre)	128
Exercice 5.5 ★★ — Log-rank (section 5.2 du livre)	129
Exercice 5.6 ★★ — Durée moyenne restreinte (section 5.2 du livre)	129
Exercice 5.7 ★★ — Lire une sortie de Cox (section 5.3 du livre)	129
Exercice 5.8 ★★ — Le biais d'immortalité (section 5.3 du livre)	129
Exercice 5.9 ★★ — Estimer la forme de Weibull de deux façons (section 5.4 du livre)	129
Exercice 5.10 ★★ ★ — Log-rank et Cox (section 5.3 du livre)	129
Exercice 5.11 ★★ ★ — Décider : le seuil de rentabilité (section 5.4 du livre)	129
Exercice 5.12 ★★ ★ — Risques concurrents à la main (section 5.5 du livre)	130
Exercice 5.13 ★★ ★ — Proportionnalité des risques sur des données réelles (section 5.3 du livre)	130
Corrigés	130

Corrigé 5.1	130
Corrigé 5.2	130
Corrigé 5.3	131
Corrigé 5.4	131
Corrigé 5.5	132
Corrigé 5.6	132
Corrigé 5.7	133
Corrigé 5.8	133
Corrigé 5.9	134
Corrigé 5.10	134
Corrigé 5.11	135
Corrigé 5.12	135
Corrigé 5.13	136
Chapitre 6 : Statistique bayésienne et simulation — exercices et applications	137
Applications	137
Préparation (à exécuter d'abord)	137
Application 6.1 — L'offre de bienvenue fonctionne-t-elle ? (section 6.1)	138
Application 6.2 — Faut-il envoyer l'offre à tout le monde ? (section 6.2)	141
Application 6.3 — La régression logistique bayésienne du rachat (section 6.3)	142
Application 6.4 — Vérifier un modèle : Poisson contre binomiale négative (section 6.4)	144
Application 6.5 — Niveaux de retour des retards de livraison (section 6.5)	145
Application 6.6 — Deux transporteurs en même temps (section 6.6)	147
Exercices	149
Exercice 6.1 ★ — Bêta-binomial (section 6.1)	149
Exercice 6.2 ★ — Gamma-Poisson (section 6.1)	149
Exercice 6.3 ★★ — Normal-normal (section 6.1)	149
Exercice 6.4 ★★ — Test A/B bayésien (section 6.1)	149
Exercice 6.5 ★ — Monte-Carlo (section 6.2)	149
Exercice 6.6 ★★ — Échantillonnage préférentiel (section 6.2)	149
Exercice 6.7 ★★ — Inversion (section 6.2)	150
Exercice 6.8 ★★ — Metropolis à la main (section 6.3)	150
Exercice 6.9 ★★ — Metropolis sur une échelle logarithmique (section 6.3)	150
Exercice 6.10 ★★ — Diagnostics (section 6.4)	150
Exercice 6.11 ★★ — Vérification prédictive (section 6.4)	150
Exercice 6.12 ★★ — Facteur de Bayes (section 6.4)	150
Exercice 6.13 ★★ — Valeurs extrêmes (section 6.5)	150
Exercice 6.14 ★★ — Dépendance de queue de Clayton (section 6.6)	150
Corrigés	151
Corrigé 6.1	151
Corrigé 6.2	151
Corrigé 6.3	151
Corrigé 6.4	152
Corrigé 6.5	152
Corrigé 6.6	153
Corrigé 6.7	153
Corrigé 6.8	153
Corrigé 6.9	154
Corrigé 6.10	154
Corrigé 6.11	155
Corrigé 6.12	155

Corrigé 6.13	156
Corrigé 6.14	156
Chapitre 7 : ☉ Inférence causale — exercices et applications	158
Applications	158
Préparation commune	158
Application 7.1 — Les résultats potentiels sur huit clients (section 7.1.2)	158
Application 7.2 — La randomisation, exhaustivement puis sur 4 000 clients (section 7.1.3)	160
Application 7.3 — Analyser l'expérience de l'offre de bienvenue (section 7.1.4)	161
Application 7.4 — Simpson, médiateur et collision : simuler les trois pièges (sections 7.1.6 à 7.1.8)	163
Application 7.5 — L'échelle d'ajustement sur l'étude observationnelle (section 7.1.9)	165
Application 7.6 — Le score de propension : estimation, chevauchement, appariement (sections 7.2.1 à 7.2.3)	166
Application 7.7 — Pondération, estimateur doublement robuste et confusion mal mesurée (sections 7.2.4 à 7.2.6)	168
Application 7.8 — La différence de différences sur le panel de villes (section 7.3)	171
Application 7.9 — La variable instrumentale : Wald, 2SLS, complaisants, instrument faible (section 7.4)	174
Exercices	178
Exercice 7.1 ★ — résultats potentiels (section 7.1.2 du livre)	178
Exercice 7.2 ★ — sous-groupes (section 7.1.4 du livre)	178
Exercice 7.3 ★ — Simpson (section 7.1.6 du livre)	178
Exercice 7.4 ★★ — choisir l'ensemble d'ajustement (sections 7.1.7 à 7.1.9 du livre)	179
Exercice 7.5 ★★ — stratification sur le score (section 7.2.4 du livre)	179
Exercice 7.6 ★★ — chevauchement (section 7.2.2 du livre)	179
Exercice 7.7 ★★ — DiD à la main (section 7.3.1 du livre)	179
Exercice 7.8 ★★ — inférence avec peu de groupes (section 7.3.3 du livre)	179
Exercice 7.9 ★★ — instrument à la main (section 7.4.3 du livre)	179
Exercice 7.10 ★★ — démonstration (section 7.4.3 du livre)	179
Exercice 7.11 ★★ — médiation (section 7.1.7 du livre)	180
Exercice 7.12 ★★ — critique d'une étude (section 7.4.7 du livre)	180
Corrigés	180
Corrigé 7.1	180
Corrigé 7.2	180
Corrigé 7.3	181
Corrigé 7.4	182
Corrigé 7.5	182
Corrigé 7.6	182
Corrigé 7.7	183
Corrigé 7.8	183
Corrigé 7.9	184
Corrigé 7.10	184
Corrigé 7.11	185
Corrigé 7.12	185
Chapitre 8 : Plans d'expériences — exercices et applications	186
Applications	186
Application 8.1 — Randomiser, répéter, comparer avec un plan « un facteur à la fois »	186
Application 8.2 — ANOVA à un facteur sur les vitrines	188

Application 8.3 — Vérifier les hypothèses, comparer les groupes, mesurer l'effet . . .	190
Application 8.4 — Les blocs : combien de bruit retire-t-on ?	192
Application 8.5 — Deux facteurs et leur interaction	194
Application 8.6 — Dimensionner une expérience : la puissance d'une ANOVA . . .	195
Application 8.7 — Un plan factoriel 2^3 de A à Z	196
Application 8.8 — Un plan 2^4 sans répétition : Lenth et le diagramme demi-normal	199
Application 8.9 — Plans fractionnaires : alias, résolution, repliement	200
Application 8.10 — Surface de réponse : trouver le meilleur réglage du four	203
Application 8.11 — Un plan D-optimal par algorithme d'échange	205
Exercices	206
Exercice 8.1 ★ — Concevoir une expérience (section 8.1 du livre)	206
Exercice 8.2 ★ — ANOVA à la main (section 8.2 du livre)	206
Exercice 8.3 ★★ — Taille d'effet (section 8.2 du livre)	206
Exercice 8.4 ★★ — Comparaisons multiples (section 8.2 du livre)	207
Exercice 8.5 ★★ — Blocs (section 8.2 du livre)	207
Exercice 8.6 ★★ — Interaction (section 8.3 du livre)	207
Exercice 8.7 ★★ — Algorithme de Yates (section 8.3 du livre)	207
Exercice 8.8 ★★★ — Puissance d'un plan factoriel (section 8.3 du livre)	207
Exercice 8.9 ★★ — Confusion (section 8.4 du livre)	207
Exercice 8.10 ★★ — Un plan 2^{6-2} (section 8.4 du livre)	207
Exercice 8.11 ★★ — Surface de réponse (section 8.4 du livre)	208
Exercice 8.12 ★★★ — Simuler l'effet des blocs (section 8.2 du livre)	208
Exercice 8.13 ★★ — Méthode de Lenth (section 8.3 du livre)	208
Corrigés	208
Corrigé 8.1	208
Corrigé 8.2	208
Corrigé 8.3	209
Corrigé 8.4	209
Corrigé 8.5	210
Corrigé 8.6	210
Corrigé 8.7	210
Corrigé 8.8	211
Corrigé 8.9	211
Corrigé 8.10	212
Corrigé 8.11	212
Corrigé 8.12	212
Corrigé 8.13	213
Chapitre 9 : ⊕ Statistique spatiale — exercices et applications	214
Applications	214
Application 9.1 — Distances et cartes (section 9.1)	214
Application 9.2 — Les ventes par zone : Moran, Geary et indices locaux (section 9.2)	217
Application 9.3 — Quand l'espace trompe la régression (section 9.2.5)	223
Application 9.4 — Du variogramme au krigeage : les délais de livraison (section 9.3)	224
Application 9.5 — Les limites du variogramme : tendance et anisotropie (section 9.3.7)	230
Application 9.6 — Trois semis de points : quadrats, plus proche voisin, fonction K (section 9.4)	232
Application 9.7 — Agrégat ou densité variable ? (section 9.4.5)	236
Exercices	237
Exercice 9.1 ★ — quel type de données ? (section 9.1.2 du livre)	237
Exercice 9.2 ★ — haversine à la main (section 9.1.3 du livre)	238

Exercice 9.4 ★★ — lire un indice local (section 9.2.4 du livre)	238
Exercice 9.5 ★★ — variogramme à la main (section 9.3.2 du livre)	238
Exercice 9.6 ★★ — krigeage à la main (section 9.3.4 du livre)	238
Exercice 9.7 ★★ — le problème de l'unité spatiale modifiable (section 9.1.4 du livre)	238
Exercice 9.8 ★★ — l'effet de bord sur les indices locaux (section 9.2.4 du livre)	238
Exercice 9.9 ★★ — nombre efficace d'observations (section 9.2.5 du livre)	239
Exercice 9.10 ★★ — cases vides (section 9.4.2 du livre)	239
Exercice 9.11 ★★★ — l'estimateur du variogramme est-il sans biais ? (section 9.3.2 du livre)	239
Exercice 9.12 ★★★ — la fonction K d'un processus de Thomas (section 9.4.4 du livre)	239
Corrigés	239
Corrigé 9.1	239
Corrigé 9.2	239
Corrigé 9.3	240
Corrigé 9.4	240
Corrigé 9.5	241
Corrigé 9.6	241
Corrigé 9.7	241
Corrigé 9.8	242
Corrigé 9.9	243
Corrigé 9.10	243
Corrigé 9.11	244
Corrigé 9.12	245
Projet du volume et auto-évaluation — exercices et applications	247
Projet du volume	247
P.1 Le cahier des charges	247
P.2 Étape 1 : charger et contrôler	248
P.3 Étape 2 : prévoir les ventes de 2026 (question 1)	249
P.4 Étape 3 : l'offre de bienvenue fait-elle revenir les clients ? (question 2)	252
P.5 Étape 4 : combien rapporte un client ? (question 3)	254
P.6 Étape 5 : combien de temps un client reste-t-il ? (question 4)	255
P.7 Étape 6 : que vaut un client, et l'offre est-elle rentable ?	256
P.8 Le rapport pour la gérante, et ce qui avait été programmé	257
P.9 Un détour par de vraies données	259
P.10 Limites, et la suite	260
Auto-évaluation	260
Régression linéaire (chapitre 1)	260
Modèles linéaires généralisés (chapitre 2)	261
Analyse multivariée (chapitre 3)	261
Séries temporelles (chapitre 4)	261
Analyse de survie (chapitre 5)	261
Statistique bayésienne et simulation (chapitre 6)	262
Chapitres facultatifs (7, 8, 9)	262
Corrigés des questions	262
Vérification des réponses chiffrées	262
Réponses	264
Grille d'auto-évaluation	266

Mode d'emploi

« On comprend en lisant, on retient en pratiquant. »

Ce cahier est le **compagnon du livre** du volume II (*Modélisation statistique*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées** sur données, des **exercices** et leurs **corrigés**, puis le **projet du volume** et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 2 : application 2.1, exercices 2.1 à 2.4.
2. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre : on peut ainsi chercher sans voir la réponse.
3. **Tapez le code vous-même** plutôt que de le copier : modifiez-le, cassez-le, lisez les messages d'erreur. C'est la seule façon d'apprendre à programmer.
4. **Faites les applications dans l'ordre** : chacune raconte une petite étude, avec une question, des étapes, du code et une lecture des résultats.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une formule ou d'une idée vue dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, une démonstration ou une petite simulation

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/` :

Fichier	Contenu
<code>clients.csv</code>	2 000 clients : âge, ville, canal d'acquisition, offre de bienvenue (tirée au sort), commandes, dépense, rachat, durée de la relation
<code>enquete_satisfaction.csv</code>	huit questions de satisfaction pour 1 212 répondants
<code>ventes_mensuelles.csv</code>	chiffre d'affaires mensuel, 2016 à 2025

Certains chapitres ajoutent leurs propres jeux (par exemple `ch07-*.csv`) : ils sont fournis, et le script qui les produit se trouve dans `build/`. Toutes les données sont **simulées**, avec des graines fixes : vos résultats seront identiques à ceux du livre.

Chaque chapitre du cahier est **autonome** : il commence par recharger ses données et refaire ses imports, de sorte qu'on peut le faire sans avoir exécuté rien d'autre. Les installations nécessaires sont décrites à la fin du livre, section « L'environnement de travail ».

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/`), car les chemins sont relatifs (`donnees/clients.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, ou dans un terminal interactif. Les blocs de R se lancent depuis le même dossier.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Vérifier son installation

Avant de commencer, ce court bloc vérifie que les données sont trouvées et que les bibliothèques principales sont installées.

```
import pandas as pd, numpy as np, scipy, statsmodels, sklearn
clients = pd.read_csv("donnees/clients.csv")
print(clients.shape)
print("statsmodels", statsmodels.__version__)
```

```
(2000, 12)
statsmodels 0.15.0
```

Vous devez lire (2000, 12), puis une version de statsmodels. Si le chargement échoue, relisez l'encadré précédent ; si une bibliothèque manque, installez-la avec `pip install statsmodels scikit-learn` (voir le livre).

Plan du cahier

Chapitre	Contenu
1. Régression linéaire	applications et exercices sur les moindres carrés, l'inférence, les diagnostics, la sélection de variables, la régularisation
2. Modèles linéaires généralisés	logistique, comptages, montants positifs, déviance
3. Analyse multivariée	ACP, analyse factorielle, classification
4. Séries temporelles	stationnarité, ARIMA, prévision
5. Analyse de survie	Kaplan-Meier, Cox, modèles paramétriques
6. Statistique bayésienne et simulation	lois a priori, Monte-Carlo, MCMC
7 à 9 (facultatifs)	inférence causale, plans d'expériences, statistique spatiale
Projet et auto-évaluation	le plan de l'année suivante de la boutique, puis 42 questions pour vérifier ses acquis

Chapitre 1 : Régression linéaire — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre. Les **applications** reprennent, pas à pas et avec le code, les études que le livre résume ; les **exercices** (★ application directe, ★★ raisonnement, ★★★ synthèse) sont corrigés à la fin du chapitre. Les données (donnees/clients.csv, donnees/enquete_satisfaction.csv, donnees/ventes_mensuelles.csv, donnees/ch01-relais.csv) sont simulées avec des graines fixes. Prérequis : le chapitre 1 du livre, et le chapitre 1 de ce cahier exécuté dans l'ordre (les blocs de code partagent les mêmes variables).

Préparation

Tout le chapitre part du même tableau : les 1 740 clients qui ont passé au moins une commande, avec le log de leur panier moyen, et les trois modèles m1 (âge), m2 (âge et canal) et m3 (avec interaction) du livre.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats

clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()           # les clients actifs
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Réseaux"])
df["a"] = df["age"] - 36                                     # âge centré sur 36 ans
df["log_panier"] = np.log(df["panier_moyen"])

m1 = smf.ols("log_panier ~ a", data=df).fit()
m2 = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
m3 = smf.ols("log_panier ~ a * C(canal)", data=df).fit()
print(len(df), "clients actifs")
```

1740 clients actifs

Figures. Le code qui trace les figures est repris dans les sources du dépôt (blocs marqués hide des fichiers cahier/01-exercices.md et sections/01-*.md) : il est exécuté à chaque vérification du livre, mais il n'est pas imprimé ici, pour ne garder que l'essentiel.

Applications

Application 1.1 — Retrouver les moindres carrés par la descente de gradient (section 1.1.3)

Objectif. Vérifier que la droite des quatre commandes (1, 22), (2, 41), (3, 66), (4, 79) s'obtient aussi en *descendant* la fonction $S(\beta) = \|y - X\beta\|^2$ pas à pas, comme au volume I (section 1.3.3). Le gradient est $-2X^T(y - X\beta)$; on le moyenne sur n pour garder un pas raisonnable.

Étape 1 — la solution exacte. C'est celle de la section 1.1.1 du livre : (3 ; 19,6).

```
x = np.array([1, 2, 3, 4]); y = np.array([22, 41, 66, 79])
X = np.column_stack([np.ones(4), x])
print("équations normales :", np.linalg.solve(X.T @ X, X.T @ y))
```

```
équations normales : [ 3. 19.6]
```

Étape 2 — la descente, avec x centré. Centrer x (lui soustraire sa moyenne) rend les deux directions de $X^T X$ orthogonales et accélère la convergence ; on retrouve ensuite la constante d'origine en retranchant $\hat{\beta}_1 \bar{x}$.

```
xc = x - x.mean() # x centré : -1,5 ; -0,5 ; 0,5 ; 1,5
Xc = np.column_stack([np.ones(4), xc])
b = np.zeros(2)
pas = 0.1 # taux d'apprentissage
for it in range(200):
    gradient = -2 / len(y) * Xc.T @ (y - Xc @ b) # gradient de la moyenne des carrés
    b = b - pas * gradient
print("descente de gradient (x centré) :", b.round(4))
print("solution exacte :", np.linalg.solve(Xc.T @ Xc, Xc.T @ y).round(4))
print("retour à l'échelle d'origine : constante =", round(b[0] - b[1] * x.mean(), 3), "| pente =",
      round(b[1], 3))
```

```
descente de gradient (x centré) : [52. 19.6]
solution exacte : [52. 19.6]
retour à l'échelle d'origine : constante = 3.0 | pente = 19.6
```

Pour aller plus loin. Remplacez `pas = 0.1` par 0,3 puis par 0,5, puis supprimez le centrage : que se passe-t-il, et pourquoi ? (Indice : les valeurs propres de $X^T X/n$ fixent le pas maximal qui garantit la convergence.)

Application 1.2 — Le panier selon l'âge et le canal, pas à pas (sections 1.1.7 à 1.1.9)

Objectif. Refaire l'étude de la section 1.1.7 : décrire le panier, ajuster m_1, m_2 et m_3 , lire les coefficients (y compris en pourcentage), retrouver « toutes choses égales par ailleurs » avec le théorème de Frisch-Waugh-Lovell, et prédire un panier moyen en euros.

Étape 1 — regarder le panier. Une variable de montants est presque toujours asymétrique ; le log la symétrise.

```
print("clients sans aucune commande :", int((clients["nb_commandes_an"] == 0).sum()), "sur",
      len(clients))
print("asymétrie du panier :", round(df["panier_moyen"].skew(), 2), "| du log du panier :",
      round(df["log_panier"].skew(), 2))
```

```
clients sans aucune commande : 260 sur 2000
asymétrie du panier : 1.44 | du log du panier : 0.11
```

Étape 2 — deux modèles. Lisez la formule : $\log_panier \sim a + C(\text{canal})$ signifie « log-panier

expliqué par l'âge centré et par le canal (variable qualitative) ». statsmodels fabrique la constante et les indicatrices.

```
print(m1.summary().tables[1])
print(m2.summary().tables[1])
print("R² :", round(m1.rsquared, 4), "->", round(m2.rsquared, 4), "| R² ajusté de m2 :",
      ↪ round(m2.rsquared_adj, 4))
print(m2.model.exog[:6].astype(int), m2.model.exog_names)      # les six premières lignes de la
      ↪ matrice X
```

```
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4.0331	0.009	426.820	0.000	4.015	4.052
a	0.0090	0.001	10.050	0.000	0.007	0.011

```
=====
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4.2206	0.017	241.369	0.000	4.186	4.255
C(canal)[T.Site]	-0.1571	0.023	-6.807	0.000	-0.202	-0.112
C(canal)[T.Réseaux]	-0.3368	0.022	-14.977	0.000	-0.381	-0.293
a	0.0092	0.001	10.862	0.000	0.008	0.011

```
=====
R² : 0.0549 -> 0.1657 | R² ajusté de m2 : 0.1643
[[ 1  0  1 -17]
 [ 1  1  0  7]
 [ 1  0  0 -1]
 [ 1  0  1 -15]
 [ 1  1  0 -6]
 [ 1  0  1 -10]] ['Intercept', 'C(canal)[T.Site]', 'C(canal)[T.Réseaux]', 'a']
```

Questions. Que représente la constante de m2 ? Combien vaut le panier **médian** d'un client de 36 ans acquis en boutique ?

Étape 3 — lire un coefficient en pourcentage. L'approximation « $100\beta\%$ » n'est correcte que pour de petits coefficients ; la variation exacte est $100(e^\beta - 1)\%$.

```
for nom, coef in m2.params.items():
    if nom == "Intercept":
        continue
    print(f"{nom:22s} coefficient = {coef:+.3f} | approximation 100*beta = {100*coef:+.1f} % | exact
      ↪ 100*(exp(beta)-1) = {100*(np.exp(coef)-1):+.1f} %")
```

```
C(canal)[T.Site]      coefficient = -0.157 | approximation 100*beta = -15.7 % | exact
      ↪ 100*(exp(beta)-1) = -14.5 %
C(canal)[T.Réseaux]   coefficient = -0.337 | approximation 100*beta = -33.7 % | exact
      ↪ 100*(exp(beta)-1) = -28.6 %
a                     coefficient = +0.009 | approximation 100*beta = +0.9 % | exact 100*(exp(beta)-1)
      ↪ = +0.9 %
```

Étape 4 — Frisch-Waugh-Lovell. Le coefficient de l'âge dans m2 est la pente entre deux résidus : celui du log-panier et celui de l'âge, une fois le canal « retiré » de l'un et de l'autre.

```
res_y = smf.ols("log_panier ~ C(canal)", data=df).fit().resid      # log-panier, une fois le canal «
      ↪ retiré »
res_a = smf.ols("a ~ C(canal)", data=df).fit().resid              # âge, une fois le canal «
      ↪ retiré »
pente_fwl = (res_a @ res_y) / (res_a @ res_a)
print("coefficient de l'âge dans le modèle complet :", round(m2.params["a"], 6))
print("pente obtenue par Frisch-Waugh-Lovell      :", round(pente_fwl, 6))
```

```
coefficient de l'âge dans le modèle complet : 0.00917
pente obtenue par Frisch-Waugh-Lovell      : 0.00917
```

Étape 5 — calculer à la main ce que fait statsmodels. Les coefficients sont la solution des équations normales sur la matrice X que le logiciel a construite ; le nombre de conditionnement n'est grand que parce que l'âge centré et les indicatrices ont des échelles très différentes.

```
Xd = m2.model.exog          # la matrice de plan d'expérience (n x p)
yd = m2.model.endog
beta_main = np.linalg.solve(Xd.T @ Xd, Xd.T @ yd)
print(pd.DataFrame({"à la main": beta_main, "statsmodels": m2.params.to_numpy()},
  ↳ index=m2.params.index).round(6))
print("n =", Xd.shape[0], "| p =", Xd.shape[1])
print("nombre de conditionnement de X'X :", round(np.linalg.cond(Xd.T @ Xd), 1))
```

	à la main	statsmodels
Intercept	4.220621	4.220621
C(canal)[T.Site]	-0.157139	-0.157139
C(canal)[T.Réseaux]	-0.336759	-0.336759
a	0.009170	0.009170

n = 1740 | p = 4
nombre de conditionnement de X'X : 1501.8

Étape 6 — prédire une moyenne en euros. Le modèle prédit un log-panier ; $e^{\hat{\mu}}$ est la prédiction de la médiane. Pour la moyenne, il faut le facteur $e^{s^2/2}$ (loi normale) ou le facteur de Duan $\frac{1}{n} \sum e^{\hat{\epsilon}_i}$.

```
cible = df[(df["canal"] == "Boutique") & (df["age"].between(33, 39))] # clients de la
  ↳ boutique, 33-39 ans
mu_hat = m2.predict(pd.DataFrame({"a": [0], "canal": pd.Categorical(["Boutique"]),
  ↳ categories=["Boutique", "Site", "Réseaux"]})).iloc[0]
s2 = m2.scale
duan = np.exp(m2.resid).mean()
print("panier moyen observé (clients boutique, 33-39 ans) :", round(cible["panier_moyen"].mean(), 2),
  ↳ "€ (n =", len(cible), ")")
print("exp(mu chapeau) [médiane prédite] :", round(np.exp(mu_hat), 2), "€")
print("exp(mu chapeau + s^2/2) [moyenne, normale] :", round(np.exp(mu_hat + s2 / 2), 2), "€")
print("exp(mu chapeau) x facteur de Duan [moyenne, libre] :", round(np.exp(mu_hat) * duan, 2), "€")
```

panier moyen observé (clients boutique, 33-39 ans) :	74.33 €	(n = 110)
exp(mu chapeau) [médiane prédite] :	68.08 €	
exp(mu chapeau + s ² /2) [moyenne, normale] :	72.91 €	
exp(mu chapeau) x facteur de Duan [moyenne, libre] :	72.96 €	

Étape 7 — l'interaction âge × canal. Les deux lignes a:C(canal)[...] mesurent la différence de pente de l'âge par rapport à la boutique ; elles sont indiscernables de zéro.

```
print(m3.summary().tables[1])
```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	4.2205	0.017	241.183	0.000	4.186	4.255
C(canal)[T.Site]	-0.1568	0.023	-6.788	0.000	-0.202	-0.112
C(canal)[T.Réseaux]	-0.3366	0.022	-14.961	0.000	-0.381	-0.292
a	0.0086	0.002	5.115	0.000	0.005	0.012
a:C(canal)[T.Site]	0.0010	0.002	0.463	0.643	-0.003	0.005
a:C(canal)[T.Réseaux]	0.0005	0.002	0.235	0.814	-0.004	0.005

Application 1.3 — Reconstruire les tableaux de statsmodels (sections 1.2.1 à 1.2.4)

Objectif. Retrouver à la main le tableau de `m2` (erreurs standard, t , p -valeurs, intervalles), tester une combinaison de coefficients, comparer deux modèles par un test F et construire un intervalle de prédiction.

Étape 1 — le tableau complet. On calcule $s^2 = \text{SCR}/(n - p)$, l'erreur standard $s\sqrt{c_{jj}}$, $t = \hat{\beta}_j/\text{se}$, la p -valeur et l'intervalle $\hat{\beta}_j \pm t_{n-p, 0.975} \text{ se}$.

```
X = m2.model.exog
y = m2.model.endog
n, p = X.shape
XtX_inv = np.linalg.inv(X.T @ X)
beta = XtX_inv @ X.T @ y
residus = y - X @ beta
s2 = residus @ residus / (n - p)           # estimation de sigma^2
se = np.sqrt(s2 * np.diag(XtX_inv))         # erreurs standard
t = beta / se                               # statistiques t pour H0 : beta_j = 0
pval = 2 * stats.t.sf(np.abs(t), df=n - p)   # p-valeur bilatérale
crit = stats.t.ppf(0.975, df=n - p)         # quantile de Student à 97,5 %
tab = pd.DataFrame({"coef": beta, "std err": se, "t": t, "P>|t|": pval,
                    "IC bas": beta - crit * se, "IC haut": beta + crit * se}, index=m2.params.index)

print(tab.round(4))
print()
print("identique à statsmodels :", np.allclose(tab["std err"], m2.bse), np.allclose(tab["t"],
↪ m2.tvalues),
      np.allclose(tab[["IC bas", "IC haut"]].to_numpy(), m2.conf_int().to_numpy()))
print(f"s = {np.sqrt(s2):.4f} | quantile t(n-p, 97,5 %) = {crit:.4f} (presque 1,96 : n - p = {n - p}
↪ est grand)")
```

	coef	std err	t	P> t	IC bas	IC haut
Intercept	4.2206	0.0175	241.3693	0.0	4.1863	4.2549
C(canal)[T.Site]	-0.1571	0.0231	-6.8065	0.0	-0.2024	-0.1119
C(canal)[T.Réseaux]	-0.3368	0.0225	-14.9770	0.0	-0.3809	-0.2927
a	0.0092	0.0008	10.8618	0.0	0.0075	0.0108

```
identique à statsmodels : True True True
s = 0.3705 | quantile t(n-p, 97,5 %) = 1.9613 (presque 1,96 : n - p = 1736 est grand)
```

Étape 2 — un test sur une combinaison. « Le Site et Réseaux ont-ils le même panier, à âge égal ? » : $H_0 : \beta_{\text{Site}} - \beta_{\text{Réseaux}} = 0$; la variance de $\mathbf{c}^T \hat{\beta}$ est $\sigma^2 \mathbf{c}^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{c}$.

```
c = np.array([0, 1, -1, 0])
diff = c @ beta
se_diff = np.sqrt(s2 * c @ XtX_inv @ c)
print(f"beta_Site - beta_Reseaux = {diff:.4f} | erreur standard = {se_diff:.4f} | t =
↪ {diff/se_diff:.2f} | p = {2*stats.t.sf(abs(diff/se_diff), n-p):.2e}")
print(m2.t_test("C(canal)[T.Site] - C(canal)[T.Réseaux] = 0"))
```

```
beta_Site - beta_Reseaux = 0.1796 | erreur standard = 0.0207 | t = 8.69 | p = 8.16e-18
Test for Constraints
```

	coef	std err	t	P> t	[0.025	0.975]
c0	0.1796	0.021	8.691	0.000	0.139	0.220

Étape 3 — le test F . Le canal compte-t-il (deux coefficients à la fois) ? Puis les autres usages du test F : interaction, cas $q = 1$, test global.

```
# Le canal compte-t-il ? M0 : log_panier ~ a (m1) contre M1 : log_panier ~ a + canal (m2)
scr0, scr1 = m1.ssr, m2.ssr
q = int(m2.df_model - m1.df_model)
F = ((scr0 - scr1) / q) / (scr1 / m2.df_resid)
pF = stats.f.sf(F, q, m2.df_resid)
print(f"SCR0 = {scr0:.2f} | SCR1 = {scr1:.2f} | q = {q} | F = {F:.2f} | p = {pF:.2e}")
print()
print(sm.stats.anova_lm(m1, m2).round(4))
```

```
SCR0 = 269.94 | SCR1 = 238.30 | q = 2 | F = 115.26 | p = 9.98e-48
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	1738.0	269.9406	0.0	NaN	NaN	NaN
1	1736.0	238.2975	2.0	31.6431	115.26	0.0

```
print("Interaction âge x canal (m2 contre m3) :")
print(sm.stats.anova_lm(m2, m3).round(4))
print()
a_t = m2.tvalues["a"]
print("cas q = 1 : t² de l'âge =", round(a_t**2, 3), "| F de la suppression de l'âge =",
      ↪ round(sm.stats.anova_lm(sm.ols("log_panier ~ C(canal)", df).fit(), m2).loc[1, "F"], 3))
print()
print("test global (m2) : F =", round(m2.fvalue, 2), "| p =", f"{m2.f_pvalue:.2e}")
```

```
Interaction âge x canal (m2 contre m3) :
df_resid      ssr df_diff ss_diff      F Pr(>F)
0    1736.0    238.2975     0.0      NaN      NaN      NaN
1    1734.0    238.2677     2.0    0.0299    0.1087    0.897
```

```
cas q = 1 : t² de l'âge = 117.979 | F de la suppression de l'âge = 117.979
```

```
test global (m2) : F = 114.93 | p = 6.88e-68
```

Étape 4 — un intervalle de prédiction à la main. Sur les quatre commandes de la section 1.1.1, quel montant prévoir pour 5 articles ? On compare la formule $\hat{y}_0 \pm ts\sqrt{h_0}$ (moyenne) et $\hat{y}_0 \pm ts\sqrt{1+h_0}$ (prédiction) au résultat de statsmodels.

```
x4 = np.array([1, 2, 3, 4]); y4 = np.array([22, 41, 66, 79])
d4 = pd.DataFrame({"x": x4, "y": y4})
mp = smf.ols("y ~ x", d4).fit()
nouveau = pd.DataFrame({"x": [5]})
pred = mp.get_prediction(nouveau).summary_frame(alpha=0.05)
print(pred.round(2).to_string())
x0 = np.array([1, 5]); X4 = np.column_stack([np.ones(4), x4])
h0 = x0 @ np.linalg.inv(X4.T @ X4) @ x0
s2_4 = mp.scale; tq = stats.t.ppf(0.975, 2)
print("à la main : h0 =", round(h0, 3), "| IC moyenne ±", round(tq*np.sqrt(s2_4*h0), 2), "| IC
      ↪ prédiction ±", round(tq*np.sqrt(s2_4*(1+h0)), 2))
```

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	101.0	4.35	82.29	119.71	76.85	125.15

```
à la main : h0 = 1.5 | IC moyenne ± 18.71 | IC prédiction ± 24.15
```

Étape 5 — les bandes de confiance et de prédiction. On trace, pour les clients venus des réseaux sociaux, la droite de m2 avec les deux bandes ; la bande de prédiction doit contenir environ 95 % des points.

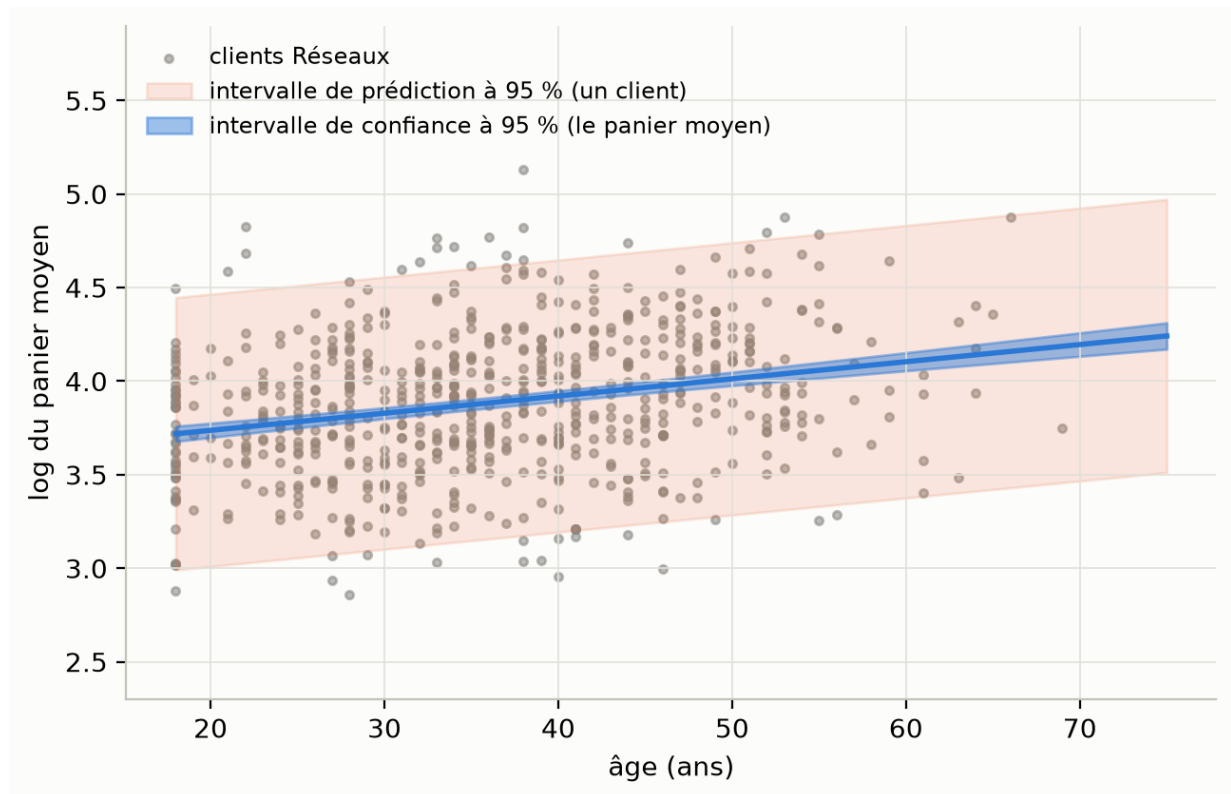


Figure 1: Clients Réseaux : log du panier selon l'âge. La bande bleue (intervalle de confiance de la moyenne) est étroite ; la bande orange (prédiction pour un client) est beaucoup plus large et contient environ 95 % des points.

Application 1.4 — Vérifier la théorie par simulation : couverture des intervalles et bootstrap (sections 1.2.5 et 1.2.6)

Objectif. Contrôler que « 95 % de confiance » veut bien dire 95 % de couverture quand le modèle est correct, puis comparer avec un bootstrap des couples, qui ne suppose pas la normalité.

Étape 1 — la couverture. On garde le vrai plan d'expérience de `m2`, on fixe de « vrais » coefficients, on simule 4 000 jeux de données et on compte combien d'intervalles contiennent la vérité.

```
rng = np.random.default_rng(12)
beta_etoile = np.array([4.22, -0.17, -0.34, 0.009]) # nos « vrais » coefficients
sigma_v = 0.37
R = 4000
Ysim = X @ beta_etoile + rng.normal(0, sigma_v, size=(R, n)) # R jeux de données de n clients
↳ (lignes)
B = Ysim @ (XtX_inv @ X.T).T # R estimations de beta (chaque ligne
↳ = un jeu)
E = Ysim - B @ X.T
S2 = (E**2).sum(axis=1) / (n - p)
SE = np.sqrt(S2[:, None] * np.diag(XtX_inv)[None, :])
Tst = (B - beta_etoile) / SE # R x p statistiques t « vraies »
↳ (centrées sur la vérité)
low, high = B - crit * SE, B + crit * SE
couverture = ((low <= beta_etoile) & (beta_etoile <= high)).mean(axis=0)
print("couverture empirique de l'IC à 95 % :")
print(pd.Series(couverture, index=m2.params.index).round(3).to_string())
print("statistique t pour Réseaux : moyenne =", Tst[:, 2].mean().round(3), "| écart-type =", Tst[:,
↳ 2].std().round(3),
      "| écart-type théorique de t(n-p) =", round(np.sqrt((n - p) / (n - p - 2)), 3))
print("part de |t| > 1,96 quand H0 est vraie (erreur de type I) pour l'âge :", (np.abs(Tst[:, 3]) >
↳ 1.96).mean().round(3))
```

```

couverture empirique de l'IC à 95 % :
Intercept          0.944
C(canal)[T.Site]   0.951
C(canal)[T.Réseaux] 0.948
a                  0.950
statistique t pour Réseaux : moyenne = 0.008 | écart-type = 1.01 | écart-type théorique de t(n-p) =
↳ 1.001
part de |t| > 1,96 quand H0 est vraie (erreur de type I) pour l'âge : 0.05

```

Étape 2 — le bootstrap des couples. On tire avec remise des clients entiers, on réajuste, et on observe la variabilité des coefficients : aucune formule, aucune hypothèse de normalité.

```

rng = np.random.default_rng(5)
B_boot = 2000
coefs = np.empty((B_boot, p))
for b in range(B_boot):
    idx = rng.integers(0, n, n)
    coefs[b] = np.linalg.lstsq(X[idx], y[idx], rcond=None)[0]
ic_boot = np.percentile(coefs, [2.5, 97.5], axis=0)
comp = pd.DataFrame({"IC t (bas)": m2.conf_int()[0].to_numpy(), "IC t (haut)":
↳ m2.conf_int()[1].to_numpy(),
                    "IC bootstrap (bas)": ic_boot[0], "IC bootstrap (haut)": ic_boot[1],
                    "se (formule)": m2.bse.to_numpy(), "se (bootstrap)": coefs.std(axis=0, ddof=1)},
↳ index=m2.params.index)
print(comp.round(4).to_string())

```

	IC t (bas)	IC t (haut)	IC bootstrap (bas)	IC bootstrap (haut)	se (formule)	se
↳ (bootstrap)						
Intercept	4.1863	4.2549	4.1868	4.2545	0.0175	
↳ 0.0172						
C(canal)[T.Site]	-0.2024	-0.1119	-0.2021	-0.1131	0.0231	
↳ 0.0227						
C(canal)[T.Réseaux]	-0.3809	-0.2927	-0.3794	-0.2931	0.0225	
↳ 0.0216						
a	0.0075	0.0108	0.0075	0.0108	0.0008	
↳ 0.0008						

Pour aller plus loin. Refaites l'étape 1 avec des erreurs très asymétriques (par exemple ε = une loi exponentielle centrée). Que devient la couverture pour $n = 1\,740$? Pour $n = 30$?

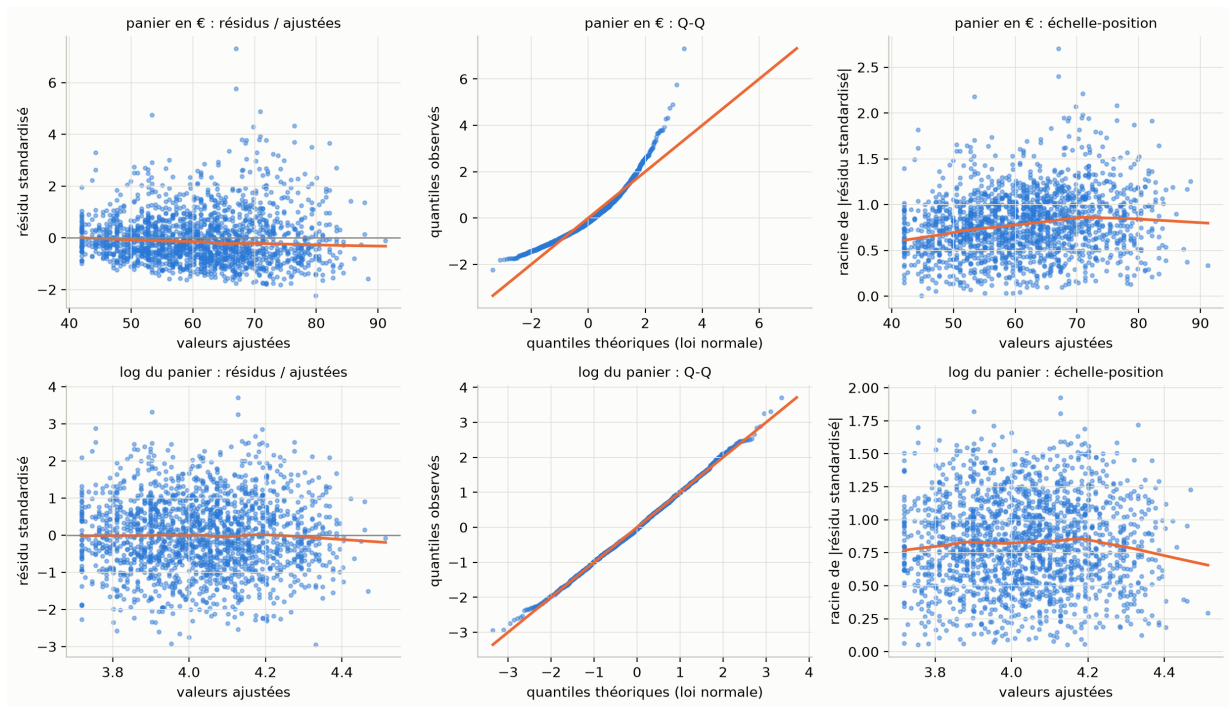


Figure 2: Diagnostics comparés. Ligne du haut : modèle sur le panier en € ; ligne du bas : modèle sur le log du panier.

Application 1.5 — Le diagnostic complet de `m2` (section 1.3)

Objectif. Passer le modèle sur le panier en euros, puis celui sur son logarithme, à la visite médicale : graphiques de résidus, tests de Breusch-Pagan et de normalité, autocorrélation sur les ventes mensuelles, levier et distance de Cook, colinéarité, erreurs standard robustes.

Étape 1 — les deux modèles et les trois graphiques. Les graphiques sont ceux du livre (résidus contre valeurs ajustées, Q-Q, échelle-position).

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

m_niv = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit() # panier en € : modèle « niveau »
ventes = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"])
ventes["t"] = np.arange(len(ventes)) # numéro du mois : 0, 1, ..., 119
```

Étape 2 — Breusch-Pagan, à la main puis avec `statsmodels`. On régresse les carrés des résidus sur X : $LM = nR_{aux}^2 \sim \chi_{p-1}^2$.

```
def breusch_pagan_main(modele):
    e2 = modele.resid.to_numpy() ** 2
    Xm = modele.model.exog
    aux = sm.OLS(e2, Xm).fit() # régression des carrés des résidus sur X
    LM = len(e2) * aux.rsquared
    return LM, stats.chi2.sf(LM, Xm.shape[1] - 1)

for nom, m in [("panier en €", m_niv), ("log du panier", m2)]:
    LM, p = breusch_pagan_main(m)
    LM_sm, p_sm = sm.stats.diagnostic.het_breuschpagan(m.resid, m.model.exog)[:2]
    print(f"{nom:14s} LM (main) = {LM:6.2f} p = {p:.2e} | statsmodels : LM = {LM_sm:6.2f} p = {p_sm:.2e}")
```

```
panier en €    LM (main) = 35.90 p = 7.86e-08 | statsmodels : LM = 35.90 p = 7.86e-08
log du panier  LM (main) = 1.92 p = 5.90e-01 | statsmodels : LM = 1.92 p = 5.90e-01
```

Étape 3 — la normalité. Asymétrie, aplatissement et tests de Shapiro et de Jarque-Bera.

```
for nom, m in [("panier en €", m_niv), ("log du panier", m2)]:
    r = m.resid
    jb, pjb, sk, ku = sm.stats.jarque_bera(r)
    print(f"{nom:14s} asymétrie = {sk:5.2f} | aplatissement (excès) = {ku-3:5.2f} | Shapiro p =
    ↳ {stats.shapiro(r).pvalue:.2e} | Jarque-Bera p = {pjb:.2e}")
```

```
panier en €    asymétrie =  1.40 | aplatissement (excès) =  3.84 | Shapiro p = 1.25e-29 | Jarque-Bera
↳ p = 0.00e+00
log du panier  asymétrie =  0.11 | aplatissement (excès) = -0.06 | Shapiro p = 2.34e-01 | Jarque-Bera
↳ p = 1.63e-01
```

Étape 4 — l'autocorrélation. Sur les ventes mensuelles, avec une tendance simple en niveau puis en logarithme : Durbin-Watson et autocorrélation complète des résidus.

```
mv_niv = smf.ols("ca ~ t", data=ventes).fit()
mv_log = smf.ols("np.log(ca) ~ t", data=ventes).fit()
for nom, m in [("ca ~ t", mv_niv), ("log(ca) ~ t", mv_log)]:
    dw = sm.stats.durbin_watson(m.resid)
    print(f"{nom:12s} R² = {m.rsquared:.3f} | Durbin-Watson = {dw:.2f} (rho ≈ {1 - dw/2:.2f}) |
    ↳ Breusch-Pagan p = {sm.stats.diagnostic.het_breuschpagan(m.resid, m.model.exog)[1]:.3f}")
```

```
ca ~ t      R² = 0.464 | Durbin-Watson = 1.56 (rho ≈ 0.22) | Breusch-Pagan p = 0.004
log(ca) ~ t R² = 0.493 | Durbin-Watson = 1.56 (rho ≈ 0.22) | Breusch-Pagan p = 0.889
```

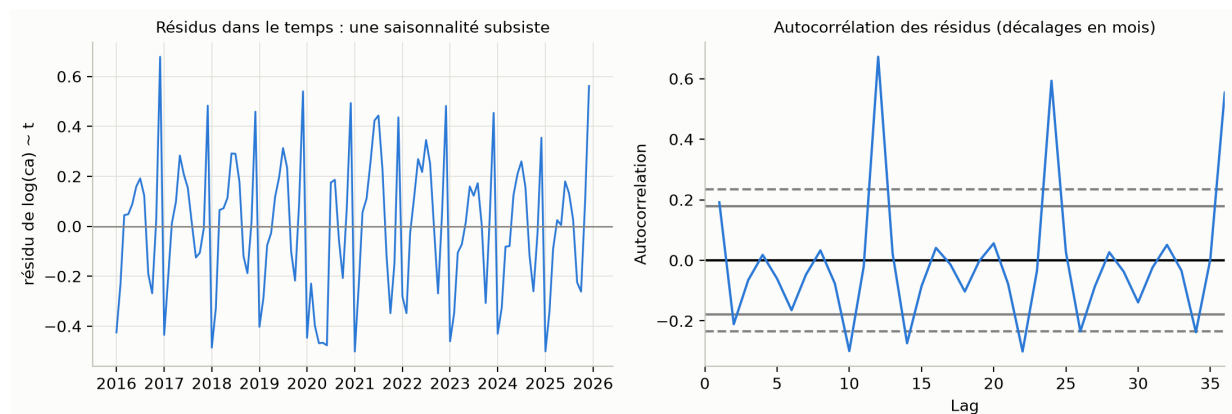


Figure 3: Résidus de $\log(ca) \sim t$: vagues saisonnières (à gauche) et autocorrélation avec des pics aux décalages 12, 24 et 36 (à droite).

Étape 5 — levier et influence : un petit exemple, puis les clients. Six points dont un très à droite : c'est son levier et sa distance de Cook qui le trahissent, pas son résidu.

```
xs = np.array([1, 2, 3, 4, 5, 12.0])
ys = np.array([2.1, 3.9, 6.2, 7.8, 10.1, 14.0]) # le dernier point devrait être vers 24 si la
↳ tendance se poursuivait
Xs = np.column_stack([np.ones(6), xs])
ms = sm.OLS(ys, Xs).fit()
m5 = sm.OLS(ys[:5], Xs[:5]).fit() # sans le sixième point
infl = ms.get_influence()
print("pente avec les 6 points :", round(ms.params[1], 3), "| sans le sixième point :",
    ↳ round(m5.params[1], 3))
print("leviers h_ii :", infl.hat_matrix_diag.round(3), "(moyenne p/n =", round(2/6, 3), ")")
print("résidus bruts :", ms.resid.round(2))
print("résidus standardisés :", infl.resid_studentized_internal.round(2))
print("distances de Cook :", infl.cooks_distance[0].round(2))
print("prévision en x = 12 sans le sixième point :", round(m5.params[0] + m5.params[1] * 12, 1), "|
    ↳ observé :", ys[5])
```

```
# vérification de la formule de Cook par réajustement sans l'observation i
# résidu studentisé externe : formule sans réajustement, comparée à statsmodels
h6, e6 = infl.hat_matrix_diag, ms.resid
s_sans = np.sqrt((e6 @ e6 - e6**2 / (1 - h6)) / (6 - 2 - 1))
print("studentisés externes (main) :", (e6 / (s_sans * np.sqrt(1 - h6))).round(2))
print("studentisés externes (statsm.):", infl.resid_studentized_external.round(2))
cook_main = []
for i in range(6):
    mi = sm.OLS(np.delete(ys, i), np.delete(Xs, i, axis=0)).fit()
    d = ms.params - mi.params
    cook_main.append(d @ Xs.T @ Xs @ d / (2 * ms.scale))
print("Cook par réajustement :", np.round(cook_main, 2))
```

```

pente avec les 6 points : 1.029 | sans le sixième point : 1.99
leviers h_ii : [0.325 0.247 0.196 0.17 0.17 0.892] (moyenne p/n = 0.333 )
résidus bruts : [-1.65 -0.88 0.39 0.96 2.24 -1.07]
résidus standardisés : [-1.23 -0.62 0.27 0.65 1.5 -1.99]
distances de Cook : [3.600e-01 6.000e-02 1.000e-02 4.000e-02 2.300e-01 1.643e+01]
prévision en x = 12 sans le sixième point : 23.9 | observé : 14.0
studentisés externes (main) : [ -1.34 -0.56 0.23 0.59 1.96 -17.24]
studentisés externes (statsm.): [ -1.34 -0.56 0.23 0.59 1.96 -17.24]
Cook par réajustement : [3.600e-01 6.000e-02 1.000e-02 4.000e-02 2.300e-01 1.643e+01]

```

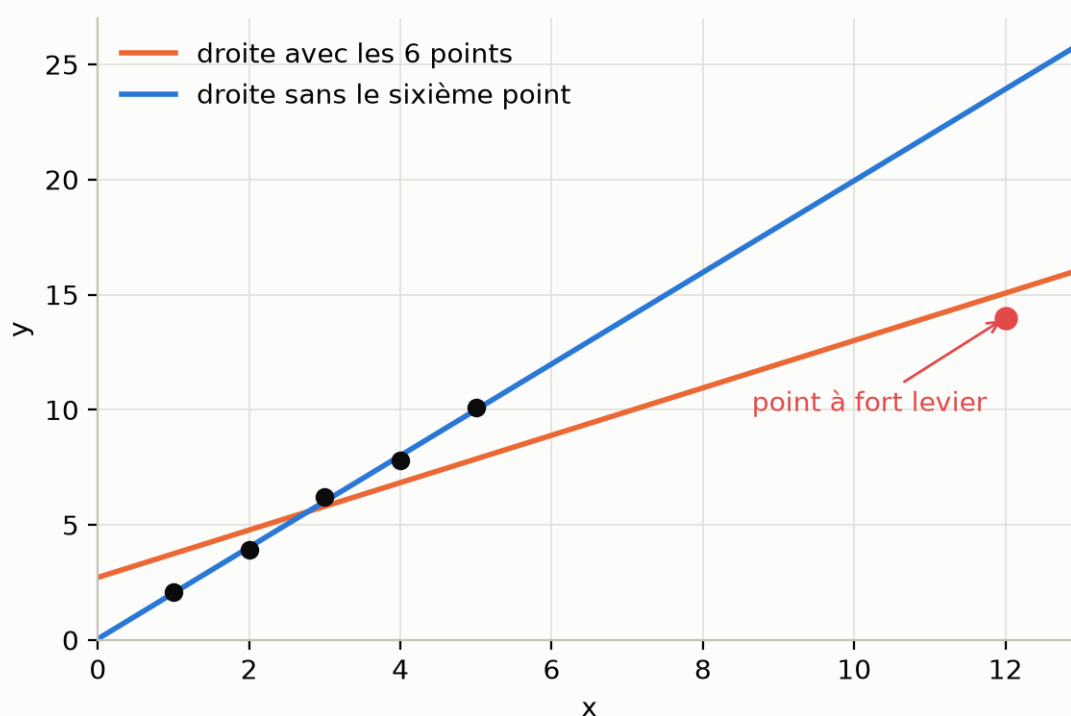


Figure 4: Un point à fort levier (rouge, $x = 12$) tire la droite orange vers lui.

```

infl = m2.get_influence()
h = infl.hat_matrix_diag
cook = infl.cooks_distance[0]
n, p = m2.model.exog.shape
print(f"n = {n}, p = {p} | levier moyen p/n = {p/n:.4f} | levier maximal = {h.max():.4f} | seuil 2p/n  

↳ = {2*p/n:.4f}")
print(f"distance de Cook maximale = {cook.max():.4f} | seuil usuel 4/n = {4/n:.4f} | seuil d'alerte  

↳ forte = 1")
print(f"nombre de clients au-dessus de 4/n : {(cook > 4/n).sum()} sur {n} ({100*(cook >  

↳ 4/n).mean():.1f} %)")
print()
```

```
top = pd.DataFrame({"age": df["age"].to_numpy(), "canal": df["canal"].to_numpy(), "panier":
↳ df["panier_moyen"].to_numpy(),
                    "résidu stud. ext.": infl.resid_studentized_external, "levier": h, "Cook":
                    ↳ cook}).sort_values("Cook", ascending=False).head(5)
print(top.round(4).to_string())
print()
print(m2.outlier_test().sort_values("unadj_p").head(3).round(4))
```

n = 1740, p = 4 | levier moyen p/n = 0.0023 | levier maximal = 0.0089 | seuil 2p/n = 0.0046
distance de Cook maximale = 0.0067 | seuil usuel 4/n = 0.0023 | seuil d'alerte forte : 1
nombre de clients au-dessus de 4/n : 84 sur 1740 (4.8 %)

	age	canal	panier	résidu stud. ext.	levier	Cook
864	43	Site	245.06	3.7254	0.0019	0.0067
747	48	Boutique	25.53	-2.9552	0.0030	0.0066
1060	66	Réseaux	131.04	1.9415	0.0061	0.0058
140	50	Site	189.89	2.8562	0.0027	0.0055
439	22	Réseaux	124.39	2.8921	0.0025	0.0052

	student_resid	unadj_p	bonf(p)
1001	3.7254	0.0002	0.3502
180	3.3254	0.0009	1.0000
1741	3.2648	0.0011	1.0000

Étape 6 — la multicollinéarité. On ajoute à m2 l'âge en mois, presque redondant avec l'âge en années : prédictions intactes, coefficients ruinés ; puis le cas des termes polynomiaux non centrés.

```
rng = np.random.default_rng(3)
df["age_mois"] = 12 * df["age"] + rng.normal(0, 3, len(df))           # presque redondant avec l'âge en
↳ années
print("corrélation âge (années) / âge (mois) :", round(np.corrcoef(df["age"], df["age_mois"])[0, 1],
↳ 4))

mc = smf.ols("log_panier ~ a + C(canal) + age_mois", data=df).fit()
print(pd.DataFrame({"m2 : coef": m2.params, "m2 : se": m2.bse, "avec age_mois : coef": mc.params,
↳ "avec age_mois : se": mc.bse}).round(4).to_string())
Xc = mc.model.exog
vif = pd.Series([variance_inflation_factor(Xc, i) for i in range(Xc.shape[1])], index=mc.params.index)
print()
print("VIF :", vif.round(1).to_dict())
print("R² de m2 :", round(m2.rsquared, 4), "| R² avec age_mois :", round(mc.rsquared, 4), "
↳ (prédictions aussi bonnes)")
```

corrélation âge (années) / âge (mois) : 0.9997

	m2 : coef	m2 : se	avec age_mois : coef	avec age_mois : se
C(canal)[T.Réseaux]	-0.3368	0.0225	-0.3377	0.0225
C(canal)[T.Site]	-0.1571	0.0231	-0.1580	0.0231
Intercept	4.2206	0.0175	3.2552	1.2998
a	0.0092	0.0008	-0.0176	0.0361
age_mois	NaN	NaN	0.0022	0.0030

VIF : {'Intercept': 1.0, 'C(canal)[T.Site]': 1.5, 'C(canal)[T.Réseaux]': 1.5, 'a': 1828.5,
↳ 'age_mois': 1828.6}
R² de m2 : 0.1657 | R² avec age_mois : 0.166 (prédictions aussi bonnes)

```
df["age2_brut"] = df["age"] ** 2
df["a2"] = df["a"] ** 2
mq_brut = smf.ols("log_panier ~ age + age2_brut + C(canal)", data=df).fit()
mq_centre = smf.ols("log_panier ~ a + a2 + C(canal)", data=df).fit()
for nom, m in [("non centré (âge, âge²)", mq_brut), ("centré (a, a²)", mq_centre)]:
    Xq = m.model.exog
    v = [round(float(variance_inflation_factor(Xq, i)), 1) for i in range(Xq.shape[1])]
    print(f"{nom:24s} VIF = {v} | p-valeur du terme quadratique = {m.pvalues.iloc[-1]::.3f}")
```

non centré (âge, âge²) VIF = [1.0, 1.5, 1.5, 33.9, 33.9] | p-valeur du terme quadratique = 0.580
centré (a, a²) VIF = [1.0, 1.5, 1.5, 1.0, 1.0] | p-valeur du terme quadratique = 0.580

Étape 7 — les erreurs standard robustes. Pour `m_niv`, on compare erreurs classiques, robustes (HC3) et bootstrap.

```
m_hc3 = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit(cov_type="HC3")
Xn, yn = m_niv.model.exog, m_niv.model.endog
rng = np.random.default_rng(8)
cb = np.array([np.linalg.lstsq(Xn[idx], yn[idx], rcond=None)[0] for idx in (rng.integers(0, len(yn),
↪ len(yn)) for _ in range(2000))])
print(pd.DataFrame({"se classique": m_niv.bse, "se robuste (HC3)": m_hc3.bse, "se bootstrap":
↪ cb.std(axis=0, ddof=1)}).round(3).to_string())
```

	se classique	se robuste (HC3)	se bootstrap
Intercept	1.149	1.306	1.274
C(canal)[T.Site]	1.518	1.688	1.659
C(canal)[T.Réseaux]	1.478	1.511	1.522
a	0.055	0.055	0.055

Étape 8 — les résidus partiels. Une courbure systématique suggérerait d'ajouter un terme en âge² ; ici, le lissage suit la droite.

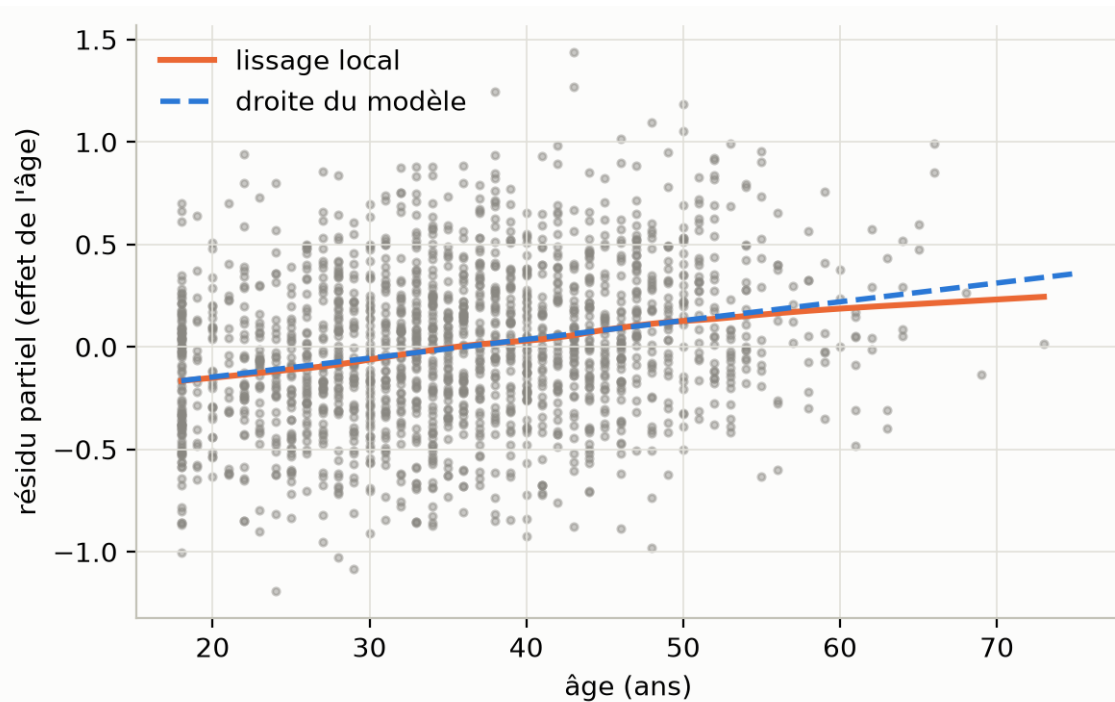


Figure 5: Résidus partiels pour l'âge : le lissage local suit la droite du modèle.

Application 1.6 — Le surajustement, puis huit modèles en concurrence (sections 1.4.1 à 1.4.4)

Objectif. Voir le surajustement, vérifier à la main AIC, BIC et PRESS, écrire une validation croisée 10-fold et comparer huit modèles candidats sur les clients ayant répondu à l'enquête.

Étape 1 — les données de l'enquête. On fait la moyenne des questions q1–q4 (produits) et q5–q8 (service) et on les rattache aux clients actifs.

```
from sklearn.model_selection import KFold

enq = pd.read_csv("donnees/enquete_satisfaction.csv")
enq["score_produit"] = enq[["q1", "q2", "q3", "q4"]].mean(axis=1)
enq["score_service"] = enq[["q5", "q6", "q7", "q8"]].mean(axis=1)
bq = df.merge(enq[["id_client", "score_produit", "score_service"]], on="id_client")    # clients
↳ actifs ayant répondu
print(len(enq), "répondants |", len(bq), "clients actifs répondants")
```

1212 répondants | 1076 clients actifs répondants

Étape 2 — le surajustement. On tire 40 clients pour entraîner un polynôme en l'âge de degré 0 à 8, on mesure l'erreur sur les autres, et on répète 300 fois.

```
z = ((df["age"] - 36) / 10).to_numpy()    # âge centré et réduit (évite les problèmes
↳ numériques des puissances)
yy = df["log_panier"].to_numpy()
rng = np.random.default_rng(21)
degres = np.arange(0, 9)
err_app, err_test = np.zeros((300, len(degres))), np.zeros((300, len(degres)))
for r in range(300):
    idx = rng.permutation(len(z))
    tr, te = idx[:40], idx[40:]
    for j, d in enumerate(degres):
        coef = np.polyfit(z[tr], yy[tr], d)
        err_app[r, j] = np.mean((yy[tr] - np.polyval(coef, z[tr])) ** 2)
        err_test[r, j] = np.mean((yy[te] - np.polyval(coef, z[te])) ** 2)
res = pd.DataFrame({"degré": degres, "erreur d'apprentissage": err_app.mean(axis=0), "erreur sur
↳ nouveaux clients": err_test.mean(axis=0)})
print(res.round(4).to_string(index=False))
```

degré	erreur d'apprentissage	erreur sur nouveaux clients
0	0.1610	0.1684
1	0.1490	0.1629
2	0.1454	0.1696
3	0.1423	0.1853
4	0.1389	0.2958
5	0.1353	1.4399
6	0.1311	50.3002
7	0.1261	612.0571
8	0.1220	12583.0488

Étape 3 — AIC et BIC à la main. La log-vraisemblance maximale du modèle gaussien est $-\frac{n}{2}(\log 2\pi + \log(\text{SCR}/n) + 1)$; on pénalise par $k = p$ (convention de statsmodels).

```
m_ex = smf.ols("log_panier ~ a + C(canal)", data=bq).fit()
n, p = m_ex.model.exog.shape
scr = m_ex.ssr
ll = -n / 2 * (np.log(2 * np.pi) + np.log(scr / n) + 1)
k = p    # convention de statsmodels : on ne compte pas
↳ sigma²
print(f"log-vraisemblance : à la main = {ll:.3f} | statsmodels = {m_ex.llf:.3f}")
print(f"AIC : à la main = {-2*ll + 2*k:.3f} | statsmodels = {m_ex.aic:.3f}")
print(f"BIC : à la main = {-2*ll + k*np.log(n):.3f} | statsmodels = {m_ex.bic:.3f}")
```

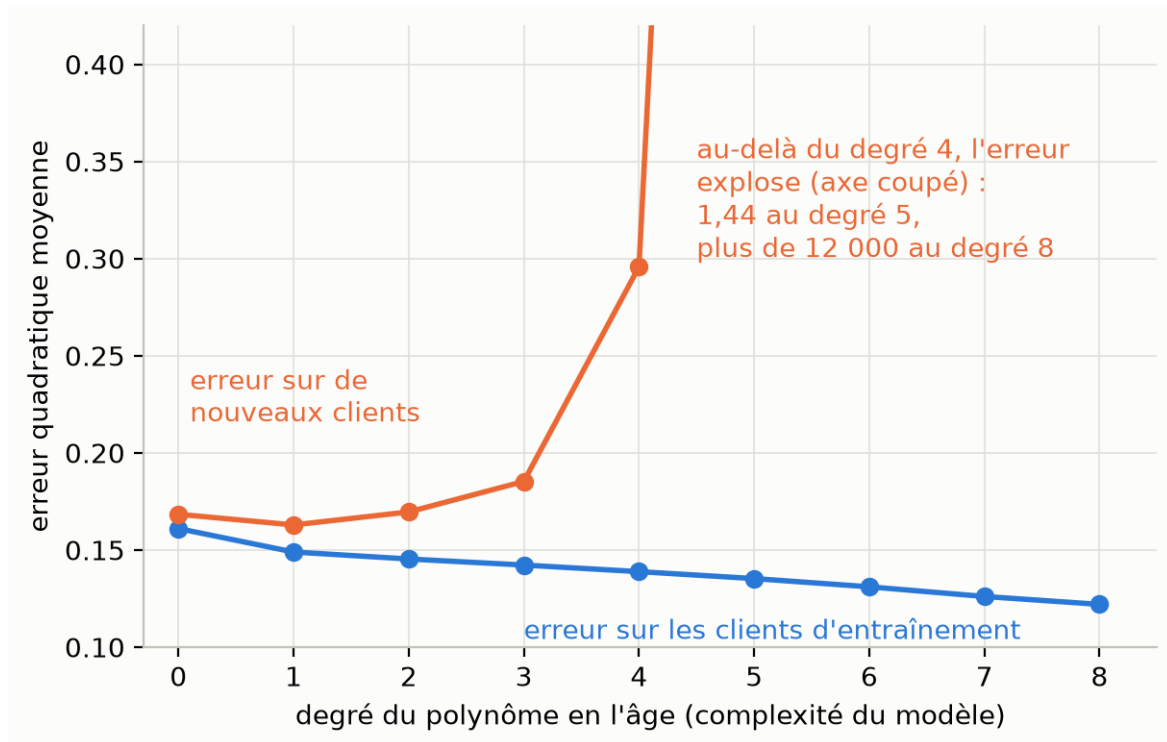


Figure 6: Erreur d'apprentissage et erreur sur de nouveaux clients selon le degré du polynôme.

```
log-vraisemblance : à la main = -431.472 | statsmodels = -431.472
AIC : à la main = 870.944 | statsmodels = 870.944
BIC : à la main = 890.868 | statsmodels = 890.868
```

Étape 4 — PRESS : le leave-one-out sans refaire n ajustements. La formule $\hat{\epsilon}_i / (1 - h_{ii})$ est comparée à une boucle explicite.

```
Xb, yb = m_ex.model.exog, m_ex.model.endog
h = m_ex.get_influence().hat_matrix_diag
press_formule = np.sum((m_ex.resid / (1 - h)) ** 2)
loo = np.empty(len(yb))
for i in range(len(yb)):
    mask = np.arange(len(yb)) != i
    b_i = np.linalg.lstsq(Xb[mask], yb[mask], rcond=None)[0]
    loo[i] = yb[i] - Xb[i] @ b_i
print(f"PRESS (formule, 1 ajustement) = {press_formule:.4f}")
print(f"PRESS (boucle, {len(yb)} ajustements) = {np.sum(loo**2):.4f}")
print(f"erreur quadratique d'apprentissage (SCR) = {scr:.4f} <- plus optimiste")
```

```
PRESS (formule, 1 ajustement) = 141.5167
PRESS (boucle, 1076 ajustements) = 141.5167
erreur quadratique d'apprentissage (SCR) = 140.4879 <- plus optimiste
```

Étape 5 — une validation croisée K-fold. Mêmes paquets pour tous les modèles comparés.

```
def cv_rmse(formule, data, K=10, graine=0):
    """RMSE de validation croisée K-fold (sur l'échelle du log-panier), mêmes paquets pour tous les
    ↪ modèles."""
    kf = KFold(n_splits=K, shuffle=True, random_state=graine)
    erreurs = []
    for tr, te in kf.split(data):
        m = smf.ols(formule, data=data.iloc[tr]).fit()
        pred = np.asarray(m.predict(data.iloc[te]))
```

```

    if pred.size == 1:                                     # modèle à constante seule : predict
        ↪ renvoie un seul nombre
        pred = np.repeat(pred, len(te))
        erreurs.append(data.iloc[te]["log_panier"].to_numpy() - pred)
    e = np.concatenate(erreurs)
    return np.sqrt(np.mean(e ** 2))

print("RMSE par validation croisée 10-fold, modèle log_panier ~ a + C(canal) :",
      ↪ round(cv_rmse("log_panier ~ a + C(canal)", bq), 4))
print("RMSE d'apprentissage                                     :", round(np.sqrt(scr /
      ↪ n), 4))

RMSE par validation croisée 10-fold, modèle log_panier ~ a + C(canal) : 0.3629
RMSE d'apprentissage                                     : 0.3613

```

Étape 6 — huit modèles candidats. M0 constante seule ; M1 âge ; M2 âge + canal ; M3 + score produit ; M4 + score service ; M5 M3 + ville + offre ; M6 M3 + âge² + interaction ; M7 tout.

```

bq = bq.copy()
bq["a2"] = bq["a"] ** 2
candidats = {
    "M0": "log_panier ~ 1",
    "M1": "log_panier ~ a",
    "M2": "log_panier ~ a + C(canal)",
    "M3": "log_panier ~ a + C(canal) + score_produit",
    "M4": "log_panier ~ a + C(canal) + score_produit + score_service",
    "M5": "log_panier ~ a + C(canal) + score_produit + C(ville) + offre_bienvenue",
    "M6": "log_panier ~ a * C(canal) + a2 + score_produit",
    "M7": "log_panier ~ a * C(canal) + a2 + C(ville) + offre_bienvenue + score_produit +
    ↪ score_service",
}

lignes, fits = [], {}
for nom, f in candidats.items():
    m = smf.ols(f, data=bq).fit()
    fits[nom] = m
    h = m.get_influence().hat_matrix_diag
    lignes.append({"modèle": nom, "paramètres p": int(m.df_model) + 1, "R²": m.rsquared, "R² ajusté":
    ↪ m.rsquared_adj,
                  "AIC": m.aic, "BIC": m.bic, "RMSE appr.": np.sqrt(m.mse_resid * m.df_resid / m.nobs),
                  "RMSE L00 (PRESS)": np.sqrt(np.mean((m.resid / (1 - h)) ** 2)), "RMSE CV10":
                  ↪ cv_rmse(f, bq)})
tab = pd.DataFrame(lignes).set_index("modèle")
with pd.option_context("display.width", 200):
    print(tab.round(4).to_string())
print()
for crit in ["R² ajusté", "AIC", "BIC", "RMSE L00 (PRESS)", "RMSE CV10"]:
    meilleur = tab[crit].idxmax() if crit == "R² ajusté" else tab[crit].idxmin()
    print(f"meilleur modèle selon {crit:18s} : {meilleur}")

```

	paramètres p	R ²	R ² ajusté	AIC	BIC	RMSE appr.	RMSE L00 (PRESS)	RMSE CV10
modèle								
M0	1	0.0000	0.0000	1082.3441	1087.3251	0.3997	0.4001	0.4005
M1	2	0.0663	0.0654	1010.5132	1020.4752	0.3863	0.3870	0.3872
M2	4	0.1829	0.1807	870.9438	890.8679	0.3613	0.3627	0.3629
M3	5	0.2542	0.2514	774.7862	799.6912	0.3452	0.3469	0.3478
M4	6	0.2567	0.2532	773.1610	803.0470	0.3446	0.3466	0.3474
M5	11	0.2548	0.2478	785.8815	840.6726	0.3451	0.3487	0.3503
M6	8	0.2549	0.2500	779.7843	819.6323	0.3451	0.3476	0.3485
M7	15	0.2582	0.2484	788.9871	863.7021	0.3443	0.3491	0.3506
meilleur modèle selon R ² ajusté				: M4				
meilleur modèle selon AIC				: M4				

```
meilleur modèle selon BIC          : M3
meilleur modèle selon RMSE L00 (PRESS) : M4
meilleur modèle selon RMSE CV10    : M4
```

Étape 7 — les tests emboîtés. Mêmes questions, posées par des tests F .

```
print("M2 -> M3 (ajout du score produit) :")
print(sm.stats.anova_lm(fits["M2"], fits["M3"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M4 (ajout du score service) :")
print(sm.stats.anova_lm(fits["M3"], fits["M4"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M5 (ajout de la ville et de l'offre de bienvenue, 6 paramètres) :")
print(sm.stats.anova_lm(fits["M3"], fits["M5"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
print("\nM3 -> M6 (âge² et interactions, 3 paramètres) :")
print(sm.stats.anova_lm(fits["M3"], fits["M6"]).round(4).iloc[:, [0, 2, 4, 5]].to_string())
```

```
M2 -> M3 (ajout du score produit) :
  df_resid df_diff      F Pr(>F)
0    1072.0     0.0     NaN     NaN
1    1071.0     1.0 102.2966     0.0
```

```
M3 -> M4 (ajout du score service) :
  df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN     NaN
1    1070.0     1.0  3.6111  0.0577
```

```
M3 -> M5 (ajout de la ville et de l'offre de bienvenue, 6 paramètres) :
  df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN     NaN
1    1065.0     6.0  0.1493  0.9892
```

```
M3 -> M6 (âge² et interactions, 3 paramètres) :
  df_resid df_diff      F Pr(>F)
0    1071.0     0.0     NaN     NaN
1    1068.0     3.0  0.3316  0.8025
```

Étape 8 — retrouver la vérité. Les données étant simulées (build/donnees2.py), on compare le modèle M3 aux vraies valeurs, puis on prédit l'effet attendu par point de note du score produit.

```
m3 = fits["M3"]
ic = m3.conf_int()
vrai_site, vrai_reseaux = 0.05 - 0.22, -0.12 - 0.22          # effets du Site et de Réseaux
↳ RELATIVEMENT à la Boutique
vrai = {"Intercept": np.nan, "C(canal)[T.Site]": vrai_site, "C(canal)[T.Réseaux]": vrai_reseaux, "a":
↳ 0.008}
comp = pd.DataFrame({"estimation (M3)": m3.params, "IC95 bas": ic[0], "IC95 haut": ic[1]})
comp["vérité"] = pd.Series(vrai)
comp["IC contient la vérité ?"] = [("oui" if (lo <= v <= hi) else "non") if not np.isnan(v) else "-"
↳ for lo, hi, v in zip(comp["IC95 bas"], comp["IC95 haut"], comp["vérité"])]
print(comp.round(4).to_string())
print()
print("Variables retenues dans M3 : âge, canal, score produit. Écartées par les critères : ville,
↳ offre, score service, âge², interactions.")
print("Dans M7 (le modèle « tout »), p-valeurs des variables sans effet réel :)")
print(fits["M7"].pvalues[["C(ville)[T.Ville A]", "C(ville)[T.Ville B]", "C(ville)[T.Ville C]",
↳ "C(ville)[T.Ville D]", "C(ville)[T.Ville E]", "offre_bienvenue", "a2",
↳ "score_service"]].round(3).to_string())
```

	estimation (M3)	IC95 bas	IC95 haut	vérité IC	contient la vérité ?
Intercept	3.6543	3.5385	3.7701	NaN	—
C(canal)[T.Site]	-0.1573	-0.2114	-0.1032	-0.170	oui
C(canal)[T.Réseaux]	-0.3519	-0.4045	-0.2993	-0.340	oui
a	0.0097	0.0078	0.0117	0.008	oui
score_produit	0.1557	0.1255	0.1859	NaN	—

Variables retenues dans M3 : âge, canal, score produit. Écartées par les critères : ville, offre, score service, âge², interactions.

Dans M7 (le modèle « tout »), p-valeurs des variables sans effet réel :

```
C(ville)[T.Ville A]    0.901
C(ville)[T.Ville B]    0.934
C(ville)[T.Ville C]    0.526
C(ville)[T.Ville D]    0.808
C(ville)[T.Ville E]    0.883
offre_bienvenue        0.788
a2                      0.300
score_service           0.051
```

```
lam = np.array([0.80, 0.70, 0.75, 0.60])
var_bruit = np.mean(1 - lam**2) / 4          # variance du bruit de la moyenne de 4
↳ questions (en unités de F1)
charge = lam.mean()                          # poids moyen de F1 dans la moyenne des
↳ questions
fiabilite = charge**2 / (charge**2 + var_bruit) # part de la variance de la note qui
↳ provient de F1
cov_F_score = 0.95 * charge
var_score = 0.95**2 * (charge**2 + var_bruit)
pente_F_sur_score = cov_F_score / var_score   # E[F1 | score] = pente * (score - moyenne)
print(f"fiabilité du score produit : {fiabilite:.2f}")
print(f"effet attendu par point de note : 0,12 x {pente_F_sur_score:.3f} = {0.12 *
↳ pente_F_sur_score:.3f} | estimé dans M3 : {m3.params['score_produit']:.3f} (IC95 % :
↳ [{ic.loc['score_produit', 0]:.3f} ; {ic.loc['score_produit', 1]:.3f}])")
```

```
fiabilité du score produit : 0.81
effet attendu par point de note : 0,12 x 1.192 = 0.143 | estimé dans M3 : 0.156 (IC95 % : [0.125 ;
↳ 0.186])
```

Application 1.7 — La sélection automatique fabrique des découvertes (section 1.4.5)

Objectif. Montrer par simulation qu'une sélection ascendante par p-valeur « découvre » des variables dans du pur bruit, et qu'une procédure honnête (choisir sur une moitié, tester sur l'autre) retrouve le bon taux d'erreur.

Étape 1 — la sélection ascendante. On ajoute à chaque étape la variable la plus significative tant que sa p-valeur est inférieure à 0,05.

```
def selection_ascendante(X, y, alpha=0.05):
    """Sélection ascendante par p-valeur. X : matrice n x q sans constante. Retourne la liste des
    ↳ colonnes retenues."""
    n, q = X.shape
    retenues = []
    while len(retenues) < q:
        meilleur, p_min = None, 1.0
        for j in range(q):
            if j in retenues:
                continue
            Z = np.column_stack([np.ones(n), X[:, retenues + [j]]])
            b, _, _, _ = np.linalg.lstsq(Z, y, rcond=None)
            e = y - Z @ b
            s2 = e @ e / (n - Z.shape[1])
            se = np.sqrt(s2 * np.linalg.inv(Z.T @ Z)[-1, -1])
            pj = 2 * stats.t.sf(abs(b[-1] / se), n - Z.shape[1])
            if pj < p_min:
                meilleur, p_min = j, pj
        if meilleur is None or p_min >= alpha:
            break
        retenues.append(meilleur)
```

```
return retenues
```

Étape 2 — une fonction qui renvoie les p-valeurs et le R^2 d'un modèle.

```
def ols_pvaleurs(X, y):
    Z = np.column_stack([np.ones(len(y)), X])
    b = np.linalg.lstsq(Z, y, rcond=None)[0]
    e = y - Z @ b
    s2 = e @ e / (len(y) - Z.shape[1])
    se = np.sqrt(s2 * np.diag(np.linalg.inv(Z.T @ Z)))
    return 2 * stats.t.sf(np.abs(b / se), len(y) - Z.shape[1][1:], 1 - (e @ e) / np.sum((y -
    ↪ y.mean()) ** 2))
```

Étape 3 — la simulation. y est indépendant des 20 variables ; 500 répétitions.

```
rng = np.random.default_rng(99)
R, n_obs, q = 500, 100, 20
nb_retenues, r2_final, sig_final, sig_honnete, nb_honnete = [], [], [], [], []
for _ in range(R):
    X = rng.normal(size=(n_obs, q)); y = rng.normal(size=n_obs) # y est indépendant de tout X
    ret = selection_ascendante(X, y)
    nb_retenues.append(len(ret))
    if ret:
        pv, r2 = ols_pvaleurs(X[:, ret], y)
        r2_final.append(r2); sig_final.append(np.mean(pv < 0.05))
    # procédure honnête : on choisit sur les 50 premiers, on teste sur les 50 autres
    ret_h = selection_ascendante(X[:50], y[:50])
    if ret_h:
        pv_h, _ = ols_pvaleurs(X[50:][:, ret_h], y[50:])
        sig_honnete.append(np.sum(pv_h < 0.05)); nb_honnete.append(len(ret_h))
nb_retenues = np.array(nb_retenues)
print(f"y est du pur bruit, indépendant des {q} variables explicatives (n = {n_obs}).")
print(f"part des jeux de données où au moins une variable est « découverte » : {np.mean(nb_retenues >
    ↪ 0):.1%} (théorie pour une seule étape : 1 - 0,95^20 = {1 - 0.95**20:.1%})")
print(f"nombre moyen de variables retenues : {nb_retenues.mean():.2f}")
print(f"R² moyen du modèle final (quand il est non vide) : {np.mean(r2_final):.3f} alors que le vrai
    ↪ R² est 0")
print(f"part de coefficients « significatifs à 5 % » dans le modèle final : {np.mean(sig_final):.1%}
    ↪ (c'est trompeur : ils ont été choisis POUR l'être)")
print(f"procédure honnête (choix sur la moitié A, test sur la moitié B) : {np.sum(sig_honnete) /
    ↪ np.sum(nb_honnete):.1%} de significatifs parmi les variables retenues (≈ 5 % attendu)")
```

```
y est du pur bruit, indépendant des 20 variables explicatives (n = 100).
part des jeux de données où au moins une variable est « découverte » : 63.4% (théorie pour une seule
    ↪ étape : 1 - 0,95^20 = 64.2%)
nombre moyen de variables retenues : 1.09
R² moyen du modèle final (quand il est non vide) : 0.093 alors que le vrai R² est 0
part de coefficients « significatifs à 5 % » dans le modèle final : 100.0% (c'est trompeur : ils ont
    ↪ été choisis POUR l'être)
procédure honnête (choix sur la moitié A, test sur la moitié B) : 5.6% de significatifs parmi les
    ↪ variables retenues (≈ 5 % attendu)
```

Questions. Pourquoi la proportion de « découvertes » est-elle proche de $1 - 0,95^{20}$? Que vaut le taux de coefficients « significatifs » dans le modèle final, et pourquoi n'est-ce pas un taux d'erreur de type I ?

Application 1.8 — Ridge et Lasso à la main (sections 1.5.1 et 1.5.2)

Objectif. Vérifier par simulation que Ridge peut battre les moindres carrés en erreur quadratique moyenne, puis écrire les formules de Ridge et du Lasso et les comparer à `scikit-learn`.

Étape 1 — l'EQM de Ridge. Deux variables corrélées à 0,98, $n = 30$, vrais coefficients (1,1) et $\sigma = 1$: biais², variance et EQM pour plusieurs λ .

```
from sklearn.linear_model import Ridge, RidgeCV, Lasso, LassoCV, ElasticNetCV, lasso_path,
↳ LinearRegression
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(4)
n_s, rho = 30, 0.98
cov = np.array([[1, rho], [rho, 1]])
Xs = rng.multivariate_normal([0, 0], cov, size=n_s)
Xs = (Xs - Xs.mean(axis=0)) / Xs.std(axis=0)           # variables centrées et standardisées,
↳ FIXES pour toute l'expérience
beta_vrai = np.array([1.0, 1.0]) / np.sqrt(n_s) * 3    # vrais coefficients (de petite
↳ taille relative au bruit)
R = 4000
resultats = {}
for lam in [0, 0.5, 1, 2, 5, 10, 20, 50]:
    M = np.linalg.solve(Xs.T @ Xs + lam * np.eye(2), Xs.T) # (X'X + lambda I)^-1 X'
    erreurs = np.empty((R, 2))
    for r in range(R):
        y_s = Xs @ beta_vrai + rng.normal(0, 1, n_s)
        erreurs[r] = M @ y_s - beta_vrai
    if lam == 0:
        erreurs_mco = erreurs.copy()
        resultats[lam] = (np.mean(np.sum(erreurs**2, axis=1)), np.sum(erreurs.mean(axis=0)**2),
        ↳ np.sum(erreurs.var(axis=0)))
tab = pd.DataFrame(resultats, index=["EQM totale", "biais² total", "variance totale"]).T
tab.index.name = "lambda"
print("corrélation entre les deux variables :", round(np.corrcoef(Xs.T)[0, 1], 3))
print(tab.round(4).to_string())
print("corrélation entre les deux estimations MCO du même échantillon :",
↳ round(np.corrcoef(erreurs_mco.T)[0, 1], 3))
```

```
corrélation entre les deux variables : 0.984
      EQM totale  biais² total  variance totale
lambda
0.0          2.0874         0.0002          2.0872
0.5          0.5132         0.0001          0.5131
1.0          0.2281         0.0003          0.2278
2.0          0.0899         0.0007          0.0892
5.0          0.0332         0.0038          0.0294
10.0         0.0292         0.0128          0.0164
20.0         0.0487         0.0378          0.0109
50.0         0.1310         0.1259          0.0052
corrélation entre les deux estimations MCO du même échantillon : -0.984
```

Étape 2 — la formule de Ridge contre `scikit-learn`. On standardise, on ne pénalise pas la constante (on centre y), et on compare à Ridge.

```
clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Réseaux"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
enq = pd.read_csv("donnees/enquete_satisfaction.csv")
enq["score_produit"] = enq[["q1", "q2", "q3", "q4"]].mean(axis=1)
enq["score_service"] = enq[["q5", "q6", "q7", "q8"]].mean(axis=1)
bq = df.merge(enq[["id_client", "score_produit", "score_service"]],
↳ on="id_client").reset_index(drop=True)
```

```
# Un petit jeu de variables : l'âge, le canal, les scores
Xp = pd.DataFrame({"age": bq["a"], "Site": (bq["canal"] == "Site").astype(float), "Réseaux":
    ↳ (bq["canal"] == "Réseaux").astype(float),
                    "score_produit": bq["score_produit"], "score_service": bq["score_service"]})
yv = bq["log_panier"].to_numpy()
sc = StandardScaler().fit(Xp)
Z = sc.transform(Xp)
yc = yv - yv.mean()
lam = 50.0
beta_main = np.linalg.solve(Z.T @ Z + lam * np.eye(Z.shape[1]), Z.T @ yc)
sk = Ridge(alpha=lam).fit(Z, yv)
print(pd.DataFrame({"à la main": beta_main, "scikit-learn": sk.coef_, "MCO (λ = 0)":
    ↳ LinearRegression().fit(Z, yv).coef_, index=Xp.columns).round(5).to_string()))
```

	à la main	scikit-learn	MCO (λ = 0)
age	0.09800	0.09800	0.10255
Site	-0.06342	-0.06342	-0.07490
Réseaux	-0.15797	-0.15797	-0.17241
score_produit	0.09827	0.09827	0.10322
score_service	0.02016	0.02016	0.02035

Étape 3 — la descente par coordonnées du Lasso. La mise à jour de chaque coefficient est un seuillage doux $S(\rho_j, \alpha) = \text{signe}(\rho_j) \max(|\rho_j| - \alpha, 0)$.

```
def lasso_coordonnees(Z, y, alpha, n_iter=500):
    n_, p_ = Z.shape
    beta = np.zeros(p_)
    for _ in range(n_iter):
        for j in range(p_):
            r_partiel = y - Z @ beta + Z[:, j] * beta[j]
            ↳ toutes les variables sauf j
            rho = Z[:, j] @ r_partiel / n_
            beta[j] = np.sign(rho) * max(abs(rho) - alpha, 0.0)
            ↳ # résidu en retirant l'effet de
            ↳ # seuillage doux
    return beta

alpha = 0.05
b_main = lasso_coordonnees(Z, yc, alpha)
b_sk = Lasso(alpha=alpha, max_iter=100000, tol=1e-12).fit(Z, yv).coef_
print(pd.DataFrame({"à la main": b_main, "scikit-learn": b_sk, "MCO": LinearRegression().fit(Z,
    ↳ yv).coef_, index=Xp.columns).round(5).to_string()))
print("coefficients exactement nuls (à la main) :", list(Xp.columns[b_main == 0]))
```

	à la main	scikit-learn	MCO
age	0.05297	0.05297	0.10255
Site	-0.00000	-0.00000	-0.07490
Réseaux	-0.07486	-0.07486	-0.17241
score_produit	0.05465	0.05465	0.10322
score_service	0.00000	0.00000	0.02035

coefficients exactement nuls (à la main) : ['Site', 'score_service']

Étape 4 — la géométrie. Le point de contact entre une ellipse qui grandit et la région admissible : sommet du losange pour le Lasso (d'où les zéros), point lisse pour Ridge.

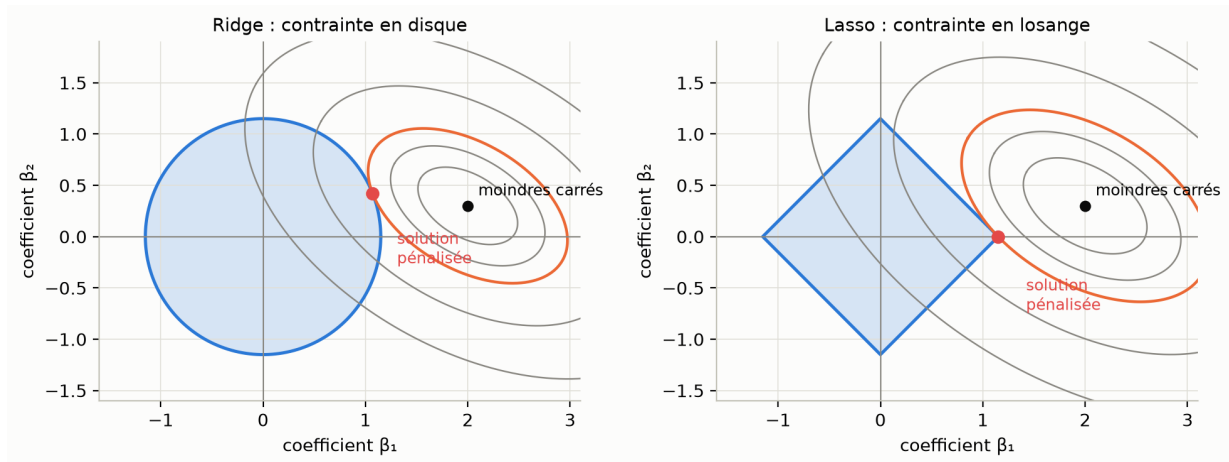


Figure 7: La géométrie de la régularisation : disque (Ridge) et losange (Lasso).

Application 1.9 — Beaucoup de variables, peu de clients (section 1.5.4)

Objectif. Reproduire l'expérience du livre : 35 variables candidates (4 vraies, des variables sans effet et 20 de pur bruit) pour 120 clients d'entraînement ; comparer les moindres carrés, un modèle de référence qui connaît les vraies variables, Ridge, Lasso et Elastic Net sur 956 clients de test.

Étape 1 — fabriquer les 35 variables.

```
rng = np.random.default_rng(31)
ville = bq["id_client"].map(clients.set_index("id_client")["ville"])
villes = pd.get_dummies(ville, prefix="ville", drop_first=True, dtype=float)      # 5 indicatrices
↳ (Autre = référence)
F = pd.DataFrame({
    "age": bq["a"].astype(float), "age_carre": bq["a"].astype(float) ** 2, "age_cube":
    ↳ bq["a"].astype(float) ** 3,
    "canal_Site": (bq["canal"] == "Site").astype(float), "canal_Reseaux": (bq["canal"] ==
    ↳ "Réseaux").astype(float),
    "offre_bienvenue":
    ↳ bq["id_client"].map(clients.set_index("id_client")["offre_bienvenue"]).astype(float),
    "score_produit": bq["score_produit"], "score_service": bq["score_service"],
    "age_x_Site": bq["a"] * (bq["canal"] == "Site").astype(float), "age_x_Reseaux": bq["a"] *
    ↳ (bq["canal"] == "Réseaux").astype(float),
})
F = pd.concat([F, villes], axis=1)
bruit = pd.DataFrame(rng.normal(size=(len(F), 20)), columns=[f"bruit_{i+1:02d}" for i in range(20)])
F = pd.concat([F, bruit], axis=1)
signal = ["age", "canal_Site", "canal_Reseaux", "score_produit"]                # les variables qui ont
↳ un VRAI effet dans la simulation
print("nombre de variables candidates :", F.shape[1], "| dont du pur bruit :", 20, "| clients :",
    ↳ len(F))
```

nombre de variables candidates : 35 | dont du pur bruit : 20 | clients : 1076

```
ordre = rng.permutation(len(F))
tr, te = ordre[:120], ordre[120:]
sc = StandardScaler().fit(F.iloc[tr])
Ztr, Zte = sc.transform(F.iloc[tr]), sc.transform(F.iloc[te])
ytr, yte = yv[tr], yv[te]
print("entraînement :", len(tr), "clients | test :", len(te), "clients")
```

entraînement : 120 clients | test : 956 clients

Étape 2 — entraîner et comparer. λ est choisi par validation croisée à 10 paquets, sur l'échantillon d'entraînement seulement.

```
def rmse(y, p):
    return float(np.sqrt(np.mean((y - p) ** 2)))

noms = list(F.columns)
idx_signal = [noms.index(v) for v in signal]
modeles = {}
modeles["MCO, 35 variables"] = LinearRegression().fit(Ztr, ytr)
ref = LinearRegression().fit(Ztr[:, idx_signal], ytr)
ridge = RidgeCV(alphas=np.logspace(-1, 3.5, 60), cv=10).fit(Ztr, ytr)
lasso = LassoCV(alphas=100, cv=10, random_state=0, max_iter=100000).fit(Ztr, ytr)
enet = ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8, 0.95], alphas=60, cv=10, random_state=0,
    ↪ max_iter=100000).fit(Ztr, ytr)
modeles.update({"Ridge ( $\lambda$  par CV)": ridge, "Lasso ( $\alpha$  par CV)": lasso, "Elastic Net (CV)": enet})

lignes = [{"modèle": "constante seule", "RMSE apprentissage": rmse(ytr, np.full(len(ytr),
    ↪ ytr.mean())), "RMSE test": rmse(yte, np.full(len(yte), ytr.mean())),
    "variables non nulles": 0, "dont bruit": 0}]
lignes.append({"modèle": "référence : 4 vraies variables", "RMSE apprentissage": rmse(ytr,
    ↪ ref.predict(Ztr[:, idx_signal])),
    "RMSE test": rmse(yte, ref.predict(Zte[:, idx_signal])), "variables non nulles": 4,
    ↪ "dont bruit": 0})
for nom, m in modeles.items():
    coef = m.coef_
    non_nuls = np.abs(coef) > 1e-10
    lignes.append({"modèle": nom, "RMSE apprentissage": rmse(ytr, m.predict(Ztr)), "RMSE test":
    ↪ rmse(yte, m.predict(Zte)),
    "variables non nulles": int(non_nuls.sum()), "dont bruit": int(sum(non_nuls[i] for
    ↪ i, n in enumerate(noms) if n.startswith("bruit")))})
res = pd.DataFrame(lignes).set_index("modèle")
print(res.round(4).to_string())
print()
print(f"Ridge : lambda choisi = {ridge.alpha_:.1f} | Lasso : alpha choisi = {lasso.alpha_:.4f} |
    ↪ Elastic Net : alpha = {enet.alpha_:.4f}, rho = {enet.l1_ratio}")
```

modèle	RMSE apprentissage	RMSE test	variables non nulles	dont bruit
constante seule	0.3803	0.4039	0	0
référence : 4 vraies variables	0.3093	0.3612	4	0
MCO, 35 variables	0.2728	0.4152	35	20
Ridge (λ par CV)	0.3200	0.3765	35	20
Lasso (α par CV)	0.3032	0.3609	15	7
Elastic Net (CV)	0.3038	0.3608	15	7

Ridge : lambda choisi = 134.0 | Lasso : alpha choisi = 0.0250 | Elastic Net : alpha = 0.0266, rho = 0.95

Étape 3 — lire les coefficients et les chemins de régularisation.

```
coef = pd.DataFrame({"MCO": modeles["MCO, 35 variables"].coef_, "Ridge": ridge.coef_, "Lasso":
    ↪ lasso.coef_, "Elastic Net": enet.coef_}, index=noms)
vus = coef[(coef["Lasso"].abs() > 1e-10) | coef.index.isin(signal)]
print("coefficients (variables standardisées) pour les variables retenues par le Lasso ou vraiment
    ↪ utiles :")
print(vus.round(3).to_string())
print()
print("variables de bruit : plus grand |coefficient| MCO =",
    ↪ round(coef.loc[coef.index.str.startswith("bruit"), "MCO"].abs().max(), 3),
    "| Ridge =", round(coef.loc[coef.index.str.startswith("bruit"), "Ridge"].abs().max(), 3),
```

```
" | Lasso =", round(coef.loc[coef.index.str.startswith("bruit"), "Lasso"].abs().max(), 3))
```

coefficients (variables standardisées) pour les variables retenues par le Lasso ou vraiment utiles :

	MC0	Ridge	Lasso	Elastic Net
age	0.165	0.033	0.050	0.050
age_carre	0.052	0.015	0.011	0.010
canal_Site	-0.174	-0.033	-0.091	-0.090
canal_Reseaux	-0.218	-0.056	-0.132	-0.131
score_produit	0.106	0.053	0.096	0.095
age_x_Reseaux	-0.005	0.022	0.026	0.026
ville_Ville D	-0.067	-0.021	-0.013	-0.013
ville_Ville E	-0.037	0.013	0.004	0.003
bruit_01	0.073	0.024	0.025	0.024
bruit_02	0.060	0.012	0.013	0.012
bruit_11	-0.034	-0.016	-0.005	-0.004
bruit_13	-0.052	-0.013	-0.004	-0.003
bruit_14	0.020	0.020	0.021	0.021
bruit_19	0.013	0.015	0.004	0.003
bruit_20	-0.054	-0.021	-0.014	-0.014

variables de bruit : plus grand |coefficient| MC0 = 0.073 | Ridge = 0.024 | Lasso = 0.025

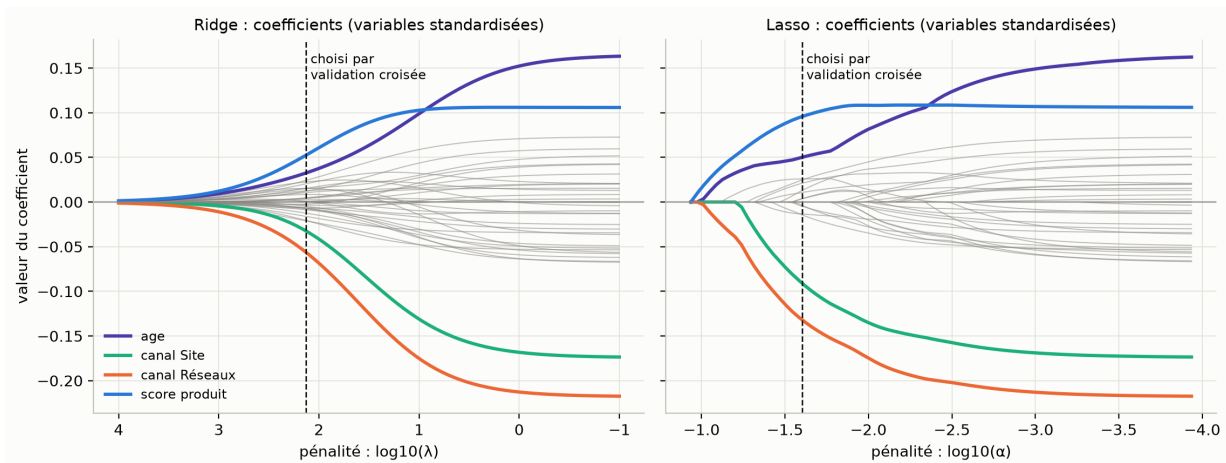


Figure 8: Chemins de régularisation : Ridge (à gauche) et Lasso (à droite).

Application 1.10 — Huber à la main et données contaminées (sections 1.6.1 à 1.6.4)

Objectif. Écrire l'algorithme des moindres carrés repondérés (IRLS) pour la fonction de Huber, puis mesurer les dégâts d'une contamination (6 % de paniers multipliés par 10) sur les moindres carrés et sur Huber ; enfin voir la limite : les points à fort levier.

Étape 1 — les données propres et les fonctions de perte.

```
import statsmodels.api as sm
df = clients[clients["nb_commandes_an"] > 0].copy().reset_index(drop=True)
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Réseaux"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
m_propre = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
```

Étape 2 — l'IRLS de Huber contre RLM.

```
def huber_irls(X, y, c=1.345, n_iter=100, tol=1e-10):
    beta = np.linalg.lstsq(X, y, rcond=None)[0]
    for _ in range(n_iter):
        # départ : les moindres carrés
```

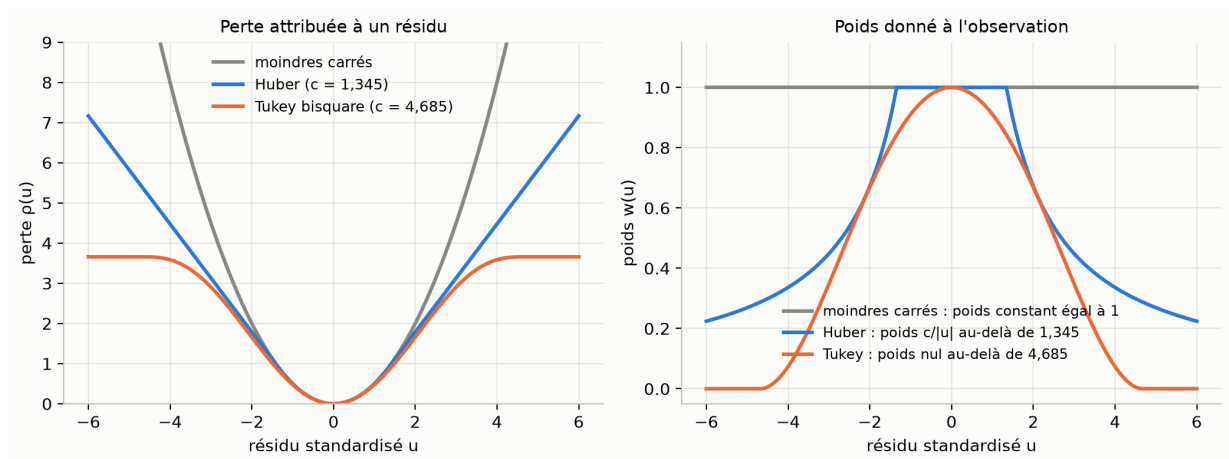


Figure 9: Fonctions de perte et de poids : moindres carrés, Huber, Tukey.

```

e = y - X @ beta
s = np.median(np.abs(e - np.median(e))) / 0.6745 # échelle robuste (MAD)
u = e / s
w = np.where(np.abs(u) <= c, 1.0, c / np.abs(u)) # poids de Huber
W = X.T * w # X' diag(w)
beta_new = np.linalg.solve(W @ X, W @ y) # régression pondérée
if np.max(np.abs(beta_new - beta)) < tol:
    beta = beta_new
    break
beta = beta_new
return beta, w, s

```

```

Xp, yp = m_propre.model.exog, m_propre.model.endog
b_main, w_main, s_main = huber_irls(Xp, yp)
rlm = sm.RLM(yp, Xp, M=sm.robust.norms.HuberT(t=1.345)).fit()
print(pd.DataFrame({"MCO": m_propre.params, "Huber (à la main)": b_main, "RLM statsmodels":
    ↳ rlm.params}).round(4).to_string())
print(f"échelle robuste s : {s_main:.4f} | écart-type résiduel des MCO :
    ↳ {np.sqrt(m_propre.scale):.4f}")
print(f"part des clients dont le poids est inférieur à 1 : {np.mean(w_main < 1):.1%}")

```

	MCO	Huber (à la main)	RLM statsmodels
Intercept	4.2206	4.2171	4.2172
C(canal)[T.Site]	-0.1571	-0.1602	-0.1602
C(canal)[T.Réseaux]	-0.3368	-0.3331	-0.3332
a	0.0092	0.0090	0.0090

échelle robuste s : 0.3667 | écart-type résiduel des MCO : 0.3705
part des clients dont le poids est inférieur à 1 : 18.1%

Étape 3 — la contamination des paniers. 6 % des paniers sont multipliés par 10 (une virgule mal placée).

```

rng = np.random.default_rng(77)
dfc = df.copy()
cible = rng.choice(len(dfc), size=int(0.06 * len(dfc)), replace=False) # 6 % de clients touchés
dfc.loc[cible, "panier_moyen"] = dfc.loc[cible, "panier_moyen"] * 10
dfc["log_panier"] = np.log(dfc["panier_moyen"])
contamine = np.zeros(len(dfc), dtype=bool); contamine[cible] = True
print(len(cible), "paniers corrompus sur", len(dfc), f"({contamine.mean():.1%})")

```

104 paniers corrompus sur 1740 (6.0%)

```

m_mco_c = smf.ols("log_panier ~ a + C(canal)", data=dfc).fit()
rlm_c = smf.rlm("log_panier ~ a + C(canal)", data=dfc, M=sm.robust.norms.HuberT(t=1.345)).fit()
tab = pd.DataFrame({"MCO données propres": m_propre.params, "MCO contaminé": m_mco_c.params, "Huber
↳ contaminé": rlm_c.params})
tab_se = pd.DataFrame({"MCO données propres": m_propre.bse, "MCO contaminé": m_mco_c.bse, "Huber
↳ contaminé": rlm_c.bse})
print("Coefficients :"); print(tab.round(4).to_string())
print("\nErreurs standard :"); print(tab_se.round(4).to_string())
print(f"\nécart-type résiduel : MCO propre = {np.sqrt(m_propre.scale):.3f} | MCO contaminé =
↳ {np.sqrt(m_mco_c.scale):.3f} | échelle robuste de Huber = {rlm_c.scale:.3f}")
print(f"panier médian prédit pour un client Boutique de 36 ans : propre =
↳ {np.exp(m_propre.params['Intercept']):.1f} € | MCO contaminé =
↳ {np.exp(m_mco_c.params['Intercept']):.1f} € | Huber contaminé =
↳ {np.exp(rlm_c.params['Intercept']):.1f} €")
poids = rlm_c.weights
print(f"poids de Huber moyen : clients corrompus = {poids[contamine].mean():.2f} | clients intacts =
↳ {poids[~contamine].mean():.2f}")

```

Coefficients :

	MCO données propres	MCO contaminé	Huber contaminé
Intercept	4.2206	4.3898	4.2628
C(canal)[T.Site]	-0.1571	-0.1853	-0.1573
C(canal)[T.Réseaux]	-0.3368	-0.3919	-0.3527
a	0.0092	0.0088	0.0091

Erreurs standard :

	MCO données propres	MCO contaminé	Huber contaminé
Intercept	0.0175	0.0314	0.0201
C(canal)[T.Site]	0.0231	0.0414	0.0265
C(canal)[T.Réseaux]	0.0225	0.0403	0.0258
a	0.0008	0.0015	0.0010

écart-type résiduel : MCO propre = 0.370 | MCO contaminé = 0.665 | échelle robuste de Huber = 0.402
 panier médian prédit pour un client Boutique de 36 ans : propre = 68.1 € | MCO contaminé = 80.6 € |
 ↳ Huber contaminé = 71.0 €
 poids de Huber moyen : clients corrompus = 0.24 | clients intacts = 0.97

Étape 4 — la limite : les âges mal saisis. Ils créent des points à très fort levier, contre lesquels Huber ne protège pas.

```

rng = np.random.default_rng(78)
dfl = df.copy()
cible_l = rng.choice(len(df), size=int(0.01 * len(df)), replace=False)
dfl.loc[cible_l, "age"] = dfl.loc[cible_l, "age"] * 10
dfl["a"] = dfl["age"] - 36
print(len(cible_l), "âges corrompus ; âge maximal après contamination :", int(dfl["age"].max()),
↳ "ans")
m_mco_l = smf.ols("log_panier ~ a + C(canal)", data=dfl).fit()
rlm_l = smf.rlm("log_panier ~ a + C(canal)", data=dfl, M=sm.robust.norms.HuberT(t=1.345)).fit()
print(pd.DataFrame({"MCO propre": m_propre.params, "MCO âges corrompus": m_mco_l.params, "Huber âges
↳ corrompus": rlm_l.params}).round(4).to_string())
print(f"coefficient de l'âge : propre = {m_propre.params['a']:.4f} | MCO corrompu =
↳ {m_mco_l.params['a']:.4f} | Huber corrompu = {rlm_l.params['a']:.4f}")
print(f"levier maximal : {m_mco_l.get_influence().hat_matrix_diag.max():.3f} (levier moyen =
↳ {4/len(dfl):.4f})")

```

17 âges corrompus ; âge maximal après contamination : 640 ans

	MCO propre	MCO âges corrompus	Huber âges corrompus
Intercept	4.2206	4.2156	4.2126
C(canal)[T.Site]	-0.1571	-0.1575	-0.1603
C(canal)[T.Réseaux]	-0.3368	-0.3349	-0.3338
a	0.0092	0.0008	0.0009

coefficient de l'âge : propre = 0.0092 | MCO corrompu = 0.0008 | Huber corrompu = 0.0009

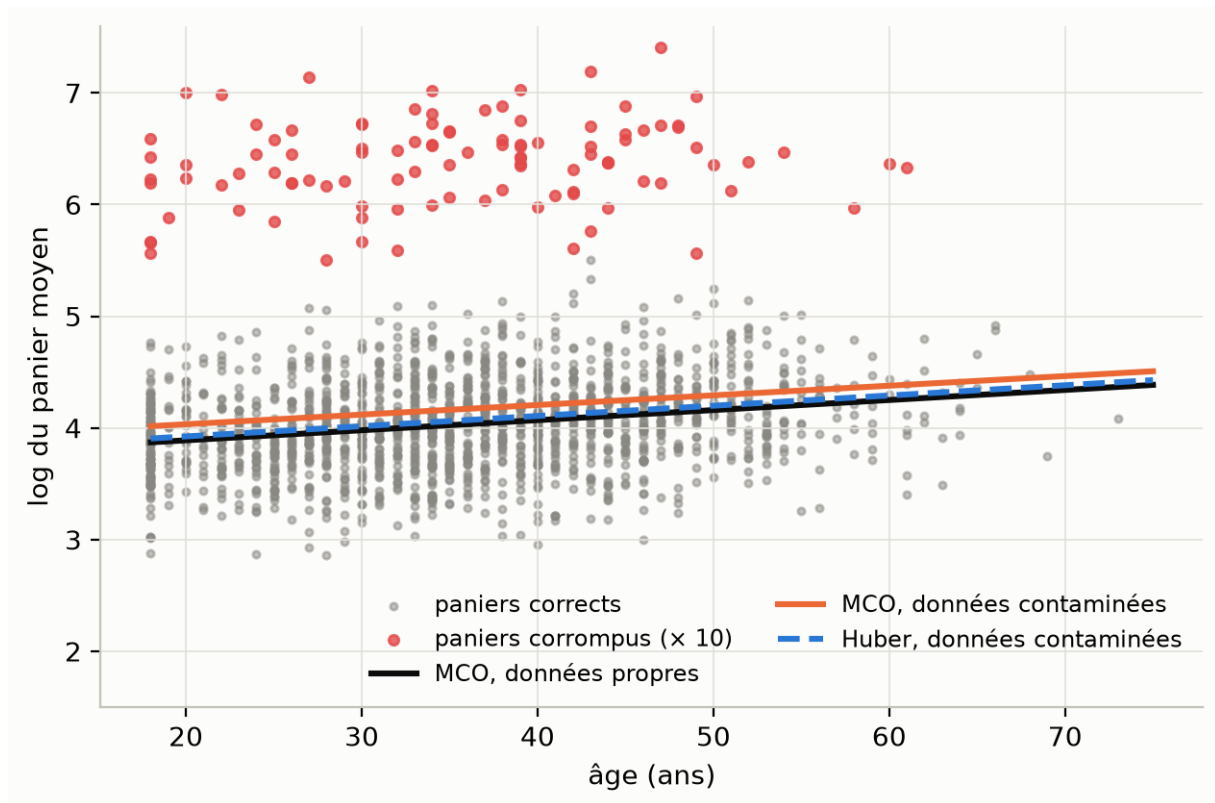


Figure 10: Log-panier selon l'âge avec 6 % de paniers corrompus : droites des moindres carrés et de Huber.

levier maximal : 0.126 (levier moyen = 0.0023)

Application 1.11 — La régression quantile (section 1.6.5)

Objectif. Décrire l'effet du canal Réseaux sur toute la distribution du panier, en euros puis en logarithme.

```
taus = [0.1, 0.25, 0.5, 0.75, 0.9]
res_niveau, res_log = [], []
for tau in taus:
    qn = smf.quantreg("panier_moyen ~ a + C(canal)", data=df).fit(q=tau)
    ql = smf.quantreg("log_panier ~ a + C(canal)", data=df).fit(q=tau)
    res_niveau.append((tau, qn.params["C(canal)[T.Réseaux]"],
    ↪ *qn.conf_int().loc["C(canal)[T.Réseaux]"])))
    res_log.append((tau, ql.params["C(canal)[T.Réseaux]"],
    ↪ *ql.conf_int().loc["C(canal)[T.Réseaux]"])))
rn = pd.DataFrame(res_niveau, columns=["tau", "coef", "bas", "haut"]).set_index("tau")
rl = pd.DataFrame(res_log, columns=["tau", "coef", "bas", "haut"]).set_index("tau")
mco_n = smf.ols("panier_moyen ~ a + C(canal)", data=df).fit()
print("Effet de Réseaux (par rapport à la Boutique) sur le panier en €, par quantile :")
print(rn.round(2).to_string())
print(f"(MCO, effet sur la moyenne : {mco_n.params['C(canal)[T.Réseaux]']:.2f} €)")
print("\nEffet de Réseaux sur le log du panier, par quantile :")
print(rl.round(3).to_string())
print(f"(MCO, effet sur la moyenne du log : {m_propre.params['C(canal)[T.Réseaux]']:.3f})")
```

```
Effet de Réseaux (par rapport à la Boutique) sur le panier en €, par quantile :
      coef  bas  haut
tau
```

```
0.10 -13.50 -16.18 -10.82
0.25 -14.40 -16.92 -11.89
0.50 -18.27 -21.44 -15.09
0.75 -24.22 -28.42 -20.02
0.90 -33.87 -40.70 -27.04
(MCO, effet sur la moyenne : -20.81 €)
```

Effet de Réseaux sur le log du panier, par quantile :

	coef	bas	haut
tau			
0.10	-0.373	-0.448	-0.298
0.25	-0.331	-0.387	-0.275
0.50	-0.310	-0.369	-0.252
0.75	-0.339	-0.400	-0.278
0.90	-0.372	-0.446	-0.299

(MCO, effet sur la moyenne du log : -0.337)

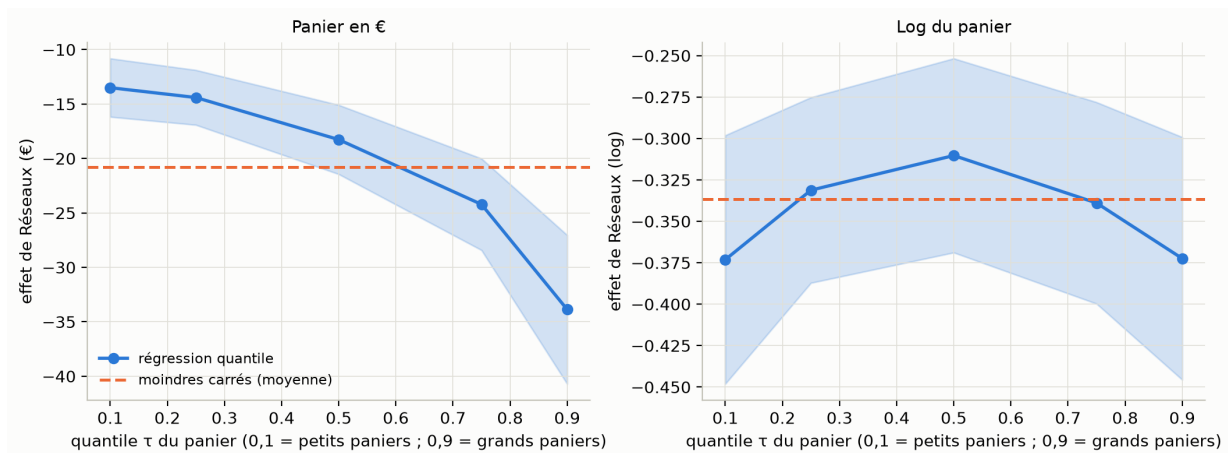


Figure 11: Effet du canal Réseaux selon le quantile du panier, en euros (à gauche) et en log (à droite).

Application 1.12 — Points relais : un modèle mixte de bout en bout (section 1.7)

Objectif. Simuler 30 points relais de tailles inégales, comparer trois traitements des groupes (mise en commun totale, nulle, partielle), ajuster un modèle à intercept puis à pente aléatoires, vérifier la formule du rétrécissement et mesurer par simulation le danger d'ignorer les groupes.

Étape 1 — simuler les relais. Le modèle vrai : $\text{note}_{ij} = 12 + 1,2 \text{ urbain}_j + u_{0j} + (-0,7 + u_{1j})(\text{délai}_{ij} - 5) + \varepsilon_{ij}$.

```
import warnings
```

```
# Simulation des 30 points relais (graine fixe)
rng = np.random.default_rng(41)
J = 30
tailles = np.clip(np.round(rng.lognormal(np.log(12), 0.8, J)), 3, 60).astype(int) # commandes par
↳ relais (très inégal)
urbain = (rng.random(J) < 0.5).astype(int)
u0 = rng.normal(0, 1.6, J) # vrai niveau de
↳ base de chaque relais
u1 = rng.normal(0, 0.3, J) # vraie sensibilité
↳ au retard de chaque relais
lignes = []
for j in range(J):
    delai = rng.integers(1, 11, tailles[j])
```

```

note = 12 + 1.2 * urbain[j] + u0[j] + (-0.7 + u1[j]) * (delai - 5) + rng.normal(0, 1.8, tailles[j])
for d, y in zip(delai, note):
    lignes.append((f"R{j + 1:02d}", urbain[j], int(d), round(float(y), 2)))
rel = pd.DataFrame(lignes, columns=["relais", "urbain", "delai", "note"])
rel.to_csv("donnees/ch01-relais.csv", index=False) # fichier fourni avec le livre (lu aussi
↳ par R plus bas)
rel["dc"] = rel["delai"] - 5 # délai centré : 0 = délai de 5 jours
print(rel.shape, "| relais :", rel["relais"].nunique(), "| relais urbains :", int(urbain.sum()))
print("commandes par relais : min =", tailles.min(), "| médiane =", int(np.median(tailles)), "| max
↳ =", tailles.max())
print("écart-type RÉALISÉ des 30 vrais niveaux de base u0 :", round(u0.std(ddof=1), 2), "(la loi dont
↳ ils sont tirés a un écart-type de 1,6)")
print(rel.head(5).to_string(index=False))

```

```

(484, 5) | relais : 30 | relais urbains : 14
commandes par relais : min = 4 | médiane = 11 | max = 58
écart-type RÉALISÉ des 30 vrais niveaux de base u0 : 1.31 (la loi dont ils sont tirés a un écart-type
↳ de 1,6)
relais  urbain  delai  note  dc
R01      0      6  12.57   1
R01      0      4  11.23  -1
R01      0      8   4.98   3
R01      0     10   7.56   5
R02      1      6   9.91   1

```

Étape 2 — mise en commun totale ou nulle.

```

m_commun = smf.ols("note ~ dc + urbain", data=rel).fit() # ignore les groupes
m_separe = smf.ols("note ~ dc + C(relais)", data=rel).fit() # une indicatrice
↳ par relais (urbain y est absorbé)
print("Mise en commun totale (MCO, groupes ignorés) :")
print(pd.DataFrame({"coef": m_commun.params, "erreur standard": m_commun.bse, "IC bas":
↳ m_commun.conf_int()[0], "IC haut": m_commun.conf_int()[1]}).round(3).to_string())
print(f"\nrésidu : s = {np.sqrt(m_commun.scale):.2f}")
print("\nPas de mise en commun : l'effet « urbain » n'est plus estimable (il est confondu avec les
↳ indicatrices de relais).")
print(f"coefficient du délai avec indicatrices de relais : {m_separe.params['dc']:.3f} (erreur
↳ standard {m_separe.bse['dc']:.3f})")

```

```

Mise en commun totale (MCO, groupes ignorés) :
      coef  erreur standard  IC bas  IC haut
Intercept  12.644           0.152  12.345  12.943
dc         -0.721           0.039  -0.796  -0.645
urbain      0.482           0.224   0.043   0.922

```

résidu : s = 2.44

```

Pas de mise en commun : l'effet « urbain » n'est plus estimable (il est confondu avec les indicatrices
↳ de relais).
coefficient du délai avec indicatrices de relais : -0.703 (erreur standard 0.034)

```

Étape 3 — l'ICC : les groupes comptent-ils ? D'abord les six villes (réponse : non), puis les relais (réponse : oui).

```

clients = pd.read_csv("donnees/clients.csv")
cl = clients[clients["nb_commandes_an"] > 0].copy().reset_index(drop=True)
cl["canal"] = pd.Categorical(cl["canal_acquisition"], categories=["Boutique", "Site", "Réseaux"])
cl["a"] = cl["age"] - 36
cl["log_panier"] = np.log(cl["panier_moyen"])
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    m_ville = smf.mixedlm("log_panier ~ a + C(canal)", cl, groups=cl["ville"]).fit(reml=True)
tau2_v, sigma2_v = m_ville.cov_re.iloc[0, 0], m_ville.scale

```

```
print(f"clients groupés par ville : variance entre villes tau² = {tau2_v:.5f} | variance résiduelle
↳ sigma² = {sigma2_v:.4f}")
print(f"ICC = tau²/(tau² + sigma²) = {tau2_v / (tau2_v + sigma2_v):.4f}")
```

```
clients groupés par ville : variance entre villes tau² = 0.00000 | variance résiduelle sigma² = 0.1373
ICC = tau²/(tau² + sigma²) = 0.0000
```

```
m_ri = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"]).fit(reml=True)
tau2, sigma2 = m_ri.cov_re.iloc[0, 0], m_ri.scale
print(m_ri.summary().tables[1])
print(f"\nvariance entre relais tau² = {tau2:.3f} (écart-type {np.sqrt(tau2):.2f}) | variance
↳ résiduelle sigma² = {sigma2:.3f} (écart-type {np.sqrt(sigma2):.2f})")
print(f"ICC = {tau2 / (tau2 + sigma2):.3f} : environ {100 * tau2 / (tau2 + sigma2):.0f} % de la
↳ variance des notes (après effet du délai et du type de relais) est due au relais")
```

	Coef.	Std.Err.	z	P> z	[0.025	0.975]
Intercept	12.489	0.364	34.284	0.000	11.775	13.202
dc	-0.710	0.034	-21.047	0.000	-0.776	-0.644
urbain	0.377	0.534	0.705	0.481	-0.671	1.424
Group Var	1.711	0.280				

```
variance entre relais tau² = 1.711 (écart-type 1.31) | variance résiduelle sigma² = 4.350 (écart-type
↳ 2.09)
ICC = 0.282 : environ 28 % de la variance des notes (après effet du délai et du type de relais) est
↳ due au relais
```

Étape 4 — le rétrécissement (BLUP). Pour un relais de n_j commandes, $\hat{u}_j = \frac{\tau^2}{\tau^2 + \sigma^2/n_j} \bar{r}_j$; on le vérifie, puis on compare à la vérité.

```
beta_ri = m_ri.fe_params
rel["r"] = rel["note"] - (beta_ri["Intercept"] + beta_ri["dc"] * rel["dc"] + beta_ri["urbain"] *
↳ rel["urbain"]) # résidus « fixes »
groupes = rel.groupby("relais")["r"].agg(["mean", "size"]).rename(columns={"mean": "r_bar", "size":
↳ "n_j"})
groupes["B"] = tau2 / (tau2 + sigma2 / groupes["n_j"])
groupes["u_formule"] = groupes["B"] * groupes["r_bar"]
groupes["u_statsmodels"] = [m_ri.random_effects[g]["Group"] for g in groupes.index]
groupes["u_vrai"] = u0
print("formule du BLUP = statsmodels :", np.allclose(groupes["u_formule"], groupes["u_statsmodels"]))
print(groupes.sort_values("n_j").iloc[[0, 1, 2, 14, 27, 28, 29]].round(3).to_string())
erreur_sans = np.sqrt(np.mean((groupes["r_bar"] - groupes["u_vrai"])**2))
erreur_part = np.sqrt(np.mean((groupes["u_statsmodels"] - groupes["u_vrai"])**2))
print(f"\nerreur quadratique moyenne par rapport aux vrais niveaux u0 : sans mise en commun =
↳ {erreur_sans:.3f} | mise en commun partielle = {erreur_part:.3f}")
petits = groupes["n_j"] <= 8
print(f" pour les {petits.sum()} petits relais (8 commandes ou moins) : sans =
↳ {np.sqrt(np.mean((groupes.loc[petits, 'r_bar'] - groupes.loc[petits, 'u_vrai']**2)):.3f} |
↳ partielle = {np.sqrt(np.mean((groupes.loc[petits, 'u_statsmodels'] - groupes.loc[petits,
↳ 'u_vrai']**2)):.3f}")
```

```
formule du BLUP = statsmodels : True
```

relais	r_bar	n_j	B	u_formule	u_statsmodels	u_vrai
R01	-1.983	4	0.611	-1.213	-1.213	-0.937
R05	0.997	4	0.611	0.610	0.610	-0.683
R10	-2.442	4	0.611	-1.493	-1.493	0.413
R19	-0.426	11	0.812	-0.346	-0.346	0.624
R12	-0.644	31	0.924	-0.595	-0.595	-0.548
R16	1.386	56	0.957	1.326	1.326	1.867
R29	1.257	58	0.958	1.204	1.204	0.725

```
erreur quadratique moyenne par rapport aux vrais niveaux u0 : sans mise en commun = 0.900 | mise en
↳ commun partielle = 0.765
```

pour les 10 petits relais (8 commandes ou moins) : sans = 1.273 | partielle = 1.031

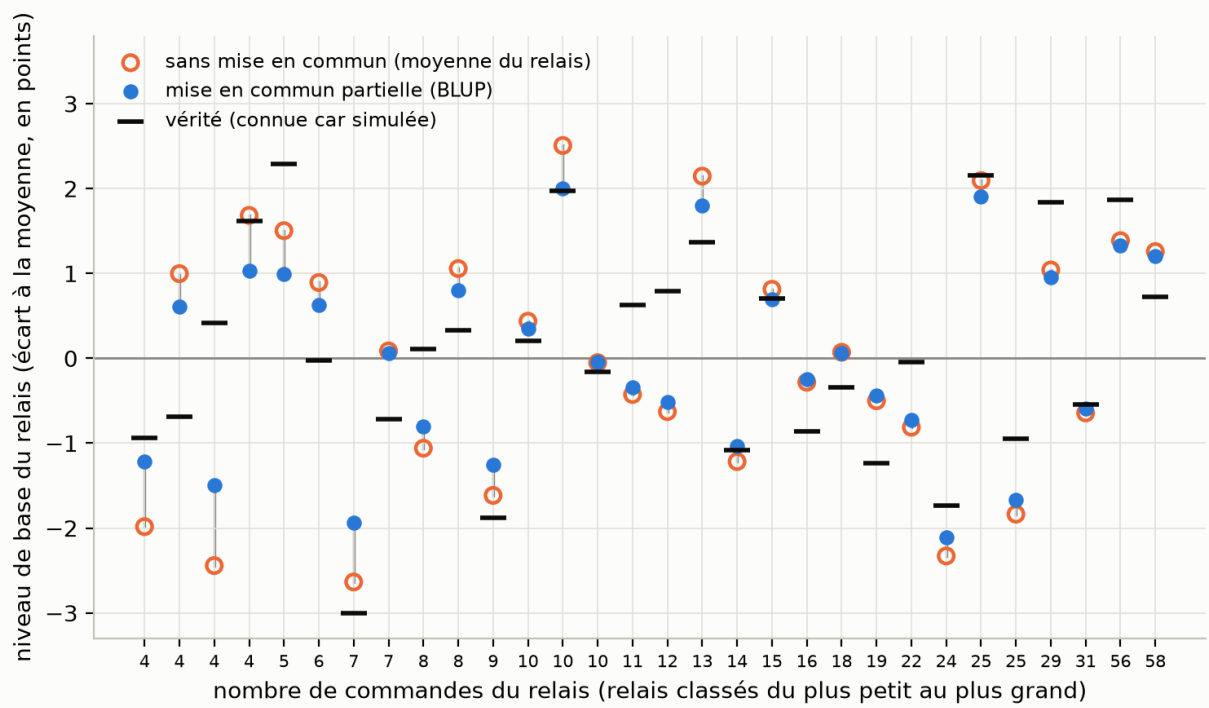


Figure 12: Niveau de base des 30 relais : moyennes brutes, estimations rétrécies et vérité.

Étape 5 — la pente aléatoire et le test du rapport de vraisemblance. Sous H_0 la variance est au bord de l'espace des paramètres : mélange 50/50 de χ_1^2 et χ_2^2 .

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    m_rs = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"],
    ↪ re_formula="~dc").fit(reml=True, method="lbfgs")
print("Converge :", m_rs.converged)
print(m_rs.summary().tables[1])
G = m_rs.cov_re
print(f"écart-type des niveaux de base : {np.sqrt(G.iloc[0, 0]):.3f} (vrai : 1,6) | écart-type des
↪ pentes : {np.sqrt(G.iloc[1, 1]):.3f} (vrai : 0,3)")
print(f"corrélacion niveau de base / pente : {G.iloc[0, 1] / np.sqrt(G.iloc[0, 0] * G.iloc[1, 1]):.2f}
↪ (vraie : 0) | écart-type résiduel : {np.sqrt(m_rs.scale):.3f} (vrai : 1,8)")
```

Converge : True

	Coef.	Std.Err.	z	P> z	[0.025	0.975]
Intercept	12.596	0.339	37.166	0.000	11.932	13.260
dc	-0.760	0.064	-11.806	0.000	-0.886	-0.634
urbain	0.277	0.501	0.553	0.581	-0.705	1.259
Group Var	1.427	0.252				
Group x dc Cov	0.072	0.044				
dc Var	0.079	0.017				

écart-type des niveaux de base : 1.195 (vrai : 1,6) | écart-type des pentes : 0.281 (vrai : 0,3)
 corrélation niveau de base / pente : 0.21 (vraie : 0) | écart-type résiduel : 1.930 (vrai : 1,8)

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    ml_ri = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"]).fit(reml=False) #
    ↪ optimiseur par défaut (voir la remarque ci-dessous)
    ml_rs = smf.mixedlm("note ~ dc + urbain", rel, groups=rel["relais"],
    ↪ re_formula="~dc").fit(reml=False, method="lbfgs")
```

```

LR = 2 * (ml_rs.llf - ml_ri.llf)
# Sous H0, la variance de la pente est sur le bord de l'espace des paramètres (>= 0) : loi limite =
  ↳ mélange 50/50 de chi²(1) et chi²(2)
p_val = 0.5 * stats.chi2.sf(LR, 1) + 0.5 * stats.chi2.sf(LR, 2)
print(f"log-vraisemblance (ML) : intercept aléatoire = {ml_ri.llf:.2f} | intercept + pente aléatoires
  ↳ = {ml_rs.llf:.2f}")
print(f"LR = {LR:.2f} | p-valeur (mélange de chi²) = {p_val:.2e}")
print(f"AIC : {-2 * ml_ri.llf + 2 * 5:.1f} (intercept aléatoire, 5 paramètres) contre {-2 * ml_rs.llf
  ↳ + 2 * 7:.1f} (intercept + pente, 7 paramètres)")

```

```

log-vraisemblance (ML) : intercept aléatoire = -1067.99 | intercept + pente aléatoires = -1047.41
LR = 41.15 | p-valeur (mélange de chi²) = 6.50e-10
AIC : 2146.0 (intercept aléatoire, 5 paramètres) contre 2108.8 (intercept + pente, 7 paramètres)

```

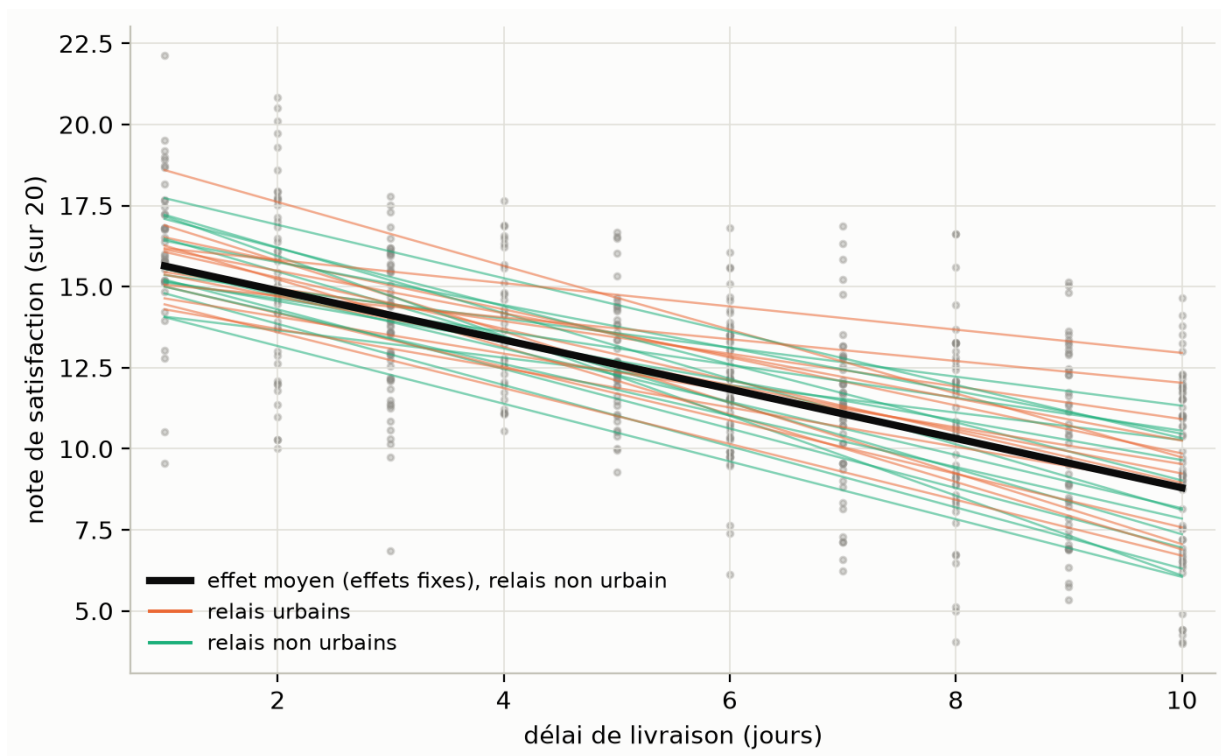


Figure 13: Droites de régression propres à chaque relais autour de l'effet moyen.

Étape 6 — la même chose avec lme4 en R.

```

suppressMessages(library(lme4))
d <- read.csv("donnees/ch01-relais.csv")
d$dc <- d$delai - 5
m <- lmer(note ~ dc + urbain + (1 + dc | relais), data = d)
print(round(fixef(m), 4))
print(VarCorr(m), digits = 4)
cat("log-vraisemblance REML :", round(as.numeric(logLik(m)), 3), "\n")

```

```

(Intercept)      dc      urbain
    12.5960    -0.7598     0.2769
Groups   Name      Std.Dev. Corr
relais   (Intercept) 1.1948
          dc         0.2812  0.21
Residual                    1.9299
log-vraisemblance REML : -1049.555

```

Étape 7 — pourquoi ignorer les groupes est dangereux. On simule 300 jeux de données où le

type de relais n'a **aucun** effet : la régression ordinaire rejette dans plus de la moitié des cas.

```
rng2 = np.random.default_rng(2024)
R = 300
rej_mco, rej_mixte, z_mixte, tau2_est = 0, 0, [], []
degenere_lbfgs, degenere_default = 0, 0
groupe = np.repeat(np.arange(J), tailles)
urb_obs = urbain[groupe]
dc_obs = rel["dc"].to_numpy()
for r in range(R):
    u = rng2.normal(0, 1.6, J)
    y = 12 + u[groupe] - 0.7 * dc_obs + rng2.normal(0, 1.8, len(groupe))      # AUCUN
    ↪ effet du type de relais
    d_sim = pd.DataFrame({"note": y, "dc": dc_obs, "urbain": urb_obs, "g": groupe})
    p_mco = sm.OLS(y, sm.add_constant(d_sim[["dc", "urbain"]])).fit().pvalues["urbain"]
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        f = smf.mixedlm("note ~ dc + urbain", d_sim, groups=d_sim["g"]).fit(reml=True)      #
    ↪ optimiseur par défaut
        if r < 60:
            ↪ même ajustement avec lbfgs, sur 60 répétitions
                f2 = smf.mixedlm("note ~ dc + urbain", d_sim, groups=d_sim["g"]).fit(reml=True,
            ↪ method="lbfgs")
                degenere_lbfgs += f2.cov_re.iloc[0, 0] < 1e-6
                degenere_default += f.cov_re.iloc[0, 0] < 1e-6
    rej_mco += p_mco < 0.05
    rej_mixte += f.pvalues["urbain"] < 0.05
    z_mixte.append(f.params["urbain"] / f.bse["urbain"])
    tau2_est.append(f.cov_re.iloc[0, 0])

print(f"quand AUCUN effet n'existe, part de tests « significatifs à 5 % » sur la variable urbain, sur
↪ {R} simulations :")
print(f" régression ordinaire (groupes ignorés) : {rej_mco / R:.1%}")
print(f" modèle mixte (intercept aléatoire) : {rej_mixte / R:.1%} | écart-type de la
↪ statistique z : {np.std(z_mixte):.2f} (théorie : 1)")
print(f"variance entre relais estimée en moyenne : {np.mean(tau2_est):.2f} (vraie valeur :
↪ {1.6**2:.2f})")
print(f"sur 60 répétitions, ajustements qui dégénèrent (variance estimée = 0) : optimiseur par défaut
↪ = {degenere_default}/60 | lbfgs = {degenere_lbfgs}/60")
```

```
quand AUCUN effet n'existe, part de tests « significatifs à 5 % » sur la variable urbain, sur 300
↪ simulations :
    régression ordinaire (groupes ignorés) : 55.3%
    modèle mixte (intercept aléatoire) : 6.7% | écart-type de la statistique z : 1.04 (théorie : 1)
    variance entre relais estimée en moyenne : 2.54 (vraie valeur : 2.56)
    sur 60 répétitions, ajustements qui dégénèrent (variance estimée = 0) : optimiseur par défaut = 0/60 |
    ↪ lbfgs = 43/60
```

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. Les exercices 1.1 à 1.8, 1.11 et 1.12 se résolvent surtout à la main ; les exercices 1.9, 1.10 et 1.13 demandent le code.

Exercice 1.1 ★ — Moindres carrés à la main (section 1.1)

Trois commandes ont x articles et un montant y en € : (1, 2), (2, 3), (4, 7). (a) Écrivez $\mathbf{X}$ et $\mathbf{y}$, calculez $\mathbf{X}^\top \mathbf{X}$ et $\mathbf{X}^\top \mathbf{y}$, puis $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1)$. (b) Calculez les valeurs ajustées et les résidus, et vérifiez que $\mathbf{X}^\top \hat{\varepsilon} = \mathbf{0}$. (c) Calculez le R^2 . (d) Quel montant prévoit-on pour 3 articles ?

Exercice 1.2 ★ — Lire un modèle en log (section 1.1)

Le modèle $\log_panier \sim a + C(\text{canal})$ du 1.1.7 donne : constante 4,2206, Site $-0,1571$, Réseaux $-0,3368$, âge centré 0,0092 par année. (a) Exprimez **exactement** (en pourcentage) l'effet du Site et de Réseaux par rapport à la boutique. (b) Quel effet pour 10 années d'âge de plus ? (c) Quel est le panier **médian** prédit d'une cliente de 46 ans acquise par le Site ? (d) Pourquoi dit-on « médian » et non « moyen » ?

Exercice 1.3 ★★ — Régression simple (section 1.1)

Dans la régression simple $y = \beta_0 + \beta_1 x + \varepsilon$, démontrez que $\hat{\beta}_1 = r \frac{s_y}{s_x}$ et que $R^2 = r^2$, où r est le coefficient de corrélation de Pearson entre x et y . Vérifiez-le sur les quatre commandes du 1.1.1 ($x = 1, 2, 3, 4$; $y = 22, 41, 66, 79$).

Exercice 1.4 ★★ — Test et intervalle à la main (section 1.2)

Pour Réseaux, statsmodels donne le coefficient $-0,3368$ et son erreur standard 0,0225, avec $n - p = 1736$ degrés de liberté (quantile de Student à 97,5 % : 1,961). (a) Calculez la statistique t et dites si l'on rejette $H_0 : \beta = 0$ à 5 %. (b) Donnez l'intervalle de confiance à 95 % de β , puis de l'effet multiplicatif e^β exprimé en pourcentage. (c) Pour l'âge : coefficient 0,0092, erreur standard 0,00085 : quelle est la statistique t ?

Exercice 1.5 ★★ — Test F (section 1.2)

Le modèle réduit $\log_panier \sim a$ a une somme des carrés résiduelle $SCR_0 = 269,94$; le modèle complet $\log_panier \sim a + C(\text{canal})$ a $SCR_1 = 238,30$, avec $n - p_1 = 1736$. (a) Calculez la statistique F du test « le canal est inutile ». Combien y a-t-il de contraintes q ? (b) La valeur critique de $F_{2, 1736}$ à 5 % est environ 3,0 : concluez. (c) Si SCR_1 valait 269,70 au lieu de 238,30 (le canal améliorerait à peine l'ajustement), que vaudrait F ? Que concluriez-vous ?

Exercice 1.6 ★★ — Levier (section 1.3)

Quatre clients ont pour âge centré $x = (1, 2, 3, 10)$. (a) Calculez le levier $h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_k (x_k - \bar{x})^2}$ de chacun. Vérifiez que leur somme vaut $p = 2$. (b) Quel client a le plus d'influence potentielle ? (c) Si y_4 augmente de 1, de combien augmente la valeur ajustée $\hat{y}_4$?

Exercice 1.7 ★★ — PRESS (section 1.4)

Reprenez les trois points de l'exercice 1. (a) Calculez les leviers h_{ii} . (b) Calculez les erreurs de prédiction « sans le point » $\hat{\varepsilon}_i / (1 - h_{ii})$ et la somme PRESS. (c) Vérifiez l'une d'elles en réajustant la droite sur les deux autres points. (d) Comparez PRESS à la somme des carrés résiduelle : que constatez-vous ?

Exercice 1.8 ★★ — Multicolinéarité (section 1.3)

- (a) Une variable explicative x_j est expliquée à 99,5 % par les autres ($R_j^2 = 0,995$) : donnez son VIF et le facteur par lequel son erreur standard est multipliée. (b) Avec deux variables explicatives de corrélation 0,9, que vaut leur VIF ? (c) **Code.** Les huit questions de l'enquête de satisfaction (q1 à q8) sont-elles gravement colinéaires entre elles ? Calculez leurs VIF.

Exercice 1.9 ★★ — Prédire une moyenne en euros (section 1.1)

Avec le modèle $\log_panier \sim a + C(\text{canal})$: (a) donnez, pour un client **Réseaux de 25 ans**, le panier médian prédit ; (b) calculez une prédiction du panier **moyen** avec la correction de Duan ; (c) comparez à la moyenne observée des clients Réseaux âgés de 23 à 27 ans. Laquelle des deux prédictions est la plus proche, et pourquoi ?

Exercice 1.10 ★★★ — Régression sur les ventes mensuelles (section 1.3)

Avec `donnees/ventes_mensuelles.csv` (120 mois), ajustez $\log(ca) \sim t + C(\text{mois}) + \text{promo} + \text{covid}$, où t est le numéro du mois (0 à 119), mois le mois civil (1 à 12), promo indique un mois de promotion et covid les mois de mars à juin 2020. (a) Quelle est la croissance annuelle estimée ? (b) Quel est l'effet estimé de décembre par rapport à janvier (en facteur multiplicatif) ? (c) Quel est l'effet du confinement de 2020 sur les ventes, en pourcentage ? (d) Que disent le R^2 et le test de Durbin-Watson sur ce modèle ? Reste-t-il de la structure dans les résidus ?

Exercice 1.11 ★★★ — Ridge, cas orthonormal (section 1.5)

Les colonnes de $\mathbf{X}$ sont orthonormales ($\mathbf{X}^\top \mathbf{X} = \mathbf{I}$). (a) Montrez que $\hat{\beta}_j^{\text{ridge}} = \hat{\beta}_j^{\text{MCO}} / (1 + \lambda)$. (b) Pour un coefficient vrai $\beta = 2$ et $\sigma = 1$, calculez l'erreur quadratique moyenne $\text{EQM}(\lambda) = \frac{\lambda^2 \beta^2 + \sigma^2}{(1 + \lambda)^2}$ pour $\lambda = 0, 0,25$ et 1. (c) Quel λ la minimise ? Vérifiez par le code sur une grille.

Exercice 1.12 ★★ — Modèle mixte, calcul à la main (section 1.7)

Dans un modèle à intercept aléatoire, $\tau^2 = 2,0$ (variance entre groupes) et $\sigma^2 = 6,0$ (variance résiduelle). (a) Calculez l'ICC. (b) Un relais a $n_j = 9$ commandes et une moyenne de résidus « fixes » $\bar{r}_j = 3,2$: calculez le facteur de rétrécissement B_j et le BLUP $\hat{u}_j$. (c) Même question pour un relais de $n_j = 2$ commandes avec la même moyenne de résidus. (d) Avec $J = 20$ relais de 9 commandes chacun, quel est l'effectif effectif pour estimer une variable qui ne varie qu'entre relais ?

Exercice 1.13 ★★ — Robustesse, simulation (section 1.6)

Simulez $n = 100$ points $y = 2 + 1,5x + \varepsilon$ (x uniforme sur $[0, 10]$, $\varepsilon \sim \mathcal{N}(0, 1)$, graine 5), puis ajoutez 25 à y pour 5 points tirés au hasard. Comparez les coefficients des moindres carrés, de Huber et de la régression médiane sur les données propres et corrompues. Que constatez-vous ?

Corrigés

Corrigé 1.1

(a) $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 4 \end{pmatrix}$, $\mathbf{y} = (2, 3, 7)^\top$. $\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 3 & 7 \\ 7 & 21 \end{pmatrix}$ (déterminant $63 - 49 = 14$), $\mathbf{X}^\top \mathbf{y} = (12, 2 + 6 + 28)^\top = (12, 36)^\top$. Donc $\hat{\beta} = \frac{1}{14} \begin{pmatrix} 21 & -7 \\ -7 & 3 \end{pmatrix} \begin{pmatrix} 12 \\ 36 \end{pmatrix} = \frac{1}{14} \begin{pmatrix} 252 - 252 \\ -84 + 108 \end{pmatrix} = \begin{pmatrix} 0 \\ 12/7 \end{pmatrix}$: la droite passe **exactement par l'origine** : $\hat{y} = \frac{12}{7}x \approx 1,714x$. (b) Ajustées : $\frac{12}{7}$, $\frac{24}{7}$, $\frac{48}{7}$; résidus : $\frac{2}{7}$, $-\frac{3}{7}$, $\frac{1}{7}$ (soit 0,286; -0,429; 0,143). Somme : 0 ✓. $\sum x_i \hat{\varepsilon}_i = \frac{2}{7} - \frac{6}{7} + \frac{4}{7} = 0$ ✓. (c) $\text{SCR} = \frac{4+9+1}{49} = \frac{2}{7}$; $\bar{y} = 4$, $\text{SCT} = 4 + 1 + 9 = 14$; $R^2 = 1 - \frac{2/7}{14} = \frac{48}{49} \approx 0,980$. (d) $\hat{y}(3) = \frac{36}{7} \approx 5,14$ €.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import matplotlib
matplotlib.use("Agg")

x = np.array([1.0, 2, 4]); y = np.array([2.0, 3, 7])
X = np.column_stack([np.ones(3), x])
beta = np.linalg.solve(X.T @ X, X.T @ y)
res = y - X @ beta
print("beta =", beta.round(4), "| 12/7 =", round(12 / 7, 4))
print("résidus =", res.round(4), "| X'e =", (X.T @ res).round(10))
print("SCR =", round(res @ res, 4), "| R² =", round(1 - res @ res / ((y - y.mean())**2).sum(), 4), "|
↳ prévision en x = 3 :", round(beta[0] + 3 * beta[1], 4))
```

```
beta = [0.      1.7143] | 12/7 = 1.7143
résidus = [ 0.2857 -0.4286  0.1429] | X'e = [-0.  0.]
SCR = 0.2857 | R² = 0.9796 | prévision en x = 3 : 5.1429
```

Corrigé 1.2

(a) Site : $e^{-0,1571} - 1 = -14,5\%$; Réseaux : $e^{-0,3368} - 1 = -28,6\%$ (et non $-15,7\%$ et $-33,7\%$: l'approximation 100β est mauvaise pour de grands coefficients, 1.1.8). (b) $e^{10 \times 0,0092} - 1 = +9,6\%$. (c) Pour 46 ans, $a = 10$: $\hat{\mu} = 4,2206 - 0,1571 + 10 \times 0,0092 = 4,1555$, donc $e^{4,1555} \approx 63,8$ €. (d) Parce que $e^{\hat{\mu}}$ est la prédiction de la **médiane** de y : pour la moyenne il faudrait la correction $e^{s^2/2}$ ou celle de Duan (1.1.8, et exercice 1.9).

```
for nom, b in [("Site", -0.1571), ("Réseaux", -0.3368)]:
    print(f"{nom}{10s}: {100 * (np.exp(b) - 1):+.1f} %")
print(f"10 ans d'âge : {100 * (np.exp(10 * 0.0092) - 1):+.1f} %")
print(f"panier médian prédit, 46 ans, Site : {np.exp(4.2206 - 0.1571 + 10 * 0.0092):.1f} €")
```

```
Site      : -14.5 %
Réseaux   : -28.6 %
10 ans d'âge : +9.6 %
panier médian prédit, 46 ans, Site : 63.8 €
```

Corrigé 1.3

On sait (1.1.1, équations normales) que $\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}$ avec $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$ et $S_{xx} = \sum (x_i - \bar{x})^2$.

Or $r = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}}$ et $\frac{s_y}{s_x} = \sqrt{S_{yy}/S_{xx}}$, donc $r \frac{s_y}{s_x} = \frac{S_{xy}}{\sqrt{S_{xx}S_{yy}}} \sqrt{\frac{S_{yy}}{S_{xx}}} = \frac{S_{xy}}{S_{xx}} = \hat{\beta}_1$. Pour le R^2 : $\hat{y}_i - \bar{y} =$

$\hat{\beta}_1(x_i - \bar{x})$, donc $\text{SCE} = \hat{\beta}_1^2 S_{xx} = \frac{S_{xy}^2}{S_{xx}}$ et $R^2 = \frac{\text{SCE}}{\text{SCT}} = \frac{S_{xy}^2}{S_{xx}S_{yy}} = r^2$. $\square$

```
x4 = np.array([1.0, 2, 3, 4]); y4 = np.array([22.0, 41, 66, 79])
r = np.corrcoef(x4, y4)[0, 1]
b1 = np.polyfit(x4, y4, 1)[0]
print(f"r = {r:.5f} | r·sy/sx = {r * y4.std(ddof=1) / x4.std(ddof=1):.4f} | pente MCO = {b1:.4f}")
print(f"r² = {r**2:.5f} | R² du modèle = {sm.OLS(y4, sm.add_constant(x4)).fit().rsquared:.5f}")
```

```
r = 0.99350 | r·sy/sx = 19.6000 | pente MCO = 19.6000
r² = 0.98705 | R² du modèle = 0.98705
```

Corrigé 1.4

(a) $t = -0,3368/0,0225 = -14,97$; $|t| \gg 1,961$: on rejette H_0 (p-valeur de l'ordre de 10^{-48}). (b) $-0,3368 \pm 1,961 \times 0,0225 = [-0,381; -0,293]$; en pourcentage : $e^{-0,381} - 1 = -31,7\%$ et $e^{-0,293} - 1 = -25,4\%$: un effet de -25% à -32% . (c) $t = 0,0092/0,00085 \approx 10,8$.

```
b, se, crit = -0.3368, 0.0225, stats.t.ppf(0.975, 1736)
print(f"t = {b / se:.2f} | p = {2 * stats.t.sf(abs(b / se), 1736):.1e} | quantile = {crit:.3f}")
lo, hi = b - crit * se, b + crit * se
print(f"IC de beta : [{lo:.3f}; {hi:.3f}] | IC de l'effet : [{100 * (np.exp(lo) - 1):.1f} % ; {100 * (np.exp(hi) - 1):.1f} %]")
print(f"t de l'âge = {0.0092 / 0.00085:.2f}")
```

```
t = -14.97 | p = 9.8e-48 | quantile = 1.961
IC de beta : [-0.381; -0.293] | IC de l'effet : [-31.7 % ; -25.4 %]
t de l'âge = 10.82
```

Corrigé 1.5

(a) $q = 2$ (les deux coefficients du canal). $F = \frac{(269,94 - 238,30)/2}{238,30/1736} = \frac{15,82}{0,1373} \approx 115,3$. (b) $115 \gg 3,0$: on rejette l'hypothèse « le canal est inutile » (p-valeur de l'ordre de 10^{-47}). (c) $F = \frac{(269,94 - 269,70)/2}{269,70/1736} = \frac{0,12}{0,1554} \approx 0,77$, inférieur à 3,0 (et même à 1) : le canal n'apporte pas plus que du bruit, on ne rejette pas H_0 .

```
def F_stat(scr0, scr1, q, ddl):
    F = ((scr0 - scr1) / q) / (scr1 / ddl)
    return F, stats.f.sf(F, q, ddl)
print("cas observé : F = %.2f, p = %.1e" % F_stat(269.94, 238.30, 2, 1736))
print("cas (c) : F = %.2f, p = %.2f" % F_stat(269.94, 269.70, 2, 1736))
print("valeur critique F(2, 1736) à 5 % :", round(stats.f.ppf(0.95, 2, 1736), 3))
```

```
cas observé : F = 115.25, p = 1.0e-47
cas (c) : F = 0.77, p = 0.46
valeur critique F(2, 1736) à 5 % : 3.001
```

Corrigé 1.6

- (a) $\bar{x} = 4$, $\sum (x_k - \bar{x})^2 = 9 + 4 + 1 + 36 = 50$. $h_{11} = \frac{1}{4} + \frac{9}{50} = 0,43$; $h_{22} = \frac{1}{4} + \frac{4}{50} = 0,33$; $h_{33} = \frac{1}{4} + \frac{1}{50} = 0,27$; $h_{44} = \frac{1}{4} + \frac{36}{50} = 0,97$. Somme : $2,00 = p$ ✓. (b) Le client d'âge 10, éloigné des autres : levier de 0,97 (presque 1). (c) $\partial \hat{y}_4 / \partial y_4 = h_{44} = 0,97$: la droite est presque **forcée** de passer par ce point ; $\hat{y}_4$ augmente de 0,97.

```
x6 = np.array([1.0, 2, 3, 10]); X6 = np.column_stack([np.ones(4), x6])
h6 = np.diag(X6 @ np.linalg.inv(X6.T @ X6) @ X6.T)
print("leviers :", h6.round(3), "| somme =", h6.sum().round(3))
y6 = np.array([2.1, 3.9, 6.2, 20.0]); y6b = y6 + np.array([0, 0, 0, 1.0])
f1, f2 = sm.OLS(y6, X6).fit(), sm.OLS(y6b, X6).fit()
print("variation de la valeur ajustée du 4e point quand y4 augmente de 1 :", round(f2.fittedvalues[3]
↵ - f1.fittedvalues[3], 3))
```

```
leviers : [0.43 0.33 0.27 0.97] | somme = 2.0
variation de la valeur ajustée du 4e point quand y4 augmente de 1 : 0.97
```

Corrigé 1.7

- (a) $\bar{x} = 7/3$, $S_{xx} = \frac{16}{9} + \frac{1}{9} + \frac{25}{9} = \frac{42}{9} = \frac{14}{3}$; $h_{11} = \frac{1}{3} + \frac{16/9}{14/3} = \frac{5}{7} \approx 0,714$; $h_{22} = \frac{1}{3} + \frac{1/9}{14/3} = \frac{5}{14} \approx 0,357$; $h_{33} = \frac{1}{3} + \frac{25/9}{14/3} = \frac{13}{14} \approx 0,929$ (somme = 2 ✓). (b) Erreurs sans le point : $\frac{2/7}{2/7} = 1$; $\frac{-3/7}{9/14} = -\frac{2}{3}$; $\frac{1/7}{1/14} = 2$. PRESS = $1 + \frac{4}{9} + 4 = \frac{49}{9} \approx 5,44$. (c) Sans le point (1,2) : la droite passe par (2,3) et (4,7), d'équation $y = 2x - 1$, et prévoit 1 en $x = 1$: l'erreur est $2 - 1 = 1$ ✓. (d) PRESS (5,44) est près de **vingt fois** plus grand que la somme des carrés résiduelle ($2/7 \approx 0,286$) : avec 3 points seulement, l'ajustement « d'apprentissage » est trompeusement optimiste, surtout pour le point $x = 4$ à fort levier (0,93), dont l'erreur sans lui est de 2 contre un résidu de 0,14.

```
Xa = np.column_stack([np.ones(3), np.array([1.0, 2, 4])]); ya = np.array([2.0, 3, 7])
H = Xa @ np.linalg.inv(Xa.T @ Xa) @ Xa.T
e = ya - H @ ya; h = np.diag(H)
print("leviers :", h.round(3))
print("erreurs sans le point (formule) :", (e / (1 - h)).round(4), "| PRESS =", round(np.sum((e / (1 -
↵ h))**2), 4), "| 49/9 =", round(49 / 9, 4))
loo = []
for i in range(3):
    m = np.arange(3) != i
    b = np.linalg.lstsq(Xa[m], ya[m], rcond=None)[0]
    loo.append(ya[i] - Xa[i] @ b)
print("erreurs sans le point (réajustement) :", np.round(loo, 4), "| SCR =", round(e @ e, 4))
```

```
leviers : [0.714 0.357 0.929]
erreurs sans le point (formule) : [ 1.      -0.6667  2.      ] | PRESS = 5.4444 | 49/9 = 5.4444
erreurs sans le point (réajustement) : [ 1.      -0.6667  2.      ] | SCR = 0.2857
```

Corrigé 1.8

- (a) $VIF = \frac{1}{1-0,995} = 200$; l'erreur standard est multipliée par $\sqrt{200} \approx 14$. (b) Avec deux variables, $R_j^2 = r^2 = 0,81$, donc $VIF = \frac{1}{0,19} \approx 5,3$: à surveiller (règle empirique : > 5). (c) Voir le code ci-dessous.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
print("VIF (a) :", 1 / (1 - 0.995), "| racine :", round(np.sqrt(200), 1), "| VIF (b) :", round(1 / (1
↵ - 0.9**2), 2))
enq = pd.read_csv("donnees/enquete_satisfaction.csv")
Q = sm.add_constant(enq[["q{i}" for i in range(1, 9)]])
```

```
vif = pd.Series([variance_inflation_factor(Q.to_numpy(), i) for i in range(1, Q.shape[1])],
               ↪ index=Q.columns[1:])
print(vif.round(2).to_string())
```

```
VIF (a) : 199.9999999999983 | racine : 14.1 | VIF (b) : 5.26
q1      1.60
q2      1.42
q3      1.47
q4      1.27
q5      1.71
q6      1.47
q7      1.63
q8      1.46
```

Les VIF des huit questions restent **modestes** (inférieurs à 2) : les questions d'un même bloc (produits d'un côté, service de l'autre) sont corrélées, mais pas au point de rendre un modèle instable. On peut donc les utiliser ensemble, ou les résumer par un score moyen comme au 1.4 (ce que l'analyse factorielle du chapitre 3 formalisera : section 3.2).

Corrigé 1.9

- (a) Panier médian prédit : $e^{\hat{\mu}}$ avec $\hat{\mu}$ la prédiction du modèle. (b) La prédiction de la moyenne s'obtient en multipliant par le facteur de Duan $\frac{1}{n} \sum_i e^{\hat{\epsilon}_i}$. (c) On compare à la moyenne observée.

```
clients = pd.read_csv("donnees/clients.csv")
df = clients[clients["nb_commandes_an"] > 0].copy()
df["canal"] = pd.Categorical(df["canal_acquisition"], categories=["Boutique", "Site", "Réseaux"])
df["a"] = df["age"] - 36
df["log_panier"] = np.log(df["panier_moyen"])
m = smf.ols("log_panier ~ a + C(canal)", data=df).fit()
profil = pd.DataFrame({"a": [25 - 36], "canal": pd.Categorical(["Réseaux"], categories=["Boutique",
↪ "Site", "Réseaux"])}))
mu = float(m.predict(profil).iloc[0])
duan = float(np.exp(m.resid).mean())
obs = df[(df["canal"] == "Réseaux") & (df["age"].between(23, 27))]["panier_moyen"]
print(f"(a) panier médian prédit      : {np.exp(mu):.2f} €")
print(f"(b) panier moyen prédit (Duan) : {np.exp(mu) * duan:.2f} €   [facteur de Duan = {duan:.4f} ;
↪  facteur exp(s²/2) = {np.exp(m.scale / 2):.4f}]")
print(f"(c) moyenne observée (Réseaux, 23-27 ans, n = {len(obs)}) : {obs.mean():.2f} € (erreur
↪ standard de cette moyenne : {obs.std() / np.sqrt(len(obs)):.2f}) | médiane observée :
↪ {obs.median():.2f} €")
# Test décisif : sur TOUS les clients, quelle prédiction reproduit la moyenne observée ?
mediane_pred = np.exp(m.fittedvalues)
print(f"tous les clients (n = {len(df)}) : moyenne observée = {df['panier_moyen'].mean():.2f} |
↪ moyenne de exp(mu) = {mediane_pred.mean():.2f} | moyenne de exp(mu) x Duan = {(mediane_pred *
↪ duan).mean():.2f}")
```

```
(a) panier médian prédit      : 43.95 €
(b) panier moyen prédit (Duan) : 47.10 €   [facteur de Duan = 1.0718 ; facteur exp(s²/2) = 1.0710]
(c) moyenne observée (Réseaux, 23-27 ans, n = 86) : 44.49 € (erreur standard de cette moyenne : 1.40)
↪ | médiane observée : 42.69 €
tous les clients (n = 1740) : moyenne observée = 61.23 | moyenne de exp(mu) = 57.12 | moyenne de
↪ exp(mu) x Duan = 61.22
```

Sur ce petit groupe (86 clients), **la moyenne observée (44,49) tombe plus près de la prédiction médiane (43,95) que de la prédiction corrigée (47,10)** : ce n'est pas une contradiction de la théorie, mais du bruit d'échantillonnage. L'erreur standard de cette moyenne observée est de 1,40 € (écart-type des paniers d'environ 13 €, divisé par $\sqrt{86}$) : la prédiction corrigée (47,10) est à 1,9 erreur standard de la moyenne observée, la prédiction médiane à 0,4 : sur un groupe aussi

petit, ces deux écarts sont tous deux plausibles par hasard. Pour **départager** vraiment, il faut un grand échantillon : sur les 1 740 clients, la moyenne observée est de 61,23 € ; la moyenne des prédictions $e^{\hat{\mu}}$ (la médiane de chacun) n'en donne que 57,12 €, soit un défaut de 7 %, alors que les prédictions corrigées $e^{\hat{\mu}} \times \text{Duan}$ redonnent 61,22 €. C'est la confirmation de la théorie du 1.1.8 : la rétro-transformation **naïve vise la médiane, pas la moyenne**, et sous-estime systématiquement le panier moyen ; la correction de Duan la rétablit. (En attendant, retenez qu'une comparaison sur 86 observations ne départage pas des écarts de 7 %.)

Corrigé 1.10

```
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"])
v["t"] = np.arange(len(v))
v["m"] = v["mois"].dt.month
mv = smf.ols("np.log(ca) ~ t + C(m) + promo + covid", data=v).fit()
ic = mv.conf_int()
print(f"(a) croissance annuelle : exp(12 x {mv.params['t']:.4f}) - 1 = {100 * (np.exp(12 *
↳ mv.params['t']) - 1):.1f} % par an (IC95 % du coefficient mensuel : [{ic.loc['t', 0]:.4f} ;
↳ {ic.loc['t', 1]:.4f}])")
print(f"(b) décembre / janvier : x {np.exp(mv.params['C(m)[T.12]']):.2f}")
print(f"(c) confinement : {100 * (np.exp(mv.params['covid']) - 1):.0f} % sur les ventes (IC95 % :
↳ [{100 * (np.exp(ic.loc['covid', 0]) - 1):.0f} % ; {100 * (np.exp(ic.loc['covid', 1]) - 1):.0f}
↳ %])")
print(f" promotion : {100 * (np.exp(mv.params['promo']) - 1):+.1f} % (IC95 % : [{100 *
↳ (np.exp(ic.loc['promo', 0]) - 1):+.1f} % ; {100 * (np.exp(ic.loc['promo', 1]) - 1):+.1f} %])")
r = mv.resid.to_numpy()
print(f"(d) R2 = {mv.rsquared:.3f} (contre 0,493 pour la tendance seule) | écart-type résiduel =
↳ {np.sqrt(mv.scale):.3f} | Durbin-Watson = {sm.stats.durbin_watson(r):.2f} | autocorrélation
↳ d'ordre 1 des résidus = {np.corrcoef(r[1:], r[:-1])[0, 1]:.2f}")
```

```
(a) croissance annuelle : exp(12 x 0.0072) - 1 = 9.0 % par an (IC95 % du coefficient mensuel :
↳ [0.0068 ; 0.0076])
(b) décembre / janvier : x 2.57
(c) confinement : -42 % sur les ventes (IC95 % : [-46 % ; -37 %])
promotion : +5.9 % (IC95 % : [+1.3 % ; +10.7 %])
(d) R2 = 0.964 (contre 0,493 pour la tendance seule) | écart-type résiduel = 0.078 | Durbin-Watson =
↳ 0.93 | autocorrélation d'ordre 1 des résidus = 0.54
```

- (a) Les ventes croissent d'environ **9 % par an** (0,72 % par mois). (b) Décembre est environ **2,6 fois** plus fort que janvier, à tendance égale (la saisonnalité est massive). (c) Le confinement de 2020 correspond à une baisse d'environ **42 %** des ventes sur les quatre mois concernés (intervalle large, de -46 % à -37 %). L'effet des promotions est estimé à environ +6 %, avec un intervalle de confiance **large** (de +1 % à +11 %) : un mois de promotion est un événement rare (une quinzaine de mois sur 120) et la dépendance entre mois rend l'intervalle encore plus incertain. (d) Le R^2 monte à **0,96** : tendance, saisonnalité et chocs expliquent presque tout. Mais le **Durbin-Watson est de 0,93** : l'autocorrélation d'ordre 1 des résidus est **0,54**. Il reste donc de la structure : les erreurs successives se ressemblent, de sorte que les p-valeurs et intervalles ci-dessus sont **trop optimistes** (1.3.3). C'est exactement le sujet du chapitre 4 : modéliser cette dépendance (modèles ARIMA, section 4.2). (Une dernière remarque : les données ayant été simulées, nous pouvons vérifier que les valeurs estimées retrouvent la vérité : croissance de 0,75 % par mois, facteur saisonnier décembre/janvier de $1,55/0,62 \approx 2,5$, effet du confinement de -0,55 en log (soit -42 %), effet des promotions de +0,10 en log (soit +10,5 % : dans l'intervalle, mais près de sa borne haute), autocorrélation des erreurs de 0,5.)

Corrigé 1.11

- (a) Avec $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$, la formule de Ridge $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$ devient $\frac{1}{1+\lambda} \mathbf{X}^\top \mathbf{y} = \frac{1}{1+\lambda} \hat{\beta}^{\text{MCO}}$. (b) Pour $\beta = 2, \sigma = 1$: $\text{EQM}(0) = 1$; $\text{EQM}(0,25) = \frac{0,0625 \times 4 + 1}{1,5625} = \frac{1,25}{1,5625} = 0,8$; $\text{EQM}(1) = \frac{4 + 1}{4} = 1,25$. (c) La dérivée s'annule en $\lambda^* = \sigma^2 / \beta^2 = 0,25$; à ce point l'EQM vaut 0,8 (une baisse de 20 % par rapport aux moindres carrés), et elle redevient supérieure à 1 pour $\lambda > 0,5$ ($\lambda = 1$ donne 1,25).

```
beta_v, sigma_v = 2.0, 1.0
eqm = lambda lam: (lam**2 * beta_v**2 + sigma_v**2) / (1 + lam)**2
for lam in [0, 0.25, 0.5, 1]:
    print(f"lambda = {lam:4.2f} : EQM = {eqm(lam):.4f}")
grille = np.linspace(0, 3, 30001)
print("lambda optimal sur la grille :", round(grille[np.argmin(eqm(grille))], 4), "| sigma²/beta² =",
      sigma_v**2 / beta_v**2)
rng = np.random.default_rng(1)
est = beta_v + rng.normal(0, sigma_v, 200000) # estimations MCO simulées
print("EQM simulée : MCO =", round(np.mean((est - beta_v)**2), 3), "| Ridge (lambda = 0,25) =",
      round(np.mean((est / 1.25 - beta_v)**2), 3))
```

```
lambda = 0.00 : EQM = 1.0000
lambda = 0.25 : EQM = 0.8000
lambda = 0.50 : EQM = 0.8889
lambda = 1.00 : EQM = 1.2500
lambda optimal sur la grille : 0.25 | sigma²/beta² = 0.25
EQM simulée : MCO = 0.998 | Ridge (lambda = 0,25) = 0.8
```

Corrigé 1.12

- (a) $\text{ICC} = \frac{2}{2+6} = 0,25$. (b) $B_j = \frac{\tau^2}{\tau^2 + \sigma^2/n_j} = \frac{2}{2+6/9} = \frac{2}{2,667} = 0,75$; $\hat{u}_j = 0,75 \times 3,2 = 2,4$: on retient 75 % de l'écart observé. (c) Avec $n_j = 2$: $B_j = \frac{2}{2+3} = 0,4$ et $\hat{u}_j = 0,4 \times 3,2 = 1,28$: un petit relais est rétréci bien davantage (60 % de l'écart est écarté, contre 25 % pour le grand). (d) Effet de plan $1 + (m-1)\rho = 1 + 8 \times 0,25 = 3$; $n = 20 \times 9 = 180$ commandes valent $180/3 = 60$ observations indépendantes.

```
tau2, sigma2 = 2.0, 6.0
print("ICC =", tau2 / (tau2 + sigma2))
for n_j in [9, 2]:
    B = tau2 / (tau2 + sigma2 / n_j)
    print(f"n_j = {n_j}: B = {B:.3f} | BLUP = {B * 3.2:.3f}")
deff = 1 + (9 - 1) * tau2 / (tau2 + sigma2)
print("effet de plan =", deff, "| effectif effectif =", 180 / deff)
```

```
ICC = 0.25
n_j = 9: B = 0.750 | BLUP = 2.400
n_j = 2: B = 0.400 | BLUP = 1.280
effet de plan = 3.0 | effectif effectif = 60.0
```

Corrigé 1.13

```
rng = np.random.default_rng(5)
n = 100
x = rng.uniform(0, 10, n)
y_propre = 2 + 1.5 * x + rng.normal(0, 1, n)
```

```

y_corr = y_propre.copy()
idx = rng.choice(n, 5, replace=False)
y_corr[idx] += 25
X = sm.add_constant(x)
lignes = {}
for nom, y in [("données propres", y_propre), ("données corrompues", y_corr)]:
    lignes[(nom, "MCO")] = sm.OLS(y, X).fit().params
    lignes[(nom, "Huber")] = sm.RLM(y, X, M=sm.robust.norms.HuberT()).fit().params
    lignes[(nom, "médiane")] = sm.QuantReg(y, X).fit(q=0.5).params
tab = pd.DataFrame(lignes, index=["constante", "pente"]).T
print("vrais coefficients : constante = 2, pente = 1,5")
print(tab.round(3).to_string())

```

```

vrais coefficients : constante = 2, pente = 1,5
                constante  pente
données propres  MCO      2.356  1.450
                  Huber    2.375  1.441
                  médiane  2.297  1.436
données corrompues MCO    2.633  1.637
                  Huber    2.424  1.451
                  médiane  2.297  1.450

```

Sur les données **propres**, les trois méthodes donnent des résultats très proches. Après corruption de 5 % des points, les **moindres carrés dérivent** (la pente passe de 1,45 à 1,64 et la constante de 2,36 à 2,63), alors que **Huber et la régression médiane** restent presque inchangés (pente entre 1,44 et 1,45 dans les deux cas). C'est l'intuition du 1.6 : l'influence des points aberrants est plafonnée par les méthodes robustes.

Chapitre 2 : Modèles linéaires généralisés — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Les **applications** reprennent, pas à pas, les calculs que le livre n'a fait que résumer (l'algorithme IRLS écrit à la main, les vérifications numériques, les évaluations de modèles) ; les **exercices** sont corrigés à la fin. Les données sont celles du livre : `donnees/clients.csv` (2 000 clients), `donnees/enquete_satisfaction.csv` (notes de 1 à 5) et `donnees/ch02-sessions.csv` (1 500 sessions de navigation, simulées). Toutes les cellules de ce chapitre s'exécutent **dans l'ordre**, en partageant leurs variables.

Préparation

Une seule cellule charge les bibliothèques et les trois fichiers ; les applications qui suivent s'en servent.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats
from scipy.special import expit

clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Réseaux",
↵ "Site"]) # référence : Boutique
enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
enquete["note_produits"] = enquete[["q1", "q2", "q3", "q4"]].mean(axis=1)
enquete["note_service"] = enquete[["q5", "q6", "q7", "q8"]].mean(axis=1)
repondants = clients.merge(enquete[["id_client", "note_produits", "note_service"]], on="id_client")
sessions = pd.read_csv("donnees/ch02-sessions.csv")
print(len(clients), "clients,", len(repondants), "répondants à l'enquête,", len(sessions), "sessions")
```

2000 clients, 1212 répondants à l'enquête, 1500 sessions

Applications

Application 2.1 — L'algorithme IRLS écrit à la main

Sections du livre : 2.1.5 et 2.2.5. **Objectif** : programmer les moindres carrés repondérés itérés, vérifier qu'ils redonnent la solution connue sur un tout petit exemple, puis qu'ils reproduisent stats-models sur le modèle de rachat.

Étape 0 — Les quatre lignes du tableau de 2.1.3. Pour chaque loi, $b'(\theta)$ et $\phi b''(\theta)$ (dérivées numériques de b) doivent coïncider avec la moyenne et la variance d'un grand échantillon simulé.

```
rng = np.random.default_rng(21)

def d1(f, x):
    h = 1e-4 * abs(x)          # pas relatif : theta varie de 1/60 à 5 selon les lois
    return (f(x + h) - f(x - h)) / (2 * h)

def d2(f, x):
    h = 1e-4 * abs(x)
    return (f(x + h) - 2 * f(x) + f(x - h)) / h**2

N = 400_000
cas = [
    # nom, theta, b, phi, échantillon simulé
    ("Normale(5 ; sigma²=4)", 5.0, lambda t: t**2 / 2, 4.0, rng.normal(5, 2, N)),
    ("Bernoulli(0,3)", np.log(0.3 / 0.7), lambda t: np.log1p(np.exp(t)), 1.0, rng.binomial(1, 0.3, N)),
    ("Poisson(3)", np.log(3.0), lambda t: np.exp(t), 1.0, rng.poisson(3, N)),
    ("Gamma(moyenne 60 ; forme 4)", -1 / 60, lambda t: -np.log(-t), 1 / 4, rng.gamma(4, 60 / 4, N)),
]
lignes = []
for nom, theta, b, phi, echantillon in cas:
    lignes.append({"loi": nom, "b'(theta)": d1(b, theta), "phi * b''(theta)": phi * d2(b, theta),
                  "moyenne simulée": echantillon.mean(), "variance simulée": echantillon.var()})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

loi	$b'(\theta)$	$\phi * b''(\theta)$	moyenne simulée	variance simulée
Normale(5 ; sigma²=4)	5.0	4.00	4.998	4.005
Bernoulli(0,3)	0.3	0.21	0.300	0.210
Poisson(3)	3.0	3.00	3.002	2.991
Gamma(moyenne 60 ; forme 4)	60.0	900.00	60.024	900.824

Étape 1 — Quatre clients, un paramètre. Quatre clients, dont trois ont racheté : $y = (1, 0, 1, 1)$, modèle logistique sans variable. La solution exacte est $\hat{\beta} = \log 3 \approx 1,0986$. À chaque itération, on calcule μ , le poids $W = \mu(1 - \mu)$, la réponse de travail $z = \eta + (y - \mu)/W$, puis la moyenne pondérée de z .

```
y = np.array([1, 0, 1, 1])
beta = 0.0
for it in range(1, 6):
    eta = beta          # modèle sans variable : eta = beta pour tous
    mu = 1 / (1 + np.exp(-eta))
    w = np.full(len(y), mu * (1 - mu)) # un poids par client : (dmu/deta)² / V(mu) = mu(1-mu)
    z = eta + (y - mu) / w          # réponse de travail
    beta = np.sum(w * z) / np.sum(w)
    print(f"itération {it} : mu = {mu:.4f}   W = {w[0]:.4f}   beta = {beta:.6f}")
print("valeur exacte : log(3) =", round(np.log(3), 6))
```

itération 1	mu = 0.5000	W = 0.2500	beta = 1.000000
itération 2	mu = 0.7311	W = 0.1966	beta = 1.096339
itération 3	mu = 0.7496	W = 0.1877	beta = 1.098611
itération 4	mu = 0.7500	W = 0.1875	beta = 1.098612

```

itération 5 : mu = 0.7500    W = 0.1875    beta = 1.098612
valeur exacte : log(3) = 1.098612

```

Les deux premières valeurs doivent être celles du calcul à la main du livre (1,000 puis 1,0963). Observez le nombre de décimales exactes à chaque pas.

Étape 2 — L'algorithme en général. On décrit une famille par son lien, son lien inverse, la dérivée $d\mu/d\eta$ et sa fonction de variance. La première cellule définit les familles, la seconde la fonction `irls`.

```

from scipy.special import expit

FAMILLES = {
    # lien, lien inverse, dmu/deta (en fonction de eta), fonction de variance V(mu), valeur initiale
    # de mu
    "binomial": dict(lien=lambda m: np.log(m / (1 - m)), inverse=expit,
                    dmu_deta=lambda e: expit(e) * (1 - expit(e)), variance=lambda m: m * (1 - m),
                    init=lambda y: (y + 0.5) / 2),
    "poisson": dict(lien=np.log, inverse=np.exp, dmu_deta=np.exp, variance=lambda m: m,
                    init=lambda y: y + 0.5),
    "gamma_log": dict(lien=np.log, inverse=np.exp, dmu_deta=np.exp, variance=lambda m: m**2,
                      init=lambda y: y),
}

def irls(X, y, famille, tol=1e-10, max_iter=50):
    """Moindres carrés repondérés itérés. Renvoie les coefficients, leur matrice de covariance (phi =
    1) et le détail."""
    f = FAMILLES[famille]
    mu = f["init"](y)
    eta = f["lien"](mu)
    for it in range(1, max_iter + 1):
        w = f["dmu_deta"](eta) ** 2 / f["variance"](mu)          # poids W_i
        z = eta + (y - mu) / f["dmu_deta"](eta)                  # réponse de travail z_i
        XtW = X.T * w
        beta = np.linalg.solve(XtW @ X, XtW @ z)                 # régression pondérée de z sur X
        nouveau_eta = X @ beta
        converge = np.max(np.abs(nouveau_eta - eta)) < tol
        eta, mu = nouveau_eta, f["inverse"](nouveau_eta)
        if converge:
            break
    w = f["dmu_deta"](eta) ** 2 / f["variance"](mu)
    cov = np.linalg.inv((X.T * w) @ X)                          # (X'WX)^-1 : covariance si phi = 1
    return dict(beta=beta, cov=cov, mu=mu, eta=eta, w=w, iterations=it)

```

Étape 3 — Deux tests. (1) Les quatre clients : IRLS doit retrouver $\log 3$. (2) Poisson sans variable sur les 2 000 clients : la solution doit être $\log \bar{y}$.

```

# Test 1 : l'exemple à quatre clients
un = np.ones((4, 1))
r = irls(un, y.astype(float), "binomial")
print("logistique, 4 clients : beta =", r["beta"].round(6), "en", r["iterations"], "itérations")

# Test 2 : Poisson sans variable sur les vraies données : la solution doit être log(moyenne)
nb = clients["nb_commandes_an"].to_numpy(dtype=float)
r = irls(np.ones((len(nb), 1)), nb, "poisson")
print("Poisson, 2000 clients : beta =", r["beta"].round(6), "| log(moyenne) =",
      round(float(np.log(nb.mean())), 6), "| itérations :", r["iterations"])

```

```

logistique, 4 clients : beta = [1.098612] en 5 itérations
Poisson, 2000 clients : beta = [1.356608] | log(moyenne) = 1.356608 | itérations : 6

```

Étape 4 — Comparaison avec statsmodels sur le modèle de rachat. On construit la matrice X

avec patsy, comme le fait statsmodels, puis on compare coefficients et erreurs-types.

```
from patsy import dmatrices

formule = "rachat_12m ~ offre_bienvenue + age + canal"
modele = smf.glm(formule, clients, family=sm.families.Binomial()).fit()
Y, X = dmatrices(formule, clients, return_type="dataframe")
r = irls(X.to_numpy(), Y.to_numpy().ravel(), "binomial")
comparaison = pd.DataFrame({
    "coef (IRLS main)": r["beta"], "coef (statsmodels)": modele.params.to_numpy(),
    "se (IRLS main)": np.sqrt(np.diag(r["cov"])), "se (statsmodels)": modele.bse.to_numpy(),
}, index=X.columns)
print(comparaison.round(6).to_string())
print("itérations :", r["iterations"], "| écart maximal sur les coefficients :",
      ↪ f"{np.max(np.abs(r['beta'] - modele.params.to_numpy())):.2e}")
```

	coef (IRLS main)	coef (statsmodels)	se (IRLS main)	se (statsmodels)
Intercept	0.588499	0.588499	0.185372	0.185372
canal[T.Réseaux]	-0.462955	-0.462955	0.115509	0.115509
canal[T.Site]	-0.167719	-0.167719	0.119619	0.119619
offre_bienvenue	0.493258	0.493258	0.090953	0.090953
age	-0.015498	-0.015498	0.004351	0.004351

itérations : 5 | écart maximal sur les coefficients : 3.13e-14

Étape 5 — Une propriété du lien canonique. Pour la régression logistique, $\sum_i (y_i - \hat{p}_i) x_{ij} = 0$ pour chaque colonne x_j , constante comprise. Vérifiez-le, puis expliquez pourquoi la somme des probabilités prédites égale le nombre de « oui ».

```
residus = clients["rachat_12m"] - modele.fittedvalues
print("somme des résidus (y - p̂) : ", round(float(residus.sum()), 8))
print("somme des p̂ = ", round(float(modele.fittedvalues.sum()), 3), "| nombre de 1 observés =",
      ↪ int(clients["rachat_12m"].sum()))
for col in ["offre_bienvenue", "age"]:
    print(f"somme des résidus × {col:16s}:", round(float((residus * clients[col]).sum()), 6))
```

```
somme des résidus (y - p̂) : 0.0
somme des p̂ = 1019.0 | nombre de 1 observés = 1019
somme des résidus × offre_bienvenue : 0.0
somme des résidus × age : 0.0
```

Application 2.2 — L'offre de bienvenue : du tableau croisé aux effets marginaux

Sections du livre : 2.2.1 à 2.2.7. Objectif : refaire, pas à pas, l'étude du rachat : cotes, rapport de cotes, modèle complet, probabilités prédites, effets marginaux, puis les notes de l'enquête.

Étape 1 — Probabilité, cote, logit, et effet d'un coefficient sur la probabilité. Calculez le tableau (probabilité, cote, logit) pour quelques valeurs, puis montrez qu'un même effet de +0,5 sur le logit ne produit pas le même gain en points de pourcentage selon le point de départ.

```
p = np.array([0.05, 0.10, 0.25, 0.50, 0.60, 0.75, 0.90, 0.95])
print(pd.DataFrame({"probabilité p": p, "cote p/(1-p)": p / (1 - p), "logit": np.log(p / (1 -
      ↪ p))}).round(3).to_string(index=False))
```

probabilité p	cote p/(1-p)	logit
0.05	0.053	-2.944
0.10	0.111	-2.197
0.25	0.333	-1.099
0.50	1.000	0.000
0.60	1.500	0.405
0.75	3.000	1.099
0.90	9.000	2.197
0.95	19.000	2.944

```

beta = 0.5 # un effet de +0,5 sur le logit : OR = exp(0,5) ≈ 1,65
p0 = np.array([0.02, 0.10, 0.30, 0.50, 0.70, 0.90, 0.98])
p1 = expit(np.log(p0 / (1 - p0)) + beta)
print("OR =", round(float(np.exp(beta)), 3))
print(pd.DataFrame({"p avant": p0, "p après": p1, "gain (points)": 100 * (p1 -
↪ p0})).round(3).to_string(index=False))

```

```

OR = 1.649
p avant  p après  gain (points)
0.02     0.033     1.255
0.10     0.155     5.483
0.30     0.414    11.404
0.50     0.622    12.246
0.70     0.794     9.369
0.90     0.937     3.686
0.98     0.988     0.777

```

Étape 2 — Un seul prédicteur binaire. Avant d'ajuster, tout se calcule à la main à partir du tableau croisé. Vérifiez que le coefficient de l'offre est exactement le logarithme du rapport de cotes (modèle saturé).

```

croise = pd.crosstab(clients["offre_bienvenue"], clients["rachat_12m"])
print(croise)
sans, avec = croise.loc[0], croise.loc[1]
p_sans, p_avec = sans.sum() / sans.sum(), avec.sum() / avec.sum()
cote_sans, cote_avec = sans[1] / sans[0], avec[1] / avec[0]
print()
print(f"sans offre : {p_sans:.4f} ont racheté | cote = {cote_sans:.4f} | logit =
↪ {np.log(cote_sans):.4f}")
print(f"avec offre : {p_avec:.4f} ont racheté | cote = {cote_avec:.4f} | logit =
↪ {np.log(cote_avec):.4f}")
print(f"rapport de cotes (a*d)/(b*c) = {cote_avec / cote_sans:.4f} | log = {np.log(cote_avec /
↪ cote_sans):.4f}")
print(f"différence de risque = {p_avec - p_sans:.4f} | risque relatif = {p_avec / p_sans:.4f}")

m_offre = smf.glm("rachat_12m ~ offre_bienvenue", clients, family=sm.families.Binomial()).fit()
print()
print(m_offre.params.round(4).to_string())
print("exp(coefficient de l'offre) =", round(float(np.exp(m_offre.params["offre_bienvenue"])), 4))

```

```

rachat_12m      0      1
offre_bienvenue
0                544  441
1                437  578

```

```

sans offre : 0.4477 ont racheté | cote = 0.8107 | logit = -0.2099
avec offre : 0.5695 ont racheté | cote = 1.3227 | logit = 0.2796
rapport de cotes (a*d)/(b*c) = 1.6316 | log = 0.4895
différence de risque = 0.1217 | risque relatif = 1.2719

```

```

Intercept      -0.2099
offre_bienvenue  0.4895
exp(coefficient de l'offre) = 1.6316

```

Étape 3 — Le modèle complet et ses rapports de cotes.

```

modele = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
↪ family=sm.families.Binomial()).fit()
print(modele.summary().tables[1])
print()
print("log-vraisemblance :", round(modele.llf, 2), "| déviance :", round(modele.deviance, 2), "| AIC
↪ :", round(modele.aic, 2), "| n =", int(modele.nobs))

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	0.5885	0.185	3.175	0.001	0.225	0.952
canal[T.Réseaux]	-0.4630	0.116	-4.008	0.000	-0.689	-0.237
canal[T.Site]	-0.1677	0.120	-1.402	0.161	-0.402	0.067
offre_bienvenue	0.4933	0.091	5.423	0.000	0.315	0.672
age	-0.0155	0.004	-3.562	0.000	-0.024	-0.007

log-vraisemblance : -1355.42 | déviance : 2710.83 | AIC : 2720.83 | n = 2000

```
ic = modele.conf_int()
rapports = pd.DataFrame({"OR": np.exp(modele.params), "IC95 bas": np.exp(ic[0]), "IC95 haut":
    np.exp(ic[1]), "p-valeur": modele.pvalues})
print(rapports.round(3).to_string())
print()
print("OR pour +10 ans d'âge :", round(float(np.exp(10 * modele.params["age"])), 3))
```

	OR	IC95 bas	IC95 haut	p-valeur
Intercept	1.801	1.253	2.590	0.001
canal[T.Réseaux]	0.629	0.502	0.789	0.000
canal[T.Site]	0.846	0.669	1.069	0.161
offre_bienvenue	1.638	1.370	1.957	0.000
age	0.985	0.976	0.993	0.000

OR pour +10 ans d'âge : 0.856

Étape 4 — Probabilités prédites pour un profil, et effets marginaux moyens. Prédisez, pour un client de 36 ans venu des réseaux, la probabilité de rachat avec et sans l'offre (puis vérifiez votre calcul avec `expit`). Calculez ensuite l'effet marginal moyen de l'offre et de l'âge, et comparez avec `get_margeff`.

```
profil = pd.DataFrame({"offre_bienvenue": [0, 1], "age": [36, 36],
    "canal": pd.Categorical(["Réseaux", "Réseaux"], categories=["Boutique",
    "Réseaux", "Site"])}))
p_profil = modele.predict(profil)
b = modele.params
a_la_main = expit(b["Intercept"] + b["canal[T.Réseaux]"] + 36 * b["age"] + b["offre_bienvenue"] *
    np.array([0, 1]))
print("probabilités prédites (sans offre, avec offre) :", p_profil.round(4).to_numpy(), "| à la main
    :", a_la_main.round(4))
print("gain en points de pourcentage pour ce profil :", round(100 * float(p_profil.iloc[1] -
    p_profil.iloc[0]), 2))
```

probabilités prédites (sans offre, avec offre) : [0.3936 0.5152] | à la main : [0.3936 0.5152]
gain en points de pourcentage pour ce profil : 12.17

```
avec_offre = clients.assign(offre_bienvenue=1)
sans_offre = clients.assign(offre_bienvenue=0)
effet_moyen = (modele.predict(avec_offre) - modele.predict(sans_offre)).mean()
print(f"effet marginal moyen de l'offre (modèle) : {100 * effet_moyen:.2f} points")
print(f"différence brute des taux de rachat : {100 * (p_avec - p_sans):.2f} points")

# effet marginal moyen de l'âge : +1 an
effet_age = (modele.predict(clients.assign(age=clients["age"] + 1)) - modele.predict(clients)).mean()
print(f"effet marginal moyen d'un an de plus : {100 * effet_age:.3f} point (soit {100 * 10 *
    effet_age:.2f} points pour 10 ans)")

# contrôle avec la fonction de statsmodels (modèle Logit)
logit = smf.logit("rachat_12m ~ offre_bienvenue + age + canal", clients).fit(dis=0)
print()
print(logit.get_margeff(at="overall", dummy=True).summary_frame().round(4).to_string())
```

effet marginal moyen de l'offre (modèle) : 12.07 points
différence brute des taux de rachat : 12.17 points
effet marginal moyen d'un an de plus : -0.376 point (soit -3.76 points pour 10 ans)

	dy/dx	Std. Err.	z	Pr(> z)	Conf. Int. Low	Conf. Int. Hi.
canal[T.Réseaux]	-0.1126	0.0277	-4.0625	0.0000	-0.1669	-0.0583
canal[T.Site]	-0.0405	0.0287	-1.4108	0.1583	-0.0967	0.0158
offre_bienvenue	0.1207	0.0220	5.4768	0.0000	0.0775	0.1640
age	-0.0038	0.0010	-3.6063	0.0003	-0.0058	-0.0017

Étape 5 — Ajouter les notes de l'enquête (sur les répondants seulement). Comparez le modèle de base et le modèle enrichi (AIC, rapports de cotes). Le rapport de cotes de l'offre change entre les deux, alors qu'elle est tirée au hasard : c'est la **non-collapsibilité**.

```
m_base = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", repondants,
↳ family=sm.families.Binomial()).fit()
m_riche = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service",
↳ repondants,
family=sm.families.Binomial()).fit()
ic = m_riche.conf_int()
print(pd.DataFrame({"coef": m_riche.params, "OR": np.exp(m_riche.params), "IC95 bas": np.exp(ic[0]),
↳ "IC95 haut": np.exp(ic[1]),
"p": m_riche.pvalues}).round(3).to_string())
print()
print("répondants :", int(m_riche.nobs), "| AIC modèle de base :", round(m_base.aic, 1), "| AIC modèle
↳ avec notes :", round(m_riche.aic, 1))
print("écart-types des notes :", repondants[["note_produits",
↳ "note_service"]].std().round(2).to_dict())
print("OR de l'offre, sur ces mêmes répondants : sans les notes =",
↳ round(float(np.exp(m_base.params["offre_bienvenue"])), 3), "| avec les notes =",
↳ round(float(np.exp(m_riche.params["offre_bienvenue"])), 3))
```

	coef	OR	IC95 bas	IC95 haut	p
Intercept	-3.626	0.027	0.010	0.069	0.000
canal[T.Réseaux]	-0.474	0.622	0.458	0.845	0.002
canal[T.Site]	-0.244	0.783	0.570	1.075	0.131
offre_bienvenue	0.607	1.834	1.442	2.334	0.000
age	-0.018	0.983	0.971	0.994	0.003
note_produits	0.764	2.148	1.781	2.589	0.000
note_service	0.420	1.522	1.285	1.803	0.000

répondants : 1212 | AIC modèle de base : 1652.3 | AIC modèle avec notes : 1545.2
écart-types des notes : {'note_produits': 0.69, 'note_service': 0.73}
OR de l'offre, sur ces mêmes répondants : sans les notes = 1.749 | avec les notes = 1.834

Application 2.3 — Évaluer un modèle de classement

Sections du livre : 2.2.8 et 2.2.9. **Objectif :** matrice de confusion, courbe ROC et AUC calculées de trois façons, calibration, évaluation hors échantillon, et un cas de séparation parfaite.

Étape 1 — Matrice de confusion à plusieurs seuils.

```
y = repondants["rachat_12m"].to_numpy()
score = m_riche.fittedvalues.to_numpy()

def matrice(y, score, seuil):
    pred = (score >= seuil).astype(int)
    vp, fp = int(((pred == 1) & (y == 1)).sum()), int(((pred == 1) & (y == 0)).sum())
    fn, vn = int(((pred == 0) & (y == 1)).sum()), int(((pred == 0) & (y == 0)).sum())
    return vp, fp, fn, vn
```

```
for seuil in (0.5, 0.4, 0.6):
    vp, fp, fn, vn = matrice(y, score, seuil)
    print(f"seuil {seuil} : VP={vp} FP={fp} FN={fn} VN={vn} | exactitude={(vp + vn) / len(y):.3f} | "
          f"sensibilité={vp / (vp + fn):.3f} | spécificité={vn / (vn + fp):.3f} | précision={vp / (vp "
          f"+ fp):.3f}")
```

```
seuil 0.5 : VP=406 FP=228 FN=206 VN=372 | exactitude=0.642 | sensibilité=0.663 | spécificité=0.620 |
↪ précision=0.640
seuil 0.4 : VP=512 FP=353 FN=100 VN=247 | exactitude=0.626 | sensibilité=0.837 | spécificité=0.412 |
↪ précision=0.592
seuil 0.6 : VP=270 FP=112 FN=342 VN=488 | exactitude=0.625 | sensibilité=0.441 | spécificité=0.813 |
↪ précision=0.707
```

Étape 2 — L'AUC de trois façons (comparaison de paires, aire sous la courbe ROC, scikit-learn)
: les trois valeurs doivent coïncider.

```
from sklearn.metrics import roc_auc_score

def auc_paires(y, s):
    pos, neg = s[y == 1], s[y == 0]
    return (pos[:, None] > neg[None, :]).mean() + 0.5 * (pos[:, None] == neg[None, :]).mean()

# courbe ROC « à la main » : on trie les clients par score décroissant et on cumule
ordre = np.argsort(-score)
y_trie = y[ordre]
tpr = np.concatenate([[0], np.cumsum(y_trie) / y.sum()])
fpr = np.concatenate([[0], np.cumsum(1 - y_trie) / (1 - y).sum()])
auc_trapezes = np.trapezoid(tpr, fpr)

print("AUC par comparaison de paires :", round(float(auc_paires(y, score)), 4))
print("AUC par aire sous la courbe :", round(float(auc_trapezes), 4))
print("AUC de scikit-learn :", round(float(roc_auc_score(y, score)), 4))
print("AUC du modèle de base (sans les notes) :", round(float(roc_auc_score(y, m_base.fittedvalues)),
↪ 4))
```

```
AUC par comparaison de paires : 0.6975
AUC par aire sous la courbe : 0.6975
AUC de scikit-learn : 0.6975
AUC du modèle de base (sans les notes) : 0.599
```

Étape 3 — Calibration : regroupez les répondants par dixième de score et comparez la probabilité prédite à la fréquence observée. Le tracé de la courbe ROC et de la calibration (figure du livre) est produit par la cellule masquée qui suit.

```
dec = pd.qcut(score, 10, labels=False)
calib = pd.DataFrame({"p prédite": score, "observé": y, "dixième": dec}).groupby("dixième").agg(
    p_predite=("p prédite", "mean"), observe=("observé", "mean"), n=("observé", "size"))
print(calib.round(3).to_string())
```

	p_predite	observe	n
dixième			
0	0.212	0.287	122
1	0.308	0.207	121
2	0.377	0.397	121
3	0.433	0.421	121
4	0.485	0.537	121
5	0.537	0.479	121
6	0.582	0.595	121
7	0.635	0.628	121
8	0.698	0.661	121
9	0.783	0.836	122

Étape 4 — Évaluer hors échantillon : découpez 70 % / 30 %, ajustez sur la première part, évaluez sur la seconde. Répétez avec d'autres graines : l'AUC en test varie-t-elle plus que l'écart avec l'apprentissage ?

```
rng = np.random.default_rng(5)
idx = rng.permutation(len(repondants))
n_app = int(0.7 * len(repondants))
app, test = repondants.iloc[idx[:n_app]], repondants.iloc[idx[n_app:]]
m_app = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service", app,
               family=sm.families.Binomial()).fit()
print("AUC en apprentissage :", round(float(roc_auc_score(app["rachat_12m"], m_app.fittedvalues)),
               4), "(", len(app), "clients)")
print("AUC en test          :", round(float(roc_auc_score(test["rachat_12m"], m_app.predict(test))),
               4), "(", len(test), "clients)")
```

```
AUC en apprentissage : 0.6841 ( 848 clients )
AUC en test          : 0.7193 ( 364 clients )
```

Étape 5 — La séparation parfaite. Six clients : les trois premiers n'ont pas racheté, les trois derniers si. Lisez attentivement l'avertissement et la valeur de la pente.

```
import warnings
x_sep = np.array([1.0, 2, 3, 4, 5, 6])
y_sep = np.array([0, 0, 0, 1, 1, 1])
with warnings.catch_warnings(record=True) as avertissements:
    warnings.simplefilter("always")
    try:
        res = sm.GLM(y_sep, sm.add_constant(x_sep), family=sm.families.Binomial()).fit()
        print("pente estimée :", round(float(res.params[1]), 1), "| erreur-type de la pente :",
              round(float(res.bse[1]), 1))
        print("probabilités prédites (arrondies) :", res.fittedvalues.round(3).tolist())
    except Exception as e:
        print(type(e).__name__, ":", e)
    for message in sorted({str(a.message) for a in avertissements}):
        print("AVERTISSEMENT :", message)
```

```
pente estimée : 41.2 | erreur-type de la pente : 25719.8
probabilités prédites (arrondies) : [0.0, 0.0, 0.0, 1.0, 1.0, 1.0]
AVERTISSEMENT : Perfect separation or prediction detected, parameter may not be identified
```

Application 2.4 — Comptages : Poisson, exposition, surdispersion

Sections du livre : 2.3.1 à 2.3.3. **Objectif :** modéliser le nombre de commandes, voir l'effet d'un oubli de l'exposition, mesurer la surdispersion et la corriger.

Étape 1 — Un modèle de Poisson avec une variable catégorielle, puis complet. Les exponentielles des coefficients sont des rapports de taux.

```
moy = clients.groupby("canal", observed=True)["nb_commandes_an"].mean()
m_canal = smf.glm("nb_commandes_an ~ canal", clients, family=sm.families.Poisson()).fit()
print("moyennes observées par canal :", moy.round(4).to_dict())
print("rapports de moyennes / Boutique :", (moy / moy["Boutique"]).round(4).to_dict())
print("exp(coefficients) Poisson :", np.exp(m_canal.params).round(4).to_dict())
print("exp(Intercept) = moyenne de la boutique :", round(float(np.exp(m_canal.params["Intercept"])),
               4))
```

```
moyennes observées par canal : {'Boutique': 4.1369, 'Réseaux': 3.4645, 'Site': 4.1971}
rapports de moyennes / Boutique : {'Boutique': 1.0, 'Réseaux': 0.8375, 'Site': 1.0145}
exp(coefficients) Poisson : {'Intercept': 4.1369, 'canal[T.Réseaux]': 0.8375, 'canal[T.Site]': 1.0145}
exp(Intercept) = moyenne de la boutique : 4.1369
```

```
poisson = smf.glm("nb_commandes_an ~ age + canal + offre_bienvenue", clients,
  ↪ family=sm.families.Poisson()).fit()
ic = poisson.conf_int()
rr = pd.DataFrame({"coef": poisson.params, "RR": np.exp(poisson.params), "IC95 bas": np.exp(ic[0]),
  ↪ "IC95 haut": np.exp(ic[1]),
  ↪ "p": poisson.pvalues})
print(rr.round(4).to_string())
```

	coef	RR	IC95 bas	IC95 haut	p
Intercept	1.4637	4.3218	3.9509	4.7275	0.0000
canal[T.Réseaux]	-0.1762	0.8385	0.7923	0.8873	0.0000
canal[T.Site]	0.0151	1.0152	0.9594	1.0742	0.6008
age	-0.0010	0.9990	0.9969	1.0011	0.3675
offre_bienvenue	-0.0189	0.9813	0.9385	1.0259	0.4051

Étape 2 — L'exposition. On simule 1 500 clients observés 3, 6, 9 ou 12 mois ; les clients anciens sont plus souvent abonnés à la newsletter ; le vrai effet de la newsletter est de multiplier le taux par 1,2. Comparez le modèle naïf, le modèle avec décalage log(mois) et le modèle où log(mois) est une variable libre.

```
rng = np.random.default_rng(23)
n = 1500
mois_obs = rng.choice([3, 6, 9, 12], size=n)
p_newsletter = pd.Series(mois_obs).map({3: 0.15, 6: 0.30, 9: 0.45, 12: 0.65}).to_numpy()
newsletter = rng.binomial(1, p_newsletter)
taux_mensuel = 0.4 * 1.2 ** newsletter # commandes par mois : vrai RR = 1,2
sim = pd.DataFrame({"mois": mois_obs, "newsletter": newsletter, "commandes": rng.poisson(taux_mensuel
  ↪ * mois_obs)})

naif = smf.glm("commandes ~ newsletter", sim, family=sm.families.Poisson()).fit()
avec_offset = smf.glm("commandes ~ newsletter", sim, family=sm.families.Poisson(),
  ↪ offset=np.log(sim["mois"])).fit()
libre = smf.glm("commandes ~ newsletter + np.log(mois)", sim, family=sm.families.Poisson()).fit()

print("part de clients abonnés à la newsletter :", round(float(sim["newsletter"].mean()), 3))
print("exposition moyenne : sans newsletter =", round(sim.loc[sim.newsletter == 0, "mois"].mean(),
  ↪ 2), "mois | avec newsletter =", round(sim.loc[sim.newsletter == 1, "mois"].mean(), 2), "mois")
g = sim.groupby("newsletter").agg(commandes=("commandes", "sum"), clients_mois=("mois", "sum"))
g["taux par client-mois"] = g["commandes"] / g["clients_mois"]
print(g.round(4).to_string())
print()
print("vrai RR de la newsletter : 1.2")
print("RR sans tenir compte de la durée :", round(float(np.exp(naif.params["newsletter"])), 3))
print("RR avec décalage log(mois) :",
  ↪ round(float(np.exp(avec_offset.params["newsletter"])), 3),
  ↪ " IC95 [", ", ".join(str(round(float(v), 3)) for v in
  ↪ np.exp(avec_offset.conf_int().loc["newsletter"])), " ]")
print("RR avec log(mois) comme variable libre :", round(float(np.exp(libre.params["newsletter"])),
  ↪ 3),
  ↪ "| coefficient de log(mois) :", round(float(libre.params["np.log(mois)"]), 3), "(attendu : 1)")
```

```
part de clients abonnés à la newsletter : 0.381
exposition moyenne : sans newsletter = 6.44 mois | avec newsletter = 9.05 mois
commandes clients_mois taux par client-mois
newsletter
0          2360          5979          0.3947
1          2451          5166          0.4744
```

```
vrai RR de la newsletter : 1.2
RR sans tenir compte de la durée : 1.69
RR avec décalage log(mois) : 1.202 IC95 [ 1.136, 1.272 ]
RR avec log(mois) comme variable libre : 1.196 | coefficient de log(mois) : 1.017 (attendu : 1)
```

Étape 3 — Mesurer la surdispersion : rapport X^2/ddl et test de Cameron-Trivedi.

```

y = clients["nb_commandes_an"].to_numpy()
mu = poisson.fittedvalues.to_numpy()
print("Pearson  $X^2$  =", round(poisson.pearson_chi2, 1), "| degrés de liberté =", int(poisson.df_resid),
      ↪ "| phi estimé =", round(poisson.pearson_chi2 / poisson.df_resid, 3))
print("déviante =", round(poisson.deviance, 1), "| déviante / ddl =", round(poisson.deviance /
      ↪ poisson.df_resid, 3))

# Test de Cameron et Trivedi (1990) : sous Poisson,  $E[(y-\mu)^2 - y] = 0$  ; sous « variance =  $\mu + \alpha\mu^2$ 
      ↪  $\mu^2$  »,  $E[(y-\mu)^2 - y] = \alpha\mu^2$ 
aux = ((y - mu) ** 2 - y) / mu
reg = sm.OLS(aux, mu).fit() # régression de  $((y-\mu)^2 - y)/\mu$  sur  $\mu$ , sans constante : la pente
      ↪ estime alpha
print("alpha estimé par Cameron-Trivedi :", round(float(reg.params[0]), 3), "| t =",
      ↪ round(float(reg.tvalues[0]), 2))

```

```

Pearson  $X^2$  = 6774.2 | degrés de liberté = 1995 | phi estimé = 3.396
déviante = 6293.0 | déviante / ddl = 3.154
alpha estimé par Cameron-Trivedi : 0.609 | t = 12.47

```

Étape 4 — Trois remèdes : quasi-Poisson, erreurs-types robustes, binomiale négative. Vérifiez au passage par simulation que le mélange Poisson-Gamma a bien pour variance $\mu + \alpha\mu^2$.

```

rng = np.random.default_rng(3)
mu0, alpha0, N = 4.0, 0.6, 400_000
lam = rng.gamma(shape=1 / alpha0, scale=mu0 * alpha0, size=N) # Gamma de moyenne mu0 et de
      ↪ variance alpha0 * mu0^2
tirages = rng.poisson(lam)
print(f"simulation : moyenne = {tirages.mean():.3f} | variance = {tirages.var():.3f}")
print(f"formule : moyenne = {mu0:.3f} | variance = {mu0 + alpha0 * mu0**2:.3f}")

```

```

simulation : moyenne = 4.004 | variance = 13.640
formule : moyenne = 4.000 | variance = 13.600

```

```

formule = "nb_commandes_an ~ age + canal + offre_bienvenue"
quasi = smf.glm(formule, clients, family=sm.families.Poisson()).fit(scale="X2")
robuste = smf.glm(formule, clients, family=sm.families.Poisson()).fit(cov_type="HC0")
nb = smf.negativebinomial(formule, clients).fit(dis=0)

comp = pd.DataFrame({
    "coef Poisson": poisson.params, "coef NB": nb.params[poisson.params.index],
    "se Poisson": poisson.bse, "se quasi-Poisson": quasi.bse, "se robuste": robuste.bse, "se NB":
    ↪ nb.bse[poisson.params.index],
})
print()
print(comp.round(4).to_string())
print()
print("ratio se quasi-Poisson / se Poisson :", round(float((quasi.bse / poisson.bse).mean()), 3), "|
      ↪ racine de phi =", round(float(np.sqrt(poisson.pearson_chi2 / poisson.df_resid)), 3))
print("alpha (binomiale négative) =", round(float(nb.params["alpha"]), 3), "| IC95 :",
      ↪ [round(float(v), 3) for v in nb.conf_int().loc["alpha"]])
lr = 2 * (nb.llf - poisson.llf)
log10_p = stats.norm.logsf(np.sqrt(lr)) / np.log(10) # 0,5 × P(khi-deux à 1 ddl > lr) = P(N(0,1)
      ↪ > sqrt(lr)) ; en log pour éviter le dépassement de capacité
print(f"AIC Poisson = {poisson.aic:.1f} | AIC binomiale négative = {nb.aic:.1f} | RV : 2(l_NB -
      ↪ l_Poisson) = {lr:.1f}, log10(p) = {log10_p:.0f}")

```

	coef Poisson	coef NB	se Poisson	se quasi-Poisson	se robuste	se NB
Intercept	1.4637	1.4681	0.0458	0.0844	0.0837	0.0840
canal[T.Réseaux]	-0.1762	-0.1765	0.0289	0.0532	0.0529	0.0520
canal[T.Site]	0.0151	0.0148	0.0288	0.0531	0.0529	0.0534
age	-0.0010	-0.0011	0.0011	0.0020	0.0019	0.0020

```
offre_bienvenue      -0.0189  -0.0156      0.0227      0.0419      0.0419  0.0411

ratio se quasi-Poisson / se Poisson : 1.843 | racine de phi = 1.843
alpha (binomiale négative) = 0.584 | IC95 : [0.528, 0.639]
AIC Poisson = 11715.5 | AIC binomiale négative = 9768.7 | RV : 2(l_NB - l_Poisson) = 1948.8, log10(p)
↳ = -425
```

Étape 5 — Quelle loi décrit la distribution entière ? Comparez les fréquences observées à celles que prédisent Poisson et la binomiale négative (la figure du livre est produite par la cellule masquée).

```
valeurs = np.arange(0, 16)
obs = np.array([(y == k).mean() for k in valeurs])
mu_p = poisson.fittedvalues.to_numpy()
mu_nb = np.asarray(nb.predict())
a = float(nb.params["alpha"])
pred_p = np.array([stats.poisson.pmf(k, mu_p).mean() for k in valeurs])
pred_nb = np.array([stats.nbinom.pmf(k, 1 / a, (1 / a) / ((1 / a) + mu_nb)).mean() for k in valeurs])
print(pd.DataFrame({"observé": obs, "Poisson": pred_p, "binomiale négative": pred_nb},
↳ index=valeurs).round(3).head(8).to_string())
print()
print("écart absolu moyen aux fréquences observées : Poisson =", round(float(np.abs(obs -
↳ pred_p).mean()), 4), "| binomiale négative =", round(float(np.abs(obs - pred_nb).mean()), 4))
```

	observé	Poisson	binomiale négative
0	0.130	0.022	0.133
1	0.154	0.082	0.157
2	0.158	0.156	0.147
3	0.130	0.199	0.126
4	0.096	0.192	0.103
5	0.088	0.149	0.081
6	0.060	0.097	0.063
7	0.048	0.055	0.048

```
écart absolu moyen aux fréquences observées : Poisson = 0.033 | binomiale négative = 0.0035
```

Application 2.5 — Montants : régression Gamma contre régression sur le logarithme

Section du livre : 2.3.4. **Objectif :** ajuster une régression Gamma à lien log sur les acheteurs, la reproduire avec IRLS, puis comparer ses prédictions en € à celles d'une régression sur $\log y$ (la médiane, pas la moyenne).

```
acheteurs = clients[clients["depense_annuelle"] > 0].copy()
print("acheteurs :", len(acheteurs), "sur", len(clients), "| dépense moyenne =",
↳ round(acheteurs["depense_annuelle"].mean(), 1), "€ | médiane =",
↳ round(acheteurs["depense_annuelle"].median(), 1), "€")

formule_d = "depense_annuelle ~ age + canal + offre_bienvenue"
gamma = smf.glm(formule_d, acheteurs,
↳ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
print()
print(gamma.summary().tables[1])
phi = gamma.scale
print()
print("dispersion phi estimée =", round(phi, 3), "| coefficient de variation implicite =",
↳ round(float(np.sqrt(phi)), 3))
```

```
acheteurs : 1740 sur 2000 | dépense moyenne = 283.9 € | médiane = 193.4 €
```

```
=====
coef      std err      z      P>|z|      [0.025      0.975]
```

```
-----
Intercept          5.6086      0.098      57.234      0.000      5.417      5.801
canal[T.Réseaux]   -0.4988      0.061      -8.181      0.000     -0.618     -0.379
canal[T.Site]      -0.1518      0.063      -2.425      0.015     -0.275     -0.029
age                 0.0075      0.002       3.295      0.001      0.003      0.012
offre_bienvenue    -0.0098      0.048      -0.203      0.839     -0.104      0.085
=====
```

```
dispersion phi estimée = 1.009 | coefficient de variation implicite = 1.004
```

Reproduction avec notre IRLS (Application 2.1) : la dispersion ϕ est inconnue, on l'estime par $X^2/(n-p)$ et l'on multiplie la covariance par $\hat{\phi}$.

```
from patsy import dmatrices

Yd, Xd = dmatrices(formule_d, acheteurs, return_type="dataframe")
r = irls(Xd.to_numpy(), Yd.to_numpy().ravel(), "gamma_log")
mu_d = r["mu"]
phi_main = np.sum((Yd.to_numpy().ravel() - mu_d) ** 2 / mu_d**2) / (len(acheteurs) - Xd.shape[1])
se_main = np.sqrt(np.diag(r["cov"]) * phi_main)
print(pd.DataFrame({"coef main": r["beta"], "coef statsmodels": gamma.params.to_numpy(), "se main":
    ↳ se_main, "se statsmodels": gamma.bse.to_numpy()},
    index=Xd.columns).round(5).to_string())
print("phi (main) =", round(float(phi_main), 4), "| phi (statsmodels) =", round(phi, 4), "|
    ↳ itérations :", r["iterations"])
```

```
coef main  coef statsmodels  se main  se statsmodels
Intercept    5.60858         5.60858  0.09799    0.09799
canal[T.Réseaux] -0.49879        -0.49879  0.06097    0.06097
canal[T.Site]  -0.15181        -0.15181  0.06260    0.06260
age           0.00754         0.00754  0.00229    0.00229
offre_bienvenue -0.00980        -0.00980  0.04819    0.04819
phi (main) = 1.0088 | phi (statsmodels) = 1.0088 | itérations : 11
```

Gamma contre OLS sur log y : les rapports de moyennes sont voisins, mais pas les prédictions en €. Testez aussi la correction de Duan.

```
ols_log = smf.ols("np.log(depense_annuelle) ~ age + canal + offre_bienvenue", acheteurs).fit()
print(pd.DataFrame({"effet (Gamma, log)": gamma.params, "effet (OLS sur log y)":
    ↳ ols_log.params}).round(4).to_string())
print()
y_d = acheteurs["depense_annuelle"]
naif_log = np.exp(ols_log.fittedvalues)
lissage = naif_log * np.mean(np.exp(ols_log.resid)) # correction de Duan (« smearing »)
print("dépense moyenne observée :", round(float(y_d.mean()), 1), "€")
print("moyenne des prédictions Gamma (lien log) :",
    ↳ round(float(gamma.fittedvalues.mean()), 1), "€")
print("moyenne des prédictions exp(OLS sur log y), sans correction :", round(float(naif_log.mean()),
    ↳ 1), "€")
print("idem avec la correction de Duan :", round(float(lissage.mean()), 1),
    ↳ "€")
print()
par_canal = pd.DataFrame({"observée": y_d.groupby(acheteurs["canal"], observed=True).mean(),
    "Gamma": gamma.fittedvalues.groupby(acheteurs["canal"], observed=True).mean(),
    "exp(OLS log)": naif_log.groupby(acheteurs["canal"], observed=True).mean()})
print(par_canal.round(1).to_string())
```

```
effet (Gamma, log)  effet (OLS sur log y)
Intercept          5.6086          5.1977
canal[T.Réseaux]   -0.4988          -0.4608
canal[T.Site]      -0.1518          -0.1381
age                 0.0075           0.0075
offre_bienvenue    -0.0098          -0.0238
```

```
dépense moyenne observée : 283.9 €
moyenne des prédictions Gamma (lien log) : 284.0 €
moyenne des prédictions exp(OLS sur log y), sans correction : 189.9 €
idem avec la correction de Duan : 283.1 €
```

	observée	Gamma	exp(OLS log)
canal			
Boutique	355.0	356.6	234.6
Réseaux	216.8	217.0	148.2
Site	307.2	306.1	204.1

Application 2.6 — Vérifier un modèle : déviance, résidus, calibration, influence

Sections du livre : 2.4.1 à 2.4.6. **Objectif** : calculer la déviance à la main, comparer des modèles emboîtés, produire les résidus quantiles aléatoires, tester la calibration, comparer par AIC/BIC et repérer les observations influentes.

Étape 1 — Déviance unitaire, calcul à la main. Quatre clients avec $y = (2, 5, 3, 8)$ commandes ; on compare avec statsmodels, puis sur les trois modèles du chapitre.

```
from scipy.special import xlogy

def deviance_unitaire(famille, y, mu):
    """Déviance unitaire d(y, mu) pour chaque observation (xlogy gère la convention 0 log 0 = 0)."""
    if famille == "normale":
        return (y - mu) ** 2
    if famille == "poisson":
        return 2 * (xlogy(y, y / mu) - (y - mu))
    if famille == "bernoulli":
        return 2 * (xlogy(y, y / mu) + xlogy(1 - y, (1 - y) / (1 - mu)))
    if famille == "gamma":
        return 2 * (-np.log(y / mu) + (y - mu) / mu)
```

```
y4 = np.array([2.0, 5, 3, 8])
mu4 = np.full(4, y4.mean())
print("déviance à la main      :", round(float(deviance_unitaire("poisson", y4, mu4).sum()), 4))
print("Pearson X² à la main    :", round(float(((y4 - mu4) ** 2) / mu4).sum()), 4))
m4 = sm.GLM(y4, np.ones((4, 1)), family=sm.families.Poisson()).fit()
print("déviance statsmodels    :", round(float(m4.deviance), 4), "| Pearson statsmodels :",
      ↪ round(float(m4.pearson_chi2), 4))
```

```
déviance à la main      : 4.5829
Pearson X² à la main    : 4.6667
déviance statsmodels    : 4.5829 | Pearson statsmodels : 4.6667
```

```
# Les trois modèles des sections 2.2 et 2.3, sur les vraies données
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Réseaux",
      ↪ "Site"])
logi = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
      ↪ family=sm.families.Binomial()).fit()
poi = smf.glm("nb_commandes_an ~ age + canal + offre_bienvenue", clients,
      ↪ family=sm.families.Poisson()).fit()
acheteurs = clients[clients["depense_annuelle"] > 0]
gam = smf.glm("depense_annuelle ~ age + canal + offre_bienvenue", acheteurs,
      ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")

lignes = []
for nom, fam, res, y in [("logistique", "bernoulli", logi, clients["rachat_12m"]), ("Poisson",
      ↪ "poisson", poi, clients["nb_commandes_an"]),
      ↪ ("Gamma", "gamma", gam, acheteurs["depense_annuelle"])]:
```

```

d_main = deviance_unitaire(fam, y.to_numpy(float), res.fittedvalues.to_numpy()).sum()
mu_nul = np.full(len(y), y.mean())
d_nul = deviance_unitaire(fam, y.to_numpy(float), mu_nul).sum()
lignes.append({"modèle": nom, "déviante (main)": d_main, "déviante (statsmodels)": res.deviance,
↪ "déviante nulle (main)": d_nul,
               "déviante nulle (statsmodels)": res.null_deviance, "part de déviante expliquée": 1
↪ - d_main / d_nul})
print()
print(pd.DataFrame(lignes).round(3).to_string(index=False))

```

	modèle	déviante (main)	déviante (statsmodels)	déviante nulle (main)	déviante nulle
↪	(statsmodels)	part de déviante expliquée			
	logistique	2710.830	2710.830	2771.867	2771.867
↪	0.022				
	Poisson	6292.954	6292.954	6357.639	6357.639
↪	0.010				
	Gamma	1389.349	1389.349	1473.452	1473.452
↪	0.057				

Étape 2 — Rapport de vraisemblance entre modèles emboîtés (logistique, puis test F pour la régression Gamma).

```

def test_rv(complet, reduit, ddl):
    delta = reduit.deviance - complet.deviance
    return delta, ddl, stats.chi2.sf(delta, ddl)

formule_c = "rachat_12m ~ offre_bienvenue + age + canal"
complet = smf.glm(formule_c, clients, family=sm.families.Binomial()).fit()
essais = [("canal (2 ddl)", "rachat_12m ~ offre_bienvenue + age", 2),
          ("age (1 ddl)", "rachat_12m ~ offre_bienvenue + canal", 1),
          ("offre_bienvenue (1 ddl)", "rachat_12m ~ age + canal", 1)]
wald = complet.wald_test_terms(scalar=True).table
lignes = []
for nom, f_reduite, ddl in essais:
    reduit = smf.glm(f_reduite, clients, family=sm.families.Binomial()).fit()
    delta, q, p_rv = test_rv(complet, reduit, ddl)
    terme = nom.split(" ")[0]
    lignes.append({"variable retirée": nom, "D réduit - D complet": delta, "p (rapport de
↪ vraisemblance)": p_rv,
                  "p (Wald)": wald.loc[terme, "pvalue"]})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
print()
print("déviante du modèle complet :", round(complet.deviance, 2), "| déviante sans l'offre :",
↪ round(smf.glm("rachat_12m ~ age + canal", clients, family=sm.families.Binomial()).fit().deviance,
↪ 2))

```

	variable retirée	D réduit - D complet	p (rapport de vraisemblance)	p (Wald)
	canal (2 ddl)	17.7355	0.0001	0.0001
	age (1 ddl)	12.7919	0.0003	0.0004
	offre_bienvenue (1 ddl)	29.6401	0.0000	0.0000

déviante du modèle complet : 2710.83 | déviante sans l'offre : 2740.47

```

f_gamma = "depense_annuelle ~ age + canal + offre_bienvenue"
g_complet = smf.glm(f_gamma, acheteurs,
↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
phi_g = g_complet.scale
for nom, f_reduite, ddl in [("canal (2 ddl)", "depense_annuelle ~ age + offre_bienvenue", 2),
↪ ("offre_bienvenue (1 ddl)", "depense_annuelle ~ age + canal", 1)]:
    g_reduit = smf.glm(f_reduite, acheteurs,
↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
    F = (g_reduit.deviance - g_complet.deviance) / ddl / phi_g

```

```
print(f"nom:24s): F = {F:7.2f} sur ({ddl}, {int(g_complet.df_resid)}) ddl | p = {stats.f.sf(F,
↪ ddl, g_complet.df_resid):.2e}")
```

```
canal (2 ddl)          : F = 37.09 sur (2, 1735) ddl | p = 1.69e-16
offre_bienvenue (1 ddl) : F = 0.04 sur (1, 1735) ddl | p = 8.39e-01
```

Étape 3 — Résidus de Pearson et de déviance du modèle de Poisson : leur écart-type devrait être voisin de 1 si le modèle était correct.

```
r_p = poi.resid_pearson.to_numpy()
r_d = poi.resid_deviance.to_numpy()
print("Poisson : somme des carrés des résidus de Pearson =", round(float((r_p ** 2).sum()), 1), "|
↪ X² =", round(poi.pearson_chi2, 1))
print("Poisson : somme des carrés des résidus de déviance =", round(float((r_d ** 2).sum()), 1), "| D
↪ =", round(poi.deviance, 1))
print("résidus de Pearson : moyenne =", round(float(r_p.mean()), 3), "| écart-type =",
↪ round(float(r_p.std()), 3), "(attendu : environ 1 si le modèle est correct)")
print("part de |résidus de Pearson| > 2 :", round(float((np.abs(r_p) > 2).mean()), 3), "(attendu pour
↪ une loi normale : environ 0,046)")
```

```
Poisson : somme des carrés des résidus de Pearson = 6774.2 | X² = 6774.2
Poisson : somme des carrés des résidus de déviance = 6293.0 | D = 6293.0
résidus de Pearson : moyenne = -0.0 | écart-type = 1.84 (attendu : environ 1 si le modèle est correct)
part de |résidus de Pearson| > 2 : 0.175 (attendu pour une loi normale : environ 0,046)
```

Étape 4 — Résidus quantiles aléatoires (Dunn et Smyth) pour quatre modèles. Pour la logistique, comparez avec le modèle sans variable : que constatez-vous ?

```
rng = np.random.default_rng(44)

def residus_quantiles(u_bas, u_haut):
    u = rng.uniform(u_bas, u_haut)
    return stats.norm.ppf(np.clip(u, 1e-12, 1 - 1e-12))

y_b = clients["rachat_12m"].to_numpy()
p_b = logi.fittedvalues.to_numpy()
rq_logi = residus_quantiles(np.where(y_b == 1, 1 - p_b, 0.0), np.where(y_b == 1, 1.0, 1 - p_b))
```

Binomiale négative et Gamma : leurs résidus quantiles se calculent avec la loi de répartition propre à chaque modèle.

```
y_c = clients["nb_commandes_an"].to_numpy()
mu_p = poi.fittedvalues.to_numpy()
rq_poi = residus_quantiles(stats.poisson.cdf(y_c - 1, mu_p), stats.poisson.cdf(y_c, mu_p))

nbm = smf.negativebinomial("nb_commandes_an ~ age + canal + offre_bienvenue", clients).fit(dis=0)
a_nb, mu_nb = float(nbm.params["alpha"]), np.asarray(nbm.predict())
n_nb, p_nb = 1 / a_nb, (1 / a_nb) / ((1 / a_nb) + mu_nb)
rq_nb = residus_quantiles(stats.nbinom.cdf(y_c - 1, n_nb, p_nb), stats.nbinom.cdf(y_c, n_nb, p_nb))
```

Résumé : le modèle sans variable sert de point de comparaison pour la logistique ; on résume les moyenne, écart-type, part de $|r| > 2$ et test de Kolmogorov-Smirnov.

```
y_g = acheteurs["depense_annuelle"].to_numpy()
mu_g = gam.fittedvalues.to_numpy()
u_g = stats.gamma.cdf(y_g, a=1 / gam.scale, scale=mu_g * gam.scale)
rq_gam = stats.norm.ppf(np.clip(u_g, 1e-12, 1 - 1e-12))

# Même le modèle SANS variable (probabilité constante) donne des résidus quantiles « parfaits » pour
↪ un 0/1
p_nul = np.full(len(y_b), y_b.mean())
rq_nul = residus_quantiles(np.where(y_b == 1, 1 - p_nul, 0.0), np.where(y_b == 1, 1.0, 1 - p_nul))
```

```

resume = []
for nom, r in [("logistique", rq_logi), ("logistique sans variable", rq_nul), ("Poisson", rq_poi),
              ("binomiale négative", rq_nb), ("Gamma", rq_gam)]:
    resume.append({"modèle": nom, "moyenne": r.mean(), "écart-type": r.std(), "part de |r| > 2":
                  (np.abs(r) > 2).mean(),
                  "p (Kolmogorov-Smirnov)": stats.kstest(r, "norm").pvalue})
print(pd.DataFrame(resume).round(3).to_string(index=False))

```

	modèle	moyenne	écart-type	part de r > 2	p (Kolmogorov-Smirnov)
	logistique	0.010	1.009	0.048	0.896
logistique	sans variable	-0.020	1.008	0.040	0.257
	Poisson	-0.148	1.675	0.228	0.000
	binomiale négative	0.004	0.991	0.045	0.849
	Gamma	0.087	0.833	0.023	0.000

Étape 5 — Dispersion et calibration. Le rapport X^2/ddl pour la binomiale négative, puis le test de Hosmer-Lemeshow (sur le modèle de rachat, puis sur des sessions de navigation dont l'effet de la durée est « en cloche »).

```

x2_nb = np.sum((y_c - mu_nb) ** 2 / (mu_nb + a_nb * mu_nb ** 2))
ddl_nb = len(y_c) - len(nbm.params) + 1          # on retire le paramètre alpha du compte des
↳ coefficients
print("Poisson          : X²/ddl =", round(poi.pearson_chi2 / poi.df_resid, 3))
print("binomiale négative : X²/ddl =", round(float(x2_nb / ddl_nb), 3))

```

```

Poisson          : X²/ddl = 3.396
binomiale négative : X²/ddl = 1.044

```

```

def hosmer_lemeshow(y, p, g=10):
    classes = pd.qcut(p, g, labels=False, duplicates="drop")
    d = pd.DataFrame({"y": y, "p": p, "k": classes}).groupby("k").agg(0=("y", "sum"), E=("p", "sum"),
    ↳ n=("y", "size"))
    d["pbar"] = d["E"] / d["n"]
    hl = (((d["0"] - d["E"]) ** 2) / (d["E"] * (1 - d["pbar"]))).sum()
    ddl = len(d) - 2
    return hl, ddl, stats.chi2.sf(hl, ddl), d

```

```

hl, ddl, p_hl, tab = hosmer_lemeshow(clients["rachat_l2m"].to_numpy(), logi.fittedvalues.to_numpy())
print(f"modèle de rachat : HL = {hl:.2f} sur {ddl} ddl, p = {p_hl:.3f}")

```

```

# Un modèle mal spécifié : sessions de navigation (simulées), effet de la durée en « cloche » mais
↳ modèle linéaire sur le logit

```

```

sessions = pd.read_csv("donnees/ch02-sessions.csv")
lineaire = smf.glm("achat ~ duree_min", sessions, family=sm.families.Binomial()).fit()
bosse = smf.glm("achat ~ duree_min + I(duree_min**2)", sessions, family=sm.families.Binomial()).fit()
for nom, res in [("linéaire sur le logit", lineaire), ("avec terme quadratique", bosse)]:
    hl_s, ddl_s, p_s, _ = hosmer_lemeshow(sessions["achat"].to_numpy(), res.fittedvalues.to_numpy())
    print(f"sessions, {nom:24s} : HL = {hl_s:6.2f} sur {ddl_s} ddl, p = {p_s:.4f} | AIC =
    ↳ {res.aic:.1f}")

```

```

modèle de rachat : HL = 6.15 sur 8 ddl, p = 0.631
sessions, linéaire sur le logit      : HL = 243.18 sur 8 ddl, p = 0.0000 | AIC = 1960.3
sessions, avec terme quadratique    : HL = 54.33 sur 8 ddl, p = 0.0000 | AIC = 1787.7

```

Étape 6 — Résidus par classes : comparez, par tranche de durée, la fréquence observée aux probabilités du modèle linéaire et du modèle quadratique. (La figure est produite par la cellule masquée.)

```

tranches = pd.cut(sessions["duree_min"], bins=[0, 3, 5, 7, 9, 11, 13, 15, 18, 22, 40])
g = sessions.assign(p_lin=lineaire.fittedvalues, p_bosse=bosse.fittedvalues,
↳ tranche=tranches).groupby("tranche", observed=True).agg(

```

```
duree=("duree_min", "mean"), observe=("achat", "mean"), p_lineaire=("p_lin", "mean"),
↳ p_bosse=("p_bosse", "mean"), n=("achat", "size"))
print(g.round(3).to_string())
```

	duree	observe	p_lineaire	p_bosse	n
tranche					
(0, 3]	2.265	0.189	0.630	0.174	37
(3, 5]	4.240	0.333	0.584	0.343	114
(5, 7]	6.113	0.381	0.539	0.489	231
(7, 9]	8.100	0.650	0.490	0.581	240
(9, 11]	10.051	0.740	0.443	0.600	223
(11, 13]	11.930	0.566	0.398	0.553	182
(13, 15]	14.071	0.320	0.350	0.422	147
(15, 18]	16.367	0.140	0.301	0.226	171
(18, 22]	19.996	0.079	0.232	0.037	101
(22, 40]	26.831	0.019	0.139	0.001	54

Étape 7 — AIC et BIC, observations influentes. Cinq modèles de rachat, de « aucune variable » à « avec interactions » : les deux critères choisissent-ils le même ? Puis levier et distance de Cook du modèle logistique.

```
formules = {
    "aucune variable": "rachat_12m ~ 1",
    "+ offre": "rachat_12m ~ offre_bienvenue",
    "+ offre + âge": "rachat_12m ~ offre_bienvenue + age",
    "+ offre + âge + canal": "rachat_12m ~ offre_bienvenue + age + canal",
    "+ offre × canal (interactions)": "rachat_12m ~ offre_bienvenue * canal + age",
}
lignes = []
ajustes = {}
for nom, f in formules.items():
    r = smf.glm(f, clients, family=sm.families.Binomial()).fit()
    ajustes[nom] = r
    lignes.append({"modèle": nom, "paramètres": len(r.params), "déviante": r.deviance, "AIC": r.aic,
    ↳ "BIC": r.bic_2llf})
res = pd.DataFrame(lignes)
res["ΔAIC"] = res["AIC"] - res["AIC"].min()
res["ΔBIC"] = res["BIC"] - res["BIC"].min()
print(res.round(1).to_string(index=False))
delta_inter = ajustes["+ offre + âge + canal"].deviance - ajustes["+ offre × canal
↳ (interactions)"].deviance
print(f"\ntest du rapport de vraisemblance pour les 2 interactions : différence de déviante =
↳ {delta_inter:.2f}, p = {stats.chi2.sf(delta_inter, 2):.3f}")
```

	modèle	paramètres	déviante	AIC	BIC	ΔAIC	ΔBIC
aucune variable		1	2771.9	2773.9	2779.5	57.8	30.6
+ offre		2	2742.1	2746.1	2757.3	30.1	8.5
+ offre + âge		3	2728.6	2734.6	2751.4	18.5	2.5
+ offre + âge + canal		5	2710.8	2720.8	2748.8	4.8	0.0
+ offre × canal (interactions)		7	2702.1	2716.1	2755.3	0.0	6.4

test du rapport de vraisemblance pour les 2 interactions : différence de déviante = 8.77, p = 0.012

```
infl = logi.get_influence()
levier = infl.hat_matrix_diag
cook = infl.cooks_distance[0]
print("levier moyen =", round(float(levier.mean()), 4), "(= p/n =", round(5 / len(clients), 4), ") |
↳ levier maximal =", round(float(levier.max()), 4))
print("distance de Cook maximale =", round(float(cook.max()), 4), "| seuil usuel 4/n =", round(4 /
↳ len(clients), 4), "| clients au-dessus du seuil :", int((cook > 4 / len(clients)).sum()))
pire = clients.loc[int(np.argmax(cook)), ["age", "canal_acquisition", "offre_bienvenue",
↳ "rachat_12m"]]
print("client le plus influent :", pire.to_dict())
```

```
print("probabilité de rachat prédite pour ce client :",
      ↪ round(float(logi.fittedvalues.iloc[int(np.argmax(cook))]), 3))

levier moyen = 0.0025 (= p/n = 0.0025 ) | levier maximal = 0.0076
distance de Cook maximale = 0.0023 | seuil usuel 4/n = 0.002 | clients au-dessus du seuil : 3
client le plus influent : {'age': 68, 'canal_acquisition': 'Boutique', 'offre_bienvenue': 0,
      ↪ 'rachat_12m': 1}
probabilité de rachat prédite pour ce client : 0.386
```

Application 2.7 — Un GAM pour la durée des sessions

Section du livre : 2.5. Objectif : comprendre une base de fonctions (la base « chapeau » à la main), comparer des splines de 3, 6 et 15 fonctions, pénaliser la courbure, puis passer le test de calibration.

Étape 1 — Une base « chapeau », à la main. Quatre nœuds, quatre coefficients : calculez $f(5)$, $f(10)$, $f(14)$, $f(25)$.

```
noeuds = np.array([0.0, 10.0, 20.0, 30.0])
def chapeau(x, k):
    return np.maximum(0.0, 1 - np.abs(x - noeuds[k]) / 10)

gamma = np.array([-1.8, 1.0, -1.2, -2.8])
for x in (5.0, 10.0, 14.0, 25.0):
    poids = [round(float(chapeau(x, k)), 2) for k in range(4)]
    f = sum(gamma[k] * chapeau(x, k) for k in range(4))
    print(f"x = {x:4.1f} : poids des 4 fonctions de base = {poids} -> f(x) = {f:.3f}")
```

```
x = 5.0 : poids des 4 fonctions de base = [0.5, 0.5, 0.0, 0.0] -> f(x) = -0.400
x = 10.0 : poids des 4 fonctions de base = [0.0, 1.0, 0.0, 0.0] -> f(x) = 1.000
x = 14.0 : poids des 4 fonctions de base = [0.0, 0.6, 0.4, 0.0] -> f(x) = 0.120
x = 25.0 : poids des 4 fonctions de base = [0.0, 0.0, 0.5, 0.5] -> f(x) = -2.000
```

Étape 2 — Splines non pénalisées de souplesse croissante, comparées à la vérité (connue, car les sessions sont simulées).

```
sessions = pd.read_csv("donnees/ch02-sessions.csv")
def vrai_p(d):
    return expit(-1.8 + 3 * np.exp(-((d - 10) / 5) ** 2) - 0.04 * d)

grille = np.linspace(1, 35, 200)
p_vrai_obs = vrai_p(sessions["duree_min"].to_numpy()) # vraie probabilité de chaque session
rmse = lambda p_hat: float(np.sqrt(np.mean((p_hat - p_vrai_obs) ** 2)))

modeles = {
    "linéaire": "achat ~ duree_min",
    "quadratique": "achat ~ duree_min + I(duree_min**2)",
    "spline, 3 fonctions de base": "achat ~ bs(duree_min, df=3, lower_bound=0, upper_bound=40)",
    "spline, 6 fonctions de base": "achat ~ bs(duree_min, df=6, lower_bound=0, upper_bound=40)",
    "spline, 15 fonctions de base": "achat ~ bs(duree_min, df=15, lower_bound=0, upper_bound=40)",
}
courbes, lignes = {}, []
for nom, f in modeles.items():
    r = smf.glm(f, sessions, family=sm.families.Binomial()).fit()
    courbes[nom] = r.predict(pd.DataFrame({"duree_min": grille})).to_numpy()
    lignes.append({"modèle": nom, "paramètres": len(r.params), "AIC": r.aic, "erreur quadratique vs
      ↪ vérité": rmse(r.fittedvalues.to_numpy())})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
```

modèle	paramètres	AIC	erreur quadratique vs vérité
linéaire	2	1960.2911	0.1962

quadratique	3	1787.7267	0.0776
spline, 3 fonctions de base	4	1739.8372	0.0630
spline, 6 fonctions de base	7	1703.4622	0.0306
spline, 15 fonctions de base	16	1708.0771	0.0491

Étape 3 — Pénaliser la courbure. On prend 12 fonctions de base et l'on balaie le paramètre de lissage λ (alpha), en gardant l'AIC minimal. Regardez comment les degrés de liberté effectifs diminuent quand λ augmente.

```
from statsmodels.gam.api import GLMGam, BSplines

base = BSplines(sessions[["duree_min"]], df=[12], degree=[3])           # 12 fonctions de base : large,
↳ la pénalité fera le tri
y_s = sessions["achat"].to_numpy()
X0 = np.ones((len(sessions), 1))                                       # constante seule : tout le reste
↳ est dans le lissage

lignes, ajustes = [], {}
for alpha in [0.01, 0.1, 1, 10, 30, 100, 1000, 10000]:
    res = GLMGam(y_s, exog=X0, smoother=base, alpha=alpha, family=sm.families.Binomial()).fit()
    ajustes[alpha] = res
    lignes.append({"alpha (lambda)": alpha, "edf du lissage": res.edf.sum() - 1, "AIC": res.aic,
                  "erreur quadratique vs vérité": rmse(res.fittedvalues)})
tab_alpha = pd.DataFrame(lignes)
print(tab_alpha.round(4).to_string(index=False))
alpha_opt = float(tab_alpha.loc[tab_alpha["AIC"].idxmin(), "alpha (lambda)"])
gam = ajustes[alpha_opt]
print("\nalpha retenu (AIC minimal) :", alpha_opt, "| edf du lissage =", round(float(gam.edf.sum() -
↳ 1), 2))
```

alpha (lambda)	edf du lissage	AIC	erreur quadratique vs vérité
0.01	10.9622	1709.0463	0.0347
0.10	10.6634	1708.4685	0.0343
1.00	9.2056	1705.9029	0.0325
10.00	6.5311	1702.7064	0.0269
30.00	5.1937	1704.1220	0.0264
100.00	3.8272	1711.9038	0.0380
1000.00	1.9769	1773.7390	0.0995
10000.00	1.2678	1900.7931	0.1715

alpha retenu (AIC minimal) : 10.0 | edf du lissage = 6.53

Étape 4 — Le GAM passe-t-il le test de Hosmer-Lemeshow ? (la fonction `hosmer_lemeshow` est définie à l'application 2.6). La figure du livre est produite par la cellule masquée.

```
for nom, p in [("linéaire", smf.glm("achat ~ duree_min", sessions,
↳ family=sm.families.Binomial()).fit().fittedvalues.to_numpy()),
              ("quadratique", smf.glm("achat ~ duree_min + I(duree_min**2)", sessions,
↳ family=sm.families.Binomial()).fit().fittedvalues.to_numpy()),
              ("GAM pénalisé", gam.fittedvalues)]:
    hl, ddl, p_val, _ = hosmer_lemeshow(y_s, p)
    print(f"{nom:14s}: HL = {hl:7.2f} sur {ddl} ddl, p = {p_val:.4f}")
```

```
linéaire      : HL = 243.18 sur 8 ddl, p = 0.0000
quadratique   : HL = 54.33 sur 8 ddl, p = 0.0000
GAM pénalisé  : HL = 6.29 sur 8 ddl, p = 0.6146
```

Étape 5 — Avec R et mgcv. Le paquet de référence choisit λ par REML et donne des intervalles de confiance.

```
suppressPackageStartupMessages(library(mgcv))
sessions <- read.csv("donnees/ch02-sessions.csv")
```

```
m <- gam(achat ~ s(duree_min), family = binomial, data = sessions, method = "REML")
print(summary(m))
```

```
Family: binomial
Link function: logit

Formula:
achat ~ s(duree_min)

Parametric coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.4779      0.0696  -6.867 6.58e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
              edf Ref.df Chi.sq p-value
s(duree_min)  6.5  7.429  243.6 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.211  Deviance explained = 17.5%
-REML = 858.59  Scale est. = 1          n = 1500
```

Étape 6 — Tester la linéarité de l'âge dans le modèle de rachat : comparez le modèle linéaire et le modèle $s(\text{age})$.

```
library(mgcv)
clients <- read.csv("donnees/clients.csv")
clients$canal <- factor(clients$canal_acquisition, levels = c("Boutique", "Réseaux", "Site"))
lineaire <- gam(rachat_12m ~ offre_bienvenue + canal + age, family = binomial, data = clients, method
  <- "REML")
souple <- gam(rachat_12m ~ offre_bienvenue + canal + s(age), family = binomial, data = clients, method
  <- "REML")
print(round(summary(souple)$s.table, 3))
print(anova(lineaire, souple, test = "Chisq"))
print(AIC(lineaire, souple))
```

```
              edf Ref.df Chi.sq p-value
s(age) 1.579  1.971  13.87  0.001
Analysis of Deviance Table

Model 1: rachat_12m ~ offre_bienvenue + canal + age
Model 2: rachat_12m ~ offre_bienvenue + canal + s(age)
  Resid. Df Resid. Dev    Df Deviance Pr(>Chi)
1    1995.0      2710.8
2    1993.6      2709.4 1.364   1.4133  0.3311
              df      AIC
lineaire 5.000000 2720.83
souple   5.971442 2721.36
```

Application 2.8 — Zéros en excès, Tweedie et modèle à deux parties

Section du livre : 2.6. Objectif : simuler un ZIP, voir que nos comptages n'ont pas d'excès de zéros mais de la surdispersion, voir un cas où l'inflation est réelle, estimer la puissance de Tweedie et comparer Tweedie au modèle à deux parties.

Étape 1 — Un ZIP simulé : comparez probabilité de zéro, moyenne et variance à la théorie.

```

rng = np.random.default_rng(26)
pi, mu = 0.2, 3.0
actif = rng.random(400_000) >= pi
y = np.where(actif, rng.poisson(mu, 400_000), 0)
print(f"ZIP simulé : P(0) = {np.mean(y == 0):.4f} | moyenne = {y.mean():.3f} | variance = {y.var():.3f} | variance / moyenne = {y.var() / y.mean():.3f}")
print(f"ZIP théorique : P(0) = {pi + (1 - pi) * np.exp(-mu):.4f} | moyenne = {(1 - pi) * mu:.3f} | variance = {(1 - pi) * mu * (1 + pi * mu):.3f} | variance / moyenne = {1 + pi * mu:.3f}")
print(f"Poisson(3) : P(0) = {np.exp(-mu):.4f}")

```

```

ZIP simulé : P(0) = 0.2407 | moyenne = 2.398 | variance = 3.836 | variance / moyenne = 1.600
ZIP théorique : P(0) = 0.2398 | moyenne = 2.400 | variance = 3.840 | variance / moyenne = 1.600
Poisson(3) : P(0) = 0.0498

```

Étape 2 — Nos comptages ont-ils des zéros en trop ? Poisson, ZIP, binomiale négative, ZINB.

```

clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Réseaux", "Site"])
X = sm.add_constant(pd.get_dummies(clients[["age", "canal"]], drop_first=True, dtype=float))
y_c = clients["nb_commandes_an"]
infl = np.ones((len(clients), 1))

poi = sm.Poisson(y_c, X).fit(dis=0)
zip_ = sm.ZeroInflatedPoisson(y_c, X, exog_infl=infl).fit(dis=0, maxiter=300)
nb = sm.NegativeBinomial(y_c, X).fit(dis=0)
zinb = sm.ZeroInflatedNegativeBinomialP(y_c, X, exog_infl=infl, p=2).fit(dis=0, maxiter=500)

lignes = []
for nom, r in [("Poisson", poi), ("ZIP", zip_), ("binomiale négative", nb), ("ZINB", zinb)]:
    pi_hat = float(expt(r.params["inflate_const"]))
    lignes.append({"modèle": nom, "paramètres": len(r.params), "log-vraisemblance": r.llf, "AIC": r.aic, "pi estimé": pi_hat})
print(pd.DataFrame(lignes).round(4).to_string(index=False))
print()
print("part de zéros observée :", round(float((y_c == 0).mean()), 4))
print("alpha (NB) =", round(float(nb.params["alpha"]), 3), "| alpha (ZINB) =", round(float(zinb.params["alpha"]), 3))

```

	modèle	paramètres	log-vraisemblance	AIC	pi estimé
	Poisson	4	-5853.0778	11714.1557	0.0000
	ZIP	5	-5536.1403	11082.2806	0.1181
binomiale négative		5	-4878.4227	9766.8453	0.0000
ZINB		6	-4878.4227	9768.8453	0.0000

```

part de zéros observée : 0.13
alpha (NB) = 0.584 | alpha (ZINB) = 0.584

```

Étape 3 — Un cas où l'inflation est réelle : 25 % de comptes dormants.

```

rng = np.random.default_rng(27)
n = 1500
x = rng.normal(size=n)
dormant = rng.random(n) < 0.25
y_sim = np.where(dormant, 0, rng.poisson(np.exp(0.9 + 0.4 * x)))
Xs = sm.add_constant(pd.DataFrame({"x": x}))
infl = np.ones((n, 1))

poi_s = sm.Poisson(y_sim, Xs).fit(dis=0)
nb_s = sm.NegativeBinomial(y_sim, Xs).fit(dis=0)
zip_s = sm.ZeroInflatedPoisson(y_sim, Xs, exog_infl=infl).fit(dis=0, maxiter=300)

print("part de zéros observée :", round(float((y_sim == 0).mean()), 3), "| part de dormants simulée :", round(float(dormant.mean()), 3))

```

```

for nom, r in [("Poisson", poi_s), ("binomiale négative", nb_s), ("ZIP", zip_s)]:
    print(f"{nom:20s}: AIC = {r.aic:8.1f}")
print()
print("ZIP : pi estimé =", round(float(expit(zip_s.params["inflate_const"])), 3), "| proportion
↪ réellement simulée de dormants :", round(float(dormant.mean()), 3), "(probabilité programmée :
↪ 0.25)")
print("ZIP : coefficients du comptage (const, x) =", zip_s.params[["const",
↪ "x"]].round(3).to_numpy(), "(vrais : 0.9, 0.4)")
print("binomiale négative : coefficients (const, x) =", nb_s.params[["const",
↪ "x"]].round(3).to_numpy(), "| alpha =", round(float(nb_s.params["alpha"]), 3))

```

```

part de zéros observée : 0.316 | part de dormants simulée : 0.23
Poisson                : AIC =   5786.0
binomiale négative     : AIC =   5505.3
ZIP                    : AIC =   5292.1

```

```

ZIP : pi estimé = 0.233 | proportion réellement simulée de dormants : 0.23 (probabilité programmée :
↪ 0.25)
ZIP : coefficients du comptage (const, x) = [0.871 0.418] (vrais : 0.9, 0.4)
binomiale négative : coefficients (const, x) = [0.606 0.407] | alpha = 0.448

```

Étape 4 — La loi de Tweedie : vérifiez par simulation la probabilité de zéro, la moyenne et la variance pour deux valeurs de μ .

```

rng = np.random.default_rng(28)
p_tw, phi_tw, N = 1.5, 20.0, 400_000
lignes = []
for mu_tw in (50.0, 200.0):
    lam = mu_tw ** (2 - p_tw) / (phi_tw * (2 - p_tw))
    alpha = (2 - p_tw) / (p_tw - 1)
    theta = phi_tw * (p_tw - 1) * mu_tw ** (p_tw - 1)
    n_cmd = rng.poisson(lam, N)
    y_tw = np.where(n_cmd > 0, rng.gamma(np.maximum(n_cmd, 1) * alpha, theta), 0.0)
    lignes.append({"moyenne mu": mu_tw, "lambda": lam, "forme alpha": alpha, "échelle theta": theta,
                  "P(0) simulé": np.mean(y_tw == 0), "P(0) = exp(-lambda)": np.exp(-lam),
                  "moyenne simulée": y_tw.mean(), "variance simulée": y_tw.var(), "phi * mu^p":
                  ↪ phi_tw * mu_tw ** p_tw})
print(pd.DataFrame(lignes).round(3).T.to_string(header=False))

```

```

moyenne mu      50.000    200.000
lambda          0.707      1.414
forme alpha      1.000      1.000
échelle theta    70.711    141.421
P(0) simulé      0.492      0.243
P(0) = exp(-lambda) 0.493      0.243
moyenne simulée  49.984    200.039
variance simulée 7048.066  56394.773
phi * mu^p      7071.068  56568.542

```

Étape 5 — Estimer la puissance p avec R (mgcv).

```

suppressPackageStartupMessages(library(mgcv))
clients <- read.csv("donnees/clients.csv")
clients$canal <- factor(clients$canal_acquisition, levels = c("Boutique", "Réseaux", "Site"))
f <- depense_annuelle ~ age + canal + offre_bienvenue

puissances <- c(1.2, 1.3, 1.4, 1.45, 1.5, 1.6, 1.7, 1.8)
logv <- sapply(puissances, function(p) as.numeric(logLik(gam(f, family = Tweedie(p = p, link =
↪ "log"), data = clients, method = "REML"))))
print(data.frame(p = puissances, log_vraisemblance = round(logv, 1)))

m <- gam(f, family = tw(), data = clients, method = "REML") # tw() estime p en même temps que
↪ les coefficients

```

```
p_hat <- m$family$getTheta(TRUE)
cat("puissance p estimée :", round(p_hat, 3), "| dispersion phi :", round(m$scale, 2), "\n")
print(round(summary(m)$p.table, 4))

mu <- fitted(m)
zeros_pred <- mean(exp(-mu^(2 - p_hat) / (m$scale * (2 - p_hat))))
cat("part de zéros prédite par le modèle de Tweedie :", round(zeros_pred, 3), "| observée :",
    ↪ round(mean(clients$depense_annuelle == 0), 3), "\n")
```

```
      p log_vraisemblance
1 1.20      -12492.1
2 1.30      -12352.9
3 1.40      -12304.5
4 1.45      -12299.8
5 1.50      -12306.0
6 1.60      -12352.6
7 1.70      -12460.1
8 1.80      -12680.0
puissance p estimée : 1.447 | dispersion phi : 19.78
      Estimate Std. Error t value Pr(>|t|)
(Intercept)    5.4730    0.0877  62.4383  0.0000
age             0.0081    0.0021   3.9372  0.0001
canalRéseaux   -0.5553    0.0547 -10.1570  0.0000
canalSite      -0.1509    0.0542  -2.7858  0.0054
offre_bienvenue -0.0142    0.0435  -0.3260  0.7444
part de zéros prédite par le modèle de Tweedie : 0.153 | observée : 0.13
```

Étape 6 — Tweedie contre modèle à deux parties (logistique × Gamma) : moyennes par canal, calibration par dixième, part de zéros prédite.

```
f_rhs = "age + canal + offre_bienvenue"
clients["achete"] = (clients["depense_annuelle"] > 0).astype(int)
partiel = smf.glm("achete ~ " + f_rhs, clients, family=sm.families.Binomial()).fit()
acheteurs = clients[clients["achete"] == 1]
partie2 = smf.glm("depense_annuelle ~ " + f_rhs, acheteurs,
    ↪ family=sm.families.Gamma(sm.families.links.Log())).fit(scale="X2")
mu_deux = partiel.predict(clients) * partie2.predict(clients) # E[Y|x] = P(achète|x)
    ↪ × E[montant|achète, x]

tw = smf.glm("depense_annuelle ~ " + f_rhs, clients, family=sm.families.Tweedie(var_power=1.45,
    ↪ link=sm.families.links.Log())).fit(scale="X2")
mu_tw = tw.fittedvalues

print("coefficients de Tweedie (p = 1,45) :", tw.params.round(4).to_dict())
print()
tab = pd.DataFrame({"observée": clients.groupby("canal", observed=True)["depense_annuelle"].mean(),
    "Tweedie": mu_tw.groupby(clients["canal"], observed=True).mean(),
    "deux parties": mu_deux.groupby(clients["canal"], observed=True).mean()})
tab.loc["tous les clients"] = [clients["depense_annuelle"].mean(), mu_tw.mean(), mu_deux.mean()]
print(tab.round(1).to_string())
```

```
coefficients de Tweedie (p = 1,45) : {'Intercept': 5.4729, 'canal[T.Réseaux]': -0.5553,
    ↪ 'canal[T.Site]': -0.151, 'age': 0.0081, 'offre_bienvenue': -0.0142}
```

	observée	Tweedie	deux parties
canal			
Boutique	316.3	316.7	317.2
Réseaux	182.5	182.8	183.0
Site	272.9	272.3	271.7
tous les clients	247.0	247.0	247.0

Calibration : on compare enfin, par dixième de dépense prédite, les deux modèles à la dépense observée, et la part de zéros prédite.

```
# calibration par dixième de dépense prédite
classes = pd.qcut(mu_deux, 10, labels=False)
dec = pd.DataFrame({"observée": clients["depense_annuelle"], "Tweedie": mu_tw, "deux parties":
    ↪ mu_deux, "dixième": classes}).groupby("dixième").mean()
print()
print(dec.round(1).to_string())
print()
print("écart absolu moyen, par dixième : Tweedie =", round(float((dec["Tweedie"] -
    ↪ dec["observée"]).abs().mean()), 2),
      "| deux parties =", round(float((dec["deux parties"] - dec["observée"]).abs().mean()), 2))
print("part de zéros prédite par le modèle à deux parties :", round(float(1 -
    ↪ partiel.fittedvalues.mean()), 3))
```

	observée	Tweedie	deux parties
dixième			
0	170.6	163.0	162.4
1	173.0	175.8	175.7
2	199.4	186.9	187.4
3	192.2	202.6	203.7
4	205.4	244.2	243.5
5	268.8	266.7	266.1
6	316.1	280.3	280.0
7	280.8	294.8	294.7
8	338.7	313.1	313.3
9	331.8	345.3	345.9

écart absolu moyen, par dixième : Tweedie = 16.3 | deux parties = 16.49
 part de zéros prédite par le modèle à deux parties : 0.13

Application 2.9 — La vérité dévoilée

Section du livre : bilan du chapitre. **Objectif** : comparer, pour les trois modèles du chapitre (rachat, commandes, dépense), les estimations à la vérité programmée dans build/donnees2.py, en erreurs-types, puis expliquer l'écart apparent sur le nombre de commandes par canal.

```
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Réseaux",
    ↪ "Site"])

# (1) rachat : logistique (vérité sur l'échelle du logit, canal relatif à la boutique)
logi = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
    ↪ family=sm.families.Binomial()).fit()
# (2) nombre de commandes : binomiale négative (log de la moyenne)
nbm = smf.negativebinomial("nb_commandes_an ~ offre_bienvenue + age + canal", clients).fit(dis=0)
# (3) dépense moyenne de tous les clients : Tweedie p = 1,45 (log de la moyenne)
twd = smf.glm("depense_annuelle ~ offre_bienvenue + age + canal", clients,
    ↪ family=sm.families.Tweedie(var_power=1.45, link=sm.families.links.Log())).fit(scale="X2")
```

Les écarts à la vérité, estimation par estimation, pour les trois modèles :

```
verite = {
    "rachat (logit)": (logi, {"offre_bienvenue": 0.55, "age": -0.015, "canal[T.Réseaux]": -0.30,
    ↪ "canal[T.Site]": -0.30}),
    "commandes (log moyenne)": (nbm, {"offre_bienvenue": 0.0, "age": -0.005, "canal[T.Réseaux]":
    ↪ -0.05, "canal[T.Site]": 0.10}),
    "dépense (log moyenne)": (twd, {"offre_bienvenue": 0.0, "age": 0.003, "canal[T.Réseaux]": -0.39,
    ↪ "canal[T.Site]": -0.07}),
}
lignes = []
for nom, (res, vrai) in verite.items():
    for param, v in vrai.items():
```

```

    est, se = float(res.params[param]), float(res.bse[param])
    lignes.append({"modèle": nom, "paramètre": param, "estimation": est, "erreur-type": se,
↳ "vérité": v, "écart (en erreurs-types)": (est - v) / se})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
print()
print("alpha (binomiale négative) :", round(float(nbm.params["alpha"]), 3), "| IC95 % :",
↳ nbm.conf_int().loc["alpha"].round(3).tolist())

```

	modèle	paramètre	estimation	erreur-type	vérité	écart (en erreurs-types)
	rachat (logit)	offre_bienvenue	0.493	0.091	0.550	-0.624
	rachat (logit)	age	-0.015	0.004	-0.015	-0.114
	rachat (logit)	canal[T.Réseaux]	-0.463	0.116	-0.300	-1.411
	rachat (logit)	canal[T.Site]	-0.168	0.120	-0.300	1.106
commandes (log moyenne)	offre_bienvenue		-0.016	0.041	0.000	-0.379
commandes (log moyenne)	age		-0.001	0.002	-0.005	1.938
commandes (log moyenne)	canal[T.Réseaux]		-0.176	0.052	-0.050	-2.430
commandes (log moyenne)	canal[T.Site]		0.015	0.053	0.100	-1.597
dépense (log moyenne)	offre_bienvenue		-0.014	0.051	0.000	-0.277
dépense (log moyenne)	age		0.008	0.002	0.003	2.112
dépense (log moyenne)	canal[T.Réseaux]		-0.555	0.064	-0.390	-2.570
dépense (log moyenne)	canal[T.Site]		-0.151	0.064	-0.070	-1.270

```
alpha (binomiale négative) : 0.584 | IC95 % : [0.528, 0.639]
```

Un contrôle de plus : les moyennes par canal du nombre de commandes sont-elles conformes à la vérité programmée ?

```

# Écarts du modèle « commandes » : les moyennes par canal sont-elles conformes à la vérité programmée ?
var_f = 0.22 ** 2 + 0.10 ** 2 + 2 * 0.22 * 0.10 * 0.3 # variance de 0,22 F1 +
↳ 0,10 F2
eff_canal = {"Boutique": 0.05, "Réseaux": 0.0, "Site": 0.15}
lignes = []
for canal, e in eff_canal.items():
    g = clients[clients["canal"] == canal]["nb_commandes_an"]
    ages = clients.loc[clients["canal"] == canal, "age"]
    attendu = np.exp(1.25 + e + var_f / 2) * np.mean(np.exp(-0.005 * (ages - 36))) # E[N] sous la
↳ vérité programmée
    lignes.append({"canal": canal, "clients": len(g), "commandes attendues": attendu, "commandes
↳ observées": g.mean(),
                    "écart (en erreurs-types de la moyenne)": (g.mean() - attendu) / (g.std() /
↳ np.sqrt(len(g)))})
print()
print(pd.DataFrame(lignes).round(3).to_string(index=False))
print()
print("hétérogénéité attendue : (1 + 0,5) × (1 + variance relative de exp(0,22 F1 + 0,10 F2)) - 1 =",
↳ round((1 + 0.5) * np.exp(0.22 ** 2 + 0.10 ** 2 + 2 * 0.22 * 0.10 * 0.3) - 1, 3))

```

	canal	clients	commandes attendues	commandes observées	écart (en erreurs-types de la moyenne)
	Boutique	504	3.818	4.137	1.930
	Réseaux	816	3.620	3.464	-1.293
	Site	680	4.219	4.197	-0.154

```
hétérogénéité attendue : (1 + 0,5) × (1 + variance relative de exp(0,22 F1 + 0,10 F2)) - 1 = 0.611
```

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse ou démonstration.

Exercice 2.1 ★ — Cotes et logit (section 2.2.1–2.2.2 du livre)

- (a) Un client a une probabilité de rachat de 0,8 : quelle est sa cote, quel est son logit ? (b) Un modèle logistique donne $\text{logit}(p) = -0,4 + 0,9x$, où x est le nombre de commandes passées l'an dernier. Calculez p pour $x = 0, 1, 2$. (c) Quel est le rapport de cotes associé à une commande de plus ? De combien de points de pourcentage la probabilité augmente-t-elle de $x = 0$ à $x = 1$, puis de $x = 1$ à $x = 2$? Pourquoi ne sont-ils pas égaux ?

Exercice 2.2 ★ — Rapport de cotes, risque relatif (section 2.2.3 du livre)

Sur 200 clients abonnés à la newsletter, 60 ont acheté ; sur 300 clients non abonnés, 45 ont acheté. Calculez à la main la différence de risque, le risque relatif et le rapport de cotes, puis un intervalle de confiance à 95 % de ce dernier (erreur-type du log-OR : $\sqrt{1/a + 1/b + 1/c + 1/d}$). Vérifiez avec une régression logistique.

Exercice 2.3 ★ — Régression de Poisson (section 2.3.1 du livre)

Un modèle de Poisson pour le nombre de commandes annuelles donne $\log \mu = 1,2 - 0,01 \times \text{âge} + 0,2 \times \text{site}$ (la variable « site » vaut 1 pour un client acquis par le site, 0 pour la boutique). (a) Nombre moyen de commandes d'un client de 40 ans acquis par le site, et d'un client de 40 ans acquis en boutique. (b) Par quel facteur le nombre moyen est-il multiplié pour 10 ans de plus ? (c) Probabilité qu'un client de 40 ans acquis par le site ne passe **aucune** commande, si la loi de Poisson est correcte.

Exercice 2.4 ★★ — IRLS à la main (section 2.1.5 du livre)

Trois clients : $x = (0, 1, 2)$ et $y = (1, 3, 5)$ commandes. On ajuste un modèle de Poisson, $\log \mu = \beta_0 + \beta_1 x$. (a) À partir de $\beta^{(0)} = (0, 0)$, calculez à la main **une itération** d'IRLS (poids, réponse de travail, régression pondérée). (b) Écrivez le code complet de l'algorithme jusqu'à convergence et comparez à statsmodels.

Exercice 2.5 ★★ — Déviance et rapport de vraisemblance (section 2.4.1–2.4.2 du livre)

Avec les données de l'exercice 2.4 : (a) calculez à la main la déviance du modèle **sans variable** ($\hat{\mu} = \bar{y}$) et la statistique de Pearson. (b) Quelle est la déviance du modèle avec x ? Testez, par le rapport de vraisemblance, l'utilité de x (1 degré de liberté).

Exercice 2.6 ★★ — Décalage (section 2.3.2 du livre)

Trois transporteurs ont livré des colis et enregistré des retards : A a livré 200 milliers de colis pour 30 retards ; B, 50 milliers pour 12 retards ; C, 400 milliers pour 40 retards. (a) Calculez les taux de retard par millier de colis et les rapports de taux B/A et C/A. (b) Ajustez une régression de Poisson avec et sans décalage (*offset*) : que concluriez-vous dans chaque cas ?

Exercice 2.7 ★★ — Surdispersion (section 2.3.3 du livre)

Une régression de Poisson, sur $n = 305$ clients avec 5 paramètres, donne une statistique de Pearson $X^2 = 540$. Le coefficient de la variable « Réseaux » est 0,30 avec une erreur-type de 0,12. (a) Estimez la dispersion $\hat{\phi}$. (b) Corrigez l'erreur-type et la statistique z . La conclusion change-t-elle au seuil de 5 % ?

Exercice 2.8 ★★ — Régression sur $\log y$ (section 2.3.4 du livre)

On régresse le logarithme des dépenses sur des variables explicatives ; pour un client donné, le modèle prédit $\log \hat{y} = 5,0$ et l'écart-type résiduel est $\hat{\sigma} = 0,9$. (a) Que vaut $e^{5,0}$? Est-ce la dépense moyenne prévue ? (b) Si les résidus sont normaux, quelle est la dépense moyenne prévue ? Vérifiez par simulation.

Exercice 2.9 ★★ — Comparer des modèles (section 2.4.5 du livre)

Les déviations de trois modèles de rachat sont $D_{\text{complet}} = 2710,83$ (offre, âge, canal : 5 paramètres), $D_{\text{sans offre}} = 2740,47$ (4 paramètres) et $D_{\text{sans canal}} = 2728,57$ (3 paramètres), pour $n = 2000$. (a) Testez par le rapport de vraisemblance l'utilité de l'offre, puis du canal. (b) Calculez la différence d'AIC et de BIC dans chaque cas. Les critères s'accordent-ils avec les tests ?

Exercice 2.10 ★★★ — Sur les données : la ville (section 2.4.2 du livre)

Le fichier `clients.csv` contient la ville de chaque client. La ville améliore-t-elle le modèle de rachat offre + âge + canal ? Utilisez le test du rapport de vraisemblance, l'AIC et le BIC, et interprétez.

Exercice 2.11 ★★ — Choisir un seuil (section 2.2.8 du livre)

Avec le modèle de rachat enrichi des notes de l'enquête (section 2.2.7), trouvez le seuil qui maximise l'indice de Youden $J = \text{sensibilité} + \text{spécificité} - 1$, et comparez-le au seuil de 0,5.

Exercice 2.12 ★★ — GAM : choisir la souplesse (section 2.5.2 du livre)

Sur les sessions de navigation (`ch02-sessions.csv`), ajustez des régressions logistiques sur des B-splines de 4 à 12 fonctions de base et choisissez le nombre qui minimise l'AIC. Quel est l'avantage de la pénalisation sur cette recherche ?

Exercice 2.13 ★★ — Zéros en excès (section 2.6.1 du livre)

Dans un modèle ZIP de moyenne de la partie Poisson $\mu = 4$, quelle probabilité d'inflation π donne un rapport variance/moyenne égal à 2 ? Quelle est alors la probabilité d'observer un zéro ? Vérifiez par simulation.

Exercice 2.14 ★★★ — Démonstration et Tweedie (section 2.1.3, 2.6.3 du livre)

- (a) Montrez que la loi Gamma de moyenne μ et de forme ν appartient à la famille exponentielle, avec $\theta = -1/\mu$, $b(\theta) = -\log(-\theta)$ et $\phi = 1/\nu$, et retrouvez $\text{Var}(Y) = \mu^2/\nu$ avec la proposition de 2.1.3. (b) Pour une loi de Tweedie de moyenne $\mu = 100$, $\phi = 15$, $p = 1,3$, calculez la probabilité de zéro, puis les paramètres $(\lambda, \alpha, \theta)$ de la somme Poisson-Gamma correspondante, et vérifiez par simulation.

Corrigés

Corrigé 2.1

- (a) $Cote = 0,8/0,2 = 4$; $logit = \log 4 \approx 1,386$. (b) $p(0) = \text{expit}(-0,4) = 0,401$; $p(1) = \text{expit}(0,5) = 0,622$; $p(2) = \text{expit}(1,4) = 0,802$. (c) Le rapport de cotes est $e^{0,9} = 2,46$ pour une commande de plus. La probabilité augmente de 22,1 points puis de 18,0 points : l'effet est **constant sur le logit** mais pas sur la probabilité (il est maximal autour de $p = 0,5$ et diminue quand on s'en éloigne : 2.2.2).

```
print("cote =", 0.8 / 0.2, "| logit =", round(float(np.log(4)), 4))
p_x = expit(-0.4 + 0.9 * np.array([0, 1, 2]))
print("probabilités :", p_x.round(4), "| OR =", round(float(np.exp(0.9)), 4))
print("gains en points :", (100 * np.diff(p_x)).round(2))
```

```
cote = 4.0 | logit = 1.3863
probabilités : [0.4013 0.6225 0.8022] | OR = 2.4596
gains en points : [22.11 17.97]
```

Corrigé 2.2

$\hat{p}_1 = 60/200 = 0,30$ et $\hat{p}_0 = 45/300 = 0,15$. Différence de risque = 0,15 (15 points) ; risque relatif = $0,30/0,15 = 2,0$; rapport de cotes = $\frac{0,30/0,70}{0,15/0,85} = \frac{0,4286}{0,1765} = 2,43$. Erreur-type du log-OR : $\sqrt{\frac{1}{60} + \frac{1}{140} + \frac{1}{45} + \frac{1}{255}} = 0,2235$; intervalle de log OR : $0,887 \pm 1,96 \times 0,2235$, soit $[0,449; 1,325]$, donc OR dans $[1,57; 3,76]$. Le RR (2,0) est plus petit que l'OR (2,43) : l'OR exagère le risque relatif car l'événement n'est pas rare (15 % à 30 %).

```
a, b, c, d = 60, 140, 45, 255 # avec : achat / non-achat ; sans : achat /
↪ non-achat
p1, p0 = a / (a + b), c / (c + d)
or_ = (a * d) / (b * c)
se = np.sqrt(1 / a + 1 / b + 1 / c + 1 / d)
print(f"différence de risque = {p1 - p0:.3f} | RR = {p1 / p0:.3f} | OR = {or_:.4f}")
print(f"IC95 % de l'OR : [{np.exp(np.log(or_) - 1.96 * se):.3f} ; {np.exp(np.log(or_) + 1.96 *
↪ se):.3f}] (erreur-type du log-OR = {se:.4f})")
tab2 = pd.DataFrame({"newsletter": [1] * 200 + [0] * 300, "achat": [1] * 60 + [0] * 140 + [1] * 45 +
↪ [0] * 255})
m2 = smf.glm("achat ~ newsletter", tab2, family=sm.families.Binomial()).fit()
print("régression logistique : OR =", round(float(np.exp(m2.params["newsletter"])), 4), "| IC95 % :",
↪ np.exp(m2.conf_int().loc["newsletter"]).round(3).tolist())
```

```
différence de risque = 0.150 | RR = 2.000 | OR = 2.4286
IC95 % de l'OR : [1.567 ; 3.764] (erreur-type du log-OR = 0.2235)
régression logistique : OR = 2.4286 | IC95 % : [1.567, 3.764]
```

Corrigé 2.3

- (a) Site, 40 ans : $\mu = e^{1,2-0,4+0,2} = e^{1,0} = 2,718$ commandes ; boutique : $e^{0,8} = 2,226$. (b) Pour 10 ans de plus : $e^{-0,1} = 0,905$, soit 9,5 % de commandes en moins. Le site est associé à $e^{0,2} = 1,221$ fois plus de commandes que la boutique. (c) $P(Y = 0) = e^{-\mu} = e^{-2,718} = 0,066$: 6,6 % de zéros attendus (sous Poisson).

```
mu_site, mu_boutique = np.exp(1.2 - 0.01 * 40 + 0.2), np.exp(1.2 - 0.01 * 40)
print("moyennes (site, boutique) :", round(float(mu_site), 3), round(float(mu_boutique), 3))
print("facteur pour +10 ans :", round(float(np.exp(-0.1)), 4), "| facteur site/boutique :",
↪ round(float(np.exp(0.2)), 4))
```

```
print("P(Y = 0 | site, 40 ans) =", round(float(stats.poisson.pmf(0, mu_site)), 4))
```

```
moyennes (site, boutique) : 2.718 2.226
facteur pour +10 ans : 0.9048 | facteur site/boutique : 1.2214
P(Y = 0 | site, 40 ans) = 0.066
```

Corrigé 2.4

- (a) Avec $\beta^{(0)} = (0, 0) : \eta = 0, \mu = e^0 = 1$ pour tous, donc $W_i = \mu_i = 1$ (pour Poisson avec lien log, $W = \mu$) et $z_i = \eta_i + (y_i - \mu_i)/\mu_i = (0, 2, 4)$. La régression de z sur x (poids tous égaux à 1) a pour pente $\frac{\sum (x-1)(z-2)}{\sum (x-1)^2} = \frac{(-1)(-2)+0+(1)(2)}{2} = 2$ et pour ordonnée $\bar{z} - 2\bar{x} = 2 - 2 = 0$: donc $\beta^{(1)} = (0; 2)$. (b) La deuxième itération, avec $\mu = (1; e^2; e^4)$, donne déjà des valeurs bien plus raisonnables ; l'algorithme converge en quelques pas (voir le code).

```
x4 = np.array([0.0, 1, 2]); y4 = np.array([1.0, 3, 5])
X4 = np.column_stack([np.ones(3), x4])
beta = np.zeros(2)
for it in range(1, 9):
    eta = X4 @ beta
    mu = np.exp(eta)
    w = mu                                # poids de Poisson (lien log)
    z = eta + (y4 - mu) / mu              # réponse de travail
    beta = np.linalg.solve(X4.T @ (w[:, None] * X4), X4.T @ (w * z))
    if it <= 4:
        print(f"itération {it} : beta = {beta.round(5)}")
m4 = sm.GLM(y4, X4, family=sm.families.Poisson()).fit()
print("après 8 itérations :", beta.round(6), "| statsmodels :", m4.params.round(6))
```

```
itération 1 : beta = [0. 2.]
itération 2 : beta = [-0.17925  1.63377]
itération 3 : beta = [0.00434  1.15568]
itération 4 : beta = [0.1661  0.82978]
après 8 itérations : [0.207929 0.723349] | statsmodels : [0.207929 0.723349]
```

Corrigé 2.5

- (a) Sans variable, $\hat{\mu} = 3 : D = 2[1 \log \frac{1}{3} + 3 \log 1 + 5 \log \frac{5}{3} - (9 - 9)] = 2[-1,0986 + 0 + 2,5541] = 2,911$; Pearson $= \frac{(1-3)^2+0+(5-3)^2}{3} = 2,667$. (b) La déviance du modèle avec x , calculée ci-dessous, est beaucoup plus petite : la différence $D_0 - D_1$ se compare à χ_1^2 .

```
mu0 = np.full(3, y4.mean())
d0 = 2 * np.sum(y4 * np.log(y4 / mu0) - (y4 - mu0))
print("déviante sans variable :", round(float(d0), 4), "| Pearson :", round(float(np.sum((y4 - mu0)
    ↪ ** 2 / mu0)), 4))
m0 = sm.GLM(y4, np.ones((3, 1)), family=sm.families.Poisson()).fit()
print("statsmodels : déviante nulle =", round(float(m0.deviance), 4), "| déviante avec x =",
    ↪ round(float(m4.deviance), 4))
delta = m0.deviance - m4.deviance
print(f"rapport de vraisemblance : {delta:.4f} sur 1 ddl, p = {stats.chi2.sf(delta, 1):.4f}")
```

```
déviante sans variable : 2.911 | Pearson : 2.6667
statsmodels : déviante nulle = 2.911 | déviante avec x = 0.1363
rapport de vraisemblance : 2.7748 sur 1 ddl, p = 0.0958
```

La déviance du modèle avec x est de 0,136 : il épouse presque parfaitement les trois points (la pente estimée est 0,7233, soit un facteur $e^{0,7233} = 2,06$ par unité de x). La différence $2,911 - 0,136 = 2,77$ donne pourtant $p = 0,096$: au seuil de 5 %, on **ne rejette pas** l'absence d'effet de x . Avec

seulement 3 observations, le test a très peu de puissance (et l'approximation par un χ^2 est de toute façon douteuse pour un si petit échantillon) : une pente énorme n'est pas « significative » faute de données.

Corrigé 2.6

- (a) Taux par millier de colis : $A = 30/200 = 0,15$; $B = 12/50 = 0,24$; $C = 40/400 = 0,10$. Rapports de taux : $B/A = 1,6$, $C/A = 0,667$: B est le transporteur le plus mauvais, C le meilleur. (b) Avec le décalage $\log(\text{exposition})$, le modèle (saturé : 3 paramètres pour 3 transporteurs) reproduit exactement ces rapports. **Sans** décalage, il compare des nombres bruts de retards (30, 12, 40) : il conclut que B a $12/30 = 0,4$ fois les retards de A et C 1,33 fois : un renversement complet, parce que C livre beaucoup plus de colis.

```
transp = pd.DataFrame({"transporteur": ["A", "B", "C"], "milliers": [200, 50, 400], "retards": [30,
↪ 12, 40]})
transp["taux"] = transp["retards"] / transp["milliers"]
print(transp.to_string(index=False))
avec = smf.glm("retards ~ transporteur", transp, family=sm.families.Poisson(),
↪ offset=np.log(transp["milliers"])).fit()
sans = smf.glm("retards ~ transporteur", transp, family=sm.families.Poisson()).fit()
print("rapports de taux avec décalage (B/A, C/A) :", np.exp(avec.params[["transporteur[T.B]",
↪ "transporteur[T.C]"]]).round(3).tolist())
print("rapports sans décalage (B/A, C/A)      :", np.exp(sans.params[["transporteur[T.B]",
↪ "transporteur[T.C]"]]).round(3).tolist())
```

transporteur	milliers	retards	taux
A	200	30	0.15
B	50	12	0.24
C	400	40	0.10

rapports de taux avec décalage (B/A, C/A) : [1.6, 0.667]
 rapports sans décalage (B/A, C/A) : [0.4, 1.333]

Corrigé 2.7

- (a) $\hat{\phi} = X^2/(n-p) = 540/(305-5) = 1,8$. (b) L'erreur-type corrigée est $0,12 \times \sqrt{1,8} = 0,161$, et $z = 0,30/0,161 = 1,86$ au lieu de 2,5. La p-valeur passe de 0,012 à 0,063 : l'effet est **significatif** à 5 % avec la loi de Poisson, mais **ne l'est plus** après correction. C'est exactement le danger de la surdispersion non corrigée.

```
phi = 540 / (305 - 5)
se_c = 0.12 * np.sqrt(phi)
z_nc, z_c = 0.30 / 0.12, 0.30 / se_c
print(f"phi = {phi:.2f} | erreur-type corrigée = {se_c:.4f}")
print(f"z non corrigé = {z_nc:.2f} (p = {2 * stats.norm.sf(z_nc):.4f}) | z corrigé = {z_c:.2f} (p = {2
↪ * stats.norm.sf(z_c):.4f})")
```

phi = 1.80 | erreur-type corrigée = 0.1610
 z non corrigé = 2.50 (p = 0.0124) | z corrigé = 1.86 (p = 0.0624)

Corrigé 2.8

- (a) $e^{5,0} = 148,4$ € : c'est la **médiane** prévue (si les résidus de $\log y$ sont symétriques), pas la moyenne. (b) Pour $\log Y \sim \mathcal{N}(5; 0,9^2)$, $E[Y] = e^{5+\sigma^2/2} = e^{5+0,405} = 148,4 \times 1,499 = 222,5$ € : la moyenne est **50 % plus grande** que e^5 . Ne pas retransformer sans correction revient à sous-estimer systématiquement la dépense moyenne (2.3.4).

```
rng = np.random.default_rng(8)
y_sim = rng.lognormal(mean=5.0, sigma=0.9, size=1_000_000)
print("exp(5,0) =", round(float(np.exp(5.0)), 1), "| médiane simulée :",
      ↪ round(float(np.median(y_sim)), 1))
print("exp(5 + sigma^2/2) =", round(float(np.exp(5.0 + 0.9 ** 2 / 2)), 1), "| moyenne simulée :",
      ↪ round(float(y_sim.mean()), 1))
```

```
exp(5,0) = 148.4 | médiane simulée : 148.6
exp(5 + sigma^2/2) = 222.5 | moyenne simulée : 222.6
```

Corrigé 2.9

- (a) Offre : $\Delta D = 2740,47 - 2710,83 = 29,64$ sur 1 ddl, $p = 5,2 \times 10^{-8}$; canal : $\Delta D = 2728,57 - 2710,83 = 17,74$ sur 2 ddl, $p = 1,4 \times 10^{-4}$. Les deux variables sont nécessaires. (b) $\Delta AIC = \Delta D - 2q$ et $\Delta BIC = \Delta D - q \log n$ (avec $\log 2000 = 7,60$) : pour l'offre, $\Delta AIC = 27,64$ et $\Delta BIC = 22,04$; pour le canal, $\Delta AIC = 13,74$ et $\Delta BIC = 2,54$. Tous les écarts sont positifs : les critères gardent chaque variable, en accord avec les tests ; le BIC est beaucoup moins enthousiaste pour le canal (+2,5 seulement), qui est le cas le plus marginal.

```
n = 2000
for nom, d_red, d_comp, q in [("offre", 2740.47, 2710.83, 1), ("canal", 2728.57, 2710.83, 2)]:
    dd = d_red - d_comp
    print(f"{nom:6s}: ΔD = {dd:6.2f} sur {q} ddl | p = {stats.chi2.sf(dd, q):.2e} | ΔAIC = {dd - 2 *
          ↪ q:6.2f} | ΔBIC = {dd - q * np.log(n):6.2f}")
```

```
offre : ΔD = 29.64 sur 1 ddl | p = 5.20e-08 | ΔAIC = 27.64 | ΔBIC = 22.04
canal : ΔD = 17.74 sur 2 ddl | p = 1.41e-04 | ΔAIC = 13.74 | ΔBIC = 2.54
```

Corrigé 2.10

Ajoutons la ville (6 modalités, donc 5 paramètres de plus) au modèle de rachat et testons.

```
clients = pd.read_csv("donnees/clients.csv")
clients["canal"] = pd.Categorical(clients["canal_acquisition"], categories=["Boutique", "Réseaux",
      ↪ "Site"])
base = smf.glm("rachat_12m ~ offre_bienvenue + age + canal", clients,
      ↪ family=sm.families.Binomial()).fit()
avec_ville = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + C(ville)", clients,
      ↪ family=sm.families.Binomial()).fit()
dd = base.deviance - avec_ville.deviance
print(f"ΔD = {dd:.2f} sur 5 ddl | p (rapport de vraisemblance) = {stats.chi2.sf(dd, 5):.3f}")
print(f"AIC : sans ville = {base.aic:.1f} | avec ville = {avec_ville.aic:.1f} | BIC : sans =
      ↪ {base.bic_1lf:.1f} | avec = {avec_ville.bic_1lf:.1f}")
ic = avec_ville.conf_int()
print(pd.DataFrame({"OR": np.exp(avec_ville.params), "IC95 bas": np.exp(ic[0]), "IC95 haut":
      ↪ np.exp(ic[1])}).filter(like="ville", axis=0).round(3).to_string())
```

```
ΔD = 1.13 sur 5 ddl | p (rapport de vraisemblance) = 0.952
AIC : sans ville = 2720.8 | avec ville = 2729.7 | BIC : sans = 2748.8 | avec = 2785.7
      OR IC95 bas IC95 haut
C(ville)[T.Ville A] 0.985 0.687 1.411
C(ville)[T.Ville B] 1.002 0.713 1.407
```

C(ville)[T.Ville C]	0.901	0.648	1.253
C(ville)[T.Ville D]	1.062	0.774	1.457
C(ville)[T.Ville E]	1.023	0.773	1.354

La ville n'améliore pas le modèle : $\Delta D = 1,13$ pour 5 degrés de liberté ($p = 0,95$, un résultat parfaitement banal si la ville n'a aucun effet). L'AIC **se dégrade** (de 2 720,8 à 2 729,7) et le BIC encore davantage (de 2 748,8 à 2 785,7) : les cinq paramètres de plus ne rapportent presque rien en vraisemblance. Les rapports de cotes de toutes les villes par rapport à la ville de référence (Autre) sont compris entre 0,90 et 1,06, avec des intervalles de confiance qui contiennent tous 1. **Conclusion : la ville n'a pas d'effet détectable sur le rachat**, et l'on garde le modèle sans elle. (Nous verrons plus bas que, dans la simulation, la ville n'a effectivement aucun rôle.)

Corrigé 2.11

On balaye les seuils de 0,05 à 0,95 et l'on garde celui qui maximise J .

```

enquete = pd.read_csv("donnees/enquete_satisfaction.csv")
enquete["note_produits"] = enquete[["q1", "q2", "q3", "q4"]].mean(axis=1)
enquete["note_service"] = enquete[["q5", "q6", "q7", "q8"]].mean(axis=1)
rep = clients.merge(enquete[["id_client", "note_produits", "note_service"]], on="id_client")
riche = smf.glm("rachat_12m ~ offre_bienvenue + age + canal + note_produits + note_service", rep,
               family=sm.families.Binomial()).fit()
yr, sr = rep["rachat_12m"].to_numpy(), riche.fittedvalues.to_numpy()

seuils = np.arange(0.05, 0.96, 0.01)
sens = np.array([(sr >= s) & (yr == 1)].sum() / (yr == 1).sum() for s in seuils])
spec = np.array([(sr < s) & (yr == 0)].sum() / (yr == 0).sum() for s in seuils])
J = sens + spec - 1
k = int(np.argmax(J))
print(f"seuil optimal (Youden) = {seuils[k]:.2f} | sensibilité = {sens[k]:.3f} | spécificité = {spec[k]:.3f} | J = {J[k]:.3f}")
k5 = int(np.argmin(np.abs(seuils - 0.5)))
print(f"seuil 0,50 : sensibilité = {sens[k5]:.3f} | spécificité = {spec[k5]:.3f} | J = {J[k5]:.3f}")
print("part de rachat dans l'échantillon :", round(float(yr.mean()), 3))

seuil optimal (Youden) = 0.57 | sensibilité = 0.526 | spécificité = 0.772 | J = 0.298
seuil 0,50 : sensibilité = 0.663 | spécificité = 0.620 | J = 0.283
part de rachat dans l'échantillon : 0.505

```

Le seuil de Youden est de **0,57** : sensibilité 0,526, spécificité 0,772, $J = 0,298$. Au seuil de 0,5, $J = 0,283$. Le gain est minuscule : la surface de J est très plate autour de son maximum. Gardez deux idées en tête. (1) Un seuil « optimal » choisi sur les données qui ont servi à l'ajuster est un peu optimiste (2.2.8). (2) L'indice de Youden donne le même poids aux deux types d'erreur ; dans la pratique, le bon seuil dépend de leurs **coûts** (ici : relancer inutilement un client coûte peu, laisser partir un client précieux coûte cher).

Corrigé 2.12

On ajuste une spline pour chaque df et l'on compare les AIC.

```

sessions = pd.read_csv("donnees/ch02-sessions.csv")
lignes = []
for k in range(4, 13):
    r = smf.glm(f"achat ~ bs(duree_min, df={k}, lower_bound=0, upper_bound=40)", sessions,
               family=sm.families.Binomial()).fit()
    lignes.append({"fonctions de base": k, "paramètres": len(r.params), "AIC": r.aic})
res = pd.DataFrame(lignes)
res["ΔAIC"] = res["AIC"] - res["AIC"].min()

```

```
print(res.round(1).to_string(index=False))
print("nombre de fonctions de base retenu :", int(res.loc[res["AIC"].idxmin(), "fonctions de base"]))
```

fonctions de base	paramètres	AIC	ΔAIC
4	5	1722.1	18.7
5	6	1704.2	0.8
6	7	1703.5	0.0
7	8	1703.6	0.1
8	9	1703.9	0.4
9	10	1705.7	2.2
10	11	1707.4	4.0
11	12	1709.1	5.7
12	13	1707.7	4.2

nombre de fonctions de base retenu : 6

L'AIC est minimal pour 6 fonctions de base (1 703,5), mais les valeurs pour 5, 7 et 8 fonctions n'en sont qu'à 0,8, 0,1 et 0,4 point : un **plateau**, pas un minimum net. À 4 fonctions la courbe est trop rigide (+18,7) ; à partir de 9 fonctions l'AIC remonte de 2 à 6 points. L'avantage de la **pénalisation** est précisément d'éviter cette recherche à tâtons : on prend une base large et c'est le paramètre de lissage λ , choisi automatiquement, qui règle la souplesse (nous avons obtenu 6,5 degrés de liberté effectifs, au milieu de ce plateau).

Corrigé 2.13

Le rapport variance/moyenne d'un ZIP vaut $1 + \pi\mu$. Avec $\mu = 4 : 1 + 4\pi = 2$, donc $\pi = 0,25$. La probabilité d'un zéro est $\pi + (1 - \pi)e^{-\mu} = 0,25 + 0,75e^{-4} = 0,25 + 0,75 \times 0,0183 = 0,2637$.

```
rng = np.random.default_rng(13)
N = 1_000_000
y_zip = np.where(rng.random(N) < 0.25, 0, rng.poisson(4, N))
print(f"variance / moyenne simulée = {y_zip.var() / y_zip.mean():.3f} (attendu 2) | P(0) simulée = {np.mean(y_zip == 0):.4f} | formule = {0.25 + 0.75 * np.exp(-4):.4f}")
```

variance / moyenne simulée = 2.001 (attendu 2) | P(0) simulée = 0.2641 | formule = 0.2637

Corrigé 2.14

- (a) La densité de la loi Gamma de moyenne μ et de forme ν est $f(y) = \frac{1}{\Gamma(\nu)} \left(\frac{\nu}{\mu}\right)^\nu y^{\nu-1} e^{-\nu y/\mu}$, donc $\log f = \nu \left(-\frac{y}{\mu} - \log \mu\right) + (\nu - 1) \log y + \nu \log \nu - \log \Gamma(\nu)$. Avec $\theta = -1/\mu$, $\phi = 1/\nu$ et $b(\theta) = -\log(-\theta) = \log \mu$, on obtient $\frac{y\theta - b(\theta)}{\phi} = \nu \left(-\frac{y}{\mu} - \log \mu\right)$: le reste ne dépend pas de θ , c'est $c(y, \phi)$. Alors $b'(\theta) = -1/\theta = \mu$ et $b''(\theta) = 1/\theta^2 = \mu^2$, d'où $\text{Var}(Y) = \phi b''(\theta) = \mu^2/\nu$: le coefficient de variation $1/\sqrt{\nu}$ est constant. (b) $P(Y = 0) = \exp(-\mu^{2-p}/(\phi(2-p)))$ avec $\mu^{0,7} = 100^{0,7} = 25,12$, donc $\lambda = 25,12/(15 \times 0,7) = 2,392$ et $P(Y = 0) = e^{-2,392} = 0,091$. Puis $\alpha = \frac{2-p}{p-1} = \frac{0,7}{0,3} = 2,333$ et $\theta = \phi(p-1)\mu^{p-1} = 15 \times 0,3 \times 100^{0,3} = 4,5 \times 3,981 = 17,91$; on vérifie $\lambda\alpha\theta = 2,392 \times 2,333 \times 17,91 \approx 100$.

```
mu_t, phi_t, p_t = 100.0, 15.0, 1.3
lam = mu_t ** (2 - p_t) / (phi_t * (2 - p_t)); alpha = (2 - p_t) / (p_t - 1); theta = phi_t * (p_t - 1) * mu_t ** (p_t - 1)
print(f"lambda = {lam:.3f} | alpha = {alpha:.3f} | theta = {theta:.3f} | lambda*alpha*theta = {lam * alpha * theta:.2f} | P(0) = exp(-lambda) = {np.exp(-lam):.4f}")
rng = np.random.default_rng(14)
N = 500_000
n_cmd = rng.poisson(lam, N)
y_t = np.where(n_cmd > 0, rng.gamma(np.maximum(n_cmd, 1) * alpha, theta), 0.0)
```

```
print(f"simulation : P(0) = {np.mean(y_t == 0):.4f} | moyenne = {y_t.mean():.2f} | variance =  
↪ {y_t.var():.1f} | phi * mu^p = {phi_t * mu_t ** p_t:.1f}")
```

```
lambda = 2.392 | alpha = 2.333 | theta = 17.915 | lambda*alpha*theta = 100.00 | P(0) = exp(-lambda) =  
↪ 0.0914
```

```
simulation : P(0) = 0.0918 | moyenne = 99.82 | variance = 5962.3 | phi * mu^p = 5971.6
```

La simulation confirme le calcul : $P(Y = 0) = 0,0918$ simulée pour 0,0914 théorique, moyenne 99,82 pour 100, variance 5 962 pour $\phi\mu^p = 5 972$. Les paramètres $(\lambda, \alpha, \theta) = (2,392; 2,333; 17,91)$ sont donc cohérents avec la loi de Tweedie annoncée.

Chapitre 3 : Analyse multivariée — exercices et applications

Ce chapitre du cahier accompagne le chapitre 3 du livre (ACP, analyse factorielle, classification, et les sections optionnelles sur les correspondances et l'analyse discriminante). Les **applications** reprennent avec le code complet les études que le livre résume ; les **exercices** (14, de ★ à ★★★) sont corrigés à la fin. Données utilisées : `enquete_satisfaction.csv` et `clients.csv` du dossier `donnees/`, entièrement simulées. Chaque application et chaque corrigé recharge ses données : on peut les traiter dans n'importe quel ordre.

Applications

Les applications reprennent, pas à pas et avec le code complet, les études que le livre ne fait que résumer. Chacune est autonome : elle recharge ses données. Les fonctions écrites dans une application peuvent être réutilisées plus loin dans le cahier.

Application 3.1 — Un indice de satisfaction pour chaque répondante

Contexte. La gérante voulait « un ou deux chiffres par cliente » à la place des huit notes du questionnaire. Les deux premières composantes d'une ACP sont ces chiffres. Reste à vérifier qu'elles disent quelque chose du comportement d'achat. *Voir section 3.1 du livre.*

Étape 1 : l'ACP sur les huit notes standardisées. Nous imposons une convention de signe (la plus grande saturation de chaque axe est positive), pour que « score élevé » signifie « cliente plus satisfaite ».

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA

q = pd.read_csv("donnees/enquete_satisfaction.csv")
c = pd.read_csv("donnees/clients.csv")
items = [f"q{j}" for j in range(1, 9)]
Z = (q[items] - q[items].mean()) / q[items].std()

pca = PCA(n_components=2).fit(Z)
signe = np.sign(pca.components_[np.arange(2), np.abs(pca.components_).argmax(axis=1)])
V = pca.components_.T * signe # directions, signe imposé
scores = pd.DataFrame(Z.to_numpy() @ V, columns=["CP1", "CP2"])
print("parts de variance :", pca.explained_variance_ratio_.round(3))
```

```
parts de variance : [0.35  0.237]
```

Étape 2 : lire les axes. Pour des variables standardisées, la corrélation d'une question avec une composante vaut $v_{ij}\sqrt{\lambda_j}$.

```
charges = pd.DataFrame(V * np.sqrt(pca.explained_variance_), index=items, columns=["CP1", "CP2"])
print(charges.round(2).T.to_string())
```

```
      q1  q2  q3  q4  q5  q6  q7  q8
CP1  0.56 0.52 0.57 0.48 0.68 0.64 0.64 0.61
CP2  0.58 0.54 0.51 0.47 -0.45 -0.39 -0.48 -0.44
```

Étape 3 : relier les scores aux comportements. On joint les scores au fichier des clientes et on compare les scores moyens selon que la cliente a racheté ou non.

```
scores["id_client"] = q["id_client"].to_numpy()
fusion = scores.merge(c[["id_client", "rachat_12m", "duree_mois", "panier_moyen"]], on="id_client")
print(fusion.groupby("rachat_12m")[["CP1", "CP2"]].mean().round(2).to_string())
print()
print(fusion[["CP1", "CP2", "duree_mois", "panier_moyen"]].corr().round(2).loc[["CP1",
↪ "CP2"]].to_string())
```

```
      CP1  CP2
rachat_12m
0      -0.47 -0.11
1       0.46  0.10

      CP1  CP2  duree_mois  panier_moyen
CP1  1.0 -0.0         0.14         0.21
CP2 -0.0  1.0        -0.09         0.13
```

Lecture. La première composante est une **satisfaction générale** : toutes les questions y ont le même signe. Son score moyen est nettement plus élevé chez les clientes qui rachètent. La seconde oppose produits et service ; elle sépare à peine les rachats (écart de l'ordre de 0,2 pour un écart-type de 1,4) mais montre des corrélations de signes contraires avec le panier (faiblement positive) et la durée de la relation (faiblement négative). Les deux dimensions semblent donc avoir des rôles distincts, ce que l'analyse factorielle (application 3.2) permettra de mieux démêler.

Pour aller plus loin. Refaites l'étape 3 avec les parts « produits » et « service » calculées comme moyennes de q1–q4 et de q5–q8 : retrouvez-vous les mêmes tendances qu'avec CP1 et CP2 ?

Application 3.2 — Deux échelles de mesure et leur lien avec le comportement

Contexte. Le but pratique d'une analyse factorielle de questionnaire est de construire des **échelles** : un score « produits » et un score « service », moyennes des questions de chaque groupe. Il faut ensuite vérifier qu'elles sont cohérentes (alpha de Cronbach) puis voir à quoi elles servent. Voir section 3.2 du livre.

Étape 1 : l'alpha de Cronbach. Pour k questions, $\alpha = \frac{k}{k-1} \left(1 - \frac{\sum \text{Var}(x_i)}{\text{Var}(\sum x_i)}\right)$.

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
c = pd.read_csv("donnees/clients.csv")

def alpha_cronbach(D):
    D = np.asarray(D, dtype=float)
    k = D.shape[1]
    return k / (k - 1) * (1 - D.var(axis=0, ddof=1).sum() / D.sum(axis=1).var(ddof=1))

produits, service = q[["q1", "q2", "q3", "q4"]], q[["q5", "q6", "q7", "q8"]]
print("alpha, échelle produits :", round(alpha_cronbach(produits), 3))
```

```
print("alpha, échelle service :", round(alpha_cronbach(service), 3))
print("alpha des 8 questions :", round(alpha_cronbach(q[[f"q{j}" for j in range(1, 9)]]), 3))
```

```
alpha, échelle produits : 0.741
alpha, échelle service : 0.784
alpha des 8 questions : 0.732
```

Étape 2 : construire les deux scores et mesurer leur corrélation.

```
scores = pd.DataFrame({"id_client": q["id_client"], "score_produits": produits.mean(axis=1),
                      "score_service": service.mean(axis=1)})
print("corrélation entre les deux échelles :",
      ↪ round(scores["score_produits"].corr(scores["score_service"]), 3))
```

```
corrélation entre les deux échelles : 0.19
```

Étape 3 : relier les échelles au comportement des clientes qui ont commandé.

```
f = scores.merge(c[["id_client", "rachat_12m", "panier_moyen", "duree_mois", "nb_commandes_an",
                  ↪ "age"]], on="id_client")
f = f[f["panier_moyen"] > 0]
cible = ["panier_moyen", "duree_mois", "nb_commandes_an", "age"]
print(f[["score_produits", "score_service"] + cible].corr().round(2).loc[["score_produits",
                                  ↪ "score_service"], cible].to_string())
print()
print(f.groupby("rachat_12m")[["score_produits", "score_service"]].mean().round(2).to_string())
```

	panier_moyen	duree_mois	nb_commandes_an	age
score_produits	0.24	0.04	0.20	0.03
score_service	0.09	0.16	0.16	-0.00

	score_produits	score_service
rachat_12m		
0	3.46	3.41
1	3.80	3.68

Lecture. Les deux échelles sont cohérentes (alpha de 0,74 et 0,78, au-dessus du repère de 0,7). Leur corrélation (0,19) est plus faible que celle des facteurs sous-jacents (0,30) : c'est l'**atténuation par l'erreur de mesure**. Le score « produits » est le mieux corrélé au panier moyen, le score « service » à la durée de la relation ; les clientes qui rachètent sont plus satisfaites sur les deux plans. Ces corrélations sont modestes (entre 0,1 et 0,25) et ne prouvent aucune causalité.

Pour aller plus loin. L'alpha augmente avec le nombre de questions : calculez l'alpha de l'échelle « produits » en retirant successivement chaque question. Laquelle est la moins utile ?

Application 3.3 — Segmenter les clientes : k-means écrit à la main, choix de k, stabilité

Contexte. La gérante voudrait savoir si ses clientes forment des « familles ». Nous écrivons l'algorithme des k-means, nous vérifions qu'il retrouve des groupes qui existent vraiment, puis nous le lançons sur les vraies clientes en nous demandant honnêtement si leurs groupes existent. Voir section 3.3 du livre.

Étape 1 : l'algorithme complet (Lloyd, initialisation k-means++, plusieurs redémarrages).

```
def kmeans(X, k, rng, n_init=10, max_iter=100):
    meilleur = None
    for _ in range(n_init):
        centres = [X[rng.integers(len(X))]]
        for _ in range(k - 1):
            d2 = ((X[:, None, :] - np.array(centres)[None, :, :]) ** 2).sum(axis=2).min(axis=1)
```

```

centres.append(X[rng.choice(len(X), p=d2 / d2.sum())])
centres = np.array(centres)
for iteration in range(max_iter):
    d = ((X[:, None, :] - centres[None, :, :]) ** 2).sum(axis=2)
    groupes = d.argmin(axis=1)
    nouveaux = np.array([X[groupe].mean(axis=0) if (groupe == j).any() else centres[j]
                          for j in range(k)])
    if np.allclose(nouveaux, centres):
        break
    centres = nouveaux
    W = ((X - centres[groupe]) ** 2).sum()
    if meilleur is None or W < meilleur["W"]:
        meilleur = {"W": W, "groupes": groupes, "centres": centres}
return meilleur

```

Étape 2 : un jeu de données avec de vrais groupes. Trois profils simulés (occasionnelles, fidèles, cadeaux), décrits par le nombre de commandes et le panier.

```

from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, silhouette_score

rng = np.random.default_rng(11)
profils = {"occasionnelles": (300, (1.5, 35), (0.7, 8)), "fidèles": (200, (6.0, 50), (1.4, 10)),
          "cadeaux": (100, (3.0, 110), (0.9, 15))}
blocs, vrais = [], []
for numero, (nom, (n_p, moyennes, ecarts)) in enumerate(profils.items()):
    blocs.append(rng.normal(moyennes, ecarts, size=(n_p, 2)))
    vrais += [numero] * n_p
sim = np.vstack(blocs)
sim[:, 0] = np.clip(sim[:, 0], 0, None)
vrais = np.array(vrais)
Zs = (sim - sim.mean(axis=0)) / sim.std(axis=0)

resultat = kmeans(Zs, 3, np.random.default_rng(0))
km = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Zs)
print("inertie : mon k-means =", round(resultat["W"], 1), "| scikit-learn =", round(km.inertia_, 1))
print("indice de Rand ajusté contre les vrais profils :", round(adjusted_rand_score(vrais,
↵ resultat["groupes"]), 3))

```

```

inertie : mon k-means = 188.1 | scikit-learn = 188.1
indice de Rand ajusté contre les vrais profils : 0.941

```

Étape 3 : choisir k. Pour chaque k , la variance intra-groupe W (le coude) et la silhouette moyenne.

```

lignes = []
for k in range(1, 8):
    r = kmeans(Zs, k, np.random.default_rng(0))
    sil = silhouette_score(Zs, r["groupes"]) if k > 1 else np.nan
    lignes.append({"k": k, "W": r["W"], "silhouette moyenne": sil})
print(pd.DataFrame(lignes).set_index("k").round(3).to_string())

```

	W	silhouette moyenne
k		
1	1200.000	NaN
2	607.380	0.533
3	188.103	0.676
4	140.201	0.582
5	123.191	0.402
6	108.734	0.387
7	97.185	0.396

Étape 4 : les vraies clientes. Nous retenons les clientes ayant commandé, décrites par quatre variables standardisées.

```

c = pd.read_csv("donnees/clients.csv")
act = c[c["nb_commandes_an"] > 0].copy()
var = ["nb_commandes_an", "panier_moyen", "duree_mois", "age"]
Zc = ((act[var] - act[var].mean()) / act[var].std()).to_numpy()

lignes = []
for k in range(1, 9):
    r = kmeans(Zc, k, np.random.default_rng(0), n_init=5)
    sil = silhouette_score(Zc, r["groupes"]) if k > 1 else np.nan
    lignes.append({"k": k, "W": r["W"], "silhouette_moyenne": sil})
print("clients retenues :", len(act), "sur", len(c))
print(pd.DataFrame(lignes).set_index("k").round(3).to_string())

```

```

clients retenues : 1740 sur 2000
      W  silhouette moyenne
k
1  6956.000             NaN
2  5526.143             0.224
3  4607.880             0.228
4  3812.099             0.242
5  3243.262             0.246
6  3029.588             0.232
7  2826.824             0.223
8  2658.348             0.201

```

Étape 5 : décrire quatre segments, puis tester leur stabilité.

```

r4 = kmeans(Zc, 4, np.random.default_rng(0), n_init=10)
act["segment"] = r4["groupes"]
profil = act.groupby("segment").agg(effectif=("id_client", "size"), commandes=("nb_commandes_an",
    ↪ "mean"),
                                   panier=("panier_moyen", "mean"), duree=("duree_mois", "mean"),
                                   age=("age", "mean")).round(1)
print(profil.sort_values("panier").to_string())

```

```

      effectif  commandes  panier  duree  age
segment
3           695         3.2   46.8   16.8  28.8
1           282         4.0   63.0   52.9  36.6
2           237        11.2   67.9   20.8  34.3
0           526         3.4   76.4   17.5  45.4

```

```

def stabilite(X, k, ref, rng, B=30):
    ari = []
    for _ in range(B):
        idx = rng.integers(0, len(X), len(X))
        r = kmeans(X[idx], k, rng, n_init=3)
        d = ((X[:, None, :] - r["centres"][None, :, :]) ** 2).sum(axis=2)
        ↪ clients
        ari.append(adjusted_rand_score(ref, d.argmin(axis=1)))
    return np.mean(ari), np.min(ari)

print("clients simulées (k = 3) : ARI moyen, minimum =", np.round(stabilite(Zs, 3,
    ↪ resultat["groupes"], np.random.default_rng(1)), 3))
print("vraies clientes (k = 4) : ARI moyen, minimum =", np.round(stabilite(Zc, 4, r4["groupes"],
    ↪ np.random.default_rng(1)), 3))

```

```

clients simulées (k = 3) : ARI moyen, minimum = [0.994 0.985]
vraies clientes (k = 4) : ARI moyen, minimum = [0.864 0.51 ]

```

Lecture. Sur les clientes simulées, le coude et la silhouette (maximale à $k = 3$) retrouvent le bon nombre de groupes et la stabilité est quasi parfaite. Sur les vraies clientes, il n'y a ni coude ni silhouette supérieure à 0,25 : les segments obtenus (jeunes à petit panier, âgées à gros panier,

habituées, anciennes) sont des **tranches commodées** d'un nuage continu. Utiles pour décider d'un message, mais pas des « types » de clientes. La stabilité (bonne en moyenne, mauvaise dans le pire cas) ne prouve pas non plus que ces groupes existent.

Pour aller plus loin. Refaites l'étape 4 avec seulement deux variables (nombre de commandes et panier) : comparez la silhouette à celle d'un nuage sans groupe (exercice 3.12).

Application 3.4 — Cartographier des associations : correspondances et ACM

Contexte. Comment « dessiner » le lien entre des variables qualitatives (canal, tranche de panier, ville...) ? L'analyse des correspondances (AC) transforme un tableau de contingence en carte ; l'ACM étend l'idée à plusieurs variables. Nous écrivons la méthode (une SVD des résidus standardisés), l'appliquons à un lien réel puis à un lien inexistant. Voir section 3.4 du livre.

Étape 1 : le tableau de contingence canal × tranche de panier.

```
from scipy import stats

c = pd.read_csv("donnees/clients.csv")
a = c[c["nb_commandes_an"] > 0].copy()
a["tranche_panier"] = pd.qcut(a["panier_moyen"], 4, labels=["T1 très petit", "T2 petit", "T3 grand",
    ↪ "T4 très grand"])
a["tranche_age"] = pd.cut(a["age"], [0, 29, 39, 200], labels=["moins de 30", "30-39", "40 et plus"])

N = pd.crosstab(a["canal_acquisition"], a["tranche_panier"])
chi2, p, ddl, _ = stats.chi2_contingency(N, correction=False)
print(N.to_string())
print(f"khi-deux = {chi2:.1f}, ddl = {ddl}, khi-deux / n = {chi2 / N.values.sum():.4f}")
```

tranche_panier	T1 très petit	T2 petit	T3 grand	T4 très grand
canal_acquisition				
Boutique	46	93	126	184
Réseaux	258	184	158	87
Site	132	157	152	163

khi-deux = 180.7, ddl = 6, khi-deux / n = 0.1038

Étape 2 : la méthode. Les résidus standardisés $S_{ij} = (p_{ij} - r_i c_j) / \sqrt{r_i c_j}$ ont pour norme au carré χ^2/n ; leur SVD donne les axes.

```
def analyse_correspondances(N):
    N = np.asarray(N, dtype=float)
    P = N / N.sum()
    r, cc = P.sum(axis=1), P.sum(axis=0)
    S = (P - np.outer(r, cc)) / np.sqrt(np.outer(r, cc))
    U, sv, Vt = np.linalg.svd(S, full_matrices=False)
    F = (U / np.sqrt(r)[:, None]) * sv          # coordonnées principales des lignes
    G = (Vt.T / np.sqrt(cc)[:, None]) * sv      # coordonnées principales des colonnes
    return {"inertie": sv**2, "F": F, "G": G}

print("exemple 2x2 :", analyse_correspondances([[30, 10], [10, 30]])["inertie"].round(4))
ac = analyse_correspondances(N.values)
print("inertie par axe :", ac["inertie"].round(4), "| somme :", ac["inertie"].sum().round(4))
```

```
exemple 2x2 : [0.25 0. ]
inertie par axe : [0.1031 0.0007 0. ] | somme : 0.1038
```

Étape 3 : lire la carte. Coordonnées des canaux puis des tranches sur les deux premiers axes.

```
lignes = pd.DataFrame(ac["F"][:, :2], index=N.index, columns=["axe 1", "axe 2"])
colonnes = pd.DataFrame(ac["G"][:, :2], index=N.columns, columns=["axe 1", "axe 2"])
print(pd.concat([lignes, colonnes]).round(3).to_string())
```

	axe 1	axe 2
Boutique	0.448	-0.026
Réseaux	-0.354	-0.015
Site	0.070	0.036
T1 très petit	-0.440	-0.024
T2 petit	-0.090	0.046
T3 grand	0.079	-0.010
T4 très grand	0.452	-0.012

Étape 4 : l'expérience inverse, avec deux variables sans aucun lien (canal et ville) : une carte a toujours l'air de dire quelque chose.

```
N2 = pd.crosstab(a["canal_acquisition"], a["ville"])
chi2_b, p_b, ddl_b, _ = stats.chi2_contingency(N2, correction=False)
ac2 = analyse_correspondances(N2.values)
print(f"canal x ville : khi-deux = {chi2_b:.1f}, ddl = {ddl_b}, p = {p_b:.2f}")
print("inertie totale :", round(chi2_b / N2.values.sum(), 4), "| part de l'axe 1 :", f"{100 *
↳ ac2['inertie'][0] / ac2['inertie'].sum():.0f} %")
```

```
canal x ville : khi-deux = 8.1, ddl = 10, p = 0.62
inertie totale : 0.0047 | part de l'axe 1 : 74 %
```

Étape 5 : l'ACM sur six variables qualitatives (tableau disjonctif complet), avec la correction de Benzécri.

```
cols = ["canal_acquisition", "ville", "tranche_age", "tranche_panier", "offre_bienvenue",
↳ "rachat_l2m"]
D = pd.get_dummies(a[cols].astype(str), dtype=float)
Q_, J = len(cols), D.shape[1]
acm = analyse_correspondances(D.values)
lam = acm["inertie"]
gardees = lam[lam > 1 / Q_]
corrigees = (Q_ / (Q_ - 1)) ** 2 * (gardees - 1 / Q_) ** 2
print("inertie totale =", lam.sum().round(4), "| (J - Q) / Q =", round((J - Q_) / Q_, 4))
print("parts brutes des deux premiers axes :", (100 * lam[:2] / lam.sum()).round(1), "%")
print("parts corrigées (Benzécri) : ", (100 * corrigees[:2] / corrigees.sum()).round(1), "%")
```

```
inertie totale = 2.3333 | (J - Q) / Q = 2.3333
parts brutes des deux premiers axes : [10.1 8.4] %
parts corrigées (Benzécri) : [77.7 15.1] %
```

Lecture. Le lien canal-panier est massivement significatif ($\chi^2/n \approx 0,10$) et tient presque entièrement sur le premier axe : Réseaux d'un côté avec les petits paniers, la boutique de l'autre avec les gros. Le lien canal-ville est nul ($p = 0,62$), mais la carte lui attribue pourtant « 74 % » sur l'axe 1 : un pourcentage d'inertie ne prouve jamais rien, l'inertie totale et le test d'abord. En ACM, l'inertie totale vaut toujours $(J - Q)/Q$ (2,33 ici) et les pourcentages bruts sont faibles par construction ; la correction de Benzécri donne une image plus réaliste.

Pour aller plus loin. Calculez les contributions de chaque variable aux axes de l'ACM (celle de la modalité j à l'axe k vaut $c_j G_{jk}^2 / \sigma_k^2$) : quelle variable construit l'axe 2 ?

Application 3.5 — Prédire le rachat avec l'analyse discriminante

Contexte. On connaît les groupes (rachat ou non) et l'on veut classer de nouvelles clientes d'après leurs scores de satisfaction et leur âge. Nous écrivons la LDA, la comparons à `scikit-learn` et à la QDA sur des données de test, puis à la régression logistique. Voir section 3.5 du livre.

Étape 1 : préparer les données et mettre de côté 362 répondantes pour le test.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score

q = pd.read_csv("donnees/enquete_satisfaction.csv")
c = pd.read_csv("donnees/clients.csv")
q["score_produits"] = q[["q1", "q2", "q3", "q4"]].mean(axis=1)
q["score_service"] = q[["q5", "q6", "q7", "q8"]].mean(axis=1)
d = q[["id_client", "score_produits", "score_service"]].merge(c[["id_client", "age", "rachat_12m"]],
                  on="id_client")

X = d[["score_produits", "score_service", "age"]].to_numpy()
y = d["rachat_12m"].to_numpy()
ordre = np.random.default_rng(5).permutation(len(d))
app, test = ordre[:850], ordre[850:]
print("apprentissage :", len(app), "| test :", len(test))
```

```
apprentissage : 850 | test : 362
```

Étape 2 : la LDA en trois fonctions. Estimer (a priori, moyennes, covariance intra poolée), calculer les scores δ_k , en déduire les probabilités.

```
def lda_ajuster(X, y):
    classes = np.unique(y)
    n, K = len(X), len(classes)
    pi = np.array([(y == k).mean() for k in classes])
    mu = np.array([X[y == k].mean(axis=0) for k in classes])
    Sw = sum((X[y == k] - mu[i]).T @ (X[y == k] - mu[i]) for i, k in enumerate(classes)) / (n - K)
    return classes, pi, mu, Sw

def lda_scores(X, modele):
    classes, pi, mu, Sw = modele
    Sinv = np.linalg.pinv(Sw)
    return np.column_stack([X @ Sinv @ mu[i] - 0.5 * mu[i] @ Sinv @ mu[i] + np.log(pi[i])
                           for i in range(len(classes))])

def lda_probas(X, modele):
    s = lda_scores(X, modele)
    e = np.exp(s - s.max(axis=1, keepdims=True)) # softmax stabilisé
    return e / e.sum(axis=1, keepdims=True)

modele = lda_ajuster(X[app], y[app])
classes, pi, mu, Sw = modele
print("a priori :", pi.round(3))
print(pd.DataFrame(mu, index=["pas de rachat", "rachat"], columns=["score_produits", "score_service",
                        "age"]).round(2).to_string())
```

```
a priori : [0.499 0.501]
           score_produits  score_service  age
pas de rachat          3.46            3.42 36.82
rachat                 3.78            3.68 35.60
```

Étape 3 : évaluer sur les données de test, et comparer avec la bibliothèque et la QDA.

```

proba = lda_probas(X[test], modele)
pred = classes[proba.argmax(axis=1)]
lda_sk = LinearDiscriminantAnalysis().fit(X[app], y[app])
qda_sk = QuadraticDiscriminantAnalysis().fit(X[app], y[app])
print("mêmes prédictions que scikit-learn :", np.array_equal(pred, lda_sk.predict(X[test])))
print("écart maximal sur les probabilités :", np.abs(proba[:, 1] - lda_sk.predict_proba(X[test])[:,
↪ 1]).max().round(4))
tab = pd.DataFrame({"précision (test)": [accuracy_score(y[test], pred), accuracy_score(y[test],
↪ qda_sk.predict(X[test]))],
                    "AUC (test)": [roc_auc_score(y[test], proba[:, 1]), roc_auc_score(y[test],
↪ qda_sk.predict_proba(X[test])[:, 1])],
                    index=["LDA", "QDA"]})
print(tab.round(3).to_string())
print(pd.crosstab(pd.Series(y[test], name="réel"), pd.Series(pred, name="prédit")).to_string())

```

```

mêmes prédictions que scikit-learn : True
écart maximal sur les probabilités : 0.0005
précision (test)  AUC (test)
LDA                0.657      0.705
QDA                0.638      0.708
prédit    0      1
réel
0         121    55
1          69   117

```

Étape 4 : LDA et régression logistique, même frontière, deux routes. La log-cote de la LDA est linéaire, de coefficients $\beta = \Sigma^{-1}(\mu_1 - \mu_0)$.

```

beta = np.linalg.solve(Sw, mu[1] - mu[0])
beta0 = np.log(pi[1] / pi[0]) - 0.5 * (mu[1] + mu[0]) @ beta
logit = LogisticRegression(C=1e6, max_iter=1000).fit(X[app], y[app]) # C très grand : pas de
↪ régularisation
coef = pd.DataFrame({"LDA (formule)": np.r_[beta0, beta],
                    "LDA (scikit-learn)": np.r_[lda_sk.intercept_, lda_sk.coef_.ravel()],
                    "régression logistique": np.r_[logit.intercept_, logit.coef_.ravel()]},
                    index=["constante", "score_produits", "score_service", "age"])
print(coef.round(3).to_string())

```

	LDA (formule)	LDA (scikit-learn)	régression logistique
constante	-3.345	-3.353	-3.341
score_produits	0.663	0.664	0.663
score_service	0.401	0.402	0.399
age	-0.013	-0.013	-0.013

Lecture. La règle écrite à la main reproduit celle de la bibliothèque. La précision (environ deux sur trois) est meilleure que le hasard mais loin d'une prédiction fiable : le rachat est intrinsèquement incertain et la satisfaction n'en est qu'un déterminant. La QDA ne fait pas mieux que la LDA. Les trois jeux de coefficients sont presque identiques : même forme, estimée différemment (modèle génératif contre discriminatif).

Pour aller plus loin. Changez les probabilités a priori (par exemple 0,7 / 0,3) et observez comment la frontière et la matrice de confusion se déplacent (voir l'exemple à la main du livre).

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 3.13 et 3.14 portent sur les sections optionnelles 3.5 et 3.4 du livre.

Exercice 3.1 ★ — ACP à la main (section 3.1 du livre)

Cinq clientes ont passé 4, 5, 6, 7, 8 commandes dans l'année, pour un panier moyen respectif de 4, 3, 6, 7, 5 dizaines de €. (a) Centrez les données et calculez la matrice de covariance. (b) Trouvez ses valeurs propres et ses vecteurs propres unitaires. (c) Quelle part de la variance la première composante explique-t-elle ? (d) Calculez les scores des cinq clientes sur cette composante. (e) Vérifiez par le code.

Exercice 3.2 ★ — Lire un éboulis (section 3.1 du livre)

Une ACP normée sur 5 variables a donné les valeurs propres 2,6; 1,1; 0,7; 0,4; 0,2. (a) Quelle est la variance totale ? (b) Calculez la part de variance de chaque composante et les parts cumulées. (c) Combien de composantes retient la règle de Kaiser ? Combien faut-il pour atteindre 80 % de la variance ? (d) Pourquoi les deux critères ne donnent-ils pas la même réponse, et lequel préférez-vous ?

Exercice 3.3 ★ — Les unités (section 3.1 du livre)

Dans la table `clients.csv`, on exprime la dépense annuelle en **centimes** de euro au lieu de euros. (a) Que devient la part de variance de la première composante d'une ACP **non standardisée** ? (b) Et d'une ACP **standardisée** ? Justifiez, puis vérifiez par le code.

Exercice 3.4 ★★ — Propriétés des scores (section 3.1 du livre)

Soit Z un tableau de variables standardisées, R sa matrice de corrélation, $\mathbf{v}_j$ et λ_j ses éléments propres, et $\mathbf{t}_j = Z\mathbf{v}_j$ les scores. (a) Démontrez que $\text{Var}(\mathbf{t}_j) = \lambda_j$ et que $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = 0$ pour $i \neq j$. (b) Démontrez que la corrélation entre la variable z_i et le score $\mathbf{t}_j$ vaut $v_{ij}\sqrt{\lambda_j}$. (c) Vérifiez les deux propriétés numériquement sur les cinq variables numériques de `clients.csv` standardisées.

Exercice 3.5 ★★ — Reconstruction (section 3.1 du livre)

Sur les huit questions standardisées du questionnaire : (a) combien de composantes faut-il pour restituer au moins 80 % de la variance ? (b) Calculez l'erreur quadratique moyenne par case (RMSE) de la reconstruction avec 2 composantes, et comparez-la à l'erreur d'une reconstruction **triviale** par la moyenne (c'est-à-dire avec 0 composante). (c) Vérifiez que l'erreur totale vaut $(n-1) \sum_{j>k} \lambda_j$.

Exercice 3.6 ★★ — Modèle factoriel à un facteur (section 3.2 du livre)

Trois questions sur la rapidité de la livraison ont les corrélations $r_{12} = 0,48$, $r_{13} = 0,40$, $r_{23} = 0,30$. (a) Calculez les saturations d'un modèle à un facteur, les communalités et les unicités. (b) Quelle question est le meilleur indicateur du facteur ? (c) Que se passe-t-il si r_{23} valait 0,15 ? Qu'est-ce que cela signifie pour le modèle ?

Exercice 3.7 ★★ — Invariance par rotation (section 3.2 du livre)

Les saturations d'un modèle à deux facteurs pour quatre questions sont $\Lambda = \begin{pmatrix} 0,7 & 0,3 \\ 0,6 & 0,4 \\ 0,2 & 0,8 \\ 0,3 & 0,7 \end{pmatrix}$. (a)

Calculez les communalités. (b) On applique une rotation d'angle $\theta = 30^\circ$: $\Lambda^* = \Lambda T$ avec $T = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$. Calculez Λ^* et vérifiez que les communalités et la matrice $\Lambda\Lambda^\top$ sont inchangées.

(c) Pourquoi cela empêche-t-il de dire « la vraie valeur » d'une saturation ?

Exercice 3.8 ★★ — Alpha de Cronbach (section 3.2 du livre)

Une échelle de trois questions standardisées a une corrélation moyenne de 0,5 entre questions. (a) Calculez la variance de la somme des trois questions, puis l'alpha de Cronbach. (b) Combien de questions de même corrélation moyenne faut-il pour atteindre $\alpha \geq 0,9$? On rappelle la formule de Spearman-Brown : pour des questions standardisées, $\alpha = \frac{k\bar{r}}{1 + (k-1)\bar{r}}$.

Exercice 3.9 ★★ — K-means : le piège du minimum local (section 3.3 du livre)

Six paniers (en dizaines de €) : 2, 3, 4, 10, 12, 20. On cherche $k = 2$ groupes. (a) Déroulez l'algorithme de Lloyd à partir des centres (2; 20) et calculez la variance intra-groupe finale W . (b) Recommencez à partir des centres (2; 10). (c) Comparez les deux solutions : que conclure sur l'algorithme ?

Exercice 3.10 ★★ — Silhouette (section 3.3 du livre)

Pour la partition $\{2, 3, 4\}$ et $\{10, 12, 20\}$ de l'exercice 3.9, calculez à la main la silhouette des points 4, 10 et 20, puis la silhouette moyenne (code). Quel point est le moins bien classé ?

Exercice 3.11 ★★ — Critères d'agrégation (section 3.3 du livre)

Quatre clientes A, B, C, D ont les distances deux à deux suivantes : $AB = 2$, $AC = 5$, $AD = 9$, $BC = 3$, $BD = 7$, $CD = 4,5$. Construisez à la main l'arbre de la classification hiérarchique pour les critères **simple**, **complet** et **moyen**, en donnant les fusions successives et leurs hauteurs. Vérifiez avec `scipy`.

Exercice 3.12 ★★★ — Une segmentation, ou pas ? (section 3.3 du livre)

La gérante veut des segments de clientes selon deux critères seulement : le nombre de commandes par an et le panier moyen (clientes ayant commandé, variables standardisées). (a) Calculez la silhouette moyenne des k-means pour $k = 2, \dots, 6$. (b) Évaluez la stabilité (indice de Rand ajusté entre deux ré-estimations sur des échantillons bootstrap) du découpage en $k = 3$. (c) Que conseillez-vous à la gérante ? Rédigez deux phrases pour elle, sans jargon.

Exercice 3.13 ★★ — Analyse discriminante (section 3.5 du livre)

Les clientes à petit panier ont un panier moyen de 40 €, celles à gros panier de 55 €, avec un écart-type de 10 € dans chaque groupe, et les gros paniers représentent 20 % des commandes. (a) À partir de quel panier la règle de Bayes (LDA à une variable) classe-t-elle une commande dans « gros panier » ? (b) Quelle est la probabilité a posteriori d'être un gros panier pour une commande de 55 € ? (c) Commentez : une commande de 55 €, égale à la moyenne du groupe « gros panier », est-elle classée dans ce groupe ?

Exercice 3.14 ★★★ — Analyse des correspondances (section 3.4 du livre)

Un tableau croise le canal (Réseaux, Boutique) et trois tranches de délai de livraison (court, moyen, long) :

	court	moyen	long
Réseaux	40	30	10
Boutique	10	30	30

- (a) Calculez χ^2 et l'inertie totale χ^2/n . (b) Combien d'axes non triviaux l'AC aura-t-elle ? (c) Vérifiez par le code que la somme des inerties des axes est égale à χ^2/n , et dites quelle modalité de délai est la plus associée à Réseaux.

Corrigés**Corrigé 3.1**

- (a) Moyennes : $\bar{x} = 6$, $\bar{y} = 5$. Écarts : $x_c = (-2, -1, 0, 1, 2)$ et $y_c = (-1, -2, 1, 2, 0)$. Variances : $s_{xx} = 10/4 = 2,5$, $s_{yy} = (1+4+1+4+0)/4 = 2,5$; covariance : $s_{xy} = (2+2+0+2+0)/4 = 1,5$.
Donc $S = \begin{pmatrix} 2,5 & 1,5 \\ 1,5 & 2,5 \end{pmatrix}$.
- (b) $\det(S - \lambda I) = (2,5 - \lambda)^2 - 1,5^2 = 0$ donne $\lambda = 2,5 \pm 1,5$, soit $\lambda_1 = 4$ et $\lambda_2 = 1$. Pour $\lambda_1 = 4$: $-1,5v_1 + 1,5v_2 = 0$, donc $\mathbf{v}_1 = (1, 1)/\sqrt{2}$. Pour $\lambda_2 = 1$: $\mathbf{v}_2 = (1, -1)/\sqrt{2}$.
- (c) Variance totale = 5 ; la première composante en explique $4/5 = 80\%$.
- (d) $t_{i1} = (x_{c,i} + y_{c,i})/\sqrt{2}$: les sommes $x_c + y_c$ valent $(-3, -3, 1, 3, 2)$, donc les scores sont $(-3, -3, 1, 3, 2)/\sqrt{2} \approx (-2,12; -2,12; 0,71; 2,12; 1,41)$. On vérifie que leur variance est $(9 + 9 + 1 + 9 + 4)/(2 \times 4) = 32/8 = 4 = \lambda_1$ ✓.

```
import numpy as np
X = np.array([[4, 4], [5, 3], [6, 6], [7, 7], [8, 5]], dtype=float)
Xc = X - X.mean(axis=0)
S = np.cov(Xc, rowvar=False)
w, V = np.linalg.eigh(S)
o = np.argsort(w)[-1]
w, V = w[o], V[:, o]
print("S =", S.tolist())
print("valeurs propres :", w.round(3), "| parts :", (w / w.sum()).round(3))
print("scores sur CP1 (au signe près) :", np.abs(Xc @ V[:, 0]).round(2))
```

```
S = [[2.5, 1.5], [1.5, 2.5]]
valeurs propres : [4. 1.] | parts : [0.8 0.2]
scores sur CP1 (au signe près) : [2.12 2.12 0.71 2.12 1.41]
```

Corrigé 3.2

- (a) Pour une ACP normée, la variance totale est le nombre de variables : 5 (et c'est bien la somme $2,6 + 1,1 + 0,7 + 0,4 + 0,2 = 5$). (b) Parts : 52 %, 22 %, 14 %, 8 %, 4 % ; cumul : 52, 74, 88, 96, 100 %. (c) Kaiser garde les valeurs propres > 1 : **deux** composantes (2,6 et 1,1). Pour atteindre 80 %, il en faut **trois** (74 % avec deux, 88 % avec trois). (d) Les deux critères répondent à des questions différentes : Kaiser compare chaque composante à « une variable d'origine » ; la règle des 80 % fixe un objectif de restitution. Ici, la deuxième composante (1,1) dépasse de très peu le seuil de 1 : c'est le cas où l'**analyse parallèle** est utile, car elle compare à ce que ferait le bruit (qui dépasse souvent 1 pour la deuxième valeur propre, comme au 3.1.6).

```
w = np.array([2.6, 1.1, 0.7, 0.4, 0.2])
print("variance totale :", w.sum())
print("parts (%)      :", (100 * w / w.sum()).round(0))
print("cumul (%)      :", (100 * w.cumsum() / w.sum()).round(0))
print("Kaiser (> 1):", int((w > 1).sum()), "| composantes pour 80 % :", int(np.argmax(w.cumsum() /
↪ w.sum() >= 0.8) + 1))
```

```
variance totale : 5.000000000000001
parts (%)       : [52. 22. 14.  8.  4.]
cumul (%)       : [ 52.  74.  88.  96. 100.]
Kaiser (> 1): 2 | composantes pour 80 % : 3
```

Corrigé 3.3

- (a) Multiplier une variable par 100 multiplie sa variance par 10 000 : la dépense, qui dominait déjà, **écrase** tout le reste. La première composante, qui expliquait déjà 98,8 % de la variance en euros, en explique alors pratiquement 100 % (le code arrondit à 1,0) et pointe sur la dépense. (b) Avec la standardisation, chaque variable est divisée par son écart-type : **changer d'unité ne change rien** (la variable standardisée est la même). Les parts de variance sont identiques à celles du 3.1.5.

```
import pandas as pd
c = pd.read_csv("donnees/clients.csv")
cols = ["age", "nb_commandes_an", "panier_moyen", "depense_annuelle", "duree_mois"]
X0 = c[cols].copy()
X1 = X0.copy(); X1["depense_annuelle"] = X1["depense_annuelle"] * 100      # en centimes

def parts(M, standardiser):
    M = (M - M.mean()) / M.std() if standardiser else M - M.mean()
    w = np.sort(np.linalg.eigvalsh(np.cov(M.to_numpy(), rowvar=False)))[::-1]
    return w / w.sum()

print("non standardisé, euros      :", parts(X0, False)[0].round(5))
print("non standardisé, centimes   :", parts(X1, False)[0].round(5))
print("standardisé, euros          :", parts(X0, True).round(3))
print("standardisé, centimes       :", parts(X1, True).round(3))
```

```
non standardisé, euros      : 0.98789
non standardisé, centimes   : 1.0
standardisé, euros          : [0.442 0.214 0.194 0.125 0.025]
standardisé, centimes       : [0.442 0.214 0.194 0.125 0.025]
```

Corrigé 3.4

(a) Les colonnes de Z sont centrées, donc $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = \frac{1}{n-1} \mathbf{v}_i^\top Z^\top Z \mathbf{v}_j = \mathbf{v}_i^\top R \mathbf{v}_j = \lambda_j \mathbf{v}_i^\top \mathbf{v}_j$ (car $R \mathbf{v}_j = \lambda_j \mathbf{v}_j$). Les vecteurs propres d'une matrice symétrique sont orthonormés : $\mathbf{v}_i^\top \mathbf{v}_j = \delta_{ij}$. D'où $\text{Cov}(\mathbf{t}_i, \mathbf{t}_j) = \lambda_j \delta_{ij}$.

(b) $\text{Cov}(\mathbf{z}_i, \mathbf{t}_j) = \frac{1}{n-1} \mathbf{z}_i^\top Z \mathbf{v}_j = (R \mathbf{v}_j)_i = \lambda_j v_{ij}$. Comme $\text{Var}(\mathbf{z}_i) = 1$ et $\text{Var}(\mathbf{t}_j) = \lambda_j$, la corrélation vaut $\frac{\lambda_j v_{ij}}{1 \times \sqrt{\lambda_j}} = v_{ij} \sqrt{\lambda_j}$.

(c)

```
Z = ((X0 - X0.mean()) / X0.std()).to_numpy()
R = np.corrcoef(Z, rowvar=False)
w, V = np.linalg.eigh(R)
o = np.argsort(w)[-1]
w, V = w[o], V[:, o]
T = Z @ V
print("variances des scores :", T.var(axis=0, ddof=1).round(4))
print("valeurs propres      :", w.round(4))
corr_scores = np.corrcoef(T, rowvar=False)
print("plus grande corrélation entre deux scores différents :", np.abs(corr_scores -
    np.eye(5)).max().round(10))
cor_var_comp = np.array([[np.corrcoef(Z[:, i], T[:, j])[0, 1] for j in range(5)] for i in range(5)])
print("cor(variable, composante) = v * sqrt(lambda) ?", np.allclose(cor_var_comp, V * np.sqrt(w)))
```

```
variances des scores : [2.2115 1.0702 0.9684 0.6256 0.1242]
valeurs propres      : [2.2115 1.0702 0.9684 0.6256 0.1242]
plus grande corrélation entre deux scores différents : 0.0
cor(variable, composante) = v * sqrt(lambda) ? True
```

Corrigé 3.5

(a) On lit les parts cumulées : 0,748 avec quatre composantes, 0,818 avec cinq : il en faut **cinq**. C'est beaucoup pour huit questions, et c'est instructif : la règle des 80 % est ici bien moins sévère que Kaiser et l'analyse parallèle, qui retiennent deux composantes ; le questionnaire contient deux dimensions nettes, mais chaque question a aussi une grande part de variance qui lui est propre (3.2). (b) La reconstruction avec k composantes a pour erreur $(n-1) \sum_{j>k} \lambda_j$; la RMSE par case vaut $\sqrt{\text{erreur}/(np)}$. Avec 0 composante, on reconstruit chaque case par 0 (la moyenne des variables standardisées) et l'erreur est la variance totale $(n-1)p$; la RMSE vaut alors à peu près 1 (l'écart-type des variables standardisées). (c) Vérification par le code : avec deux composantes, la RMSE tombe de 1,000 à 0,643, et l'erreur totale (4 007,5) coïncide exactement avec la formule.

```
q = pd.read_csv("donnees/enquete_satisfaction.csv")
Q = q[[f"q{j}" for j in range(1, 9)]]
Zq = ((Q - Q.mean()) / Q.std()).to_numpy()
n, p = Zq.shape
w, V = np.linalg.eigh(np.corrcoef(Zq, rowvar=False))
o = np.argsort(w)[-1]
w, V = w[o], V[:, o]

cumul = (w / w.sum()).cumsum()
print("composantes pour au moins 80 % :", int(np.argmax(cumul >= 0.8)) + 1, "(parts cumulées :",
    cumul.round(3).tolist(), ")")

for k in (0, 2):
    Vk = V[:, :k]
    erreur = ((Zq - Zq @ Vk @ Vk.T) ** 2).sum()
```

```
print(f"k = {k} : erreur totale = {erreur:.1f}, théorie (n-1) x somme(valeurs jetées) = {(n - 1) *
↳ w[k:].sum():.1f}, RMSE par case = {np.sqrt(erreur / (n * p)):.3f}")
```

composantes pour au moins 80 % : 5 (parts cumulées : [0.35, 0.586, 0.675, 0.748, 0.818, 0.887, 0.946, 1.0])

k = 0 : erreur totale = 9688.0, théorie (n-1) x somme(valeurs jetées) = 9688.0, RMSE par case = 1.000

k = 2 : erreur totale = 4007.5, théorie (n-1) x somme(valeurs jetées) = 4007.5, RMSE par case = 0.643

Corrigé 3.6

(a) $\ell_1^2 = \frac{r_{12}r_{13}}{r_{23}} = \frac{0,48 \times 0,40}{0,30} = 0,64$, donc $\ell_1 = 0,8$; $\ell_2 = 0,48/0,8 = 0,6$; $\ell_3 = 0,40/0,8 = 0,5$.

Communalités : 0,64; 0,36; 0,25. Unicités : 0,36; 0,64; 0,75. Contrôle : $\ell_2\ell_3 = 0,30 = r_{23}$ ✓.

(b) La question 1 : saturation la plus forte (0,8) et unicité la plus faible. (c) Avec $r_{23} = 0,15$: $\ell_1^2 = 0,48 \times 0,40/0,15 = 1,28 > 1$. Une communalité supérieure à 1 est **impossible** (elle dépasserait la variance de la variable, soit une unicité négative $1 - 1,28 = -0,28$) : c'est un **cas de Heywood**. Il signale que le modèle à un facteur est incompatible avec ces corrélations (par exemple, la question 1 est trop corrélée aux deux autres par rapport à leur corrélation mutuelle), ou que l'échantillon est trop petit.

```
for r23 in (0.30, 0.15):
    l1_carre = 0.48 * 0.40 / r23
    print(f"r23 = {r23} : l1^2 = {l1_carre:.2f}", "-> impossible (cas de Heywood)" if l1_carre > 1
    ↳ else f"-> l1 = {np.sqrt(l1_carre):.2f}")
```

r23 = 0.3 : l1^2 = 0.64 -> l1 = 0.80

r23 = 0.15 : l1^2 = 1.28 -> impossible (cas de Heywood)

Corrigé 3.7

(a) Communalités : $0,49 + 0,09 = 0,58$; $0,36 + 0,16 = 0,52$; $0,04 + 0,64 = 0,68$; $0,09 + 0,49 = 0,58$.

(b) Voir le code : Λ^* est différente de Λ mais communalités et $\Lambda\Lambda^T$ sont identiques, puisque $\Lambda^*\Lambda^{*T} = \Lambda T T^T \Lambda^T = \Lambda\Lambda^T$. (c) Comme l'ajustement est strictement le même pour tous les angles, les données **ne peuvent pas choisir** entre « la saturation de la question 1 sur le facteur 1 vaut 0,7 » et « elle vaut 0,66 » : seule la convention de rotation (varimax, etc.) fixe une valeur.

```
Lam = np.array([[0.7, 0.3], [0.6, 0.4], [0.2, 0.8], [0.3, 0.7]])
th = np.deg2rad(30)
T = np.array([[np.cos(th), -np.sin(th)], [np.sin(th), np.cos(th)]])
Lam2 = Lam @ T
print("communalités avant :", (Lam**2).sum(axis=1).round(3))
print("communalités après :", (Lam2**2).sum(axis=1).round(3))
print("Lambda* =\n", Lam2.round(3))
print("même Lambda Lambda' :", np.allclose(Lam @ Lam.T, Lam2 @ Lam2.T))
```

communalités avant : [0.58 0.52 0.68 0.58]

communalités après : [0.58 0.52 0.68 0.58]

Lambda* =

```
[[ 0.756 -0.09 ]
 [ 0.72   0.046]
 [ 0.573  0.593]
 [ 0.61   0.456]]
```

même Lambda Lambda' : True

Corrigé 3.8

- (a) Variance de la somme de $k = 3$ variables de variance 1 et de covariance 0,5 : $3 + 3 \times 2 \times 0,5 = 6$.
 Alpha : $\frac{3}{2} \left(1 - \frac{3}{6}\right) = 0,75$. Cohérent avec Spearman-Brown : $\frac{3 \times 0,5}{1 + 2 \times 0,5} = 0,75$ ✓. (b) On résout $\frac{0,5k}{1 + 0,5(k-1)} = 0,9$, soit $0,5k = 0,9(0,5 + 0,5k) = 0,45 + 0,45k$, donc $0,05k = 0,45$ et $k = 9$. Il faut **neuf** questions de corrélation moyenne 0,5 pour atteindre $\alpha = 0,9$: l'alpha augmente avec la longueur de l'échelle, ce qui rend toute comparaison entre échelles de longueurs différentes délicate.

```
def alpha_std(k, r): return k * r / (1 + (k - 1) * r)
print("alpha, k = 3 :", alpha_std(3, 0.5))
print("k pour alpha >= 0,9 :", next(k for k in range(2, 50) if alpha_std(k, 0.5) >= 0.9), "| alpha(k = 9) =", round(alpha_std(9, 0.5), 3))
```

```
alpha, k = 3 : 0.75
k pour alpha >= 0,9 : 9 | alpha(k = 9) = 0.9
```

Corrigé 3.9

- (a) Centres (2;20). *Affectation* : 2,3,4 vont avec 2 ; 10 est à 8 de 2 et à 10 de 20 : il va avec 2 ; 12 est à 10 de 2 et à 8 de 20 : il va avec 20. Groupes {2,3,4,10} et {12,20}. *Mise à jour* : centres 4,75 et 16. *Affectation* : 10 est à 5,25 de 4,75 et à 6 de 16 ; 12 est à 7,25 et à 4 : rien ne change. Fin. $W = (2,75^2 + 1,75^2 + 0,75^2 + 5,25^2) + (4^2 + 4^2) = 7,5625 + 3,0625 + 0,5625 + 27,5625 + 32 = 70,75$. (b) Centres (2;10) : groupes {2,3,4} et {10,12,20} ; centres 3 et 14 ; plus rien ne change. $W = (1 + 0 + 1) + (16 + 4 + 36) = 58$. (c) Les deux exécutions **convergent**, mais vers deux solutions différentes (70,75 et 58) : la seconde est meilleure. L'algorithme de Lloyd ne trouve qu'un **minimum local**, qui dépend de l'initialisation : d'où les redémarrages multiples et k-means++.

```
pts = np.array([2, 3, 4, 10, 12, 20.0])
for init in ([2.0, 20.0], [2.0, 10.0]):
    centres = np.array(init)
    while True:
        g = np.abs(pts[:, None] - centres[None, :]).argmin(axis=1)
        nouveaux = np.array([pts[g == j].mean() for j in range(2)])
        if np.allclose(nouveaux, centres):
            break
        centres = nouveaux
    W = sum([(pts[g == j] - centres[j]) ** 2].sum() for j in range(2)])
    print(f"départ {init} -> groupes {g.tolist()}, centres {centres.round(2).tolist()}, W = {W:.2f}")
```

```
départ [2.0, 20.0] -> groupes [0, 0, 0, 0, 1, 1], centres [4.75, 16.0], W = 70.75
départ [2.0, 10.0] -> groupes [0, 0, 0, 1, 1, 1], centres [3.0, 14.0], W = 58.00
```

Corrigé 3.10

Point 4 (groupe {2,3,4}) : $a = (2 + 1)/2 = 1,5$; $b = (6 + 8 + 16)/3 = 10$; $s = (10 - 1,5)/10 = 0,85$.
 Point 10 (groupe {10,12,20}) : $a = (2 + 10)/2 = 6$; $b = (8 + 7 + 6)/3 = 7$; $s = (7 - 6)/7 \approx 0,143$.
 Point 20 : $a = (10 + 8)/2 = 9$; $b = (18 + 17 + 16)/3 = 17$; $s = (17 - 9)/17 \approx 0,471$. Le point 10 est le moins bien classé : presque à égale distance des deux groupes (son groupe est étiré par le point extrême 20).

```
from sklearn.metrics import silhouette_samples
s = silhouette_samples(pts.reshape(-1, 1), np.array([0, 0, 0, 1, 1, 1]))
print("silhouettes :", dict(zip(pts.astype(int).tolist(), s.round(3).tolist())))
print("silhouette moyenne :", s.mean().round(3))
```

```
silhouettes : {2: 0.875, 3: 0.909, 4: 0.85, 10: 0.143, 12: 0.444, 20: 0.471}
silhouette moyenne : 0.615
```

Corrigé 3.11

Lien simple (distance minimale entre groupes) : fusion de $\{A,B\}$ à 2 ; puis $\{A,B\}-C$ vaut $\min(5;3) = 3$, $\{A,B\}-D$ vaut $\min(9;7) = 7$, $C-D$ vaut 4,5 : on fusionne C à $\{A,B\}$ à la hauteur 3 ; enfin D à la hauteur $\min(9;7;4,5) = 4,5$. Hauteurs : 2; 3; 4,5 (effet de **chaîne** : A,B,C,D s'enchaînent). *Lien complet* (distance maximale) : $\{A,B\}$ à 2 ; puis $\{A,B\}-C$ vaut $\max(5;3) = 5$, $\{A,B\}-D$ vaut 9, $C-D$ vaut 4,5 : on fusionne $\{C,D\}$ à 4,5 ; enfin $\{A,B\}-\{C,D\}$ vaut $\max(5;9;3;7) = 9$. Hauteurs : 2; 4,5; 9. *Lien moyen* : $\{A,B\}$ à 2 ; $\{A,B\}-C$ vaut $(5 + 3)/2 = 4$, $\{A,B\}-D$ vaut 8, $C-D$ vaut 4,5 : on fusionne C à $\{A,B\}$ à 4 ; puis D à $(9 + 7 + 4,5)/3 \approx 6,83$. Hauteurs : 2; 4; 6,83. Les **trois arbres diffèrent** : le critère simple et le moyen rattachent C à $\{A,B\}$, le critère complet préfère former $\{C,D\}$.

```
from scipy.cluster.hierarchy import linkage
from scipy.spatial.distance import squareform
D = np.array([[0, 2, 5, 9], [2, 0, 3, 7], [5, 3, 0, 4.5], [9, 7, 4.5, 0]], dtype=float)
for m in ["single", "complete", "average"]:
    L = linkage(squareform(D), method=m)
    print(f"{m:9s} : fusions (indices) = {L[:, :2].astype(int).tolist()}, hauteurs = {L[:, :2].round(3).tolist()}")
```

```
single      : fusions (indices) = [[0, 1], [2, 4], [3, 5]], hauteurs = [2.0, 3.0, 4.5]
complete    : fusions (indices) = [[0, 1], [2, 3], [4, 5]], hauteurs = [2.0, 4.5, 9.0]
average      : fusions (indices) = [[0, 1], [2, 4], [3, 5]], hauteurs = [2.0, 4.0, 6.833]
```

Corrigé 3.12

- (a) et (b) par le code, ainsi qu'un point de comparaison indispensable : la silhouette qu'obtiendrait un nuage **sans aucun groupe**, mais de même taille et d'asymétrie comparable (variables log-normales, comme le sont le nombre de commandes et le panier). (c) Voir la discussion sous le code.

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score, silhouette_score

act = c[c["nb_commandes_an"] > 0]
Z2 = ((act[["nb_commandes_an", "panier_moyen"]] - act[["nb_commandes_an", "panier_moyen"]].mean())
      / act[["nb_commandes_an", "panier_moyen"]].std()).to_numpy()

# nuage témoin : mêmes effectifs, variables asymétriques (log-normales) corrélées, mais AUCUN groupe
rng = np.random.default_rng(0)
temoin = np.exp(rng.multivariate_normal([0, 0], [[0.4, 0.15], [0.15, 0.3]], len(Z2)))
temoin = (temoin - temoin.mean(axis=0)) / temoin.std(axis=0)

lignes = []
for k in range(2, 7):
    sil_reel = silhouette_score(Z2, KMeans(n_clusters=k, n_init=10, random_state=0).fit(Z2).labels_)
    sil_temoin = silhouette_score(temoin, KMeans(n_clusters=k, n_init=10,
    random_state=0).fit(temoin).labels_)
    lignes.append({"k": k, "silhouette (clientes)": sil_reel, "silhouette (nuage sans groupe)":
    sil_temoin})
print(pd.DataFrame(lignes).set_index("k").round(3).to_string())
```

```
silhouette (clientes)  silhouette (nuage sans groupe)
k
```

2	0.422	0.511
3	0.450	0.496
4	0.392	0.420
5	0.375	0.416
6	0.358	0.390

Puis la stabilité du découpage en trois groupes :

```
ref = KMeans(n_clusters=3, n_init=10, random_state=0).fit(Z2)
rng = np.random.default_rng(4)
ari = []
for _ in range(30):
    idx = rng.integers(0, len(Z2), len(Z2))
    km = KMeans(n_clusters=3, n_init=3, random_state=int(rng.integers(10**6))).fit(Z2[idx])
    ari.append(adjusted_rand_score(ref.labels_, km.predict(Z2)))
print(f"stabilité (k = 3) : ARI moyen = {np.mean(ari):.3f}, minimum = {np.min(ari):.3f}")
```

```
stabilité (k = 3) : ARI moyen = 0.920, minimum = 0.824
```

- (a) La silhouette moyenne des clientes atteint son maximum en $k = 3$ (0,450). Cette valeur tombe dans la zone « structure faible » (0,25 à 0,50) et l'on pourrait être tenté de conclure à trois familles. Mais le **nuage témoin**, qui ne contient **aucun** groupe, obtient une silhouette **plus élevée à chaque k** (0,511 en $k = 2$, 0,496 en $k = 3$). L'explication : des variables fortement **asymétriques** (longue queue vers les grandes valeurs) font que les k-means découpent le nuage en tranches « cœur dense » / « queue », et la silhouette récompense ce découpage. **Moralité : une silhouette ne se lit jamais seule, il faut une valeur de comparaison obtenue sur des données sans structure** ; ici, les clientes ne montrent aucune structure en groupes au-delà de ce que produit une simple asymétrie.
- (b) La stabilité du découpage en trois groupes est bonne (ARI moyen 0,92, minimum 0,82) : la partition est reproductible. Comme au 3.3.5, **stabilité n'est pas existence** : ce sont des tranches reproductibles d'un nuage continu.
- (c) *Conseil à la gérante* : « Vos clientes ne forment pas des familles distinctes selon le nombre de commandes et le panier : elles se répartissent sur un continuum, avec beaucoup de petites clientes et quelques très grosses. Pour vos relances, vous pouvez néanmoins utiliser trois tranches pratiques (peu actives, régulières, très actives), à condition de les voir comme des repères commodes, pas comme des types de clientes qui existeraient vraiment. »

Corrigé 3.13

- (a) $\text{Seuil} = \frac{40 + 55}{2} + \frac{10^2}{55 - 40} \ln \frac{0,8}{0,2} = 47,5 + 6,667 \times 1,386 \approx 56,74 \text{ €}$. (b) $\mathbb{P}(\text{gros} \mid 55) = \frac{0,2f_{55}(55)}{0,8f_{40}(55) + 0,2f_{55}(55)}$: calculé par le code, environ 0,435. (c) **Non** : malgré un panier égal à la moyenne du groupe « gros panier », la commande est classée « petit panier » ($0,435 < 0,5$). Les gros paniers étant quatre fois plus rares, il faut une commande plus élevée ($\geq 56,74$) pour faire basculer la décision : c'est l'effet de la probabilité a priori (3.5.1).

```
from scipy.stats import norm
mu0, mu1, sig, pi0 = 40, 55, 10, 0.8
seuil = (mu0 + mu1) / 2 + sig**2 / (mu1 - mu0) * np.log(pi0 / (1 - pi0))
print("seuil de décision :", round(seuil, 2), "€")
for x in (50, 55, 60):
    a = pi0 * norm.pdf(x, mu0, sig); b = (1 - pi0) * norm.pdf(x, mu1, sig)
    print(f"panier de {x} € : P(gros panier | x) = {b / (a + b):.3f}")
```

```

seuil de décision : 56.74 €
panier de 50 € : P(gros panier | x) = 0.267
panier de 55 € : P(gros panier | x) = 0.435
panier de 60 € : P(gros panier | x) = 0.620

```

Corrigé 3.14

- (a) $n = 150$; marges des lignes 80 et 70 ; marges des colonnes 50, 60, 40. Effectifs attendus : $\frac{80 \times 50}{150} \approx 26,67$, $\frac{80 \times 60}{150} = 32$, $\frac{80 \times 40}{150} \approx 21,33$ pour Réseaux, et 23,33, 28, 18,67 pour la boutique.
 $\chi^2 = \frac{13,33^2}{26,67} + \frac{2^2}{32} + \frac{11,33^2}{21,33} + \frac{13,33^2}{23,33} + \frac{2^2}{28} + \frac{11,33^2}{18,67} \approx 6,67 + 0,13 + 6,02 + 7,62 + 0,14 + 6,88 \approx 27,5$
; inertie totale $\chi^2/n \approx 0,183$. (b) Un tableau 2×3 a $\min(2, 3) - 1 = 1$ axe non trivial. (c) Voir le code ; la modalité de délai la plus proche du canal Réseaux sur l'axe est « court » (Réseaux a 50 % de délais courts contre 14 % pour la boutique).

```

from scipy import stats
N = np.array([[40, 30, 10], [10, 30, 30]], dtype=float)
chi2 = stats.chi2_contingency(N, correction=False)[0]
P = N / N.sum(); r, cc = P.sum(axis=1), P.sum(axis=0)
S = (P - np.outer(r, cc)) / np.sqrt(np.outer(r, cc))
U, sv, Vt = np.linalg.svd(S, full_matrices=False)
F = (U / np.sqrt(r)[:, None]) * sv
G = (Vt.T / np.sqrt(cc)[:, None]) * sv
print("khi-deux =", round(chi2, 2), "| khi-deux / n =", round(chi2 / N.sum(), 4))
print("inerties des axes :", (sv**2).round(4), "| somme :", (sv**2).sum().round(4))
print("axe 1, lignes   :", pd.Series(F[:, 0], index=["Réseaux", "Boutique"]).round(3).to_dict())
print("axe 1, colonnes :", pd.Series(G[:, 0], index=["court", "moyen", "long"]).round(3).to_dict())

```

```

khi-deux = 27.46 | khi-deux / n = 0.183
inerties des axes : [0.183 0.   ] | somme : 0.183
axe 1, lignes   : {'Réseaux': -0.4, 'Boutique': 0.457}
axe 1, colonnes : {'court': -0.535, 'moyen': 0.067, 'long': 0.568}

```

Chapitre 4 : Séries temporelles — exercices et applications

Ce chapitre du cahier accompagne le chapitre 4 du livre. Il contient **quatre applications guidées** (prévoir 2026, programmer le filtre de Kalman, écrire un modèle à la Prophet, valider à origine glissante) et **treize exercices corrigés**. Les données sont celles du livre : `donnees/ventes_mensuelles.csv` (120 mois de chiffre d'affaires, de nombre de commandes, avec les indicatrices promo et covid). Cherchez d'abord à la main ou sur papier, vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Le chapitre est **autonome** : le bloc suivant prépare les données et les imports communs à tout ce qui suit.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
y = np.log(v["ca"])
X = v[["promo", "covid"]].astype(float)
train, test = y[:"2023-12"], y["2024-01":]
```

on modélise le logarithme du chiffre d'affaires
variables explicatives connues
96 mois pour apprendre, 24 mois de test

Applications

Application 4.1 — Les ventes de 2026

Contexte. La gérante prépare son budget de 2026. Elle veut la prévision **mois par mois**, avec une fourchette, et le **total de l'année** avec son incertitude. Nous reprenons le modèle retenu à la section 4.3 du livre, le SARIMAX(1,0,0)(0,1,1)₁₂ avec tendance linéaire et les variables promo et covid, que nous ajustons cette fois sur les **120** mois disponibles.

Étape 1 : prévoir 2026 mois par mois. Un modèle avec variables explicatives exige leurs valeurs futures : nous supposons aucune crise (covid nul) et une promotion en décembre (décidée par la gérante).

```
X_2026 = pd.DataFrame({"promo": [0.0] * 11 + [1.0], "covid": [0.0] * 12},
                      index=pd.date_range("2026-01-01", periods=12, freq="MS"))
final = SARIMAX(y, exog=X, order=(1, 0, 0), seasonal_order=(0, 1, 1, 12), trend="ct").fit(dispatch=False,
↪ maxiter=300)
pf = final.get_forecast(12, exog=X_2026)
ic = np.asarray(pf.conf_int(alpha=0.05)) # intervalle à 95 %, en logarithme
resume = pd.DataFrame({"prévision (€)": np.exp(pf.predicted_mean.values), "IC95 bas": np.exp(ic[:,
↪ 0]), "IC95 haut": np.exp(ic[:, 1])},
                      index=X_2026.index.strftime("%Y-%m")).round(0).astype(int)
print(resume.to_string())
```

	prévision (€)	IC95 bas	IC95 haut
2026-01	1414	1214	1648
2026-02	1694	1434	2001
2026-03	2130	1799	2522
2026-04	2207	1863	2614
2026-05	2495	2106	2956
2026-06	2881	2431	3413
2026-07	2864	2418	3393
2026-08	2637	2226	3124
2026-09	2067	1745	2449
2026-10	1873	1581	2219
2026-11	2602	2197	3083
2026-12	4195	3541	4970

On revient à l'échelle des euros par l'exponentielle (section 4.3.2 du livre : on obtient la **médiane**, et les bornes se transforment sans difficulté). Le creux de janvier, le plateau d'été et le pic de décembre ressortent clairement.

Étape 2 : le total de l'année, par simulation. Les erreurs des douze mois sont **corrélées** : on ne peut pas simplement additionner les bornes mensuelles. On tire donc 2 000 trajectoires futures plausibles du modèle (en tirant nous-mêmes l'état initial et les chocs, pour que le résultat soit reproductible) et l'on additionne les 12 mois de chacune.

```
a_fin = final.predicted_state[:, -1] # loi de l'état caché juste après la dernière
↪ observation
P_fin = final.predicted_state_cov[:, :, -1]
sigma = np.sqrt(final.params["sigma2"])

def une_trajetoire(graine):
    rng = np.random.default_rng(graine)
    etat0 = rng.multivariate_normal(a_fin, P_fin, method="eigh")
    chocs = rng.normal(0, sigma, size=(12, 1))
    return final.simulate(12, measurement_shocks=np.zeros((12, 1)), state_shocks=chocs,
↪ initial_state=etat0,
                           anchor="end", exog=X_2026).values

totaux = np.exp(np.array([une_trajetoire(k) for k in range(2000)]).T).sum(axis=0)
total_2025 = v["ca"]["2025-01":"2025-12"].sum()
p10, p50, p90 = np.percentile(totaux, [10, 50, 90])
p025, p975 = np.percentile(totaux, [2.5, 97.5])
```

```
print(f"total 2025 observé : {total_2025:,.0f} €")
print(f"total 2026 prévu : médiane {p50:,.0f} € | intervalle à 80 % [{p10:,.0f} ; {p90:,.0f}] | à 95 % [{p025:,.0f} ; {p975:,.0f}]" )
print(f"croissance prévue sur 2025 : {100 * (p50 / total_2025 - 1):+.1f} %")
```

```
total 2025 observé : 27,630 €
total 2026 prévu : médiane 29,169 € | intervalle à 80 % [27,823 ; 30,597] | à 95 % [27,073 ; 31,397]
croissance prévue sur 2025 : +5.6 %
```

La fourchette à 80 % est un bon outil de budget : « dans huit cas sur dix, l'année se situera entre ces deux montants ». Elle suppose que la structure observée de 2016 à 2025 se prolonge.

Étape 3 : un scénario. Que devient décembre si la gérante renonce à la promotion ? Il suffit de changer la variable explicative future.

```
X_sans = X_2026.copy()
X_sans.loc["2026-12-01", "promo"] = 0.0
sans = final.get_forecast(12, exog=X_sans).predicted_mean
avec = pf.predicted_mean
print(f"décembre 2026 : {np.exp(avec.iloc[-1]):,.0f} € avec promotion, {np.exp(sans.iloc[-1]):,.0f} € ↪ sans")
print(f"rapport sans / avec : {np.exp(sans.iloc[-1] - avec.iloc[-1]):.3f}")
```

```
décembre 2026 : 4,195 € avec promotion, 3,896 € sans
rapport sans / avec : 0.929
```

À vous. (1) Refaites l'étape 1 en supposant **deux** promotions (juin et décembre). (2) Un nouveau choc de type COVID de trois mois en 2026 est-il dans les intervalles ? Que faudrait-il changer pour le traiter ? (3) Comparez la médiane annuelle à celle obtenue par le naïf saisonnier avec dérive (section 4.3.4 du livre).

Application 4.2 — Le filtre de Kalman en numpy

Contexte. Le livre (section 4.5) décrit le filtre de Kalman d'un niveau local : une croyance $\mathcal{N}(a, P)$ sur le niveau, qui alterne **prédiction** (l'incertitude grandit) et **mise à jour** (on corrige d'une fraction K de la surprise). Vous allez le programmer, le comparer à statsmodels, estimer des variances, et retrouver le lissage exponentiel.

Étape 1 : la fonction. Elle traite aussi les observations manquantes (NaN) : dans ce cas, on prédit sans mettre à jour. Elle cumule la log-vraisemblance gaussienne des innovations.

```
def kalman_niveau_local(y, s2_eps, s2_eta, a0, P0):
    """Filtre de Kalman du niveau local. (a0, P0) = croyance sur le niveau AVANT la première
    ↪ observation."""
    n = len(y)
    sortie = {k: np.zeros(n) for k in ["a_pred", "P_pred", "F", "K", "v", "a_filtre", "P_filtre"]}
    a, P, loglik = a0, P0, 0.0
    for t in range(n):
        sortie["a_pred"][t], sortie["P_pred"][t] = a, P
        if np.isnan(y[t]):
            # observation manquante : pas de mise à jour
            sortie["F"][t] = sortie["K"][t] = sortie["v"][t] = np.nan
            a_f, P_f = a, P
        else:
            F = P + s2_eps
            v_t = y[t] - a
            K = P / F
            a_f, P_f = a + K * v_t, P * (1 - K)
            loglik += -0.5 * (np.log(2 * np.pi * F) + v_t ** 2 / F)
            sortie["F"][t], sortie["K"][t], sortie["v"][t] = F, K, v_t
        sortie["a_filtre"][t], sortie["P_filtre"][t] = a_f, P_f
```

```

a, P = a_f, P_f + s2_eta                                # prédiction pour la date suivante
sortie["loglik"] = loglik
return sortie

```

Étape 2 : la vérifier. Sur l'exemple à la main du livre ($\sigma_\varepsilon^2 = 4$, $\sigma_\eta^2 = 1$, croyance initiale $\mathcal{N}(100, 5)$, observations 103, 101, 106), puis contre statsmodels.

```

from statsmodels.tsa.statespace.structural import UnobservedComponents

obs = np.array([103.0, 101.0, 106.0])
r = kalman_niveau_local(obs, s2_eps=4.0, s2_eta=1.0, a0=100.0, P0=5.0)
print(pd.DataFrame({k: r[k] for k in ["a_pred", "P_pred", "F", "K", "v", "a_filtre", "P_filtre"]},
    ↪ index=[1, 2, 3]).round(4).to_string())

mod = UnobservedComponents(obs, level="llevel")
mod.ssm.initialize_known(np.array([100.0]), np.array([[5.0]]))
mod.ssm.loglikelihood_burn = 0                                # par défaut, statsmodels ignore le premier terme de
    ↪ la vraisemblance
res_sm = mod.smooth([4.0, 1.0])                              # variances : irrégulière (4), niveau (1)
print("\nétat filtré statsmodels :", res_sm.filtered_state[0].round(4))
print("log-vraisemblance : numpy =", round(r["loglik"], 4), "| statsmodels =", round(res_sm.llf, 4))

```

	a_pred	P_pred	F	K	v	a_filtre	P_filtre
1	100.0000	5.0000	9.0000	0.5556	3.0000	101.6667	2.2222
2	101.6667	3.2222	7.2222	0.4462	-0.6667	101.3692	1.7846
3	101.3692	2.7846	6.7846	0.4104	4.6308	103.2698	1.6417

```

état filtré statsmodels : [101.6667 101.3692 103.2698]
log-vraisemblance : numpy = -7.9124 | statsmodels = -7.9124

```

Étape 3 : estimer les variances. Sur un niveau local simulé de 200 points (variances programmées : 4 pour le bruit d'observation, 1 pour le niveau), on les estime par maximum de vraisemblance.

```

rng = np.random.default_rng(5)
n = 200
niveau = 100 + np.cumsum(rng.normal(0, 1.0, n))              # sigma_eta = 1
y_sim = niveau + rng.normal(0, 2.0, n)                       # sigma_eps = 2
ajust = UnobservedComponents(y_sim, level="llevel").fit(dis=False)
print(pd.DataFrame({"programmé": [4.0, 1.0], "estimé": ajust.params, "erreur type": ajust.bse},
    ↪ index=["variance du bruit d'observation", "variance du
    ↪ niveau])).round(3).to_string())

```

	programmé	estimé	erreur type
variance du bruit d'observation	4.0	3.051	0.445
variance du niveau	1.0	1.453	0.382

Étape 4 : le régime permanent et le lissage exponentiel. Le gain K_t se stabilise ; une fois constant, le filtre est un lissage exponentiel de paramètre $\alpha = K_\infty$. Pour $\sigma_\varepsilon^2 = 4$ et $\sigma_\eta^2 = 1$, la variance prédite stationnaire vaut $p = (1 + \sqrt{17})/2$ et $K_\infty = p/(p + 4)$.

```

r = kalman_niveau_local(y_sim, 4.0, 1.0, a0=100.0, P0=25.0)
p = (1 + np.sqrt(17)) / 2
alpha = p / (p + 4)
print("gain final du filtre :", round(r["K"][-1], 5), "| théorie p/(p+4) :", round(alpha, 5))

lisse = np.zeros(n)
lisse[0] = r["a_filtre"][0]
for t in range(1, n):
    lisse[t] = alpha * y_sim[t] + (1 - alpha) * lisse[t - 1]
print("écart maximal lissage exponentiel / filtre, après 30 pas :", f"{np.abs(lisse[30:] -
    ↪ r['a_filtre'][30:]).max():.1e}")

```

```
gain final du filtre : 0.39039 | théorie p/(p+4) : 0.39039
écart maximal lissage exponentiel / filtre, après 30 pas : 2.2e-07
```

À vous. (1) Effacez 20 observations consécutives de `y_sim` (remplacez-les par `np.nan`) et regardez comment l'incertitude `P_filtre` évolue dans le trou. (2) Faites varier le rapport $q = \sigma_\eta^2 / \sigma_\varepsilon^2$ et tracez K_∞ en fonction de q .

Application 4.3 — Un modèle « à la Prophet » écrit à la main

Contexte. Prophet (section 4.6 du livre) est un modèle additif : tendance linéaire par morceaux, saisonnalité de Fourier, régresseurs. Nous ne pouvons pas l'installer ici, mais le même modèle s'écrit comme une **régression pénalisée** : les moindres carrés avec un terme $\lambda \sum \delta_j^2$ sur les changements de pente. Vous allez le construire et choisir ses deux réglages (le nombre K d'harmoniques et la pénalité λ) **sur l'apprentissage seul**.

Étape 1 : la matrice de dessin. Colonnes non pénalisées : constante, temps, harmoniques de Fourier, promo, covid ; colonnes pénalisées : un changement de pente à chacune des 20 dates de rupture candidates.

```
temps = np.arange(120.0)                                # 0 = janvier 2016
num_mois = v.index.month.values.astype(float)
Xv = X.values

def colonnes(i_t, i_mois, K, ruptures, promo, covid):
    t = i_t / 120.0
    fourier = []
    for k in range(1, K + 1):
        for f in (np.sin, np.cos):
            c = f(2 * np.pi * k * i_mois / 12)
            if np.abs(c).max() > 1e-9:                    # pour k = 6, sin(pi m) est nul : on
                ↪ l'écarte
                fourier.append(c)
    base = [np.ones_like(t), t] + fourier + [promo, covid]
    sauts = [np.maximum(0.0, t - s / 120.0) for s in ruptures]
    return np.column_stack(base + sauts), len(base)      # (matrice, nombre de colonnes NON
    ↪ pénalisées)
```

Étape 2 : l'ajustement pénalisé et la prévision. On empile $\sqrt{\lambda}$ fois la pénalité sous la matrice de dessin : les moindres carrés ordinaires de la matrice empilée sont les moindres carrés pénalisés (méthode stable, sans inverser de matrice).

```
def ajuster_predire(y_app, i_app, i_prev, m_app, m_prev, X_app, X_prev, K, lam, n_rupt=20):
    ruptures = np.linspace(6, 0.85 * len(i_app), n_rupt) # points de rupture candidats
    A, nb = colonnes(i_app, m_app, K, ruptures, X_app[:, 0], X_app[:, 1])
    B, _ = colonnes(i_prev, m_prev, K, ruptures, X_prev[:, 0], X_prev[:, 1])
    pen = np.zeros(A.shape[1])
    pen[nb:] = 1.0                                     # on ne pénalise que les changements de pente
    A_aug = np.vstack([A, np.sqrt(lam) * np.diag(pen)])
    y_aug = np.r_[y_app, np.zeros(A.shape[1])]
    beta = np.linalg.lstsq(A_aug, y_aug, rcond=None)[0]
    return B @ beta, A @ beta
```

Étape 3 : choisir K et λ par validation. On apprend sur les 72 premiers mois et on évalue sur les 24 suivants (le test de 2024-2025 reste intact).

```
import itertools
lignes = []
for K, lam in itertools.product((1, 2, 3, 4, 5, 6), (0.001, 0.01, 0.1, 1, 10)):
    f, _ = ajuster_predire(train.values[:72], temps[:72], temps[72:96], num_mois[:72],
    ↪ num_mois[72:96], Xv[:72], Xv[72:96], K, lam)
```

```
lignes.append({"K": K, "lambda": lam, "RMSE": np.sqrt(np.mean((train.values[72:96] - f) ** 2))})
grille = pd.DataFrame(lignes)
print(grille.pivot(index="K", columns="lambda", values="RMSE").round(3).to_string())
meilleur = grille.sort_values("RMSE").iloc[0]
K_opt, lam_opt = int(meilleur["K"]), float(meilleur["lambda"])
print(f"\nréglage retenu : K = {K_opt}, lambda = {lam_opt} (RMSE de validation :
↳ {meilleur['RMSE']:.3f})")
```

lambda	0.001	0.010	0.100	1.000	10.000
K					
1	0.386	0.347	0.323	0.307	0.293
2	0.294	0.284	0.277	0.271	0.260
3	0.183	0.190	0.197	0.199	0.192
4	0.133	0.125	0.133	0.145	0.141
5	0.139	0.096	0.099	0.112	0.110
6	0.153	0.094	0.090	0.104	0.103

réglage retenu : K = 6, lambda = 0.1 (RMSE de validation : 0.090)

Étape 4 : ajuster sur les 96 mois et prévoir le test.

```
f_test, ajuste = ajuster_predire(train.values, temps[:96], temps[96:], num_mois[:96], num_mois[96:],
↳ Xv[:96], Xv[96:], K_opt, lam_opt)
print("RMSE (log) sur les 24 mois de test :", round(np.sqrt(np.mean((test.values - f_test) ** 2)), 4),
      "| biais (y - prévision) :", round(float(np.mean(test.values - f_test)), 4))
print("RMSE d'ajustement sur l'apprentissage :", round(np.sqrt(np.mean((train.values - ajuste) **
↳ 2)), 4))
```

RMSE (log) sur les 24 mois de test : 0.0801 | biais (y - prévision) : 0.0173
 RMSE d'ajustement sur l'apprentissage : 0.0679

À vous. (1) Pourquoi $K = 6$ ne laisse-t-il « rien à lisser » sur des données mensuelles ? (Combien de paramètres saisonniers cela fait-il ?) (2) Supprimez la pénalité ($\lambda=0$) : que devient la prévision ? (3) Remplacez la pénalité par $\lambda \sum |\delta_j|$ (comme le Lasso du volume II, section 1.5) en vous aidant de `sklearn.linear_model.Lasso`.

Application 4.4 — La validation à origine glissante

Contexte. Un seul découpage de 24 mois donne un classement bruité. La **validation à origine glissante** (section 4.3.6 du livre) recommence plusieurs fois en avançant l'origine : à chaque date T , on ajuste sur les données jusqu'à T , on prévoit les 12 mois suivants, on mesure l'erreur. Vous allez comparer le naïf saisonnier, le naïf avec dérive et deux modèles de la section 4.2.

Étape 1 : les prévisionnistes. Chaque fonction prend l'origine i (nombre de mois connus) et renvoie la prévision des 12 mois suivants, en logarithme.

```
H = 12
origines = list(range(96, 109)) # 13 origines : fin décembre 2023, ..., fin
↳ décembre 2024

def prev_naif(i): # même mois l'an dernier
    return y.iloc[i - 12:i - 12 + H].values

def prev_derive(i): # idem, plus la croissance annuelle moyenne
    tr = y.iloc[:i]
    gi = (tr - tr.shift(12)).mean()
    return np.array([tr.iloc[-12 + (h % 12)] + gi * (1 + h // 12) for h in range(H)])

def prev_modele_A(i):
    m = SARIMAX(y.iloc[:i], exog=X.iloc[:i], order=(1, 1, 1), seasonal_order=(0, 1, 1,
↳ 12)).fit(dispatch=False)
```

```

    return m.forecast(H, exog=X.iloc[i:i + H]).values

def prev_modele_B(i):
    m = SARIMAX(y.iloc[:i], exog=X.iloc[:i], order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="ct").fit(dispatch=False, maxiter=300)
    return m.forecast(H, exog=X.iloc[i:i + H]).values

```

Étape 2 : le concours. On calcule les prévisions à chaque origine, puis l'erreur (RMSE en logarithme) globale et aux horizons 1 et 12.

```

fonctions = {"naïf saisonnier": prev_naif, "naïf saisonnier + dérive": prev_derive,
             "A : SARIMAX(1,1,1)(0,1,1)": prev_modele_A, "B : (1,0,0)(0,1,1) + tendance":
             ↪ prev_modele_B}
P = {nom: np.array([f(i) for i in origines]) for nom, f in fonctions.items()}      # forme (13
    ↪ origines, 12 horizons)
reel = np.array([y.iloc[i:i + H].values for i in origines])

lignes = []
for nom, f in P.items():
    e = reel - f
    lignes.append({"modèle": nom, "RMSE global": np.sqrt((e ** 2).mean()), "h = 1": np.sqrt((e[:, 0]
    ↪ ** 2).mean()),
                  "h = 12": np.sqrt((e[:, 11] ** 2).mean()), "biais": e.mean()})
print(pd.DataFrame(lignes).set_index("modèle").sort_values("RMSE global").round(4).to_string())

```

modèle	RMSE global	h = 1	h = 12	biais
B : (1,0,0)(0,1,1) + tendance	0.0670	0.0712	0.0889	0.0037
A : SARIMAX(1,1,1)(0,1,1)	0.0763	0.0737	0.0890	-0.0416
naïf saisonnier + dérive	0.0839	0.0753	0.1052	-0.0175
naïf saisonnier	0.1081	0.1213	0.1264	0.0707

Étape 3 : une combinaison. La moyenne des prévisions de A et B est-elle meilleure que chacune ?

```

comb = (P["A : SARIMAX(1,1,1)(0,1,1)"] + P["B : (1,0,0)(0,1,1) + tendance"]) / 2
print("RMSE de la combinaison A+B :", round(np.sqrt(((reel - comb) ** 2).mean()), 4))

```

RMSE de la combinaison A+B : 0.0662

À vous. (1) Ajoutez le modèle C de la section 4.2.7 du livre (régression sur tendance, mois et variables explicatives, erreurs AR(1)). (2) Les origines choisies sont celles de la période de test : quelle précaution le livre recommande-t-il ? (3) Tracez le RMSE selon l'horizon pour chaque méthode.

Exercices

Exercice 4.1 ★ — Autocorrélation à la main (section 4.1.4 du livre)

Six chiffres d'affaires hebdomadaires (en centaines de €) : 8, 6, 9, 5, 7, 4. Calculez à la main les autocorrélations r_1 et r_2 . Quel signe attendiez-vous pour r_1 en regardant la série, et pourquoi ?

Exercice 4.2 ★ — Stationnaire ou non ? (section 4.1.3 du livre)

Pour chaque processus (ε_t est un bruit blanc), dites s'il est stationnaire, et si non, quel remède appliquer : (a) $Y_t = 5 + \varepsilon_t$; (b) $Y_t = Y_{t-1} + \varepsilon_t$; (c) $Y_t = 0,9 Y_{t-1} + \varepsilon_t$; (d) $Y_t = 2t + \varepsilon_t$; (e) $Y_t = 1,1 Y_{t-1} + \varepsilon_t$.

Exercice 3 ★ (prévoir avec un AR(1)). Un processus suit $Y_t - 10 = 0,8(Y_{t-1} - 10) + \varepsilon_t$ avec $\sigma = 2$. (a) Quelle est sa variance, et les autocorrélations $\rho(1), \rho(2), \rho(3)$? (b) On observe $Y_T = 14$: donnez les prévisions à 1, 2 et 3 pas, avec les demi-largeurs des intervalles à 95 %.

Exercice 4 ★★ (MA(1) et inversibilité). (a) Calculez $\rho(1)$ pour $\theta = 0,8$, puis pour $\theta = 1,25$. Que remarquez-vous ? (b) Montrez qu'un MA(1) ne peut jamais avoir $|\rho(1)| > 0,5$. Si vous observez $r_1 = 0,6$ sur une longue série, que concluez-vous ?

Exercice 4.5 ★★ — Ljung-Box à la main (section 4.1.5 du livre)

Sur $n = 50$ résidus, on trouve $r_1 = 0,30$ et $r_2 = 0,20$. Calculez $Q(2)$ et sa p-valeur (indice : pour 2 degrés de liberté, $P(\chi_2^2 > x) = e^{-x/2}$). Que concluez-vous ?

Exercice 4.6 ★★ — Du logarithme aux euros (section 4.3.2 du livre)

Un modèle prévoit $\log(\text{ca}) = 7,60$ pour décembre, avec une erreur type de prévision de 0,08. Donnez la prévision **médiane** en euros, la prévision de la **moyenne**, et l'intervalle de prévision à 95 %.

Exercice 4.7 ★★ — MASE et MAPE (section 4.3.4 du livre)

Sur l'apprentissage, le naïf saisonnier a une MAE de 150 €. Le modèle A a une MAE de 120 € sur le test et le modèle B de 160 €. (a) Calculez les MASE. (b) Pourquoi le MAPE est-il dangereux quand une valeur réelle est nulle ? (c) Pourquoi favorise-t-il les prévisions trop basses ?

Exercice 4.8 ★★ — Identifier un modèle (section 4.2.1 et 4.2.4 du livre)

Trois séries de 300 points, X , Y et Z , ont été simulées avec des modèles ARMA différents. Voici leurs autocorrélations et autocorrélations partielles. **Identifiez** pour chacune le modèle le plus probable (type et ordre), puis vérifiez avec les critères d'information.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.arima_process import ArmaProcess
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import acf, pacf

mystere = {"X": ([1, -0.75], [1]), "Y": ([1], [1, 0.7, 0.5]), "Z": ([1, -0.5, -0.3], [1])}
series = {nom: ArmaProcess(ar, ma).generate_sample(300, distrvs=np.random.default_rng(100 +
    ↵ j).standard_normal)
    for j, (nom, (ar, ma)) in enumerate(mystere.items())}
bande = 1.96 / np.sqrt(300)
print("bande de confiance : +/-", round(bande, 3))
for nom, s in series.items():
    print(f"série {nom} : ACF décalages 1-6 :", acf(s, nlags=6, fft=False)[1:].round(2))
    print(f"          PACF décalages 1-6 :", pacf(s, nlags=6, method="ols")[1:].round(2))
```

```

bande de confiance : +/- 0.113
série X : ACF décalages 1-6 : [0.73 0.58 0.43 0.28 0.2 0.12]
          PACF décalages 1-6 : [ 0.74 0.1 -0.05 -0.1 0.03 -0.03]
série Y : ACF décalages 1-6 : [ 0.6 0.17 -0.19 -0.25 -0.23 -0.17]
          PACF décalages 1-6 : [ 0.6 -0.28 -0.27 0.1 -0.12 -0.12]
série Z : ACF décalages 1-6 : [0.69 0.57 0.48 0.36 0.29 0.28]
          PACF décalages 1-6 : [ 0.69 0.19 0.08 -0.07 -0. 0.09]

```

Exercice 4.9 ★★★ — La sur-différenciation, en théorie (section 4.1.6 du livre)

Soit Y_t un AR(1) stationnaire de coefficient φ , et $W_t = Y_t - Y_{t-1}$. (a) Calculez $\text{Var}(W_t)$ et $\text{Cov}(W_t, W_{t-1})$ en fonction de $\gamma(0)$ et de φ , puis montrez que $\rho_W(1) = -\frac{1-\varphi}{2}$. (b) Que vaut ce résultat pour $\varphi = 0$? pour $\varphi \rightarrow 1$? Interprétez. (c) Vérifiez par simulation pour $\varphi = 0,5$ et $\varphi = 0,9$.

Exercice 4.10 ★★★ — Filtre de Kalman à la main (section 4.5.2 du livre)

Niveau local de variances $\sigma_\varepsilon^2 = 9$ (observation) et $\sigma_\eta^2 = 3$ (niveau), croyance initiale $\mathcal{N}(50, 12)$ avant la première mesure. On observe $y_1 = 56$ puis $y_2 = 52$. (a) Calculez à la main les gains K_1, K_2 , les niveaux filtrés $a_{1|1}, a_{2|2}$ et leurs variances. (b) Quel est le gain en régime permanent ? (c) À quel paramètre de lissage exponentiel cela correspond-il ?

Exercice 4.11 ★★★ — Régression fallacieuse et remède (section 4.4.2 du livre)

Deux marches aléatoires indépendantes de 200 points sont régressées l'une sur l'autre. (a) Quelle part de régressions « significatives » à 5 % attendez-vous, en niveaux ? (b) Montrez par simulation que la régression **des différences** Δa_t sur Δb_t redonne le bon taux de 5 %. (c) Pourquoi ce remède fonctionne-t-il, et quelle information perd-on ?

Exercice 4.12 ★★ — GARCH à la main (section 4.4.3 du livre)

Un GARCH(1,1) a pour paramètres $\omega = 0,1, \alpha = 0,2, \beta = 0,7$. (a) Quelle est la variance de long terme ? (b) Hier, $\varepsilon_{t-1} = 2$ et $\sigma_{t-1}^2 = 1,5$: que vaut σ_t^2 ? (c) Quelle est la prévision de σ_{t+1}^2 ? (d) En combien de pas l'excès de variance (au-dessus de la variance de long terme) est-il divisé par deux ?

Exercice 4.13 ★★★ — Le concours complet sur une autre série (section 4.3 du livre)

Appliquez la démarche de 4.3 à la série nb_commandes (nombre de commandes par mois) : travaillez en logarithme, apprenez sur 2016-2023, prévoyez 2024-2025 avec (i) le naïf saisonnier avec dérive et (ii) un SARIMAX(1, 0, 0)(0, 1, 1)₁₂ avec tendance et variables promo et covid. Calculez la MASE (en nombre de commandes) de chacun. Le modèle bat-il la référence ? Que pensez-vous de la précision atteignable sur cette série, plus bruitée que le chiffre d'affaires ?

Corrigés

Corrigé 4.1

Moyenne = $39/6 = 6,5$; écarts : 1,5 ; -0,5 ; 2,5 ; -1,5 ; 0,5 ; -2,5 ; somme des carrés : $2,25 + 0,25 + 6,25 + 2,25 + 0,25 + 6,25 = 17,5$. **Décalage 1** : produits $(-0,5)(1,5) = -0,75$; $(2,5)(-0,5) = -1,25$; $(-1,5)(2,5) = -3,75$; $(0,5)(-1,5) = -0,75$; $(-2,5)(0,5) = -1,25$; somme -7,75, donc $r_1 = -7,75/17,5 \approx -0,443$. **Décalage 2** : $(2,5)(1,5) = 3,75$; $(-1,5)(-0,5) = 0,75$; $(0,5)(2,5) = 1,25$; $(-2,5)(-1,5) = 3,75$; somme 9,5, donc $r_2 \approx 0,543$. On attendait un r_1 **négalif** : la série fait du « zigzag » (haut, bas, haut, bas) autour de sa moyenne, et un zigzag correspond à une corrélation négative entre deux mois consécutifs et positive à deux mois d'écart. Avec seulement six points, ces valeurs sont très incertaines (bande de $\pm 0,8$).

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd

def acf_manuel(x, k):
    d = np.asarray(x, float) - np.mean(x)
    return (d[:len(d) - k] * d[k:]).sum() / (d ** 2).sum()

x = [8, 6, 9, 5, 7, 4]
print("r1 =", round(acf_manuel(x, 1), 4), "| r2 =", round(acf_manuel(x, 2), 4), "| bande
↳ 1,96/racine(6) =", round(1.96 / np.sqrt(6), 2))
```

```
r1 = -0.4429 | r2 = 0.5429 | bande 1,96/racine(6) = 0.8
```

Corrigé 4.2

- (a) **Stationnaire** : moyenne 5, variance σ^2 , pas de mémoire. (b) **Marche aléatoire** : non stationnaire ($\text{Var} = t\sigma^2$) ; remède : **différencier**. (c) **Stationnaire** ($|\rho| = 0,9 < 1$), mais très persistant : la mémoire décroît lentement ($\rho(k) = 0,9^k$), et sur un échantillon court elle se confond facilement avec une racine unitaire (c'est le manque de puissance de 4.1.6). (d) **Non stationnaire** : la moyenne $2t$ varie ; remède : **retirer la tendance** (régression sur t), car la tendance est déterministe, pas stochastique (différencier fonctionnerait aussi, mais surdifférencierait). (e) **Explosif** ($|\rho| > 1$) : non stationnaire, aucune transformation simple ne le « répare » ; c'est rare en pratique.

```
rng = np.random.default_rng(3)
n = 400
e = rng.normal(size=n)
def generer(phi, tendance=0.0, depart=0.0):
    y = np.zeros(n); y[0] = depart
    for t in range(1, n):
        y[t] = phi * y[t - 1] + e[t] + tendance * t
    return y
cas = {"(a) 5 + bruit": 5 + e, "(b) marche aléatoire": generer(1.0), "(c) AR(1), phi = 0,9":
↳ generer(0.9),
      "(d) 2t + bruit": 2 * np.arange(n) + e}
lignes = [{"processus": k, "moyenne 1re moitié": s[:n // 2].mean(), "moyenne 2e moitié": s[n //
↳ 2:].mean(),
          "écart-type 1re moitié": s[:n // 2].std(), "écart-type 2e moitié": s[n // 2:].std()} for k,
↳ s in cas.items()]
print(pd.DataFrame(lignes).round(1).to_string(index=False))
```

processus	moyenne 1re moitié	moyenne 2e moitié	écart-type 1re moitié	écart-type 2e moitié
(a) 5 + bruit	5.0	5.0	1.0	1.0

(b) marche aléatoire	-2.8	7.4	4.3	4.1
(c) AR(1), $\phi = 0,9$	0.3	0.2	2.2	2.3
(d) $2t + \text{bruit}$	199.0	599.0	115.6	115.5

La simulation montre que les moyennes diffèrent nettement d'une moitié à l'autre pour (b) et (d), alors que (a) et (c) se ressemblent. (Pour la marche aléatoire (b), c'est la variance **à travers les trajectoires** qui croît avec t (4.1.3) ; sur une seule trajectoire, comparer les écarts-types de deux moitiés n'est pas probant, et le tableau ne le montre d'ailleurs pas. C'est le déplacement de la moyenne qui trahit la non-stationnarité.)

Corrigé 4.3

(a) $\gamma(0) = \frac{\sigma^2}{1 - \phi^2} = \frac{4}{1 - 0,64} = 11,11$ (écart-type 3,33) ; $\rho(1) = 0,8$, $\rho(2) = 0,64$, $\rho(3) = 0,512$.

(b) $\hat{y}_{T+h} = 10 + 0,8^h \times (14 - 10) : 13,2 ; 12,56 ; 12,048$. Demi-largeurs : $1,96 \sigma \sqrt{\sum_{j < h} \phi^{2j}} : h = 1 : 1,96 \times 2 = 3,92 ; h = 2 : 3,92 \times \sqrt{1,64} = 5,02 ; h = 3 : 3,92 \times \sqrt{1,64 + 0,4096} = 5,61$. Limite : $1,96 \times 3,33 = 6,53$.

```
from statsmodels.tsa.arima.model import ARIMA
phi, mu, sigma, yT = 0.8, 10.0, 2.0, 14.0
print("variance :", round(sigma ** 2 / (1 - phi ** 2), 2), "| rho(1..3) :", [round(phi ** k, 3) for k
    in (1, 2, 3)])
print("prévisions :", [round(mu + phi ** h * (yT - mu), 3) for h in (1, 2, 3)])
print("demi-largeurs :", np.round([1.96 * sigma * np.sqrt(sum(phi ** (2 * j) for j in range(h))) for h
    in (1, 2, 3)], 2),
    "| limite :", round(1.96 * sigma / np.sqrt(1 - phi ** 2), 2))
# vérification avec statsmodels (paramètres fixés : constante, phi, sigma2)
fixe = ARIMA(np.r_[np.full(50, mu), yT], order=(1, 0, 0), trend="c").filter([mu, phi, sigma ** 2])
p = fixe.get_forecast(3)
print("statsmodels :", p.predicted_mean.round(3), ((p.conf_int()[:, 1] - p.conf_int()[:, 0]) /
    2).round(2))
```

```
variance : 11.11 | rho(1..3) : [0.8, 0.64, 0.512]
prévisions : [13.2, 12.56, 12.048]
demi-largeurs : [3.92 5.02 5.61] | limite : 6.53
statsmodels : [13.2 12.56 12.048] [3.92 5.02 5.61]
```

Corrigé 4.4

(a) $\rho(1) = \frac{0,8}{1 + 0,64} = 0,488$ et $\rho(1) = \frac{1,25}{1 + 1,5625} = 0,488$: **les deux valeurs donnent la même autocorrélation** (θ et $1/\theta$). On retient la valeur **inversible** $\theta = 0,8$. (b) $1 + \theta^2 \geq 2|\theta|$ (car $(1 - |\theta|)^2 \geq 0$), donc $|\rho(1)| = \frac{|\theta|}{1 + \theta^2} \leq \frac{1}{2}$, avec égalité pour $\theta = \pm 1$. Si l'on observe $r_1 = 0,6$ sur une longue série (assez longue pour que 0,6 ne soit pas du bruit d'échantillonnage), **ce n'est pas un MA(1)** : il faut un AR (dont la mémoire peut produire une autocorrélation d'ordre 1 proche de 1), ou un ARMA.

```
from statsmodels.tsa.arima_process import arma_acf
for th in (0.8, 1.25):
    print(f"theta = {th} : rho(1) = {arma_acf([1], [1, th], 2)[1]:.4f}")
thetas = np.linspace(-5, 5, 100001)
rho1 = thetas / (1 + thetas ** 2)
print("maximum de |rho(1)| sur theta dans [-5, 5] :", round(np.abs(rho1).max(), 4), "atteint pour
    theta =", round(abs(thetas[np.abs(rho1).argmax()]), 3))
```

```
theta = 0.8 : rho(1) = 0.4878
theta = 1.25 : rho(1) = 0.4878
maximum de |rho(1)| sur theta dans [-5, 5] : 0.5 atteint pour theta = 1.0
```

Corrigé 4.5

$$Q(2) = n(n+2) \left[\frac{r_1^2}{n-1} + \frac{r_2^2}{n-2} \right] = 50 \times 52 \times \left[\frac{0,09}{49} + \frac{0,04}{48} \right] = 2600 \times (0,001837 + 0,000833) = 6,94.$$

Pour 2 degrés de liberté, $p = e^{-6,94/2} = e^{-3,47} \approx 0,031$. **p < 0,05** : on rejette l'hypothèse « bruit blanc » ; le modèle n'a pas tout expliqué. (Si les résidus provenaient d'un modèle avec deux paramètres AR/MA estimés, il faudrait comparer à un khi-deux à $2 - 2 = 0$ degré de liberté : le test serait inutilisable à 2 retards ; on utiliserait plus de retards.)

```
from scipy import stats
n, r = 50, np.array([0.30, 0.20])
Q = n * (n + 2) * np.sum(r ** 2 / (n - np.arange(1, 3)))
print("Q(2) =", round(Q, 3), "| p-valeur =", round(np.exp(-Q / 2), 4), "| via scipy :", round(1 -
↳ stats.chi2.cdf(Q, 2), 4))
```

Q(2) = 6.942 | p-valeur = 0.0311 | via scipy : 0.0311

Corrigé 4.6

La **médiane** est $e^{7,60} \approx 1\,998$ €. La **moyenne** vaut $e^{7,60+0,08^2/2} = e^{7,6032} \approx 2\,005$ € (un facteur $e^{0,0032} \approx 1,003$: négligeable). L'intervalle à 95 % : $\exp(7,60 \pm 1,96 \times 0,08) = \exp(7,60 \pm 0,1568) = [1\,708 ; 2\,337]$ € : environ ± 16 % autour de la médiane, de façon **asymétrique** en euros (l'intervalle est un peu plus étendu vers le haut : +339 € contre -290 €).

```
m, s = 7.60, 0.08
print("médiane :", round(np.exp(m), 1), "| moyenne :", round(np.exp(m + s ** 2 / 2), 1))
print("IC95 % :", round(np.exp(m - 1.96 * s), 1), "à", round(np.exp(m + 1.96 * s), 1),
      "| écarts à la médiane :", round(np.exp(m - 1.96 * s) - np.exp(m), 1), "et +", round(np.exp(m +
↳ 1.96 * s) - np.exp(m), 1))
```

médiane : 1998.2 | moyenne : 2004.6
IC95 % : 1708.2 à 2337.4 | écarts à la médiane : -290.0 et + 339.2

Corrigé 4.7

- (a) $MASE_A = 120/150 = 0,80$ (le modèle fait **20 % mieux** que le naïf saisonnier de l'apprentissage) ; $MASE_B = 160/150 \approx 1,07$ (**moins bien** que le naïf). (b) Le MAPE divise par $|y_t|$: si $y_t = 0$ (un mois sans vente), le terme est indéfini (division par zéro), et pour des valeurs réelles très petites il explose. (c) Une sous-prévision ne peut pas dépasser **100 %** d'erreur (prévoir 0 pour une valeur de 100 donne 100 %), alors qu'une surestimation est **illimitée** (prévoir 300 pour 100 donne 200 %). En minimisant le MAPE, un modèle est donc incité à prévoir **trop bas**.

```
actuel = 100
for prevu in (0, 50, 150, 300):
    print(f"réel = {actuel}, prévu = {prevu:3d} -> erreur en % du réel = {100 * abs(actuel - prevu) /
↳ actuel:.0f} %")
print("MASE A =", 120 / 150, "| MASE B =", round(160 / 150, 3))
```

réel = 100, prévu = 0 -> erreur en % du réel = 100 %
réel = 100, prévu = 50 -> erreur en % du réel = 50 %

```

réel = 100, prévu = 150 -> erreur en % du réel = 50 %
réel = 100, prévu = 300 -> erreur en % du réel = 200 %
MASE A = 0.8 | MASE B = 1.067

```

Corrigé 4.8

Règle (4.2.1) : $AR(p) \Rightarrow$ PACF qui s'arrête après p , ACF qui décroît ; $MA(q) \Rightarrow$ ACF qui s'arrête après q , PACF qui décroît. Lecture des sorties de l'énoncé (bande de $\pm 0,113$) :

- X : ACF qui décroît lentement (0,73 ; 0,58 ; 0,43 ; 0,28...), PACF avec **un seul** pic net, au décalage 1 (0,74), puis des valeurs dans la bande : un **AR(1)**, de coefficient voisin de 0,74.
- Z : ACF qui décroît, PACF de 0,69 puis 0,19 (au-dessus de la bande), puis dans la bande : un **AR(2)** est plausible (un AR(1) est possible : le deuxième pic est faible).
- Y : PACF qui décroît avec alternance de signes (0,60 ; -0,28 ; -0,27 ; puis dans la bande) : une signature de **MA** ; mais l'ACF (0,60 ; 0,17 ; puis -0,19, -0,25, -0,23 au-delà de la bande) ne s'arrête **pas** proprement après le décalage 2. Cette série est **ambiguë** à l'œil : un MA(2), mais peut-être un MA plus long ou un ARMA.

Voilà la réalité de l'identification : avec 300 points, les autocorrélations estimées fluctuent d'environ $\pm 0,11$, et les signatures théoriques de la figure de 4.2.1 se brouillent. D'où le rôle des critères d'information, qui comparent les modèles candidats **entre eux** :

```

candidats = {"AR(1)": (1, 0, 0), "AR(2)": (2, 0, 0), "MA(1)": (0, 0, 1), "MA(2)": (0, 0, 2),
             ↪ "ARMA(1,1)": (1, 0, 1)}
ajustements = {nom: {c: ARIMA(s, order=o, trend="n").fit() for c, o in candidats.items()} for nom, s
             ↪ in series.items()}
aic = pd.DataFrame({nom: {c: m.aic for c, m in d.items()} for nom, d in ajustements.items()})
bic = pd.DataFrame({nom: {c: m.bic for c, m in d.items()} for nom, d in ajustements.items()})
print("AIC par modèle (une colonne par série) :")
print(aic.round(1).to_string())
print("\nBIC par modèle :")
print(bic.round(1).to_string())
print("\nmeilleur modèle par AIC :", aic.idxmin().to_dict())
print("meilleur modèle par BIC :", bic.idxmin().to_dict())
print("vérité : X = AR(1), phi = 0,75 | Y = MA(2), theta = (0,7 ; 0,5) | Z = AR(2), phi = (0,5 ; 0,3)")

```

AIC par modèle (une colonne par série) :

	X	Y	Z
AR(1)	850.5	906.2	882.7
AR(2)	849.5	883.7	873.3
MA(1)	944.1	929.2	964.5
MA(2)	897.8	864.3	930.8
ARMA(1,1)	850.0	896.0	871.9

BIC par modèle :

	X	Y	Z
AR(1)	858.0	913.6	890.2
AR(2)	860.6	894.8	884.4
MA(1)	951.5	936.6	971.9
MA(2)	908.9	875.4	941.9
ARMA(1,1)	861.1	907.1	883.0

meilleur modèle par AIC : {'X': 'AR(2)', 'Y': 'MA(2)', 'Z': 'ARMA(1,1)'}

meilleur modèle par BIC : {'X': 'AR(1)', 'Y': 'MA(2)', 'Z': 'ARMA(1,1)'}

vérité : X = AR(1), phi = 0,75 | Y = MA(2), theta = (0,7 ; 0,5) | Z = AR(2), phi = (0,5 ; 0,3)

Lecture des critères. Pour Y , le MA(2) l'emporte nettement, par l'AIC comme par le BIC : le critère résout l'ambiguïté que l'œil ne résolvait pas. Pour X et Z , ce sont des modèles **voisins** qui se disputent la première place avec un ou deux points d'écart : un AR(2) dont le second coefficient est petit ressemble à un AR(1), et un ARMA(1,1) imite un AR(2). **Un écart d'AIC de**

l'ordre de 1 à 2 ne permet pas de trancher (la règle empirique est qu'il faut un écart de plus de 2 à 4 points pour préférer un modèle). Le BIC, plus sévère avec la complexité, retrouve la vérité pour X (AR(1)) et pour Y (MA(2)), mais préfère encore l'ARMA(1,1) à l'AR(2) pour Z , avec 1,4 point d'écart (883,0 contre 884,4) : pour ces deux modèles voisins, les données n'ont pas de quoi décider. En pratique, on retient alors le modèle **le plus simple** parmi les ex æquo, et on valide par les résidus et par la prévision.

Corrigé 4.9

- (a) Soit $\gamma(k) = \varphi^k \gamma(0)$. $\text{Var}(W_t) = 2\gamma(0) - 2\gamma(1) = 2\gamma(0)(1 - \varphi)$. $\text{Cov}(W_t, W_{t-1}) = \text{Cov}(Y_t - Y_{t-1}, Y_{t-1} - Y_{t-2}) = \gamma(1) - \gamma(2) - \gamma(0) + \gamma(1) = \gamma(0)(2\varphi - \varphi^2 - 1) = -\gamma(0)(1 - \varphi)^2$. Donc $\rho_W(1) = \frac{-\gamma(0)(1 - \varphi)^2}{2\gamma(0)(1 - \varphi)} = -\frac{1 - \varphi}{2}$. (b) Pour $\varphi = 0$ (Y est un bruit blanc), on retrouve $-0,5$ (section 4.1.6 du livre). Pour $\varphi \rightarrow 1$ (quasi-marche aléatoire), $\rho_W(1) \rightarrow 0$: la différence d'une marche aléatoire est un bruit blanc : **la différenciation est alors le bon remède**. Entre les deux, plus la série est « loin » d'une racine unitaire, plus la différenciation fabrique une autocorrélation négative artificielle. (c) Simulation :

```
from statsmodels.tsa.arima_process import ArmaProcess
for phi in (0.0, 0.5, 0.9):
    x = ArmaProcess([1, -phi], [1]).generate_sample(300000,
    distrvs=np.random.default_rng(7).standard_normal)
    print(f"phi = {phi} : rho_W(1) simulé = {acf_manuel(np.diff(x), 1):+.3f} | théorie -(1-phi)/2 =
    {-(1 - phi) / 2:+.3f}")
```

```
phi = 0.0 : rho_W(1) simulé = -0.502 | théorie -(1-phi)/2 = -0.500
phi = 0.5 : rho_W(1) simulé = -0.252 | théorie -(1-phi)/2 = -0.250
phi = 0.9 : rho_W(1) simulé = -0.052 | théorie -(1-phi)/2 = -0.050
```

Corrigé 4.10

- (a) **Date 1.** $P_{1|0} = 12$, $F_1 = 12 + 9 = 21$, $K_1 = 12/21 = 0,5714$, $v_1 = 56 - 50 = 6$, $a_{1|1} = 50 + 0,5714 \times 6 = 53,43$, $P_{1|1} = 12 \times \frac{9}{21} = 5,143$. **Date 2.** $P_{2|1} = 5,143 + 3 = 8,143$, $F_2 = 17,143$, $K_2 = 8,143/17,143 = 0,475$, $v_2 = 52 - 53,43 = -1,43$, $a_{2|2} = 53,43 + 0,475 \times (-1,43) = 52,75$, $P_{2|2} = 8,143 \times \frac{9}{17,143} = 4,275$. (b) En régime permanent, la variance prédite p vérifie $p = \frac{p\sigma_\varepsilon^2}{p + \sigma_\varepsilon^2} + \sigma_\eta^2$, soit $p^2 = \sigma_\eta^2 p + \sigma_\eta^2 \sigma_\varepsilon^2$, donc $p = \frac{3 + \sqrt{9 + 4 \times 27}}{2} = \frac{3 + \sqrt{117}}{2} \approx 6,908$ et $K_\infty = \frac{p}{p + 9} \approx 0,434$. (c) Le lissage exponentiel simple de paramètre $\alpha = K_\infty \approx 0,43$.

```
r = kalman_niveau_local(np.array([56.0, 52.0]), s2_eps=9.0, s2_eta=3.0, a0=50.0, P0=12.0) #
↳ fonction définie en 4.5.3
print(pd.DataFrame({k: r[k] for k in ["P_pred", "F", "K", "v", "a_filtre", "P_filtre"]}, index=[1,
↳ 2])).round(4).to_string())
p_inf = (3 + np.sqrt(9 + 4 * 27)) / 2
print("régime permanent : p =", round(p_inf, 4), "| K_inf =", round(p_inf / (p_inf + 9), 4))
longue = kalman_niveau_local(np.random.default_rng(1).normal(100, 3, 300), 9.0, 3.0, 100.0, 12.0)
print("gain après 300 observations :", round(longue["K"][-1], 4))
```

	P_pred	F	K	v	a_filtre	P_filtre
1	12.0000	21.0000	0.5714	6.0000	53.4286	5.1429
2	8.1429	17.1429	0.4750	-1.4286	52.7500	4.2750

régime permanent : p = 6.9083 | K_inf = 0.4343
gain après 300 observations : 0.4343

Corrigé 4.11

- (a) En niveaux, la pente est « significative » dans **plus des trois quarts** des cas (77 % avec 100 points en 4.4.2 ; plus encore avec 200 points, voir la sortie : plus les séries sont longues, plus l'illusion est fréquente), pas 5 %. (b) Sur les différences (qui sont des bruits blancs indépendants), le taux retombe autour de 5 % (la sortie donne une valeur proche, à l'incertitude de la simulation près). (c) La différenciation rend les séries **stationnaires** : les hypothèses du chapitre 1 s'appliquent de nouveau, donc les p-valeurs sont justes. On **perd l'information de long terme** : si les séries sont cointégrées (4.4.2), une régression sur les différences ignore la relation d'équilibre ; il faut alors le modèle à correction d'erreur.

```
import statsmodels.api as sm
rng = np.random.default_rng(55)
niveaux, differences = [], []
for _ in range(2000):
    a = np.cumsum(rng.normal(size=200))
    b = np.cumsum(rng.normal(size=200))
    niveaux.append(abs(sm.OLS(a, sm.add_constant(b)).fit().tvalues[1]) > 1.96)
    differences.append(abs(sm.OLS(np.diff(a), sm.add_constant(np.diff(b))).fit().tvalues[1]) > 1.96)
print("part de pentes « significatives » à 5 % : en niveaux =", np.mean(niveaux).round(3), "| sur les  
↳ différences =", np.mean(differences).round(3))
```

part de pentes « significatives » à 5 % : en niveaux = 0.824 | sur les différences = 0.052

Corrigé 4.12

- (a) $\sigma^2 = \frac{0,1}{1 - 0,2 - 0,7} = 1$. (b) $\sigma_t^2 = 0,1 + 0,2 \times 2^2 + 0,7 \times 1,5 = 0,1 + 0,8 + 1,05 = 1,95$. (c) $\mathbb{E}[\sigma_{t+1}^2] = \omega + (\alpha + \beta)\sigma_t^2 = 0,1 + 0,9 \times 1,95 = 1,855$ (car $\mathbb{E}[\varepsilon_t^2 | \text{passé}] = \sigma_t^2$). (d) L'excès $\sigma_{t+h}^2 - 1$ est multiplié par $\alpha + \beta = 0,9$ à chaque pas : il est divisé par deux quand $0,9^h = 0,5$, soit $h = \ln 0,5 / \ln 0,9 \approx 6,6$: **environ 7 jours**.

```
omega, alpha, beta = 0.1, 0.2, 0.7
var_lt = omega / (1 - alpha - beta)
sig2_t = omega + alpha * 2 ** 2 + beta * 1.5
prev = [omega + (alpha + beta) * sig2_t]
for _ in range(19):
    prev.append(omega + (alpha + beta) * prev[-1])
print("variance de long terme :", round(var_lt, 4), "| sigma_t^2 =", round(sig2_t, 3), "| prévision de  
↳ sigma_{t+1}^2 =", round(prev[0], 3))
print("excès sur la variance de long terme aux pas 1, 5, 10, 20 :", np.round(np.array(prev)[[0, 4, 9,  
↳ 19]] - var_lt, 3))
print("demi-vie de l'excès :", round(np.log(0.5) / np.log(alpha + beta), 2), "pas")
```

variance de long terme : 1.0 | sigma_t^2 = 1.95 | prévision de sigma_{t+1}^2 = 1.855
excès sur la variance de long terme aux pas 1, 5, 10, 20 : [0.855 0.561 0.331 0.115]
demi-vie de l'excès : 6.58 pas

Corrigé 4.13

On applique la démarche de 4.3 à $\log(\text{nb_commandes})$: même découpage, un naïf saisonnier avec dérive, un SARIMAX, et la MASE en **nombre de commandes** (échelle : MAE du naïf saisonnier sur l'apprentissage).

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
v = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"]).set_index("mois")
v.index.freq = "MS"
z = np.log(v["nb_commandes"])
```

```

train_z, test_z = z[:"2023-12"], z["2024-01":]
Xe = v[["promo", "covid"]].astype(float)

g = (train_z - train_z.shift(12)).mean()
naif = pd.Series([train_z.iloc[-12 + (h % 12)] + g * (1 + h // 12) for h in range(24)],
    ↪ index=test_z.index)
modele = SARIMAX(train_z, exog=Xe["2023-12"], order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="ct").fit(dispatch=False, maxiter=300)
prevu = pd.Series(modele.get_forecast(24, exog=Xe["2024-01:"]).predicted_mean.values,
    ↪ index=test_z.index)

nb_train = np.exp(train_z)
echelle = np.mean(np.abs(nb_train.values[12:] - nb_train.values[:-12]))
lignes = {}
for nom, f in [("naïf saisonnier + dérive", naif), ("SARIMAX(1,0,0)(0,1,1)12 + tendance", prevu)]:
    mae = np.mean(np.abs(np.exp(test_z) - np.exp(f)))
    lignes[nom] = {"MAE (commandes)": mae, "MASE": mae / echelle, "RMSE (log)":
    ↪ np.sqrt(np.mean((test_z - f) ** 2)), "biais (log)": float(np.mean(test_z - f))}
print("MAE du naïf saisonnier sur l'apprentissage :", round(echelle, 2), "commandes")
print(pd.DataFrame(lignes).T.round(3).to_string())
print("écart-type des résidus du SARIMAX (log) :", round(float(np.sqrt(modele.params['sigma2'])), 3),
    ↪ "| à comparer à celui du chiffre d'affaires (4.2) : environ 0,06 à 0,07")

```

```

MAE du naïf saisonnier sur l'apprentissage : 7.92 commandes
                                MAE (commandes)  MASE  RMSE (log)  biais (log)
naïf saisonnier + dérive                7.786  0.983      0.273      -0.139
SARIMAX(1,0,0)(0,1,1)12 + tendance        6.146  0.776      0.216       0.023
écart-type des résidus du SARIMAX (log) : 0.262 | à comparer à celui du chiffre d'affaires (4.2) :
    ↪ environ 0,06 à 0,07

```

Le résultat se lit en trois temps. (1) Le **bruit** de cette série est nettement plus grand que celui du chiffre d'affaires : un nombre de commandes est un **comptage** (de loi de Poisson, de variance égale à la moyenne), donc son erreur relative, de l'ordre de $1/\sqrt{\text{moyenne}}$ (environ 19 % pour 28 commandes par mois), est irréductible. L'écart-type des résidus du SARIMAX est d'environ 0,26 en logarithme, près de quatre fois celui du chiffre d'affaires. (2) Le SARIMAX fait **mieux que la référence simple** (MASE de 0,78 contre 0,98, soit environ 20 % d'erreur de moins ; sa MAE est de 6,1 commandes par mois), mais la référence est déjà **à peine meilleure que le naïf saisonnier de l'apprentissage** (MASE proche de 1) : à ce niveau de bruit, le gain d'un bon modèle se mesure en dizaines de pour cent d'une erreur de toute façon élevée, pas en précision. (3) La bonne conclusion est donc moins « quel modèle ? » que « quelle précision peut-on espérer ? » : une prévision de commandes mensuelles doit s'accompagner d'une **fourchette large**.

Chapitre 5 : Analyse de survie — exercices et applications

Ce chapitre du cahier accompagne le chapitre 5 du livre (censure, Kaplan-Meier, Cox, modèles paramétriques, risques concurrents). Vous y trouverez **six applications guidées**, qui écrivent à la main les estimateurs du chapitre et les comparent aux bibliothèques, puis **treize exercices corrigés**. Les données sont celles du livre : `donnees/clients.csv` (2 000 clients, simulés) et, pour la dernière application, une table de risques concurrents simulée dans le chapitre. Prérequis : les sections 5.1 à 5.5 du livre.

Préparation

Les estimateurs du chapitre sont écrits dans `build/outils_ch05.py` (on y trouve le code complet de `kaplan_meier`, `logrank`, `cox_ph`, `test_ph`, `ajuster`, `incidence_cumulee`, etc.). Chaque application en rappelle l'idée et l'utilise ; les exercices les réutilisent. Commençons par charger la boîte à outils et les données.

```
import sys
sys.path.insert(0, "build")                # dossier de la boîte à outils du chapitre
import numpy as np
import pandas as pd
from scipy import stats
from outils_ch05 import *                  # kaplan_meier, logrank, cox_ph, ajuster,
↳ incidence_cumulee...

c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()    # durées et départs observés des 2
↳ 000 clients
y8 = np.array([3, 5, 6, 8, 10, 12, 14, 14.])                # les huit clients du livre (5.1.1)
d8 = np.array([1, 1, 0, 1, 0, 1, 0, 0])
print(len(c), "clients,", int(d.sum()), "départs observés")
```

2000 clients, 977 départs observés

Applications

Application 5.1 — L'expérience de la censure

Objectif. Voir de ses yeux que les moyennes « naïves » sont biaisées, mesurer la part des pertes de vue, et estimer un taux de départ constant avec la vraisemblance censurée (livre, 5.1).

Étape 1 : une expérience contrôlée. Nous simulons 5 000 clients dont nous connaissons les vraies durées (loi de Weibull de forme 1,35 et d'échelle 36 mois), puis nous les observons pendant une durée aléatoire entre 6 et 60 mois.

```
from math import gamma

rng = np.random.default_rng(51)
n, k, echelle = 5000, 1.35, 36.0
T_vraie = echelle * rng.weibull(k, n)          # durées complètes (inconnues en pratique)
C = rng.uniform(6, 60, n)                    # durée pendant laquelle on peut observer chaque client
yy = np.minimum(T_vraie, C)                  # ce que l'on voit
parti = (T_vraie <= C).astype(int)

print(f"part de clients censurés : {100 * (1 - parti.mean()):.1f} %")
print(f"VRAIE durée moyenne (formule exacte) : {echelle * gamma(1 + 1 / k):.1f} mois")
print(f"(1) moyenne de toutes les durées observées : {yy.mean():.1f} mois")
print(f"(2) moyenne des seuls clients partis : {yy[parti == 1].mean():.1f} mois")

part de clients censurés : 44.8 %
VRAIE durée moyenne (formule exacte) : 33.0 mois
(1) moyenne de toutes les durées observées : 21.5 mois
(2) moyenne des seuls clients partis : 18.5 mois
```

Lecture. Les deux moyennes naïves sont très en dessous de la vérité, et aucune ne se rapproche si l'on ajoute des clients : ce sont des estimateurs biaisés. À essayer : observer pendant 30 à 84 mois au lieu de 6 à 60. Que deviennent les deux moyennes ?

Étape 2 : d'où viennent les durées censurées du fichier ? La date de fin d'observation est le 31 décembre 2025. Un client censuré dont la durée vaut exactement son suivi possible est un censuré administratif ; les autres sont des pertes de vue.

```
c["date_inscription"] = pd.to_datetime(c["date_inscription"])
c["suivi_possible"] = (pd.Timestamp("2025-12-31") - c["date_inscription"]).dt.days / 30.4375

censures = c[c["churn"] == 0]
admin = (censures["duree_mois"] - censures["suivi_possible"]).abs() < 0.01
print("censurés administratifs :", int(admin.sum()), "| pertes de vue :", int((~admin).sum()))
print("suivi possible de", round(c["suivi_possible"].min(), 1), "à", round(c["suivi_possible"].max(), 1), "mois")

censurés administratifs : 660 | pertes de vue : 363
suivi possible de 6.0 à 84.0 mois
```

Étape 3 : le taux de départ constant. Pour un risque constant, le maximum de vraisemblance donne $\hat{\lambda} = D / \sum y_i$. Vérifions-le sur les huit clients avec un optimiseur (la fonction `log_vraisemblance_exp` est dans la boîte à outils), puis appliquons-le aux 2 000 clients.

```
from scipy.optimize import minimize_scalar

opt = minimize_scalar(lambda l: -log_vraisemblance_exp(l, y8, d8), bounds=(1e-4, 1), method="bounded")
print(f"8 clients : λ^(optimiseur) = {opt.x:.4f} | formule D/Σy = {d8.sum() / y8.sum():.4f}")

lam = d.sum() / y.sum()
se = lam / np.sqrt(d.sum())
```

```
print(f"2000 clients : D = {d.sum()}, exposition = {y.sum():.0f} mois-clients")
print(f"λ̂ = {lam:.5f} par mois (IC95 : {lam - 1.96 * se:.5f} à {lam + 1.96 * se:.5f})")
print(f"durée moyenne 1/λ̂ = {1 / lam:.1f} mois | médiane ln2/λ̂ = {np.log(2) / lam:.1f} mois | moyenne
↳ naïve : {y.mean():.1f} mois")
```

```
8 clients : λ̂(optimiseur) = 0.0556 | formule D/Σy = 0.0556
2000 clients : D = 977, exposition = 46,761 mois-clients
λ̂ = 0.02089 par mois (IC95 : 0.01958 à 0.02220)
durée moyenne 1/λ̂ = 47.9 mois | médiane ln2/λ̂ = 33.2 mois | moyenne naïve : 23.4 mois
```

Lecture. L'estimation exponentielle donne environ 48 mois de durée moyenne, plus du double de la moyenne naïve. Mais elle suppose un risque constant : l'application 5.2 montrera que Kaplan-Meier ne le confirme pas.

Application 5.2 — Kaplan-Meier écrit à la main

Objectif. Construire l'estimateur de Kaplan-Meier, le comparer à statsmodels et à R, en donner l'incertitude et deux résumés : la médiane et la durée moyenne restreinte (livre, 5.2.1 à 5.2.4, 5.2.7).

Étape 1 : les huit clients. Le tableau donne, à chaque instant de départ, l'ensemble à risque n_j , les départs d_j et la survie cumulée. La fonction `kaplan_meier(y, d)` de la boîte à outils renvoie les instants, les effectifs à risque, les départs, la survie et la somme de Greenwood.

```
print(tableau_km(y8, d8).to_string(index=False))
tj8, n8, dj8, S8, gw8 = kaplan_meier(y8, d8)
print("survie aux instants de départ :", S8.round(4))
```

t _j	n _j (à risque)	d _j (départs)	censurés en t _j	d _j /n _j	S(t _j)	somme Greenwood
3.0	8	1	0	0.1250	0.875	0.0179
5.0	7	1	0	0.1429	0.750	0.0417
8.0	5	1	0	0.2000	0.600	0.0917
12.0	3	1	0	0.3333	0.400	0.2583

survie aux instants de départ : [0.875 0.75 0.6 0.4]

Lecture. On retrouve les valeurs du livre : 0,875, 0,75, 0,6, puis 0,4. Les censurés sortent de l'ensemble à risque sans faire bouger la courbe.

Étape 2 : les 2 000 clients, comparés à statsmodels.

```
from statsmodels.duration.survfunc import SurvfuncRight

tj, n, dj, S, gw = kaplan_meier(y, d)
sf = SurvfuncRight(y, d)
print(f"{'mois':>5} {'à la main':>10} {'statsmodels':>12} {'ET Greenwood':>13} {'ET
↳ statsmodels':>15}")
for t in (12, 24, 36, 48, 60):
    i = np.searchsorted(sf.surv_times, t, side="right") - 1
    s = surv_at(t, tj, S)
    print(f"{'t':>5} {'s':>10.5f} {'sf.surv_prob[i]:>12.5f} {'s * np.sqrt(surv_at(t, tj, gw,
↳ avant=0.0)):>13.5f} {'sf.surv_prob_se[i]:>15.5f}")
```

mois	à la main	statsmodels	ET Greenwood	ET statsmodels
12	0.82519	0.82519	0.00888	0.00888
24	0.63376	0.63376	0.01211	0.01211
36	0.45275	0.45275	0.01396	0.01396
48	0.30758	0.30758	0.01498	0.01498
60	0.24754	0.24754	0.01558	0.01558

Et avec la référence de la discipline, le paquet R `survival` :

```
library(survival)
clients <- read.csv("donnees/clients.csv")
km_all <- survfit(Surv(duree_mois, churn) ~ 1, data = clients)
print(summary(km_all, times = c(12, 24, 36, 48, 60)))
```

Call: survfit(formula = Surv(duree_mois, churn) ~ 1, data = clients)

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
12	1378	323	0.825	0.00888	0.808	0.843
24	814	286	0.634	0.01211	0.610	0.658
36	416	200	0.453	0.01396	0.426	0.481
48	186	111	0.308	0.01498	0.280	0.338
60	96	31	0.248	0.01558	0.219	0.280

Lecture. Les trois sources donnent la même courbe et les mêmes erreurs standard. Remarquez la colonne n. risk de R : à 60 mois, il reste 96 clients sous observation contre 1 378 à 12 mois.

Étape 3 : les intervalles de confiance. Trois constructions (plan, log, log-log), à 36 et 60 mois, puis pour les huit clients :

```
for t in (36, 60):
    s, plan, log_, loglog = intervalles(t, tj, S, gw)
    print(f"t = {t} : S = {s:.4f} | plan [{plan[0]:.4f} ; {plan[1]:.4f}] | log [{log_[0]:.4f} ;
    ↪ {log_[1]:.4f}] | log-log [{loglog[0]:.4f} ; {loglog[1]:.4f}]")
s8, plan8, log8, ll8 = intervalles(12, tj8, S8, gw8)
print(f"8 clients, t = 12 : S = {s8:.2f} | plan [{plan8[0]:.2f} ; {plan8[1]:.2f}] | log-log
    ↪ [{ll8[0]:.2f} ; {ll8[1]:.2f}]")
```

```
t = 36 : S = 0.4528 | plan [0.4254 ; 0.4801] | log [0.4262 ; 0.4810] | log-log [0.4252 ; 0.4799]
t = 60 : S = 0.2475 | plan [0.2170 ; 0.2781] | log [0.2188 ; 0.2800] | log-log [0.2176 ; 0.2786]
8 clients, t = 12 : S = 0.40 | plan [0.00 ; 0.80] | log-log [0.07 ; 0.73]
```

Lecture. Sur 2 000 clients, les trois intervalles sont presque identiques. Sur huit clients, l'intervalle plan est immense (de presque 0 à 0,80) ; le log-log est plus raisonnable.

Étape 4 : médiane et durée moyenne restreinte.

```
z = 1.959964
S_bas, S_haut = S * np.exp(-z * np.sqrt(gw)), S * np.exp(z * np.sqrt(gw)) # bande « log »
print(f"médiane : {tj[S <= 0.5][0]:.2f} mois ; IC95 [{tj[S_bas <= 0.5][0]:.1f} ; {tj[S_haut <=
    ↪ 0.5][0]:.1f}]")
print("durée moyenne restreinte, 8 clients, tau = 14 :", round(rmst(tj8, S8, 14), 2), "mois")
for tau in (24, 36, 60):
    print(f"2000 clients, tau = {tau} : {rmst(tj, S, tau):.2f} mois")
```

```
médiane : 32.45 mois ; IC95 [29.9 ; 34.2]
durée moyenne restreinte, 8 clients, tau = 14 : 10.2 mois
2000 clients, tau = 24 : 19.76 mois
2000 clients, tau = 36 : 26.19 mois
2000 clients, tau = 60 : 33.94 mois
```

Étape 5 : et si les pertes de vue étaient des départs ? (analyse de sensibilité) On recompte comme départs les clients censurés avant la fin de l'étude : c'est le pire cas.

```
c["date_inscription"] = pd.to_datetime(c["date_inscription"])
suivi = (pd.Timestamp("2025-12-31") - c["date_inscription"]).dt.days / 30.4375
perdu = ((c["churn"] == 0) & (suivi - c["duree_mois"] > 0.01)).to_numpy()
tj_p, _, _, S_p, _ = kaplan_meier(y, np.where(perdu, 1, d)) # pertes de vue recomptées comme
    ↪ départs
print("pertes de vue :", int(perdu.sum()))
for t in (12, 24, 36):
    print(f"{t}>3} mois : censure non informative {surv_at(t, tj, S):.3f} | pire cas {surv_at(t, tj_p,
    ↪ S_p):.3f}")
```

```
pertes de vue : 363
12 mois : censure non informative 0.825 | pire cas 0.753
24 mois : censure non informative 0.634 | pire cas 0.526
36 mois : censure non informative 0.453 | pire cas 0.341
```

Lecture. À 36 mois, la survie passe de 0,45 à 0,34 dans le pire cas : l'hypothèse de censure non informative pèse plus que l'incertitude statistique.

Application 5.3 — Comparer des groupes, et le piège de l'entrée tardive

Objectif. Écrire le test du log-rank, le comparer aux logiciels, corriger les comparaisons multiples et mesurer l'effet d'une entrée tardive (livre, 5.2.5 et 5.2.6).

Étape 1 : le log-rank sur les huit clients. Imaginons que C, E, F et G aient reçu l'offre. `logrank(y, d, groupe)` renvoie les départs observés et attendus, le χ^2 , ses degrés de liberté et la p-valeur.

```
groupe8 = np.array(["sans", "sans", "avec", "sans", "avec", "avec", "avec", "sans"]) # clients A
↪ à H
O8, E8, ddl8, p8 = logrank(y8, d8, groupe8)
print(f"observés {O8} | attendus {E8.round(3)} | chi2 = {chi2:.2f} | p = {p8:.3f}")
```

```
observés [1. 3.] | attendus [2.338 1.662] | chi2 = 1.87 | p = 0.171
```

Étape 2 : l'offre de bienvenue sur les 2 000 clients, avec trois outils.

```
from statsmodels.duration.survfunc import survdiff

O, E, chi2, ddl, p = logrank(y, d, c["offre_bienvenue"])
print(f"à la main : observés {O.astype(int)}, attendus {E.round(1)}, chi2 = {chi2:.2f}, p = {p:.1e}")
print("statsmodels : chi2 = %.2f, p = %.1e" % survdiff(y, d, c["offre_bienvenue"]))
```

```
à la main : observés [528 449], attendus [435.6 541.4], chi2 = 35.54, p = 2.5e-09
statsmodels : chi2 = 35.54, p = 2.5e-09
```

```
print(survdiff(Surv(duree_mois, churn) ~ offre_bienvenue, data = clients))
```

```
Call:
survdiff(formula = Surv(duree_mois, churn) ~ offre_bienvenue,
          data = clients)
```

	N	Observed	Expected	(O-E) ² /E	(O-E) ² /V
offre_bienvenue=0	985	528	436	19.6	35.5
offre_bienvenue=1	1015	449	541	15.8	35.5

```
Chisq= 35.5 on 1 degrees of freedom, p= 3e-09
```

Étape 3 : trois canaux, puis deux à deux avec la correction de Holm.

```
O3, E3, chi3, ddl3, p3 = logrank(y, d, c["canal_acquisition"])
print(f"3 canaux : chi2 = {chi3:.1f} à {ddl3} ddl, p = {p3:.1e}")

paires = [("Boutique", "Réseaux"), ("Boutique", "Site"), ("Site", "Réseaux")]
brutes = []
for a, b in paires:
    m = c["canal_acquisition"].isin([a, b]).to_numpy()
    brutes.append(logrank(y[m], d[m], c["canal_acquisition"]).to_numpy()[m])[4])
holm, courant = np.empty(3), 0.0
for rang, i in enumerate(np.argsort(brutes)):
    courant = max(courant, min(1.0, (3 - rang) * brutes[i])) # (m - rang) x p, puis suite
↪ croissante
```

```

    holm[i] = courant
print(pd.DataFrame({"comparaison": [f"{a} / {b}" for a, b in paires], "p brute": brutes, "p Holm":
    ↪ holm}).to_string(index=False, float_format=lambda x: f"{x:.2e}"))

```

```

3 canaux : chi2 = 55.0 à 2 ddl, p = 1.1e-12
      comparaison  p brute    p Holm
Boutique / Réseaux 5.27e-13 1.58e-12
Boutique / Site   8.61e-04 8.61e-04
Site / Réseaux   3.02e-05 6.05e-05

```

Étape 4 : autres pondérations du log-rank.

```

for nom, w, kw in [("log-rank", None, {}), ("Gehan-Breslow", "gb", {}), ("Tarone-Ware", "tw", {}),
    ↪ ("Fleming-Harrington p=1", "fh", {"fh_p": 1})]:
    chi, pv = survdiff(y, d, c["offre_bienvenue"], weight_type=w, **kw)
    print(f"{nom:<24} chi2 = {chi:6.2f}    p = {pv:.1e}")

```

```

log-rank          chi2 = 35.54    p = 2.5e-09
Gehan-Breslow     chi2 = 31.45    p = 2.0e-08
Tarone-Ware       chi2 = 35.12    p = 3.1e-09
Fleming-Harrington p=1  chi2 = 34.98    p = 3.3e-09

```

Étape 5 : l'entrée tardive. 6 000 clients de loi de Weibull ne sont enregistrés qu'à leur adhésion au programme de fidélité (uniforme entre 0 et 30 mois après le premier achat) s'ils sont encore clients. Ignorer cette règle surestime la survie ; `kaplan_meier(..., entree=...)` la respecte.

```

rng = np.random.default_rng(52)
N = 6000
T = 36 * rng.weibull(1.35, N)                # durées vraies
E = rng.uniform(0, 30, N)                   # date d'adhésion (mois après le premier achat)
vus = T > E                                  # seuls les clients encore là à l'adhésion sont
    ↪ enregistrés
Tv, Ev = T[vus], E[vus]
Cv = Ev + rng.uniform(6, 60, vus.sum())      # fin d'observation après l'adhésion
Yv, Dv = np.minimum(Tv, Cv), (Tv <= Cv).astype(int)

tj_n, _, _, S_n, _ = kaplan_meier(Yv, Dv)      # entrées tardives ignorées
tj_c, _, _, S_c, _ = kaplan_meier(Yv, Dv, entree=Ev) # entrées tardives prises en compte
sf_e = SurvfuncRight(Yv, Dv, entry=Ev)
print("clients vus :", int(vus.sum()), "sur", N)
print(f"{ 'mois':>5} { 'vraie S(t)':>11} { 'ignorer':>9} { 'avec entrée':>12} { 'statsmodels':>12}")
for t in (6, 12, 24, 36):
    i = np.searchsorted(sf_e.surv_times, t, side="right") - 1
    print(f"{t:>5} {np.exp(-(t / 36) ** 1.35):>11.3f} {surv_at(t, tj_n, S_n):>9.3f} {surv_at(t, tj_c,
    ↪ S_c):>12.3f} {sf_e.surv_prob[i]:>12.3f}")

```

```

clients vus : 4417 sur 6000
mois  vraie S(t)  ignorer  avec entrée  statsmodels
6      0.915      0.988      0.913      0.913
12     0.797      0.940      0.804      0.804
24     0.561      0.759      0.577      0.577
36     0.368      0.490      0.365      0.365

```

Lecture. Ignorer l'entrée donne 0,76 à 24 mois pour une vérité de 0,56. En la prenant en compte, on est à deux points près.

Application 5.4 — Le modèle de Cox à la main

Objectif. Calculer la vraisemblance partielle, la maximiser par Newton-Raphson, ajuster Cox sur les 2 000 clients, tester la proportionnalité des risques et prédire des courbes (livre, 5.3).

Étape 1 : Newton-Raphson sur les huit clients. La fonction `score_info(beta, y, d, x)` calcule le score $U(\beta)$ et l'information $I(\beta)$ d'une covariable. On suppose que C, E, F et G ont reçu l'offre.

```
x8 = np.array([0, 0, 1, 0, 1, 1, 1, 0], float)          # 1 = a reçu l'offre

U0, I0 = score_info(0.0, y8, d8, x8)
print(f"en beta = 0 : U = {U0:.4f}, I = {I0:.4f}, U²/I = {U0 ** 2 / I0:.3f} (chi2 du log-rank des huit  
↪ clients : 1.87)")
beta = 0.0
for it in range(5):                                     # Newton-Raphson : beta <- beta + U/I
    U, I = score_info(beta, y8, d8, x8)
    beta += U / I
    print(f"itération {it + 1} : beta = {beta:8.5f}")
print("rapport de risques estimé :", round(np.exp(beta), 3))

en beta = 0 : U = -1.3381, I = 0.9571, U²/I = 1.871 (chi2 du log-rank des huit clients : 1.87)
itération 1 : beta = -1.39804
itération 2 : beta = -1.45965
itération 3 : beta = -1.46056
itération 4 : beta = -1.46057
itération 5 : beta = -1.46057
rapport de risques estimé : 0.232
```

Lecture. La statistique de score en $\beta = 0$ égale le χ^2 du log-rank (1,87), et la méthode converge en quatre itérations.

Étape 2 : les 2 000 clients. `cox_ph(y, d, X)` maximise la vraisemblance partielle (ex aequo à la Breslow) et renvoie les coefficients et leur matrice de covariance.

```
X = pd.get_dummies(c[["offre_bienvenue", "age", "canal_acquisition"]], drop_first=True, dtype=float)
fit = cox_ph(y, d, X.to_numpy())
se = np.sqrt(np.diag(fit["cov"]))
tableau = pd.DataFrame({"coef": fit["beta"], "ET": se, "HR": np.exp(fit["beta"]),
                        "HR bas": np.exp(fit["beta"] - 1.96 * se), "HR haut": np.exp(fit["beta"] + 1.96
                        ↪ * se)}, index=X.columns)
print(tableau.round(4).to_string())
```

	coef	ET	HR	HR bas	HR haut
offre_bienvenue	-0.4061	0.0645	0.6662	0.5871	0.7561
age	-0.0136	0.0030	0.9865	0.9806	0.9924
canal_acquisition_Réseaux	0.6351	0.0842	1.8872	1.6002	2.2256
canal_acquisition_Site	0.3258	0.0887	1.3851	1.1641	1.6481

Comparaison avec statsmodels et avec R :

```
from statsmodels.duration.hazard_regression import PHReg

ph = PHReg(y, X, status=d, ties="breslow").fit()
print("statsmodels :", np.round(ph.params, 5))
print("à la main   :", np.round(fit["beta"], 5))
```

```
statsmodels : [-0.40612 -0.01363  0.63508  0.3258 ]
à la main   : [-0.40612 -0.01363  0.63508  0.3258 ]
```

```
clients$canal_acquisition <- factor(clients$canal_acquisition, levels = c("Boutique", "Réseaux",
↪ "Site"))
cox_r <- coxph(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data = clients,
↪ ties = "breslow")
```

```
print(round(summary(cox_r)$coefficients[, c("coef", "se(coef)"]), 5))
```

	coef	se(coef)
offre_bienvenue	-0.40612	0.06454
age	-0.01363	0.00304
canal_acquisitionRéseaux	0.63508	0.08415
canal_acquisitionSite	0.32580	0.08869

Étape 3 : la proportionnalité des risques. test_ph calcule le test de score de Grambsch-Therneau (le temps est remplacé par son rang) pour chaque variable et globalement.

```
chi2_j, chi2_glob = test_ph(y, d, X.to_numpy(), fit)
res_ph = pd.DataFrame({"chi2": chi2_j, "p": stats.chi2.sf(chi2_j, 1)}, index=X.columns)
res_ph.loc["GLOBAL (4 ddl)"] = [chi2_glob, stats.chi2.sf(chi2_glob, 4)]
print(res_ph.round(4).to_string())
```

	chi2	p
offre_bienvenue	0.5588	0.4548
age	0.7292	0.3931
canal_acquisition_Réseaux	2.2063	0.1375
canal_acquisition_Site	0.0146	0.9037
GLOBAL (4 ddl)	4.9502	0.2924

```
print(cox.zph(cox_r, transform = "rank", terms = FALSE))
```

	chisq	df	p
offre_bienvenue	0.5588	1	0.45
age	0.7292	1	0.39
canal_acquisitionRéseaux	2.2063	1	0.14
canal_acquisitionSite	0.0146	1	0.90
GLOBAL	4.9502	4	0.29

Étape 4 : des risques qui se croisent. Deux groupes de 750 clients : risque décroissant (Weibull de forme 0,8) pour A, croissant (forme 1,8) pour B. Cox donne un seul rapport de risques, que le test dénonce.

```
rng = np.random.default_rng(54)
nn = 1500
g = np.repeat([0, 1], nn // 2)
T = np.where(g == 0, 30 * rng.weibull(0.8, nn), 40 * rng.weibull(1.8, nn))
C = rng.uniform(10, 80, nn)
yc, dc = np.minimum(T, C), (T <= C).astype(int)
Xg = g[:, None].astype(float)

fit_c = cox_ph(yc, dc, Xg)
chi2_c, _ = test_ph(yc, dc, Xg, fit_c)
print(f"HR (B contre A) = {np.exp(fit_c['beta'][0]):.2f} | test de proportionnalité : chi2 = {chi2_c[0]:.1f}, p = {stats.chi2.sf(chi2_c[0], 1):.1e}")
```

```
HR (B contre A) = 0.73 | test de proportionnalité : chi2 = 216.3, p = 5.7e-49
```

Étape 5 : le biais d'immortalité. La carte de fidélité (aucun effet réel) est remise au mois 12 aux clients encore là. Traitée comme variable fixe, elle semble protectrice ; traitée comme variable dépendant du temps, elle est neutre.

```
rng = np.random.default_rng(53)
N = 3000
T = rng.exponential(1 / 0.03, N)
C = rng.uniform(24, 60, N)
Y, D = np.minimum(T, C), (T <= C).astype(int)
carte = (Y > 12) & (rng.random(N) < 0.5)
↪ clients encore là
```

risque constant, indépendant de la carte

carte remise au mois 12 à la moitié des

```

naif = PHReg(Y, carte.astype(float)[: , None], status=D, ties="efron").fit()
porteurs = np.where(carte)[0] # deux épisodes pour les porteurs : [0, 12[
↳ puis [12, Y]
debut = np.concatenate([np.zeros(N), np.full(len(porteurs), 12.0)])
fin = np.concatenate([np.where(carte, 12.0, Y), Y[porteurs]])
statut = np.concatenate([np.where(carte, 0, D), D[porteurs]])
x_carte = np.concatenate([np.zeros(N), np.ones(len(porteurs))])
juste = PHReg(fin, x_carte[: , None], status=statut, entry=debut, ties="efron").fit()
print(f"analyse naïve : HR = {np.exp(naif.params[0]):.2f}")
print(f"analyse correcte : HR = {np.exp(juste.params[0]):.2f}")

analyse naïve : HR = 0.46
analyse correcte : HR = 1.03

```

Étape 6 : prédire des courbes de survie. Le risque cumulé de base est estimé par Breslow ; $S(t | x) = \exp[-H_0(t) e^{x^T \beta}]$.

```

H0 = np.cumsum(fit["dj"] / fit["s0"]) # risque cumulé de base
def survie_pred(t, x):
    i = np.searchsorted(fit["tj"], t, side="right") - 1
    return float(np.exp(-H0[i] if i >= 0 else 0.0) * np.exp(np.asarray(x, float) @ fit["beta"]))

profils = {"A : Réseaux, 25 ans, sans offre": [0, 25, 1, 0], "B : Réseaux, 25 ans, avec offre": [1,
↳ 25, 1, 0],
          "C : Boutique, 45 ans, avec offre": [1, 45, 0, 0]} # colonnes : offre, âge,
↳ Réseaux, Site
for nom, x in profils.items():
    print(f"{nom:<34}", [round(survie_pred(t, x), 3) for t in (12, 24, 36, 60)])

A : Réseaux, 25 ans, sans offre      [0.711, 0.438, 0.23, 0.069]
B : Réseaux, 25 ans, avec offre     [0.797, 0.577, 0.375, 0.168]
C : Boutique, 45 ans, avec offre    [0.912, 0.801, 0.673, 0.487]

```

Étape 7 : l'indice de concordance.

```

print("indice de concordance :", round(indice_concordance(y, d, X.to_numpy() @ fit["beta"]), 4))

indice de concordance : 0.6052

```

Application 5.5 — Modèles paramétriques et valeur vie client

Objectif. Ajuster par maximum de vraisemblance quatre lois de durée, choisir par l'AIC, puis chiffrer une valeur vie client avec une analyse de sensibilité (livre, 5.4). **Les hypothèses de marge, d'actualisation et de coût de l'offre sont inventées :** à remplacer par les vôtres.

Étape 1 : quatre lois. `ajuster(loi, y, d, X)` maximise la vraisemblance d'un modèle de temps de vie accéléré ; la première colonne de X est la constante.

```

Xd = pd.get_dummies(c[["offre_bienvenue", "age", "canal_acquisition"]], drop_first=True, dtype=float)
Xc = np.column_stack([np.ones(len(c)), Xd.to_numpy()])
noms = ["(constante)"] + list(Xd.columns)

ajust = {loi: ajuster(loi, y, d, Xc) for loi in ("weibull", "lognormale", "loglogistique")}
ajust["exponentielle"] = ajuster("weibull", y, d, Xc, sigma_fixe=1.0)

wb = ajust["weibull"]
se_w = np.sqrt(np.diag(wb["cov"]))
print(pd.DataFrame({"gamma": wb["theta"][:-1], "ET": se_w[:-1], "e^gamma": np.exp(wb["theta"][:-1])},
↳ index=noms).round(4).to_string())
print("sigma =", round(np.exp(wb["theta"][-1]), 4), "-> forme k =", round(1 /
↳ np.exp(wb["theta"][-1]), 4), "| log-vraisemblance :", round(wb["ll"], 3))

```

	gamma	ET	e^gamma
(constante)	3.5257	0.0970	33.9765
offre_bienvenue	0.3132	0.0489	1.3678
age	0.0102	0.0023	1.0102
canal_acquisition_Réseaux	-0.4799	0.0636	0.6188
canal_acquisition_Site	-0.2452	0.0671	0.7825

sigma = 0.7563 -> forme k = 1.3223 | log-vraisemblance : -4657.378

```
w_r <- survreg(Surv(duree_mois, churn) ~ offre_bienvenue + age + canal_acquisition, data = clients,
  ↪ dist = "weibull")
print(round(summary(w_r)$table, 4))
```

	Value	Std. Error	z	p
(Intercept)	3.5257	0.0970	36.3418	0e+00
offre_bienvenue	0.3132	0.0489	6.4037	0e+00
age	0.0102	0.0023	4.4328	0e+00
canal_acquisitionRéseaux	-0.4799	0.0636	-7.5472	0e+00
canal_acquisitionSite	-0.2452	0.0671	-3.6565	3e-04
Log(scale)	-0.2794	0.0253	-11.0435	0e+00

Étape 2 : choisir la loi.

```
lignes = [{"loi": loi, "paramètres": r["k"], "log-vraisemblance": round(r["ll"], 2), "AIC": round(2 *
  ↪ r["k"] - 2 * r["ll"], 2)} for loi, r in ajust.items()]
print(pd.DataFrame(lignes).sort_values("AIC").to_string(index=False))
lr = 2 * (ajust["weibull"]["ll"] - ajust["exponentielle"]["ll"])
print(f"exponentielle contre Weibull : chi2 = {lr:.1f} (1 ddl), p = {stats.chi2.sf(lr, 1):.1e}")
```

loi	paramètres	log-vraisemblance	AIC
weibull	6	-4657.38	9326.76
loglogistique	6	-4663.73	9339.45
lognormale	6	-4697.20	9406.40
exponentielle	5	-4710.58	9431.17

exponentielle contre Weibull : chi2 = 106.4 (1 ddl), p = 6.0e-25

Étape 3 : conversion vers Cox. Pour la Weibull, $\beta = -\gamma/\sigma$: les coefficients convertis doivent être proches de ceux de l'application 5.4.

```
sigma = np.exp(wb["theta"][-1])
print(pd.DataFrame({"Weibull converti": -wb["theta"][:, -1] / sigma, "Cox (application 5.4)":
  ↪ fit["beta"]}, index=noms[1:]).round(4).to_string())
```

	Weibull converti	Cox (application 5.4)
offre_bienvenue	-0.4141	-0.4061
age	-0.0135	-0.0136
canal_acquisition_Réseaux	0.6346	0.6351
canal_acquisition_Site	0.3242	0.3258

Étape 4 : la valeur vie client. Marge de 6 € par mois et par client actif, taux d'actualisation de 1 % par mois, horizon de 240 mois (hypothèses). La survie du modèle de Weibull est moyennée sur chaque groupe.

```
m_mensuelle, taux, mois = 6.0, 0.01, np.arange(0, 240)
k_hat = 1 / sigma
lam = np.exp(Xc @ wb["theta"][:-1]) # échelle e^{x gamma} de chaque client

def S_groupe(masque):
    return np.array([np.mean(np.exp(-(t / lam[masque]) ** k_hat)) for t in mois])

groupes = {"tous": np.ones(len(c), bool), "sans offre": (c["offre_bienvenue"] == 0).to_numpy(), "avec
  ↪ offre": (c["offre_bienvenue"] == 1).to_numpy()})
```

```
for nom, masque in groupes.items():
    S_g = S_groupe(masque)
    print(f"{nom:<12} survie à 36 mois = {S_g[36]:.3f} | CLV = {m_mensuelle * np.sum(S_g / (1 + taux)
    ↪ ** mois):.1f} €")
```

```
tous          survie à 36 mois = 0.453 | CLV = 186.1 €
sans offre    survie à 36 mois = 0.384 | CLV = 165.8 €
avec offre     survie à 36 mois = 0.521 | CLV = 205.8 €
```

Étape 5 : l'offre est-elle rentable ? Gain net = $m \sum_t (S_1 - S_0)(1 + r)^{-t} - \text{coût}$, selon la marge et le taux d'actualisation.

```
S0, S1 = S_groupe(groupe["sans offre"]), S_groupe(groupe["avec offre"])
cout_offre = 10.0
res = pd.DataFrame({"marge {m} €": [m * np.sum((S1 - S0) * (1 + r) ** (-mois)) - cout_offre for r in
    ↪ (0.005, 0.01, 0.02)] for m in (3, 6, 9)},
    index=[f"taux {100 * r:.1f} %/mois" for r in (0.005, 0.01, 0.02)])
print(res.round(1).to_string())
```

	marge 3 €	marge 6 €	marge 9 €
taux 0.5 %/mois	16.5	42.9	69.4
taux 1.0 %/mois	10.0	30.0	50.0
taux 2.0 %/mois	2.4	14.7	27.1

Lecture. Le gain net est positif dans les neuf cas, mais son ordre de grandeur varie d'un facteur 30 : la conclusion « l'offre est rentable » est robuste, pas le chiffre.

Application 5.6 — Risques concurrents

Objectif. Estimer les incidences cumulées de deux causes de sortie (départ volontaire, fermeture forcée), voir pourquoi « 1 – Kaplan-Meier » trompe, et comparer modèle par cause et modèle de Fine et Gray (livre, 5.5).

Étape 1 : dix clients. Issues : 0 = censuré, 1 = départ volontaire, 2 = fermeture forcée.

```
t10 = np.array([2, 3, 4, 5, 6, 7, 8, 9, 10, 12.])
c10 = np.array([1, 2, 1, 0, 2, 1, 0, 1, 2, 0])
tj10, S10, (F1_10, F2_10) = incidence_cumulee(t10, c10)
print(pd.DataFrame({"t_j": tj10, "S": S10, "F1": F1_10, "F2": F2_10, "S+F1+F2": S10 + F1_10 +
    ↪ F2_10}).round(3).to_string(index=False))

tj_n, S_n, (F1_naif,) = incidence_cumulee(t10, np.where(c10 == 1, 1, 0)) # cause 2 traitée
    ↪ comme censure
print(f"1 - KM naïf à t = 9 : {1 - S_n[-1]:.3f} | incidence correcte F1(9) = {F1_10[5]:.3f}")
```

t_j	S	F1	F2	S+F1+F2
2.0	0.900	0.100	0.000	1.0
3.0	0.800	0.100	0.100	1.0
4.0	0.700	0.200	0.100	1.0
6.0	0.583	0.200	0.217	1.0
7.0	0.467	0.317	0.217	1.0
9.0	0.311	0.472	0.217	1.0
10.0	0.156	0.472	0.372	1.0

1 - KM naïf à t = 9 : 0.580 | incidence correcte F1(9) = 0.472

Étape 2 : une simulation à vérité connue. 6 000 clients : départ volontaire de loi de Weibull (forme 1,3), allongé par l'offre ; fermeture forcée à taux constant de 0,6 % par mois, multiplié par 2,2 pour le canal Réseaux, sans effet de l'offre ; censure uniforme entre 12 et 72 mois.

```

rng = np.random.default_rng(55)
n = 6000
offre = rng.integers(0, 2, n)
reseaux = (rng.random(n) < 0.45).astype(int)
T1 = 40 * np.exp(0.40 * offre) * rng.weibull(1.3, n) # départ volontaire
T2 = rng.exponential(1 / (0.006 * np.exp(0.8 * reseaux))) # fermeture forcée
C = rng.uniform(12, 72, n)
t_obs = np.minimum.reduce([T1, T2, C])
cause = np.where(C <= np.minimum(T1, T2), 0, np.where(T1 <= T2, 1, 2))
rc = pd.DataFrame({"duree": t_obs, "cause": cause, "offre": offre, "reseaux": reseaux})
rc.to_csv("donnees/ch05-risques-concurrents.csv", index=False) # relu par R plus bas
print("issues (0 = censuré) :", rc["cause"].value_counts().sort_index().to_dict())

```

```
issues (0 = censuré) : {0: 2079, 1: 2655, 2: 1266}
```

Étape 3 : incidences cumulées, estimées de trois façons.

```

tj, S, (F1, F2) = incidence_cumulee(rc["duree"], rc["cause"])
tj_1, S_1, (F1_naif,) = incidence_cumulee(rc["duree"], np.where(rc["cause"] == 1, 1, 0))
for t in (12, 24, 36, 48):
    i, il = np.searchsorted(tj, t, side="right") - 1, np.searchsorted(tj_1, t, side="right") - 1
    print(f"{t}>3} mois : S = {S[i]:.3f} | F1 = {F1[i]:.3f} | F2 = {F2[i]:.3f} | 1 - KM naïf = {F1_naif[i]:.3f}")

```

```

12 mois : S = 0.762 | F1 = 0.143 | F2 = 0.095 | 1 - KM naïf = 0.151
24 mois : S = 0.538 | F1 = 0.299 | F2 = 0.163 | 1 - KM naïf = 0.335
36 mois : S = 0.367 | F1 = 0.422 | F2 = 0.211 | 1 - KM naïf = 0.494
48 mois : S = 0.251 | F1 = 0.514 | F2 = 0.235 | 1 - KM naïf = 0.627

```

```

library(cmprsk)
rc <- read.csv("donnees/ch05-risques-concurrents.csv")
ci <- cuminc(rc$duree, rc$cause)
print(round(timepoints(ci, c(12, 24, 36, 48))$est, 4))

```

```

      12      24      36      48
1 1 0.1428 0.2993 0.4217 0.5141
1 2 0.0948 0.1629 0.2109 0.2350

```

```

from lifelines import AalenJohansenFitter

aj1 = AalenJohansenFitter(calculate_variance=False, seed=1).fit(rc["duree"], rc["cause"],
    ↪ event_of_interest=1)
print([round(float(aj1.cumulative_density_.loc[:,t].iloc[-1, 0]), 4) for t in (12, 24, 36, 48)])

```

```
[0.1428, 0.2993, 0.4217, 0.5141]
```

Étape 4 : modèle par cause et modèle de Fine et Gray.

```

from statsmodels.duration.hazard_regression import PHReg

for k in (1, 2):
    m = PHReg(rc["duree"], rc[["offre", "reseaux"]], status=(rc["cause"] == k).astype(int),
    ↪ ties="efron").fit()
    print(f"cause {k} : HR propres à la cause", {nom: round(float(v), 3) for nom, v in zip(["offre",
    ↪ "reseaux"], np.exp(m.params))})

```

```

cause 1 : HR propres à la cause {'offre': 0.584, 'reseaux': 0.974}
cause 2 : HR propres à la cause {'offre': 1.033, 'reseaux': 2.316}

```

```

cov <- cbind(offre = rc$offre, reseaux = rc$reseaux)
for (k in 1:2) {
    f <- crr(rc$duree, rc$cause, cov, failcode = k, cencode = 0)
}

```

```
cat("Fine-Gray, cause", k, "\n"); print(round(summary(f)$conf.int[, c(1, 3, 4)], 3))
}
```

```
Fine-Gray, cause 1
      exp(coef) 2.5% 97.5%
offre      0.604 0.560 0.652
reseaux    0.792 0.733 0.855
Fine-Gray, cause 2
      exp(coef) 2.5% 97.5%
offre      1.219 1.091 1.361
reseaux    2.280 2.036 2.554
```

Lecture. Dans le modèle par cause, l'offre réduit le risque de départ volontaire ($HR \approx 0,58$) et n'a aucun effet sur celui de fermeture ; dans le modèle de Fine et Gray, elle *augmente* pourtant l'incidence de la fermeture forcée ($\approx 1,22$), par effet de compétition. De même, Réseaux n'a aucun effet direct sur le départ volontaire, mais un rapport de sous-distribution de 0,79 : les clients de ce canal sont plus souvent éliminés avant.

Exercices

Les exercices sont classés par difficulté : ★ (application directe), ★★ (demande de réfléchir), ★★★ (synthèse). **Cherchez d'abord seul(e)**, à la main quand c'est demandé, avant de lire le corrigé. Les fonctions de `build/outils_ch05.py` (`kaplan_meier`, `logrank`, `cox_ph`, `test_ph`, `ajuster`, `incidence_cumulee`, etc.), importées dans la préparation, sont réutilisées dans les corrigés.

Exercice 5.1 ★ — Censure et Kaplan-Meier à la main (sections 5.1, 5.2 du livre)

La gérante suit six clients : (4;parti), (7;censuré), (9;parti), (12;parti), (15;censuré), (20;censuré) (durées en mois). (a) Calculez la moyenne de toutes les durées, puis celle des seuls clients partis. (b) Calculez la courbe de Kaplan-Meier à la main. (c) Donnez la médiane de survie. (d) Calculez la durée moyenne restreinte jusqu'à 20 mois.

Exercice 5.2 ★ — Relations entre les fonctions (section 5.1 du livre)

Un client a , à l'âge t de la relation (en mois), un risque instantané $h(t) = 0,0008t$. (a) Déduisez $H(t)$ et $S(t)$. (b) Calculez $S(24)$ et la médiane. (c) De quelle loi de Weibull s'agit-il ? Donnez sa durée moyenne.

Exercice 5.3 ★ — Taux constant (section 5.1 du livre)

Sur un échantillon, on observe 40 départs pour un total de 1 600 mois-clients d'exposition. (a) Estimez le taux de départ mensuel λ (exponentielle), son écart-type et un intervalle de confiance à 95 %. (b) Estimez $S(24)$ et la durée moyenne. (c) Testez $H_0 : \lambda = 0,03$ par le test de Wald et par le rapport de vraisemblance.

Exercice 5.4 ★★ — Greenwood (section 5.2 du livre)

Huit clients : (2;1), (3;1), (3;1), (5;0), (6;1), (8;0), (9;1), (11;0) (durée ; 1 = parti, 0 = censuré). Construisez le tableau de Kaplan-Meier (avec les ex aequo), puis donnez $\hat{S}(6)$, son erreur standard de Greenwood et son intervalle de confiance log-log à 95 %.

Exercice 5.5 ★★ — Log-rank (section 5.2 du livre)

Deux groupes de cinq clients. Groupe A : (3;1), (6;1), (8;0), (10;1), (12;0). Groupe B : (5;1), (7;1), (9;1), (11;0), (13;0). Calculez à la main le test du log-rank (tableau aux instants de départ : ensembles à risque, départs attendus, variances), concluez, puis vérifiez avec `statsmodels`.

Exercice 5.6 ★★ — Durée moyenne restreinte (section 5.2 du livre)

Les courbes de survie de deux campagnes sont des escaliers. Campagne A : $S = 1$ jusqu'à 6 mois, 0,8 sur $[6, 12[$, 0,5 sur $[12, 18[$, 0,3 ensuite. Campagne B : $S = 1$ jusqu'à 9 mois, 0,9 sur $[9, 15[$, 0,7 sur $[15, 21[$, 0,6 ensuite. Calculez la durée moyenne restreinte à 24 mois de chaque campagne et leur différence. Que signifie ce nombre ?

Exercice 5.7 ★★ — Lire une sortie de Cox (section 5.3 du livre)

Un modèle de Cox donne : offre de bienvenue $\hat{\beta} = -0,40$ (ET 0,065), âge $\hat{\beta} = -0,0136$ (ET 0,0030), canal Réseaux (contre Boutique) $\hat{\beta} = 0,635$ (ET 0,084). (a) Donnez pour chaque variable le rapport de risques, son IC95 et le z de Wald. (b) Quel est l'effet de dix années d'âge de plus ? (c) Quel est le rapport de risques d'un client Réseaux *avec* offre contre un client Boutique *sans* offre, à âge égal ? (d) La survie à 24 mois d'un client de référence est de 0,80 : quelle est celle du même client avec l'offre ? (e) Un AFT Weibull donne $\hat{\gamma}_{\text{offre}} = 0,35$ avec $\hat{\sigma} = 0,74$: quel rapport de risques équivalent ?

Exercice 5.8 ★★ — Le biais d'immortalité (section 5.3 du livre)

Simulez 2 000 clients dont les durées sont exponentielles de taux 2 % par mois (graine 8), censurées uniformément entre 18 et 48 mois. Un cadeau est remis au mois 6 à la moitié des clients encore présents. Estimez l'effet du cadeau (qui n'en a aucun) (a) en traitant « a reçu le cadeau » comme une variable fixe, (b) correctement, avec une variable dépendant du temps. Commentez.

Exercice 5.9 ★★ — Estimer la forme de Weibull de deux façons (section 5.4 du livre)

Pour l'ensemble des 2 000 clients (sans covariables), estimez la forme k de la loi de Weibull (a) par le graphique de Weibull : régression de $\ln(-\ln \hat{S}_{KM})$ sur $\ln t$ entre 6 et 60 mois ; (b) par maximum de vraisemblance. Comparez, et expliquez l'écart avec la valeur $k = 1,35$ utilisée pour simuler.

Exercice 5.10 ★★★ — Log-rank et Cox (section 5.3 du livre)

- (a) Montrez que le score du modèle de Cox à une variable binaire en $\beta = 0$ vaut $O_1 - E_1$. (b) Vérifiez numériquement, sur les 2 000 clients (variable `offre_bienvenue`), que la statistique de score U^2/I est très proche du χ^2 du log-rank. Pourquoi n'est-elle pas *exactement* égale ?

Exercice 5.11 ★★★ — Décider : le seuil de rentabilité (section 5.4 du livre)

Reprenez les survies moyennes avec et sans offre du 5.4.5. L'offre de bienvenue coûte maintenant 25 € par client. À partir de quelle **marge mensuelle** par client actif est-elle rentable, pour un taux d'actualisation de 1 % par mois ? Et pour 2 % ?

Exercice 5.12 ★★★ — Risques concurrents à la main (section 5.5 du livre)

Huit clients, durées et causes de sortie (0 = censuré, 1 = départ volontaire, 2 = fermeture forcée) : (1;1), (2;2), (3;1), (4;0), (5;2), (6;1), (7;0), (9;2). Calculez à la main les incidences cumulées $\hat{F}_1$ et $\hat{F}_2$ (estimateur d'Aalen-Johansen) et la survie totale. Vérifiez que $\hat{S} + \hat{F}_1 + \hat{F}_2 = 1$. Comparez $\hat{F}_1$ à « 1 – KM » où la cause 2 est traitée comme une censure.

Exercice 5.13 ★★★ — Proportionnalité des risques sur des données réelles (section 5.3 du livre)

Reprenez les données de récurrence de Rossi (5.3.9). Appliquez le test de score de proportionnalité des risques du 5.3.5 à chacune des sept covariables. Quelles variables posent problème ? Que feriez-vous ?

Corrigés**Corrigé 5.1**

- (a) Moyenne de toutes les durées : $(4 + 7 + 9 + 12 + 15 + 20)/6 \approx 11,17$ mois ; moyenne des seuls clients partis : $(4 + 9 + 12)/3 \approx 8,33$ mois. Les deux sont biaisées (5.1.1). (b) Départs aux mois 4, 9 et 12. À 4 mois, 6 clients à risque : $5/6 = 0,833$. À 9 mois (le client censuré à 7 est sorti), 4 clients à risque : $0,833 \times 3/4 = 0,625$. À 12 mois, 3 clients à risque : $0,625 \times 2/3 = 0,417$. (c) La courbe passe sous 0,5 au mois 12 : **médiane = 12 mois**. (d) $RMST(20) = 4 \times 1 + 5 \times 0,833 + 3 \times 0,625 + 8 \times 0,417 = 4 + 4,167 + 1,875 + 3,333 = 13,375$ mois.

```
import numpy as np
import pandas as pd
from scipy import stats

y1 = np.array([4, 7, 9, 12, 15, 20.]); d1 = np.array([1, 0, 1, 1, 0, 0])
tj1, n1, dj1, S1_, gw1 = kaplan_meier(y1, d1)
print("moyenne de toutes les durées :", round(y1.mean(), 2), "| des seuls partis :", round(y1[d1 ==
↪ 1].mean(), 2))
print("S aux instants de départ", tj1, ":", S1_.round(4), "| à risque :", n1)
print("médiane :", tj1[S1_ <= 0.5][0], "| RMST(20) :", round(rmst(tj1, S1_, 20), 3))
```

```
moyenne de toutes les durées : 11.17 | des seuls partis : 8.33
S aux instants de départ [ 4.  9. 12.] : [0.8333 0.625  0.4167] | à risque : [6 4 3]
médiane : 12.0 | RMST(20) : 13.375
```

Corrigé 5.2

- (a) $H(t) = \int_0^t 0,0008u \, du = 0,0004 t^2$, donc $S(t) = \exp(-0,0004 t^2)$. (b) $S(24) = \exp(-0,0004 \times 576) = e^{-0,2304} \approx 0,794$; la médiane vérifie $0,0004 t^2 = \ln 2$, donc $t = \sqrt{\ln 2 / 0,0004} \approx 41,6$ mois. (c) Une Weibull a $H(t) = (t/\sigma)^k$: on lit $k = 2$ et $\sigma^{-2} = 0,0004$, soit $\sigma = 50$ mois. La durée moyenne vaut $\sigma \Gamma(1 + 1/k) = 50 \Gamma(1,5) \approx 44,3$ mois.

```
from math import gamma, log
from scipy.integrate import quad
print("S(24) =", round(np.exp(-0.0004 * 24 ** 2), 4), "| médiane =", round(np.sqrt(log(2) / 0.0004),
↪ 2))
print("moyenne par la formule :", round(50 * gamma(1.5), 2), "| par intégration de S :",
↪ round(quad(lambda t: np.exp(-0.0004 * t ** 2), 0, np.inf)[0], 2))
```

$S(24) = 0.7942$ | médiane = 41.63
moyenne par la formule : 44.31 | par intégration de S : 44.31

Corrigé 5.3

(a) $\hat{\lambda} = D/E = 40/1600 = 0,025$ par mois ; $\widehat{se} = \hat{\lambda}/\sqrt{D} = 0,025/\sqrt{40} \approx 0,00395$; IC95 : $0,025 \pm 1,96 \times 0,00395$, soit $[0,0173 ; 0,0327]$. (b) $\hat{S}(24) = e^{-0,025 \times 24} = e^{-0,6} \approx 0,549$; durée moyenne $1/\hat{\lambda} = 40$ mois. (c) Wald : $z = (0,025 - 0,03)/0,00395 \approx -1,26$, $p \approx 0,21$. Rapport de vraisemblance : $2[\ell(\hat{\lambda}) - \ell(0,03)] = 2[D \ln(\hat{\lambda}/0,03) - (\hat{\lambda} - 0,03)E] = 2[40 \ln(0,8333) + 8] \approx 1,41$, $p \approx 0,23$. Les deux tests concluent de la même façon : **on ne rejette pas** $\lambda = 0,03$ (les données sont compatibles aussi avec ce taux).

```
D, E = 40, 1600
lam = D / E; se = lam / np.sqrt(D)
print(f"lambda = {lam:.4f}, ET = {se:.5f}, IC95 = [{lam - 1.96 * se:.4f} ; {lam + 1.96 * se:.4f}]")
print(f"S(24) = {np.exp(-lam * 24):.4f}, durée moyenne = {1 / lam:.1f} mois")
z = (lam - 0.03) / se
lr = 2 * (D * np.log(lam / 0.03) - (lam - 0.03) * E)
print(f"Wald : z = {z:.3f}, p = {2 * stats.norm.sf(abs(z)):.3f} | rapport de vraisemblance : chi2 = {lr:.3f}, p = {stats.chi2.sf(lr, 1):.3f}")
```

lambda = 0.0250, ET = 0.00395, IC95 = [0.0173 ; 0.0327]
S(24) = 0.5488, durée moyenne = 40.0 mois
Wald : z = -1.265, p = 0.206 | rapport de vraisemblance : chi2 = 1.414, p = 0.234

Corrigé 5.4

Départs : mois 2 (1 départ, 8 à risque : 7/8), mois 3 (2 départs, 7 à risque : 5/7), mois 6 (1 départ, 4 à risque, car le client censuré à 5 est sorti : 3/4), mois 9 (1 départ, 2 à risque : 1/2). D'où $\hat{S}(2) = 0,875$, $\hat{S}(3) = 0,875 \times 5/7 = 0,625$, $\hat{S}(6) = 0,625 \times 3/4 = 0,469$, $\hat{S}(9) = 0,234$. Greenwood : $\sum \frac{d_j}{n_j(n_j - d_j)} = \frac{1}{8 \times 7} + \frac{2}{7 \times 5} + \frac{1}{4 \times 3} = 0,0179 + 0,0571 + 0,0833 = 0,1583$; l'erreur standard de $\hat{S}(6)$ vaut $0,469 \times \sqrt{0,1583} \approx 0,187$. Intervalle log-log : $\hat{S}^{\exp(\pm 1,96 \sqrt{G}/|\ln \hat{S}|)}$ avec $1,96 \times \sqrt{0,1583}/|\ln 0,469| = 0,780/0,757 = 1,03$, d'où $[0,469^{2,80} ; 0,469^{1/2,80}] \approx [0,12 ; 0,76]$: un intervalle immense, faute de données.

```
y4 = np.array([2, 3, 3, 5, 6, 8, 9, 11.]); d4 = np.array([1, 1, 1, 0, 1, 0, 1, 0])
tj4, n4, dj4, S4, gw4 = kaplan_meier(y4, d4)
print(pd.DataFrame({"t_j": tj4, "à risque": n4, "départs": dj4, "S": S4.round(4), "somme Greenwood": gw4.round(4)}).to_string(index=False))
s, plan, log_, loglog = intervalles(6, tj4, S4, gw4)
print(f"S(6) = {s:.4f} ; ET = {s * np.sqrt(surv_at(6, tj4, gw4)):.4f} ; IC log-log = [{loglog[0]:.3f} ; {loglog[1]:.3f}])")
```

t_j	à risque	départs	S	somme Greenwood
2.0	8	1	0.8750	0.0179
3.0	7	2	0.6250	0.0750
6.0	4	1	0.4688	0.1583
9.0	2	1	0.2344	0.6583

S(6) = 0.4688 ; ET = 0.1865 ; IC log-log = [0.120 ; 0.763]

Corrigé 5.5

Aux instants de départ 3, 5, 6, 7, 9 et 10, on a respectivement $n_j = 10, 9, 8, 7, 5, 4$ clients à risque, dont $n_{Aj} = 5, 4, 4, 3, 2, 2$ dans le groupe A. Chaque instant compte un seul départ : $E_{Aj} = n_{Aj}/n_j$ et $V_j = \frac{n_{Aj}}{n_j} (1 - \frac{n_{Aj}}{n_j})$. Le code donne le détail :

```
y5 = np.array([3, 6, 8, 10, 12, 5, 7, 9, 11, 13.]); d5 = np.array([1, 1, 0, 1, 0, 1, 1, 1, 0, 0])
g5 = np.array(["A"] * 5 + ["B"] * 5)
lignes, O1, E1, V1 = [], 0, 0.0, 0.0
for t in np.unique(y5[d5 == 1]):
    n_ = int(np.sum(y5 >= t)); nA = int(np.sum((y5 >= t) & (g5 == "A")))
    dd = int(np.sum((y5 == t) & (d5 == 1))); dA = int(np.sum((y5 == t) & (d5 == 1) & (g5 == "A")))
    e = dd * nA / n_; v = dd * (nA / n_) * (1 - nA / n_) * (n_ - dd) / (n_ - 1)
    lignes.append((t, n_, nA, dd, dA, round(e, 4), round(v, 4))); O1 += dA; E1 += e; V1 += v
print(pd.DataFrame(lignes, columns=["t_j", "n_j", "n_Aj", "d_j", "départ en A", "E_Aj",
    ↪ "V_j"]).to_string(index=False))
chi5 = (O1 - E1) ** 2 / V1
print(f"O_A = {O1}, E_A = {E1:.3f}, V = {V1:.3f}, chi2 = {chi5:.3f}, p = {stats.chi2.sf(chi5, 1):.3f}")
from statsmodels.duration.survfunc import survdiff
print("statsmodels : chi2 = %.3f, p = %.3f" % survdiff(y5, d5, g5))
```

t_j	n_j	n_Aj	d_j	départ en A	E_Aj	V_j
3.0	10	5	1	1	0.5000	0.2500
5.0	9	4	1	0	0.4444	0.2469
6.0	8	4	1	1	0.5000	0.2500
7.0	7	3	1	0	0.4286	0.2449
9.0	5	2	1	0	0.4000	0.2400
10.0	4	2	1	1	0.5000	0.2500

O_A = 3, E_A = 2.773, V = 1.482, chi2 = 0.035, p = 0.852
statsmodels : chi2 = 0.035, p = 0.852

Le groupe A a 3 départs pour 2,77 attendus : $\chi^2 \approx 0,03, p \approx 0,85$. Avec dix clients, on ne peut rien conclure ; c'est tout à fait normal, et c'est la raison pour laquelle on ne compare pas des groupes aussi petits.

Corrigé 5.6

$RMST_A(24) = 6 \times 1 + 6 \times 0,8 + 6 \times 0,5 + 6 \times 0,3 = 6 + 4,8 + 3 + 1,8 = 15,6$ mois ; $RMST_B(24) = 9 \times 1 + 6 \times 0,9 + 6 \times 0,7 + 3 \times 0,6 = 9 + 5,4 + 4,2 + 1,8 = 20,4$ mois. La différence est de **4,8 mois** : en moyenne, sur les 24 premiers mois, un client de la campagne B reste 4,8 mois de plus dans la clientèle qu'un client de la campagne A. Contrairement au rapport de risques, cette quantité s'interprète sans hypothèse de proportionnalité et se lit directement en unités de temps (ou, multipliée par la marge mensuelle, en euros).

```
tjA, SA = np.array([6, 12, 18.]), np.array([0.8, 0.5, 0.3])
tjB, SB = np.array([9, 15, 21.]), np.array([0.9, 0.7, 0.6])
print("RMST(24) A :", round(rmst(tjA, SA, 24), 2), "| B :", round(rmst(tjB, SB, 24), 2), "| différence
    ↪ :", round(rmst(tjB, SB, 24) - rmst(tjA, SA, 24), 2))
```

RMST(24) A : 15.6 | B : 20.4 | différence : 4.8

Corrigé 5.7

- (a) $HR = e^{\hat{\beta}}$, $IC = e^{\hat{\beta} \pm 1.96 se}$, $z = \hat{\beta}/se$: offre 0,670 ([0,590 ; 0,761], $z = -6,15$) ; âge 0,986 ([0,981 ; 0,992], $z = -4,53$) ; Réseaux 1,887 ([1,60 ; 2,22], $z = 7,56$). (b) $e^{10 \times (-0,0136)} \approx 0,873$: dix ans de plus réduisent le risque d'environ 13 %. (c) Les effets se multiplient : $e^{-0,40+0,635} = e^{0,235} \approx 1,26$ (le canal Réseaux l'emporte sur l'offre). (d) $S(t | x) = S_0(t) \exp(x^T \hat{\beta})$: $0,80^{0,670} \approx 0,861$. (e) $\hat{\beta} = -\hat{\gamma}/\hat{\sigma} = -0,35/0,74 \approx -0,473$, soit $HR = e^{-0,473} \approx 0,623$.

```
b = np.array([-0.40, -0.0136, 0.635]); se_ = np.array([0.065, 0.0030, 0.084])
print(pd.DataFrame({"HR": np.exp(b), "IC bas": np.exp(b - 1.96 * se_), "IC haut": np.exp(b + 1.96 *
↪ se_), "z": b / se_}, index=["offre", "age", "Réseaux"]).round(3).to_string())
print("+10 ans :", round(np.exp(10 * b[1]), 3), "| Réseaux avec offre / Boutique sans offre :",
↪ round(np.exp(b[0] + b[2]), 3))
print("S(24) avec offre :", round(0.80 ** np.exp(b[0]), 3), "| HR depuis AFT :", round(np.exp(-0.35 /
↪ 0.74), 3))
```

	HR	IC bas	IC haut	z
offre	0.670	0.590	0.761	-6.154
age	0.986	0.981	0.992	-4.533
Réseaux	1.887	1.601	2.225	7.560

+10 ans : 0.873 | Réseaux avec offre / Boutique sans offre : 1.265
 S(24) avec offre : 0.861 | HR depuis AFT : 0.623

Corrigé 5.8

Dans l'analyse (a), le cadeau n'est donné qu'à ceux qui ont **survécu** jusqu'au mois 6 : ils ont une avance garantie, et le modèle y voit un effet protecteur qui n'existe pas. Dans l'analyse (b), le cadeau est une variable qui passe de 0 à 1 au mois 6 (deux épisodes pour les clients concernés), et seuls les clients **présents au mois 6** sont comparés entre eux.

```
from statsmodels.duration.hazard_regression import PHReg

rng = np.random.default_rng(8)
N = 2000
T = rng.exponential(1 / 0.02, N)
C = rng.uniform(18, 48, N)
Y, D = np.minimum(T, C), (T <= C).astype(int)
cadeau = (Y > 6) & (rng.random(N) < 0.5)
naif = PHReg(Y, cadeau.astype(float)[: , None], status=D, ties="efron").fit()
porteurs = np.where(cadeau)[0]
debut = np.concatenate([np.zeros(N), np.full(len(porteurs), 6.0)])
fin = np.concatenate([np.where(cadeau, 6.0, Y), Y[porteurs]])
statut = np.concatenate([np.where(cadeau, 0, D), D[porteurs]])
x_tv = np.concatenate([np.zeros(N), np.ones(len(porteurs))])
juste = PHReg(fin, x_tv[: , None], status=statut, entry=debut, ties="efron").fit()
for nom, m in (("naïve (variable fixe)", naif), ("correcte (dépend du temps)", juste)):
    print(f"{nom:<28} HR = {np.exp(m.params[0]):.2f} IC95 [{np.exp(m.params[0] - 1.96 *
↪ m.bse[0]):.2f} ; {np.exp(m.params[0] + 1.96 * m.bse[0]):.2f}])")
```

naïve (variable fixe)	HR = 0.64	IC95 [0.56 ; 0.74]
correcte (dépend du temps)	HR = 1.00	IC95 [0.86 ; 1.16]

L'analyse naïve conclut à un effet protecteur net (un HR bien inférieur à 1), l'analyse correcte à un effet nul (HR proche de 1, intervalle contenant 1) : c'est le **bias d'immortalité**.

Corrigé 5.9

- (a) Si $S(t) = \exp[-(t/\sigma)^k]$, alors $\ln(-\ln S) = k \ln t - k \ln \sigma$: la pente de la droite est k . (b) Le maximum de vraisemblance sans covariable est l'estimation de $k = 1/\hat{\sigma}$ dans ajuster avec une seule colonne de 1.

```
c = pd.read_csv("donnees/clients.csv")
y, d = c["duree_mois"].to_numpy(), c["churn"].to_numpy()
tj_, _, _, S_, _ = kaplan_meier(y, d)
garde = (tj_ >= 6) & (tj_ <= 60)
pente, ord_, r, _, _ = stats.linregress(np.log(tj_[garde]), np.log(-np.log(S_[garde])))
mv = ajuster("weibull", y, d, np.ones((len(y), 1)))
print(f"forme par le graphique de Weibull : k = {pente:.3f} (r = {r:.3f}) | forme par maximum de
↪ vraisemblance : k = {1 / np.exp(mv['theta'][-1]):.3f}")
```

```
forme par le graphique de Weibull : k = 1.296 (r = 1.000) | forme par maximum de vraisemblance : k =
↪ 1.281
```

Les deux méthodes donnent une forme voisine de 1,3 (1,30 et 1,28) et légèrement inférieure à la vraie valeur 1,35. La raison est celle du 5.4.7 : en **ignorant les covariables** (offre, canal, âge, facteur de service non observé), le risque de la population est un mélange de risques individuels, dont la croissance apparente est plus lente. L'ajout des covariables mesurées dans le modèle du 5.4.3 faisait passer la forme à 1,32, et l'ajout du facteur non observé (modèle « oracle » du 5.4.7) à 1,40 : la vraie valeur (1,35) n'est approchée, à l'erreur d'échantillonnage près, que si l'on tient compte de *toutes* les covariables, y compris celles qu'on ne mesure pas. Les deux méthodes d'estimation (graphique, maximum de vraisemblance) s'accordent entre elles ; c'est le *modèle* qui est incomplet, pas la méthode.

Corrigé 5.10

- (a) En $\beta = 0$, $e^{\beta x_k} = 1$: la moyenne pondérée de x dans l'ensemble à risque R_j est la proportion n_{1j}/n_j de clients du groupe 1. Le score est $\sum_j \sum_{i \text{ parti en } t_j} (x_i - n_{1j}/n_j) = \sum_j (d_{1j} - d_j n_{1j}/n_j) = O_1 - E_1$. (b) L'information en $\beta = 0$ est $\sum_i \text{Var}_{R_i}(x) = \sum_i \frac{n_1}{n} (1 - \frac{n_1}{n})$ (somme sur *chaque* départ), alors que le log-rank utilise la variance hypergéométrique, avec le facteur $\frac{n_j - d_j}{n_j - 1}$ qui corrige les départs simultanés. Les deux coïncident exactement quand il n'y a **jamais** d'ex aequo.

```
off = c["offre_bienvenue"].to_numpy().astype(float)
U0, I0 = score_info(0.0, y, d, off) # fonction du 5.3.2 : boucle sur chaque
↪ départ (ex aequo à la Breslow)
O, E, chi_lr, _, _ = logrank(y, d, off)
print(f"Cox : score U = {U0:.3f}, information I = {I0:.3f}, U²/I = {U0 ** 2 / I0:.3f}")
print(f"log-rank : O1 - E1 = {O[1] - E[1]:.3f}, chi2 = {chi_lr:.3f}")
```

```
Cox : score U = -92.383, information I = 240.231, U²/I = 35.527
log-rank : O1 - E1 = -92.383, chi2 = 35.535
```

Le score est exactement $O_1 - E_1$ (même valeur, signe compris, puisque $x = 1$ désigne le groupe avec offre), et les deux statistiques ne diffèrent que de 0,008 sur 35,5 : la différence vient de la correction des ex aequo dans la variance.

Corrigé 5.11

Le gain actualisé de l'offre vaut $\Delta = \sum_t (S_1(t) - S_0(t))(1+r)^{-t}$ « mois de présence actualisés » ; l'offre est rentable si $m\Delta > 25$, c'est-à-dire $m > m^* = 25/\Delta$.

```
Xc = np.column_stack([np.ones(len(c)), pd.get_dummies(c[["offre_bienvenue", "age",
↳ "canal_acquisition"]], drop_first=True, dtype=float).to_numpy())
wb = ajuster("weibull", y, d, Xc) # Weibull du livre (5.4.3) : forme k et
↳ échelle de chaque client
k_hat, lam_i, mois = 1 / np.exp(wb["theta"][-1]), np.exp(Xc @ wb["theta"][:-1]), np.arange(240)
S_groupe = lambda masque: np.array([np.mean(np.exp(-(t / lam_i[masque]) ** k_hat)) for t in mois])
S0, S1 = S_groupe((c["offre_bienvenue"] == 0).to_numpy()), S_groupe((c["offre_bienvenue"] ==
↳ 1).to_numpy())
cout = 25.0
for r in (0.01, 0.02):
    v = (1 + r) ** (-mois)
    delta = np.sum((S1 - S0) * v)
    print(f"taux {100 * r:.0f} %/mois : gain en mois de présence actualisés = {delta:.3f} -> marge de
↳ rentabilité m* = {cout / delta:.2f} €/mois")
```

```
taux 1 %/mois : gain en mois de présence actualisés = 6.663 -> marge de rentabilité m* = 3.75 €/mois
taux 2 %/mois : gain en mois de présence actualisés = 4.124 -> marge de rentabilité m* = 6.06 €/mois
```

Avec une actualisation de 1 % par mois, l'offre devient rentable dès que la marge mensuelle par client actif dépasse environ 3,75 € ; avec 2 %, il faut environ 6 €. L'offre est donc beaucoup moins coûteuse à justifier si la marge est élevée : c'est la lecture pratique du tableau de sensibilité du 5.4.5.

Corrigé 5.12

Instants de sortie : 1, 2, 3, 5, 6, 9 (les censures aux mois 4 et 7 réduisent seulement les ensembles à risque). Aux mois 1 ($n = 8$) : $d_1 = 1$; 2 ($n = 7$) : $d_2 = 1$; 3 ($n = 6$) : $d_1 = 1$; 5 ($n = 4$) : $d_2 = 1$; 6 ($n = 3$) : $d_1 = 1$; 9 ($n = 1$) : $d_2 = 1$. On applique $\hat{F}_k(t) = \sum \hat{S}(t_j^-) d_{kj}/n_j$.

```
y12 = np.array([1, 2, 3, 4, 5, 6, 7, 9.]); c12 = np.array([1, 2, 1, 0, 2, 1, 0, 2])
tj12, S12, (F1_12, F2_12) = incidence_cumulee(y12, c12)
print(pd.DataFrame({"t_j": tj12, "S": S12, "F1": F1_12, "F2": F2_12, "S + F1 + F2": S12 + F1_12 +
↳ F2_12}).round(4).to_string(index=False))
tj_n12, S_n12, (F1_naif12,) = incidence_cumulee(y12, np.where(c12 == 1, 1, 0))
print("1 - KM (cause 2 traitée comme censure), aux instants de la cause 1 :", (1 - S_n12).round(4), "|
↳ F1 correcte au dernier instant de cause 1 :", round(F1_12[-2], 4))
```

t_j	S	F1	F2	S + F1 + F2
1.0	0.8750	0.1250	0.0000	1.0
2.0	0.7500	0.1250	0.1250	1.0
3.0	0.6250	0.2500	0.1250	1.0
5.0	0.4688	0.2500	0.2812	1.0
6.0	0.3125	0.4062	0.2812	1.0
9.0	0.0000	0.4062	0.5938	1.0

1 - KM (cause 2 traitée comme censure), aux instants de la cause 1 : [0.125 0.2708 0.5139] | F1
 ↳ correcte au dernier instant de cause 1 : 0.4062

La somme $\hat{S} + \hat{F}_1 + \hat{F}_2$ vaut 1 à chaque instant. À 6 mois, la probabilité de départ volontaire est $\hat{F}_1(6) = 0,406$; en traitant la cause 2 comme une censure, on obtiendrait « 1 - KM » = 0,514, soit 11 points de trop : la compétition des fermetures forcées (aux mois 2, 5 et 9) est ignorée.

Corrigé 5.13

On applique `test_ph` à l'ajustement de Cox des données de Rossi (la fonction `cox_ph` du 5.3.3 accepte n'importe quelle matrice de covariables).

```
from lifelines.datasets import load_rossi

rossi = load_rossi()
Xr = rossi.drop(columns=["week", "arrest"])
fit_r = cox_ph(rossi["week"].to_numpy(), rossi["arrest"].to_numpy(), Xr.to_numpy())
chi_r, chi_glob_r = test_ph(rossi["week"].to_numpy(), rossi["arrest"].to_numpy(), Xr.to_numpy(),
    ↪ fit_r)
out = pd.DataFrame({"HR (Breslow)": np.exp(fit_r["beta"]), "chi2 PH": chi_r, "p":
    ↪ stats.chi2.sf(chi_r, 1)}, index=Xr.columns)
print(out.round(3).to_string())
print(f"test global : chi2 = {chi_glob_r:.2f} (7 ddl), p = {stats.chi2.sf(chi_glob_r, 7):.3f}")
```

	HR (Breslow)	chi2 PH	p
fin	0.685	1.371	0.242
age	0.944	0.893	0.345
race	1.369	2.245	0.134
wexp	0.860	4.128	0.042
mar	0.649	0.089	0.765
paro	0.919	0.021	0.886
prio	1.095	1.618	0.203
test global	chi2 = 10.93 (7 ddl), p = 0.142		

La proportionnalité est plausible pour six des sept covariables. Seule l'expérience professionnelle (`wexp`) a une p -valeur inférieure à 0,05 ($p = 0,042$), et le test **global** ne rejette rien ($p = 0,14$). Avec sept tests, obtenir une p -valeur à 0,04 arrive souvent par hasard (volume I, section 3.5.5) : ce n'est pas une preuve de violation. La démarche prudente : tracer le graphique log-log de `wexp`, et si un doute subsiste, **stratifier** sur cette variable (elle est binaire) ou ajouter un effet qui dépend du temps, puis voir si les autres coefficients, en particulier celui de l'aide financière, bougent. S'ils ne bougent pas, la conclusion principale est robuste.

Chapitre 6 : Statistique bayésienne et simulation — exercices et applications

Ce chapitre du cahier accompagne le chapitre 6 du livre. Il contient six **applications** guidées (une par section du livre) et quatorze **exercices** corrigés. Les données sont celles du volume : `donnees/clients.csv` (2 000 clients simulés) ; certaines applications simulent leurs propres données avec une graine fixe. Prérequis : avoir lu la section correspondante du livre.

Applications

Préparation (à exécuter d'abord)

Toutes les applications et les corrigés partagent les outils ci-dessous : imports, données, et quatre petites fonctions (marche aléatoire de Metropolis en dimension 1 et en dimension quelconque, taille d'échantillon effective, $\hat{R}$ découpé). Les algorithmes sont expliqués dans les sections 6.3 et 6.4 du livre.

```
import numpy as np
import pandas as pd
from scipy import stats
```

```
clients = pd.read_csv("donnees/clients.csv")
print(clients.shape, "| canaux :", sorted(clients["canal_acquisition"].unique()))
```

```
(2000, 12) | canaux : ['Boutique', 'Réseaux', 'Site']
```

```
def metropolis_1d(logp, x0, n, pas, rng):
    """Marche aléatoire de Metropolis en dimension 1 : proposition  $x' = x + pas * N(0, 1)$ ."""
    x, lp = x0, logp(x0)
    chaine, acceptes = np.empty(n), 0
    for i in range(n):
        prop = x + pas * rng.standard_normal()
        lpp = logp(prop)
        if np.log(rng.random()) < lpp - lp:  # critère d'acceptation, en logarithmes
            x, lp = prop, lpp
            acceptes += 1
        chaine[i] = x  # on recompte la valeur même si on a refusé
    return chaine, acceptes / n
```

```
def metropolis_multi(logp, x0, n, cov_prop, rng):
    """Même algorithme en dimension quelconque : proposition gaussienne de covariance cov_prop."""
```

```

L = np.linalg.cholesky(cov_prop)
x, lp = x0.copy(), logp(x0)
chaine, acceptes = np.empty((n, len(x0))), 0
for i in range(n):
    prop = x + L @ rng.standard_normal(len(x0))
    lpp = logp(prop)
    if np.log(rng.random()) < lpp - lp:
        x, lp = prop, lpp
        acceptes += 1
    chaine[i] = x
return chaine, acceptes / n

def autocorr(x, max_lag):
    x = np.asarray(x, dtype=float) - np.mean(x)
    n = len(x)
    f = np.fft.rfft(x, 2 * n) # transformée de Fourier avec remplissage
    ↪ de zéros
    ac = np.fft.irfft(f * np.conj(f))[:n]
    return (ac / ac[0])[: max_lag + 1]

def ess(x):
    """Taille d'échantillon effective ; la somme des autocorrélations s'arrête à la première
    ↪ négative."""
    rho = autocorr(x, max_lag=min(len(x) // 2, 1000))
    somme = 0.0
    for r in rho[1:]:
        if r < 0:
            break
        somme += r
    return len(x) / (1 + 2 * somme)

def split_rhat(ch):
    """ch : tableau (m chaînes, n itérations). Split-R-chapeau de Gelman-Rubin."""
    m, n = ch.shape
    parts = np.concatenate([ch[:, : n // 2], ch[:, n // 2 : 2 * (n // 2)]], axis=0) # 2m chaînes de
    ↪ longueur n/2
    N = parts.shape[1]
    W = parts.var(axis=1, ddof=1).mean()
    B = N * parts.mean(axis=1).var(ddof=1)
    return np.sqrt(((N - 1) / N * W + B / N) / W)

```

Application 6.1 — L'offre de bienvenue fonctionne-t-elle ? (section 6.1)

Objectif. Quantifier, avec des lois a posteriori, l'effet de l'offre de bienvenue (attribuée au hasard) sur le rachat à 12 mois, puis tester la solidité de la conclusion (a priori différents) et faire une prédiction.

Étape 1 — les effectifs. Combien de clients et de rachats dans chaque groupe ?

```

bilan = clients.groupby("offre_bienvenue")["rachat_12m"].agg(clients="size", rachats="sum")
bilan["frequence"] = (bilan["rachats"] / bilan["clients"]).round(4)
print(bilan)

```

	clients	rachats	frequence
offre_bienvenue			
0	985	441	0.4477
1	1015	578	0.5695

Étape 2 — une loi a posteriori par groupe. Avec un a priori Beta(1, 1), chaque groupe a pour a posteriori Beta(1 + y, 1 + n - y).

```

post = {}
for groupe, ligne in bilan.iterrows():
    a_post = 1 + ligne["rachats"]
    b_post = 1 + ligne["clients"] - ligne["rachats"]
    post[groupe] = stats.beta(a_post, b_post)
    bas, haut = post[groupe].ppf([0.025, 0.975])
    print(f"offre = {groupe} : Beta({a_post:.0f}, {b_post:.0f}) | moyenne {post[groupe].mean():.4f} |
    ↪      f"IC de crédibilité à 95 % : [{bas:.4f} ; {haut:.4f}]"

```

```

offre = 0 : Beta(442, 545) | moyenne 0.4478 | IC de crédibilité à 95 % : [0.4169 ; 0.4789]
offre = 1 : Beta(579, 438) | moyenne 0.5693 | IC de crédibilité à 95 % : [0.5388 ; 0.5996]

```

Étape 3 — la loi de la différence. On tire dans les deux lois et on soustrait.

```

rng = np.random.default_rng(61)
S = 200_000
theta1 = post[1].rvs(S, random_state=rng)
theta0 = post[0].rvs(S, random_state=rng)
delta = theta1 - theta0
print(f"moyenne de la différence      : {delta.mean():.4f}")
print(f"IC de crédibilité à 95 %      : [{np.percentile(delta, 2.5):.4f} ; {np.percentile(delta,
    ↪ 97.5):.4f}]")
print(f"P(l'offre augmente le rachat) : {(delta > 0).mean():.5f}")
print(f"P(l'effet dépasse 5 points)   : {(delta > 0.05).mean():.4f}")
print(f"P(l'effet dépasse 10 points)  : {(delta > 0.10).mean():.4f}")

```

```

moyenne de la différence      : 0.1214
IC de crédibilité à 95 %      : [0.0778 ; 0.1647]
P(l'offre augmente le rachat) : 1.00000
P(l'effet dépasse 5 points)   : 0.9993
P(l'effet dépasse 10 points)  : 0.8346

```

Lecture. Aucun des 200 000 tirages ne donne une différence négative : l'offre augmente presque certainement le rachat, d'environ 12 points (entre 8 et 16 avec 95 % de probabilité). La probabilité que l'effet dépasse 10 points est de 83 %.

Étape 4 — intervalle à queues égales ou HPD ? Pour une loi asymétrique (un nouveau canal testé auprès de 20 clients, un seul rachat), les deux intervalles diffèrent.

```

def hpd(loi, masse=0.95, pas=2000):
    """Plus court intervalle de probabilité `masse` : on fait glisser une fenêtre de quantiles."""
    bas = np.linspace(0, 1 - masse, pas)
    largeurs = loi.ppf(bas + masse) - loi.ppf(bas)
    i = largeurs.argmin()
    return loi.ppf(bas[i]), loi.ppf(bas[i] + masse)

asym = stats.beta(1 + 1, 1 + 19)
print("Beta(2, 20)  moyenne :", round(asym.mean(), 4))
print("à queues égales : [{:.4f} ; {:.4f}]  largeur {:.4f}" % (asym.ppf([0.025, 0.975]),
    ↪ np.diff(asym.ppf([0.025, 0.975]))[0]))
h = hpd(asym)
print("HPD          : [{:.4f} ; {:.4f}]  largeur {:.4f}" % (*h, h[1] - h[0]))

```

```

Beta(2, 20)  moyenne : 0.0909
à queues égales : [0.0117 ; 0.2382]  largeur 0.2264
HPD          : [0.0026 ; 0.2080]  largeur 0.2054

```

Étape 5 — sensibilité à l'a priori. Refaites le calcul pour quatre a priori et trois tailles d'échantillon.

```

avec_offre = clients.loc[clients["offre_bienvenue"] == 1, "rachat_12m"].to_numpy()
priors = {"Uniforme Beta(1,1)": (1, 1), "Jeffreys Beta(.5,.5)": (0.5, 0.5),
         "Informatif Beta(20,20)": (20, 20), "Informatif faux Beta(2,18)": (2, 18)}
lignes = []
for n in (10, 100, len(avec_offre)):
    y = avec_offre[:n].sum()
    for nom, (a, b) in priors.items():
        loi = stats.beta(a + y, b + n - y)
        bas, haut = loi.ppf([0.025, 0.975])
        lignes.append({"n": n, "a priori": nom, "moyenne": round(lois.mean(), 3), "IC95": f"{{bas:.3f}}
↵ ; {{haut:.3f}}"])
print(pd.DataFrame(lignes).to_string(index=False))

```

n	a priori	moyenne	IC95
10	Uniforme Beta(1,1)	0.333	[0.109 ; 0.610]
10	Jeffreys Beta(.5,.5)	0.318	[0.093 ; 0.606]
10	Informatif Beta(20,20)	0.460	[0.325 ; 0.598]
10	Informatif faux Beta(2,18)	0.167	[0.058 ; 0.317]
100	Uniforme Beta(1,1)	0.549	[0.452 ; 0.644]
100	Jeffreys Beta(.5,.5)	0.550	[0.452 ; 0.645]
100	Informatif Beta(20,20)	0.536	[0.453 ; 0.617]
100	Informatif faux Beta(2,18)	0.475	[0.387 ; 0.564]
1015	Uniforme Beta(1,1)	0.569	[0.539 ; 0.600]
1015	Jeffreys Beta(.5,.5)	0.569	[0.539 ; 0.600]
1015	Informatif Beta(20,20)	0.567	[0.537 ; 0.597]
1015	Informatif faux Beta(2,18)	0.560	[0.530 ; 0.590]

Lecture. Avec $n = 10$ les quatre a priori donnent des réponses très différentes (l'a priori faux ramène la moyenne à 0,167) ; avec $n = 1\,015$ elles sont toutes comprises entre 0,560 et 0,569.

Étape 6 — la loi prédictive. Sur 100 nouveaux clients avec offre, combien rachèteront ? Comparez le plug-in, la loi bêta-binomiale exacte et la simulation.

```

n_nouv = 100
plug = stats.binom(n_nouv, avec_offre.mean())
pred = stats.betabinom(n_nouv, post[1].args[0], post[1].args[1])
for nom, loi in (("plug-in Bin(100, 0)", plug), ("prédictive bêta-binomiale", pred)):
    print(f"{{nom:28s}} moyenne {{loi.mean():6.2f}} | écart-type {{loi.std():5.2f}} | "
          f"intervalle à 95 % [{{loi.ppf(0.025):.0f}} ; {{loi.ppf(0.975):.0f}}]")
rng = np.random.default_rng(63)
theta_s = post[1].rvs(100_000, random_state=rng)
y_s = rng.binomial(n_nouv, theta_s)
print(f"simulation : moyenne {{y_s.mean():.2f}}, écart-type {{y_s.std():.2f}}, "
      f"intervalle à 95 % [{{np.percentile(y_s, 2.5):.0f}} ; {{np.percentile(y_s, 97.5):.0f}}]")

```

```

plug-in Bin(100, 0)      moyenne 56.95 | écart-type 4.95 | intervalle à 95 % [47 ; 67]
prédictive bêta-binomiale  moyenne 56.93 | écart-type 5.19 | intervalle à 95 % [47 ; 67]
simulation : moyenne 56.94, écart-type 5.20, intervalle à 95 % [47 ; 67]

```

Pour aller plus loin — la couverture. Un intervalle de crédibilité à 95 % couvre-t-il la vraie valeur dans 95 % des échantillons ? Simulez 20 000 échantillons de taille 30 pour quatre vraies valeurs de θ et comparez Wald, Wilson et crédibilité.

```

from statsmodels.stats.proportion import proportion_confint

def couverture(theta_vrai, n=30, reps=20_000, graine=62):
    rng = np.random.default_rng(graine)
    y = rng.binomial(n, theta_vrai, reps)
    res = {}
    for methode in ("normal", "wilson"):
        bas, haut = proportion_confint(y, n, alpha=0.05, method=methode)
        res[methode] = np.mean((bas <= theta_vrai) & (theta_vrai <= haut))
    loi = stats.beta(1 + y, 1 + n - y)

```

```
res["crédibilité"] = np.mean((loi.ppf(0.025) <= theta_vrai) & (theta_vrai <= loi.ppf(0.975)))
return res
```

```
print(pd.DataFrame([{"vrai θ": th, **{k: round(v, 3) for k, v in couverture(th).items()} for th in
↳ (0.05, 0.10, 0.30, 0.50)]).to_string(index=False))
```

vrai θ	normal	wilson	crédibilité
0.05	0.781	0.940	0.940
0.10	0.806	0.974	0.974
0.30	0.954	0.930	0.930
0.50	0.958	0.958	0.958

Application 6.2 — Faut-il envoyer l'offre à tout le monde ? (section 6.2)

Objectif. Propager toute l'incertitude (deux taux de rachat, une marge) jusqu'à une décision, par Monte-Carlo. Hypothèses (inventées, à remplacer par les vraies valeurs) : marge de 30 % du panier, coût de l'offre de 1,50 € par client, un client qui rachète génère un panier tiré parmi les paniers observés.

Étape 1 — les ingrédients.

```
resume = clients.groupby("offre_bienvenue")["rachat_12m"].agg(["size", "sum"])
post0 = stats.beta(1 + resume.loc[0, "sum"], 1 + resume.loc[0, "size"] - resume.loc[0, "sum"])
post1 = stats.beta(1 + resume.loc[1, "sum"], 1 + resume.loc[1, "size"] - resume.loc[1, "sum"])
paniers = clients.loc[clients["panier_moyen"] > 0, "panier_moyen"].to_numpy()
marge, cout = 0.30, 1.50
print(f"panier moyen des acheteurs : {paniers.mean():.2f} € ; marge moyenne par rachat : {marge *
↳ paniers.mean():.2f} €")
```

```
panier moyen des acheteurs : 61.23 € ; marge moyenne par rachat : 18.37 €
```

Étape 2 — des scénarios. Dans chacun des 100 000 scénarios, on tire les deux taux et une marge moyenne (bootstrap), puis on calcule $G = (\theta_1 - \theta_0) m - c$.

```
rng = np.random.default_rng(625)
S = 100_000
theta1 = post1.rvs(S, random_state=rng)
theta0 = post0.rvs(S, random_state=rng)
m_boot = marge * np.array([rng.choice(paniers, len(paniers)).mean() for _ in range(S // 20)])
m = rng.choice(m_boot, S)
G = (theta1 - theta0) * m - cout # gain net par client, S scénarios
print(f"gain net moyen par client : {G.mean():+.3f} €")
print(f"intervalle à 90 % : [{np.percentile(G, 5):+.3f} ; {np.percentile(G, 95):+.3f}] €")
print(f"P(l'offre est rentable) : {(G > 0).mean():.3f}")
print(f"pour 5 000 nouveaux clients : gain attendu {5000 * G.mean():+.0f} € ; "
      f"dans 95 % des scénarios le gain dépasse {5000 * np.percentile(G, 95):+.0f} €")
```

```
gain net moyen par client : +0.731 €
intervalle à 90 % : [+0.062 ; +1.400] €
P(l'offre est rentable) : 0.964
pour 5 000 nouveaux clients : gain attendu +3656 € ; dans 95 % des scénarios le gain dépasse +312 €
```

Lecture. L'offre est très probablement rentable (96 %), mais le gain par client reste modeste (0,73 €) et l'intervalle à 90 % s'approche de zéro. **À faire :** refaites le calcul avec un coût de 2 € et une marge de 25 % ; que devient la probabilité de rentabilité ?

Étape 3 — les méthodes de simulation. Trois variantes d'un même calcul, pour comparer leur précision.

```

rng = np.random.default_rng(628)
n, reps = 10_000, 300
naif = np.array([(rng.standard_normal(n) > 4).mean() for _ in range(reps)])
def preferentiel(n, rng):
    x = rng.normal(4, 1, n) # on tire autour de 4
    return np.mean((x > 4) * stats.norm.pdf(x) / stats.norm.pdf(x, loc=4, scale=1))
pref = np.array([preferentiel(n, rng) for _ in range(reps)])
print(f"exact : {stats.norm.sf(4):.3e} | naïf : écart-type {naif.std():.3e} ({(naif == 0).mean():.0%}
↳ d'estimations nulles) "
    f"| préférentiel : écart-type {pref.std():.3e}")

exact : 3.167e-05 | naïf : écart-type 5.798e-05 (69% d'estimations nulles) | préférentiel : écart-type
↳ 6.752e-07

```

Application 6.3 — La régression logistique bayésienne du rachat (section 6.3)

Objectif. Estimer par Metropolis les cinq coefficients d'une régression logistique du rachat (offre, âge standardisé, canaux Réseaux et Site ; Boutique en référence), a priori $\mathcal{N}(0, 2,5^2)$.

Étape 1 — la matrice des variables.

```

import statsmodels.api as sm
d = clients.copy()
d["age_c"] = (d["age"] - d["age"].mean()) / d["age"].std()
X = pd.get_dummies(d[["offre_bienvenue", "canal_acquisition", "age_c"]],
↳ columns=["canal_acquisition"], drop_first=True, dtype=float)
X = X.rename(columns={"canal_acquisition_Réseaux": "Réseaux", "canal_acquisition_Site": "Site",
↳ "offre_bienvenue": "offre"})
X = sm.add_constant(X)
noms = list(X.columns)
Xm, y = X.to_numpy(), d["rachat_l2m"].to_numpy()
print("colonnes :", noms, "| n =", len(y), "| rachat moyen :", y.mean().round(3))

colonnes : ['const', 'offre', 'age_c', 'Réseaux', 'Site'] | n = 2000 | rachat moyen : 0.509

```

Étape 2 — le log-posterior (toute la modélisation) et le maximum de vraisemblance comme point de repère.

```

def log_post(beta):
    eta = Xm @ beta
    log_vrais = np.sum(y * eta - np.logaddexp(0, eta)) # somme de y*eta - log(1 + exp(eta))
    log_prior = -0.5 * np.sum(beta**2) / 2.5**2 # a priori normal centré, écart-type 2,5
    return log_vrais + log_prior

emv = sm.Logit(y, Xm).fit(disps=0)
print("log-posterior en l'EMV :", round(log_post(emv.params), 2))

log-posterior en l'EMV : -1355.46

```

Étape 3 — quatre chaînes. Proposition gaussienne de covariance $\frac{2,38^2}{d} \hat{\Sigma}$ ($\hat{\Sigma}$: covariance de l'EMV), points de départ dispersés.

```

dim = len(noms)
cov_prop = (2.38**2 / dim) * emv.cov_params()
rng = np.random.default_rng(633)
chaines, taux = [], []
for c in range(4):
    depart = emv.params + rng.normal(0, 1.0, dim)
    ch, tx = metropolis_multi(log_post, depart, n=6000, cov_prop=cov_prop, rng=rng)
    chaines.append(ch); taux.append(tx)
chaines = np.array(chaines) # forme (4 chaînes, 6000 itérations, 5
↳ paramètres)

```

```
print("taux d'acceptation :", np.round(taux, 3), "| forme :", chaines.shape)
```

```
taux d'acceptation : [0.284 0.297 0.293 0.274] | forme : (4, 6000, 5)
```

Étape 4 — résumé a posteriori (chauffe de 1 000 itérations écartée), à côté du maximum de vraisemblance.

```
apres = chaines[:, 1000:, :]
tirages = apres.reshape(-1, dim)
resume = pd.DataFrame({
    "EMV": emv.params, "moyenne post.": tirages.mean(axis=0), "écart-type post.": tirages.std(axis=0),
    "2,5 %": np.percentile(tirages, 2.5, axis=0), "97,5 %": np.percentile(tirages, 97.5, axis=0),
    "ESS": [sum(ess(apres[c, :, j]) for c in range(4)) for j in range(dim)], index=noms})
print(resume.round(3).to_string())
```

	EMV	moyenne post.	écart-type post.	2,5 %	97,5 %	ESS
const	0.034	0.033	0.098	-0.162	0.227	1280.705
offre	0.493	0.493	0.089	0.316	0.663	1276.029
age_c	-0.163	-0.163	0.045	-0.256	-0.076	1114.281
Réseaux	-0.463	-0.459	0.113	-0.681	-0.232	1158.399
Site	-0.168	-0.163	0.118	-0.393	0.073	1205.256

Étape 5 — tout devient une moyenne. Rapport de cotes de l'offre et effet sur la probabilité de rachat d'un client de référence (Boutique, âge moyen).

```
b0, b1 = tirages[:, 0], tirages[:, 1]
rc = np.exp(b1)
print(f"rapport de cotes de l'offre : médiane {np.median(rc):.3f}, IC95 [{np.percentile(rc, 2.5):.3f} ; {np.percentile(rc, 97.5):.3f}]")
print(f"P(rapport de cotes > 1) = {(rc > 1).mean():.4f} | P(rapport de cotes > 1,5) = {(rc > 1.5).mean():.4f}")
sigmoide = lambda u: 1 / (1 + np.exp(-u))
effet = sigmoide(b0 + b1) - sigmoide(b0)
print(f"effet de l'offre sur la probabilité de rachat : {effet.mean():+.3f} IC95 [{np.percentile(effet, 2.5):+.3f} ; {np.percentile(effet, 97.5):+.3f}]")
brut = clients.groupby("offre_bienvenue")["rachat_12m"].mean()
print(f"rapport : différence brute des fréquences = {brut[1] - brut[0]:+.3f}")
```

```
rapport de cotes de l'offre : médiane 1.639, IC95 [1.372 ; 1.940]
P(rapport de cotes > 1) = 1.0000 | P(rapport de cotes > 1,5) = 0.8357
effet de l'offre sur la probabilité de rachat : +0.120 IC95 [+0.078 ; +0.161]
rapport : différence brute des fréquences = +0.122
```

Étape 6 — pourquoi 0,49 et pas 0,55 ? Le générateur de données utilise un effet de l'offre de 0,55 sur le logit, mais aussi deux facteurs latents que notre modèle ne voit pas. Vérifiez sur un très grand échantillon que l'omission de variables *atténue* le coefficient.

```
rng = np.random.default_rng(636)
N = 400_000
F1 = rng.normal(size=N)
F2 = 0.3 * F1 + np.sqrt(1 - 0.3**2) * rng.normal(size=N) # deux facteurs latents corrélés
offre = rng.integers(0, 2, N).astype(float)
eta = -0.35 + 0.45 * F1 + 0.35 * F2 + 0.55 * offre # vrai modèle : l'offre vaut 0,55
yy = rng.binomial(1, 1 / (1 + np.exp(-eta)))
marginal = sm.Logit(yy, sm.add_constant(offre)).fit(dis=0).params[1]
condit = sm.Logit(yy, sm.add_constant(np.c_[offre, F1, F2])).fit(dis=0).params[1]
print(f"coefficient de l'offre sans les facteurs latents : {marginal:.3f}")
print(f"coefficient de l'offre avec les facteurs latents : {condit:.3f} (vrai : 0.55)")
```

```
coefficient de l'offre sans les facteurs latents : 0.499
coefficient de l'offre avec les facteurs latents : 0.548 (vrai : 0.55)
```

Application 6.4 — Vérifier un modèle : Poisson contre binomiale négative (section 6.4)

Objectif. Montrer par une vérification prédictive a posteriori que le modèle de Poisson est faux pour le nombre de commandes annuelles des clients du canal Boutique, le remplacer par une binomiale négative estimée par MCMC, et revérifier. (On réutilise `metropolis_multi`, `ess` et `split_rhat` de la préparation.)

Étape 1 — le modèle de Poisson et ses répliques. A posteriori $\text{Gamma}(2 + \sum y; 0,5 + n)$; trois statistiques-test : variance/moyenne, part de zéros, maximum.

```
b = clients.loc[clients["canal_acquisition"] == "Boutique", "nb_commandes_an"].to_numpy()
n_b = len(b)

def stats_test(yrep):
    """Trois statistiques sur chaque jeu (lignes) : variance/moyenne, part de zéros, maximum."""
    return np.c_[yrep.var(axis=1, ddof=1) / yrep.mean(axis=1), (yrep == 0).mean(axis=1),
                 yrep.max(axis=1)]

T_obs = stats_test(b[None, :])[0]
noms_T = ["variance / moyenne", "part de zéros", "maximum"]
rng = np.random.default_rng(642)
lam = stats.gamma(a=2 + b.sum(), scale=1 / (0.5 + n_b)).rvs(2000, random_state=rng)
yrep_pois = rng.poisson(lam[:, None], size=(2000, n_b))
T_pois = stats_test(yrep_pois)

def tableau_ppc(T_rep):
    return pd.DataFrame({"statistique": noms_T, "observée": T_obs.round(3), "répliques : moyenne":
                        T_rep.mean(axis=0).round(3),
                        "2,5 %": np.percentile(T_rep, 2.5, axis=0).round(3), "97,5 %":
                        np.percentile(T_rep, 97.5, axis=0).round(3),
                        "p bayésien": (T_rep >= T_obs).mean(axis=0).round(3)})
print(tableau_ppc(T_pois).to_string(index=False))
```

	statistique	observée	répliques : moyenne	2,5 %	97,5 %	p bayésien
variance / moyenne		3.324	1.000	0.879	1.127	0.0
part de zéros		0.109	0.016	0.006	0.030	0.0
maximum		21.000	11.541	10.000	14.000	0.0

Lecture. Le modèle de Poisson est rejeté sur les trois statistiques ($p_B = 0$) : les clients sont hétérogènes.

Étape 2 — la binomiale négative par MCMC. On paramètre $\theta = (\log \mu, \log k)$, a priori $\mathcal{N}(1, 2^2)$ et $\mathcal{N}(0, 2^2)$.

```
def log_post_nb(theta):
    mu, k = np.exp(theta[0]), np.exp(theta[1])
    log_vrais = stats.nbinom.logpmf(b, k, k / (k + mu)).sum()
    log_prior = -0.5 * ((theta[0] - 1) / 2) ** 2 - 0.5 * ((theta[1] - 0) / 2) ** 2
    return log_vrais + log_prior

rng = np.random.default_rng(643)
pilote, _ = metropolis_multi(log_post_nb, np.array([np.log(b.mean()), 0.0]), 3000, np.diag([0.003,
                                                0.02]), rng)
cov_nb = np.cov(pilote[500:].T) * (2.38**2 / 2) # covariance estimée sur une course pilote
```

Étape 3 — quatre chaînes et diagnostics.

```
chaines_nb, taux_nb = [], []
for c in range(4):
    depart = np.array([np.log(b.mean()) + rng.normal(0, 0.3), rng.normal(0, 1.0)])
    ch, tx = metropolis_multi(log_post_nb, depart, 5000, cov_nb, rng)
    chaines_nb.append(ch[1000:]); taux_nb.append(tx)
```

```
chaines_nb = np.array(chaines_nb)
print("taux d'acceptation :", np.round(taux_nb, 3))
print("R-chapeau (log mu, log k) :", [round(float(split_rhat(chaines_nb[:, :, i])), 4) for i in
    range(2)])
mu_s, k_s = np.exp(chaines_nb[:, :, 0].ravel()), np.exp(chaines_nb[:, :, 1].ravel())
print(f"mu : moyenne {mu_s.mean():.3f} IC95 [{np.percentile(mu_s, 2.5):.3f} ; {np.percentile(mu_s,
    97.5):.3f}]")
print(f"k : médiane {np.median(k_s):.3f} IC95 [{np.percentile(k_s, 2.5):.3f} ; {np.percentile(k_s,
    97.5):.3f}]")
```

```
taux d'acceptation : [0.365 0.352 0.357 0.354]
R-chapeau (log mu, log k) : [1.0029, 1.0019]
mu : moyenne 4.134 IC95 [3.817 ; 4.457]
k : médiane 1.817 IC95 [1.513 ; 2.199]
```

Étape 4 — vérifier. Le modèle réparé reproduit-il les trois statistiques ?

```
idx = rng.choice(len(mu_s), 2000, replace=False)
p_nb = k_s[idx] / (k_s[idx] + mu_s[idx])
yrep_nb = rng.negative_binomial(k_s[idx][:, None], p_nb[:, None], size=(2000, n_b))
print(tableau_ppc(stats_test(yrep_nb)).to_string(index=False))
```

	statistique	observée	répliques :	moyenne	2,5 %	97,5 %	p bayésien
variance / moyenne		3.324		3.287	2.653	4.049	0.434
part de zéros		0.109		0.117	0.083	0.155	0.667
maximum		21.000		22.964	17.000	32.000	0.714

Pour aller plus loin — un cas où la convergence échoue. Prenez la loi à deux bosses (mélange de $\mathcal{N}(-4, 1)$ et $\mathcal{N}(4, 1)$) et quatre chaînes de pas 0,5 puis 6 ; comparez les moyennes de chaque chaîne et le $\hat{R}$.

```
def log_bimodale(x):
    return np.logaddexp(stats.norm.logpdf(x, -4, 1), stats.norm.logpdf(x, 4, 1))

for pas in (0.5, 6.0):
    rng = np.random.default_rng(641)
    ch = np.array([metropolis_ld(log_bimodale, x0, n=5000, pas=pas, rng=rng)[0] for x0 in (-6, -2, 2,
        6)])
    print(f"pas {pas} : moyennes {np.round(ch[:, 1000:].mean(axis=1), 2)} | R-chapeau
        {split_rhat(ch[:, 1000:]):.3f}")
```

```
pas 0.5 : moyennes [-4.08  3.71  4.06  3.92] | R-chapeau 3.072
pas 6.0 : moyennes [-0.16  0.16 -0.28  0.17] | R-chapeau 1.002
```

Application 6.5 — Niveaux de retour des retards de livraison (section 6.5)

Objectif. Estimer des niveaux de retour par les maxima mensuels (GEV) et par les excès au-dessus d'un seuil (GPD), et comparer à la vérité, qu'on connaît car les données sont simulées (graine 671 ; colis en nombre de Poisson de moyenne 3 par jour sur 3 652 jours ; durée = Pareto généralisée décalée de 1 jour, $\xi = 0,25$, $\sigma = 1,5$).

Étape 1 — générer les colis.

```
rng = np.random.default_rng(671)
XI0, SIGMA0, COLIS_PAR_JOUR, JOURS = 0.25, 1.5, 3.0, 3652
n_j = rng.poisson(COLIS_PAR_JOUR, JOURS)
n_tot = n_j.sum()
u = rng.random(n_tot)
duree = 1 + SIGMA0 / XI0 * (u ** (-XI0) - 1) # inversion de la fonction de survie
date = pd.Timestamp("2016-01-01") + pd.to_timedelta(np.repeat(np.arange(JOURS), n_j), unit="D")
colis = pd.DataFrame({"date": date, "duree": duree})
```

```
print(f"{n_tot} colis ; durée médiane {colis['duree'].median():.2f} j ; maximum {duree.max():.2f} j")
print(f"> 5 jours : {(duree > 5).mean():.3%} | > 10 jours : {(duree > 10).mean():.3%} | > 20 jours :
↳ {(duree > 20).mean():.3%}")
```

```
10981 colis ; durée médiane 2.14 j ; maximum 56.32 j
> 5 jours : 13.114% | > 10 jours : 2.723% | > 20 jours : 0.373%
```

Étape 2 — maxima par blocs : GEV. (scipy paramètre la GEV par $c = -\xi$.)

```
maxima_m = colis.groupby(colis["date"].dt.to_period("M"))["duree"].max().to_numpy()
c_hat, mu_hat, sig_hat = stats.genextreme.fit(maxima_m)
xi_hat = -c_hat
print(f"{len(maxima_m)} maxima mensuels ; GEV : mu = {mu_hat:.3f}, sigma = {sig_hat:.3f}, xi =
↳ {xi_hat:.3f}")
rng = np.random.default_rng(672)
boot = np.array([stats.genextreme.fit(rng.choice(maxima_m, len(maxima_m))) for _ in range(300)])
xi_boot = -boot[:, 0]
print(f"xi : IC95 bootstrap [{np.percentile(xi_boot, 2.5):.3f} ; {np.percentile(xi_boot, 97.5):.3f}]"
```

```
120 maxima mensuels ; GEV : mu = 13.741, sigma = 4.772, xi = 0.324
xi : IC95 bootstrap [0.213 ; 0.459]
```

Étape 3 — niveaux de retour et comparaison à la loi normale et à la vérité.

```
def niveau_gev(T, mu, sigma, xi):
    return mu - sigma / xi * (1 - (-np.log(1 - 1 / T)) ** (-xi))

lam_m = COLIS_PAR_JOUR * 30.4375
sig_vrai = SIGMA0 * lam_m ** XI0
mu_vrai = 1 + SIGMA0 * (lam_m ** XI0 - 1) / XI0
m_norm, s_norm = maxima_m.mean(), maxima_m.std(ddof=1)
lignes = []
for T, nom in ((12, "1 an"), (120, "10 ans"), (1200, "100 ans")):
    ni_boot = np.array([niveau_gev(T, bb[1], bb[2], -bb[0]) for bb in boot])
    lignes.append({"retour": nom, "vérité": round(niveau_gev(T, mu_vrai, sig_vrai, XI0), 1),
                  "GEV": round(niveau_gev(T, mu_hat, sig_hat, xi_hat), 1),
                  "IC95 GEV": f"[{np.percentile(ni_boot, 2.5):.0f} ; {np.percentile(ni_boot,
↳ 97.5):.0f}]",
                  "normale": round(stats.norm.ppf(1 - 1 / T, m_norm, s_norm), 1)})
print(pd.DataFrame(lignes).to_string(index=False))
```

retour	vérité	GEV	IC95 GEV	normale
1 an	29.1	31.5	[27 ; 37]	31.5
10 ans	56.3	68.5	[51 ; 101]	41.1
100 ans	104.2	145.8	[89 ; 289]	48.2

Étape 4 — excès au-dessus d'un seuil : GPD. Variation du seuil, puis niveaux de retour.

```
x_all = colis["duree"].to_numpy()

def ajuste_gpd(x, seuil):
    exces = x[x > seuil] - seuil
    xi_, loc_, sc_ = stats.genpareto.fit(exces, floc=0) # loc = 0 : excès comptés depuis
    ↳ le seuil
    return xi_, sc_, len(exces)

for q in (0.80, 0.90, 0.95, 0.98):
    u_ = np.quantile(x_all, q)
    xi_, sc_, k_ = ajuste_gpd(x_all, u_)
    print(f"seuil au centile {q:.0%} : u = {u_:5.2f} j, {k_:4d} excès, xi = {xi_:3f}, sigma_u =
↳ {sc_:3f}")
```

```

seuil au centile 80% : u = 4.02 j, 2196 excès, xi = 0.273, sigma_u = 2.237
seuil au centile 90% : u = 5.69 j, 1098 excès, xi = 0.252, sigma_u = 2.795
seuil au centile 95% : u = 7.73 j, 549 excès, xi = 0.239, sigma_u = 3.440
seuil au centile 98% : u = 11.16 j, 220 excès, xi = 0.289, sigma_u = 4.073

```

```

u_ = np.quantile(x_all, 0.95)
xi_p, sc_p, k_p = ajuste_gpd(x_all, u_)
zeta = (x_all > u_).mean()
niveau_pot = lambda m, xi_, sc_, u_, zeta_: u_ + sc_ / xi_ * ((m * zeta_) ** xi_ - 1)
niveau_vrai = lambda m: 1 + SIGMA0 / XI0 * (m ** XI0 - 1)
for ans, nom in ((1, "1 an"), (10, "10 ans"), (100, "100 ans")):
    m = n_tot * ans / 10.0 # nombre de colis dans la période de
    ↪ retour
    print(f"{nom:8s} vérité {niveau_vrai(m):6.1f} j | POT {niveau_pot(m, xi_p, sc_p, u_, zeta):6.1f}
    ↪ j")

```

```

1 an    vérité  29.5 j | POT  30.8 j
10 ans  vérité  56.4 j | POT  58.3 j
100 ans vérité 104.2 j | POT 105.9 j

```

Lecture. À 100 ans, la loi normale annonce moins de 50 jours quand la vérité est de 104 ; la GEV et surtout le POT s'en approchent, avec des intervalles larges. **À faire :** ajoutez un bootstrap au POT pour obtenir ses intervalles (300 rééchantillonnages des colis) et comparez-les à ceux de la GEV.

Application 6.6 — Deux transporteurs en même temps (section 6.6)

Objectif. Montrer qu'à même tau de Kendall, la copule choisie change d'un facteur 3 la probabilité de deux retards extrêmes le même jour. Les retards de deux transporteurs sont simulés (graine 683) avec des marginales Pareto généralisées et une vraie dépendance de Clayton *retournée* ($\theta = 1,5$).

Étape 1 — les simulateurs de copules.

```

from scipy.stats import norm, kendalltau, multivariate_normal

def sim_gauss(rho, n, rng):
    z = rng.multivariate_normal([0, 0], [[1, rho], [rho, 1]], n)
    return norm.cdf(z)

def sim_clayton(theta, n, rng):
    v = rng.gamma(1 / theta, 1.0, n) # facteur commun
    e = rng.exponential(size=(n, 2))
    return (1 + e / v[:, None]) ** (-1 / theta)

tau = 0.5
rho, theta = np.sin(np.pi * tau / 2), 2 * tau / (1 - tau)
rng = np.random.default_rng(680)
ug, uc = sim_gauss(rho, 200_000, rng), sim_clayton(theta, 200_000, rng)
for q in (0.10, 0.05, 0.01):
    print(f"q = {q:.2f} : P(les deux sous q) gaussienne {np.mean((ug < q).all(axis=1)):.5f} | Clayton
    ↪ {np.mean((uc < q).all(axis=1)):.5f}")

```

```

q = 0.10 : P(les deux sous q) gaussienne 0.04682 | Clayton 0.07184
q = 0.05 : P(les deux sous q) gaussienne 0.01959 | Clayton 0.03565
q = 0.01 : P(les deux sous q) gaussienne 0.00281 | Clayton 0.00681

```

Étape 2 — les données des transporteurs.

```

def gpd_inv(u_, xi_, sg): # fonction de répartition inverse de la GPD
    ↪ décalée de 1

```

```

    return 1 + sg / xi_ * ((1 - u_) ** (-xi_) - 1)

theta_v = 1.5
rng = np.random.default_rng(683)
n_jours = 1500
u_vrai = 1 - sim_clayton(theta_v, n_jours, rng)          # Clayton retournée : dépendance dans la
↳ queue supérieure
A = gpd_inv(u_vrai[:, 0], 0.25, 1.5)
B = gpd_inv(u_vrai[:, 1], 0.20, 2.0)
print(f"retard moyen A : {A.mean():.2f} j, B : {B.mean():.2f} j ; retard maximal A : {A.max():.1f} j,
↳ B : {B.max():.1f} j")

retard moyen A : 2.94 j, B : 3.39 j ; retard maximal A : 23.1 j, B : 31.4 j

```

Étape 3 — ajuster les copules sur les pseudo-observations (rangs divisés par $n + 1$).

```

def pseudo_obs(x, y):
    n_ = len(x)
    return stats.rankdata(x) / (n_ + 1), stats.rankdata(y) / (n_ + 1)

def ll_gauss(u, v, rho_):
    x, y = norm.ppf(u), norm.ppf(v)
    return np.sum(-0.5 * np.log(1 - rho_**2) - (rho_**2 * (x**2 + y**2) - 2 * rho_ * x * y) / (2 * (1
↳ - rho_**2)))

def ll_clayton(u, v, th):
    return np.sum(np.loglp(th) - (th + 1) * (np.log(u) + np.log(v)) - (2 + 1 / th) * np.log(u**th +
↳ v**th - 1))

u, v = pseudo_obs(A, B)
tau_h = kendalltau(A, B).statistic
rho_h, th_h = np.sin(np.pi * tau_h / 2), 2 * tau_h / (1 - tau_h)
print(f"tau estimé : {tau_h:.3f} -> rho = {rho_h:.3f} (gaussienne) ; theta = {th_h:.2f} (Clayton)")
print(f"log-vraisemblance : gaussienne {ll_gauss(u, v, rho_h):.1f} | Clayton {ll_clayton(u, v,
↳ th_h):.1f} | "
      f"Clayton retournée {ll_clayton(1 - u, 1 - v, th_h):.1f}")

tau estimé : 0.416 -> rho = 0.608 (gaussienne) ; theta = 1.42 (Clayton)
log-vraisemblance : gaussienne 332.6 | Clayton 11.0 | Clayton retournée 431.7

```

Étape 4 — la probabilité qui compte. Probabilité que les deux retards dépassent leur 99^e centile le même jour.

```

qq = 0.99
p_indep = (1 - qq) ** 2
p_gauss = multivariate_normal([0, 0], [[1, rho_h], [rho_h, 1]]).cdf([-norm.ppf(qq), -norm.ppf(qq)])
p_clay_ret = (2 * (1 - qq) ** (-th_h) - 1) ** (-1 / th_h)
p_vrai = (2 * (1 - qq) ** (-theta_v) - 1) ** (-1 / theta_v)
p_emp = np.mean((u > qq) & (v > qq))
for nom, p in (("indépendance", p_indep), ("copule gaussienne", p_gauss), ("Clayton retournée",
↳ p_clay_ret), ("vérité", p_vrai), ("observé", p_emp)):
    print(f"{nom:18s} {p:.5f} ({p / p_indep:5.1f} fois l'indépendance ; une fois tous les {round(1 /
↳ p):6d} jours)")

indépendance      0.00010 ( 1.0 fois l'indépendance ; une fois tous les 10000 jours)
copule gaussienne 0.00193 ( 19.3 fois l'indépendance ; une fois tous les 518 jours)
Clayton retournée 0.00615 ( 61.5 fois l'indépendance ; une fois tous les 163 jours)
vérité            0.00630 ( 63.0 fois l'indépendance ; une fois tous les 159 jours)
observé           0.00533 ( 53.3 fois l'indépendance ; une fois tous les 188 jours)

```

Lecture. La copule gaussienne prévoit des doubles retards extrêmes 3,3 fois moins fréquents que la vérité. **À faire :** reprenez l'étape 3 avec la dépendance faible de clients.csv (panier moyen

et nombre de commandes des acheteurs, en départageant les égalités par un bruit uniforme sur $[-0,5; 0,5]$) : la dépendance de queue est-elle un enjeu ici ?

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 6.8 à 6.10 réutilisent les fonctions de la préparation (`metropolis_1d`, `ess`, `split_rhat`).

Exercice 6.1 ★ — Bêta-binomial (section 6.1)

La gérante teste un nouvel emballage : 3 clients sur 8 le jugent « excellent ». Avec l'a priori $\text{Beta}(2,2)$ (« je pense plutôt autour de 50 % »), donnez (a) la loi a posteriori, (b) sa moyenne, (c) le poids de l'a priori dans cette moyenne, (d) la comparaison avec l'estimation du maximum de vraisemblance.

Exercice 6.2 ★ — Gamma-Poisson (section 6.1)

Le site reçoit 2, 4 et 1 commandes lors de trois soirées. A priori $\lambda \sim \text{Gamma}(3; \text{taux } 1)$ (moyenne 3 commandes par soirée). Donnez la loi a posteriori de λ , sa moyenne, et un intervalle de crédibilité à 95 %.

Exercice 6.3 ★★ — Normal-normal (section 6.1)

On modélise le log du panier avec $\sigma = 0,35$ connu et l'a priori $\mathcal{N}(4,0; 0,5^2)$. À partir de combien d'observations le poids des données dans la moyenne a posteriori dépasse-t-il 90 % ? Calculez-le à la main, puis vérifiez avec le code.

Exercice 6.4 ★★ — Test A/B bayésien (section 6.1)

La version A d'une page convertit 30 visiteurs sur 100, la version B 42 sur 110. Avec des a priori $\text{Beta}(1,1)$, calculez par simulation la probabilité que B soit meilleure que A, le gain attendu en points de pourcentage, et la probabilité que le gain dépasse 5 points. Comparez avec le test de proportions fréquentiste.

Exercice 6.5 ★ — Monte-Carlo (section 6.2)

Soient U_1, U_2 uniformes indépendantes sur $[0,1]$. (a) Calculez à la main $\mathbb{E}[\max(U_1, U_2)]$. (b) Estimez-la par Monte-Carlo avec 100 000 tirages, avec son erreur type et son intervalle de confiance. (c) Estimez $P(U_1 + U_2 > 1,5)$ et comparez avec la valeur exacte $1/8$.

Exercice 6.6 ★★ — Échantillonnage préférentiel (section 6.2)

Soit $X \sim \text{Exp}(1)$. On veut $P(X > 5) = e^{-5}$. (a) Estimez-la naïvement avec $n = 10\,000$ tirages. (b) Utilisez la proposition « $5 + \text{Exp}(1)$ ». Que valent les poids ? Que remarquez-vous sur la variance de l'estimateur ?

Exercice 6.7 ★★ — Inversion (section 6.2)

La durée de vie T (en mois) d'un abonnement suit une loi de Weibull de fonction de répartition $F(t) = 1 - \exp(-(t/\lambda)^k)$, avec $\lambda = 24$ et $k = 1,5$. (a) Déterminez F^{-1} . (b) Simulez 100 000 durées. (c) Vérifiez la médiane théorique et la moyenne théorique $\lambda \Gamma(1 + 1/k)$.

Exercice 6.8 ★★ — Metropolis à la main (section 6.3)

On veut une chaîne sur trois états $\{1, 2, 3\}$ de loi stationnaire proportionnelle à $(1, 2, 1)$. La proposition choisit l'un des deux autres états avec probabilité $\frac{1}{2}$. (a) Écrivez la matrice de transition de Metropolis. (b) Vérifiez le bilan détaillé. (c) Vérifiez par le code, et par simulation.

Exercice 6.9 ★★ — Metropolis sur une échelle logarithmique (section 6.3)

Six semaines de commandes : 3, 5, 4, 6, 2, 5. Modèle : $\text{Poisson}(\lambda)$, a priori $\text{Gamma}(2; \text{taux } 0,5)$. (a) Donnez la loi a posteriori exacte. (b) Écrivez un Metropolis sur $\theta = \log \lambda$ (attention à la transformation de la densité) et comparez avec la loi exacte.

Exercice 6.10 ★★ — Diagnostics (section 6.4)

Une chaîne autorégressive $x_t = \varphi x_{t-1} + \sqrt{1 - \varphi^2} \varepsilon_t$ ($\varepsilon_t \sim \mathcal{N}(0, 1)$) a pour loi stationnaire $\mathcal{N}(0, 1)$ et pour autocorrélation $\rho_k = \varphi^k$. (a) Montrez que son ESS théorique vaut environ $n \frac{1 - \varphi}{1 + \varphi}$. (b) Pour $\varphi = 0,9$ et $n = 20\,000$, comparez avec l'ESS calculée. (c) Quatre chaînes de $\varphi = 0,99$ lancées de $-10, -3, 3, 10$: calculez le $\widehat{R}$ avec et sans élimination des 300 premiers points.

Exercice 6.11 ★★ — Vérification prédictive (section 6.4)

Modélisez les paniers des acheteurs du canal Boutique (a) par une loi normale sur le panier en €, (b) par une loi normale sur le **logarithme** du panier. Avec la statistique-test « asymétrie » (skewness) et le plus petit panier, quel modèle passe la vérification prédictive a posteriori ?

Exercice 6.12 ★★ — Facteur de Bayes (section 6.4)

Neuf clients sur dix préfèrent le nouvel emballage. $H_0 : \theta = 0,5$; $H_1 : \theta \sim \mathcal{U}(0, 1)$. Calculez à la main BF_{10} , comparez à la p-valeur exacte bilatérale, puis calculez la probabilité a posteriori de H_1 si l'on pense au départ qu'il y a 1 chance sur 4 que l'emballage ait un effet.

Exercice 6.13 ★★ — Valeurs extrêmes (section 6.5)

Pour des retards de colis, on a choisi le seuil $u = 5$ jours ; 5 % des colis le dépassent ($\zeta_u = 0,05$) et la loi des excès est GPD de $\xi = 0,2$ et $\sigma_u = 2$ jours. (a) Calculez à la main le retard dépassé en moyenne une fois tous les 1 000 colis. (b) Et tous les 10 000 colis ? (c) Que devient (b) si $\xi = 0,4$ au lieu de 0,2 ? Qu'en concluez-vous ?

Exercice 6.14 ★★★ — Dépendance de queue de Clayton (section 6.6)

- (a) Démontrez que, pour la copule de Clayton, $P(V \leq q \mid U \leq q) = (2 - q^\theta)^{-1/\theta}$, puis que cette quantité tend vers $2^{-1/\theta}$ quand $q \rightarrow 0$. (b) Vérifiez numériquement avec $\theta = 2$ pour $q = 0,1; 0,01; 0,001$. (c) Par la formule de survie, que vaut $P(U > 1 - q, V > 1 - q)$ pour la copule de Clayton *retournée* ? Comparez avec une copule gaussienne de même tau.

Corrigés

Corrigé 6.1

- (a) Succès $y = 3$, échecs $n - y = 5$: $\text{Beta}(2 + 3, 2 + 5) = \text{Beta}(5, 7)$. (b) Moyenne $5/12 = 0,4167$.
 (c) Le poids de l'a priori est $\frac{a+b}{a+b+n} = \frac{4}{4+8} = \frac{1}{3}$: un tiers de l'a priori (moyenne 0,5), deux tiers des données (fréquence 0,375) : $\frac{1}{3} \times 0,5 + \frac{2}{3} \times 0,375 = 0,4167$ ✓. (d) Le maximum de vraisemblance est $3/8 = 0,375$; l'a posteriori est **tiré vers 0,5**. Avec 8 observations seulement, l'a priori compte beaucoup (voir 6.1.6).

```
post = stats.beta(2 + 3, 2 + 5)
print(f"a posteriori Beta(5, 7) : moyenne {post.mean():.4f} | IC95 [{post.ppf(0.025):.3f} ;
↳ {post.ppf(0.975):.3f}]")
print(f"poids de l'a priori : {4 / 12:.3f} | EMV : {3 / 8:.3f} | moyenne pondérée : {4 / 12 * 0.5 + 8
↳ / 12 * 3 / 8:.4f}")
```

```
a posteriori Beta(5, 7) : moyenne 0.4167 | IC95 [0.167 ; 0.692]
poids de l'a priori : 0.333 | EMV : 0.375 | moyenne pondérée : 0.4167
```

Corrigé 6.2

$\sum y_i = 7$, $n = 3$: $\lambda \mid y \sim \text{Gamma}(3 + 7; 1 + 3) = \text{Gamma}(10; \text{taux } 4)$, de moyenne $10/4 = 2,5$ commandes par soirée. Elle est **comprise entre la moyenne a priori (3) et celle des données** ($7/3 = 2,33$), comme il se doit pour un compromis ; elle est plus proche des données car celles-ci pèsent $3/4$ (3 observations contre un a priori valant « 1 observation » : le taux de l'a priori est 1).

```
post = stats.gamma(a=10, scale=1 / 4)
print(f"Gamma(10, taux 4) : moyenne {post.mean():.3f} | IC95 [{post.ppf(0.025):.3f} ;
↳ {post.ppf(0.975):.3f}]")
```

```
Gamma(10, taux 4) : moyenne 2.500 | IC95 [1.199 ; 4.271]
```

Corrigé 6.3

Le poids des données est $w = \frac{n/\sigma^2}{1/\tau_0^2 + n/\sigma^2}$. On veut $w > 0,9 \iff n/\sigma^2 > 9/\tau_0^2 \iff n > 9\sigma^2/\tau_0^2 = 9 \times 0,1225/0,25 = 4,41$. Il faut donc $n \geq 5$ observations (avec $n = 5$: $w = 0,911$). L'a priori est donc déjà « oublié » à 90 % avec cinq clients seulement : il est assez large par rapport à σ .

```
sigma, tau0 = 0.35, 0.5
for n in range(1, 8):
    w = (n / sigma**2) / (1 / tau0**2 + n / sigma**2)
    print(f"n = {n} : poids des données = {w:.3f}" + (" <- premier n au-dessus de 90 %" if w > 0.9
↳ and (n == 1 or ((n - 1) / sigma**2) / (1 / tau0**2 + (n - 1) / sigma**2)) <= 0.9) else "")
```

```
n = 1 : poids des données = 0.671
n = 2 : poids des données = 0.803
n = 3 : poids des données = 0.860
n = 4 : poids des données = 0.891
n = 5 : poids des données = 0.911 <- premier n au-dessus de 90 %
n = 6 : poids des données = 0.924
n = 7 : poids des données = 0.935
```

Corrigé 6.4

A : Beta(31,71), B : Beta(43,69). On tire dans chaque loi et on compare.

```
rng = np.random.default_rng(690)
S = 400_000
pA = stats.beta(1 + 30, 1 + 70).rvs(S, random_state=rng)
pB = stats.beta(1 + 42, 1 + 68).rvs(S, random_state=rng)
gain = pB - pA
print(f"P(B > A) = {(gain > 0).mean():.4f} | gain moyen = {100 * gain.mean():.1f} points | P(gain > 5
↪ points) = {(gain > 0.05).mean():.4f}")
print(f"IC de crédibilité à 95 % du gain : [{100 * np.percentile(gain, 2.5):.1f} ; {100 *
↪ np.percentile(gain, 97.5):.1f}] points")

from statsmodels.stats.proportion import proportions_ztest
z, p = proportions_ztest([42, 30], [110, 100])
print(f"test fréquentiste : z = {z:.3f}, p-valeur bilatérale = {p:.4f} (unilatérale : {p / 2:.4f})")
```

```
P(B > A) = 0.8925 | gain moyen = 8.0 points | P(gain > 5 points) = 0.6800
IC de crédibilité à 95 % du gain : [-4.7 ; 20.6] points
test fréquentiste : z = 1.248, p-valeur bilatérale = 0.2122 (unilatérale : 0.1061)
```

La probabilité bayésienne que B soit meilleure est de 89 % ; le test fréquentiste donne une p-valeur bilatérale de 0,21 (unilatérale 0,106), **non significative** à 5 %. Remarquez que $1 - 0,106 = 0,894$ est presque égal à la probabilité bayésienne de 0,8925 : avec un a priori plat et des données assez abondantes, la p-valeur unilatérale et la probabilité a posteriori de l'hypothèse « opposée » **coïncident presque** (c'est un résultat classique pour les proportions). Mais **la p-valeur ne dit pas « la probabilité que B soit meilleure »** (volume I, section 3.5.2), alors que la probabilité bayésienne le dit : « 89 % de chances que B soit meilleure, gain probable de 8 points, avec une probabilité de 68 % que le gain dépasse 5 points ». L'intervalle de crédibilité à 95 % du gain, $[-4,7 ; +20,6]$ points, est très large : avec une centaine de visiteurs par version, on ne sait pas grand-chose, et c'est exactement ce que dit le test non significatif.

Corrigé 6.5

(a) $P(\max \leq x) = x^2$, de densité $2x$: $\mathbb{E}[\max] = \int_0^1 x \cdot 2x \, dx = 2/3$. (b), (c) :

```
rng = np.random.default_rng(691)
n = 100_000
u = rng.random((n, 2))
g = u.max(axis=1)
est, se = g.mean(), g.std(ddof=1) / np.sqrt(n)
print(f"E[max] estimée : {est:.4f} ± {1.96 * se:.4f} (IC95) | exacte : {2 / 3:.4f}")
h = (u.sum(axis=1) > 1.5)
est, se = h.mean(), h.std(ddof=1) / np.sqrt(n)
print(f"P(U1 + U2 > 1,5) estimée : {est:.4f} ± {1.96 * se:.4f} | exacte : {1 / 8:.4f}")
```

```
E[max] estimée : 0.6682 ± 0.0015 (IC95) | exacte : 0.6667
P(U1 + U2 > 1,5) estimée : 0.1265 ± 0.0021 | exacte : 0.1250
```

L'intervalle de confiance de Monte-Carlo contient la valeur exacte dans chaque cas (dans 95 % des répétitions de l'expérience, en moyenne).

Corrigé 6.6

- (a) L'estimateur naïf a une variance $p(1-p)/n \approx 6,7 \times 10^{-7}$, soit un écart-type de $8,2 \times 10^{-4}$ pour une valeur de $6,7 \times 10^{-3}$: environ 12 % d'erreur relative. (b) Tirons $Y = 5 + E$, $E \sim \text{Exp}(1)$ (densité $h(y) = e^{-(y-5)}$ pour $y > 5$). Alors $f(y)/h(y) = e^{-y}/e^{-(y-5)} = e^{-5}$ **pour tout** $y > 5$: tous les tirages sont dans la zone d'intérêt (indicatrice égale à 1) et **tous les poids valent** e^{-5} . L'estimateur vaut donc e^{-5} **exactement**, avec une **variance nulle** : c'est la proposition idéale, qui est *proportionnelle* à $g \times f$. En pratique, on ne la connaît pas (sinon on ne simulerait pas), mais on s'en approche.

```
rng = np.random.default_rng(692)
n = 10_000
naif = (rng.exponential(1, n) > 5).mean()
y = 5 + rng.exponential(1, n)
poids = np.exp(-y) / np.exp(-(y - 5))          # f / h
pref = np.mean((y > 5) * poids)
print(f"exact : {np.exp(-5):.6f} | naïf : {naif:.6f} | préférentiel : {pref:.6f} | poids min / max :
↪ {poids.min():.6f} / {poids.max():.6f}")
```

exact : 0.006738 | naïf : 0.006700 | préférentiel : 0.006738 | poids min / max : 0.006738 / 0.006738

Corrigé 6.7

- (a) On résout $u = 1 - \exp(-(t/\lambda)^k) : t = \lambda[-\ln(1 - u)]^{1/k}$. (b) et (c) : la médiane théorique est $\lambda(\ln 2)^{1/k}$.

```
from math import gamma, log
lam, k = 24, 1.5
rng = np.random.default_rng(693)
u = rng.random(100_000)
t = lam * (-np.log(1 - u)) ** (1 / k)
print(f"médiane simulée {np.median(t):.3f} | théorique {lam * log(2) ** (1 / k):.3f}")
print(f"moyenne simulée {t.mean():.3f} | théorique {lam * gamma(1 + 1 / k):.3f}")
print(f"P(T > 36) simulée {np.mean(t > 36):.4f} | théorique {np.exp(-(36 / lam) ** k):.4f}")
```

médiane simulée 18.892 | théorique 18.797
moyenne simulée 21.713 | théorique 21.666
P(T > 36) simulée 0.1587 | théorique 0.1593

Corrigé 6.8

Cible $\pi = (0,25; 0,5; 0,25)$. (a) $P_{12} = \frac{1}{2} \min(1, 2) = \frac{1}{2}$; $P_{13} = \frac{1}{2} \min(1, 1) = \frac{1}{2}$; $P_{11} = 0$. Depuis 2 : $P_{21} = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$, $P_{23} = \frac{1}{4}$, $P_{22} = \frac{1}{2}$. Depuis 3 : $P_{31} = \frac{1}{2}$, $P_{32} = \frac{1}{2} \min(1, 2) = \frac{1}{2}$, $P_{33} = 0$. (b) $\pi_1 P_{12} = 0,25 \times \frac{1}{2} = 0,125 = \pi_2 P_{21} = 0,5 \times \frac{1}{4} \checkmark$; $\pi_1 P_{13} = 0,125 = \pi_3 P_{31} \checkmark$; $\pi_2 P_{23} = 0,5 \times \frac{1}{4} = 0,125 = \pi_3 P_{32} = 0,25 \times \frac{1}{2} \checkmark$.

```
poids = np.array([1.0, 2.0, 1.0]); pi = poids / poids.sum(); k3 = 3
P = np.zeros((k3, k3))
for i in range(k3):
    for j in range(k3):
        if i != j:
            P[i, j] = 0.5 * min(1, poids[j] / poids[i])
    P[i, i] = 1 - P[i].sum()
print(P)
print("bilan détaillé (écart max) :", np.abs(pi[:, None] * P - (pi[:, None] * P).T).max())
rng = np.random.default_rng(694)
etat, compte = 0, np.zeros(3)
for _ in range(60_000):
```

```
etat = rng.choice(3, p=P[etat]); compte[etat] += 1
print("fréquences simulées :", (compte / 60_000).round(3), "| cible :", pi)
```

```
[[0.  0.5  0.5 ]
 [0.25 0.5  0.25]
 [0.5  0.5  0.  ]]
bilan détaillé (écart max) : 0.0
fréquences simulées : [0.251 0.5  0.249] | cible : [0.25 0.5  0.25]
```

Corrigé 6.9

- (a) $\sum y = 25, n = 6 : \lambda \mid y \sim \text{Gamma}(2 + 25; 0,5 + 6) = \text{Gamma}(27; \text{taux } 6,5)$, de moyenne $27/6,5 = 4,154$. (b) Si $\theta = \log \lambda$, la densité de θ est celle de λ multipliée par le jacobien $|d\lambda/d\theta| = \lambda = e^\theta$. L'a posteriori **non normalisé** de θ est donc $\propto e^{(27-1)\theta} e^{-6,5e^\theta} \cdot e^\theta = e^{27\theta - 6,5e^\theta}$: $\log \pi(\theta) = 27\theta - 6,5e^\theta$. **Oublier le jacobien est l'erreur classique** : on échantillonnerait alors une autre loi.

```
def log_cible(theta):
    return 27 * theta - 6.5 * np.exp(theta)

rng = np.random.default_rng(695)
chaîne, taux = metropolis_ld(log_cible, x0=np.log(4.0), n=30_000, pas=0.5, rng=rng)
lam_s = np.exp(chaîne[2000:])
exacte = stats.gamma(a=27, scale=1 / 6.5)
print(f"taux d'acceptation {taux:.3f}")
print(f"MH : moyenne {lam_s.mean():.4f}, écart-type {lam_s.std():.4f}, quantiles 2,5 / 50 / 97,5 %
↪ {np.percentile(lam_s, [2.5, 50, 97.5]).round(3)}")
print(f"exact : moyenne {exacte.mean():.4f}, écart-type {exacte.std():.4f}, quantiles
↪ {exacte.ppf([0.025, 0.5, 0.975]).round(3)}")

taux d'acceptation 0.420
MH : moyenne 4.1457, écart-type 0.8114, quantiles 2,5 / 50 / 97,5 % [2.712 4.076 5.874]
exact : moyenne 4.1538, écart-type 0.7994, quantiles [2.737 4.103 5.861]
```

Corrigé 6.10

- (a) $\text{ESS} = n / (1 + 2 \sum_{k \geq 1} \rho_k)$ et $\sum_{k \geq 1} \varphi^k = \frac{\varphi}{1 - \varphi}$, donc $1 + 2 \frac{\varphi}{1 - \varphi} = \frac{1 + \varphi}{1 - \varphi}$ et $\text{ESS} = n \frac{1 - \varphi}{1 + \varphi}$.
Pour $\varphi = 0,9$: $n/19$, soit environ 1 053 sur 20 000. (b), (c) :

```
def arl(phi, n, x0, rng):
    x = np.empty(n); x[0] = x0
    for t in range(1, n):
        x[t] = phi * x[t - 1] + np.sqrt(1 - phi**2) * rng.standard_normal()
    return x

rng = np.random.default_rng(696)
x = arl(0.9, 20_000, 0.0, rng)
print(f"ESS théorique {20_000 * (1 - 0.9) / (1 + 0.9):.0f} | ESS calculée {ess(x):.0f}")
chaines4 = np.array([arl(0.99, 1500, x0, rng) for x0 in (-10, -3, 3, 10)])
print(f"R-chapeau avec les 1 500 points : {split_rhat(chaines4):.3f} | sans les 300 premiers :
↪ {split_rhat(chaines4[:, 300:]):.3f}")
print(f"ESS théorique par chaîne (phi = 0,99) : {1200 * (1 - 0.99) / (1 + 0.99):.0f} sur 1 200 points")

ESS théorique 1053 | ESS calculée 1111
R-chapeau avec les 1 500 points : 1.142 | sans les 300 premiers : 1.063
ESS théorique par chaîne (phi = 0,99) : 6 sur 1 200 points
```

L'ESS calculée est proche de la théorie. Pour $\varphi = 0,99$, la chaîne est si lente que l'ESS d'une

chaîne de 1 200 points est de l'ordre de 6 seulement : le $\widehat{R}$ **détecte le problème** (valeur nettement supérieure à 1,01, quoi qu'on fasse), parce que chaque chaîne n'a pas eu le temps de quitter son voisinage de départ. C'est exactement la situation où il faut **allonger** ou **reparamétriser**.

Corrigé 6.11

Données : les paniers des 449 acheteurs du canal Boutique. Sous le modèle normal avec a priori $p(\mu, \sigma^2) \propto 1/\sigma^2$, la loi a posteriori est : $\sigma^2 | y \sim (n-1)s^2/\chi_{n-1}^2$, puis $\mu | \sigma^2, y \sim \mathcal{N}(\bar{y}, \sigma^2/n)$, ce qui donne directement des tirages (sans MCMC).

```
pan = clients[(clients["canal_acquisition"] == "Boutique") & (clients["panier_moyen"] >
↳ 0)][["panier_moyen"]].to_numpy()

def ppc_normal(y_, graine, S=2000):
    rng = np.random.default_rng(graine)
    n_, ybar, s2 = len(y_), y_.mean(), y_.var(ddof=1)
    sig2 = (n_ - 1) * s2 / rng.chisquare(n_ - 1, S)
    mu = rng.normal(ybar, np.sqrt(sig2 / n_))
    yrep = rng.normal(mu[:, None], np.sqrt(sig2)[0, None], size=(S, n_))
    T = lambda a: np.c_[stats.skew(a, axis=-1), a.min(axis=-1)]
    To, Tr = T(y_[None, :])[0], T(yrep)
    return To, Tr, (Tr >= To).mean(axis=0)

for nom, donnees in (("normale sur le panier (€)", pan), ("normale sur le log du panier",
↳ np.log(pan))):
    To, Tr, pb = ppc_normal(donnees, 697)
    print(f"{nom:30s} asymétrie : obs. {To[0]:6.3f}, rép. {Tr[:, 0].mean():6.3f} (p {pb[0]:.3f}) | "
        f"minimum : obs. {To[1]:7.3f}, rép. {Tr[:, 1].mean():7.3f} (p {pb[1]:.3f})")

normale sur le panier (€)      asymétrie : obs.  1.037, rép.  0.003 (p 0.000) | minimum : obs.
↳ 25.530, rép. -11.756 (p 0.000)
normale sur le log du panier  asymétrie : obs.  0.046, rép.  0.003 (p 0.360) | minimum : obs.
↳ 3.240, rép.   3.094 (p 0.144)
```

Le modèle sur le panier brut échoue : les données ont une asymétrie de 1,04 (queue à droite) que le modèle symétrique ne reproduit jamais (asymétrie répliquée : 0,00 ; $p_B = 0$), et il prédit des **paniers négatifs** (le minimum répliqué vaut en moyenne -11,8 €, ce qui est absurde), alors que le plus petit panier observé est de 25,5 €. Le modèle sur le logarithme (c'est-à-dire une loi log-normale pour le panier) passe les deux vérifications ($p_B = 0,36$ pour l'asymétrie et 0,14 pour le minimum). C'est la raison pour laquelle on modélise les montants positifs sur une échelle logarithmique.

Corrigé 6.12

$n = 10, y = 9$. $p(y | H_0) = \binom{10}{9} 0,5^{10} = 10/1024 = 0,00977$; $p(y | H_1) = 1/11 = 0,0909$.
 $BF_{10} = \frac{1/11}{10/1024} = \frac{1024}{110} = 9,31$: des données environ 9 fois plus probables sous H_1 . La p-valeur exacte bilatérale est $2[(\binom{10}{9} + \binom{10}{10})]/1024 = 22/1024 = 0,0215$. Avec une cote a priori de $\frac{1/4}{3/4} = \frac{1}{3}$ en faveur de H_1 , la cote a posteriori est $9,31 \times \frac{1}{3} = 3,10$, soit une probabilité $3,10/4,10 = 0,756$ pour H_1 : on a gagné en crédibilité mais on est loin de la certitude, alors que la p-valeur de 0,02 « rejette H_0 au seuil de 5 % ». C'est la différence de langage entre « rejeter » et « mettre à jour ses croyances ».

```
from scipy.special import comb
m0 = comb(10, 9) * 0.5**10
m1 = 1 / 11
bf = m1 / m0
```

```
print(f"p(y|H0) = {m0:.5f} | p(y|H1) = {m1:.5f} | BF10 = {bf:.3f} (à la main : 1024/110 = {1024 / 110:.3f})")
print(f"p-valeur exacte bilatérale : {stats.binomtest(9, 10, 0.5).pvalue:.4f} (à la main : 22/1024 = {22 / 1024:.4f})")
cote_post = bf * (0.25 / 0.75)
print(f"cote a posteriori {cote_post:.3f} -> P(H1 | données) = {cote_post / (1 + cote_post):.3f}")
```

```
p(y|H0) = 0.00977 | p(y|H1) = 0.09091 | BF10 = 9.309 (à la main : 1024/110 = 9.309)
p-valeur exacte bilatérale : 0.0215 (à la main : 22/1024 = 0.0215)
cote a posteriori 3.103 -> P(H1 | données) = 0.756
```

Corrigé 6.13

On utilise $x_m = u + \frac{\sigma_u}{\xi} [(m\zeta_u)^\xi - 1]$. (a) $m = 1000 : m\zeta_u = 50, 50^{0.2} = e^{0.2 \ln 50} = e^{0.7824} = 2.187$; $x = 5 + \frac{2}{0.2}(2.187 - 1) = 5 + 10 \times 1.187 = 16.87$ jours. (b) $m = 10000 : m\zeta_u = 500, 500^{0.2} = e^{0.2 \ln 500} = 3.466$, donc $x = 5 + 10 \times 2.466 = 29.66$ jours. (c) Avec $\xi = 0.4 : x_{10000} = 5 + \frac{2}{0.4}(500^{0.4} - 1) = 5 + 5 \times (12.01 - 1) = 60.1$ jours. **Doubler ξ double (à peu près) le niveau de retour à 10 000 colis : une petite erreur sur l'indice de queue a une conséquence énorme sur l'extrapolation.**

```
def niveau_pot_ex(m, xi, sc, u, zeta):
    return u + sc / xi * ((m * zeta) ** xi - 1)

for xi_ in (0.2, 0.4):
    print(f"xi = {xi_} : " + " | ".join(f"1 colis sur {m:>6,}: {niveau_pot_ex(m, xi_, 2.0, 5.0, 0.05):6.2f} jours".replace(", ", " ") for m in (1_000, 10_000)))
```

```
xi = 0.2 : 1 colis sur 1 000: 16.87 jours | 1 colis sur 10 000: 29.66 jours
xi = 0.4 : 1 colis sur 1 000: 23.91 jours | 1 colis sur 10 000: 60.06 jours
```

Corrigé 6.14

(a) $P(V \leq q | U \leq q) = \frac{C_\theta(q, q)}{q} = \frac{(2q^{-\theta} - 1)^{-1/\theta}}{q}$. Or $(2q^{-\theta} - 1)^{-1/\theta} = [q^{-\theta}(2 - q^\theta)]^{-1/\theta} = q(2 - q^\theta)^{-1/\theta}$; en divisant par $q : (2 - q^\theta)^{-1/\theta}$. Quand $q \rightarrow 0, q^\theta \rightarrow 0$ (car $\theta > 0$) et l'on obtient $2^{-1/\theta}$. $\square$ (c) Pour la Clayton retournée, $(U, V) = (1 - U', 1 - V')$ avec (U', V') de Clayton : $P(U > 1 - q, V > 1 - q) = P(U' < q, V' < q) = C_\theta(q, q) = (2q^{-\theta} - 1)^{-1/\theta}$. Avec $\theta = 2$ ($\tau = 0.5, \rho = \sin(\pi/4) = 0.7071$ pour la gaussienne), on compare.

```
theta, rho = 2.0, np.sin(np.pi / 4)
lignes = []
for q in (0.1, 0.01, 0.001):
    cond = (2 - q**theta) ** (-1 / theta)
    c_qq = (2 * q ** (-theta) - 1) ** (-1 / theta)
    gauss = multivariate_normal([0, 0], [[1, rho], [rho, 1]]).cdf([norm.ppf(q), norm.ppf(q)])
    lignes.append({"q": q, "P(V<=q | U<=q)": round(cond, 4), "limite": round(2 ** (-1 / theta), 4),
                  "Clayton retournée": f"{c_qq:.3e}", "gaussienne": f"{gauss:.3e}", "rapport":
                  round(c_qq / gauss, 1)})
print(pd.DataFrame(lignes).to_string(index=False))
```

q	P(V<=q U<=q)	limite	Clayton retournée	gaussienne	rapport
0.100	0.7089	0.7071	7.089e-02	4.739e-02	1.5
0.010	0.7071	0.7071	7.071e-03	2.735e-03	2.6
0.001	0.7071	0.7071	7.071e-04	1.654e-04	4.3

La probabilité conditionnelle de Clayton converge vers $2^{-1/2} = 0,707$ (la limite λ), tandis que celle de la copule gaussienne tend vers 0 quand $q \rightarrow 0$. Le rapport entre les probabilités conjointes de la Clayton retournée et de la gaussienne **croît sans cesse** à mesure que q diminue : c'est la signature de la dépendance de queue.

Chapitre 7 : ⊕ Inférence causale — exercices et applications

Ce chapitre du cahier accompagne le chapitre 7 du livre (inférence causale, entièrement facultatif). Les **applications** refont, avec leur code complet, les analyses dont le livre ne donne que le raisonnement et les résultats : elles utilisent `clients.csv` (l'expérience randomisée) et les fichiers simulés `ch07-*.csv` produits par `build/sim_ch07.py`. Les **exercices** (12, notés ★ à ★★★★★) se font d'abord à la main ; les corrigés suivent. Comme les données sont simulées, nous connaissons la vérité et pouvons juger chaque méthode.

Applications

Préparation commune

Tous les blocs de ce chapitre du cahier partagent le même espace de noms : exécutez-les dans l'ordre.

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
from scipy.special import expit
```

Application 7.1 — Les résultats potentiels sur huit clients (section 7.1.2)

Objectif. Voir le « problème fondamental » sur un petit tableau où l'on connaît, exceptionnellement, les deux dépenses potentielles de chaque client, puis calculer l'ATE, l'ATT, l'ATU et la décomposition de la différence naïve.

Étape 1 — le tableau et les trois effets moyens.

```
import itertools
import numpy as np
import pandas as pd

clients = pd.DataFrame({
    "client": list("ABCDEFGH"),
    "y0": [100, 150, 180, 130, 50, 80, 60, 90],      # dépense SANS offre (€)
    "y1": [120, 165, 190, 145, 60, 85, 75, 100],      # dépense AVEC offre (€)
    "offre": [1, 1, 1, 1, 0, 0, 0, 0],               # ce que la gérante a décidé
})
```

```
clients["effet"] = clients["y1"] - clients["y0"]
print("Le tableau vu par Dieu :")
print(clients.to_string(index=False))
print()
print("Le tableau vu par la gérante (une moitié du tableau manque toujours) :")
vu = clients.assign(y0=clients["y0"].where(clients["offre"] == 0),
                    y1=clients["y1"].where(clients["offre"] == 1))
print(vu[["client", "offre", "y0", "y1"]].to_string(index=False))
```

```
Le tableau vu par Dieu :
client y0 y1 offre effet
A 100 120 1 20
B 150 165 1 15
C 180 190 1 10
D 130 145 1 15
E 50 60 0 10
F 80 85 0 5
G 60 75 0 15
H 90 100 0 10
```

```
Le tableau vu par la gérante (une moitié du tableau manque toujours) :
client offre y0 y1
A 1 NaN 120.0
B 1 NaN 165.0
C 1 NaN 190.0
D 1 NaN 145.0
E 0 50.0 NaN
F 0 80.0 NaN
G 0 60.0 NaN
H 0 90.0 NaN
```

Étape 2 — ATE, ATT, ATU, différence naïve. Les effets individuels sont connus ici ; la gérante, elle, n'observe qu'une colonne par client.

```
ate = clients["effet"].mean()
att = clients.loc[clients["offre"] == 1, "effet"].mean()
atu = clients.loc[clients["offre"] == 0, "effet"].mean()
print(f"ATE = {ate} ATT = {att} ATU = {atu}")

observe = np.where(clients["offre"] == 1, clients["y1"], clients["y0"])
moy_offre = observe[clients["offre"] == 1].mean()
moy_sans = observe[clients["offre"] == 0].mean()
print(f"Dépense moyenne observée avec offre : {moy_offre} sans offre : {moy_sans}")
print(f"Différence naïve (ce que calcule la gérante) : {moy_offre - moy_sans}")
```

```
ATE = 12.5 ATT = 15.0 ATU = 10.0
```

```
Dépense moyenne observée avec offre : 155.0 sans offre : 70.0
Différence naïve (ce que calcule la gérante) : 85.0
```

Étape 3 — le biais de sélection. On vérifie que « différence naïve = ATT + biais de sélection » (section 7.1.2).

```
y0_traites = clients.loc[clients["offre"] == 1, "y0"].mean()
y0_non_traites = clients.loc[clients["offre"] == 0, "y0"].mean()
print("Sans offre, les traités auraient dépensé :", y0_traites, "| les non-traités dépensent :",
      ↪ y0_non_traites)
print("Biais de sélection :", y0_traites - y0_non_traites)
print("ATT + biais =", att + (y0_traites - y0_non_traites), "= différence naïve", moy_offre -
      ↪ moy_sans)
```

```
Sans offre, les traités auraient dépensé : 140.0 | les non-traités dépensent : 70.0
Biais de sélection : 70.0
```

ATT + biais = 85.0 = différence naïve 85.0

Lecture. L'ATT vaut 15, l'ATU 10, l'ATE 12,5 ; la différence naïve vaut 85 parce que les clients choisis auraient dépensé 70 € de plus que les autres **même sans offre** : $15 + 70 = 85$.

Pour aller plus loin. Modifiez la colonne offre (par exemple en donnant l'offre aux clients A, C, E, G) : quel est le nouveau biais de sélection ?

Application 7.2 — La randomisation, exhaustivement puis sur 4 000 clients (section 7.1.3)

Objectif. Vérifier que la différence naïve est sans biais quand l'attribution est tirée au sort, d'abord en énumérant les 70 attributions possibles sur les huit clients, puis par simulation sur l'étude observationnelle (ch07-observationnel.csv, offre **ciblée**).

Étape 1 — les 70 attributions possibles (on reprend le tableau de l'application 7.1).

```
estimations = []
for groupe_offre in itertools.combinations(range(8), 4):
    T = np.zeros(8, dtype=int)
    T[list(groupe_offre)] = 1
    y = np.where(T == 1, clients["y1"], clients["y0"])
    estimations.append(y[T == 1].mean() - y[T == 0].mean())
estimations = np.array(estimations)
print(len(estimations), "attributions possibles")
print("moyenne des 70 différences naïves :", estimations.mean(), " | ATE réel :", ate)
print("plus petite / plus grande :", estimations.min(), "/", estimations.max())
```

```
70 attributions possibles
moyenne des 70 différences naïves : 12.5 | ATE réel : 12.5
plus petite / plus grande : -60.0 / 85.0
```

Étape 2 — l'étude observationnelle : ciblage de la gérante contre attributions aléatoires. Le fichier ch07-observationnel-verite.csv contient les deux dépenses potentielles (information inaccessible en pratique).

```
obs = pd.read_csv("donnees/ch07-observationnel.csv")
verite = pd.read_csv("donnees/ch07-observationnel-verite.csv")
d = obs.merge(verite, on="id_client")
ate_vrai = (d["y1"] - d["y0"]).mean()
att_vrai = (d["y1"] - d["y0"])[d["offre"] == 1].mean()
print(f"{len(d)} clients ; {d['offre'].mean():.1%} ont reçu l'offre")
print(f"ATE vrai = {ate_vrai:.2f} € ATT vrai = {att_vrai:.2f} €")

naif = d.loc[d["offre"] == 1, "depense"].mean() - d.loc[d["offre"] == 0, "depense"].mean()
print(f"Différence naïve avec le ciblage de la gérante : {naif:.2f} €")

rng = np.random.default_rng(1)
diffs_alea = []
for _ in range(2000):
    T = rng.permutation(d["offre"].to_numpy()) # même proportion de traités, mais tirés au
    ↪ sort
    y = np.where(T == 1, d["y1"], d["y0"])
    diffs_alea.append(y[T == 1].mean() - y[T == 0].mean())
diffs_alea = np.array(diffs_alea)
print(f"Avec attribution aléatoire : moyenne {diffs_alea.mean():.2f}, écart-type
    ↪ {diffs_alea.std():.2f}")
print(f" 95 % des tirages entre {np.percentile(diffs_alea, 2.5):.1f} et {np.percentile(diffs_alea,
    ↪ 97.5):.1f}")
```

```

4000 clients ; 46.2% ont reçu l'offre
ATE vrai = 15.53 €   ATT vrai = 16.64 €
Différence naïve avec le ciblage de la gérante : 50.50 €
Avec attribution aléatoire : moyenne 15.46, écart-type 2.26
95 % des tirages entre 11.1 et 19.9

```

Lecture. Avec le ciblage de la gérante, la différence naïve est de 50,5 € alors que l'ATE vrai vaut 15,5 € ; avec des attributions tirées au sort, elle se répartit autour de 15,5 (écart-type 2,3). Le tableau d'ainsi construit sert aux applications 7.5 à 7.7.

Application 7.3 — Analyser l'expérience de l'offre de bienvenue (section 7.1.4)

Objectif. Analyser une expérience randomisée comme le ferait un data scientist : vérifier l'équilibre, estimer l'effet sur le rachat et sur la dépense, avec intervalle de confiance, puis comparer à la vérité.

Étape 1 — la randomisation a-t-elle « marché » ? Tableau d'équilibre (différences moyennes standardisées, SMD) et tests.

```

from scipy import stats

c = pd.read_csv("donnees/clients.csv")
traite = c[c["offre_bienvenue"] == 1]
temoin = c[c["offre_bienvenue"] == 0]
print("effectifs : offre =", len(traite), " | pas d'offre =", len(temoin))

def smd(x1, x0):
    return (x1.mean() - x0.mean()) / np.sqrt((x1.var(ddof=1) + x0.var(ddof=1)) / 2)

lignes = [("age", smd(traite["age"], temoin["age"]), traite["age"].mean(), temoin["age"].mean())]
for modalite in ["Réseaux", "Site", "Boutique"]:
    u1 = (traite["canal_acquisition"] == modalite).astype(float)
    u0 = (temoin["canal_acquisition"] == modalite).astype(float)
    lignes.append((f"canal = {modalite}", smd(u1, u0), u1.mean(), u0.mean()))
for v in sorted(c["ville"].unique()):
    u1 = (traite["ville"] == v).astype(float)
    u0 = (temoin["ville"] == v).astype(float)
    lignes.append((f"ville = {v}", smd(u1, u0), u1.mean(), u0.mean()))
equilibre = pd.DataFrame(lignes, columns=["variable", "SMD", "moy. offre", "moy. témoin"]).round(3)
print(equilibre.to_string(index=False))
print()
print("p-valeur (Welch) pour l'âge :", round(stats.ttest_ind(traite["age"], temoin["age"],
    ↪ equal_var=False).pvalue, 3))
print("p-valeur (khi-deux) pour le canal :",
    ↪ round(stats.chi2_contingency(pd.crosstab(c["canal_acquisition"], c["offre_bienvenue"]))[1], 3))
print("p-valeur (khi-deux) pour la ville :", round(stats.chi2_contingency(pd.crosstab(c["ville"],
    ↪ c["offre_bienvenue"]))[1], 3))

```

```

effectifs : offre = 1015 | pas d'offre = 985
      variable  SMD  moy. offre  moy. témoin
      age -0.068    35.397    36.114
canal = Réseaux  0.020     0.413     0.403
      canal = Site  0.029     0.347     0.333
canal = Boutique -0.054     0.240     0.264
      ville = Autre -0.021     0.147     0.154
ville = Ville A  0.016     0.106     0.102
ville = Ville B -0.011     0.124     0.128
ville = Ville C -0.012     0.139     0.143
ville = Ville D -0.011     0.167     0.171
ville = Ville E  0.032     0.317     0.303

```

```
p-valeur (Welch) pour l'âge : 0.128
```

p-valeur (khi-deux) pour le canal : 0.473
 p-valeur (khi-deux) pour la ville : 0.976

Étape 2 — l'effet sur le rachat à 12 mois (différence de proportions et intervalle de Wald).

```
n1, n0 = len(traité), len(temoin)
p1, p0 = traité["rachat_12m"].mean(), témoin["rachat_12m"].mean()
ate_rachat = p1 - p0
se = np.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
print(f"rachat avec offre : {p1:.3f} sans offre : {p0:.3f}")
print(f"effet moyen : {ate_rachat:.3f} (IC 95 % : {ate_rachat - 1.96 * se:.3f} ; {ate_rachat + 1.96 *
↪ se:.3f})")
print(f"effet relatif : {ate_rachat / p0:+.1%} | une offre de plus = {ate_rachat:.3f} rachat de plus
↪ en moyenne,")
print(f"soit environ 1 client de plus qui rachète pour {1 / ate_rachat:.1f} offres envoyées")
```

rachat avec offre : 0.569 sans offre : 0.448
 effet moyen : 0.122 (IC 95 % : 0.078 ; 0.165)
 effet relatif : +27.2% | une offre de plus = 0.122 rachat de plus en moyenne,
 soit environ 1 client de plus qui rachète pour 8.2 offres envoyées

Étape 3 — le même effet par régression logistique, en passant du rapport de cotes à une différence de probabilités (effet marginal moyen).

```
import statsmodels.formula.api as smf

logit = smf.logit("rachat_12m ~ offre_bienvenue", data=c).fit(disp=0)
print(f"coefficient (log-cote) :", round(logit.params["offre_bienvenue"], 3), "| rapport de cotes :",
↪ round(np.exp(logit.params["offre_bienvenue"]), 2))
marg = logit.get_margeff(dummy=True).summary_frame().iloc[0] # dummy=True : vraie différence de
↪ probabilités 1 - 0
print(f"effet marginal moyen : {marg['dy/dx']:.3f} (IC 95 % : {marg['Conf. Int. Low']:.3f} ;
↪ {marg['Conf. Int. Hi.']::.3f})")
```

coefficient (log-cote) : 0.49 | rapport de cotes : 1.63
 effet marginal moyen : 0.122 (IC 95 % : 0.078 ; 0.165)

Étape 4 — l'effet sur la dépense annuelle (test de Welch), puis l'ajustement sur covariables, qui n'améliore ici que la précision.

```
d1, d0 = traité["depense_annuelle"], témoin["depense_annuelle"]
res = stats.ttest_ind(d1, d0, equal_var=False)
ic = res.confidence_interval(0.95)
print(f"dépense moyenne avec offre : {d1.mean():.1f} € sans offre : {d0.mean():.1f} €")
print(f"effet moyen : {d1.mean() - d0.mean():+.1f} € (IC 95 % : {ic.low:.1f} ; {ic.high:.1f}) p =
↪ {res.pvalue:.2f}")
```

dépense moyenne avec offre : 243.5 € sans offre : 250.5 €
 effet moyen : -7.0 € (IC 95 % : -33.1 ; 19.0) p = 0.60

```
simple = smf.ols("rachat_12m ~ offre_bienvenue", data=c).fit(cov_type="HC1")
ajuste = smf.ols("rachat_12m ~ offre_bienvenue + age + C(canal_acquisition) + C(ville)",
↪ data=c).fit(cov_type="HC1")
for nom, m in [("sans covariables", simple), ("avec covariables", ajuste)]:
    b, s = m.params["offre_bienvenue"], m.bse["offre_bienvenue"]
    print(f"{nom} : effet = {b:.3f} erreur-type = {s:.4f} IC 95 % = [{b - 1.96 * s:.3f} ; {b +
↪ 1.96 * s:.3f}])")
```

sans covariables : effet = 0.122 erreur-type = 0.0222 IC 95 % = [0.078 ; 0.165]
 avec covariables : effet = 0.121 erreur-type = 0.0221 IC 95 % = [0.077 ; 0.164]

Étape 5 — la vérité. Dans le simulateur, l'offre ajoute 0,55 à la log-cote du rachat et n'a aucun effet

sur la dépense. On calcule l'effet moyen vrai en probabilité sur une grande population simulée.

```
rng = np.random.default_rng(1)
N = 400_000
age = np.clip(np.round(rng.normal(36, 11, N)), 18, 75)
canal = rng.choice(["Réseaux", "Site", "Boutique"], N, p=[0.40, 0.35, 0.25])
z = rng.normal(size=(N, 2))
F1 = z[:, 0] # facteurs latents du simulateur (corrélation 0,3)
F2 = 0.3 * z[:, 0] + np.sqrt(1 - 0.3 ** 2) * z[:, 1]
eta = -0.35 + 0.45 * F1 + 0.35 * F2 - 0.015 * (age - 36) + 0.3 * (canal == "Boutique")
expit = lambda x: 1 / (1 + np.exp(-x))
print(f"effet moyen vrai sur la probabilité de rachat : {(expit(eta + 0.55) -
    expit(eta)).mean():.4f}")
print("effet vrai sur la dépense annuelle : 0 (par construction du simulateur)")
```

```
effet moyen vrai sur la probabilité de rachat : 0.1239
effet vrai sur la dépense annuelle : 0 (par construction du simulateur)
```

Lecture. L'estimation expérimentale (0,122) est très proche de la vérité (0,124) ; l'effet sur la dépense (−7,0 €, intervalle de −33,1 à +19,0) est compatible avec zéro, et l'on sait ici qu'il est nul.

Pour aller plus loin. Estimez l'effet de l'offre sur le rachat dans chaque canal (c'est l'exercice 7.2) et demandez-vous ce que vaut la comparaison entre canaux.

Application 7.4 — Simpson, médiateur et collision : simuler les trois pièges (sections 7.1.6 à 7.1.8)

Objectif. Voir à l'œuvre, sur des données simulées, les trois structures élémentaires d'un graphe causal et ce que fait (ou ne fait pas) un ajustement par régression.

Étape 1 — la fourche (paradoxe de Simpson). Le tableau de la section 7.1.6 (240 clients), analysé avec et sans ajustement sur le canal.

```
simpson = pd.DataFrame({
    "canal": ["Boutique", "Boutique", "Réseaux", "Réseaux"],
    "offre": [1, 0, 1, 0],
    "clients": [20, 100, 100, 20],
    "rachats": [18, 80, 40, 6],
})
simpson["taux"] = simpson["rachats"] / simpson["clients"]
glob = simpson.groupby("offre")[["clients", "rachats"]].sum()
print("global :", (glob["rachats"] / glob["clients"]).round(3).to_dict())

par_canal = simpson.pivot(index="canal", columns="offre", values="taux")
par_canal["effet"] = par_canal[1] - par_canal[0]
poids = simpson.groupby("canal")["clients"].sum() / simpson["clients"].sum()
print(par_canal.round(3))
print("poids des canaux :", poids.round(2).to_dict())
print("effet standardisé :", round((par_canal["effet"] * poids).sum(), 3))

# La même chose avec une régression logistique sur les 240 clients (une ligne par client)
lignes = pd.DataFrame([{"canal": r.canal, "offre": r.offre, "rachat": int(k < r.rachats)}
    for r in simpson.itertuples() for k in range(r.clients)])
print(len(lignes), "clients, taux de rachat global :", round(lignes["rachat"].mean(), 3))
m_brut = smf.logit("rachat ~ offre", lignes).fit(dis=0)
m_ajuste = smf.logit("rachat ~ offre + C(canal)", lignes).fit(dis=0)
print(f"ncoefficient de l'offre SANS ajustement sur le canal : {m_brut.params['offre']:.2f}")
print(f"coeffcient de l'offre AVEC ajustement sur le canal : {m_ajuste.params['offre']:.2f}")
```

```
global : {0: 0.717, 1: 0.483}
offre    0    1  effet
canal
```

```
Boutique 0.8 0.9 0.1
Réseaux 0.3 0.4 0.1
poids des canaux : {'Boutique': 0.5, 'Réseaux': 0.5}
effet standardisé : 0.1
240 clients, taux de rachat global : 0.6
```

```
coefficient de l'offre SANS ajustement sur le canal : -0.99
coefficient de l'offre AVEC ajustement sur le canal : +0.56
```

Étape 2 — la chaîne : ne pas ajuster sur un médiateur. L'offre est randomisée ; elle agit par un code promotionnel (30 €) et directement (8 €).

```
rng = np.random.default_rng(71)
n = 100_000
motivation = rng.normal(size=n)
offre = rng.integers(0, 2, n)
p_code = expit(0.4 + 0.9 * motivation)
↳ l'offre
code_si_offre = rng.random(n) < p_code
code = offre * code_si_offre
bruit = rng.normal(0, 25, n)
depense = 100 + 8 * offre + 30 * code + 20 * motivation + bruit

y1 = 100 + 8 + 30 * code_si_offre + 20 * motivation + bruit
y0 = 100 + 20 * motivation + bruit
print(f"effet total vrai : {(y1 - y0).mean():.2f} € (= 8 + 30 x {code_si_offre.mean():.2f}, où
↳ {code_si_offre.mean():.0%} des clients utilisent le code s'ils reçoivent l'offre)")

df_m = pd.DataFrame({"depense": depense, "offre": offre, "code": code})
total = smf.ols("depense ~ offre", df_m).fit()
sur_ajuste = smf.ols("depense ~ offre + code", df_m).fit()
print(f"sans ajuster sur le médiateur : effet de l'offre = {total.params['offre']:.2f} (erreur-type
↳ {total.bse['offre']:.2f})")
print(f"en ajustant sur le médiateur : effet de l'offre = {sur_ajuste.params['offre']:.2f}
↳ (erreur-type {sur_ajuste.bse['offre']:.2f})")

# Pourquoi ? Parmi les clients SANS code, les traités et les non-traités n'ont pas la même motivation :
m_traites_sans_code = motivation[(offre == 1) & (code == 0)].mean()
m_temoins = motivation[offre == 0].mean()
print(f"motivation moyenne, offre reçue mais code non utilisé : {m_traites_sans_code:+.2f} | pas
↳ d'offre : {m_temoins:+.2f}")
print(f"écart de dépense dû à cette seule différence de motivation : {20 * (m_traites_sans_code -
↳ m_temoins):+.1f} €")
```

```
effet total vrai : 25.47 € (= 8 + 30 x 0.58, où 58% des clients utilisent le code s'ils reçoivent
↳ l'offre)
sans ajuster sur le médiateur : effet de l'offre = 25.62 (erreur-type 0.22)
en ajustant sur le médiateur : effet de l'offre = -0.80 (erreur-type 0.26)
motivation moyenne, offre reçue mais code non utilisé : -0.45 | pas d'offre : -0.00
écart de dépense dû à cette seule différence de motivation : -9.0 €
```

Étape 3 — la collision : ne pas conditionner sur un effet commun. Qualité et attrait sont indépendants, mais seuls les prototypes réussis sont gardés au catalogue.

```
rng = np.random.default_rng(72)
n = 6000
qualite = rng.normal(size=n)
attrait = rng.normal(size=n)
score = qualite + attrait + rng.normal(0, 0.5, n)
retenu = score > 0.8

print(f"corrélation qualité-attrait, tous les prototypes : {np.corrcoef(qualite, attrait)[0,
↳ 1]:+.3f}")
```

```

print(f"corr lation qualit -attrait, produits retenus      : {np.corrcoef(qualite[retenu],
  ↳ attrait[retenu])[0, 1]:+.3f}")
print(f"({retenu.sum()}) produits retenus sur {n}")

# Ce que ferait un analyste qui n'observe QUE le catalogue :
cat = pd.DataFrame({"qualite": qualite, "attrait": attrait, "retenu": retenu})
brut = smf.ols("qualite ~ attrait", cat).fit()
dans_catalogue = smf.ols("qualite ~ attrait", cat[cat.retenu]).fit()
print(f"pente de la qualit  sur l'attrait, tous : {brut.params['attrait']:+.3f} | catalogue seulement
  ↳ : {dans_catalogue.params['attrait']:+.3f}")

corr lation qualit -attrait, tous les prototypes : +0.011
corr lation qualit -attrait, produits retenus      : -0.479
(1811 produits retenus sur 6000)
pente de la qualit  sur l'attrait, tous : +0.012 | catalogue seulement : -0.494

```

Lecture. Sans ajustement sur le canal, le coefficient de l'offre est $-0,99$; avec le canal, $+0,56$. En ajustant sur le m diateur, l'estimation tombe   $-0,8$   alors que l'effet total est de $25,5$   et l'effet direct de 8  . Sur les seuls produits retenus, la corr lation qualit -attrait passe de $+0,01$   $-0,48$.

Application 7.5 — L' chelle d'ajustement sur l' tude observationnelle (section 7.1.9)

Objectif. Montrer par une suite de r gressions que seul l'ensemble d'ajustement qui bloque tous les chemins de confusion ( ge, canal **et** engagement) retrouve la v rit . On r utilise le tableau d de l'application 7.2.

```

modeles = [
    ("aucun ajustement", "depense ~ offre"),
    ("+  ge", "depense ~ offre + age"),
    ("+  ge + canal", "depense ~ offre + age + C(canal)"),
    ("+  ge + canal + engagement", "depense ~ offre + age + C(canal) + engagement"),
]
lignes = []
for nom, formule in modeles:
    m = smf.ols(formule, d).fit()
    b, s = m.params["offre"], m.bse["offre"]
    lignes.append({"ensemble d'ajustement": nom, "effet estim ": b, "IC95 bas": b - 1.96 * s, "IC95
  ↳ haut": b + 1.96 * s})
tab = pd.DataFrame(lignes).round(1)
print(tab.to_string(index=False))
print(f"\nv rit  : ATE = {ate_vrai:.1f}    ATT = {att_vrai:.1f}")

```

ensemble d'ajustement	effet estim�	IC95 bas	IC95 haut
aucun ajustement	50.5	46.3	54.7
+ �ge	46.1	41.9	50.3
+ �ge + canal	47.0	42.8	51.2
+ �ge + canal + engagement	14.8	11.0	18.7

v rit  : ATE = 15.5 ATT = 16.6

Lecture. Tant que l'engagement manque, l'estimation reste entre 46 et 51   (v rit  : ATE = $15,5$  ) ; avec les trois variables, elle tombe   $14,8$   avec un intervalle de $11,0$   $18,7$.

Application 7.6 — Le score de propension : estimation, chevauchement, appariement (sections 7.2.1 à 7.2.3)

Objectif. Estimer l'effet de l'offre ciblée avec un score de propension : malédiction de la dimension, estimation du score, diagnostic de chevauchement, appariement au plus proche voisin avec calibre, bootstrap et équilibre des covariables.

Étape 1 — pourquoi un score ? Avec trois covariables seulement, combien de clients ont un « jumeau » de l'autre groupe ?

```
cellules = d.groupby(["age", "canal", "engagement"])["offre"].agg(["size", "sum"])
mixtes = cellules[(cellules["sum"] > 0) & (cellules["sum"] < cellules["size"])]
print(f"{len(d)} clients répartis en {len(cellules)} cellules (âge x canal x engagement)")
print(f"cellules où l'on trouve à la fois un client avec offre et un sans : {len(mixtes)}")
print(f"clients se trouvant dans une telle cellule : {int(mixtes['size'].sum())} sur {len(d)}")
```

```
4000 clients répartis en 2923 cellules (âge x canal x engagement)
cellules où l'on trouve à la fois un client avec offre et un sans : 413
clients se trouvant dans une telle cellule : 1035 sur 4000
```

Étape 2 — estimer le score et regarder le chevauchement.

```
from sklearn.metrics import roc_auc_score

modele_ps = smf.logit("offre ~ age + C(canal) + engagement", data=d).fit(dis=0)
print(modele_ps.params.round(3).to_string())
d["ps"] = modele_ps.predict(d)
print(f"\nAUC du modèle d'attribution : {roc_auc_score(d['offre'], d['ps']):.3f}")
print(d.groupby("offre")["ps"].describe().round(3).to_string())
```

```
Intercept          -2.462
C(canal)[T.Réseaux]  0.454
C(canal)[T.Site]    -0.060
age                 -0.032
engagement          0.063
```

```
AUC du modèle d'attribution : 0.761
```

	count	mean	std	min	25%	50%	75%	max
offre								
0	2150.0	0.368	0.198	0.021	0.207	0.342	0.502	0.966
1	1850.0	0.572	0.204	0.049	0.423	0.582	0.731	0.969

```
bas_commun = max(d.loc[d.offre == 1, "ps"].min(), d.loc[d.offre == 0, "ps"].min())
haut_commun = min(d.loc[d.offre == 1, "ps"].max(), d.loc[d.offre == 0, "ps"].max())
hors = ((d["ps"] < bas_commun) | (d["ps"] > haut_commun)).sum()
print(f"support commun : [{bas_commun:.3f} ; {haut_commun:.3f}] -> {hors} clients en dehors")
```

```
support commun : [0.049 ; 0.966] -> 22 clients en dehors
```

Étape 3 — l'appariement (au plus proche voisin sur le logit du score, avec remise et calibre de 0,2 écart-type).

```
from sklearn.neighbors import NearestNeighbors

def apparier(df, calibre=0.2):
    """Renvoie (effet ATT, nombre de traités appariés, indices des témoins appariés)."""
    logit_ps = np.log(df["ps"] / (1 - df["ps"])).to_numpy()
    T = df["offre"].to_numpy()
    traites, temoins = np.where(T == 1)[0], np.where(T == 0)[0]
    nn = NearestNeighbors(n_neighbors=1).fit(logit_ps[temoins].reshape(-1, 1))
    dist, pos = nn.kneighbors(logit_ps[traites].reshape(-1, 1))
    ok = dist[:, 0] <= calibre * logit_ps.std()
    appar_t = traites[ok]
```

```

    appar_c = temoins[pos[ok, 0]]
    y = df["depense"].to_numpy()
    return (y[appar_t] - y[appar_c]).mean(), ok.sum(), appar_t, appar_c

att_appar, n_appar, idx_t, idx_c = apparier(d)
print(f"{n_appar} traités appariés sur {int(d['offre'].sum())}")
print(f"témoins distincts utilisés : {len(np.unique(idx_c))} (un même témoin sert en moyenne
↪ {len(idx_c) / len(np.unique(idx_c)):.1f} fois)")
print(f"effet estimé par appariement (ATT) : {att_appar:.2f} € | ATT vrai : {att_vrai:.2f}")

```

1843 traités appariés sur 1850
 témoins distincts utilisés : 814 (un même témoin sert en moyenne 2.3 fois)
 effet estimé par appariement (ATT) : 14.41 € | ATT vrai : 16.64

Étape 4 — l'incertitude par bootstrap : on refait *toute* la procédure (estimation du score comprise) sur chaque échantillon rééchantillonné.

```

def ps_et_appariement(df):
    m = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(dis=0)
    df = df.assign(ps=m.predict(df))
    return apparier(df)[0]

rng = np.random.default_rng(2)
boot = []
for _ in range(200):
    echantillon = d.sample(len(d), replace=True,
↪ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True)
    boot.append(ps_et_appariement(echantillon))
boot = np.array(boot)
print(f"erreur-type bootstrap : {boot.std():.2f} IC 95 % (percentiles) : [{np.percentile(boot,
↪ 2.5):.1f} ; {np.percentile(boot, 97.5):.1f}]")

```

erreur-type bootstrap : 3.66 IC 95 % (percentiles) : [6.6 ; 20.2]

Étape 5 — l'équilibre avant/après appariement (différences moyennes standardisées pondérées).

```

def smd_pondere(x, T, w):
    """Différence moyenne standardisée entre traités (T=1) et témoins (T=0), avec poids w."""
    x, T, w = np.asarray(x, float), np.asarray(T), np.asarray(w, float)
    m1 = np.average(x[T == 1], weights=w[T == 1])
    m0 = np.average(x[T == 0], weights=w[T == 0])
    v1 = np.average((x[T == 1] - m1) ** 2, weights=w[T == 1])
    v0 = np.average((x[T == 0] - m0) ** 2, weights=w[T == 0])
    return (m1 - m0) / np.sqrt((v1 + v0) / 2)

covariables = pd.DataFrame({
    "âge": d["age"], "engagement": d["engagement"],
    "canal = Réseaux": (d["canal"] == "Réseaux").astype(float),
    "canal = Site": (d["canal"] == "Site").astype(float),
    "canal = Boutique": (d["canal"] == "Boutique").astype(float)})

poids_appar = np.zeros(len(d))
poids_appar[idx_t] += 1 # chaque traité apparié compte une fois
np.add.at(poids_appar, idx_c, 1) # chaque témoin compte autant de fois qu'il
↪ est utilisé
avant = {c: smd_pondere(covariables[c], d["offre"], np.ones(len(d))) for c in covariables}
apres_appar = {c: smd_pondere(covariables[c], d["offre"], poids_appar) for c in covariables}
print(pd.DataFrame({"SMD avant": avant, "SMD après appariement": apres_appar}).round(3).to_string())

```

	SMD avant	SMD après appariement
âge	-0.336	0.003
engagement	0.915	0.006

canal = Réseaux	0.306	-0.013
canal = Site	-0.204	0.041
canal = Boutique	-0.118	-0.028

Lecture. 1 843 traités sur 1 850 sont appariés ; l'ATT estimé est de 14,4 € (vrai : 16,6 €) avec un intervalle bootstrap de 6,6 à 20,2 € ; après appariement, toutes les SMD sont inférieures à 0,05.

Application 7.7 — Pondération, estimateur doublement robuste et confusion mal mesurée (sections 7.2.4 à 7.2.6)

Objectif. Estimer l'ATE et l'ATT par pondération par l'inverse du score (IPW), vérifier les poids, construire l'estimateur doublement robuste, le mettre à l'épreuve en rendant l'un des deux modèles faux, et mesurer l'effet d'un facteur de confusion mal mesuré. On réutilise les objets des applications 7.2 et 7.6.

Étape 1 — IPW : ATE (Horvitz-Thompson et version normalisée) et ATT.

```
e = d["ps"].to_numpy()
T = d["offre"].to_numpy()
Y = d["depense"].to_numpy()

w_ate = np.where(T == 1, 1 / e, 1 / (1 - e))
ht = np.mean(T * Y / e - (1 - T) * Y / (1 - e)) # Horvitz-Thompson
hajek = np.average(Y[T == 1], weights=w_ate[T == 1]) - np.average(Y[T == 0], weights=w_ate[T == 0])
w_att = np.where(T == 1, 1.0, e / (1 - e))
att_ipw = np.average(Y[T == 1]) - np.average(Y[T == 0], weights=w_att[T == 0])

print(f"ATE par IPW (Horvitz-Thompson) : {ht:.2f}")
print(f"ATE par IPW (normalisé) : {hajek:.2f} | ATE vrai : {ate_vrai:.2f}")
print(f"ATT par IPW : {att_ipw:.2f} | ATT vrai : {att_vrai:.2f}")
```

```
ATE par IPW (Horvitz-Thompson) : 13.44
ATE par IPW (normalisé) : 14.36 | ATE vrai : 15.53
ATT par IPW : 11.84 | ATT vrai : 16.64
```

Étape 2 — les poids sont-ils raisonnables ? Effectif effectif et troncature des poids.

```
def effectif_effectif(w):
    return w.sum() ** 2 / (w ** 2).sum()

for nom, mask in [("avec offre", T == 1), ("sans offre", T == 0)]:
    w = w_ate[mask]
    print(f"{nom} : n = {mask.sum()}, poids min/médian/max = {w.min():.2f} / {np.median(w):.2f} / {w.max():.1f}, effectif effectif = {effectif_effectif(w):.0f}")

# Troncature : on borne les poids aux percentiles 1 et 99
bas, haut = np.percentile(w_ate, [1, 99])
w_tronque = np.clip(w_ate, bas, haut)
hajek_tronque = np.average(Y[T == 1], weights=w_tronque[T == 1]) - np.average(Y[T == 0], weights=w_tronque[T == 0])
print(f"\nATE par IPW avec poids tronqués à [{bas:.2f} ; {haut:.1f}] : {hajek_tronque:.2f}")
```

```
avec offre : n = 1850, poids min/médian/max = 1.03 / 1.72 / 20.5, effectif effectif = 1261
sans offre : n = 2150, poids min/médian/max = 1.02 / 1.52 / 29.3, effectif effectif = 1508
```

```
ATE par IPW avec poids tronqués à [1.06 ; 7.4] : 17.52
```

Étape 3 — l'incertitude par bootstrap, puis l'équilibre après pondération.

```
def ipw_ate_att(df):
    e_b = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(dis=0).predict(df).to_numpy()
```

```

Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
w = np.where(Tb == 1, 1 / e_b, 1 / (1 - e_b))
ate_b = np.average(Yb[Tb == 1], weights=w[Tb == 1]) - np.average(Yb[Tb == 0], weights=w[Tb == 0])
att_b = Yb[Tb == 1].mean() - np.average(Yb[Tb == 0], weights=(e_b / (1 - e_b))[Tb == 0])
return ate_b, att_b

rng = np.random.default_rng(4)
boot_ipw = np.array([ipw_ate_att(d.sample(len(d), replace=True,
↳ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True))
                    for _ in range(200)])
for nom, i, vrai in [("ATE", 0, ate_vrai), ("ATT", 1, att_vrai)]:
    b = boot_ipw[:, i]
    print(f"IPW {nom} : erreur-type bootstrap {b.std():.2f} IC 95 % : [{np.percentile(b, 2.5):.1f} ;
↳ {np.percentile(b, 97.5):.1f}] (vérité {vrai:.1f})")

```

```

IPW ATE : erreur-type bootstrap 2.50 IC 95 % : [9.6 ; 19.1] (vérité 15.5)
IPW ATT : erreur-type bootstrap 3.37 IC 95 % : [4.7 ; 17.8] (vérité 16.6)

```

```

apres_ipw = {c: smd_pondere(covariables[c], T, w_ate) for c in covariables}
print(pd.DataFrame({"SMD avant": avant, "SMD après IPW": apres_ipw}).round(3).to_string())

```

	SMD avant	SMD après IPW
âge	-0.336	-0.030
engagement	0.915	-0.022
canal = Réseaux	0.306	-0.003
canal = Site	-0.204	-0.001
canal = Boutique	-0.118	0.004

Étape 4 — l'estimateur doublement robuste, mis à l'épreuve. On rend volontairement faux l'un des deux modèles (résultat sans covariables, ou score constant).

```

X = np.column_stack([np.ones(len(d)), d["age"], (d["canal"] == "Réseaux"), (d["canal"] == "Site"),
↳ d["engagement"]]).astype(float)

def mu_hat(X, Y, T, bon_modele):
    """Prédictions de E[Y | X, T=t] pour t = 0 et 1, par moindres carrés séparés dans chaque groupe."""
    sorties = []
    for t in (0, 1):
        cols = slice(None) if bon_modele else slice(0, 1) # mauvais modèle = constante seule
        beta, *_ = np.linalg.lstsq(X[T == t][:, cols], Y[T == t], rcond=None)
        sorties.append(X[:, cols] @ beta)
    return sorties

def e_hat(X, T, bon_modele):
    if not bon_modele:
        return np.full(len(T), T.mean())
    m = smf.logit("offre ~ age + C(canal) + engagement", data=d).fit(dis=0)
    return m.predict(d).to_numpy()

def estimateurs(bon_resultat, bon_score):
    mu0, mu1 = mu_hat(X, Y, T, bon_resultat)
    e = e_hat(X, T, bon_score)
    regression = np.mean(mu1 - mu0)
    ipw = np.average(Y[T == 1], weights=1 / e[T == 1]) - np.average(Y[T == 0], weights=1 / (1 - e[T ==
↳ 0]))
    aipw = np.mean(mu1 - mu0 + T * (Y - mu1) / e - (1 - T) * (Y - mu0) / (1 - e))
    return regression, ipw, aipw

lignes = []
for br in (True, False):
    for bs in (True, False):
        reg, ipw, aipw = estimateurs(br, bs)
        lignes.append({"modèle de résultat": "correct" if br else "faux", "modèle d'attribution":
↳ "correct" if bs else "faux",

```

```

    "régression": reg, "IPW": ipw, "doublement robuste": aipw))
print(pd.DataFrame(lignes).round(1).to_string(index=False))
print(f"\nATE vrai : {ate_vrai:.1f}    (différence naïve : {Y[T == 1].mean() - Y[T == 0].mean():.1f})")

```

modèle de résultat	modèle d'attribution	régression	IPW	doublement robuste
correct	correct	15.0	14.4	14.9
correct	faux	15.0	50.5	15.0
faux	correct	50.5	14.4	14.3
faux	faux	50.5	50.5	50.5

ATE vrai : 15.5 (différence naïve : 50.5)

Étape 5 — l'estimation principale par AIPW, avec son bootstrap.

```

def aipw_complet(df):
    Xb = np.column_stack([np.ones(len(df)), df["age"], (df["canal"] == "Réseaux"), (df["canal"] ==
    ↪ "Site"), df["engagement"]]).astype(float)
    Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
    mu0, mu1 = mu_hat(Xb, Yb, Tb, True)
    eb = smf.logit("offre ~ age + C(canal) + engagement", data=df).fit(disps=0).predict(df).to_numpy()
    return np.mean(mu1 - mu0 + Tb * (Yb - mu1) / eb - (1 - Tb) * (Yb - mu0) / (1 - eb))

est = aipw_complet(d)
rng = np.random.default_rng(3)
boot = np.array([aipw_complet(d.sample(len(d), replace=True,
    ↪ random_state=int(rng.integers(1_000_000_000))).reset_index(drop=True))
    for _ in range(200)])
print(f"ATE doublement robuste : {est:.2f}    erreur-type bootstrap : {boot.std():.2f}    IC 95 % :
    ↪ [{est - 1.96 * boot.std():.1f} ; {est + 1.96 * boot.std():.1f}])")
print(f"ATE vrai : {ate_vrai:.2f}")

```

ATE doublement robuste : 14.89 erreur-type bootstrap : 2.44 IC 95 % : [10.1 ; 19.7]
 ATE vrai : 15.53

Étape 6 — récapitulatif, puis confusion mal mesurée. On cache l'engagement, ou on ne le mesure qu'avec du bruit.

```

recap = pd.DataFrame({
    "méthode": ["différence naïve", "régression (7.1.9)", "IPW (ATE)", "doublement robuste (ATE)",
    ↪ "appariement (ATT)", "IPW (ATT)"],
    "estimation": [Y[T == 1].mean() - Y[T == 0].mean(),
        smf.ols("depense ~ offre + age + C(canal) + engagement", d).fit().params["offre"],
        hajek, est, att_appar, att_ipw],
    "vérité": [ate_vrai, ate_vrai, ate_vrai, ate_vrai, att_vrai, att_vrai]).round(1)
print(recap.to_string(index=False))

```

	méthode	estimation	vérité
	différence naïve	50.5	15.5
	régression (7.1.9)	14.8	15.5
	IPW (ATE)	14.4	15.5
	doublement robuste (ATE)	14.9	15.5
	appariement (ATT)	14.4	16.6
	IPW (ATT)	11.8	16.6

```

def estimation_aipw_avec_engagement(bruit_sd, graine=5):
    """AIPW quand l'engagement n'est connu qu'avec un bruit gaussien d'écart-type bruit_sd (None =
    ↪ engagement caché)."""
    df = d.copy()
    if bruit_sd is None:
        formule_ps = "offre ~ age + C(canal)"
        cols = [np.ones(len(df)), df["age"], (df["canal"] == "Réseaux"), (df["canal"] == "Site")]
    else:
        r = np.random.default_rng(graine)

```

```

df["eng_mesure"] = df["engagement"] + r.normal(0, bruit_sd, len(df))
formule_ps = "offre ~ age + C(canal) + eng_mesure"
cols = [np.ones(len(df)), df["age"], (df["canal"] == "Réseaux"), (df["canal"] == "Site"),
↪ df["eng_mesure"]]
Xb = np.column_stack(cols).astype(float)
Tb, Yb = df["offre"].to_numpy(), df["depense"].to_numpy()
mu0, mu1 = mu_hat(Xb, Yb, Tb, True)
eb = smf.logit(formule_ps, data=df).fit(dis=0).predict(df).to_numpy()
return np.mean(mu1 - mu0 + Tb * (Yb - mu1) / eb - (1 - Tb) * (Yb - mu0) / (1 - eb))

resultats = [("engagement parfaitement mesuré", estimation_aipw_avec_engagement(0.0))]
for sd in (15, 30, 60):
    resultats.append((f"engagement mesuré avec du bruit (écart-type {sd})",
↪ estimation_aipw_avec_engagement(sd)))
resultats.append(("engagement non observé", estimation_aipw_avec_engagement(None)))
print(pd.DataFrame(resultats, columns=["situation", "ATE doublement
↪ robuste"]).round(1).to_string(index=False))
print(f"\nATE vrai : {ate_vrai:.1f}")
print(f"écart-type de l'engagement lui-même : {d['engagement'].std():.1f}")

```

	situation	ATE doublement robuste
	engagement parfaitement mesuré	14.9
	engagement mesuré avec du bruit (écart-type 15)	32.5
	engagement mesuré avec du bruit (écart-type 30)	41.8
	engagement mesuré avec du bruit (écart-type 60)	45.6
	engagement non observé	46.9

ATE vrai : 15.5

écart-type de l'engagement lui-même : 15.5

Lecture. Les estimations de l'ATE se situent entre 14,4 et 14,9 € (vérité : 15,5), contre 50,5 € pour la différence naïve. Quand le modèle de résultat et le score sont tous deux faux, l'AIPW échoue aussi ; et quand l'engagement est mesuré avec un bruit de 15 (autant que son propre écart-type), l'estimation « doublement robuste » est déjà à 32,5 €.

Application 7.8 — La différence de différences sur le panel de villes (section 7.3)

Objectif. Estimer l'effet d'une campagne lancée dans 8 villes sur 20, avec la DiD à la main, en logarithme, par régression à effets fixes (erreurs-types groupées), puis diagnostiquer l'hypothèse de tendances parallèles (étude d'événement, test placebo) et en voir la violation.

Étape 1 — les données. Le simulateur `panel_villes` est dans `build/sim_ch07.py` ; le fichier est chargé et vérifié.

```

import sys
sys.path.insert(0, "build")
from sim_ch07 import panel_villes, iv # simulateurs du chapitre (graines fixes)

p = pd.read_csv("donnees/ch07-panel-villes.csv", parse_dates=["mois"])
regen = panel_villes()
regen["mois"] = pd.to_datetime(regen["mois"])
print("le simulateur reproduit exactement le fichier :", p.equals(regen))
p["apres"] = (p["t"] >= 18).astype(int)
p["vid"] = pd.factorize(p["ville"])[0] # identifiant numérique de ville
↪ (erreurs-types groupées)
print(f"{p['ville'].nunique()} villes, {p['t'].nunique()} mois, {len(p)} lignes ;
↪ {p.groupby('ville')['groupe_traite'].first().sum()} villes traitées")

```

le simulateur reproduit exactement le fichier : True
 20 villes, 24 mois, 480 lignes ; 8 villes traitées

Étape 2 — la DiD à la main : moyennes avant/après dans chaque groupe, en niveau puis en logarithme, et le diagnostic de l'échelle.

```
moy = p.groupby(["groupe_traite", "apres"])[["commandes"]].mean().unstack().round(1)
moy.index = ["témoins (12 villes)", "traitées (8 villes)"]
moy.columns = ["avant", "après"]
print(moy)
```

	avant	après
témoins (12 villes)	30.6	36.1
traitées (8 villes)	46.9	63.0

```
moyenne_mois = p.groupby(["groupe_traite", "mois"])[["commandes"]].mean().unstack(0)
moyenne_mois.columns = ["témoins", "traitées"]
debut = pd.Timestamp("2025-07-01")
```

```
avant_lancement = moyenne_mois[moyenne_mois.index < debut]
ecart = avant_lancement["traitées"] - avant_lancement["témoins"]
rapport = avant_lancement["traitées"] / avant_lancement["témoins"]
print(f"écart de niveau entre les groupes : de {ecart.min():.1f} à {ecart.max():.1f} commandes selon
↳ le mois (moyenne {ecart.mean():.1f})")
print(f"rapport traitées / témoins : de {rapport.min():.2f} à {rapport.max():.2f} selon le mois
↳ (moyenne {rapport.mean():.2f})")
print(f"variabilité relative (écart-type / moyenne) : écart {ecart.std() / ecart.mean():.0%}, rapport
↳ {rapport.std() / rapport.mean():.0%}")
```

```
écart de niveau entre les groupes : de 5.7 à 24.5 commandes selon le mois (moyenne 16.3)
rapport traitées / témoins : de 1.25 à 1.77 selon le mois (moyenne 1.54)
variabilité relative (écart-type / moyenne) : écart 30%, rapport 9%
```

```
lm = p.assign(log_cmd=np.log(p["commandes"])).groupby(["groupe_traite",
↳ "apres"])[["log_cmd"]].mean().unstack()
did_log = (lm.loc[1, 1] - lm.loc[1, 0]) - (lm.loc[0, 1] - lm.loc[0, 0])
print("moyennes du logarithme des commandes :")
print(lm.round(3).rename(index={0: "témoins", 1: "traitées"}, columns={0: "avant", 1: "après"}))
print(f"\nDiD en logarithme : {did_log:.3f} soit un effet relatif de {np.exp(did_log) - 1:+.1%}")
print(f"(rappel : DiD en niveau = {(moy.iloc[1, 1] - moy.iloc[1, 0]) - (moy.iloc[0, 1] - moy.iloc[0,
↳ 0]):.1f} commandes par mois)")
```

```
moyennes du logarithme des commandes :
apres      avant  après
groupe_traite
témoins      3.369  3.537
traitées      3.784  4.089
```

```
DiD en logarithme : 0.138 soit un effet relatif de +14.8%
(rappel : DiD en niveau = 10.6 commandes par mois)
```

Étape 3 — la régression à effets fixes (TWFE), en logarithme et en modèle de Poisson, avec erreurs-types groupées par ville.

```
def twfe(df, debut=18, fin=None, tendance_groupe=False, poisson=False):
    """DiD par régression à effets fixes ville + mois ; erreurs-types groupées par ville."""
    d = df if fin is None else df[df["t"] < fin]
    d = d.assign(camp=((d["groupe_traite"] == 1) & (d["t"] >= debut)).astype(int))
    formule = "commandes ~ camp + C(ville) + C(t)" + (" + groupe_traite:t" if tendance_groupe else "")
    if poisson:
        m = smf.glm(formule, d, family=sm.families.Poisson()).fit(cov_type="cluster",
↳ cov_kws={"groups": d["vid"]})
    else:
        m = smf.ols(formule.replace("commandes", "np.log(commandes)"), d).fit(cov_type="cluster",
↳ cov_kws={"groups": d["vid"]})
    return m.params["camp"], m.bse["camp"]
```

```

b, se = twfe(p)
print(f"TWFE (log) : effet = {b:.3f}  erreur-type = {se:.3f}  IC 95 % = [{b - 1.96 * se:.3f} ; {b +
↳ 1.96 * se:.3f}]  ({np.exp(b) - 1:+.1%}")
b_p, se_p = twfe(p, poisson=True)
print(f"Poisson : effet = {b_p:.3f}  erreur-type = {se_p:.3f}  IC 95 % = [{b_p - 1.96 * se_p:.3f} ;
↳ {b_p + 1.96 * se_p:.3f}]  ({np.exp(b_p) - 1:+.1%}")
print(f"(rappel du calcul à la main : {did_log:.3f}")

```

```

TWFE (log) : effet = 0.138  erreur-type = 0.032  IC 95 % = [0.076 ; 0.200]  (+14.8%)
Poisson : effet = 0.131  erreur-type = 0.028  IC 95 % = [0.076 ; 0.186]  (+14.0%)
(rappel du calcul à la main : 0.138)

```

Étape 4 — l'étude d'événement : un coefficient par mois relatif au lancement (référence : le mois précédent).

```

def etude_evenement(df, ref=-1):
    """Coefficient mois par mois (traitées vs témoins), relatif au mois de lancement (t = 18) ;
    ↳ référence : k = -1."""
    d = df.copy()
    d["rel"] = d["t"] - 18
    termes = []
    for k in sorted(d["rel"].unique()):
        if k == ref:
            continue
        nom = f"ev_{'m' if k < 0 else 'p'}{abs(k)}"
        d[nom] = ((d["groupe_traite"] == 1) & (d["rel"] == k)).astype(int)
        termes.append((k, nom))
    formule = "np.log(commandes) ~ " + " + ".join(n for _, n in termes) + " + C(ville) + C(t)"
    m = smf.ols(formule, d).fit(cov_type="cluster", cov_kws={"groups": d["vid"]})
    return pd.DataFrame({"k": [k for k, _ in termes], "coef": [m.params[n] for _, n in termes],
        "se": [m.bse[n] for _, n in termes]})

ee = etude_evenement(p)
avant = ee[ee["k"] < 0]
apres = ee[ee["k"] >= 0]
print(f"avant le lancement : coefficient moyen = {avant['coef'].mean():+.3f} ; plus grand écart en
↳ valeur absolue = {avant['coef'].abs().max():.3f}")
print(f"nombre de mois « avant » dont l'IC à 95 % exclut 0 : {int(((avant['coef'].abs() - 1.96 *
↳ avant['se']) > 0).sum())} sur {len(avant)}")
print(f"après le lancement : coefficient moyen = {apres['coef'].mean():+.3f}  (effet vrai : 0.150)")

```

```

avant le lancement : coefficient moyen = +0.054 ; plus grand écart en valeur absolue = 0.188
nombre de mois « avant » dont l'IC à 95 % exclut 0 : 0 sur 17
après le lancement : coefficient moyen = +0.189  (effet vrai : 0.150)

```

Étape 5 — quand les tendances ne sont pas parallèles : les grandes villes croissent 1 % plus vite par mois, hors campagne. On regarde l'estimation, le placebo, puis le remède de la tendance de groupe.

```

q = panel_villes(tendance_diff=0.01)
q["vid"] = pd.factorize(q["ville"])[0]

b_q, se_q = twfe(q)
print(f"Effet estimé par DiD avec tendances NON parallèles : {b_q:.3f}  (erreur-type {se_q:.3f})  |
↳ effet vrai : 0.150")
ee_q = etude_evenement(q)
avant_q = ee_q[ee_q["k"] < 0]
print(f"mois « avant » dont l'IC à 95 % exclut 0 : {int(((avant_q['coef'].abs() - 1.96 *
↳ avant_q['se']) > 0).sum())} sur {len(avant_q)}")

```

Effet estimé par DiD avec tendances NON parallèles : 0.315 (erreur-type 0.031) | effet vrai : 0.150
 mois « avant » dont l'IC à 95 % exclut 0 : 2 sur 17

```
for nom, df in [("tendances parallèles", p), ("tendances non parallèles", q)]:
    b_pl, se_pl = twfe(df, debut=9, fin=18)
    print(f"placebo ({nom}) : effet « fictif » = {b_pl:+.3f} (erreur-type {se_pl:.3f}, rapport {b_pl
    ↪ / se_pl:+.1f})")
```

placebo (tendances parallèles) : effet « fictif » = -0.062 (erreur-type 0.038, rapport -1.6)
 placebo (tendances non parallèles) : effet « fictif » = +0.097 (erreur-type 0.042, rapport +2.3)

```
b_t, se_t = twfe(q, tendance_groupe=True)
print(f"avec tendance linéaire propre au groupe traité : effet = {b_t:.3f} (erreur-type {se_t:.3f})
    ↪ | vrai : 0.150")
b_t2, se_t2 = twfe(p, tendance_groupe=True)
print(f"(le même modèle sur les données initiales, où il n'était pas nécessaire : effet = {b_t2:.3f},
    ↪ erreur-type {se_t2:.3f})")
```

avec tendance linéaire propre au groupe traité : effet = 0.185 (erreur-type 0.063) | vrai : 0.150
 (le même modèle sur les données initiales, où il n'était pas nécessaire : effet = 0.181, erreur-type
 ↪ 0.062)

Lecture. La DiD en logarithme vaut 0,138 (+14,8 % ; vérité : 0,15 soit +16,2 %). Avec des tendances non parallèles, l'estimation passe à 0,315, le double de la vérité, alors que son erreur-type reste de 0,031 : c'est l'étude d'événement (2 mois « avant » sur 17 déjà significatifs) et le placebo (rapport +2,3) qui donnent l'alerte.

Application 7.9 — La variable instrumentale : Wald, 2SLS, complaisants, instrument faible (section 7.4)

Objectif. Estimer l'effet de « suivre le compte de la boutique » sur la dépense malgré un facteur de confusion non observé (la passion), avec un rappel envoyé au hasard comme instrument : première étape, estimateur de Wald, moindres carrés en deux étapes, effet pour les complaisants, instrument faible et exclusion violée.

Étape 1 — les données et la comparaison naïve (ch07-iv.csv, simulateur iv de build/sim_ch07.py).

```
from statsmodels.sandbox.regression.gmm import IV2SLS

d = pd.read_csv("donnees/ch07-iv.csv")
print("le simulateur reproduit exactement le fichier :", d.equals(iv()))
print(d.head(5).to_string(index=False))
print(f"\n{len(d)} clients ; {d['suit_compte'].mean():.1%} suivent le compte ;
    ↪ {d['rappel'].mean():.1%} ont reçu le rappel")
print(d.groupby("suit_compte")["depense"].agg(["count", "mean"]).round(1).rename(index={0: "ne suit
    ↪ pas", 1: "suit"}))
```

le simulateur reproduit exactement le fichier : True

id_client	age	rappel	suit_compte	depense
1	39	1	1	5.76
2	40	0	0	124.21
3	37	1	1	142.74
4	35	0	1	69.39
5	58	1	1	51.32

5000 clients ; 62.3% suivent le compte ; 48.8% ont reçu le rappel

	count	mean
suit_compte		
ne suit pas	1886	68.7
suit	3114	109.8

```
ols = smf.ols("depense ~ suit_compte + age", d).fit(cov_type="HC1")
b_ols, se_ols = ols.params["suit_compte"], ols.bse["suit_compte"]
print(f"MCO : effet de suivre le compte = {b_ols:.1f} € (erreur-type {se_ols:.1f}, IC 95 % : [{b_ols
↳ - 1.96 * se_ols:.1f} ; {b_ols + 1.96 * se_ols:.1f}])")
```

MCO : effet de suivre le compte = 41.9 € (erreur-type 1.4, IC 95 % : [39.1 ; 44.7])

Étape 2 — l'instrument : indépendance (équilibre) et pertinence (première étape, statistique F).

```
print("âge moyen avec rappel :", round(d.loc[d.rappel == 1, "age"].mean(), 2), "| sans rappel :",
↳ round(d.loc[d.rappel == 0, "age"].mean(), 2))
print(f"SMD de l'âge : {(d.loc[d.rappel == 1, 'age'].mean() - d.loc[d.rappel == 0, 'age'].mean()) /
↳ np.sqrt((d.loc[d.rappel == 1, 'age'].var() + d.loc[d.rappel == 0, 'age'].var()) / 2):+.3f}")
```

âge moyen avec rappel : 36.47 | sans rappel : 36.38
SMD de l'âge : +0.008

```
etapel = smf.ols("suit_compte ~ rappel + age", d).fit(cov_type="HC1")
print("part de clients qui suivent le compte, sans rappel :", round(d.loc[d.rappel == 0,
↳ "suit_compte"].mean(), 3),
      "| avec rappel :", round(d.loc[d.rappel == 1, "suit_compte"].mean(), 3))
print(f"première étape : effet du rappel sur la probabilité de suivre = {etapel.params['rappel']:.3f}
↳ (erreur-type {etapel.bse['rappel']:.3f})")
print(f"statistique F de l'instrument : {etapel.tvalues['rappel'] ** 2:.0f}")
```

part de clients qui suivent le compte, sans rappel : 0.441 | avec rappel : 0.813
première étape : effet du rappel sur la probabilité de suivre = 0.372 (erreur-type 0.013)
statistique F de l'instrument : 874

Étape 3 — l'estimateur de Wald : le rapport de deux différences de moyennes.

```
moy = d.groupby("rappel")[["suit_compte", "depense"]].mean().round(3)
moy.index = ["sans rappel", "avec rappel"]
print(moy)
pi_hat = moy.loc["avec rappel", "suit_compte"] - moy.loc["sans rappel", "suit_compte"]
rho_hat = moy.loc["avec rappel", "depense"] - moy.loc["sans rappel", "depense"]
print(f"\npremière étape pi = {pi_hat:.3f}")
print(f"forme réduite rho = {rho_hat:.2f} €")
print(f"estimateur de Wald rho / pi = {rho_hat / pi_hat:.1f} €")
```

	suit_compte	depense
sans rappel	0.441	90.020
avec rappel	0.813	98.825

première étape pi = 0.372
forme réduite rho = 8.81 €
estimateur de Wald rho / pi = 23.7 €

Étape 4 — les moindres carrés en deux étapes, écrits trois fois (deux régressions, formule matricielle, estimateur IV), puis les bonnes erreurs-types et la vérification avec statsmodels.

```
n = len(d)
y = d["depense"].to_numpy()
X = np.column_stack([np.ones(n), d["suit_compte"], d["age"]]).astype(float)      # régresseurs, dont
↳ le traitement
Z = np.column_stack([np.ones(n), d["rappel"], d["age"]]).astype(float)          # instruments +
↳ covariables exogènes

# Étape 1 : traitement expliqué par l'instrument
pi = np.linalg.lstsq(Z, X[:, 1], rcond=None)[0]
T_chapeau = Z @ pi
```

```

X_chapeau = X.copy()
X_chapeau[:, 1] = T_chapeau
# Étape 2 : régression de Y sur le traitement « prédit »
beta = np.linalg.lstsq(X_chapeau, y, rcond=None)[0]

# Même résultat avec la formule matricielle  $(X' P_Z X)^{-1} X' P_Z y$ 
PzX = Z @ np.linalg.solve(Z.T @ Z, Z.T @ X)
beta_matrice = np.linalg.solve(PzX.T @ X, PzX.T @ y)
# Cas « juste identifié » (un instrument pour un traitement) :  $(Z' X)^{-1} Z' y$ 
beta_iv = np.linalg.solve(Z.T @ X, Z.T @ y)
print("2SLS en deux régressions :", beta.round(3))
print("2SLS, formule matricielle:", beta_matrice.round(3))
print("estimateur IV  $(Z'X)^{-1} Z'y$  :", beta_iv.round(3))

```

```

2SLS en deux régressions : [107.609  23.839 -0.773]
2SLS, formule matricielle: [107.609  23.839 -0.773]
estimateur IV  $(Z'X)^{-1} Z'y$  : [107.609  23.839 -0.773]

```

```

k = X.shape[1]
sigma2_correct = np.sum((y - X @ beta) ** 2) / (n - k) # résidus avec le VRAI traitement
sigma2_naif = np.sum((y - X_chapeau @ beta) ** 2) / (n - k) # résidus avec le traitement «
↳ « prédit » (faux)
var = np.linalg.inv(X_chapeau.T @ X_chapeau)
se_correct = np.sqrt(sigma2_correct * var[1, 1])
se_naif = np.sqrt(sigma2_naif * var[1, 1])
print(f"effet du suivi (2SLS) : {beta[1]:.2f} €")
print(f"erreur-type correcte : {se_correct:.2f} | erreur-type « naïve » de la seconde régression :
↳ {se_naif:.2f}")

# Vérification avec l'implémentation de statsmodels
res = IV2SLS(y, X, Z).fit()
print(f"statsmodels IV2SLS : coefficient = {res.params[1]:.2f} erreur-type = {res.bse[1]:.2f}")
print(f"IC 95 % : [{beta[1] - 1.96 * se_correct:.1f} ; {beta[1] + 1.96 * se_correct:.1f}"]

```

```

effet du suivi (2SLS) : 23.84 €
erreur-type correcte : 3.80 | erreur-type « naïve » de la seconde régression : 4.03
statsmodels IV2SLS : coefficient = 23.84 erreur-type = 3.80
IC 95 % : [16.4 ; 31.3]

```

Étape 5 — ce que l'on estime vraiment : l'effet pour les complaisants.

```

rng = np.random.default_rng(74)
n = 400_000
u = rng.normal(size=n) # passion
z = rng.integers(0, 2, n) # rappel aléatoire
v = rng.logistic(size=n) # même « bruit » individuel dans les deux
↳ mondes
d0 = (-0.3 + 0.8 * u + v > 0).astype(int) # suivrait-il SANS rappel ?
d1 = (-0.3 + 2.0 + 0.8 * u + v > 0).astype(int) # suivrait-il AVEC rappel ?
tau = 25 + 20 * u # effet individuel : plus fort chez les
↳ passionnés
T = np.where(z == 1, d1, d0)
Y = 80 + tau * T + 30 * u + rng.normal(0, 40, n)

complaisants = (d0 == 0) & (d1 == 1)
print(f"toujours-abonnés : {(d0 == 1).mean():.1%} | jamais-abonnés : {(d1 == 0).mean():.1%} |
↳ complaisants : {complaisants.mean():.1%}")
print(f"effet moyen sur tous les clients (ATE) : {tau.mean():.2f} €")
print(f"effet moyen sur les complaisants (LATE vrai) : {tau[complaisants].mean():.2f} €")
print(f"estimateur de Wald : {(Y[z == 1].mean() - Y[z == 0].mean()) / (T[z
↳ == 1].mean() - T[z == 0].mean()):.2f} €")
print(f"passion moyenne : complaisants {u[complaisants].mean():+.2f}, toujours-abonnés {u[d0 ==
↳ 1].mean():+.2f}, jamais-abonnés {u[d1 == 0].mean():+.2f}")

```

```
toujours-abonnés : 43.4% | jamais-abonnés : 18.1% | complaisants : 38.4%
effet moyen sur tous les clients (ATE)      : 24.99 €
effet moyen sur les complaisants (LATE vrai) : 21.64 €
estimateur de Wald                          : 21.75 €
passion moyenne : complaisants -0.17, toujours-abonnés +0.40, jamais-abonnés -0.60
```

Étape 6 — instrument faible, puis exclusion violée.

```
dw = iv(force=0.25)
et1 = smf.ols("suit_compte ~ rappel + age", dw).fit(cov_type="HC1")
Xw = np.column_stack([np.ones(len(dw)), dw["suit_compte"], dw["age"]]).astype(float)
Zw = np.column_stack([np.ones(len(dw)), dw["rappel"], dw["age"]]).astype(float)
rw = IV2SLS(dw["depense"].to_numpy(), Xw, Zw).fit()
print(f"part d'abonnés sans rappel : {dw.loc[dw.rappel == 0, 'suit_compte'].mean():.3f} avec rappel
↳ : {dw.loc[dw.rappel == 1, 'suit_compte'].mean():.3f}")
print(f"statistique F de l'instrument : {et1.tvalues['rappel'] ** 2:.1f}")
print(f"IV : effet = {rw.params[1]:.1f} € erreur-type = {rw.bse[1]:.1f} IC 95 % : [{rw.params[1] -
↳ 1.96 * rw.bse[1]:.0f} ; {rw.params[1] + 1.96 * rw.bse[1]:.0f}"])
```

```
part d'abonnés sans rappel : 0.441 avec rappel : 0.484
statistique F de l'instrument : 9.2
IV : effet = 14.9 € erreur-type = 33.8 IC 95 % : [-51 ; 81]
```

```
def estimation_iv_simple(df):
    """Estimateur IV (Wald) et son erreur-type robuste, sans covariable. Renvoie (estimation,
    ↳ erreur-type, statistique F)."""
    zc = df["rappel"].to_numpy(float) - df["rappel"].mean()
    xc = df["suit_compte"].to_numpy(float) - df["suit_compte"].mean()
    yc = df["depense"].to_numpy(float) - df["depense"].mean()
    b = (zc @ yc) / (zc @ xc)
    e = yc - b * xc
    se = np.sqrt(np.sum(zc ** 2 * e ** 2) / abs(zc @ xc))
    r = np.corrcoef(zc, xc)[0, 1]
    F = r ** 2 * (len(df) - 2) / (1 - r ** 2)
    return b, se, F

cas = [("forte (force 2,0, n = 5000)", 5000, 2.0), ("moyenne (force 0,25, n = 5000)", 5000, 0.25),
↳ ("faible (force 0,25, n = 1000)", 1000, 0.25)]
resultats = {}
for nom, n_mc, force in cas:
    est = np.array([estimation_iv_simple(iv(n=n_mc, seed=10_000 + k, force=force)) for k in
↳ range(500)])
    couverture = np.mean(np.abs(est[:, 0] - 25) <= 1.96 * est[:, 1])
    resultats[nom] = est[:, 0]
    q5, q25, q50, q75, q95 = np.percentile(est[:, 0], [5, 25, 50, 75, 95])
    print(f"{nom:32s} F médian {np.median(est[:, 2]):6.1f} | estimation médiane {q50:5.1f}, 5%-95% :
↳ [{q5:7.1f} ; {q95:6.1f}] | l'IC contient 25 dans {couverture:.0%} des cas")
```

```
forte (force 2,0, n = 5000)      F médian  936.7 | estimation médiane  25.1, 5%-95% : [ 19.3 ;
↳ 31.1] | l'IC contient 25 dans 97% des cas
moyenne (force 0,25, n = 5000)  F médian   14.7 | estimation médiane  25.5, 5%-95% : [ -24.7 ;
↳ 67.5] | l'IC contient 25 dans 98% des cas
faible (force 0,25, n = 1000)   F médian    3.0 | estimation médiane  30.8, 5%-95% : [ -185.0 ;
↳ 159.1] | l'IC contient 25 dans 99% des cas
```

```
dv = iv()
dv["depense"] = dv["depense"] + 15 * dv["rappel"] # effet direct du rappel sur la
↳ dépense : viole l'exclusion
Xv = np.column_stack([np.ones(len(dv)), dv["suit_compte"], dv["age"]]).astype(float)
Zv = np.column_stack([np.ones(len(dv)), dv["rappel"], dv["age"]]).astype(float)
rv = IV2SLS(dv["depense"].to_numpy(), Xv, Zv).fit()
print(f"IV avec exclusion violée : effet = {rv.params[1]:.1f} € (erreur-type {rv.bse[1]:.1f}, IC 95 %
↳ : [{rv.params[1] - 1.96 * rv.bse[1]:.0f} ; {rv.params[1] + 1.96 * rv.bse[1]:.0f}]) | vérité :
↳ 25")
```

```
print(f"biais théorique : effet direct / première étape = 15 / {pi_hat:.3f} = {15 / pi_hat:.1f} €")
```

```
IV avec exclusion violée : effet = 64.2 € (erreur-type 3.8, IC 95 % : [57 ; 72]) | vérité : 25
biais théorique : effet direct / première étape = 15 / 0.372 = 40.3 €
```

Lecture. La régression ordinaire donne 41,9 € (biaisée par la passion) ; Wald et 2SLS donnent 23,7 et 23,8 € (vérité : 25 €) avec une erreur-type de 3,8 € contre 1,4 €. Avec un instrument faible ($F = 9,2$), l'intervalle va de -51 à $+81$ € ; si le rappel contenait un bon de réduction (exclusion violée), l'estimation passe à 64,2 € pour une vérité de 25 €.

Pour aller plus loin. Faites varier `force` dans `iv(force=...)` et tracez la largeur de l'intervalle en fonction de la statistique F .

Exercices

Comment s'y prendre. Faites d'abord l'exercice **à la main**, puis vérifiez avec le code. Les exercices sont notés ★ (application directe), ★★ (il faut combiner deux idées) et ★★★ (démonstration ou analyse critique). Les corrigés suivent tous les énoncés.

```
clients = pd.read_csv("donnees/clients.csv")
panel = pd.read_csv("donnees/ch07-panel-villes.csv")
panel["vid"] = pd.factorize(panel["ville"])[0]
print(clients.shape, panel.shape)
```

```
(2000, 12) (480, 7)
```

Exercice 7.1 ★ — résultats potentiels (section 7.1.2 du livre)

Six clients ont les dépenses potentielles suivantes (en €) : A ($Y(0) = 60$, $Y(1) = 75$), B (40, 50), C (100, 105), D (80, 100), E (30, 40), F (90, 95). Les trois premiers ont reçu l'offre, les trois derniers non. (a) Calculez à la main l'ATE, l'ATT et l'ATU. (b) Quelle est la différence naïve des moyennes observées ? (c) Décomposez-la en ATT plus biais de sélection.

Exercice 7.2 ★ — sous-groupes (section 7.1.4 du livre)

Dans l'expérience de `clients.csv`, calculez l'effet de l'offre sur le rachat (`rachat_12m`) **dans chaque canal d'acquisition**, avec intervalle de confiance à 95 %. La gérante remarque : « l'offre marche presque cinq fois mieux sur Réseaux que sur le site ». (a) Testez cette différence. (b) Que devez-vous répondre à la gérante ?

Exercice 7.3 ★ — Simpson (section 7.1.6 du livre)

Une campagne d'e-mails a été envoyée à des clients en ville (1 000 clients dont 600 destinataires) et à la campagne (1 000 clients dont 200 destinataires). En ville, le rachat vaut 50 % chez les destinataires et 40 % chez les autres ; à la campagne, 30 % chez les destinataires et 20 % chez les autres. (a) Calculez le rachat global chez les destinataires et chez les non-destinataires. (b) Que dit la différence globale ? Que dit la différence par zone ? (c) Laquelle croire si la zone est une cause commune de la décision d'envoi et du rachat ? Et si la zone était une conséquence de l'e-mail ?

Exercice 7.4 ★★ — choisir l'ensemble d'ajustement (sections 7.1.7 à 7.1.9 du livre)

On simule : un facteur de confusion C ; un traitement T qui en dépend ; un médiateur M affecté par T ; un résultat $Y = 5T + 3M + 4C + \varepsilon$; et un effet commun $K = T + Y + \varepsilon'$. (a) Quel est l'effet **total** de T sur Y ? (b) Estimez l'effet par régression avec quatre ensembles d'ajustement : $\emptyset$, $\{C\}$, $\{C, M\}$, $\{C, K\}$. (c) Lequel est correct, et que mesurent les autres ?

Exercice 7.5 ★★ — stratification sur le score (section 7.2.4 du livre)

Les clients ont été classés en trois strates de score de propension. Strate A : 400 clients, probabilité d'offre 0,2, dépense moyenne 130 (avec offre) et 100 (sans). Strate B : 400 clients, probabilité 0,5, moyennes 150 et 115. Strate C : 200 clients, probabilité 0,8, moyennes 190 et 150. (a) Calculez la différence naïve. (b) Calculez l'ATE en pondérant les effets par strate. (c) Retrouvez-le par IPW.

Exercice 7.6 ★★ — chevauchement (section 7.2.2 du livre)

Reprenez l'étude observationnelle de 7.2 (ch07-observationnel.csv). (a) Restreignez l'analyse aux clients dont le score de propension est compris entre 0,1 et 0,9 : combien en reste-t-il ? (b) Réestimez l'ATE par IPW sur cet échantillon, et comparez avec l'estimation sur tous les clients. (c) Quelle quantité estime-t-on désormais ?

Exercice 7.7 ★★ — DiD à la main (section 7.3.1 du livre)

Chiffre d'affaires moyen par ville (en milliers de €) : villes traitées 120 avant, 150 après ; villes témoins 80 avant, 92 après. (a) Calculez la DiD en niveau. (b) Calculez-la en logarithme et interprétez en pourcentage. (c) Laquelle des deux hypothèses de tendances parallèles est la plus plausible si le chiffre d'affaires évolue en pourcentage ?

Exercice 7.8 ★★ — inférence avec peu de groupes (section 7.3.3 du livre)

Dans le panel, ne gardez que les 12 villes témoins, et attribuez **au hasard** à 4 d'entre elles une « fausse campagne » à partir de $t = 18$: il n'y a **aucun effet** à trouver. Répétez 300 fois (graine 81) et comptez à quelle fréquence la DiD avec erreurs-types groupées paraît « significative » à 5 %. (a) Avec le seuil normal $|t| > 1,96$. (b) Avec le seuil de Student à 11 degrés de liberté. (c) Que concluez-vous ?

Exercice 7.9 ★★ — instrument à la main (section 7.4.3 du livre)

Dans une étude, on observe : avec l'instrument ($Z = 1$), 60 % des clients suivent le compte et la dépense moyenne est de 52 € ; sans ($Z = 0$), 30 % le suivent et la dépense moyenne est de 46 €. (a) Calculez l'estimateur de Wald. (b) Quelle proportion de complaisants peut-on au mieux estimer ? (c) Si l'instrument avait en réalité un effet direct de 1,5 € sur la dépense, de combien l'estimation serait-elle faussée ?

Exercice 7.10 ★★★ — démonstration (section 7.4.3 du livre)

Soit Z binaire de probabilité $q = \mathbb{P}(Z = 1)$. (a) Montrez que $\text{Cov}(Z, Y) = q(1 - q)(\mathbb{E}[Y | Z = 1] - \mathbb{E}[Y | Z = 0])$. (b) Déduisez que $\text{Cov}(Z, Y) / \text{Cov}(Z, T)$ est égal au rapport de Wald. (c) Vérifiez-le numériquement sur ch07-iv.csv.

Exercice 7.11 ★★★ — médiation (section 7.1.7 du livre)

Reprenez la simulation de 7.1.7 (offre randomisée ; médiateur « code utilisé » ; motivation qui influence à la fois l'utilisation du code et la dépense). Cette fois, **supposez la motivation observée**. (a) Estimez l'effet direct de l'offre en ajustant sur le code **et** la motivation. (b) Déduisez l'effet indirect (par le code), et comparez avec la valeur vraie $30 \times \mathbb{P}(\text{code} \mid \text{offre})$. (c) Pourquoi l'ajustement sur le médiateur marche-t-il ici et pas en 7.1.7 ?

Exercice 7.12 ★★★ — critique d'une étude (section 7.4.7 du livre)

Une collègue veut estimer l'effet de « venir en boutique » (traitement) sur la dépense annuelle (résultat). Elle propose comme instrument la **distance** entre le domicile du client et la boutique. Discutez chacune des trois conditions d'un instrument, proposez une vérification ou un ajustement pour chacune, et dites ce que l'on estimerait si l'instrument était valide.

Corrigés**Corrigé 7.1**

- (a) Effets individuels : A +15, B +10, C +5, D +20, E +10, F +5. ATT = $(15 + 10 + 5)/3 = 10$; ATU = $(20 + 10 + 5)/3 = 11,67$; ATE = $65/6 = 10,83$. (b) Observé : traités (les Y(1) des trois premiers) 75, 50, 105, moyenne 76,67 ; non traités (les Y(0) des trois derniers) 80, 30, 90, moyenne 66,67 ; différence naïve = 10. (c) $\mathbb{E}[Y(0) \mid T = 1] = (60 + 40 + 100)/3 = 66,67$ et $\mathbb{E}[Y(0) \mid T = 0] = 66,67$: le biais de sélection est **nul** (par hasard dans ce petit exemple), donc la différence naïve égale l'ATT (10). Vérifions :

```
ex1 = pd.DataFrame({"client": ["A", "B", "C", "D", "E", "F"],
                    "y0": [60, 40, 100, 80, 30, 90], "y1": [75, 50, 105, 100, 40, 95], "offre": [1, 1,
                    ↪ 1, 0, 0, 0]})
ex1["effet"] = ex1["y1"] - ex1["y0"]
obs = np.where(ex1.offre == 1, ex1.y1, ex1.y0)
print(f"ATE = {ex1.effet.mean():.2f}  ATT = {ex1.effet[ex1.offre == 1].mean():.2f}  ATU =
    ↪ {ex1.effet[ex1.offre == 0].mean():.2f}")
print(f"différence naïve = {obs[ex1.offre == 1].mean() - obs[ex1.offre == 0].mean():.2f}")
print(f"biais de sélection = {ex1.y0[ex1.offre == 1].mean() - ex1.y0[ex1.offre == 0].mean():.2f}")
```

```
ATE = 10.83  ATT = 10.00  ATU = 11.67
différence naïve = 10.00
biais de sélection = 0.00
```

Moralité : un biais de sélection **nul** n'est pas impossible, mais on ne peut jamais le savoir sans connaître les Y(0) des traités ; la différence naïve n'est donc fiable que si l'on a une raison de croire que la sélection est ignorable.

Corrigé 7.2

- (a) Dans chaque canal, la différence de proportions et son intervalle de Wald :

```
lignes = []
for canal, g in clients.groupby("canal_acquisition"):
    t, u = g[g.offre_bienvenue == 1]["rachat_12m"], g[g.offre_bienvenue == 0]["rachat_12m"]
    diff = t.mean() - u.mean()
    se = np.sqrt(t.var() / len(t) + u.var() / len(u))
    lignes.append({"canal": canal, "clients": len(g), "effet": diff, "IC95 bas": diff - 1.96 * se,
    ↪ "IC95 haut": diff + 1.96 * se})
```

```
print(pd.DataFrame(lignes).round(3).to_string(index=False))

complet = smf.logit("rachat_12m ~ offre_bienvenue * C(canal_acquisition)", clients).fit(dis=0)
reduit = smf.logit("rachat_12m ~ offre_bienvenue + C(canal_acquisition)", clients).fit(dis=0)
lr = 2 * (complet.llf - reduit.llf)
print(f"\ntest de l'interaction offre x canal (rapport de vraisemblance, 2 ddl) : p =
↳ {stats.chi2.sf(lr, 2):.4f}")
```

canal	clients	effet	IC95 bas	IC95 haut
Boutique	504	0.116	0.029	0.202
Réseaux	816	0.196	0.129	0.263
Site	680	0.042	-0.033	0.117

```
test de l'interaction offre x canal (rapport de vraisemblance, 2 ddl) : p = 0.0105
```

L'effet est de +19,6 points sur Réseaux contre +4,2 sur le site, avec des intervalles larges ; le test d'interaction (rapport de vraisemblance, chapitre 2) donne $p \approx 0,01$. (b) Que répondre ? **Pru-dence**. Cette analyse par sous-groupe est *exploratoire* : si la gérante avait regardé cinq découpages différents (canal, ville, âge, année d'inscription...), le seuil de Bonferroni serait $0,05/5 = 0,01$, que $p = 0,0105$ ne franchit pas (volume I, section 3.5.5). Et l'expérience n'était **pas dimensionnée** pour détecter des différences entre sous-groupes (chaque canal n'a que quelques centaines de clients par bras). **Vérité révélée** : dans le simulateur, l'effet de l'offre est **le même** dans les trois canaux (12,3 à 12,4 points, comme on peut le vérifier par la même intégration qu'en 7.1.4) ; l'écart observé n'est qu'une fluctuation d'échantillonnage, de celles qui arrivent environ une fois sur cent. La bonne réponse : « c'est une piste, pas une conclusion ; on peut la **confirmer** par une nouvelle expérience prévue pour cela ».

Corrigé 7.3

- (a) Destinataires : $600 \times 0,5 + 200 \times 0,3 = 300 + 60 = 360$ rachats sur 800, soit 45 %. Non-destinataires : $400 \times 0,4 + 800 \times 0,2 = 160 + 160 = 320$ rachats sur 1200, soit 26,7 %. (b) Globalement l'e-mail est associé à **+18,3 points** ; par zone, à **+10 points** dans chaque zone. L'écart global **surestime** l'effet, car les destinataires sont surtout des citadins, qui rachètent davantage de toute façon (ici le sens est l'inverse du paradoxe de 7.1.6, mais le mécanisme est le même). (c) Si la zone est une cause commune de l'envoi et du rachat, il faut **ajuster** : l'effet standardisé est $0,5 \times 10 + 0,5 \times 10 = 10$ points. Si la zone était une **conséquence** de l'e-mail (l'e-mail pousserait les clients à déménager en ville !), ajuster sur elle serait une erreur : la différence globale deviendrait l'effet total.

```
ex3 = pd.DataFrame({"zone": ["ville", "ville", "campagne", "campagne"], "mail": [1, 0, 1, 0],
                    "n": [600, 400, 200, 800], "taux": [0.5, 0.4, 0.3, 0.2]})
ex3["rachats"] = ex3["n"] * ex3["taux"]
glob = ex3.groupby("mail")[["n", "rachats"]].sum()
print("taux global :", (glob.rachats / glob.n).round(3).to_dict())
par_zone = ex3.pivot(index="zone", columns="mail", values="taux")
poids = ex3.groupby("zone")["n"].sum() / ex3["n"].sum()
print("effet par zone :", (par_zone[1] - par_zone[0]).round(2).to_dict(), "| poids :",
↳ poids.round(2).to_dict())
print("effet standardisé :", round(((par_zone[1] - par_zone[0]) * poids).sum(), 3))
```

```
taux global : {0: 0.267, 1: 0.45}
effet par zone : {'campagne': 0.1, 'ville': 0.1} | poids : {'campagne': 0.5, 'ville': 0.5}
effet standardisé : 0.1
```

Corrigé 7.4

- (a) T agit directement (5) et par M (qui vaut $2T$ en moyenne, et pèse 3 dans Y) : effet total $= 5 + 3 \times 2 = 11$. (b) et (c) :

```
rng = np.random.default_rng(401)
n = 50_000
conf = rng.normal(size=n) # facteur de confusion (nommé « conf »
↳ pour ne pas masquer C() de patsy)
T = (conf + rng.normal(size=n) > 0).astype(int)
M = 2 * T + rng.normal(size=n)
Y = 5 * T + 3 * M + 4 * conf + rng.normal(size=n)
K = T + Y + rng.normal(size=n)
df4 = pd.DataFrame({"Y": Y, "T": T, "M": M, "conf": conf, "K": K})
for nom, f in [("", "Y ~ T"), ("{C}", "Y ~ T + conf"), ("{C, M}", "Y ~ T + conf + M"), ("{C, K}", "Y
↳ ~ T + conf + K")]:
    print(f"ensemble {nom:7s} : effet estimé de T = {smf.ols(f, df4).fit().params['T']:.2f}")

ensemble  : effet estimé de T = 15.50
ensemble {C} : effet estimé de T = 11.00
ensemble {C, M} : effet estimé de T = 5.01
ensemble {C, K} : effet estimé de T = 0.08
```

Seul $\{C\}$ donne l'effet **total** (11). Sans ajustement, on garde la **confusion** (15,5 : C pousse à la fois T et Y) ; avec $\{C, M\}$, on retire la voie par le médiateur et on mesure l'effet **direct** (5), ce qui peut être voulu mais ne répond pas à la question « que fait T ? » ; avec $\{C, K\}$, on conditionne sur un effet commun : l'estimation s'effondre vers 0, **en créant un biais**, alors que K semble une covariable « utile ».

Corrigé 7.5

- (a) Nombres de traités par strate : $0,2 \times 400 = 80$, $0,5 \times 400 = 200$, $0,8 \times 200 = 160$ (total 440) ; de témoins : 320, 200, 40 (total 560). Moyenne des traités $= (80 \times 130 + 200 \times 150 + 160 \times 190) / 440 = 160,9$; moyenne des témoins $= (320 \times 100 + 200 \times 115 + 40 \times 150) / 560 = 108,9$; différence naïve $= 52,0$. (b) Effets par strate : 30, 35, 40 ; ATE $= (400 \times 30 + 400 \times 35 + 200 \times 40) / 1000 = 34,0$. (c) IPW : poids $1/e$ pour les traités, $1/(1-e)$ pour les témoins. Dans chaque strate, le poids total des traités est n et celui des témoins aussi ; la pseudo-population a donc la même composition dans les deux groupes. L'ATE par IPW coïncide avec 34.

```
ex5 = pd.DataFrame({"strate": ["A", "B", "C"], "n": [400, 400, 200], "e": [0.2, 0.5, 0.8],
                    "m1": [130, 150, 190], "m0": [100, 115, 150]})
ex5["n1"], ex5["n0"] = ex5.n * ex5.e, ex5.n * (1 - ex5.e)
naif = (ex5.n1 * ex5.m1).sum() / ex5.n1.sum() - (ex5.n0 * ex5.m0).sum() / ex5.n0.sum()
ate = ((ex5.m1 - ex5.m0) * ex5.n).sum() / ex5.n.sum()
m1_ipw = (ex5.n1 * ex5.m1 / ex5.e).sum() / (ex5.n1 / ex5.e).sum()
m0_ipw = (ex5.n0 * ex5.m0 / (1 - ex5.e)).sum() / (ex5.n0 / (1 - ex5.e)).sum()
print(f"différence naïve = {naif:.1f} | ATE par strate = {ate:.1f} | ATE par IPW = {m1_ipw -
↳ m0_ipw:.1f}")
```

différence naïve = 52.0 | ATE par strate = 34.0 | ATE par IPW = 34.0

Corrigé 7.6

- (a)-(b) On refait l'estimation du score, puis on restreint :

```
obs = pd.read_csv("donnees/ch07-observationnel.csv")
verite = pd.read_csv("donnees/ch07-observationnel-verite.csv")
d6 = obs.merge(verite, on="id_client")
d6["ps"] = smf.logit("offre ~ age + C(canal) + engagement", d6).fit(dis=0).predict(d6)
```

```

ate_vrai = (d6.y1 - d6.y0).mean()

def ipw_ate(df):
    w = np.where(df.offre == 1, 1 / df.ps, 1 / (1 - df.ps))
    return (np.average(df.depense[df.offre == 1], weights=w[df.offre == 1])
            - np.average(df.depense[df.offre == 0], weights=w[df.offre == 0]))

restreint = d6[(d6.ps >= 0.1) & (d6.ps <= 0.9)]
ate_restreint_vrai = (restreint.y1 - restreint.y0).mean()
print(f"clients conservés : {len(restreint)} sur {len(d6)} ({len(restreint) / len(d6):.1%})")
print(f"IPW, tous les clients      : {ipw_ate(d6):.2f} (ATE vrai de la population : {ate_vrai:.2f})")
print(f"IPW, scores dans [0,1 ; 0,9] : {ipw_ate(restreint):.2f} (ATE vrai de ce sous-échantillon :
↪ {ate_restreint_vrai:.2f})")

clients conservés : 3778 sur 4000 (94.5%)
IPW, tous les clients      : 14.36 (ATE vrai de la population : 15.53)
IPW, scores dans [0,1 ; 0,9] : 14.73 (ATE vrai de ce sous-échantillon : 15.59)

```

- (c) En restreignant, on change la **population cible** : on estime l'effet pour les clients dont la probabilité d'offre n'est ni très faible ni très forte (la population de « chevauchement »), et non plus pour tous. Ici, la différence est **minime** : 94,5 % des clients restent, et l'ATE vrai du sous-échantillon (15,59) est presque celui de la population (15,53), parce que le chevauchement était déjà bon ; l'estimation passe de 14,36 à 14,73. Elle serait beaucoup plus importante si l'on devait écarter une grande partie des clients : l'ATE vrai du sous-échantillon pourrait alors différer sensiblement de celui de la population dès que l'effet varie avec le score (ici, il varie avec le canal, et le canal influence le score). Dans tous les cas, l'estimation répond à une question légèrement différente (celle de la population de chevauchement), à **dire explicitement** dans le rapport.

Corrigé 7.7

- (a) En niveau : $(150 - 120) - (92 - 80) = 30 - 12 = 18$ milliers de €. (b) En logarithme : $\ln(150/120) - \ln(92/80) = \ln 1,25 - \ln 1,15 = 0,2231 - 0,1398 = 0,0833$ (le code donne 0,0834, sans l'arrondi intermédiaire), soit un effet relatif d'environ +8,7 % ($e^{0,0833} - 1$). Les villes traitées auraient crû de 25 % ; les témoins de 15 % ; sans campagne, les traitées auraient crû de 15 % aussi, soit 138, ce qui donne un effet de $150 - 138 = 12$ milliers, et non 18. (c) Si le chiffre d'affaires évolue en **pourcentage**, les tendances parallèles sont plausibles **en logarithme** : l'effet de 18 milliers en niveau surestime l'effet réel (il suppose que la ville traitée, plus grosse, aurait crû de 12 milliers comme la petite, alors qu'une croissance de 15 % sur 120 fait 18 : la hausse « naturelle » des grandes villes est plus grande en niveau).

```

a_b, a_a, t_b, t_a = 80, 92, 120, 150
print(f"DiD en niveau : {(t_a - t_b) - (a_a - a_b)} | DiD en log : {np.log(t_a / t_b) - np.log(a_a /
↪ a_b):.4f} (soit {np.exp(np.log(t_a / t_b) - np.log(a_a / a_b)) - 1:+.1%})")
print(f"contrefactuel multiplicatif pour les villes traitées : {t_b * a_a / a_b:.0f} -> effet {t_a -
↪ t_b * a_a / a_b:.0f}")

```

```

DiD en niveau : 18 | DiD en log : 0.0834 (soit +8.7%)
contrefactuel multiplicatif pour les villes traitées : 138 -> effet 12

```

Corrigé 7.8

```

temoins = panel[panel["groupe_traite"] == 0]
villes = temoins["ville"].unique()
rng = np.random.default_rng(81)
t_stats = []

```

```

for _ in range(300):
    faux = set(rng.choice(villes, 4, replace=False))
    d8 = temoins.assign(camp=(temoins["ville"].isin(faux) & (temoins["t"] >= 18)).astype(int))
    m = smf.ols("np.log(commandes) ~ camp + C(ville) + C(t)", d8).fit(cov_type="cluster",
    ↪ cov_kws={"groups": d8["vid"]})
    t_stats.append(m.params["camp"] / m.bse["camp"])
t_stats = np.array(t_stats)
print(f"seuil normal 1,96          : {np.mean(np.abs(t_stats) > 1.96):.1%} de « découvertes »")
print(f"seuil de Student (11 ddl)   : {np.mean(np.abs(t_stats) > stats.t.ppf(0.975, 11)):.1%} de «
    ↪ découvertes »")

```

```

seuil normal 1,96          : 8.0% de « découvertes »
seuil de Student (11 ddl)   : 4.7% de « découvertes »

```

(a)-(b) Avec le seuil normal, le test « découvrir » un effet là où il n'y en a aucun dans environ 8 % des simulations au lieu des 5 % annoncés ; avec le seuil de Student à 11 degrés de liberté, le taux retombe à près de 5 % (4,7 %). (c) Quand le nombre de groupes (ici 12 villes, dont 4 « traitées ») est petit, les erreurs-types groupées sont **trop optimistes** et le seuil normal est trop laxiste : on obtient des faux positifs en excès. Remèdes : seuil de Student avec peu de degrés de liberté, bootstrap par groupes, ou tests de permutation (volume I, section 3.7) : c'est exactement ce que nous venons de faire, puisque la distribution de l'effet « fictif » est la distribution de référence d'un test de permutation.

Corrigé 7.9

- (a) $\hat{\beta} = \frac{52 - 46}{0,60 - 0,30} = \frac{6}{0,3} = 20$ €. (b) La part de **complaisants** est $\pi = 0,60 - 0,30 = 30\%$ (sous la monotonie) : l'effet de 20 € est l'effet **pour ces 30 %** de clients. (c) Un effet direct $\gamma_Z = 1,5$ ajouterait 1,5 à la forme réduite ($6 \rightarrow 7,5$) sans changer la première étape : l'estimation deviendrait $7,5/0,3 = 25$, soit un biais de $\gamma_Z/\pi = 1,5/0,3 = 5$ €.

```

pi, rho = 0.60 - 0.30, 52 - 46
print(f"Wald = {rho / pi:.1f} | avec effet direct de 1,5 : {(rho + 1.5) / pi:.1f} | biais = {1.5 /
    ↪ pi:.1f}")

```

```

Wald = 20.0 | avec effet direct de 1,5 : 25.0 | biais = 5.0

```

Corrigé 7.10

- (a) Comme $Z \in \{0, 1\}$: $\text{Cov}(Z, Y) = \mathbb{E}[ZY] - \mathbb{E}[Z] \mathbb{E}[Y]$. On a $\mathbb{E}[ZY] = q \mathbb{E}[Y | Z = 1]$ et $\mathbb{E}[Y] = q \mathbb{E}[Y | Z = 1] + (1 - q) \mathbb{E}[Y | Z = 0]$. Donc $\text{Cov}(Z, Y) = q \mu_1 - q(q \mu_1 + (1 - q) \mu_0) = q(1 - q)(\mu_1 - \mu_0)$ avec $\mu_z = \mathbb{E}[Y | Z = z]$. (b) La même formule vaut pour T à la place de Y ; dans le rapport, le facteur $q(1 - q)$ se **simplifie**, et il reste $\frac{\mathbb{E}[Y | Z = 1] - \mathbb{E}[Y | Z = 0]}{\mathbb{E}[T | Z = 1] - \mathbb{E}[T | Z = 0]}$, le rapport de Wald. (c) Vérification :

```

iv_data = pd.read_csv("donnees/ch07-iv.csv")
z, tt, yy = iv_data["rappel"].to_numpy(float), iv_data["suit_compte"].to_numpy(float),
    ↪ iv_data["depense"].to_numpy()
q = z.mean()
cov_zy, cov_zt = np.cov(z, yy, ddof=0)[0, 1], np.cov(z, tt, ddof=0)[0, 1]
rho = yy[z == 1].mean() - yy[z == 0].mean()
pi = tt[z == 1].mean() - tt[z == 0].mean()
print(f"Cov(Z,Y) = {cov_zy:.4f} vs q(1-q) x rho = {q * (1 - q) * rho:.4f}")
print(f"Cov(Z,Y)/Cov(Z,T) = {cov_zy / cov_zt:.4f} | rapport de Wald rho/pi = {rho / pi:.4f}")

```

$\text{Cov}(Z,Y) = 2.2000$ vs $q(1-q) \times \rho = 2.2000$
 $\text{Cov}(Z,Y)/\text{Cov}(Z,T) = 23.6563$ | rapport de Wald $\rho/\pi = 23.6563$

Corrigé 7.11

(a)-(b) On réutilise la simulation de 7.1.7 (même graine), mais avec la motivation dans la régression :

```
rng = np.random.default_rng(71)
n = 100_000
motivation = rng.normal(size=n)
offre = rng.integers(0, 2, n)
code_si_offre = rng.random(n) < 1 / (1 + np.exp(-(0.4 + 0.9 * motivation)))
code = offre * code_si_offre
depense = 100 + 8 * offre + 30 * code + 20 * motivation + rng.normal(0, 25, n)
d11 = pd.DataFrame({"depense": depense, "offre": offre, "code": code, "motivation": motivation})

total = smf.ols("depense ~ offre", d11).fit().params["offre"]
direct = smf.ols("depense ~ offre + code + motivation", d11).fit().params["offre"]
print(f"effet total estimé (offre seule) : {total:.2f}")
print(f"effet direct estimé (offre + code + motivation) : {direct:.2f} (vrai : 8)")
print(f"effet indirect = total - direct = {total - direct:.2f} (vrai : 30 x")
↳ f"code_si_offre.mean():.2f} = {30 * code_si_offre.mean():.2f}")
```

```
effet total estimé (offre seule) : 25.62
effet direct estimé (offre + code + motivation) : 8.27 (vrai : 8)
effet indirect = total - direct = 17.35 (vrai : 30 x 0.58 = 17.47)
```

- (c) En 7.1.7, la motivation était **cachée** : conditionner sur le médiateur ouvrait un chemin biaisé $\text{offre} \rightarrow \text{code} \leftarrow \text{motivation} \rightarrow \text{dépense}$ (le code est un effet commun de l'offre et de la motivation). Quand la motivation est **observée et incluse**, ce chemin est bloqué, et le coefficient de l'offre redevient l'effet **direct**. Morale : l'analyse de médiation exige de contrôler **tous** les facteurs de confusion entre le médiateur et le résultat, une hypothèse supplémentaire, plus exigeante que celle de l'effet total (qui n'en demande aucune dans une expérience randomisée).

Corrigé 7.12

Il n'y a pas de code ici : c'est une analyse critique. *Pertinence* : la distance doit réellement influencer la fréquentation (plausible : plus on habite loin, moins on vient) ; on le **vérifie** avec la première étape et sa statistique *F*. *Indépendance* : les clients **choisissent** où habiter, et ce choix dépend du revenu, du mode de vie, de l'âge : la distance n'est pas tirée au sort. Les citadins aisés habitent peut-être près du centre, où se trouve la boutique, et dépensent plus pour cette raison : l'instrument est corrélé à un facteur de confusion. On peut atténuer en **ajustant** sur les variables socio-économiques observées et sur la ville, mais jamais complètement. *Exclusion* : la distance ne doit affecter la dépense **que** via la fréquentation de la boutique ; or elle peut affecter la dépense par d'autres voies (un client éloigné achète davantage en ligne, ou fait de plus gros achats par déplacement). On peut chercher des **tests de plausibilité** (l'effet de la distance sur la dépense en ligne devrait être nul chez ceux qui ne viennent jamais en boutique) sans pouvoir prouver l'exclusion. *Ce qu'on estimerait si l'instrument était valide* : l'effet moyen de venir en boutique pour les **complaisants**, c'est-à-dire les clients dont la fréquentation **dépend** de la distance (ceux qui viendraient s'ils habitaient près et ne viendraient pas s'ils habitaient loin) : pas pour les habitués, ni pour ceux qui ne viendraient jamais. Pour une variable continue comme la distance, l'interprétation est une moyenne pondérée de tels effets, plus difficile à énoncer.

Chapitre 8 : Plans d'expériences — exercices et applications

Ce chapitre du cahier accompagne le chapitre 8 du livre (**⊕ Plans d'expériences**, entièrement optionnel). Vous y trouverez onze **applications guidées**, avec le code complet des simulations et des analyses que le livre ne fait que résumer, puis treize **exercices corrigés**. Il faut avoir lu les sections correspondantes ; les données sont celles du livre : des fichiers simulés `donnees/ch08-*.csv` (graines fixes, script `build/donnees_ch08.py`), dont nous connaissons la vérité. Les applications 8.2 et 8.3 se suivent (exécutez-les dans l'ordre) ; les autres sont indépendantes.

Préparation commune : les bibliothèques utilisées dans tout le chapitre.

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm
```

Applications

Application 8.1 — Randomiser, répéter, comparer avec un plan « un facteur à la fois »

Section 8.1 du livre. Trois résultats du livre sont des simulations : l'affectation naïve contre l'affectation aléatoire, la pseudo-réplication, et la comparaison d'un plan « un facteur à la fois » (OFAT) avec un plan factoriel. Refaites-les, puis changez les paramètres (écart-type du bruit, nombre de jours) pour voir comment les conclusions bougent.

Étape 1 : affectation naïve contre affectation aléatoire. Deux vitrines A et B sont *équivalentes* ; le week-end vend 60 € de plus. La version naïve met A du lundi au jeudi et B du vendredi au dimanche ; la version aléatoire tire 14 jours pour A et 14 pour B.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(81)
jours = np.arange(28)                # 4 semaines ; le jour 0 est un lundi
jour_semaine = jours % 7             # 0 = lundi ... 6 = dimanche
effet_jour = np.where(jour_semaine >= 4, 60, 0)  # vendredi, samedi, dimanche : +60 €

naif_A = jour_semaine <= 3           # A : lundi-jeudi (16 jours) ; B : vendredi-dimanche (12
↳ jours)
```

```

ecarts = {"naïve": [], "aléatoire": []}
rejets = {"naïve": 0, "aléatoire": 0}
n_sim = 2000
for _ in range(n_sim):
    y = 200 + effet_jour + rng.normal(0, 25, 28) # AUCUN effet de la vitrine : A = B
    lab_A = {"naïve": naif_A, "aléatoire": rng.permutation(np.r_[np.ones(14, bool), np.zeros(14,
    ↪ bool)])}
    for nom, A in lab_A.items():
        ecarts[nom].append(y[~A].mean() - y[A].mean())
        rejets[nom] += stats.ttest_ind(y[~A], y[A], equal_var=False).pvalue < 0.05

for nom in ecarts:
    e = np.array(ecarts[nom])
    print(f"affectation {nom:9s}: écart moyen B - A = {e.mean():6.1f} € ; écart-type = {e.std():5.1f}
    ↪ ; "
        f"'effet significatif' dans {100 * rejets[nom] / n_sim:5.1f} % des expériences")

```

```

affectation naïve      : écart moyen B - A =   60.3 € ; écart-type =    9.4 ; 'effet significatif' dans
↪ 100.0 % des expériences
affectation aléatoire: écart moyen B - A =   -0.1 € ; écart-type =   14.8 ; 'effet significatif' dans
↪  5.2 % des expériences

```

On attend un écart moyen B – A d'environ 60 € avec l'affectation naïve (et un « effet significatif » presque à chaque expérience), et d'environ 0 avec le tirage au sort (un test qui se trompe dans 5 % des cas, comme son niveau l'annonce). Vérifiez ces ordres de grandeur sur votre sortie.

Étape 2 : la pseudo-réplication. Deux vitrines sans aucune différence, 5 jours chacune, 40 clients par jour, avec un aléa propre à chaque jour. On compare le test « 200 clients contre 200 clients » (faux : les clients d'un même jour ne sont pas indépendants) au test « 5 jours contre 5 jours » (juste).

```

rng = np.random.default_rng(83)

faux, bon = 0, 0
n_sim = 3000
for _ in range(n_sim):
    aleas_jour = rng.normal(0, 25, (2, 5)) # un aléa par (vitrine, jour)
    y = 200 + aleas_jour[:, :, None] + rng.normal(0, 40, (2, 5, 40)) # forme : (vitrine, jour, client)
    # (a) on traite les 200 clients de chaque vitrine comme indépendants : FAUX
    faux += stats.ttest_ind(y[0].ravel(), y[1].ravel()).pvalue < 0.05
    # (b) on résume chaque journée par sa moyenne : l'unité expérimentale est le jour (5 contre 5)
    bon += stats.ttest_ind(y[0].mean(axis=1), y[1].mean(axis=1)).pvalue < 0.05

print(f"Test sur 200 clients contre 200 clients : 'effet' trouvé dans {100 * faux / n_sim:.1f} % des
↪ cas")
print(f"Test sur 5 jours contre 5 jours          : 'effet' trouvé dans {100 * bon / n_sim:.1f} % des
↪ cas")

```

```

Test sur 200 clients contre 200 clients : 'effet' trouvé dans 56.9 % des cas
Test sur 5 jours contre 5 jours          : 'effet' trouvé dans 5.1 % des cas

```

Le premier test voit un effet qui n'existe pas dans plus de la moitié des cas ; le second respecte son niveau de 5 %. *Pour aller plus loin* : passez à 20 jours par vitrine : que devient la puissance du test juste si l'on ajoute un vrai effet de 20 € ?

Étape 3 : un facteur à la fois contre plan factoriel. Le tableau des quatre moyennes sans bruit, puis la comparaison, à nombre d'essais égal, de la variance de l'estimation de l'effet du cadeau.

```

import pandas as pd

vrai = pd.DataFrame({"prix normal": [50, 55], "promo -10 %": [62, 60]}, index=["standard", "cadeau"])
print(vrai)

```

```

effet_A_prix_normal = vrai.loc["cadeau", "prix normal"] - vrai.loc["standard", "prix normal"]
effet_A_promo = vrai.loc["cadeau", "promo -10 %"] - vrai.loc["standard", "promo -10 %"]
effet_B_standard = vrai.loc["standard", "promo -10 %"] - vrai.loc["standard", "prix normal"]
effet_B_cadeau = vrai.loc["cadeau", "promo -10 %"] - vrai.loc["cadeau", "prix normal"]
print()
print("effet du cadeau au prix normal :", effet_A_prix_normal, "| avec la promo :", effet_A_promo)
print("effet de la promo en standard :", effet_B_standard, "| en cadeau :", effet_B_cadeau)

# effets « principaux » et interaction (définitions précisées en 8.3)
eff_A = (effet_A_prix_normal + effet_A_promo) / 2
eff_B = (effet_B_standard + effet_B_cadeau) / 2
eff_AB = (effet_A_promo - effet_A_prix_normal) / 2
print(f"effet principal du cadeau A = {eff_A}, de la promo B = {eff_B}, interaction AB = {eff_AB}")

```

	prix normal	promo -10 %
standard	50	62
cadeau	55	60

```

effet du cadeau au prix normal : 5 | avec la promo : -2
effet de la promo en standard : 12 | en cadeau : 5
effet principal du cadeau A = 1.5, de la promo B = 8.5, interaction AB = -3.5

```

```

rng = np.random.default_rng(84)
sigma, n_sim = 4.0, 100000
mu = {"std_normal": 50, "cadeau_normal": 55, "std_promo": 62, "cadeau_promo": 60}

# plan factoriel : 4 essais, un par case
y = {k: v + rng.normal(0, sigma, n_sim) for k, v in mu.items()}
A_fact = ((y["cadeau_normal"] + y["cadeau_promo"]) - (y["std_normal"] + y["std_promo"])) / 2

# OFAT : 4 essais aussi (départ répété 2 fois, puis A changé, puis B changé)
depart = (mu["std_normal"] + rng.normal(0, sigma, n_sim) + mu["std_promo"] + rng.normal(0, sigma,
↵ n_sim)) / 2
A_ofat = (mu["cadeau_normal"] + rng.normal(0, sigma, n_sim)) - depart

print(f"variance de l'estimation de A : factoriel = {A_fact.var():.1f} (théorie {sigma**2:.1f})")
print(f"                                OFAT      = {A_ofat.var():.1f} (théorie {1.5 * sigma**2:.1f})")
print(f"moyenne estimée de A          : factoriel = {A_fact.mean():.2f} (effet principal vrai 1.5) ; "
      f"OFAT = {A_ofat.mean():.2f} (effet de A au prix normal : 5)")

```

```

variance de l'estimation de A : factoriel = 16.1 (théorie 16.0)
                                OFAT      = 24.0 (théorie 24.0)
moyenne estimée de A          : factoriel = 1.48 (effet principal vrai 1.5) ; OFAT = 5.01 (effet de A
↵ au prix normal : 5)

```

La variance vaut $\sigma^2 = 16$ pour le plan factoriel et $1,5\sigma^2 = 24$ pour l'OFAT. Vérifiez aussi que l'OFAT estime l'effet du cadeau *au prix normal* (5), alors que le factoriel estime son effet *moyen* (1,5).

Application 8.2 — ANOVA à un facteur sur les vitrines

Sections 8.2.1 à 8.2.4. On analyse l'expérience des quatre agencements de vitrine (48 journées tirées au hasard, 12 par agencement) : de la décomposition de la variance au test F , puis le lien avec le test de Student et la régression.

Étape 1 : les données.

```

import numpy as np
import pandas as pd
from scipy import stats

```

```
df = pd.read_csv("donnees/ch08-vitrines.csv")
print(df.head(6).to_string(index=False))
print()
resume = df.groupby("agencement")["ventes"].agg(n="count", moyenne="mean", ecart_type="std").round(1)
print(resume.sort_values("moyenne").to_string())
print("\nmoyenne générale :", round(df["ventes"].mean(), 1))
```

```
jour  agencement  ventes
1  Par couleur   226.7
2   Vedette     249.3
3  Par thème    259.0
4   Vedette     272.2
5  Par thème    269.0
6  Par thème    243.2
```

```
          n  moyenne  ecart_type
agencement
Classique   12    196.2      38.1
Vedette    12    215.4      28.3
Par couleur 12    224.6      40.7
Par thème   12    237.0      25.5
```

moyenne générale : 218.3

Étape 2 : la décomposition sur un exemple de neuf nombres. Trois agencements, trois journées chacun. Les sommes de carrés doivent valoir $SS_B = 2450$ et $SS_W = 600$, et leur somme doit retomber sur la variabilité totale (3050).

```
y = np.array([[190, 200, 210], [205, 215, 225], [230, 240, 250]], dtype=float) # une ligne par groupe
k, n = y.shape
moy_gen, moy_groupes = y.mean(), y.mean(axis=1)

SS_entre = n * ((moy_groupes - moy_gen) ** 2).sum()
SS_dans = ((y - moy_groupes[:, None]) ** 2).sum()
SS_total = ((y - moy_gen) ** 2).sum()
print(f"SS entre = {SS_entre:.0f}, SS dans = {SS_dans:.0f}, somme = {SS_entre + SS_dans:.0f}, SS total
↪ = {SS_total:.0f}")

MS_entre, MS_dans = SS_entre / (k - 1), SS_dans / (k * n - k)
F = MS_entre / MS_dans
print(f"F = ({SS_entre:.0f}/{k - 1}) / ({SS_dans:.0f}/{k * n - k}) = {MS_entre:.0f} / {MS_dans:.0f} =
↪ {F:.2f}")
print(f"p-valeur = {stats.f.sf(F, k - 1, k * n - k):.4f} (scipy f_oneway :
↪ {stats.f_oneway(*y).pvalue:.4f})")
```

```
SS entre = 2450, SS dans = 600, somme = 3050, SS total = 3050
F = (2450/2) / (600/6) = 1225 / 100 = 12.25
p-valeur = 0.0076 (scipy f_oneway : 0.0076)
```

Étape 3 : le test F sur les données de la gérante, à la main puis avec statsmodels et scipy. Les trois calculs doivent coïncider : $F = 3,10$ avec 3 et 44 degrés de liberté, $p = 0,036$.

```
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

k, N = df["agencement"].nunique(), len(df)
g = df.groupby("agencement")["ventes"]
moy_gen = df["ventes"].mean()
SSB = (g.size() * (g.mean() - moy_gen) ** 2).sum()
SSW = ((df["ventes"] - g.transform("mean")) ** 2).sum()
SST = ((df["ventes"] - moy_gen) ** 2).sum()
MSB, MSW = SSB / (k - 1), SSW / (N - k)
F = MSB / MSW
```

```
print(f"SSB = {SSB:.0f}   SSW = {SSW:.0f}   SST = {SST:.0f}   (SSB + SSW = {SSB + SSW:.0f})")
print(f"MSB = {MSB:.0f}   MSW = {MSW:.0f}   F = {F:.3f}   p = {stats.f.sf(F, k - 1, N - k):.4f}   (ddl :
↳ {k - 1} et {N - k})")
print()
modele = ols("ventes ~ C(agencement)", data=df).fit()
print(anova_lm(modele).round(3).to_string())
print("\nscipy f_oneway :", stats.f_oneway(*[x.to_numpy() for _, x in g]))
```

```
SSB = 10595   SSW = 50168   SST = 60763   (SSB + SSW = 60763)
MSB = 3532   MSW = 1140   F = 3.097   p = 0.0363   (ddl : 3 et 44)
```

	df	sum_sq	mean_sq	F	PR(>F)
C(agencement)	3.0	10595.062	3531.687	3.097	0.036
Residual	44.0	50168.171	1140.186	NaN	NaN

```
scipy f_oneway : F_onewayResult(statistic=np.float64(3.097466867203288),
↳ pvalue=np.float64(0.03633773429167083))
```

Étape 4 : l'ANOVA est un test de Student, et une régression.

```
deux = df[df["agencement"].isin(["Classique", "Par thème"])]
t = stats.ttest_ind(deux.loc[deux["agencement"] == "Par thème", "ventes"],
                    deux.loc[deux["agencement"] == "Classique", "ventes"], equal_var=True)
F2 = stats.f_oneway(*[x["ventes"].to_numpy() for _, x in deux.groupby("agencement")])
print(f"Student : t = {t.statistic:.4f}, t² = {t.statistic ** 2:.4f}, p = {t.pvalue:.5f}")
print(f"ANOVA : F = {F2.statistic:.4f}, p = {F2.pvalue:.5f}")
```

```
Student : t = 3.0796, t² = 9.4837, p = 0.00548
ANOVA : F = 9.4837, p = 0.00548
```

```
print(modele.params.round(2).to_string())
print(f"\nF global de la régression = {modele.fvalue:.3f}, p = {modele.f_pvalue:.4f}")
print(f"R² = {modele.rsquared:.4f} et SSB/SST = {SSB / SST:.4f}")
```

Intercept	196.25
C(agencement)[T.Par couleur]	28.36
C(agencement)[T.Par thème]	40.72
C(agencement)[T.Vedette]	19.19

```
F global de la régression = 3.097, p = 0.0363
R² = 0.1744 et SSB/SST = 0.1744
```

Avec deux groupes, $F = t^2$. Avec quatre, le test F global de la régression sur indicatrices est celui de l'ANOVA, et $R^2 = SS_B / SS_T$.

Application 8.3 — Vérifier les hypothèses, comparer les groupes, mesurer l'effet

Sections 8.2.5 à 8.2.7. Cette application prolonge la précédente (mêmes variables).

Étape 1 : les hypothèses. Normalité des résidus (Shapiro-Wilk), égalité des variances (Levene, Bartlett), puis les deux solutions de repli (ANOVA de Welch, Kruskal-Wallis).

```
residus = modele.resid
groupes = [x["ventes"].to_numpy() for _, x in df.groupby("agencement")]

print("Shapiro-Wilk (normalité des résidus) : p =", round(stats.shapiro(residus).pvalue, 3))
print("Levene/Brown-Forsythe (variances égales) : p =", round(stats.levene(*groupes,
↳ center="median").pvalue, 3))
print("Bartlett (variances égales, sensible à la non-normalité) : p =",
↳ round(stats.bartlett(*groupes).pvalue, 3))
sd = df.groupby("agencement")["ventes"].std()
print(f"rapport plus grand / plus petit écart-type : {sd.max() / sd.min():.2f}")
```

```

Shapiro-Wilk (normalité des résidus) : p = 0.521
Levene/Brown-Forsythe (variances égales) : p = 0.276
Bartlett (variances égales, sensible à la non-normalité) : p = 0.369
rapport plus grand / plus petit écart-type : 1.60

```

```

from statsmodels.stats.oneway import anova_oneway

welch = anova_oneway(groupees, use_var="unequal", welch_correction=True)
print(f"ANOVA de Welch : F = {welch.statistic:.3f}, p = {welch.pvalue:.4f}")
print(f"Kruskal-Wallis : H = {stats.kruskal(*groupees).statistic:.3f}, p =
↳ {stats.kruskal(*groupees).pvalue:.4f}")

```

```

ANOVA de Welch : F = 3.259, p = 0.0390
Kruskal-Wallis : H = 7.009, p = 0.0716

```

Le Kruskal-Wallis ($p = 0,072$) est un peu moins tranché que l'ANOVA ($p = 0,036$) : la preuve est limite, et il faut le dire.

Étape 2 : les comparaisons deux à deux. On calcule le seuil de Tukey à la main, puis avec la bibliothèque ; on compare avec Bonferroni.

```

from statsmodels.stats.multicomp import pairwise_tukeyhsd

n_par = N // k
q = stats.studentized_range.ppf(0.95, k, N - k)
hsd = q * np.sqrt(MSW / n_par)
print(f"q(0.95 ; k={k}, ddl={N - k}) = {q:.3f} -> HSD = {q:.3f} x sqrt({MSW:.0f}/{n_par}) =
↳ {hsd:.1f} €")
lsd = stats.t.ppf(0.975, N - k) * np.sqrt(2 * MSW / n_par)
print(f"(seuil d'un test de Student non corrigé, pour une seule paire : {lsd:.1f} €)\n")

moy = g.mean()
paires = [(a, b) for i, a in enumerate(moy.index) for b in moy.index[i + 1:]]
for a, b in paires:
    ecart = moy[b] - moy[a]
    print(f"{b:12s} - {a:12s} : écart = {ecart:6.1f} € {'> HSD : significatif' if abs(ecart) > hsd
↳ else '<= HSD'}")

tukey = pairwise_tukeyhsd(df["ventes"], df["agencement"], alpha=0.05)
tab = pd.DataFrame(tukey._results_table.data[1:], columns=tukey._results_table.data[0])
print()
print(tab.round(3).to_string(index=False))

```

```

q(0.95 ; k=4, ddl=44) = 3.776 -> HSD = 3.776 x sqrt(1140/12) = 36.8 €
(seuil d'un test de Student non corrigé, pour une seule paire : 27.8 €)

```

```

Par couleur - Classique : écart = 28.4 € <= HSD
Par thème - Classique : écart = 40.7 € > HSD : significatif
Vedette - Classique : écart = 19.2 € <= HSD
Par thème - Par couleur : écart = 12.4 € <= HSD
Vedette - Par couleur : écart = -9.2 € <= HSD
Vedette - Par thème : écart = -21.5 € <= HSD

```

group1	group2	meandiff	p-adj	lower	upper	reject
Classique	Par couleur	28.358	0.183	-8.448	65.165	False
Classique	Par thème	40.725	0.025	3.918	77.532	True
Classique	Vedette	19.192	0.511	-17.615	55.998	False
Par couleur	Par thème	12.367	0.806	-24.440	49.173	False
Par couleur	Vedette	-9.167	0.910	-45.973	27.640	False
Par thème	Vedette	-21.533	0.410	-58.340	15.273	False

```
brut = {}
for a, b in paires:
    brut[(a, b)] = stats.ttest_ind(g.get_group(b), g.get_group(a), equal_var=True).pvalue
for (a, b), p in brut.items():
    print(f"{b:12s} - {a:12s} : p brute = {p:.4f} ; p Bonferroni (x6) = {min(1, 6 * p):.4f}")
```

```
Par couleur - Classique : p brute = 0.0919 ; p Bonferroni (x6) = 0.5514
Par thème - Classique : p brute = 0.0055 ; p Bonferroni (x6) = 0.0329
Vedette - Classique : p brute = 0.1752 ; p Bonferroni (x6) = 1.0000
Par thème - Par couleur : p brute = 0.3823 ; p Bonferroni (x6) = 1.0000
Vedette - Par couleur : p brute = 0.5288 ; p Bonferroni (x6) = 1.0000
Vedette - Par thème : p brute = 0.0632 ; p Bonferroni (x6) = 0.3794
```

Étape 3 : un contraste planifié et la taille d'effet. « Par thème » contre la moyenne des trois autres, puis η^2 , ω^2 et f de Cohen.

```
poids = pd.Series({"Classique": -1 / 3, "Par couleur": -1 / 3, "Par thème": 1.0, "Vedette": -1 / 3})
estim = (poids * moy).sum()
se = np.sqrt(MSW * (poids ** 2 / g.size()).sum())
t_c = estim / se
ic = (estim - stats.t.ppf(0.975, N - k) * se, estim + stats.t.ppf(0.975, N - k) * se)
print(f"contraste thème - moyenne des autres = {estim:.1f} € (écart-type {se:.1f})")
print(f"t = {t_c:.2f}, p = {2 * stats.t.sf(abs(t_c), N - k):.4f}, IC95 = [{ic[0]:.1f} ; {ic[1]:.1f}]"
```

```
contraste thème - moyenne des autres = 24.9 € (écart-type 11.3)
t = 2.21, p = 0.0324, IC95 = [2.2 ; 47.6]
```

```
eta2 = SSB / SST
omega2 = (SSB - (k - 1) * MSW) / (SST + MSW)
f_cohen = np.sqrt(eta2 / (1 - eta2))
print(f"eta² = {eta2:.3f} omega² = {omega2:.3f} f de Cohen = {f_cohen:.3f}")
```

```
eta² = 0.174 omega² = 0.116 f de Cohen = 0.460
```

Pour aller plus loin : quel est le plus petit nombre de jours par agencement pour lequel Tukey détecterait l'écart « Par thème – Classique » observé (40,7 €) ?

Application 8.4 — Les blocs : combien de bruit retire-t-on ?

Section 8.2.8. L'expérience refaite en 8 semaines (les blocs), chaque agencement testé une fois par semaine.

Étape 1 : la décomposition en blocs sur neuf nombres.

```
Y = np.array([[10, 12, 14], # semaine 1 : traitements A, B, C
              [20, 22, 27], # semaine 2
              [30, 31, 35]], dtype=float)
b_, k_ = Y.shape
mg = Y.mean()
SS_blocs = k_ * ((Y.mean(axis=1) - mg) ** 2).sum()
SS_trait = b_ * ((Y.mean(axis=0) - mg) ** 2).sum()
SS_err = ((Y - Y.mean(axis=1, keepdims=True) - Y.mean(axis=0, keepdims=True) + mg) ** 2).sum()
SS_tot = ((Y - mg) ** 2).sum()
print(f"blocs : {SS_blocs:.2f} traitements : {SS_trait:.2f} erreur : {SS_err:.2f}")
print(f"somme = {SS_blocs + SS_trait + SS_err:.2f} total = {SS_tot:.2f}")
print(f"ddl : blocs {b_ - 1}, traitements {k_ - 1}, erreur {(b_ - 1) * (k_ - 1)}")
```

```
blocs : 602.00 traitements : 44.67 erreur : 3.33
somme = 650.00 total = 650.00
ddl : blocs 2, traitements 2, erreur 4
```

Étape 2 : les vraies données, avec et sans blocs. Même mesure, mêmes 32 ventes, deux analyses.

```
bl = pd.read_csv("donnees/ch08-vitrines-blocs.csv")
print(bl.pivot(index="semaine", columns="agencement",
  ↪ values="ventes").round(0).astype(int).to_string())

sans_bloc = anova_lm(ols("ventes ~ C(agencement)", bl).fit())
avec_bloc = anova_lm(ols("ventes ~ C(agencement) + C(semaine)", bl).fit())
print("\n--- en ignorant les semaines (ANOVA à un facteur) ---")
print(sans_bloc.round(3).to_string())
print("\n--- en tenant compte des semaines (blocs) ---")
print(avec_bloc.round(3).to_string())
```

agencement	Classique	Par couleur	Par thème	Vedette
semaine				
1	202	191	236	208
2	130	167	192	162
3	226	256	276	219
4	142	114	164	134
5	234	261	272	231
6	184	220	230	195
7	132	153	191	174
8	185	210	217	203

```
--- en ignorant les semaines (ANOVA à un facteur) ---
              df      sum_sq  mean_sq      F  PR(>F)
C(agencement)  3.0    7932.631  2644.210   1.542   0.225
Residual      28.0   48007.159  1714.541    NaN    NaN
```

```
--- en tenant compte des semaines (blocs) ---
              df      sum_sq  mean_sq      F  PR(>F)
C(agencement)  3.0    7932.631  2644.210  15.228   0.0
C(semaine)     7.0   44360.712  6337.245  36.496   0.0
Residual      21.0    3646.447   173.640    NaN   NaN
```

Étape 3 : l'efficacité relative et le seuil de Tukey.

```
b, kk = bl["semaine"].nunique(), bl["agencement"].nunique()
MSE_bloc = avec_bloc.loc["Residual", "mean_sq"]
MS_blocs = avec_bloc.loc["C(semaine)", "mean_sq"]
ER = ((b - 1) * MS_blocs + b * (kk - 1) * MSE_bloc) / ((b * kk - 1) * MSE_bloc)
print(f"efficacité relative du plan en blocs = {ER:.1f}")

# comparaison des agencements à l'intérieur des blocs (Tukey avec le carré moyen de l'erreur du modèle
  ↪ à blocs)
moyennes = bl.groupby("agencement")["ventes"].mean()
hsd_bloc = stats.studentized_range.ppf(0.95, kk, (b - 1) * (kk - 1)) * np.sqrt(MSE_bloc / b)
print(f"HSD (blocs) = {hsd_bloc:.1f} €")
print((moyennes - moyennes["Classique"]).round(1).to_string())
```

```
efficacité relative du plan en blocs = 9.0
HSD (blocs) = 18.4 €
agencement
Classique      0.0
Par couleur    17.1
Par thème      42.9
Vedette        11.3
```

On doit trouver un carré moyen d'erreur divisé par dix (1 715 puis 174), un F de 1,5 puis 15,2, une efficacité relative d'environ 9 et un seuil de Tukey de 18 € (contre 37 € sans blocs).

Application 8.5 — Deux facteurs et leur interaction

Section 8.2.9. L'emballage (Kraft, Tissu, Coffret) et le canal (Site, Réseaux), 10 commandes par combinaison.

Étape 1 : le tableau des moyennes et la lecture de l'interaction.

```
ec = pd.read_csv("donnees/ch08-emballage-canal.csv")
table = ec.pivot_table(index="emballage", columns="canal", values="panier",
    ↪ aggfunc="mean").loc[["Kraft", "Tissu", "Coffret"]]
table["moyenne ligne"] = table.mean(axis=1)
table.loc["moyenne colonne"] = table.mean()
print(table.round(1).to_string())
```

canal	Réseaux	Site	moyenne ligne
emballage			
Kraft	43.5	48.0	45.7
Tissu	48.0	50.3	49.2
Coffret	69.8	60.3	65.0
moyenne colonne	53.8	52.9	53.3

Étape 2 : le tableau d'analyse de variance, et la vérification à la main d'une somme de carrés.

```
mod2 = ols("panier ~ C(emballage) * C(canal)", data=ec).fit()
aov2 = anova_lm(mod2)
print(aov2.round(3).to_string())

# vérification à la main de SS_emballage = r * b * somme des (moyenne de l'emballage - moyenne
    ↪ générale)2
r, nb_canaux = 10, 2
mg = ec["panier"].mean()
ss_emb = r * nb_canaux * ((ec.groupby("emballage")["panier"].mean() - mg) ** 2).sum()
print(f"\nSS emballage à la main = {ss_emb:.1f} (tableau : {aov2.loc['C(emballage)',
    ↪ 'sum_sq']:.1f})")
```

	df	sum_sq	mean_sq	F	PR(>F)
C(emballage)	2.0	4248.196	2124.098	39.749	0.000
C(canal)	1.0	12.513	12.513	0.234	0.630
C(emballage):C(canal)	2.0	569.809	284.905	5.331	0.008
Residual	54.0	2885.656	53.438	NaN	NaN

SS emballage à la main = 4248.2 (tableau : 4248.2)

Étape 3 : les effets simples, c'est-à-dire l'effet de l'emballage à chaque niveau du canal.

```
for canal in ["Site", "Réseaux"]:
    sous = ec[ec["canal"] == canal]
    a = anova_lm(ols("panier ~ C(emballage)", sous).fit())
    m = sous.groupby("emballage")["panier"].mean()
    print(f"{canal:10s}: F emballage = {a.loc['C(emballage)', 'F']:.1f}, p = {a.loc['C(emballage)',
        ↪ 'PR(>F)']:.4f} ; "
        f"gain Coffret - Kraft = {m['Coffret'] - m['Kraft']:.1f} €")
```

Site : F emballage = 7.1, p = 0.0032 ; gain Coffret - Kraft = 12.3 €
 Réseaux : F emballage = 42.1, p = 0.0000 ; gain Coffret - Kraft = 26.3 €

Interprétez : pourquoi l'effet principal du canal est-il presque nul alors que le canal modifie nettement l'effet du Coffret ?

Application 8.6 — Dimensionner une expérience : la puissance d'une ANOVA

Section 8.2.10. Moyennes vraies prévues : 200, 215, 240 et 205 €, écart-type 30 €, 12 jours par agencement.

Étape 1 : la puissance par la loi de Fisher non centrale, par statsmodels, et par simulation.

```
from statsmodels.stats.power import FTestAnovaPower

mu_vrai = np.array([200, 215, 240, 205])
sigma = 30
f_plan = np.sqrt(((mu_vrai - mu_vrai.mean()) ** 2).mean()) / sigma
print(f"effet de Cohen prévu : f = {f_plan:.3f}")

def puissance(n_par_groupe, f=f_plan, k=4, alpha=0.05):
    N = n_par_groupe * k
    ddl1, ddl2 = k - 1, N - k
    seuil = stats.f.ppf(1 - alpha, ddl1, ddl2)
    return stats.ncf.sf(seuil, ddl1, ddl2, N * f ** 2)    # lambda = N f^2

print(f"puissance avec 12 jours par agencement : {puissance(12):.3f}")
print(f"(statsmodels : {FTestAnovaPower().power(effect_size=f_plan, nobs=48, alpha=0.05,
↪ k_groups=4):.3f})")

# vérification par simulation : on rejoue l'expérience 5000 fois
rng = np.random.default_rng(85)
rejets = 0
for _ in range(5000):
    echantillons = [m + rng.normal(0, sigma, 12) for m in mu_vrai]
    rejets += stats.f.oneway(*echantillons).pvalue < 0.05
print(f"fréquence de rejet simulée : {rejets / 5000:.3f}")

effet de Cohen prévu : f = 0.514
puissance avec 12 jours par agencement : 0.826
(statsmodels : 0.826)
fréquence de rejet simulée : 0.820
```

Étape 2 : et si l'effet était deux fois plus petit ? Puissance de la même expérience, puis nombre de jours nécessaire pour atteindre 80 %.

```
f_moitie = f_plan / 2
print(f"effet moitié moindre : f = {f_moitie:.3f} -> puissance avec 12 jours par agencement =
↪ {puissance(12, f_moitie):.3f}")

def jours_pour_80(f):
    for n_g in range(3, 400):
        if puissance(n_g, f) >= 0.80:
            return n_g

for f, nom in [(f_plan, "effet prévu"), (f_moitie, "effet moitié moindre")]:
    n_req = jours_pour_80(f)
    print(f"{nom:22s} (f = {f:.3f}) : {n_req} jours par agencement pour 80 % de puissance, soit {4 *
↪ n_req} jours au total")
print("statsmodels (N total, avant arrondi à des groupes égaux) :",
      int(np.ceil(FTestAnovaPower().solve_power(effect_size=f_plan, power=0.8, alpha=0.05,
↪ k_groups=4))))

effet moitié moindre : f = 0.257 -> puissance avec 12 jours par agencement = 0.266
effet prévu (f = 0.514) : 12 jours par agencement pour 80 % de puissance, soit 48 jours au
↪ total
effet moitié moindre (f = 0.257) : 43 jours par agencement pour 80 % de puissance, soit 172 jours au
↪ total
statsmodels (N total, avant arrondi à des groupes égaux) : 46
```

On doit trouver 83 % de puissance prévue, 27 % si l'effet est moitié moindre, et 43 jours par agencement (172 au total) pour retrouver 80 %.

Application 8.7 — Un plan factoriel 2^3 de A à Z

Section 8.3. Trois facteurs (emballage A, prix B, relance C), huit combinaisons, deux répétitions : 16 semaines.

Étape 1 : le tableau des signes et l'orthogonalité.

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

def plan_2k(k):
    """Plan factoriel 2^k en ordre standard : A varie le plus vite, puis B, etc."""
    return np.array([(1 if (i >> j) & 1 else -1) for j in range(k)] for i in range(2 ** k))

X = plan_2k(3)
S = pd.DataFrame(X, columns=["A", "B", "C"])
S["AB"], S["AC"], S["BC"] = S.A * S.B, S.A * S.C, S.B * S.C
S["ABC"] = S.A * S.B * S.C
S.insert(0, "I", 1)
S.index = ["(1)", "a", "b", "ab", "c", "ac", "bc", "abc"] # notation classique : on nomme les
↳ lettres « hautes »
print(S.to_string())
print("\nS' S (produit scalaire de chaque paire de colonnes) :")
print((S.T @ S).to_string())
```

	I	A	B	C	AB	AC	BC	ABC
(1)	1	-1	-1	-1	1	1	1	-1
a	1	1	-1	-1	-1	-1	1	1
b	1	-1	1	-1	-1	1	-1	1
ab	1	1	1	-1	1	-1	-1	-1
c	1	-1	-1	1	1	-1	-1	1
ac	1	1	-1	1	-1	1	-1	-1
bc	1	-1	1	1	-1	-1	1	-1
abc	1	1	1	1	1	1	1	1

S' S (produit scalaire de chaque paire de colonnes) :

	I	A	B	C	AB	AC	BC	ABC
I	8	0	0	0	0	0	0	0
A	0	8	0	0	0	0	0	0
B	0	0	8	0	0	0	0	0
C	0	0	0	8	0	0	0	0
AB	0	0	0	0	8	0	0	0
AC	0	0	0	0	0	8	0	0
BC	0	0	0	0	0	0	8	0
ABC	0	0	0	0	0	0	0	8

Étape 2 : les données et les effets à la main.

```
f3 = pd.read_csv("donnees/ch08-factoriel-2p3.csv")
print("Les 5 premières semaines dans l'ordre d'exécution (tiré au hasard) :")
print(f3.head(5)[["ordre", "A", "B", "C", "commandes"]].to_string(index=False))

cel = f3.pivot_table(index=["C", "B", "A"], columns="replicat", values="commandes")
cel["moyenne"] = cel.mean(axis=1)
cel.index = S.index # même ordre standard que le tableau des
↳ signes
print("\nRésultats par combinaison :")
```

```
print(cel.round(2).to_string())
```

Les 5 premières semaines dans l'ordre d'exécution (tiré au hasard) :

ordre	A	B	C	commandes
1	1	-1	-1	57.1
2	1	1	-1	56.9
3	1	1	1	78.1
4	-1	-1	1	45.8
5	1	1	-1	68.4

Résultats par combinaison :

replicat	1	2	moyenne
(1)	49.9	45.9	47.90
a	57.1	65.3	61.20
b	69.4	64.6	67.00
ab	68.4	56.9	62.65
c	45.8	51.5	48.65
ac	62.1	59.8	60.95
bc	70.1	64.4	67.25
abc	73.2	78.1	75.65

```
ybar = cel["moyenne"].to_numpy()
effets = {c: (S[c].to_numpy() * ybar).sum() / 4 for c in ["A", "B", "C", "AB", "AC", "BC", "ABC"]}
print("Détail pour A : moyenne des cellules A haut =", round(ybar[S.A.to_numpy() == 1].mean(), 2),
      "; A bas =", round(ybar[S.A.to_numpy() == -1].mean(), 2))
print(pd.Series(effets).round(2).to_string())
print("\nmoyenne générale :", round(ybar.mean(), 2))
```

Détail pour A : moyenne des cellules A haut = 65.11 ; A bas = 57.7

A	7.41
B	13.46
C	3.44
AB	-5.39
AC	2.94
BC	3.19
ABC	3.44

moyenne générale : 61.41

Étape 3 : l'algorithme de Yates. On doit retrouver exactement les mêmes effets.

```
def yates(y):
    cols, col = [np.array(y, float)], np.array(y, float)
    for _ in range(int(np.log2(len(y)))):
        col = np.concatenate([col[0::2] + col[1::2], col[1::2] - col[0::2]])
        cols.append(col)
    return np.array(cols).T

Y = yates(ybar)
noms = ["I", "A", "B", "AB", "C", "AC", "BC", "ABC"]
tab = pd.DataFrame(Y, index=noms, columns=["moyennes", "étape 1", "étape 2", "contraste (étape 3)"])
tab["effet = contraste / 4"] = tab["contraste (étape 3)"] / 4
tab.loc["I", "effet = contraste / 4"] = tab.loc["I", "contraste (étape 3)"] / 8 # la moyenne se
↳ divise par 8
print(tab.round(2).to_string())
print("\nIdentique aux effets calculés plus haut :", all(np.isclose(tab.loc[c, "effet = contraste /
↳ 4"], effets[c]) for c in effets))
```

	moyennes	étape 1	étape 2	contraste (étape 3)	effet = contraste / 4
I	47.90	109.10	238.75	491.25	61.41
A	61.20	129.65	252.50	29.65	7.41
B	67.00	109.60	8.95	53.85	13.46
AB	62.65	142.90	20.70	-21.55	-5.39

C	48.65	13.30	20.55	13.75	3.44
AC	60.95	-4.35	33.30	11.75	2.94
BC	67.25	12.30	-17.65	12.75	3.19
ABC	75.65	8.40	-3.90	13.75	3.44

Identique aux effets calculés plus haut : True

Étape 4 : tout par régression, avec l'erreur pure des répétitions ; puis la décomposition de la variance.

```
mod = smf.ols("commandes ~ A * B * C", data=f3).fit() # A*B*C = tous les effets principaux et
↳ interactions
N = len(f3)
res = pd.DataFrame({"effet": 2 * mod.params, "ET": 2 * mod.bse, "t": mod.tvalues, "p":
↳ mod.pvalues}).drop("Intercept")
print(res.round(3).to_string())

# erreur pure « à la main » : écarts des deux répétitions à la moyenne de leur combinaison
moy_cel = f3.groupby(["A", "B", "C"])["commandes"].transform("mean")
s2 = ((f3["commandes"] - moy_cel) ** 2).sum() / (N - 8)
print(f"erreur pure : s² = {s2:.2f} (ddl = {N - 8}) ; statsmodels : {mod.mse_resid:.2f}")
print(f"écart-type d'un effet = 2 s / sqrt(N) = 2 x {np.sqrt(s2):.2f} / {np.sqrt(N):.0f} = {2 *
↳ np.sqrt(s2 / N):.3f}")
```

	effet	ET	t	p
A	7.413	2.28	3.251	0.012
B	13.463	2.28	5.904	0.000
A:B	-5.388	2.28	-2.363	0.046
C	3.437	2.28	1.507	0.170
A:C	2.937	2.28	1.288	0.234
B:C	3.188	2.28	1.398	0.200
A:B:C	3.438	2.28	1.507	0.170

erreur pure : $s^2 = 20.80$ (ddl = 8) ; statsmodels : 20.80
écart-type d'un effet = $2 s / \sqrt{N} = 2 \times 4.56 / 4 = 2.280$

```
aov = anova_lm(mod)
ss = (N * (res["effet"] / 2) ** 2).round(1)
verif = pd.DataFrame({"SS (tableau d'ANOVA)": aov["sum_sq"].drop("Residual").round(1).to_numpy(),
↳ "SS = N x (effet/2)²": ss.to_numpy()}, index=res.index)
print(verif.to_string())
print(f"SS erreur pure = {aov.loc['Residual', 'sum_sq']:.1f} ; SS total = {(f3['commandes'] -
↳ f3['commandes'].mean()) ** 2).sum():.1f} ; "
f"somme des SS des effets + erreur = {aov['sum_sq'].sum():.1f}")
```

	SS (tableau d'ANOVA)	SS = N x (effet/2)²
A	219.8	219.8
B	725.0	725.0
A:B	116.1	116.1
C	47.3	47.3
A:C	34.5	34.5
B:C	40.6	40.6
A:B:C	47.3	47.3
SS erreur pure = 166.4 ; SS total = 1396.9 ; somme des SS des effets + erreur = 1396.9		

Étape 5 : le modèle réduit et la prédiction des quatre réglages.

```
red = smf.ols("commandes ~ A + B + A:B", data=f3).fit()
print(red.params.round(3).to_string())
print(f"\nR² complet = {mod.rsquared:.3f} ; R² réduit = {red.rsquared:.3f} ; s (erreur) =
↳ {np.sqrt(red.mse_resid):.2f} (ddl {int(red.df_resid)})\n")

grille = pd.DataFrame([(a, b) for b in (-1, 1) for a in (-1, 1)], columns=["A", "B"])
```

```

pred = red.get_prediction(grille).summary_frame(alpha=0.05)
grille["prédiction"] = pred["mean"].round(1)
grille["IC95 de la moyenne"] = [f"[{lo:.1f} ; {hi:.1f}]" for lo, hi in zip(pred["mean_ci_lower"],
↪ pred["mean_ci_upper"])]
print(grille.to_string(index=False))

```

```

Intercept    61.406
A             3.706
B             6.731
A:B          -2.694

```

R^2 complet = 0.881 ; R^2 réduit = 0.759 ; s (erreur) = 5.29 (ddl 12)

```

A B  prédiction IC95 de la moyenne
-1 -1      48.3    [42.5 ; 54.0]
 1 -1      61.1    [55.3 ; 66.8]
-1  1      67.1    [61.4 ; 72.9]
 1  1      69.2    [63.4 ; 74.9]

```

Étape 6 : la puissance du plan pour un effet de 4 commandes.

```

sigma, delta_effet = 3.5, 4.0
lignes = []
for r in (1, 2, 3, 4, 6):
    N_r = 8 * r
    ddl = N_r - 8 if r > 1 else None
    if ddl is None:                                     # pas de répétition : pas d'erreur pure ; voir 8.3.6
        lignes.append((r, N_r, None, 2 * sigma / np.sqrt(N_r), None))
        continue
    se = 2 * sigma / np.sqrt(N_r)
    seuil = stats.t.ppf(0.975, ddl)
    puissance = stats.nct.sf(seuil, ddl, delta_effet / se) + stats.nct.cdf(-seuil, ddl, delta_effet /
↪ se)
    lignes.append((r, N_r, ddl, se, puissance))
tab = pd.DataFrame(lignes, columns=["répétitions r", "essais N = 8 r", "ddl erreur pure", "ET d'un
↪ effet", "puissance (effet = 4)"])
print(tab.round(3).to_string(index=False))

```

répétitions r	essais N = 8 r	ddl erreur pure	ET d'un effet	puissance (effet = 4)
1	8	NaN	2.475	NaN
2	16	8.0	1.750	0.520
3	24	16.0	1.429	0.748
4	32	24.0	1.237	0.873
6	48	40.0	1.010	0.971

Application 8.8 — Un plan 2^4 sans répétition : Lenth et le diagramme demi-normal

Section 8.3.6. Quatre facteurs (on ajoute D, un message personnalisé), seize essais, aucune répétition : quinze effets, zéro degré de liberté pour l'erreur.

Étape 1 : la méthode de Lenth. On estime l'écart-type du bruit à partir des effets eux-mêmes (le principe de parcimonie), puis on déclare « actifs » ceux qui dépassent la marge simultanée.

```

g = pd.read_csv("donnees/ch08-factoriel-2p4.csv")
mod4 = smf.ols("commandes ~ A * B * C * D", data=g).fit()
eff = (2 * mod4.params).drop("Intercept")
eff.index = [c.replace(":", "") for c in eff.index]
print("Nombre d'effets estimés :", len(eff), "; ddl de l'erreur :", int(mod4.df_resid), "(aucun :
↪ modèle saturé)\n")

absolu = eff.abs().sort_values()
m = len(absolu)

```

```

s0 = 1.5 * np.median(absolu)
pse = 1.5 * np.median(absolu[absolu < 2.5 * s0])
d = m / 3
ME = stats.t.ppf(0.975, d) * pse
SME = stats.t.ppf((1 + 0.95 ** (1 / m)) / 2, d) * pse
print(f"s0 = {s0:.3f}   PSE = {pse:.3f}   ME = {ME:.2f}   SME = {SME:.2f}\n")
tab = pd.DataFrame({"effet": eff[absolu.index].round(2), "|effet|": absolu.round(2), "actif (|effet|
↳ > SME)": absolu > SME})
print(tab.iloc[::1].head(8).to_string())

```

Nombre d'effets estimés : 15 ; ddl de l'erreur : 0 (aucun : modèle saturé)

s0 = 1.012 PSE = 0.900 ME = 2.31 SME = 4.70

	effet	effet	actif (effet > SME)
B	12.20	12.20	True
A	9.53	9.53	True
AB	-5.68	5.68	True
D	5.20	5.20	True
ABC	1.30	1.30	False
ACD	-1.10	1.10	False
ABD	-0.92	0.92	False
CD	-0.67	0.67	False

Étape 2 : confirmer sur les seuls effets actifs.

```

red4 = smf.ols("commandes ~ A + B + A:B + D", data=g).fit()
t4 = pd.DataFrame({"effet": 2 * red4.params, "ET": 2 * red4.bse, "p": red4.pvalues}).drop("Intercept")
print(t4.round(3).to_string())
print(f"s = {np.sqrt(red4.mse_resid):.2f} avec {int(red4.df_resid)} ddl ; R² = {red4.rsquared:.3f}")

```

	effet	ET	p
A	9.525	0.727	0.0
B	12.200	0.727	0.0
A:B	-5.675	0.727	0.0
D	5.200	0.727	0.0

s = 1.45 avec 11 ddl ; R² = 0.981

Quatre effets actifs doivent ressortir : B, A, AB et D.

Application 8.9 — Plans fractionnaires : alias, résolution, repliement

Section 8.4.1 à 8.4.4.

Étape 1 : combien d'effets, de quel ordre ?

```

import numpy as np
import pandas as pd
from math import comb
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

lignes = []
for k in (3, 4, 5, 7):
    ordres = [comb(k, j) for j in range(1, k + 1)]
    lignes.append({"facteurs k": k, "essais 2^k": 2 ** k, "principaux": ordres[0], "interactions
↳ d'ordre 2": ordres[1],
                  "d'ordre 3": ordres[2], "d'ordre 4 et plus": sum(ordres[3:]), "total d'effets":
                  ↳ sum(ordres)})
print(pd.DataFrame(lignes).to_string(index=False))

```

facteurs k ↪ d'effets	essais 2^k	principaux	interactions d'ordre 2	d'ordre 3	d'ordre 4 et plus	total
3	8	3	3	1	0	7
4	16	4	6	4	1	15
5	32	5	10	10	6	31
7	128	7	21	35	64	127

Étape 2 : la demi-fraction 2^{4-1} avec $D = ABC$. On prend, parmi les 16 essais de l'application précédente, les 8 qui vérifient $ABCD = +1$, et on compare à ce que donne le plan complet : chaque contraste estime la *somme* de deux effets confondus.

```
def plan_2k(k):
    return np.array([(1 if (i >> j) & 1 else -1) for j in range(k)] for i in range(2 ** k))

xa, xb, xc = plan_2k(3).T
xd = xa * xb * xc                                # générateur : D = ABC
print("A = BCD :", np.array_equal(xa, xb * xc * xd), "| AB = CD :", np.array_equal(xa * xb, xc * xd))

g4 = pd.read_csv("donnees/ch08-factoriel-2p4.csv")
complet = (2 * smf.ols("commandes ~ A * B * C * D", data=g4).fit().params).drop("Intercept")
demi = g4[g4.A * g4.B * g4.C * g4.D == 1]          # la demi-fraction I = ABCD : 8 essais sur 16
eff_demi = (2 * smf.ols("commandes ~ A * B * C", data=demi).fit().params).drop("Intercept")
alias = {"A": ["A", "B:C:D"], "B": ["B", "A:C:D"], "C": ["C", "A:B:D"], "A:B": ["A:B", "C:D"],
         "A:C": ["A:C", "B:D"], "B:C": ["B:C", "A:D"], "A:B:C": ["A:B:C", "D"]}
tab = pd.DataFrame({"demi-fraction": eff_demi.round(2),
                    "somme des alias (plan complet)": [round(sum(complet[e] for e in alias[i]), 2) for
                    ↪ i in eff_demi.index]})
print(tab)
```

```
A = BCD : True | AB = CD : True
      demi-fraction  somme des alias (plan complet)
A           9.00           9.00
B          11.10          11.10
A:B         -6.35         -6.35
C           -1.20          -1.20
A:C          -0.45          -0.45
B:C           0.15           0.15
A:B:C         6.50           6.50
```

Les deux colonnes doivent être égales au centième près.

Étape 3 : les classes de confusion du plan 2^{5-2} ($D = AB, E = AC$), calculées par programme : le produit de deux ensembles de lettres est leur différence symétrique.

```
def produit(u, v):
    return "".join(sorted(set(u) ^ set(v)))          # lettres communes éliminées

mots = ["ABD", "ACE", "BCDE"]
tous = ["A", "B", "C", "D", "E", "AB", "AC", "AD", "AE", "BC", "BD", "BE", "CD", "CE", "DE", "ABC",
        ↪ "ABD", "ABE",
        "ACD", "ACE", "ADE", "BCD", "BCE", "BDE", "CDE", "ABCD", "ABCE", "ABDE", "ACDE", "BCDE",
        ↪ "ABCDE"]
classes = {}
for e in tous:
    classe = tuple(sorted({e} | {produit(e, w) for w in mots}, key=lambda s: (len(s), s)))
    classes[classe] = classes.get(classe, 0) + 1
print(len(tous), "effets possibles, répartis en", len(classes), "classes de confusion (8 essais = 7
        ↪ contrastes + la moyenne) :\n")
for c in sorted(classes, key=lambda c: (len(c[0]), c[0])):
    print("  " + " = ".join(x or "I" for x in c))
```

31 effets possibles, répartis en 8 classes de confusion (8 essais = 7 contrastes + la moyenne) :

```

I = ABD = ACE = BCDE
A = BD = CE = ABCDE
B = AD = CDE = ABCE
C = AE = BDE = ABCD
D = AB = BCE = ACDE
E = AC = BCD = ABDE
BC = DE = ABE = ACD
BE = CD = ABC = ADE

```

Étape 4 : le piège de la résolution III, puis le repliement. On suppose $A = +6$, $B = +4$, $AB = +8$ et *aucun* effet de D. Les 8 essais attribuent à tort un effet d'environ 8 à D (= AB) ; le repliement (tous les signes inversés) le corrige.

```

rng = np.random.default_rng(87)
xa, xb, xc = plan_2k(3).T
xd, xe = xa * xb, xa * xc          # générateurs D = AB et E = AC
vrai = lambda a, b, c, d, e: 50 + 3 * a + 2 * b + 4 * a * b          # effets : A = 6, B = 4, AB = 8,
↪ le reste 0
y = vrai(xa, xb, xc, xd, xe) + rng.normal(0, 0.8, 8)
contrastes = {"A": xa, "B": xb, "C": xc, "D (= AB)": xd, "E (= AC)": xe, "BC (= DE)": xb * xc, "ABC (=
↪ CD = BE)": xa * xb * xc}
print(pd.Series({nom: (col * y).sum() / 4 for nom, col in contrastes.items()})).round(2))

```

```

A          5.95
B          4.76
C         -0.56
D (= AB)   8.17
E (= AC)  -0.23
BC (= DE)  0.09
ABC (= CD = BE) -0.51
dtype: float64

```

```

X1 = plan_2k(3)
bloc1 = pd.DataFrame({"A": X1[:, 0], "B": X1[:, 1], "C": X1[:, 2]})
bloc2 = -bloc1          # repliement : tous les signes inversés
plan = pd.concat([bloc1.assign(bloc=1), bloc2.assign(bloc=2)], ignore_index=True)
plan["D"] = np.where(plan.bloc == 1, plan.A * plan.B, -plan.A * plan.B)
plan["E"] = np.where(plan.bloc == 1, plan.A * plan.C, -plan.A * plan.C)
# vérification : sur les 16 essais, seule la relation BCDE = + 1 subsiste
print("BCDE = +1 sur tous les essais :", bool((plan.B * plan.C * plan.D * plan.E == 1).all()),
      "| ABD = +1 :", bool((plan.A * plan.B * plan.D == 1).all()), "| ACE = +1 :", bool((plan.A *
↪ plan.C * plan.E == 1).all()))

plan["y"] = vrai(plan.A, plan.B, plan.C, plan.D, plan.E) + rng.normal(0, 0.8, 16)
formule = "y ~ A + B + C + D + E + A:B + A:C + A:D + A:E + B:C + B:D + B:E"          # BC=DE, BD=CE,
↪ BE=CD restent confondus deux à deux
fo = smf.ols(formule, data=plan).fit()
t = pd.DataFrame({"effet": 2 * fo.params, "p": fo.pvalues}).drop("Intercept")
print()
print(t.round(3).to_string())
print(f"\n(ddl de l'erreur : {int(fo.df_resid)} ; s = {np.sqrt(fo.mse_resid):.2f})")

```

BCDE = +1 sur tous les essais : True | ABD = +1 : False | ACE = +1 : False

	effet	p
A	6.157	0.001
B	3.624	0.004
C	0.337	0.490
D	-0.098	0.834
E	-0.107	0.820
A:B	8.082	0.000
A:C	0.686	0.209
A:D	-0.381	0.441

```
A:E -0.223 0.640
B:C -0.329 0.500
B:D -0.281 0.560
B:E -0.780 0.167
```

```
(ddl de l'erreur : 3 ; s = 0.86)
```

L'effet de D tombe près de zéro et l'interaction A:B réapparaît à environ 8 : les deux, confondus dans le plan à 8 essais, sont maintenant séparés.

Application 8.10 — Surface de réponse : trouver le meilleur réglage du four

Section 8.4.5 à 8.4.7. Température (autour de 1 000 °C) et durée (autour de 6 h) ; réponse : pourcentage de pièces sans défaut. Plan composite centré à 13 essais : 4 points factoriels, 4 axiaux, 5 répétitions du centre.

Étape 1 : y a-t-il de la courbure ?

```
cc = pd.read_csv("donnees/ch08-ccd-cuisson.csv")
print(cc.sort_values("ordre").to_string(index=False))
```

ordre	x1	x2	temperature_C	duree_h	reussite
1	0.0000	0.0000	1000.0	6.00	85.5
2	-1.0000	-1.0000	960.0	5.00	73.8
3	1.0000	-1.0000	1040.0	5.00	73.0
4	-1.0000	1.0000	960.0	7.00	69.9
5	0.0000	0.0000	1000.0	6.00	84.0
6	-1.4142	0.0000	943.4	6.00	71.4
7	1.0000	1.0000	1040.0	7.00	79.8
8	1.4142	0.0000	1056.6	6.00	81.2
9	0.0000	0.0000	1000.0	6.00	83.0
10	0.0000	0.0000	1000.0	6.00	84.0
11	0.0000	1.4142	1000.0	7.41	72.1
12	0.0000	-1.4142	1000.0	4.59	70.9
13	0.0000	0.0000	1000.0	6.00	82.5

```
centre = cc[(cc.x1 == 0) & (cc.x2 == 0)]
fact = cc[(cc.x1.abs() == 1) & (cc.x2.abs() == 1)]
ss_pe = ((centre.reussite - centre.reussite.mean()) ** 2).sum()
df_pe = len(centre) - 1
s_pe = np.sqrt(ss_pe / df_pe)
courbure = fact.reussite.mean() - centre.reussite.mean()
se_c = s_pe * np.sqrt(1 / len(fact) + 1 / len(centre))
t_c = courbure / se_c
print(f"moyenne factoriels = {fact.reussite.mean():.2f} ; moyenne au centre =
  ↳ {centre.reussite.mean():.2f} ; écart = {courbure:.2f}")
print(f"erreur pure : s = {s_pe:.2f} ({df_pe} ddl) ; t = {t_c:.2f} ; p = {2 * stats.t.sf(abs(t_c),
  ↳ df_pe):.4f}")
b1 = (fact.x1 * fact.reussite).sum() / len(fact)
b2 = (fact.x2 * fact.reussite).sum() / len(fact)
print(f"pentes du premier ordre (points factoriels) : b1 = {b1:.2f}, b2 = {b2:.2f}")
```

```
moyenne factoriels = 74.12 ; moyenne au centre = 83.80 ; écart = -9.67
erreur pure : s = 1.15 (4 ddl) ; t = -12.53 ; p = 0.0002
pentes du premier ordre (points factoriels) : b1 = 2.27, b2 = 0.72
```

Étape 2 : le modèle quadratique et le défaut d'ajustement.

```
q = smf.ols("reussite ~ x1 + x2 + I(x1**2) + I(x2**2) + x1:x2", data=cc).fit()
noms = {"Intercept": "β0", "x1": "β1 (x1)", "x2": "β2 (x2)", "I(x1 ** 2)": "β11 (x1²)", "I(x2 ** 2)":
  ↳ "β22 (x2²)", "x1:x2": "β12 (x1 x2)"}
coef = pd.DataFrame({"estimation": q.params, "ET": q.bse, "t": q.tvalues, "p":
  ↳ q.pvalues}).rename(index=noms)
```

```
print(coef.round(3).to_string())
print(f"\nR² = {q.rsquared:.3f} ; R² ajusté = {q.rsquared_adj:.3f} ; s = {np.sqrt(q.mse_resid):.2f}
↪ ({int(q.df_resid)} ddl)")
```

	estimation	ET	t	p
β_0	83.800	0.490	170.916	0.000
β_1 (x1)	2.870	0.388	7.404	0.000
β_2 (x2)	0.575	0.388	1.482	0.182
β_{11} (x1 ²)	-3.694	0.416	-8.886	0.000
β_{22} (x2 ²)	-6.094	0.416	-14.660	0.000
β_{12} (x1 x2)	2.675	0.548	4.880	0.002

R² = 0.980 ; R² ajusté = 0.966 ; s = 1.10 (7 ddl)

```
ss_def, df_def = q.ssr - ss_pe, q.df_resid - df_pe
F_def = (ss_def / df_def) / (ss_pe / df_pe)
print(f"SS résidu = {q.ssr:.2f} = SS erreur pure {ss_pe:.2f} ({df_pe} ddl) + SS défaut d'ajustement
↪ {ss_def:.2f} ({int(df_def)} ddl)")
print(f"F de défaut d'ajustement = {F_def:.2f} ; p = {stats.f.sf(F_def, df_def, df_pe):.3f}")
```

SS résidu = 8.41 = SS erreur pure 5.30 (4 ddl) + SS défaut d'ajustement 3.11 (3 ddl)

F de défaut d'ajustement = 0.78 ; p = 0.562

Étape 3 : le point stationnaire, sa nature, sa précision.

```
pq = q.params
bq = np.array([pq["x1"], pq["x2"]])
Bq = np.array([[pq["I(x1 ** 2)"], pq["x1:x2"] / 2], [pq["x1:x2"] / 2, pq["I(x2 ** 2)"]]])
xs = -0.5 * np.linalg.solve(Bq, bq)
ys = pq["Intercept"] + 0.5 * bq @ xs
lam, vecs = np.linalg.eigh(Bq)
print(f"point stationnaire (codé) : x1 = {xs[0]:.3f}, x2 = {xs[1]:.3f} ; distance au centre =
↪ {np.linalg.norm(xs):.2f}")
print(f"en unités naturelles : température = {1000 + 40 * xs[0]:.0f} °C, durée = {6 + xs[1]:.2f} h ;
↪ réponse prédite : {ys:.2f} %")
print(f"valeurs propres de B : {lam.round(2)} -> {'maximum' if (lam < 0).all() else 'minimum' if (lam
↪ > 0).all() else 'col'}")
pred = q.get_prediction(pd.DataFrame({"x1": [xs[0]], "x2": [xs[1]]}).summary_frame(alpha=0.05)
print(f"IC95 de la réponse moyenne : [{pred['mean_ci_lower'][0]:.1f} ;
↪ {pred['mean_ci_upper'][0]:.1f}]")
print(f"intervalle de prédiction à 95 % : [{pred['obs_ci_lower'][0]:.1f} ;
↪ {pred['obs_ci_upper'][0]:.1f}]")
```

point stationnaire (codé) : x1 = 0.441, x2 = 0.144 ; distance au centre = 0.46

en unités naturelles : température = 1018 °C, durée = 6.14 h ; réponse prédite : 84.47 %

valeurs propres de B : [-6.69 -3.1] -> maximum

IC95 de la réponse moyenne : [83.3 ; 85.6]

intervalle de prédiction à 95 % : [81.6 ; 87.3]

Étape 4 : confirmer par de nouveaux essais. Trois fournées simulées au sommet estimé, avec le vrai processus (que nous connaissons).

```
def vrai_taux(x1, x2):
    return 84 + 3 * x1 + 1 * x2 - 4 * x1 ** 2 - 6 * x2 ** 2 + 2.5 * x1 * x2

rng = np.random.default_rng(88)
confirm = vrai_taux(xs[0], xs[1]) + rng.normal(0, 0.9, 3)
dans = ((confirm >= pred["obs_ci_lower"][0]) & (confirm <= pred["obs_ci_upper"][0])).all()
print("trois fournées de confirmation :", confirm.round(1), "| toutes dans l'intervalle de prédiction
↪ :", bool(dans))
```

trois fournées de confirmation : [84.1 84.2 83.8] | toutes dans l'intervalle de prédiction : True

Le sommet attendu est à environ 1 018 °C et 6,14 h, avec 84,5 % de pièces sans défaut ; le vrai optimum est à 1 017 °C et 6,17 h (84,7 %).

Application 8.11 — Un plan D-optimal par algorithme d'échange

Section 8.4.8 (optionnelle). On cherche les 9 essais qui maximisent $\det(X^T X)$ pour le modèle quadratique, d'abord sur le carré, puis sous une contrainte qui rend un coin impossible.

Étape 1 : le critère et l'algorithme d'échange.

```
def info(points):
    x1, x2 = points[:, 0], points[:, 1]
    Xm = np.column_stack([np.ones(len(points)), x1, x2, x1 ** 2, x2 ** 2, x1 * x2])    # modèle
    ↪ quadratique
    return np.linalg.det(Xm.T @ Xm)

def echange(candidats, n, rng, depart=20):
    meilleur = (-1, None)
    for _ in range(depart):
        idx = list(rng.choice(len(candidats), n, replace=True))
        change = True
        while change:
            change = False
            for pos in range(n):
                best_j, best_d = idx[pos], info(candidats[idx])
                for j in range(len(candidats)):
                    essai = idx.copy()
                    essai[pos] = j
                    d = info(candidats[essai])
                    if d > best_d * (1 + 1e-9):
                        best_j, best_d, change = j, d, True
            idx[pos] = best_j
        d = info(candidats[idx])
        if d > meilleur[0]:
            meilleur = (d, idx.copy())
    return meilleur
```

Étape 2 : sur le carré, puis avec la contrainte $x_1 + x_2 \leq 1$.

```
rng = np.random.default_rng(89)
grille = np.array([(a, b) for a in np.linspace(-1, 1, 5) for b in np.linspace(-1, 1, 5)])
d_opt, idx = echange(grille, 9, rng)
factoriel_3x3 = np.array([(a, b) for a in (-1, 0, 1) for b in (-1, 0, 1)])
au_hasard = np.median([info(grille[rng.choice(25, 9, replace=False)]) for _ in range(2000)])
print("9 essais choisis parmi la grille 5 x 5 :")
print(pd.Series([tuple(grille[i]) for i in idx]).value_counts().sort_index().to_string())
print(f"\ndet(X'X) : D-optimal = {d_opt:.0f} ; factoriel 3x3 = {info(factoriel_3x3):.0f} ; "
      f"9 points au hasard (médiane sur 2000 tirages) = {au_hasard:.0f}")

# avec une contrainte : la combinaison « tout haut » (x1 + x2 > 1) est impossible (le four ne le
↪ permet pas)
possible = grille[grille.sum(axis=1) <= 1.0]
d_c, idx_c = echange(possible, 9, rng)
print(f"\nSous la contrainte x1 + x2 <= 1 ({len(possible)} candidats), plan D-optimal à 9 essais (det
↪ = {d_c:.0f}) :")
print(pd.Series([tuple(possible[i]) for i in idx_c]).value_counts().sort_index().to_string())
```

```
9 essais choisis parmi la grille 5 x 5 :
(-1.0, -1.0)    1
(-1.0, 0.0)     1
```

```
(-1.0, 1.0)    1
(0.0, -1.0)    1
(0.0, 0.0)     1
(0.0, 1.0)     1
(1.0, -1.0)    1
(1.0, 0.0)     1
(1.0, 1.0)     1
```

$\det(X'X)$: D-optimal = 5184 ; factoriel 3×3 = 5184 ; 9 points au hasard (médiane sur 2000 tirages) = 94

Sous la contrainte $x_1 + x_2 \leq 1$ (22 candidats), plan D-optimal à 9 essais ($\det = 1920$) :

```
(-1.0, -1.0)    2
(-1.0, 0.0)     1
(-1.0, 1.0)     1
(0.0, -1.0)     1
(0.0, 0.0)      1
(0.0, 1.0)      1
(1.0, -1.0)     1
(1.0, 0.0)      1
```

Sur le carré, l'algorithme retrouve la grille 3×3 classique (même déterminant, 5 184) ; avec la contrainte, il propose la grille privée du coin impossible, avec le coin opposé répété.

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 2 à 4 se font avec les mêmes trois groupes ; 7, 9 et 10 se font entièrement à la main.

Exercice 8.1 ★ — Concevoir une expérience (section 8.1 du livre)

La gérante veut comparer deux présentations de la page d'accueil de son site, A et B. Elle propose : « affichage A la semaine prochaine, affichage B la semaine suivante, et je compare les commandes ». (a) Quelle est l'unité expérimentale ? (b) Citez deux raisons pour lesquelles la comparaison sera biaisée. (c) Proposez un plan qui applique les trois principes de Fisher (randomisation, répétition, blocage).

Exercice 8.2 ★ — ANOVA à la main (section 8.2 du livre)

Trois fournisseurs de papier d'emballage, quatre lots chacun ; on mesure la résistance à la déchirure (en newtons) : Fournisseur 1 : 52, 48, 50, 50 ; Fournisseur 2 : 56, 58, 54, 56 ; Fournisseur 3 : 62, 60, 64, 62. Calculez les moyennes, SS_B , SS_W , les carrés moyens et la statistique F . Combien de degrés de liberté ? Que conclure ?

Exercice 8.3 ★★ — Taille d'effet (section 8.2 du livre)

Avec les données de l'exercice 2, calculez η^2 et ω^2 . Pourquoi $\omega^2 < \eta^2$? Vérifiez ensuite que la régression sur indicatrices redonne le même F et le même R^2 .

Exercice 8.4 ★★ — Comparaisons multiples (section 8.2 du livre)

Toujours avec les données de l'exercice 2, calculez le seuil HSD de Tukey (utilisez $q_{0,95;3,9} \approx 3,95$) et dites quelles paires de fournisseurs diffèrent. Pourquoi ne pas simplement faire trois tests de Student à 5 % ?

Exercice 8.5 ★★ — Blocs (section 8.2 du livre)

Quatre traitements sont testés dans trois blocs (trois semaines) ; ventes en dizaines de € :

	traitement 1	traitement 2	traitement 3	traitement 4
semaine 1	10	14	12	16
semaine 2	20	25	22	27
semaine 3	31	33	32	36

- (a) Calculez SS_{blocs} , $SS_{\text{traitements}}$, SS_E et leurs degrés de liberté. (b) Comparez le F des traitements avec et sans les blocs. (c) Que s'est-il passé ?

Exercice 8.6 ★★ — Interaction (section 8.3 du livre)

On teste deux facteurs, A et B, à deux niveaux ; les moyennes de réponse sont $\bar{y}_{A-B-} = 20$, $\bar{y}_{A+B-} = 30$, $\bar{y}_{A-B+} = 25$, $\bar{y}_{A+B+} = 15$. Calculez l'effet principal de A, celui de B et l'interaction AB. L'effet principal de A est nul : A n'a-t-il donc aucune influence ? Quel est le meilleur réglage ?

Exercice 8.7 ★★ — Algorithme de Yates (section 8.3 du livre)

Plan 2^3 sans répétition, essais en ordre standard : $(1) = 10$, $a = 14$, $b = 12$, $ab = 20$, $c = 11$, $ac = 15$, $bc = 13$, $abc = 21$. Calculez les sept effets et la moyenne par l'algorithme de Yates, puis par les contrastes. Quels effets sont non nuls ?

Exercice 8.8 ★★★ — Puissance d'un plan factoriel (section 8.3 du livre)

Dans un plan 2^3 répliqué r fois, l'écart-type du bruit est $\sigma = 3$. On veut détecter un effet de $\Delta = 4$ avec une puissance d'au moins 80 %, au seuil de 5 %. (a) Donnez l'écart-type d'un effet en fonction de r . (b) Estimez à la main un ordre de grandeur de r avec l'approximation normale. (c) Calculez la valeur exacte avec la loi de Student non centrale.

Exercice 8.9 ★★ — Confusion (section 8.4 du livre)

On veut un plan 2^{4-1} (8 essais, 4 facteurs). (a) Avec le générateur $D = ABC$, donnez la relation de définition, la résolution et les alias de AB. (b) Avec $D = AB$, mêmes questions. (c) Lequel choisir et pourquoi ?

Exercice 8.10 ★★★ — Un plan 2^{6-2} (section 8.4 du livre)

On construit 6 facteurs en 16 essais avec les générateurs $E = ABC$ et $F = BCD$. (a) Donnez la relation de définition complète. (b) Quelle est la résolution ? (c) Donnez les alias de A et de AB. (d) Peut-on séparer les interactions AB et CE ?

Exercice 8.11 ★★ — Surface de réponse (section 8.4 du livre)

Un modèle du second ordre ajusté sur un plan composite centré à 2 facteurs ($\alpha = \sqrt{2}$) est $\hat{y} = 70 + 4x_1 + 2x_2 - 3x_1^2 - x_2^2 + x_1x_2$. (a) Trouvez le point stationnaire et la réponse prédite. (b) Est-ce un maximum ? (c) Peut-on faire confiance à ce point ?

Exercice 8.12 ★★★ — Simuler l'effet des blocs (section 8.2 du livre)

On compare deux traitements avec 12 unités. L'effet vrai du traitement est de +15, l'écart-type du bruit de 10 et l'écart-type entre blocs (les semaines) de 30. Simulez 3 000 expériences et comparez la puissance (a) d'un plan **complètement randomisé** (12 unités issues de 12 blocs différents, 6 par traitement, analysées par un test de Student à deux échantillons) et (b) d'un plan **en blocs** (6 blocs, chacun contenant une unité de chaque traitement, analysé par un test de Student apparié).

Exercice 8.13 ★★ — Méthode de Lenth (section 8.3 du livre)

Un plan 2^3 non répliqué donne les sept effets 12,0 ; -1,0 ; 0,5 ; 8,0 ; -0,8 ; 0,4 ; 0,6. Calculez s_0 et le PSE de Lenth, et dites quels effets sont actifs (marge d'erreur $ME = t_{0,975;7/3} \times \text{PSE}$).

Corrigés**Corrigé 8.1**

- (a) L'unité expérimentale est **la semaine** (tous les visiteurs d'une même semaine voient le même affichage) : elle n'a ici que **deux unités**, une par traitement. (b) D'abord, l'affichage est **confondu avec la semaine** : si la semaine 1 contient une fête ou une promotion, on attribuera à A ce qui vient de la période ; ensuite, il n'y a **aucune répétition** : on ne peut pas estimer le bruit entre semaines, donc aucun test n'est possible. (c) Plan en **blocs** : prendre par exemple 8 semaines ; **dans chaque semaine** (le bloc), afficher A trois ou quatre jours et B les autres, avec un **tirage au sort** des jours ; si l'on peut, afficher A et B en même temps à des visiteurs tirés au hasard (l'unité devient alors le visiteur, plus fine, et la répétition immédiate). On compare A et B **à l'intérieur de chaque semaine** (test apparié), ce qui élimine l'effet de semaine, et on dispose de plusieurs répétitions pour estimer le bruit.

Corrigé 8.2

Moyennes : 50, 56, 62 ; moyenne générale 56. $SS_B = 4[(50 - 56)^2 + 0 + (62 - 56)^2] = 4 \times 72 = 288$. Dans chaque groupe, la somme des carrés des écarts vaut $4 + 4 + 0 + 0 = 8$ (groupe 1), $0 + 4 + 4 + 0 = 8$ (groupe 2), $0 + 4 + 4 + 0 = 8$ (groupe 3) : $SS_W = 24$. Degrés de liberté : $k - 1 = 2$ et $N - k = 12 - 3 = 9$. $MS_B = 144$, $MS_W = 24/9 \approx 2,667$ et $F = 144/2,667 = 54$. C'est très au-dessus du seuil de la loi $\mathcal{F}(2, 9)$ (4,26 à 5 %, et $p \approx 10^{-5}$) : les fournisseurs diffèrent nettement.

```
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.formula.api as smf
from statsmodels.stats.anova import anova_lm

dat = {"F1": [52, 48, 50, 50], "F2": [56, 58, 54, 56], "F3": [62, 60, 64, 62]}
y = np.array(list(dat.values()), dtype=float)
k, n = y.shape
```

```

mg = y.mean()
SSB = n * ((y.mean(axis=1) - mg) ** 2).sum()
SSW = ((y - y.mean(axis=1, keepdims=True)) ** 2).sum()
MSB, MSW = SSB / (k - 1), SSW / (k * n - k)
print("moyennes :", y.mean(axis=1), "; moyenne générale :", mg)
print(f"SSB = {SSB:.0f}, SSW = {SSW:.0f}, MSB = {MSB:.0f}, MSW = {MSW:.3f}, F = {MSB / MSW:.1f}")
print(f"seuil F(2, 9) à 5 % = {stats.f.ppf(0.95, k - 1, k * n - k):.2f} ; p = {stats.f.sf(MSB / MSW, k
↪ - 1, k * n - k):.1e}")

```

```

moyennes : [50. 56. 62.] ; moyenne générale : 56.0
SSB = 288, SSW = 24, MSB = 144, MSW = 2.667, F = 54.0
seuil F(2, 9) à 5 % = 4.26 ; p = 9.7e-06

```

Corrigé 8.3

$\eta^2 = SS_B / SS_T = 288 / 312 \approx 0,923$: le fournisseur explique 92 % de la variance. $\omega^2 = (SS_B - (k - 1)MS_W) / (SS_T + MS_W) = (288 - 2 \times 2,667) / (312 + 2,667) \approx 0,898$. $\omega^2 < \eta^2$ parce que η^2 attribue au facteur une part du **bruit d'échantillonnage** (même sans effet réel, $SS_B > 0$ en général) ; ω^2 retranche cette part attendue, $(k - 1)MS_W$, et est donc moins optimiste.

```

SST = SSB + SSW
print(f"eta² = {SSB / SST:.4f} ; omega² = {(SSB - (k - 1) * MSW) / (SST + MSW):.4f}")
long = pd.DataFrame({"fournisseur": np.repeat(list(dat), n), "resistance": y.ravel()})
mod = smf.ols("resistance ~ fournisseur", data=long).fit() # (la colonne de texte est traitée
↪ comme un facteur)
print(f"régression : F = {mod.fvalue:.1f}, R² = {mod.rsquared:.4f} (= eta²)")

```

```

eta² = 0.9231 ; omega² = 0.8983
régression : F = 54.0, R² = 0.9231 (= eta²)

```

Corrigé 8.4

$MS_W = 2,667$, $n = 4$: $\sqrt{MS_W/n} = \sqrt{0,667} \approx 0,816$, donc $HSD = 3,95 \times 0,816 \approx 3,22$ N. Les écarts de moyennes sont $|56 - 50| = 6$, $|62 - 56| = 6$ et $|62 - 50| = 12$, tous supérieurs à 3,22 : **les trois fournisseurs diffèrent deux à deux**, le troisième étant le plus résistant. Trois tests de Student à 5 % donneraient un risque global de fausse alerte proche de $1 - 0,95^3 \approx 14\%$; Tukey contrôle ce risque à 5 % pour l'ensemble des comparaisons.

```

from statsmodels.stats.multicomp import pairwise_tukeyhsd
q = stats.studentized_range.ppf(0.95, k, k * n - k)
print(f"q = {q:.3f} ; HSD = {q * np.sqrt(MSW / n):.2f}")
res = pairwise_tukeyhsd(long["resistance"], long["fournisseur"])
print(pd.DataFrame(res._results_table.data[1:],
↪ columns=res._results_table.data[0]).round(3).to_string(index=False))

```

```

q = 3.948 ; HSD = 3.22
group1 group2 meandiff p-adj lower upper reject
F1 F2 6.0 0.002 2.776 9.224 True
F1 F3 12.0 0.000 8.776 15.224 True
F2 F3 6.0 0.002 2.776 9.224 True

```

Corrigé 8.5

Moyennes des blocs : 13, 23,5, 33 ; des traitements : 20,33, 24, 22,0, 26,33 ; moyenne générale 23,17.

(a) $SS_{\text{blocs}} = 4 \sum (\bar{y}_j - \bar{y})^2$, $SS_{\text{trait}} = 3 \sum (\bar{y}_i - \bar{y})^2$, et SS_E par différence, avec 2, 3 et $(2)(3) = 6$ degrés de liberté ; on obtient $SS_{\text{blocs}} \approx 800,7$, $SS_{\text{trait}} \approx 60,3$ et $SS_E \approx 2,67$ (leur somme est la somme totale des carrés, vérifiez-le). (b) Avec les blocs : $F = \frac{60,3/3}{2,67/6} \approx 45,3$, très significatif ($p = 0,0002$). Sans

les blocs, le résidu absorbe la variabilité entre semaines : $F = \frac{60,3/3}{(800,7 + 2,67)/8} \approx 0,20$ ($p = 0,89$), aucun effet visible. (c) Les semaines diffèrent énormément (13, 23,5, 33), alors que les traitements diffèrent peu : le facteur « semaine » domine, et sans le bloc, l'effet des traitements est invisible.

```
Y = np.array([[10, 14, 12, 16], [20, 25, 22, 27], [31, 33, 32, 36]], dtype=float) # lignes = blocs
b, t = Y.shape
mg = Y.mean()
SSbl = t * ((Y.mean(axis=1) - mg) ** 2).sum()
SStr = b * ((Y.mean(axis=0) - mg) ** 2).sum()
SSe = ((Y - Y.mean(axis=1, keepdims=True) - Y.mean(axis=0, keepdims=True) + mg) ** 2).sum()
print(f"SS blocs = {SSbl:.1f} (ddl {b - 1}) ; SS traitements = {SStr:.1f} (ddl {t - 1}) ; SS erreur = {SSe:.2f} (ddl {(b - 1) * (t - 1)})")
F_avec = (SStr / (t - 1)) / (SSe / ((b - 1) * (t - 1)))
F_sans = (SStr / (t - 1)) / ((SSbl + SSe) / (b * t - t))
print(f"F traitements avec blocs = {F_avec:.2f} (p = {stats.f.sf(F_avec, t - 1, (b - 1) * (t - 1)):.4f})")
print(f"F traitements sans blocs = {F_sans:.2f} (p = {stats.f.sf(F_sans, t - 1, b * t - t):.4f})")
```

```
SS blocs = 800.7 (ddl 2) ; SS traitements = 60.3 (ddl 3) ; SS erreur = 2.67 (ddl 6)
F traitements avec blocs = 45.25 (p = 0.0002)
F traitements sans blocs = 0.20 (p = 0.8933)
```

Corrigé 8.6

effet(A) = $\frac{(30+15)-(20+25)}{2} = 0$; effet(B) = $\frac{(25+15)-(20+30)}{2} = -5$; AB = $\frac{(15-25)-(30-20)}{2} = -10$. L'effet principal de A est nul **parce que deux effets de signes opposés se compensent** : A fait **monter** la réponse de +10 quand B est bas (20 → 30) et la fait **baissier** de 10 quand B est haut (25 → 15). A a donc une influence majeure, mais elle **dépend** de B : c'est exactement le piège de l'interaction qui annule un effet principal. Le meilleur réglage est (A+, B-) avec 30.

```
m = {"-","-": 20, ("+", "-"): 30, ("-", "+"): 25, ("+", "+"): 15}
eff_A = ((m[("+", "-")] + m[("+", "+")]) - (m[("-","-")] + m[("-","+")])) / 2
eff_B = ((m[("-","+")] + m[("+","+")]) - (m[("-","-")] + m[("+","-")])) / 2
Aeff_B = ((m[("+","+")] - m[("-","+")]) - (m[("+","-")] - m[("-","-")])) / 2
print("A =", eff_A, "; B =", eff_B, "; AB =", eff_AB, "; meilleur réglage :", max(m, key=m.get))
```

```
A = 0.0 ; B = -5.0 ; AB = -3.5 ; meilleur réglage : ('+', '-')
```

Corrigé 8.7

Moyenne = $116/8 = 14,5$. Effets (contraste / 4) : $A = (14 + 20 + 15 + 21 - 10 - 12 - 11 - 13)/4 = 24/4 = 6$; $B = (12 + 20 + 13 + 21 - 10 - 14 - 11 - 15)/4 = 16/4 = 4$; $C = (11 + 15 + 13 + 21 - 10 - 14 - 12 - 20)/4 = 4/4 = 1$; AB : signes + pour (1), ab, c, abc : $(10 + 20 + 11 + 21 - 14 - 12 - 15 - 13)/4 = 8/4 = 2$; $AC = BC = ABC = 0$. Les effets non nuls sont donc A, B, C et AB, avec $A > B > AB > C$.

```
def yates(y):
    col = np.array(y, float)
    for _ in range(int(np.log2(len(y)))):
        col = np.concatenate([col[0::2] + col[1::2], col[1::2] - col[0::2]])
    return col
```

```

y = [10, 14, 12, 20, 11, 15, 13, 21]
contr = yates(y)
noms = ["I", "A", "B", "AB", "C", "AC", "BC", "ABC"]
print({nom: float(v) for nom, v in zip(noms, np.round(np.r_[contr[0] / 8, contr[1:] / 4], 2))})

```

```
{'I': 14.5, 'A': 6.0, 'B': 4.0, 'AB': 2.0, 'C': 1.0, 'AC': 0.0, 'BC': 0.0, 'ABC': 0.0}
```

Corrigé 8.8

- (a) $N = 8r$, donc l'écart-type d'un effet vaut $ET = 2\sigma / \sqrt{8r} = 6 / \sqrt{8r}$. (b) Avec l'approximation normale, la puissance de 80 % exige $\Delta/ET \approx 1,96 + 0,84 = 2,8$, soit $ET \leq 4/2,8 = 1,43$ et $8r \geq (6/1,43)^2 \approx 17,6$, donc $r \geq 2,2$: environ **3 répétitions**. (c) Avec la loi de Student, qui a peu de degrés de liberté quand r est petit (8 pour $r = 2$), il faut un peu plus de marge. Le calcul exact confirme ce qu'annonce l'approximation :

```

sigma, delta = 3.0, 4.0
for r in (2, 3, 4):
    N = 8 * r
    ddl = N - 8
    se = 2 * sigma / np.sqrt(N)
    seuil = stats.t.ppf(0.975, ddl)
    puissance = stats.nct.sf(seuil, ddl, delta / se) + stats.nct.cdf(-seuil, ddl, delta / se)
    print(f"r = {r} : N = {N}, ET = {se:.3f}, ddl = {ddl}, puissance = {puissance:.3f}")

```

```

r = 2 : N = 16, ET = 1.500, ddl = 8, puissance = 0.648
r = 3 : N = 24, ET = 1.225, ddl = 16, puissance = 0.865
r = 4 : N = 32, ET = 1.061, ddl = 24, puissance = 0.951

```

La puissance est de 65 % pour $r = 2$, de 86,5 % pour $r = 3$ et de 95 % pour $r = 4$: le seuil de 80 % est atteint à partir de $r = 3$ (24 essais), comme l'annonçait l'approximation normale. La loi de Student, qui tient compte du petit nombre de degrés de liberté de l'erreur pure, rend le calcul exact un peu plus exigeant pour les petites valeurs de r .

Corrigé 8.9

- (a) $D = ABC \Rightarrow I = ABCD$. Un seul mot de 4 lettres : **résolution IV**. $AB = AB \cdot ABCD = CD$. (b) $D = AB \Rightarrow I = ABD$ (mot de 3 lettres) : **résolution III** ; $AB = AB \cdot ABD = D$: l'interaction AB est confondue avec le **facteur principal D**, ce qui est le pire cas ; de plus, $A = BD$, $B = AD$. (c) Le premier : à nombre d'essais égal, la résolution IV garantit que les effets principaux ne sont confondus qu'avec des interactions d'ordre 3 ; la résolution III les confond avec des interactions d'ordre 2, souvent non négligeables.

```

def produit(u, v):
    return "".join(sorted(set(u) ^ set(v)))
for gen, mots in [("D = ABC", ["ABCD"]), ("D = AB", ["ABD"])] :
    print(f"{gen} : I = {' '.join(mots)} ; résolution {min(len(w) for w in mots)} ; "
          f"AB = {' '.join(produit('AB', w) for w in mots)} ; A = {' '.join(produit('A', w) for w
          in mots)}")

```

```

D = ABC : I = ABCD ; résolution 4 ; AB = CD ; A = BCD
D = AB : I = ABD ; résolution 3 ; AB = D ; A = BD

```

Corrigé 8.10

- (a) Les mots générateurs : $E = ABC \Rightarrow I = ABCE$; $F = BCD \Rightarrow I = BCDF$. Leur produit est aussi un mot : $ABCE \cdot BCDF = ADE F$ (les lettres B et C, communes, s'éliminent). Relation complète : $I = ABCE = BCDF = ADEF$. (b) Mots de longueurs 4, 4 et 4 : **résolution IV**. (c) $A = A \cdot ABCE = BCE$; $A \cdot BCDF = ABCDF$; $A \cdot ADEF = DEF$: $A = BCE = ABCDF = DEF$ (effets d'ordre 3 ou plus : **A est propre**). $AB = CE = ACDF = BDEF$. (d) Non : AB est confondue avec CE (elle l'est par le mot ABCE) : en résolution IV, certaines interactions d'ordre 2 sont confondues entre elles ; pour les séparer, il faudrait un plan de résolution V (plus d'essais) ou un repliement bien choisi.

```
mots = ["ABCE", "BCDF", produit("ABCE", "BCDF")]
print("relation de définition : I = " + " = ".join(mots))
for e in ["A", "AB"]:
    print(f"{e} = " + " = ".join(produit(e, w) for w in mots))
```

```
relation de définition : I = ABCE = BCDF = ADEF
A = BCE = ABCDF = DEF
AB = CE = ACDF = BDEF
```

Corrigé 8.11

- (a) $b = (4, 2)^T$, $B = \begin{pmatrix} -3 & 0,5 \\ 0,5 & -1 \end{pmatrix}$ (le terme croisé $1 \cdot x_1 x_2$ se répartit en $0,5 + 0,5$). $\det B = 3 - 0,25 = 2,75$ et $B^{-1} = \frac{1}{2,75} \begin{pmatrix} -1 & -0,5 \\ -0,5 & -3 \end{pmatrix}$, donc $B^{-1}b = \frac{1}{2,75}(-5, -8)^T$ et $x_s = -\frac{1}{2}B^{-1}b \approx (0,909; 1,455)$. La réponse prédite est $\hat{y}_s = 70 + \frac{1}{2}b^T x_s = 70 + \frac{1}{2}(4 \times 0,909 + 2 \times 1,455) \approx 73,27$. (b) La trace de B vaut -4 et son déterminant $2,75 > 0$: les valeurs propres sont $\frac{-4 \pm \sqrt{16-11}}{2} \approx -0,88$ et $-3,12$, **toutes deux négatives** : c'est un **maximum**. (c) La distance au centre est $\sqrt{0,909^2 + 1,455^2} \approx 1,72$, **supérieure** au rayon $\sqrt{2} \approx 1,41$ des points axiaux : le sommet est **en dehors du domaine expérimental**. C'est une extrapolation : on ne doit pas lui faire confiance, mais déplacer le plan dans cette direction et recommencer.

```
b2 = np.array([4.0, 2.0])
B2 = np.array([[-3.0, 0.5], [0.5, -1.0]])
xs = -0.5 * np.linalg.solve(B2, b2)
print("point stationnaire :", xs.round(3), "; réponse :", round(70 + 0.5 * b2 @ xs, 2))
print("valeurs propres de B :", np.linalg.eigvalsh(B2).round(2))
print(f"distance au centre = {np.linalg.norm(xs):.2f} ; rayon du domaine (points axiaux) = {np.sqrt(2):.2f}")
```

```
point stationnaire : [0.909 1.455] ; réponse : 73.27
valeurs propres de B : [-3.12 -0.88]
distance au centre = 1.72 ; rayon du domaine (points axiaux) = 1.41
```

Corrigé 8.12

Dans un plan complètement randomisé, chaque unité vient d'un bloc différent : la variabilité entre blocs (écart-type 30) s'ajoute au bruit (10), soit un écart-type de $\sqrt{30^2 + 10^2} \approx 31,6$ par unité. L'écart-type de la différence de deux moyennes de 6 unités vaut $31,6\sqrt{2/6} \approx 18,3$, pour un effet de 15 : le rapport signal/bruit est d'environ 0,8 et le test est peu puissant. Dans le plan en blocs, la **différence** entre les deux traitements au sein d'un même bloc élimine l'effet de bloc : l'écart-type d'une différence est de $10\sqrt{2} \approx 14,1$, celui de la moyenne des 6 différences $14,1/\sqrt{6} \approx 5,8$, soit un rapport signal/bruit de 2,6 : bien meilleur. La simulation le chiffre.

```

rng = np.random.default_rng(90)
effet, sig, sig_bloc, n_par_trait, n_sim = 15.0, 10.0, 30.0, 6, 3000
rej_crd, rej_blocs = 0, 0
for _ in range(n_sim):
    # (a) plan complètement randomisé : 12 unités, chacune avec SON propre effet de bloc (indépendants)
    y1 = rng.normal(0, sig_bloc, n_par_trait) + rng.normal(0, sig, n_par_trait)
    y2 = effet + rng.normal(0, sig_bloc, n_par_trait) + rng.normal(0, sig, n_par_trait)
    rej_crd += stats.ttest_ind(y2, y1).pvalue < 0.05
    # (b) plan en blocs : 6 blocs, une unité de chaque traitement par bloc (l'effet de bloc est PARTAGÉ)
    blocs = rng.normal(0, sig_bloc, n_par_trait)
    z1 = blocs + rng.normal(0, sig, n_par_trait)
    z2 = blocs + effet + rng.normal(0, sig, n_par_trait)
    rej_blocs += stats.ttest_rel(z2, z1).pvalue < 0.05
print(f"puissance, plan complètement randomisé (Student, 6 contre 6) : {rej_crd / n_sim:.3f}")
print(f"puissance, plan en blocs (Student apparié, 6 blocs) : {rej_blocs / n_sim:.3f}")

```

```

puissance, plan complètement randomisé (Student, 6 contre 6) : 0.112
puissance, plan en blocs (Student apparié, 6 blocs) : 0.552

```

Les deux plans utilisent **le même nombre d'unités** (12) et le même effet, mais le plan en blocs le détecte dans environ 55 % des expériences, contre environ 11 % pour le plan complètement randomisé : un facteur 5 de puissance obtenu **sans une unité de plus**, simplement en organisant l'expérience. (Un piège à éviter : si l'on appliquait un test de Student à deux échantillons à des données **appariées par un bloc partagé**, on traiterait comme indépendantes des mesures corrélées ; le test deviendrait trop conservateur, avec une puissance inférieure même à son seuil de 5 %. C'est une erreur d'analyse, pas un plan complètement randomisé.)

Corrigé 8.13

Valeurs absolues triées : 0,4; 0,5; 0,6; 0,8; 1,0; 8,0; 12,0 ; médiane = 0,8, donc $s_0 = 1,5 \times 0,8 = 1,2$ et le seuil $2,5s_0 = 3,0$. On ne garde que les $|c| < 3$: 0,4; 0,5; 0,6; 0,8; 1,0, de médiane 0,6 : $PSE = 1,5 \times 0,6 = 0,9$. Avec $d = 7/3 \approx 2,33$ degrés de liberté, $t_{0,975} \approx 3,76$ (très grand, faute de degrés de liberté) : $ME \approx 3,39$. Les effets 12 et 8 dépassent largement la marge ; les cinq autres n'en approchent pas : **deux effets actifs**.

```

c = np.array([12.0, -1.0, 0.5, 8.0, -0.8, 0.4, 0.6])
a = np.abs(c)
s0 = 1.5 * np.median(a)
pse = 1.5 * np.median(a[a < 2.5 * s0])
d = len(c) / 3
ME = stats.t.ppf(0.975, d) * pse
print(f"s0 = {s0:.2f} ; PSE = {pse:.2f} ; d = {d:.2f} ; t = {stats.t.ppf(0.975, d):.2f} ; ME = {ME:.2f}")
print("effets actifs (|c| > ME) :", c[a > ME])

```

```

s0 = 1.20 ; PSE = 0.90 ; d = 2.33 ; t = 3.76 ; ME = 3.39
effets actifs (|c| > ME) : [12.  8.]

```

Chapitre 9 : ⊕ Statistique spatiale — exercices et applications

Ce chapitre du cahier accompagne le chapitre 9 du livre. Il contient le **code complet** que le livre exécute « en coulisses » (sept applications guidées, qui reconstruisent pas à pas les distances, les indices de Moran et de Geary, le variogramme, le krigeage et les processus ponctuels) puis **douze exercices corrigés**. Aucune bibliothèque de cartographie : NumPy, SciPy, matplotlib (et mgcv en R pour un point de comparaison). Les données sont **simulées** et le décor est une **région fictive** (villes, zones et coordonnées inventées) ; les fichiers sont dans `donnees/` (`ch09-zones.csv`, `ch09-livraisons.csv`, `clients.csv`) et leurs générateurs dans `build/donnees_ch09.py`.

Le fichier est exécuté **d'un seul tenant, de haut en bas** : les fonctions définies dans une application servent dans les suivantes et dans les corrigés. Si vous travaillez dans un notebook, exécutez les cellules dans l'ordre.

Applications

Application 9.1 — Distances et cartes (section 9.1)

Objectif. Repérer des villes par leurs coordonnées, calculer des distances sur la sphère (haversine), mesurer l'erreur de deux raccourcis, et dessiner des cartes sans fond de carte.

Étape 1 : les villes et la formule de haversine. Dix villes fictives ; la fonction accepte des tableaux grâce à la diffusion de NumPy.

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

villes = pd.DataFrame({
    "ville": ["Ville A", "Ville B", "Ville C", "Ville D", "Ville E", "Ville F", "Ville G", "Ville H",
             "Ville I", "Ville J"],
    "lat":  [47.30, 46.90, 46.40, 45.80, 45.30, 44.70, 47.00, 46.00, 45.20, 44.30],
    "lon":  [ 3.20,  4.60,  3.60,  4.90,  3.40,  5.30,  6.40,  6.70,  6.60,  3.70],
})
```

```
R = 6371.0 # rayon moyen de la Terre, en km
```

```
def haversine(lat1, lon1, lat2, lon2):
    """Distance de grand cercle en km (arguments en degrés ; NumPy diffuse sur les tableaux)."""
```

```

p1, p2 = np.radians(lat1), np.radians(lat2)
a = np.sin((p2 - p1) / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(np.radians(lon2 - lon1) / 2) ** 2
return 2 * R * np.arcsin(np.sqrt(a))

print("1 degré de latitude      :", round(float(haversine(46.0, 4.0, 47.0, 4.0)), 1), "km")
print("1 degré de longitude à 45° N :", round(float(haversine(45.0, 4.0, 45.0, 5.0)), 1), "km")
print("Ville A - Ville B      :", round(float(haversine(47.30, 3.20, 46.90, 4.60)), 1), "km")
↪ (calcul à la main : 114,9)

```

```

1 degré de latitude      : 111.2 km
1 degré de longitude à 45° N : 78.6 km
Ville A - Ville B      : 114.9 km (calcul à la main : 114,9)

```

Étape 2 : la matrice des distances. Transformer les vecteurs en colonnes et en lignes calcule les 10×10 paires d'un coup. La matrice est symétrique, de diagonale nulle ; la plus grande distance est entre G et J (366 km), la plus petite entre H et I (89 km).

```

lat, lon = villes["lat"].to_numpy(), villes["lon"].to_numpy()
D = haversine(lat[:, None], lon[:, None], lat[None, :], lon[None, :]) # (10, 10)
lettres = [v[-1] for v in villes["ville"]]
print(pd.DataFrame(D.round(0).astype(int), index=lettres, columns=lettres).to_string())
print("matrice symétrique :", np.allclose(D, D.T), "| diagonale nulle :", np.allclose(np.diag(D), 0))

```

	A	B	C	D	E	F	G	H	I	J
A	0	115	105	211	223	331	244	304	350	336
B	115	0	94	124	201	251	137	189	244	297
C	105	94	0	120	123	231	224	243	268	234
D	211	124	120	0	129	126	176	141	148	192
E	223	201	123	129	0	164	299	268	251	114
F	331	251	231	126	164	0	270	181	116	134
G	244	137	224	176	299	270	0	114	201	366
H	304	189	243	141	268	181	114	0	89	302
I	350	244	268	148	251	116	201	89	0	250
J	336	297	234	192	114	134	366	302	250	0

matrice symétrique : True | diagonale nulle : True

Étape 3 : l'erreur des raccourcis. (a) traiter les degrés comme des unités égales ; (b) la projection équirectangulaire centrée sur la latitude moyenne φ_0 . Le raccourci (a) se trompe de 1 % à 46 % selon la direction du trajet, le raccourci (b) de moins de 2,5 %.

```

def naif(lat1, lon1, lat2, lon2):
    return 111.19 * np.hypot(lat2 - lat1, lon2 - lon1)

def equirect(lat1, lon1, lat2, lon2, lat0):
    c = np.cos(np.radians(lat0))
    return R * np.radians(1) * np.hypot(lat2 - lat1, (lon2 - lon1) * c)

lat0 = villes["lat"].mean()
paires = [("Ville A", "Ville B"), ("Ville A", "Ville J"), ("Ville A", "Ville G"), ("Ville G", "Ville J"), ("Ville C", "Ville H")]
lignes = []
for a, b in paires:
    ra, rb = villes.set_index("ville").loc[a], villes.set_index("ville").loc[b]
    vrai = float(haversine(ra.lat, ra.lon, rb.lat, rb.lon))
    n = float(naif(ra.lat, ra.lon, rb.lat, rb.lon))
    e = float(equirect(ra.lat, ra.lon, rb.lat, rb.lon, lat0))
    lignes.append({"paire": f"{a[-1]}-{b[-1]}", "haversine_km": vrai, "degres_bruts_km": n,
    ↪ "erreur_%": 100 * (n / vrai - 1),
    "equirect_km": e, "erreur_eq_%": 100 * (e / vrai - 1)})
print(pd.DataFrame(lignes).round(1).to_string(index=False))

```

paire	haversine_km	degres_bruts_km	erreur_%	equirect_km	erreur_eq_%
A-B	114.9	161.9	40.9	117.1	1.9
A-J	335.8	338.2	0.7	335.8	-0.0
A-G	244.3	357.4	46.3	249.9	2.3
G-J	366.3	424.6	15.9	365.8	-0.1
C-H	242.7	347.5	43.2	244.0	0.6

Étape 4 : une carte à symboles proportionnels. La surface des disques est proportionnelle au nombre de clients (villes A à E du fichier `clients.csv`). À gauche, les degrés sont dessinés comme des unités égales ; à droite, le rapport d'aspect $1/\cos \varphi_0$ rétablit les proportions.

```
clients = pd.read_csv("donnees/clients.csv")
par_ville = clients.groupby("ville").agg(clients=("id_client", "size"), depense=("depense_annuelle",
↪ "mean")).round(1)
print(par_ville.to_string())

carte = villes.merge(par_ville, left_on="ville", right_index=True, how="left") # F à J : pas de
↪ clients dans ce fichier
BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
```

	clients	depense
ville		
Autre	301	274.0
Ville A	208	217.9
Ville B	252	243.6
Ville C	282	245.6
Ville D	337	247.0
Ville E	620	245.5

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 5.2))
for ax, titre, corrige in zip(axes, ["Degrés traités comme des unités égales", "Aspect corrigé (1 /
↪ cos de la latitude)"], [False, True]):
    ax.scatter(carte["lon"], carte["lat"], s=18, color=ORANGE, zorder=3)
    ax.scatter(carte["lon"], carte["lat"], s=carte["clients"].fillna(0) * 1.5, color=BLEU,
↪ alpha=0.35, zorder=2)
    for _, r in carte.iterrows():
        ax.annotate(r["ville"][-1], (r["lon"], r["lat"]), xytext=(6, 5), ha="left", textcoords="offset
↪ points", fontsize=9)
    ax.set_title(titre, fontsize=10)
    ax.set_xlabel("longitude (°E)")
    ax.set_xlim(2.4, 7.6)
    ax.set_ylim(43.8, 47.8)
    ax.set_aspect(1 / np.cos(np.radians(lat0)) if corrige else 1.0)
axes[0].set_ylabel("latitude (°N)")
plt.tight_layout()
plt.savefig("figures/ch09-carte-villes.png", dpi=200, bbox_inches="tight")
```

Étape 5 : les deux jeux du chapitre, côte à côte. Une carte choroplèthe pour les zones (surfactive) et un nuage de points colorés pour les livraisons (géostatistique).

```
zones = pd.read_csv("donnees/ch09-zones.csv")
liv = pd.read_csv("donnees/ch09-livraisons.csv")
print("zones :", zones.shape, "| livraisons :", liv.shape)
print(liv["delai_jours"].describe().round(2).to_string())
```

```
zones : (144, 8) | livraisons : (200, 4)
count    200.00
mean      4.27
std       1.10
min       1.59
25%      3.62
50%      4.26
75%      5.07
```

max 7.53

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.6))
grille = zones["ventes_hab"].to_numpy().reshape(12, 12) # une case = 5 km x 5 km
im = ax1.imshow(grille, origin="lower", extent=(0, 60, 0, 60), cmap="Blues")
ax1.set_title("Ventes par habitant (€) par zone")
ax1.set_xlabel("x (km)"); ax1.set_ylabel("y (km)")
fig.colorbar(im, ax=ax1, shrink=0.8, label="€ par habitant")

sc = ax2.scatter(liv["x"], liv["y"], c=liv["delai_jours"], cmap="Oranges", s=46, edgecolor="#555555",
               ↪ linewidth=0.3)
ax2.set_title("Délai de livraison (jours) à 200 adresses")
ax2.set_xlabel("x (km)"); ax2.set_ylabel("y (km)")
ax2.set_aspect("equal")
fig.colorbar(sc, ax=ax2, shrink=0.8, label="jours")
plt.tight_layout()
plt.savefig("figures/ch09-donnees-chapitre.png", dpi=200, bbox_inches="tight")
```

Pour aller plus loin. Remplacez la distance à vol d'oiseau par une matrice de temps de trajet inventée (non symétrique) et recalculez la carte : quelles propriétés des distances sont indispensables aux outils du chapitre ?

Application 9.2 — Les ventes par zone : Moran, Geary et indices locaux (section 9.2)

Objectif. Construire les matrices de voisinage, programmer l'indice de Moran (hand-check sur une grille 3×3), le tester par approximation normale et par permutations, mesurer sa sensibilité au choix de W , puis l'indice de Geary et les indices locaux (LISA) corrigés par Benjamini-Hochberg.

Étape 1 : la grille 3×3 et la standardisation par ligne. Les coins ont 2 voisines, les bords 3, le centre 4 : $S_0 = 24$.

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

def contiguite_tour(n_lig, n_col):
    """Matrice binaire des voisins par un côté (haut, bas, gauche, droite)."""
    n = n_lig * n_col
    W = np.zeros((n, n))
    for i in range(n_lig):
        for j in range(n_col):
            for di, dj in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                if 0 <= i + di < n_lig and 0 <= j + dj < n_col:
                    W[i * n_col + j, (i + di) * n_col + (j + dj)] = 1
    return W

W3 = contiguite_tour(3, 3)
print(W3.astype(int))
print("nombre de voisins par zone :", W3.sum(axis=1).astype(int))
print("S0 (somme de tous les poids) :", int(W3.sum()))
```

```
[[0 1 0 1 0 0 0 0 0]
 [1 0 1 0 1 0 0 0 0]
 [0 1 0 0 0 1 0 0 0]
 [1 0 0 0 1 0 1 0 0]
 [0 1 0 1 0 1 0 1 0]
 [0 0 1 0 1 0 0 0 1]
```

```
[0 0 0 1 0 0 0 1 0]
[0 0 0 0 1 0 1 0 1]
[0 0 0 0 0 1 0 1 0]]
nombre de voisins par zone : [2 3 2 3 4 3 2 3 2]
S0 (somme de tous les poids) : 24
```

```
def std_lignes(W):
    """Standardise par ligne : chaque ligne somme à 1 (les lignes de zéros sont laissées intactes)."""
    s = W.sum(axis=1, keepdims=True)
    return np.divide(W, s, out=np.zeros_like(W), where=s > 0)

W3s = std_lignes(W3)
print(np.round(W3s[4], 2), "<- ligne du centre : 1/4 pour chacun de ses 4 voisins")
print("somme de chaque ligne :", W3s.sum(axis=1))
```

```
[0.  0.25 0.  0.25 0.  0.25 0.  0.25 0. ] <- ligne du centre : 1/4 pour chacun de ses 4 voisins
somme de chaque ligne : [1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Étape 2 : l'indice de Moran. Sur la progression 1..9 on retrouve $I = 0,5$ à la main, sur le damier -1 ; avec les poids standardisés, 0,556 ; l'espérance sous le hasard est $-1/(n-1) = -0,125$.

```
def moran(y, W):
    """Indice de Moran de y pour la matrice de poids W (standardisée ou non)."""
    z = np.asarray(y, dtype=float) - np.mean(y)
    return len(z) / W.sum() * (z @ W @ z) / (z @ z)

progression = np.arange(1, 10)
damier = np.array([1, 9, 1, 9, 1, 9, 1, 9, 1], dtype=float)
print("progression 1..9, poids binaires      :", round(moran(progression, W3), 4))
print("damier,          poids binaires      :", round(moran(damier, W3), 4))
print("progression 1..9, poids standardisés :", round(moran(progression, W3s), 4))
print("E[I] sans autocorrélation, n = 9      :", round(-1 / (9 - 1), 4))
```

```
progression 1..9, poids binaires      : 0.5
damier,          poids binaires      : -1.0
progression 1..9, poids standardisés : 0.5556
E[I] sans autocorrélation, n = 9      : -0.125
```

Étape 3 : le générateur des ventes (modèle autorégressif spatial). On propage des bruits indépendants par $z = (I - \rho W)^{-1}e$ avec $\rho = 0,9$; le résultat est identique au fichier fourni. La variable ventes_bruit n'a aucune structure spatiale : c'est le témoin.

```
def contiguite_reine(n_lig, n_col):
    """Voisins par un côté ou un coin (8 directions)."""
    n = n_lig * n_col
    W = np.zeros((n, n))
    for i in range(n_lig):
        for j in range(n_col):
            for di in (-1, 0, 1):
                for dj in (-1, 0, 1):
                    if (di, dj) != (0, 0) and 0 <= i + di < n_lig and 0 <= j + dj < n_col:
                        W[i * n_col + j, (i + di) * n_col + (j + dj)] = 1
    return W
```

Le générateur proprement dit :

```
def grille_zones(seed=9, n_lig=12, n_col=12, rho=0.9):
    rng = np.random.default_rng(seed)
    W = std_lignes(contiguite_reine(n_lig, n_col))
    n = n_lig * n_col
    e = rng.normal(size=n)
    z = np.linalg.solve(np.eye(n) - rho * W, e) # z = (I - rho W)^-1 e
    pop = np.round(rng.lognormal(8.5, 0.5, n)).astype(int)
```

```

lig, col = np.divmod(np.arange(n), n_col)
return pd.DataFrame({"id": np.arange(n), "lig": lig, "col": col, "x": col * 5.0 + 2.5, "y": lig *
↳ 5.0 + 2.5,
                    "population": pop, "ventes_hab": np.round(50 + 8 * z, 2),
                    "ventes_bruit": np.round(rng.normal(50, 8, n), 2)})

zones = grille_zones()
fichier = pd.read_csv("donnees/ch09-zones.csv")
print("identique au fichier fourni :", np.allclose(zones[["ventes_hab", "ventes_bruit"]],
↳ fichier[["ventes_hab", "ventes_bruit"]]))
print(zones[["ventes_hab", "ventes_bruit"]].describe().round(2).to_string())

```

```

identique au fichier fourni : True
      ventes_hab  ventes_bruit
count      144.00      144.00
mean         52.89         50.22
std          13.51          8.15
min          19.29         29.97
25%          44.29         43.90
50%          53.43         50.36
75%          62.67         56.50
max          81.49         71.64

```

Étape 4 : test de Moran, par la formule normale et par permutations. Pour ventes_hab : $I = 0,621$, $z = 14,2$, p-valeur par permutations 10^{-4} (le minimum avec 9 999 permutations) ; pour le témoin : $I = -0,021$, p-valeur 0,61.

```

W = std_lignes(contiguite_reine(12, 12))
n = len(zones)

def stats_poids(W):
    S0 = W.sum()
    S1 = 0.5 * ((W + W.T) ** 2).sum()
    S2 = ((W.sum(axis=1) + W.sum(axis=0)) ** 2).sum()
    return S0, S1, S2

def moran_normal(y, W):
    """I, E[I], écart-type sous normalité et z_I."""
    n = len(y)
    S0, S1, S2 = stats_poids(W)
    E = -1 / (n - 1)
    var = (n**2 * S1 - n * S2 + 3 * S0**2) / ((n**2 - 1) * S0**2) - E**2
    I = moran(y, W)
    return I, E, np.sqrt(var), (I - E) / np.sqrt(var)

```

Les fonctions de test, puis leur application aux deux variables :

```

def moran_permutations(y, W, B=9999, seed=1):
    """B valeurs de I obtenues en mélangeant y sur les zones."""
    rng = np.random.default_rng(seed)
    z = np.asarray(y, dtype=float) - np.mean(y)
    idx = np.argsort(rng.random((B, len(z))), axis=1)           # B permutations de 0..n-1
    Z = z[idx]                                                    # chaque ligne est un mélange de z
    return len(z) / W.sum() * ((Z @ W) * Z).sum(axis=1) / (z @ z)

lignes = []
for nom in ["ventes_hab", "ventes_bruit"]:
    y = zones[nom].to_numpy()
    I, E, sd, zI = moran_normal(y, W)
    sim = moran_permutations(y, W)
    p_perm = (1 + (sim >= I).sum()) / (len(sim) + 1)
    lignes.append({"variable": nom, "I": I, "E[I]": E, "ecart_type": sd, "z": zI, "p_normale":
↳ stats.norm.sf(zI),

```

```

        "p_permutation": p_perm, "sd_perm": sim.std())
res = pd.DataFrame(lignes)
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 150):
    print(res.to_string(index=False))

```

variable	I	E[I]	ecart_type	z	p_normale	p_permutation	sd_perm
ventes_hab	0.6207	-0.006993	0.04429	14.17	6.802e-46	0.0001	0.04468
ventes_bruit	-0.0214	-0.006993	0.04429	-0.3253	0.6275	0.6132	0.04445

Étape 5 : le diagramme de Moran (nuage de Wz contre z ; la pente est I) et la loi de I obtenue en mélangeant les valeurs.

```

BLEU, ORANGE, AQUA, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#e34948"
y = zones["ventes_hab"].to_numpy()
zc = y - y.mean()
lag = W @ zc
I_obs = moran(y, W)
sim = moran_permutations(y, W)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.4))
ax1.scatter(zc, lag, s=16, color=BLEU, alpha=0.7)
pente, ordonnee = np.polyfit(zc, lag, 1)
xs = np.array([zc.min(), zc.max()])
ax1.plot(xs, pente * xs + ordonnee, color=ORANGE, lw=2)
ax1.axhline(0, color="#999999", lw=0.8); ax1.axvline(0, color="#999999", lw=0.8)
ax1.set_xlabel("écart à la moyenne de la zone z"); ax1.set_ylabel("moyenne des écarts des voisines W
↪ z")
ax1.set_title(f"Diagramme de Moran (pente = I = {pente:.3f})")

```

La loi de I par permutations :

```

ax2.hist(sim, bins=50, color="#b9d0f0", edgecolor="white")
ax2.axvline(I_obs, color=ORANGE, lw=2)
ax2.annotate(f"I observé = {I_obs:.3f}", (I_obs, 400), xytext=(-8, 0), textcoords="offset points",
↪ ha="right", color=ORANGE)
ax2.set_xlabel("I après mélange des valeurs (9999 permutations)"); ax2.set_ylabel("effectif")
ax2.set_title("Ce que produirait le hasard")
plt.tight_layout()
plt.savefig("figures/ch09-moran.png", dpi=200, bbox_inches="tight")
print("pente du diagramme :", round(pente, 4), "| I :", round(I_obs, 4), "| identiques :",
↪ np.isclose(pente, I_obs))

```

pente du diagramme : 0.6207 | I : 0.6207 | identiques : True

Étape 6 : sensibilité au choix de W . Cinq définitions du voisinage : I varie de 0,57 à 0,63, la conclusion ne change pas, le témoin reste proche de zéro.

```

xy = zones[["x", "y"]].to_numpy()
Dd = np.sqrt(((xy[:, None, :] - xy[None, :, :]) ** 2).sum(axis=2)) # distances euclidiennes (km)

def poids_bande(D, d):
    W = ((D > 0) & (D <= d)).astype(float)
    return W

def poids_knn(D, k):
    W = np.zeros_like(D)
    for i in range(len(D)):
        voisins = np.argsort(D[i])[1:k + 1] # on saute la distance nulle
        ↪ (soi-même)
        W[i, voisins] = 1
    return W

```

Les cinq matrices et le calcul de I :

```

variantes = {
    "tour (distance <= 5 km)": poids_bande(Dd, 5.0),
    "reine (distance <= 7,1 km)": poids_bande(Dd, 7.1),
    "bande de 10 km": poids_bande(Dd, 10.0),
    "4 plus proches voisins": poids_knn(Dd, 4),
    "8 plus proches voisins": poids_knn(Dd, 8),
}
lignes = []
for nom, Wv in variantes.items():
    Ws = std_lignes(Wv)
    sim_v = moran_permutations(y, Ws, B=999, seed=3)
    I_v = moran(y, Ws)
    lignes.append({"definition": nom, "voisins_moyens": Wv.sum(axis=1).mean(), "I": I_v,
                  "I_temoin": moran(zones["ventes_bruit"].to_numpy(), Ws),
                  "p_permutation": (1 + (sim_v >= I_v).sum()) / 1000})
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))

```

	definition	voisins_moyens	I	I_temoin	p_permutation
	tour (distance <= 5 km)	3.667	0.627	-0.034	0.001
	reine (distance <= 7,1 km)	7.028	0.621	-0.021	0.001
	bande de 10 km	10.361	0.572	0.009	0.001
	4 plus proches voisins	4.000	0.618	-0.013	0.001
	8 plus proches voisins	8.000	0.607	0.011	0.001

Étape 7 : l'indice de Geary. $C = 0,395$ pour ventes_hab (nettement inférieur à 1), 0,991 pour le témoin.

```

def geary(y, W):
    y = np.asarray(y, dtype=float)
    z = y - y.mean()
    diff2 = (y[:, None] - y[None, :]) ** 2
    return (len(y) - 1) * (W * diff2).sum() / (2 * W.sum() * (z @ z))

def geary_permutations(y, W, B=999, seed=4):
    rng = np.random.default_rng(seed)
    return np.array([geary(rng.permutation(y), W) for _ in range(B)])

lignes = []
for nom in ["ventes_hab", "ventes_bruit"]:
    yy = zones[nom].to_numpy()
    C = geary(yy, W)
    sim_c = geary_permutations(yy, W)
    lignes.append({"variable": nom, "C": C, "moyenne_perm": sim_c.mean(), "p_perm (C petit)": (1 +
    ↪ (sim_c <= C).sum()) / (len(sim_c) + 1)})
with pd.option_context("display.float_format", "{:.4f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))

```

	variable	C	moyenne_perm	p_perm (C petit)
	ventes_hab	0.3950	1.0033	0.0010
	ventes_bruit	0.9909	0.9997	0.4030

Étape 8 : les indices locaux (LISA). Permutation conditionnelle pour chaque zone, puis correction de Benjamini-Hochberg : 35 zones significatives pour ventes_hab (11 HH, 22 LL, 2 LH), aucune pour le témoin (5 zones à $p < 0,05$ sans correction).

```

def lisa(y, W, B=999, seed=2):
    """Moran local, quadrant, et p-valeur bilatérale par permutation conditionnelle."""
    rng = np.random.default_rng(seed)
    y = np.asarray(y, dtype=float)
    n = len(y)
    z = y - y.mean()
    m2 = (z @ z) / n

```

```

lag = W @ z
Ii = z * lag / m2
pv = np.empty(n)
for i in range(n):
    voisins = np.flatnonzero(W[i])
    poids = W[i, voisins]
    r = rng.random((B, n))
    r[:, i] = 2.0 # exclut la zone i du tirage
    tire = np.argsort(r, axis=1)[:len(voisins)] # valeurs tirées pour les voisins
    sim = z[i] * (z[tire] * poids).sum(axis=1) / m2
    p_haut = (1 + (sim >= Ii[i]).sum()) / (B + 1)
    p_bas = (1 + (sim <= Ii[i]).sum()) / (B + 1)
    pv[i] = min(1.0, 2 * min(p_haut, p_bas))
quadrant = np.where(z > 0, np.where(lag > 0, "HH", "HL"), np.where(lag > 0, "LH", "LL"))
return Ii, quadrant, pv

```

La correction de Benjamini-Hochberg :

```

def benjamini_hochberg(p, alpha=0.05):
    """Renvoie un tableau booléen des hypothèses rejetées (FDR contrôlé à alpha)."""
    p = np.asarray(p)
    m = len(p)
    ordre = np.argsort(p)
    ok = p[ordre] <= (np.arange(1, m + 1) / m) * alpha
    rejet = np.zeros(m, dtype=bool)
    if ok.any():
        rejet[ordre[: np.max(np.flatnonzero(ok)) + 1]] = True
    return rejet

```

Application aux deux variables :

```

lignes = []
resultats = {}
for nom in ["ventes_hab", "ventes_bruit"]:
    Ii, quad, pv = lisa(zones[nom].to_numpy(), W)
    rej = benjamini_hochberg(pv)
    resultats[nom] = (Ii, quad, pv, rej)
    comptes = pd.Series(np.where(rej, quad, "non significatif")).value_counts()
    lignes.append({"variable": nom, "somme_Ii/n": Ii.sum() / len(Ii), "I_global":
        moran(zones[nom].to_numpy(), W),
        "p<0.05 brut": int((pv < 0.05).sum()), "significatifs (BH)": int(rej.sum()),
        "HH": int(comptes.get("HH", 0)), "LL": int(comptes.get("LL", 0)),
        "HL": int(comptes.get("HL", 0)), "LH": int(comptes.get("LH", 0))})
with pd.option_context("display.float_format", "{:.4f}".format, "display.width", 170):
    print(pd.DataFrame(lignes).to_string(index=False))

```

variable	somme_Ii/n	I_global	p<0.05 brut	significatifs (BH)	HH	LL	HL	LH
ventes_hab	0.6207	0.6207	49	35	11	22	0	2
ventes_bruit	-0.0214	-0.0214	5	0	0	0	0	0

Étape 9 : la carte des zones significatives.

```

couleurs = {"HH": ROUGE, "LL": BLEU, "HL": ORANGE, "LH": AQUA, "ns": "#e8e6df"}
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.8))
for ax, nom in zip(axes, ["ventes_hab", "ventes_bruit"]):
    Ii, quad, pv, rej = resultats[nom]
    cat = np.where(rej, quad, "ns").reshape(12, 12)
    for i in range(12):
        for j in range(12):
            ax.add_patch(plt.Rectangle((j * 5, i * 5), 5, 5, facecolor=couleurs[cat[i, j]],
            edgecolor="white", linewidth=0.8))
    ax.set_xlim(0, 60); ax.set_ylim(0, 60); ax.set_aspect("equal")
    ax.set_title("ventes par habitant (structurées)" if nom == "ventes_hab" else "témoin (bruit sans
    structure)", fontsize=10)

```

```

    ax.set_xlabel("x (km)")
    axes[0].set_ylabel("y (km)")
    poignees = [plt.Rectangle((0, 0), 1, 1, facecolor=couleurs[k]) for k in ["HH", "LL", "HL", "LH",
    ↪ "ns"]]
    fig.legend(poignees, ["HH : haut entouré de haut", "LL : bas entouré de bas", "HL : haut entouré de
    ↪ bas", "LH : bas entouré de haut", "non significatif"],
        loc="lower center", ncol=5, frameon=False, fontsize=8)
    plt.tight_layout(rect=(0, 0.07, 1, 1))
    plt.savefig("figures/ch09-lisa.png", dpi=200, bbox_inches="tight")
    print("figure enregistrée")

```

figure enregistrée

Pour aller plus loin. Recommencez avec $\rho = 0,5$ puis $\rho = 0,95$: comment évoluent I , le nombre de zones significatives et l'écart entre z normal et p -valeur par permutations ?

Application 9.3 — Quand l'espace trompe la régression (section 9.2.5)

Objectif. Mesurer à quel point ignorer l'autocorrélation fausse un test, puis rétablir le niveau avec les moindres carrés généralisés (le modèle SAR rend le blanchiment explicite). On réutilise W , n , moran et $\text{moran_permutations}$ de l'application 9.2.

Étape 1 : deux champs sans aucun lien. 1 000 paires de champs SAR indépendants ($\rho = 0,9$) ; on régresse l'un sur l'autre par moindres carrés ordinaires et on teste la pente à 5 %. Résultat : 46 % de rejets au lieu de 5 %, et 5,5 observations indépendantes « efficaces » sur 144.

```

from scipy import stats

A = np.linalg.inv(np.eye(n) - 0.9 * W)                # (I - rho W)^-1, rho = 0,9
rng = np.random.default_rng(5)
reps = 1000
Y = A @ rng.normal(size=(n, reps))                    # 1000 champs « y », indépendants entre eux
X = A @ rng.normal(size=(n, reps))                    # 1000 champs « x », indépendants de y

# MCO vectorisée : pente et test de Student sur la pente, pour chacune des 1000 paires
xc, yc = X - X.mean(axis=0), Y - Y.mean(axis=0)
b = (xc * yc).sum(axis=0) / (xc ** 2).sum(axis=0)
res = yc - b * xc
s2 = (res ** 2).sum(axis=0) / (n - 2)
t = b / np.sqrt(s2 / (xc ** 2).sum(axis=0))
p_mco = 2 * stats.t.sf(np.abs(t), n - 2)
print("MCO : proportion de rejets à 5 % alors qu'il n'y a AUCUN lien :", round((p_mco < 0.05).mean(),
    ↪ 3))

```

MCO : proportion de rejets à 5 % alors qu'il n'y a AUCUN lien : 0.459

(suite du code)

```

# Nombre « efficace » d'observations indépendantes pour estimer une moyenne
var_y = Y.var()                                         # variance d'une zone
var_moy = Y.mean(axis=0).var()                         # variance de la moyenne des 144 zones
print("variance d'une zone :", round(var_y, 2), "| variance de la moyenne de 144 zones :",
    ↪ round(var_moy, 3),
    "| si indépendantes, ce serait", round(var_y / n, 3))
print("nombre efficace d'observations indépendantes :", round(var_y / var_moy, 1), "sur", n)

```

```

variance d'une zone : 3.9 | variance de la moyenne de 144 zones : 0.706 | si indépendantes, ce serait
    ↪ 0.027
nombre efficace d'observations indépendantes : 5.5 sur 144

```

Étape 2 : le remède. Les moindres carrés généralisés, avec ρ connu, ramènent le niveau à 5,3 % ;

l'indice de Moran des résidus de la régression ordinaire (0,425, $p = 0,001$) est le signal d'alarme à calculer dans une vraie analyse.

```
T = np.eye(n) - 0.9 * W # opérateur de blanchiment (rho connu)
un = np.ones(n)
rejets = 0
for r in range(reps):
    ys = T @ Y[:, r]
    Xs = np.column_stack([T @ un, T @ X[:, r]]) # constante et x, blanchis
    coef, *_ = np.linalg.lstsq(Xs, ys, rcond=None)
    resid = ys - Xs @ coef
    s2 = resid @ resid / (n - 2)
    cov = s2 * np.linalg.inv(Xs.T @ Xs)
    tt = coef[1] / np.sqrt(cov[1, 1])
    rejets += 2 * stats.t.sf(abs(tt), n - 2) < 0.05
print("MCG avec rho connu : proportion de rejets à 5 % :", rejets / reps)
```

MCG avec rho connu : proportion de rejets à 5 % : 0.053

(suite du code)

```
# Et dans une seule paire (la première) : les résidus des MCO révèlent le problème
Xd = np.column_stack([np.ones(n), X[:, 0]])
beta = np.linalg.lstsq(Xd, Y[:, 0], rcond=None)[0]
residus = Y[:, 0] - Xd @ beta
sim_r = moran_permutations(residus, W, B=999, seed=6)
print("Moran des résidus MCO (première paire) :", round(moran(residus, W), 3),
      "| p-valeur par permutation :", round((1 + (sim_r >= moran(residus, W)).sum()) / 1000, 3))
```

Moran des résidus MCO (première paire) : 0.425 | p-valeur par permutation : 0.001

Pour aller plus loin. Estimez ρ au lieu de le supposer connu (maximum de vraisemblance sur une grille de valeurs) et vérifiez que le niveau du test reste proche de 5 %.

Application 9.4 — Du variogramme au krigeage : les délais de livraison (section 9.3)

Objectif. Estimer le variogramme empirique, ajuster trois modèles, mesurer l'incertitude de l'ajustement, prédire par krigeage ordinaire avec sa variance, comparer à d'autres méthodes et valider par validation croisée.

Étape 1 : le variogramme empirique, validé sur quatre points alignés ($\hat{\gamma} = 1,0 ; 2,5 ; 2,0$ à la main).

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats, optimize

def distances(P, Q=None):
    Q = P if Q is None else Q
    return np.sqrt(((P[:, None, :] - Q[None, :, :]) ** 2).sum(axis=2))
```

L'estimateur par classes de distance, puis sa validation sur les quatre points :

```
def variogramme_empirique(coords, z, largeur=5.0, hmax=50.0, angle=None, tol=22.5):
    """Semi-variogramme empirique par classes de distance. Si `angle` (en degrés, 0 = est, 90 = nord)
    est donné, on ne garde que les paires orientées dans cette direction (à +/- tol degrés près)."""
    n = len(z)
    i, j = np.triu_indices(n, 1)
    h = distances(coords)[i, j]
```

```

g = 0.5 * (z[i] - z[j]) ** 2
garde = np.ones(len(h), dtype=bool)
if angle is not None:
    theta = np.degrees(np.arctan2(coords[j, 1] - coords[i, 1], coords[j, 0] - coords[i, 0])) % 180
    ecart = np.abs(theta - angle)
    garde = np.minimum(ecart, 180 - ecart) <= tol
classes = np.arange(0, hmax + largeur, largeur)
lignes = []
for a, b in zip(classes[:-1], classes[1:]):
    m = garde & (h > a) & (h <= b)
    if m.any():
        lignes.append({"h": h[m].mean(), "gamma": g[m].mean(), "paires": int(m.sum())})
return pd.DataFrame(lignes)

# Validation sur les quatre points de l'exemple à la main (classes de 1 km)
P4 = np.array([[0.0, 0], [1, 0], [2, 0], [3, 0]])
z4 = np.array([2.0, 3, 5, 4])
print(variogramme_empirique(P4, z4, largeur=1.0, hmax=3.0).to_string(index=False))

```

h	gamma	paires
1.0	1.0	3
2.0	2.5	2
3.0	2.0	1

Étape 2 : les 200 livraisons et leur générateur (champ gaussien de covariance exponentielle, pépite 0,4). Le résultat est identique au fichier fourni (moyenne 4,27 jours, variance 1,21).

```

def livraisons(seed=27, n=200, cote=100.0, sill=1.0, a=12.0, pepite=0.4, moyenne=4.0):
    rng = np.random.default_rng(seed)
    g = np.linspace(2, cote - 2, 25)
    gx, gy = np.meshgrid(g, g)
    grille = np.column_stack([gx.ravel(), gy.ravel()])
    obs = rng.uniform(0, cote, size=(n, 2))
    pts = np.vstack([obs, grille])
    C = sill * np.exp(-distances(pts) / a)
    ↪ exp(-h/a)
    champ = moyenne + np.linalg.cholesky(C + 1e-8 * np.eye(len(pts))) @ rng.normal(size=len(pts))
    z = champ[:n] + rng.normal(0, np.sqrt(pepite), n)
    ↪ # champ + bruit de mesure (la pépite)
    df = pd.DataFrame({"x": np.round(obs[:, 0], 2), "y": np.round(obs[:, 1], 2), "delai_jours":
    ↪ np.round(z, 3)})
    df["pli"] = rng.permutation(np.arange(n) % 5)
    verite = pd.DataFrame({"x": grille[:, 0], "y": grille[:, 1], "champ": champ[n:]})
    ↪ # 5 plis pour la validation croisée
    return df, verite

```

(suite du code)

```

liv, verite = livraisons()
fichier = pd.read_csv("donnees/ch09-livraisons.csv")
print("identique au fichier fourni :", np.allclose(liv[["x", "y", "delai_jours"]], fichier[["x", "y",
↪ "delai_jours"]]))
coords = liv[["x", "y"]].to_numpy()
z = liv["delai_jours"].to_numpy()
print(f"n = {len(z)} | moyenne = {z.mean():.2f} jours | variance = {z.var(ddof=1):.2f}")

```

```

identique au fichier fourni : True
n = 200 | moyenne = 4.27 jours | variance = 1.21

```

Étape 3 : le variogramme empirique des livraisons (classes de 5 km jusqu'à 50 km).

```

emp = variogramme_empirique(coords, z, largeur=5.0, hmax=50.0)
print(emp.round(3).to_string(index=False))

```

h	gamma	paires
3.328	0.473	139
7.817	0.784	384
12.655	1.057	683
17.572	1.265	849
22.550	1.250	992
27.537	1.190	1193
32.569	1.150	1278
37.539	1.211	1237
42.466	1.373	1332
47.408	1.426	1343

Étape 4 : trois modèles ajustés par moindres carrés pondérés (poids de Cressie). Les trois donnent un palier total voisin (1,28 à 1,32), mais des décompositions différentes entre pépite et portée.

```
def gamma_sph(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    g = np.where(h < a, c0 + c * (1.5 * h / a - 0.5 * (h / a) ** 3), c0 + c)
    return np.where(h > 0, g, 0.0)

def gamma_exp(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    return np.where(h > 0, c0 + c * (1 - np.exp(-h / a)), 0.0)

def gamma_gau(h, c0, c, a):
    h = np.asarray(h, dtype=float)
    return np.where(h > 0, c0 + c * (1 - np.exp(-(h / a) ** 2)), 0.0)
```

Les moyens d'ajuster :

```
MODELES = {"sphérique": gamma_sph, "exponentiel": gamma_exp, "gaussien": gamma_gau}
PORTEE_PRATIQUE = {"sphérique": lambda a: a, "exponentiel": lambda a: 3 * a, "gaussien": lambda a:
    ↪ np.sqrt(3) * a}

def ajuster(emp, modele, v0=None):
    """Ajuste (c0, c, a) par moindres carrés pondérés (poids de Cressie) ; renvoie les paramètres et
    ↪ le critère."""
    h, g, N = emp["h"].to_numpy(), emp["gamma"].to_numpy(), emp["paires"].to_numpy()
    v = g.max() if v0 is None else v0
    def residus(p):
        return np.sqrt(N) * (g / np.maximum(modele(h, *p), 1e-9) - 1)
    meilleur = None
    for a0 in (h.max() / 6, h.max() / 3, h.max() / 1.5):          # plusieurs départs : le critère
        ↪ n'est pas convexe
        r = optimize.least_squares(residus, x0=[0.1 * v, v, a0], bounds=([0, 1e-6, 1e-3], [10 * v, 10
        ↪ * v, 10 * h.max()]))
        if meilleur is None or r.cost < meilleur.cost:
            meilleur = r
    return meilleur.x, 2 * meilleur.cost
```

L'ajustement des trois modèles :

```
lignes, ajustements = [], {}
for nom, f in MODELES.items():
    p, crit = ajuster(emp, f)
    ajustements[nom] = p
    lignes.append({"modele": nom, "pepite": p[0], "palier_partiel": p[1], "palier_total": p[0] + p[1],
        "param_a": p[2], "portee_pratique_km": PORTEE_PRATIQUE[nom](p[2]), "critere": crit})
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))
```

modele	pepite	palier_partiel	palier_total	param_a	portee_pratique_km	critere
sphérique	0.215	1.065	1.280	20.792	20.792	48.096
exponentiel	0.038	1.277	1.315	8.136	24.407	47.196
gaussien	0.390	0.893	1.283	10.436	18.075	48.425

Étape 5 : la figure.

```
BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
hh = np.linspace(0.01, 50, 300)
fig, ax = plt.subplots(figsize=(7.4, 4.6))
ax.scatter(emp["h"], emp["gamma"], s=np.sqrt(emp["paires"]) * 3, color="#444444", zorder=3,
           ↪ label="empirique (taille = nb de paires)")
for (nom, f), couleur in zip(MODELES.items(), [AQUA, ORANGE, VIOLET]):
    ax.plot(hh, f(hh, *ajustements[nom]), color=couleur, lw=2, label=nom)
ax.axhline(z.var(ddof=1), color="#999999", lw=0.8, ls="--")
ax.annotate("variance empirique des données", (50, z.var(ddof=1)), xytext=(-4, 4), textcoords="offset
           ↪ points", ha="right", fontsize=8, color="#666666")
ax.set_xlabel("distance h (km)"); ax.set_ylabel("semi-variance γ(h) (jours²)")
ax.set_title("Variogramme empirique des délais de livraison et trois modèles ajustés")
ax.set_xlim(0, 52); ax.set_ylim(0, None)
ax.legend(frameon=False, loc="lower right", fontsize=8)
plt.tight_layout()
plt.savefig("figures/ch09-variogramme.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Étape 6 : cet ajustement est-il typique ? Trente autres graines : la pépite varie de 0,10 à 0,54 et la portée pratique de 16 à 52 km (vérités : 0,4 et 36 km).

```
rangs = []
for s in range(100, 130):
    d_s, _ = livraisons(seed=s)
    e_s = variogramme_empirique(d_s[["x", "y"]].to_numpy(), d_s["delai_jours"].to_numpy())
    p, _ = ajuster(e_s, gamma_exp)
    rangs.append({"pepite": p[0], "palier_partiel": p[1], "param_a": p[2], "portee_pratique": 3 *
           ↪ p[2], "palier_total": p[0] + p[1]})
rangs = pd.DataFrame(rangs)
verite_p = {"pepite": 0.4, "palier_partiel": 1.0, "param_a": 12.0, "portee_pratique": 36.0,
           ↪ "palier_total": 1.4}
resume = pd.DataFrame({"vérité": pd.Series(verite_p), "médiane": rangs.median(), "10e perc.":
           ↪ rangs.quantile(0.10), "90e perc.": rangs.quantile(0.90)})
print(resume.round(2).to_string())
```

	vérité	médiane	10e perc.	90e perc.
pepite	0.4	0.35	0.10	0.54
palier_partiel	1.0	1.04	0.78	1.38
param_a	12.0	9.64	5.29	17.33
portee_pratique	36.0	28.92	15.88	52.00
palier_total	1.4	1.38	1.05	1.63

Étape 7 : le krigeage ordinaire. La fonction résout le système bordé $\begin{pmatrix} \Gamma & \mathbf{1} \\ \mathbf{1}^\top & 0 \end{pmatrix}$; elle est testée sur l'exemple à la main (poids $\frac{2}{3}, \frac{1}{3}$, prédiction 12, variance $\frac{4}{3}$).

```
def krigeage_ordinaire(coords, z, cibles, modele, params):
    """Krigage ordinaire : renvoie prédictions, variances de krigeage et poids (une colonne par
    ↪ cible)."""
    n = len(z)
    G = modele(distances(coords), *params) # Gamma (diagonale nulle)
    A = np.zeros((n + 1, n + 1))
    A[:n, :n] = G
```

```

A[:, n] = 1.0
A[n, :] = 1.0
g0 = modele(distances(coords, cibles), *params) # gamma_0, une colonne par cible
B = np.vstack([g0, np.ones((1, cibles.shape[0]))])
sol = np.linalg.solve(A, B)
lam, m = sol[:n], sol[n]
pred = lam.T @ z
var = (lam * g0).sum(axis=0) + m
return pred, var, lam

lineaire = lambda h, pente: np.where(np.asarray(h) > 0, pente * np.asarray(h), 0.0)
pred, var, lam = krigage_ordinaire(np.array([[0.0, 0], [3, 0]]), np.array([10.0, 16.0]),
    ↪ np.array([[1.0, 0]]), lineaire, (1.0,))
print("poids :", lam.ravel().round(4), "| prédiction :", pred.round(4), "| variance :", var.round(4))

```

```
poids : [0.6667 0.3333] | prédiction : [12.] | variance : [1.3333]
```

Étape 8 : une prédiction au point (40, 60) : 4,75 jours, écart-type de krigage 0,77 ; 29 poids négatifs sur 200 (effet d'écran).

```

mod, par = gamma_exp, ajustements["exponentiel"]
cible = np.array([[40.0, 60.0]])
pred, var, lam = krigage_ordinaire(coords, z, cible, mod, par)
lam = lam.ravel()
dist = distances(coords, cible).ravel()
ordre = np.argsort(-np.abs(lam))[:6]
print(f"prédiction en (40, 60) : {pred[0]:.2f} jours | écart-type de krigage : {np.sqrt(var[0]):.2f}
    ↪ jour")
print("somme des poids :", round(lam.sum(), 6), "| poids négatifs :", int((lam < 0).sum()), "sur",
    ↪ len(lam),
    f"| plus petit : {lam.min():.3f} | plus grand : {lam.max():.3f}")
print(pd.DataFrame({"distance_km": dist[ordre], "poids": lam[ordre], "delai":
    ↪ z[ordre]}).round(3).to_string(index=False))

```

```

prédiction en (40, 60) : 4.75 jours | écart-type de krigage : 0.77 jour
somme des poids : 1.0 | poids négatifs : 29 sur 200 | plus petit : -0.026 | plus grand : 0.532
distance_km  poids  delai
      2.724    0.532   4.753
      4.740    0.158   4.849
      6.420    0.105   3.997
      8.405    0.102   5.156
     11.762    0.054   5.439
     11.745    0.032   3.196

```

Étape 9 : cartes de prédiction et d'incertitude. RMSE par rapport au vrai champ : moyenne 0,916 ; inverse de la distance 0,671 ; krigage 0,645.

```

grille = verite[["x", "y"]].to_numpy()
pred_k, var_k, _ = krigage_ordinaire(coords, z, grille, mod, par)

def idw(coords, z, cibles, puissance=2.0):
    d = np.maximum(distances(cibles, coords), 1e-9)
    w = 1.0 / d ** puissance
    return (w * z).sum(axis=1) / w.sum(axis=1)

pred_i = idw(coords, z, grille)
vrai = verite["champ"].to_numpy()
rmse = lambda a, b: float(np.sqrt(np.mean((a - b) ** 2)))
print("RMSE par rapport au VRAI champ sur les 625 points de la grille :")
print(f"  moyenne globale      : {rmse(np.full_like(vrai, z.mean()), vrai):.3f}")
print(f"  inverse de la distance^2 : {rmse(pred_i, vrai):.3f}")
print(f"  krigage ordinaire       : {rmse(pred_k, vrai):.3f}")
print(f"écart-type de krigage : de {np.sqrt(var_k.min()):.2f} à {np.sqrt(var_k.max()):.2f} jour
    ↪ (moyenne {np.sqrt(var_k).mean():.2f})")

```

RMSE par rapport au VRAI champ sur les 625 points de la grille :

```
moyenne globale      : 0.916
inverse de la distance^2 : 0.671
krigeage ordinaire    : 0.645
```

écart-type de krigeage : de 0.35 à 1.08 jour (moyenne 0.77)

```
fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.4))
vmin, vmax = min(vrai.min(), pred_k.min()), max(vrai.max(), pred_k.max())
cartes = [(vrai, "Vrai champ (connu par simulation)", "Oranges", dict(vmin=vmin, vmax=vmax)),
          (pred_k, "Krigeage : prédiction", "Oranges", dict(vmin=vmin, vmax=vmax)),
          (np.sqrt(var_k), "Krigeage : écart-type d'erreur", "Purples", {})]
for ax, (champ, titre, cmap, kw) in zip(axes, cartes):
    im = ax.imshow(champ.reshape(25, 25), origin="lower", extent=(0, 100, 0, 100), cmap=cmap, **kw)
    ax.set_title(titre, fontsize=10)
    ax.set_xlabel("x (km)")
    fig.colorbar(im, ax=ax, shrink=0.78)
    if "écart-type" in titre:
        ax.scatter(coords[:, 0], coords[:, 1], s=5, color="#222222", alpha=0.6)
axes[0].set_ylabel("y (km)")
plt.tight_layout()
plt.savefig("figures/ch09-krigeage.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Étape 10 : de quoi dépend l'incertitude ? L'écart-type de krigeage croît avec la distance à la mesure la plus proche (corrélation 0,92).

```
dmin = distances(grille, coords).min(axis=1) # distance au point mesuré le plus
↳ proche
sd_k = np.sqrt(var_k)
bord = np.minimum.reduce([grille[:, 0], 100 - grille[:, 0], grille[:, 1], 100 - grille[:, 1]])
print("corrélation entre l'écart-type de krigeage et la distance au point le plus proche :",
↳ round(np.corrcoef(sd_k, dmin)[0, 1], 3))
for a_, b_ in [(0, 2), (2, 4), (4, 6), (6, 10)]:
    m = (dmin >= a_) & (dmin < b_)
    print(f" points de grille à {a_}-{b_} km d'une mesure : {int(m.sum()):3d} points, écart-type
↳ moyen {sd_k[m].mean():.2f} jour")
print(f"écart-type moyen à moins de 8 km d'un bord : {sd_k[bord < 8].mean():.3f} | à l'intérieur :
↳ {sd_k[bord >= 8].mean():.3f}")
```

```
corrélation entre l'écart-type de krigeage et la distance au point le plus proche : 0.924
points de grille à 0-2 km d'une mesure : 145 points, écart-type moyen 0.59 jour
points de grille à 2-4 km d'une mesure : 252 points, écart-type moyen 0.77 jour
points de grille à 4-6 km d'une mesure : 173 points, écart-type moyen 0.87 jour
points de grille à 6-10 km d'une mesure : 54 points, écart-type moyen 0.97 jour
écart-type moyen à moins de 8 km d'un bord : 0.779 | à l'intérieur : 0.770
```

Étape 11 : validation croisée honnête (variogramme réajusté à chaque pli) : RMSE 1,097 ; 0,994 ; 0,969 ; variance des résidus standardisés 1,71 ; couverture 89,5 % au lieu de 95 %.

```
plis = liv["pli"].to_numpy()
pred_cv = {"moyenne": np.empty(len(z)), "IDW": np.empty(len(z)), "krigeage": np.empty(len(z))}
sig_cv = np.empty(len(z))
for k in range(5):
    tr, te = plis != k, plis == k
    emp_k = variogramme_empirique(coords[tr], z[tr])
    p_k, _ = ajuster(emp_k, gamma_exp) # variogramme réajusté sur
↳ l'entraînement
    pk, vk, _ = krigeage_ordinaire(coords[tr], z[tr], coords[te], gamma_exp, p_k)
    pred_cv["krigeage"][te] = pk
    sig_cv[te] = np.sqrt(vk)
```

```
pred_cv["IDW"][te] = idw(coords[tr], z[tr], coords[te])
pred_cv["moyenne"][te] = z[tr].mean()
```

Les trois méthodes, pli par pli :

```
tab = pd.DataFrame({nom: {"RMSE (jours)": rmse(p, z), "biais moyen": float(np.mean(p - z))} for nom, p
↪ in pred_cv.items()}).T
print(tab.round(3).to_string())
std_res = (z - pred_cv["krigeage"]) / sig_cv
couvert = np.mean(np.abs(z - pred_cv["krigeage"]) <= 1.96 * sig_cv)
print(f"\nrésidus standardisés du krigeage : moyenne = {std_res.mean():.3f}, variance =
↪ {std_res.var(ddof=1):.3f} (visé : 0 et 1)")
print(f"couverture des intervalles de krigeage à 95 % : {couvert:.3f}")
```

	RMSE (jours)	biais moyen
moyenne	1.097	0.000
IDW	0.994	0.040
krigeage	0.969	0.023

résidus standardisés du krigeage : moyenne = -0.021, variance = 1.710 (visé : 0 et 1)
couverture des intervalles de krigeage à 95 % : 0.895

Étape 12 : une alternative sans variogramme, la spline de plaque mince (R, mgcv) : RMSE 0,974, quasi identique au krigeage.

```
library(mgcv)
d <- read.csv("donnees/ch09-livraisons.csv")
pred <- numeric(nrow(d))
for (k in 0:4) {
  tr <- d[d$pli != k, ]
  te <- d[d$pli == k, ]
  g <- gam(delai_jours ~ s(x, y, k = 60), data = tr, method = "REML")
  pred[d$pli == k] <- predict(g, newdata = te)
}
cat("RMSE par validation croisée (spline de plaque mince, mgcv) :", round(sqrt(mean((d$delai_jours -
↪ pred)^2)), 3), "\n")
cat("biais moyen :", round(mean(pred - d$delai_jours), 3), "\n")
```

Loading required package: nlme
This is mgcv 1.9-1. For overview type 'help("mgcv-package")'.
RMSE par validation croisée (spline de plaque mince, mgcv) : 0.974
biais moyen : 0.009

Pour aller plus loin. Remplacez le modèle exponentiel par le sphérique dans la validation croisée : l'erreur change-t-elle ? Et la couverture des intervalles ?

Application 9.5 — Les limites du variogramme : tendance et anisotropie (section 9.3.7)

Objectif. Voir une tendance faire « monter » le variogramme, la retirer par régression, puis calibrer par simulation un écart apparent entre deux directions.

Étape 1 : une tendance de 0,04 jour/km vers l'est. La régression retrouve une pente de 0,042 (vérité 0,04) ; une fois retirée, la courbe redevient celle du processus sans tendance.

```
z_tend = z + 0.04 * coords[:, 0] # tendance ajoutée : +0,04 jour/km
↪ vers l'est
X = np.column_stack([np.ones(len(z)), coords])
beta = np.linalg.lstsq(X, z_tend, rcond=None)[0]
z_res = z_tend - X @ beta # résidus de la régression sur (x, y)
e_tend = variogramme_empirique(coords, z_tend, largeur=5.0, hmax=70.0)
e_res = variogramme_empirique(coords, z_res, largeur=5.0, hmax=70.0)
```

```
e_ref = variogramme_empirique(coords, z, largeur=5.0, hmax=70.0)
print("tendance estimée par régression : ", beta.round(3), "(constante, pente en x, pente en y)")
print(pd.DataFrame({"h": e_ref["h"].round(1), "sans_tendance": e_ref["gamma"], "avec_tendance":
    ↪ e_tend["gamma"],
    "tendance_retiree": e_res["gamma"]})).round(2).iloc[[1, 3, 5, 7, 9, 11,
    ↪ 13]].to_string(index=False))
```

```
tendance estimée par régression : [ 4.325e+00  4.200e-02 -3.000e-03] (constante, pente en x, pente en
    ↪ y)
   h  sans_tendance  avec_tendance  tendance_retiree
7.8      0.78         0.84         0.78
17.6     1.27         1.41         1.27
27.5     1.19         1.50         1.19
37.5     1.21         1.79         1.22
47.4     1.43         2.36         1.42
57.4     1.35         2.64         1.35
67.4     1.21         3.29         1.19
```

Étape 2 : la figure (tendance, puis variogrammes directionnels est-ouest et nord-sud).

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))
ax1.plot(e_ref["h"], e_ref["gamma"], "o-", color=AQUA, label="sans tendance")
ax1.plot(e_tend["h"], e_tend["gamma"], "o-", color=ROUGE, label="avec une tendance linéaire non
    ↪ retirée")
ax1.plot(e_res["h"], e_res["gamma"], "o-", color=BLEU, label="tendance retirée par régression")
ax1.set_xlabel("distance h (km)"); ax1.set_ylabel("semi-variance (jours²)"); ax1.set_title("Une
    ↪ tendance fait « monter » le variogramme")
ax1.legend(frameon=False, fontsize=8, loc="upper left")
for angle, couleur, nom in [(0, ORANGE, "est-ouest (0°)", (90, VIOLET, "nord-sud (90°)"]):
    e_dir = variogramme_empirique(coords, z, largeur=7.5, hmax=45.0, angle=angle, tol=22.5)
    ax2.plot(e_dir["h"], e_dir["gamma"], "o-", color=couleur, label=nom)
e_om = variogramme_empirique(coords, z, largeur=7.5, hmax=45.0)
ax2.plot(e_om["h"], e_om["gamma"], "--", color="#444444", label="toutes directions")
ax2.set_xlabel("distance h (km)"); ax2.set_ylabel("semi-variance (jours²)");
    ↪ ax2.set_title("Variogrammes directionnels")
ax2.legend(frameon=False, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("figures/ch09-limites-variogramme.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

Étape 3 : calibrer l'écart apparent. Le rapport $\gamma_{N-S}/\gamma_{E-O}$ observé (1,24) est comparé à celui de 60 jeux simulés isotropes : environ 5 % l'atteignent. Cas limite, qu'on ne tranche pas sur un graphique.

```
def rapport_ns_eo(c, zz):
    ns_ = variogramme_empirique(c, zz, largeur=7.5, hmax=45.0, angle=90, tol=22.5)["gamma"].to_numpy()
    eo_ = variogramme_empirique(c, zz, largeur=7.5, hmax=45.0, angle=0, tol=22.5)["gamma"].to_numpy()
    m_ = min(len(ns_), len(eo_))
    return float(np.mean(ns_[:m_] / eo_[:m_]))

r_obs = rapport_ns_eo(coords, z)
r_sim = []
for s_ in range(100, 160):
    d_s, _ = livraisons(seed=s_)
    r_sim.append(rapport_ns_eo(d_s[["x", "y"]].to_numpy(), d_s["delai_jours"].to_numpy()))
r_sim = np.array(r_sim)
print(f"rapport N-S / E-O observé : {r_obs:.3f}")
print(f"rapport sur 60 jeux isotropes : médiane {np.median(r_sim):.3f}, 10e-90e percentiles
    ↪ {np.percentile(r_sim, 10):.3f} - {np.percentile(r_sim, 90):.3f}")
print(f"part des jeux isotropes dont le rapport est au moins aussi grand : {np.mean(r_sim >=
    ↪ r_obs):.3f}")
```

```

rapport N-S / E-O observé : 1.243
rapport sur 60 jeux isotropes : médiane 1.020, 10e-90e percentiles 0.908 - 1.167
part des jeux isotropes dont le rapport est au moins aussi grand : 0.050

```

Pour aller plus loin. Doublez le nombre de graines (120 jeux) et suivez la part de jeux isotropes au moins aussi extrêmes : se stabilise-t-elle ?

Application 9.6 — Trois semis de points : quadrats, plus proche voisin, fonction K (section 9.4)

Objectif. Fabriquer trois semis (hasard complet, agrégat de Thomas, régulier), les distinguer par le test des quadrats, l'indice de Clark-Evans (avec correction de bord de Donnelly et p-valeur de Monte-Carlo), la fonction G et la fonction K de Ripley avec enveloppes.

Étape 1 : les trois semis (fenêtre de 10 km de côté ; 100, 75 et 100 points).

```

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

COTE = 10.0 # fenêtre carrée de 10 km de côté
AIRE = COTE ** 2 # 100 km²

```

Les trois générateurs :

```

def distances(P, Q=None):
    Q = P if Q is None else Q
    return np.sqrt(((P[:, None, :] - Q[None, :, :]) ** 2).sum(axis=2))

def semis_poisson(n, rng):
    """CSR conditionnel à n : n points indépendants et uniformes."""
    return rng.uniform(0, COTE, size=(n, 2))

def semis_thomas(kappa, mu, sigma, rng):
    """Agrégat de Thomas : centres de Poisson (densité kappa), mu descendants en moyenne, étalement
    ↪ sigma."""
    marge = 3 * sigma # on génère aussi des centres hors fenêtre
    n_centres = rng.poisson(kappa * (COTE + 2 * marge) ** 2)
    centres = rng.uniform(-marge, COTE + marge, size=(n_centres, 2))
    nb = rng.poisson(mu, size=n_centres)
    pts = np.repeat(centres, nb, axis=0) + rng.normal(0, sigma, size=(nb.sum(), 2))
    return pts[((pts >= 0) & (pts <= COTE)).all(axis=1)] # on ne garde que ce qui tombe dans
    ↪ la fenêtre

```

Le tirage et la figure :

```

def semis_inhibition(n, rmin, rng):
    """Inhibition séquentielle simple : points uniformes refusés s'ils sont à moins de rmin d'un point
    ↪ accepté."""
    pts = []
    while len(pts) < n:
        p = rng.uniform(0, COTE, size=2)
        if not pts or (np.hypot(*(np.array(pts) - p).T) >= rmin).all():
            pts.append(p)
    return np.array(pts)

rng = np.random.default_rng(2025)
semis = {
    "aléatoire (CSR)": semis_poisson(100, rng),

```

```

    "agrégé (Thomas)": semis_thomas(kappa=0.08, mu=12, sigma=0.4, rng=rng),
    "régulier (inhibition)": semis_inhibition(100, 0.7, rng),
}
for nom, pts in semis.items():
    print(f"{nom:24s} n = {len(pts):3d} points (intensité estimée {len(pts) / AIRE:.2f} par km²)")

```

aléatoire (CSR)	n = 100 points	(intensité estimée 1.00 par km²)
agrégé (Thomas)	n = 75 points	(intensité estimée 0.75 par km²)
régulier (inhibition)	n = 100 points	(intensité estimée 1.00 par km²)

(suite de la figure)

```

BLEU, ORANGE, AQUA, VIOLET, ROUGE = "#2a78d6", "#eb6834", "#1baf7a", "#4a3aa7", "#e34948"
fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.3))
for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
    ax.scatter(pts[:, 0], pts[:, 1], s=16, color=couleur, edgecolor="white", linewidth=0.4)
    ax.set_xlim(0, COTE); ax.set_ylim(0, COTE); ax.set_aspect("equal")
    ax.set_title(f"{nom} : {len(pts)} adresses", fontsize=10)
    ax.set_xlabel("x (km)")
axes[0].set_ylabel("y (km)")
plt.tight_layout()
plt.savefig("figures/ch09-semis.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Étape 2 : le test des quadrats. $\chi^2 = 8,8$ (hasard), 143,7 (agrégé), 4,0 (régulier) pour 15 degrés de liberté.

```

def comptages(pts, m=4):
    """Comptages par case d'un quadrillage m x m de la fenêtre."""
    bornes = np.linspace(0, COTE, m + 1)
    H, _, _ = np.histogram2d(pts[:, 0], pts[:, 1], bins=[bornes, bornes])
    return H.ravel()

def test_quadrats(pts, m=4):
    c = comptages(pts, m)
    chi2 = ((c - c.mean()) ** 2).sum() / c.mean()
    ddl = len(c) - 1
    return {"chi2": chi2, "ddl": ddl, "VMR": c.var(ddof=1) / c.mean(),
            "p_agregat (chi2 grand)": stats.chi2.sf(chi2, ddl), "p_regulier (chi2 petit)":
            stats.chi2.cdf(chi2, ddl)}

```

(suite du code)

```

# l'exemple à la main
print("exemple (1, 1, 1, 9) :", round(((np.array([1, 1, 1, 9]) - 3) ** 2).sum() / 3, 2), "| valeur
↳ critique chi2(3) à 5 % :", round(stats.chi2.ppf(0.95, 3), 2))
res = pd.DataFrame({nom: test_quadrats(pts) for nom, pts in semis.items()}).T
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 160):
    print(res.to_string())
print("\ncomptages par case du semis agrégé (4 x 4) :")
print(comptages(semis["agrégé (Thomas)"]).reshape(4, 4).astype(int))

```

exemple (1, 1, 1, 9) :	16.0	valeur critique chi2(3) à 5 % :	7.81
	chi2	ddl	VMR
aléatoire (CSR)	8.8	15	0.5867
agrégé (Thomas)	143.7	15	9.578
régulier (inhibition)	4	15	0.2667
	p_agregat (chi2 grand)	p_regulier (chi2 petit)	
	0.8877	0.1123	
	4.342e-23	1	
	0.9977	0.002263	

```

comptages par case du semis agrégé (4 x 4) :
[[ 1  0  1 11]
 [19  0  0  0]]

```

```
[12  2 14  1]
[ 0  0  0 14]]
```

Étape 3 : l'effet de bord sur le plus proche voisin. La formule naïve sous-estime la moyenne (0,500 contre 0,523 simulé) ; celle de Donnelly colle à la simulation.

```
def plus_proches(pts):
    D = distances(pts)
    np.fill_diagonal(D, np.inf)
    return D.min(axis=1)

rng = np.random.default_rng(1)
n0 = 100
dbar = np.array([plus_proches(semis_poisson(n0, rng)).mean() for _ in range(4000)])
P = 4 * COTE
naif = 0.5 * np.sqrt(AIRE / n0)
don = 0.5 * np.sqrt(AIRE / n0) + (0.0514 + 0.041 / np.sqrt(n0)) * P / n0
sd_naif = np.sqrt((4 - np.pi) / (4 * np.pi * n0 * (n0 / AIRE)))
sd_don = np.sqrt(0.070 * AIRE / n0 ** 2 + 0.037 * P * np.sqrt(AIRE / n0 ** 5))
print(f"{'':28s}{'moyenne de d':>14s}{'écart-type de d':>18s}")
print(f"{'simulation (4000 semis CSR)':28s}{dbar.mean():14.4f}{dbar.std():18.4f}")
print(f"{'formule naïve (plan infini)':28s}{naif:14.4f}{sd_naif:18.4f}")
print(f"{'formule de Donnelly':28s}{don:14.4f}{sd_don:18.4f}")
```

	moyenne de d	écart-type de d
simulation (4000 semis CSR)	0.5224	0.0288
formule naïve (plan infini)	0.5000	0.0261
formule de Donnelly	0.5222	0.0291

Étape 4 : l'indice de Clark-Evans, avec trois voies : R naïf, R corrigé et p-valeur de Monte-Carlo.

```
def clark_evans(pts, B=999, seed=7):
    n = len(pts)
    d = plus_proches(pts).mean()
    R_naif = d / (0.5 * np.sqrt(AIRE / n))
    E_don = 0.5 * np.sqrt(AIRE / n) + (0.0514 + 0.041 / np.sqrt(n)) * P / n
    s_don = np.sqrt(0.070 * AIRE / n ** 2 + 0.037 * P * np.sqrt(AIRE / n ** 5))
    z_don = (d - E_don) / s_don
    rng = np.random.default_rng(seed)
    sim = np.array([plus_proches(semis_poisson(n, rng)).mean() for _ in range(B)])
    p_mc = 2 * (1 + min((sim <= d).sum(), (sim >= d).sum())) / (B + 1) # bilatérale
    return {"n": n, "d_moyen": d, "R (naïf)": R_naif, "R (Donnelly)": d / E_don, "z (Donnelly)": z_don,
            "p (Donnelly)": 2 * stats.norm.sf(abs(z_don)), "p (Monte-Carlo)": min(1.0, p_mc)}

res = pd.DataFrame({nom: clark_evans(pts) for nom, pts in semis.items()}).T
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 170):
    print(res.to_string())
```

	n	d_moyen	R (naïf)	R (Donnelly)	z (Donnelly)	p (Donnelly)	p (Monte-Carlo)
aléatoire (CSR)	100	0.5353	1.071	1.025	0.4506	0.6523	0.628
agrégé (Thomas)	75	0.2026	0.3509	0.3336	-10.28	8.252e-25	0.002
régulier (inhibition)	100	0.829	1.658	1.587	10.53	6.024e-26	0.002

Étape 5 : la fonction G et son enveloppe de Monte-Carlo (499 semis aléatoires de même taille).

```
def G_emp(pts, rs):
    d = plus_proches(pts)
    return np.array([(d <= r).mean() for r in rs])

rs_G = np.linspace(0, 1.6, 81)
rng = np.random.default_rng(11)
fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.9), sharey=True)
lignes = []
```

```

for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
    n = len(pts)
    sims = np.array([G_emp(semis_poisson(n, rng), rs_G) for _ in range(499)])
    bas, haut = np.percentile(sims, [2.5, 97.5], axis=0)
    theorique = 1 - np.exp(-(n / AIRE) * np.pi * rs_G ** 2)
    ax.fill_between(rs_G, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %)")
    ax.plot(rs_G, theorique, color="#555555", lw=1, ls="--", label="théorie (plan infini)")
    ax.plot(rs_G, G_emp(pts, rs_G), color=couleur, lw=2.2, label="observé")
    ax.set_title(nom, fontsize=10); ax.set_xlabel("distance r (km)")
    sortie = (G_emp(pts, rs_G) < bas) | (G_emp(pts, rs_G) > haut)
    lignes.append({"semis": nom, "part des r hors enveloppe": sortie.mean()})
axes[0].set_ylabel("G(r)")
axes[0].legend(frameon=False, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("figures/ch09-fonction-G.png", dpi=200, bbox_inches="tight")
print(pd.DataFrame(lignes).round(3).to_string(index=False))

```

	semis	part des r hors enveloppe
aléatoire (CSR)		0.000
agréé (Thomas)		0.741
régulier (inhibition)		0.531

Étape 6 : la fonction K de Ripley : estimateur avec correction de bord (méthode du bord), contrôle sur l'exemple à la main (0,333), puis mesure du biais de bord (10,4 sans correction, 12,3 avec, 12,6 en théorie à $r = 2$ km).

```

def ripley_K(pts, rs, bord=True):
    """Estimateur de K, avec ou sans correction de bord (méthode du bord)."""
    n = len(pts)
    D = distances(pts)
    np.fill_diagonal(D, np.inf)
    b = np.minimum.reduce([pts[:, 0], COTE - pts[:, 0], pts[:, 1], COTE - pts[:, 1]])
    K = np.full(len(rs), np.nan)
    for k, r in enumerate(rs):
        garde = (b >= r) if bord else np.ones(n, dtype=bool)
        nr = garde.sum()
        if nr > 0:
            K[k] = AIRE * (D[garde] <= r).sum() / ((n - 1) * nr)
    return K

```

(suite du code)

```

# Contrôle sur l'exemple à la main (fenêtre unité) : on neutralise le bord pour retrouver 0,333
P4 = np.array([[0.1, 0.1], [0.2, 0.1], [0.8, 0.8], [0.85, 0.9]])
D4 = distances(P4); np.fill_diagonal(D4, np.inf)
print("K(0,2) de l'exemple à la main :", round(1.0 * (D4 <= 0.2).sum() / (4 * 3), 4), "| pi r^2 =",
      round(np.pi * 0.2 ** 2, 4))

# L'effet de bord, mesuré : moyenne de K sur 300 semis CSR, avec et sans correction, à r = 2 km
rs = np.linspace(0.1, 2.0, 20)
rng = np.random.default_rng(3)
Kn, Kc = [], []
for _ in range(300):
    pts_sim = semis_poisson(100, rng)
    Kn.append(ripley_K(pts_sim, rs, bord=False))
    Kc.append(ripley_K(pts_sim, rs, bord=True))
Kn, Kc = np.array(Kn), np.array(Kc)
print(f"à r = 2 km -> pi r^2 = {np.pi * 4:.2f} | moyenne sans correction : {Kn[:, -1].mean():.2f} |
      ↪ moyenne avec correction : {np.nanmean(Kc[:, -1]):.2f}")

```

K(0,2) de l'exemple à la main : 0.3333 | $\pi r^2 = 0.1257$
à $r = 2$ km -> $\pi r^2 = 12.57$ | moyenne sans correction : 10.42 | moyenne avec correction : 12.30

Étape 7 : $L(r) - r$, enveloppes et test global ($T = \max_r |\hat{L}(r) - r|$).

```
def L_centre(pts, rs):
    return np.sqrt(ripley_K(pts, rs) / np.pi) - rs

rng = np.random.default_rng(21)
fig, axes = plt.subplots(1, 3, figsize=(12.5, 3.9), sharey=True)
lignes = []
```

La boucle : une enveloppe, une statistique T et une courbe par semis :

```
for ax, (nom, pts), couleur in zip(axes, semis.items(), [BLEU, ORANGE, AQUA]):
    n = len(pts)
    sims = np.array([L_centre(semis_poisson(n, rng), rs) for _ in range(499)])
    bas, haut = np.nanpercentile(sims, [2.5, 97.5], axis=0)
    obs = L_centre(pts, rs)
    T_obs = np.nanmax(np.abs(obs))
    T_sim = np.nanmax(np.abs(sims), axis=1)
    lignes.append({"semis": nom, "T observé": T_obs, "T sim. (médiane)": np.median(T_sim),
                  "p global (Monte-Carlo)": (1 + (T_sim >= T_obs).sum()) / (len(T_sim) + 1),
                  "L-r moyen": np.nanmean(obs)})
    ax.fill_between(rs, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %, point par point)")
    ax.axhline(0, color="#555555", lw=1, ls="--")
    ax.plot(rs, obs, color=couleur, lw=2.2, label="observé")
    ax.set_title(nom, fontsize=10); ax.set_xlabel("distance r (km)")
```

Les légendes, puis le résumé :

```
axes[0].set_ylabel("L(r) - r (0 = hasard complet)")
axes[0].legend(frameon=False, fontsize=8, loc="lower left")
plt.tight_layout()
plt.savefig("figures/ch09-ripley.png", dpi=200, bbox_inches="tight")
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 170):
    print(pd.DataFrame(lignes).to_string(index=False))
```

	semis	T observé	T sim. (médiane)	p global (Monte-Carlo)	L-r moyen
	aléatoire (CSR)	0.092	0.1002	0.672	-0.03473
	agrégé (Thomas)	1.095	0.133	0.002	0.7666
	régulier (inhibition)	0.7	0.1023	0.002	-0.2289

Pour aller plus loin. Variez l'étalement σ de l'agrégat (0,2 ; 0,4 ; 0,8 km) et relevez l'échelle r où $L(r) - r$ est maximal : suit-elle σ ?

Application 9.7 — Agrégat ou densité variable ? (section 9.4.5)

Objectif. Montrer qu'une densité qui varie, sans aucune interaction entre les points, fait rejeter le hasard complet. On réutilise les fonctions de l'application 9.6.

Étape 1 : cent adresses indépendantes, avec une densité forte au centre. Le test global rejette le hasard complet ($T = 0,687$, $p = 0,002$) alors qu'aucun point n'attire les autres.

```
def semis_inhomogene(n, rng, etalement=2.5):
    """100 points indépendants, mais avec une densité qui décroît du centre vers les bords."""
    pts = []
    while len(pts) < n:
        p = rng.uniform(0, COTE, size=2)
        if rng.random() < np.exp(-((p - COTE / 2) ** 2).sum() / (2 * etalement ** 2)):
            pts.append(p)
    return np.array(pts)

rng = np.random.default_rng(31)
inho = semis_inhomogene(100, rng)
```

```

obs = L_centre(inho, rs)
sims = np.array([L_centre(semis_poisson(100, rng), rs) for _ in range(499)])
T_obs, T_sim = np.nanmax(np.abs(obs)), np.nanmax(np.abs(sims), axis=1)
print(f"semis à densité variable, SANS interaction : T = {T_obs:.3f} | p global CSR = {(1 + (T_sim >=
↪ T_obs).sum()) / 500:.3f}")
q = comptages(inho, 4).reshape(4, 4)
print("comptages par case (4 x 4) :")
print(q.astype(int))

```

```

semis à densité variable, SANS interaction : T = 0.687 | p global CSR = 0.002
comptages par case (4 x 4) :
[[ 3  8  2  1]
 [ 4 13 15  7]
 [ 5 10 11  6]
 [ 1  8  5 11]]

```

Le test et le dessin :

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
ax1.scatter(inho[:, 0], inho[:, 1], s=16, color=VIOLET, edgecolor="white", linewidth=0.4)
ax1.set_xlim(0, COTE); ax1.set_ylim(0, COTE); ax1.set_aspect("equal")
ax1.set_title("100 adresses indépendantes, densité forte au centre", fontsize=10); ax1.set_xlabel("x
↪ (km)"); ax1.set_ylabel("y (km)")
bas, haut = np.nanpercentile(sims, [2.5, 97.5], axis=0)
ax2.fill_between(rs, bas, haut, color="#cfcfc8", alpha=0.8, label="enveloppe CSR (95 %)")
ax2.axhline(0, color="#555555", lw=1, ls="--")
ax2.plot(rs, obs, color=VIOLET, lw=2.2, label="observé")
ax2.set_xlabel("distance r (km)"); ax2.set_ylabel("L(r) - r"); ax2.set_title("La fonction K voit un «
↪ agrégat » qui n'en est pas un", fontsize=10)
ax2.legend(frameon=False, fontsize=8, loc="upper left")
plt.tight_layout()
plt.savefig("figures/ch09-inhomogene.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

Pour aller plus loin. Divisez la fenêtre en deux moitiés de densités différentes mais homogènes : le test de K sur chaque moitié rejette-t-il encore le hasard ? Que concluez-vous sur l'échelle d'analyse ?

Exercices

Cherchez d'abord à la main (ou sur papier), vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les corrigés réutilisent les fonctions définies dans les applications 9.1 à 9.7 ci-dessus (haversine, moran, variogramme_empirique, krigeage_ordinaire, semis_thomas, riple_K...).

Exercice 9.1 ★ — quel type de données ? (section 9.1.2 du livre)

Pour chaque situation, dites s'il s'agit de données **géostatistiques**, **surfaciennes** ou d'un **semis de points**, et nommez l'outil de ce chapitre qui répond à la question. (a) la gérante relève la **température** à l'intérieur de 40 entrepôts de stockage répartis dans la région et veut estimer la température dans un entrepôt qu'elle n'a pas visité. (b) Elle dispose du **taux de retour de colis** de chacun des 24 départements de la région et se demande si les départements voisins ont des taux semblables. (c) Elle a les **adresses** de tous ses clients d'un même quartier et se demande

si elles se regroupent. (d) Un transporteur mesure le **délai de livraison** à 300 adresses et veut cartographier le délai moyen attendu sur toute la zone.

Exercice 9.2 ★ — haversine à la main (section 9.1.3 du livre)

Calculez à la main la distance entre la Ville C (46,40°N; 3,60°E) et la Ville D (45,80°N; 4,90°E) de la région fictive, en utilisant la latitude moyenne pour le facteur $\cos \varphi$. Comparez au résultat de la formule de haversine. ### Exercice 9.3 ★★ — Moran sur quatre zones (section 9.2.2 du livre)

Quatre zones alignées A–B–C–D (chacune voisine de la précédente et de la suivante) ont pour valeurs 1, 2, 3, 4. (a) Calculez à la main l'indice de Moran avec les poids binaires, puis avec les poids standardisés par ligne. (b) Quelle est son espérance sous l'hypothèse nulle ? (c) Il n'y a que $4! = 24$ façons de ranger ces valeurs sur les zones : calculez la distribution **exacte** de I et la p-valeur de l'alignement observé.

Exercice 9.4 ★★ — lire un indice local (section 9.2.4 du livre)

Dans une carte, la zone i a un écart à la moyenne $z_i = +12$ et la moyenne des écarts de ses voisines vaut $(Wz)_i = -8$. La variance empirique est $m_2 = 100$. (a) Dans quel quadrant se trouve la zone ? (b) Que vaut son indice local I_i ? (c) Peut-on conclure que c'est une valeur atypique significative ? Que faudrait-il faire ?

Exercice 9.5 ★★ — variogramme à la main (section 9.3.2 du livre)

Cinq mesures alignées aux abscisses 0, 1, 2, 3 et 4 km ont pour valeurs 3, 5, 4, 8, 7. Calculez le variogramme empirique aux distances 1, 2, 3 et 4 km. Que pouvez-vous dire de la forme de la courbe et de la fiabilité de ces estimations ?

Exercice 9.6 ★★ — krigeage à la main (section 9.3.4 du livre)

En dimension 1, avec le variogramme linéaire $\gamma(h) = 0,5h$, on a mesuré 20 en $x = 0$ et 30 en $x = 4$. (a) Écrivez et résolvez le système de krigeage ordinaire pour prédire en $x = 1$; donnez la prédiction et la variance. (b) Si les valeurs mesurées étaient 0 et 100 au lieu de 20 et 30, la variance de krigeage changerait-elle ? Pourquoi ?

Exercice 9.7 ★★ — le problème de l'unité spatiale modifiable (section 9.1.4 du livre)

On construit une variable x corrélée aux ventes_{hab} des 144 zones (la graine est donnée dans le corrigé). On **agrège** ensuite les zones en blocs de 2×2 , 3×3 , 4×4 et 6×6 cases (moyenne par bloc). Comment évoluent la corrélation entre x et les ventes, l'écart-type des ventes et l'indice de Moran ? Qu'en conclure pour l'interprétation d'une corrélation calculée sur des zones ?

Exercice 9.8 ★★ — l'effet de bord sur les indices locaux (section 9.2.4 du livre)

Sur la grille 12×12 avec voisinage « reine », les cases ont 3 voisines (coins), 5 (bords) ou 8 (intérieur). Sans autocorrélation, la variance de l'indice local I_i devrait varier comme $1/k$ où k est le nombre de voisines. Vérifiez-le par simulation et expliquez pourquoi les zones de bord sont plus souvent « significatives » avec une p-valeur naïve.

Exercice 9.9 ★★ — nombre efficace d'observations (section 9.2.5 du livre)

Pour un modèle SAR sur la grille 12×12 avec $\rho \in \{0,3; 0,5; 0,7; 0,9; 0,95\}$, estimez par simulation le **nombre efficace d'observations indépendantes** pour estimer une moyenne, défini par $n_{\text{eff}} = \text{Var}(y_i) / \text{Var}(\bar{y})$. Commentez.

Exercice 9.10 ★★ — cases vides (section 9.4.2 du livre)

On répartit 100 points au hasard (CSR) dans une fenêtre de 10×10 km, divisée en 4×4 cases. (a) Quelle est, à la main, la probabilité qu'une case donnée soit vide, et le nombre moyen de cases vides ? (b) Vérifiez par simulation, puis comparez au nombre de cases vides du semis agrégé de la section 9.4. Que cela dit-il ?

Exercice 9.11 ★★★ — l'estimateur du variogramme est-il sans biais ? (section 9.3.2 du livre)

Le variogramme empirique est une moyenne de $\frac{1}{2}(z_i - z_j)^2$. Montrez que $\mathbb{E}[\frac{1}{2}(Z(s+h) - Z(s))^2] = \gamma(h)$ pour un processus stationnaire de moyenne μ , puis vérifiez par simulation, en moyennant le variogramme empirique de 60 jeux de livraisons indépendants (même processus que 9.3) et en le comparant au variogramme **vrai** $\gamma(h) = 0,4 + 1,0(1 - e^{-h/12})$.

Exercice 9.12 ★★★ — la fonction K d'un processus de Thomas (section 9.4.4 du livre)

Pour un processus de Thomas de densité de centres κ et d'étalement σ (par coordonnée), on admet que la fonction de corrélation de paires est $g(r) = 1 + \frac{1}{4\pi\kappa\sigma^2}e^{-r^2/(4\sigma^2)}$. (a) Montrez que $K(r) = \pi r^2 + \frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$. (b) Vérifiez par simulation pour $\kappa = 0,5$, $\mu = 2$, $\sigma = 0,4$, puis pour $\kappa = 0,08$, $\mu = 12$, $\sigma = 0,4$. Que constatez-vous ?

Corrigés

Corrigé 9.1

- (a) **Géostatistique** : la température existe en tout point de la région et on la mesure en 40 lieux ; on veut prédire ailleurs : variogramme et **krigeage**. (b) **Surfacique** : une valeur par zone, la question est la ressemblance entre zones voisines : matrice de poids et **indice de Moran** (avec 24 zones seulement, on teste par permutations et on indique la définition du voisinage). (c) **Semis de points** : ce sont les positions qui sont le phénomène ; on les compare au hasard complet avec les quadrats, le plus proche voisin et la fonction K de **Ripley**, en pensant à la densité de population qui peut varier (9.4.5). (d) **Géostatistique** : un délai défini en tout point mesuré en 300 adresses ; **variogramme et krigeage** donnent la carte du délai attendu et une carte d'incertitude.

Corrigé 9.2

$\Delta\phi = 0,60^\circ$ donne $0,60 \times 111,2 \approx 66,7$ km vers le sud. $\Delta\lambda = 1,30^\circ$: avec $\cos(46,1^\circ) \approx 0,693$, $1,30 \times 111,2 \times 0,693 \approx 100,2$ km vers l'est. La distance est $\sqrt{66,7^2 + 100,2^2} \approx 120,4$ km.

```
print("haversine C - D      :", round(float(haversine(46.40, 3.60, 45.80, 4.90)), 1), "km")
print("estimation à la main :", round(float(np.hypot(0.60 * 111.2, 1.30 * 111.2 *
↳ np.cos(np.radians(46.1)))), 1), "km")
```

```
haversine C - D      : 120.4 km
estimation à la main : 120.4 km
```

Corrigé 9.3

- (a) Les écarts à la moyenne $\bar{y} = 2,5$ sont $z = (-1,5; -0,5; 0,5; 1,5)$ et $\sum z^2 = 5$. Les frontières sont A-B, B-C et C-D : produits $(-1,5)(-0,5) = 0,75$, $(-0,5)(0,5) = -0,25$, $(0,5)(1,5) = 0,75$, de somme 1,25. **Poids binaires** ($S_0 = 6$ liens orientés) : $\sum_{ij} w_{ij} z_i z_j = 2 \times 1,25 = 2,5$ et $I = \frac{4}{6} \times \frac{2,5}{5} = \frac{1}{3}$. **Poids standardisés** ($S_0 = 4$) : A et D n'ont qu'une voisine (poids 1), B et C deux (poids $\frac{1}{2}$ chacune), donc $z^T W z = z_A z_B + \frac{1}{2} z_B (z_A + z_C) + \frac{1}{2} z_C (z_B + z_D) + z_D z_C = 0,75 + 0,25 + 0,25 + 0,75 = 2$ et $I = \frac{2}{5} = 0,4$. (b) $\mathbb{E}[I] = -1/(n-1) = -\frac{1}{3}$: avec $n = 4$ zones seulement, l'espérance sous le hasard est loin de zéro. (c) On énumère les 24 permutations.

```
from itertools import permutations

W4 = std_lignes(np.array([[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]], dtype=float))
W4b = np.array([[0, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0]], dtype=float)
vals = np.array([1.0, 2, 3, 4])
print("I (poids binaires)      :", round(moran(vals, W4b), 4), "| I (poids standardisés) :",
      ↪ round(moran(vals, W4), 4))
toutes = np.array([moran(np.array(p), W4) for p in permutations(vals)])
print("nombre de permutations :", len(toutes), "| valeurs distinctes de I :",
      ↪ np.unique(toutes.round(4)).tolist())
print("moyenne de I sur les 24 permutations :", round(toutes.mean(), 4), "(= -1/(n-1) =", round(-1 /
      ↪ 3, 4), ")")
print("p-valeur exacte (I >= observé) :", round(float((toutes >= moran(vals, W4) - 1e-12).mean()), 4))
```

```
I (poids binaires)      : 0.3333 | I (poids standardisés) : 0.4
nombre de permutations : 24 | valeurs distinctes de I : [-0.9, -0.6, -0.5, -0.3, 0.0, 0.3, 0.4]
moyenne de I sur les 24 permutations : -0.3333 (= -1/(n-1) = -0.3333 )
p-valeur exacte (I >= observé) : 0.0833
```

L'espérance exacte sur les permutations est bien $-\frac{1}{3}$. L'alignement observé (1, 2, 3, 4) et son image inversée (4, 3, 2, 1) sont les deux seuls arrangements qui atteignent cette valeur maximale : la p-valeur exacte est $2/24 \approx 0,083$, **supérieure à 5 %**. Avec quatre zones, aucune autocorrélation, même parfaite, ne peut être significative : c'est le prix du petit nombre de permutations.

Corrigé 9.4

- (a) $z_i > 0$ (zone haute) et $(Wz)_i < 0$ (voisines basses) : quadrant **HL** (haute entourée de basses). (b) $I_i = \frac{z_i (Wz)_i}{m_2} = \frac{12 \times (-8)}{100} = -0,96$: négatif, car la zone s'oppose à ses voisines. (c) Non : un indice local négatif ne dit rien de sa significativité. Il faut une **permutation conditionnelle** (fixer z_i , mélanger les autres valeurs) pour obtenir une p-valeur, puis **corriger** pour les tests multiples (Benjamini-Hochberg), comme au 9.2.4.

Corrigé 9.5

Distance 1 : paires (3,5), (5,4), (4,8), (8,7) ; carrés des écarts 4, 1, 16, 1, total 22 ; $\hat{\gamma}(1) = \frac{22}{2 \times 4} = 2,75$.
 Distance 2 : (3,4), (5,8), (4,7) ; carrés 1, 9, 9, total 19 ; $\hat{\gamma}(2) = \frac{19}{2 \times 3} \approx 3,17$. Distance 3 : (3,8), (5,7) ; carrés 25, 4 ; $\hat{\gamma}(3) = \frac{29}{2 \times 2} = 7,25$. Distance 4 : (3,7) ; $\hat{\gamma}(4) = \frac{16}{2} = 8$.

```
P5 = np.array([[0.0, 0], [1, 0], [2, 0], [3, 0], [4, 0]])
z5 = np.array([3.0, 5, 4, 8, 7])
print(variogramme_empirique(P5, z5, largeur=1.0, hmax=4.0).round(3).to_string(index=False))
```

h	gamma	paires
1.0	2.750	4
2.0	3.167	3
3.0	7.250	2
4.0	8.000	1

La courbe est **croissante** (2,75 ; 3,17 ; 7,25 ; 8) et ne montre aucun palier : on ne peut pas estimer de portée. Deux raisons : il y a très peu de données, et ces valeurs montrent une **tendance** (elles augmentent avec l'abscisse), qui fait monter le variogramme (9.3.7). Surtout, le nombre de paires **diminue** avec la distance (4, 3, 2, 1) : l'estimation à 4 km repose sur **une seule paire** et n'a quasiment aucune fiabilité. C'est l'inverse de ce qui se passe en deux dimensions, où les paires lointaines sont nombreuses, mais cela rappelle qu'un variogramme se lit en regardant les effectifs.

Corrigé 9.6

- (a) $\Gamma = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$ (car $\gamma(4) = 2$) et $\gamma_0 = (\gamma(1), \gamma(3))^T = (0,5; 1,5)^T$. Les équations sont $2\lambda_2 + m = 0,5$, $2\lambda_1 + m = 1,5$ et $\lambda_1 + \lambda_2 = 1$. En soustrayant, $2(\lambda_1 - \lambda_2) = 1$, donc $\lambda_1 = 0,75$, $\lambda_2 = 0,25$ et $m = 0$. Prédiction : $0,75 \times 20 + 0,25 \times 30 = 22,5$, variance $\lambda^T \gamma_0 + m = 0,75 \times 0,5 + 0,25 \times 1,5 = 0,75$. C'est l'interpolation linéaire (de 20 à 30 sur 4 km, soit 22,5 à 1 km). (b) **Non** : la variance de krigeage $\lambda^T \gamma_0 + m$ ne dépend que de la **géométrie** (les distances) et du variogramme, jamais des valeurs mesurées. Les poids sont identiques, donc la prédiction vaudrait $0,75 \times 0 + 0,25 \times 100 = 25$ et la variance resterait 0,75. Le krigeage dit à quel point le *plan d'échantillonnage* est informatif, pas si les données observées sont « surprenantes ».

```
for valeurs in ([20.0, 30.0], [0.0, 100.0]):
    pred, var, lam = krigeage_ordinaire(np.array([[0.0, 0], [4, 0]]), np.array(valeurs),
    ↪ np.array([[1.0, 0]]), lineaire, (0.5,))
    print("valeurs", valeurs, "-> poids", lam.ravel().round(3), "| prédiction", pred.round(3), "|
    ↪ variance", var.round(3))
```

```
valeurs [20.0, 30.0] -> poids [0.75 0.25] | prédiction [22.5] | variance [0.75]
valeurs [0.0, 100.0] -> poids [0.75 0.25] | prédiction [25.] | variance [0.75]
```

Corrigé 9.7

Nous construisons x comme un mélange de ventes_hab standardisées et d'un autre champ SAR indépendant, puis nous agrégeons.

```
Wr = std_lignes(contiguite_reine(12, 12))
rng7 = np.random.default_rng(5)
A7 = np.linalg.inv(np.eye(144) - 0.9 * Wr)
y7 = zones["ventes_hab"].to_numpy()
champ = A7 @ rng7.normal(size=144)
x7 = 0.5 * (y7 - y7.mean()) / y7.std() + 0.87 * champ / champ.std()
```

```
def agrege(v, k):
    m = 12 // k
    return v.reshape(12, 12).reshape(m, k, m, k).mean(axis=(1, 3)).ravel()

lignes = []
for k in (1, 2, 3, 4, 6):
    m = 12 // k
    ya, xa = agrege(y7, k), agrege(x7, k)
    I_k = moran(ya, std_lignes(contiguite_reine(m, m))) if m >= 3 else np.nan
    lignes.append({"bloc": f"{k} x {k}", "zones": m * m, "corr(x, ventes)": np.corrcoef(xa, ya)[0, 1],
                  "ecart_type_ventes": ya.std(), "Moran_ventes": I_k, "E[I]": -1 / (m * m - 1) if m >=
                  3 else np.nan})
with pd.option_context("display.float_format", "{:.3f}".format, "display.width", 150):
    print(pd.DataFrame(lignes).to_string(index=False))
```

	bloc	zones	corr(x, ventes)	ecart_type_ventes	Moran_ventes	E[I]
1	x 1	144	0.597	13.468	0.621	-0.007
2	x 2	36	0.619	11.142	0.570	-0.029
3	x 3	16	0.646	10.010	0.291	-0.067
4	x 4	9	0.774	9.350	-0.054	-0.125
6	x 6	4	0.869	8.431	NaN	NaN

La **corrélation augmente** avec l'agrégation (de 0,60 sur les 144 zones à 0,87 sur les 4 blocs de 6×6), l'**écart-type** des ventes diminue (de 13,5 à 8,4 : les moyennes de blocs lissent les fluctuations) et l'**indice de Moran** chute : de 0,62 à 0,57 (blocs de 2 × 2), à 0,29 (3 × 3, soit 16 zones) puis à -0,05 (4 × 4, soit 9 zones), à comparer à -0,125 sous le hasard pour 9 zones : à cette échelle, l'autocorrélation a pratiquement disparu (et avec 9 zones, un test aurait de toute façon très peu de puissance). Les mêmes personnes, les mêmes ventes, trois « résultats » différents selon le découpage : c'est le **problème de l'unité spatiale modifiable** (MAUP). Une corrélation calculée sur des zones n'est vraie que pour **ce découpage** : on ne peut pas la transposer à l'échelle des individus (sophisme écologique), ni à un autre découpage, sans précaution.

Corrigé 9.8

On simule le hasard en mélangeant les valeurs de ventes_bruit et on calcule, pour chaque case, l'indice local $I_i = z_i(Wz)_i / m_2$.

```
rng8 = np.random.default_rng(8)
zb = zones["ventes_bruit"].to_numpy() - zones["ventes_bruit"].mean()
m2 = zb @ zb / 144
perms = np.argsort(rng8.random((3000, 144)), axis=1)
Zs = zb[perms]
Ii_sim = Zs * (Zs @ Wr.T) / m2 # (3000, 144) : indices locaux sous H0
k_vois = contiguite_reine(12, 12).sum(axis=1)
lignes = []
for kk in (3, 5, 8):
    cols = k_vois == kk
    lignes.append({"voisines": kk, "cases": int(cols.sum()), "variance de I_i": Ii_sim[:,
    ↪ cols].var(), "variance x k": Ii_sim[:, cols].var() * kk})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

voisines	cases	variance de I_i	variance x k
3	4	0.333	1.000
5	40	0.191	0.957
8	100	0.117	0.936

La variance de I_i est environ **2,8 fois plus grande** pour un coin (3 voisines) que pour une case intérieure (8 voisines), à comparer au rapport théorique $8/3 \approx 2,67$; la colonne « variance x k » est à peu près constante, ce qui confirme la loi en $1/k$: la moyenne des voisines est plus bruitée quand

il y a moins de voisines. Une zone de bord ressemble donc plus facilement à une valeur extrême par hasard, et une p-valeur calculée avec la **même** loi pour toutes les zones la déclarerait trop souvent significative. Les **permutations conditionnelles** du 9.2.4 évitent ce piège, car elles tirent exactement le bon nombre de voisines pour chaque zone. Par prudence, on examine séparément les zones de bord.

Corrigé 9.9

Pour chaque ρ , on simule 4 000 champs SAR et on compare la variance d'une zone à celle de la moyenne des 144 zones.

```
rng9 = np.random.default_rng(9)
lignes = []
for rho in (0.3, 0.5, 0.7, 0.9, 0.95):
    Y9 = np.linalg.inv(np.eye(144) - rho * Wr) @ rng9.normal(size=(144, 4000))
    lignes.append({"rho": rho, "variance d'une zone": Y9.var(), "variance de la moyenne":
        ↪ Y9.mean(axis=0).var(), "n_eff": Y9.var() / Y9.mean(axis=0).var()})
print(pd.DataFrame(lignes).round(3).to_string(index=False))
```

rho	variance d'une zone	variance de la moyenne	n_eff
0.30	1.052	0.014	75.715
0.50	1.179	0.028	42.577
0.70	1.555	0.077	20.203
0.90	3.917	0.708	5.532
0.95	8.383	2.805	2.989

Le nombre efficace d'observations **s'effondre** quand ρ augmente : d'environ 76 pour $\rho = 0,3$, il tombe à 43 pour $\rho = 0,5$, à 20 pour $\rho = 0,7$, à 5,5 pour $\rho = 0,9$ (la valeur de la section 9.2.5) et à 3 pour $\rho = 0,95$. Même pour une dépendance modérée ($\rho = 0,5$), les 144 zones n'apportent l'information que d'environ 43 observations indépendantes. C'est la raison de la surconfiance des tests usuels.

Corrigé 9.10

- (a) Les cases ont une aire de $2,5 \times 2,5 = 6,25 \text{ km}^2$ et l'intensité vaut $\lambda = 1$ par km^2 : le nombre de points d'une case suit approximativement une loi de Poisson de moyenne 6,25, donc $\mathbb{P}(\text{vide}) = e^{-6,25} \approx 0,0019$. Plus exactement, conditionnellement à $n = 100$ points, chaque point tombe hors d'une case donnée avec la probabilité $\frac{15}{16}$: $\mathbb{P}(\text{vide}) = (15/16)^{100} \approx 0,0016$. Pour 16 cases, le nombre moyen de cases vides est $16 \times 0,0016 \approx 0,025$: sous le hasard complet, on s'attend à **ne presque jamais** voir de case vide.

```
print("e^-6,25 =", round(float(np.exp(-6.25)), 5), "| (15/16)^100 =", round((15 / 16) ** 100, 5), "|
↪ 16 x (15/16)^100 =", round(16 * (15 / 16) ** 100, 4))
rng10 = np.random.default_rng(10)
vides = [(comptages(semis_poisson(100, rng10)) == 0).sum() for _ in range(5000)]
print("nombre moyen de cases vides sur 5000 semis CSR :", round(float(np.mean(vides)), 4), "| part des
↪ semis avec au moins une case vide :", round(float(np.mean(np.array(vides) > 0)), 4))
print("cases vides dans le semis agrégé :", int((comptages(semis["agrégé (Thomas)"]) == 0).sum()),
↪ "sur 16")
```

```
e^-6,25 = 0.00193 | (15/16)^100 = 0.00157 | 16 x (15/16)^100 = 0.0252
nombre moyen de cases vides sur 5000 semis CSR : 0.0226 | part des semis avec au moins une case vide :
↪ 0.0224
cases vides dans le semis agrégé : 7 sur 16
```

- (b) La simulation confirme le calcul (environ 0,025 case vide en moyenne). Le semis agrégé de 9.4 a **7 cases vides sur 16** : un événement quasi impossible sous le hasard, qui est un autre

visage de l'agrégation. (Attention : ce semis compte 75 points et non 100, ce qui rend les cases vides un peu plus plausibles, sans changer la conclusion.)

Corrigé 9.11

Notons $\delta = Z(s + h) - Z(s)$. Par stationnarité, $\mathbb{E}[Z(s + h)] = \mathbb{E}[Z(s)] = \mu$, donc $\mathbb{E}[\delta] = 0$, et $\mathbb{E}[\delta^2] = \text{Var}(\delta)$. Par définition du semi-variogramme, $\gamma(h) = \frac{1}{2}\mathbb{E}[\delta^2]$. L'estimateur $\hat{\gamma}(h)$ est une moyenne de $\frac{1}{2}\delta^2$ sur des paires **dont la distance est h** : son espérance est donc $\gamma(h)$; il est sans biais à μ **inconnue** (ce qui le distingue de la covariance empirique, qui exige d'estimer la moyenne). Le seul biais vient du regroupement par classes (nous comparons à γ à la distance moyenne de la classe) et d'éventuelles tendances. Vérifions par simulation.

```
accumule = []
for graine in range(300, 360):
    d_sim, _ = livraisons(seed=graine)
    accumule.append(variogramme_empirique(d_sim[["x", "y"]].to_numpy(),
    ↪ d_sim["delai_jours"].to_numpy()))
h_moy = np.mean([e["h"].to_numpy() for e in accumule], axis=0)
g_moy = np.mean([e["gamma"].to_numpy() for e in accumule], axis=0)
g_sd = np.std([e["gamma"].to_numpy() for e in accumule], axis=0, ddof=1) / np.sqrt(len(accumule))
vrai_g = gamma_exp(h_moy, 0.4, 1.0, 12.0)
print(pd.DataFrame({"h": h_moy, "gamma_moyen": g_moy, "erreur_type_moyenne": g_sd, "gamma_vrai":
    ↪ vrai_g, "ecart_%": 100 * (g_moy / vrai_g - 1)}).round(3).to_string(index=False))
```

h	gamma_moyen	erreur_type_moyenne	gamma_vrai	ecart_%
3.324	0.632	0.011	0.642	-1.592
7.745	0.867	0.014	0.876	-0.956
12.636	1.033	0.019	1.051	-1.751
17.593	1.164	0.022	1.169	-0.416
22.548	1.242	0.027	1.247	-0.389
27.549	1.303	0.030	1.299	0.277
32.529	1.329	0.032	1.334	-0.326
37.509	1.367	0.037	1.356	0.809
42.512	1.404	0.036	1.371	2.385
47.503	1.405	0.037	1.381	1.719

La moyenne sur 60 jeux colle au variogramme vrai à toutes les distances, à moins de 2,5 % près, et les écarts restent de l'ordre de l'erreur-type de la moyenne (colonne suivante) : rien ne montre de biais. Mais regardons la colonne des erreurs-types : elle **augmente avec la distance**. C'est la signature d'une corrélation forte entre classes de distance voisines et de grandes fluctuations d'un jeu à l'autre aux distances élevées : une moyenne de 60 jeux est précise, mais **un seul jeu** de données réelles est un tirage unique, d'où la dispersion observée en 9.3.3 sur les paramètres ajustés.

Corrigé 9.12

- (a) Par définition, $K(r) = \int_0^r 2\pi s g(s) ds$ (le nombre moyen d'autres points à moins de r , divisé par λ , est l'intégrale de la fonction de corrélation de paires sur le disque). Donc $K(r) = \pi r^2 + \frac{2\pi}{4\pi\kappa\sigma^2} \int_0^r s e^{-s^2/(4\sigma^2)} ds$. Or $\int_0^r s e^{-s^2/(4\sigma^2)} ds = 2\sigma^2(1 - e^{-r^2/(4\sigma^2)})$, d'où $K(r) = \pi r^2 + \frac{1}{2\kappa\sigma^2} \cdot 2\sigma^2(1 - e^{-r^2/(4\sigma^2)}) = \pi r^2 + \frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$. Remarquons que K ne dépend ni de μ ni de l'intensité totale : l'excès par rapport au hasard est $\frac{1}{\kappa}(1 - e^{-r^2/(4\sigma^2)})$, et il tend vers $1/\kappa$ (le nombre moyen de centres par km^2 est κ , donc $1/\kappa$ est l'aire d'« un agrégat »). (b) Simulons.

```
def K_thomas_theorique(r, kappa, sigma):
    return np.pi * r ** 2 + (1 / kappa) * (1 - np.exp(-r ** 2 / (4 * sigma ** 2)))

rs12 = np.array([0.25, 0.5, 1.0, 1.5, 2.0])
for kappa, mu, sigma, B in [(0.5, 2, 0.4, 500), (0.08, 12, 0.4, 300)]:
    rng12 = np.random.default_rng(12)
    Ks, effectifs = [], []
    while len(Ks) < B:
        pts12 = semis_thomas(kappa, mu, sigma, rng12)
        if len(pts12) > 10:
            Ks.append(ripley_K(pts12, rs12))
            effectifs.append(len(pts12))
    Ks = np.array(Ks)
    theo = K_thomas_theorique(rs12, kappa, sigma)
    tab = pd.DataFrame({"r": rs12, "pi r^2 (hasard)": np.pi * rs12 ** 2, "K théorique": theo, "K
    simulé (moyenne)": np.nanmean(Ks, axis=0)})
    tab["écart_%"] = 100 * (tab["K simulé (moyenne)"] / tab["K théorique"] - 1)
    print(f"kappa = {kappa}, mu = {mu}, sigma = {sigma} ({B} semis) | nombre de points : moyenne
    {np.mean(effectifs):.0f}, "
          f"écart-type {np.std(effectifs):.0f}, coefficient de variation {np.std(effectifs) /
          np.mean(effectifs):.0%}")
    print(tab.round(2).to_string(index=False))
    print()
```

```
kappa = 0.5, mu = 2, sigma = 0.4 (500 semis) | nombre de points : moyenne 101, écart-type 17,
    coefficient de variation 16%
   r  pi r^2 (hasard)  K théorique  K simulé (moyenne)  écart_%
0.25          0.20         0.38          0.38      -0.15
0.50          0.79         1.43          1.43      -0.19
1.00          3.14         4.72          4.68      -0.82
1.50          7.07         9.01          8.81     -2.25
2.00         12.57        14.56         13.96     -4.13
```

```
kappa = 0.08, mu = 12, sigma = 0.4 (300 semis) | nombre de points : moyenne 97, écart-type 31,
    coefficient de variation 32%
   r  pi r^2 (hasard)  K théorique  K simulé (moyenne)  écart_%
0.25          0.20         1.36          1.46       7.63
0.50          0.79         4.83          5.16       6.94
1.00          3.14        13.02         13.67       4.95
1.50          7.07        19.20         19.05      -0.77
2.00         12.57        25.04         23.21     -7.34
```

Pour le premier réglage (agrégats faibles et nombreux : en moyenne 2 descendants par centre), la simulation retrouve la théorie à moins de 1 % jusqu'à $r = 1$ km, puis s'en écarte de $-2,3\%$ à $1,5$ km et de $-4,1\%$ à 2 km. Pour le second (agrégats forts : 12 descendants par centre), les écarts sont nettement plus importants et de **signe variable** : $+7,6\%$ à $0,25$ km, $+5\%$ à 1 km, $-7,3\%$ à 2 km. **Cela ne signifie pas que la formule est fausse**, mais que l'estimateur $\hat{K}$ est biaisé quand le nombre total de points varie beaucoup d'un semis à l'autre : le coefficient de variation de n est de 16 % pour le premier réglage et de 32 % pour le second (voir la ligne imprimée au-

dessus de chaque tableau). Une cause probable est que l'estimateur divise par un nombre de points aléatoire, lui-même corrélé aux comptages de voisins : l'espérance d'un rapport n'est pas le rapport des espérances. Ajoutons que, pour les grandes distances, la méthode du bord n'utilise qu'une petite fraction des points (à 2 km, seuls ceux situés à plus de 2 km du bord, soit 36 %). Morale : **ne pas surinterpréter $\hat{K}$ pour des semis très agrégés ni aux grandes distances**, ce que résume la règle de ne pas dépasser le quart du côté de la fenêtre.

Projet du volume et auto-évaluation — exercices et applications

Ce chapitre du cahier clôt le volume II. Il contient le **projet du volume** (une étude de modélisation complète : prévoir les ventes, comprendre qui rachète, valoriser un client, mesurer sa durée de vie, puis décider) et **quarante-deux questions d'auto-évaluation** avec leurs réponses. Il accompagne les points clés du livre et utilise les données `clients.csv` et `ventes_mensuelles.csv` du dossier `donnees/` : ces données sont **simulées**, ce qui permet de dévoiler à la fin ce qui avait été programmé.

Projet du volume

« Prévoir, c'est facile : il suffit de se tromper de façon honnête, et de dire de combien.
»

Dans le cahier du volume I, le projet de clôture assemblait des mesures simples. Ici, nous assemblons des **modèles** : une série temporelle pour prévoir, un modèle linéaire généralisé pour comprendre qui rachète et combien chaque client dépense, un modèle de survie pour savoir combien de temps un client reste. À la fin, ces trois résultats se combinent en une **décision** : l'offre de bienvenue vaut-elle son coût ?

Comment lire ce projet. Il n'introduit aucune notion nouvelle : chaque étape renvoie à la section où la méthode est expliquée. Le plus profitable : lire le cahier des charges (P.1), fermer le livre, essayer de répondre vous-même, puis comparer. Les données sont **simulées** (voir l'introduction du volume) : en P.8, nous dévoilerons ce qui avait été programmé et nous verrons ce que nos modèles en ont retrouvé.

P.1 Le cahier des charges

Début janvier 2026, la gérante vous écrit :

« Bonjour ! 2025 est bouclée, je prépare le budget de 2026. J'ai quatre questions.

1. Combien vais-je vendre en 2026, mois par mois ? J'ai besoin d'une fourchette, pas seulement d'un chiffre : je dois prévoir ma trésorerie.
2. L'offre de bienvenue que je tire au sort pour les nouveaux clients, est-ce qu'elle les fait revenir ? Et de combien ?

3. Un client, ça me rapporte combien par an ? Ça dépend de l'âge, du canal ?

4. Et combien de temps un client reste-t-il ? Au fond, je voudrais savoir ce que vaut un client, et si l'offre de bienvenue (elle me coûte environ 10 € par client) vaut la peine. Merci ! »

La méthode suit six étapes. Chacune s'appuie sur un chapitre du volume.

Étape	Question	Outil	Chapitre
1. Contrôler	Les données sont-elles fiables ?	vérifications, assertions	cahier du volume I (projet)
2. Prévoir	Les ventes de 2026	SARIMA avec variables exogènes, rétro-test	4
3. Comprendre le rachat	Effet de l'offre de bienvenue	régression logistique	2 (2.2, 2.4)
4. Valoriser	Combien rapporte un client ?	modèle Tweedie / Gamma	2 (2.3, 2.6), 1
5. Durer	Combien de temps reste un client ?	Kaplan-Meier, Cox	5
6. Décider	Que vaut un client ? L'offre est-elle rentable ?	combinaison des modèles + incertitude	5, 6 (bootstrap : volume I, section 3.3.5)

P.2 Étape 1 : charger et contrôler

```
import warnings
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")          # les avertissements de convergence sont examinés un par un
↳ dans le texte

clients = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
ventes = pd.read_csv("donnees/ventes_mensuelles.csv", parse_dates=["mois"], index_col="mois")
ventes.index.freq = "MS"
```

Avant toute analyse, on **teste** les données : chaque contrôle est une question dont on connaît la réponse attendue.

```
controles = {
    "2 000 clients, identifiants uniques": clients["id_client"].is_unique and len(clients) == 2000,
    "aucune valeur manquante (clients)": int(clients.isna().sum().sum()) == 0,
    "offre_bienvenue ne vaut que 0 ou 1": set(clients["offre_bienvenue"]) == {0, 1},
    "dépense nulle si et seulement si aucune commande": bool(((clients["depense_annuelle"] == 0) ==
    ↳ (clients["nb_commandes_an"] == 0)).all()),
    "durée de relation positive ou nulle": bool((clients["duree_mois"] >= 0).all()),
    "120 mois consécutifs sans trou": len(ventes) == 120 and bool((ventes.index ==
    ↳ pd.date_range("2016-01-01", periods=120, freq="MS")).all()),
    "chiffre d'affaires strictement positif": bool((ventes["ca"] > 0).all()),
}
for nom, ok in controles.items():
    print("OK " if ok else "ÉCHEC", nom)
assert all(controles.values())
```

```

OK 2 000 clients, identifiants uniques
OK aucune valeur manquante (clients)
OK offre_bienvenue ne vaut que 0 ou 1
OK dépense nulle si et seulement si aucune commande
OK durée de relation positive ou nulle
OK 120 mois consécutifs sans trou
OK chiffre d'affaires strictement positif

```

Comme dans le projet du volume I, un contrôle qui échoue **arrête** le programme : on préfère un plantage bruyant à un rapport faux.

P.3 Étape 2 : prévoir les ventes de 2026 (question 1)

P.3.1 Regarder, puis fixer des repères

Une série temporelle se **dessine** avant de se modéliser (chapitre 4, section 4.1). Sur l'échelle logarithmique, la saisonnalité devient additive et la tendance presque linéaire :

```

y = np.log(ventes["ca"])
fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
ax[0].plot(ventes.index, ventes["ca"], color="#2a78d6", lw=1.6)
ax[0].axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2020-06-30"), color="#e34948", alpha=0.15)
ax[0].text(pd.Timestamp("2020-03-15"), ventes["ca"].max() * 0.93, "2020", color="#e34948",
           fontsize=9)
ax[0].set_title("Chiffre d'affaires mensuel (€)")
ax[0].set_ylabel("€")
ax[1].plot(ventes.index, y, color="#2a78d6", lw=1.6)
ax[1].set_title("Même série en logarithme")
ax[1].set_ylabel("log(€)")
plt.tight_layout()
plt.savefig("figures/ch10-serie-ventes.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")

```

figure enregistrée

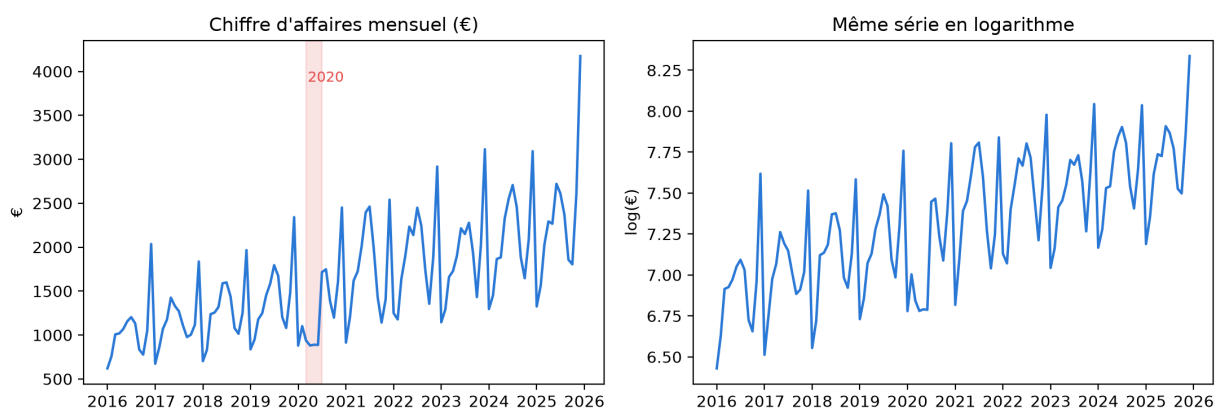


Figure 14: À gauche : chiffre d'affaires mensuel de la boutique de 2016 à 2025 (la bande rouge marque l'arrêt du printemps 2020). À droite : la même série en logarithme, où la saisonnalité est de même amplitude à tous les niveaux.

Pour juger un modèle de prévision, il faut un **repère** : une méthode si simple que ne pas la battre serait inquiétant. Le plus utile ici est la **prévision naïve saisonnière** : « chaque mois ressemblera au même mois de l'an passé ».

On s'entraîne sur 2016-2023 et on teste sur 2024-2025 : le modèle ne voit jamais les mois qu'il doit prévoir (chapitre 4, section 4.3). La mesure d'erreur est le **MAPE**, l'erreur absolue moyenne en

pourcentage.

```
X = ventes[["promo", "covid"]].astype(float)
train, test = slice(None, "2023-12-01"), slice("2024-01-01", None)

def mape(reel, prevu):
    return float(np.mean(np.abs(reel - prevu) / reel) * 100)

naif_saisonnier = ventes["ca"].shift(12)[test]
print(f"MAPE du naïf saisonnier sur 2024-2025 : {mape(ventes['ca'][test], naif_saisonnier):.2f} %")
```

```
MAPE du naïf saisonnier sur 2024-2025 : 9.52 %
```

P.3.2 Ajuster et comparer des modèles SARIMA

Quatre candidats, tous avec une composante saisonnière annuelle et les deux variables exogènes promo (promotion ce mois-ci) et covid (arrêt de 2020) :

```
candidats = {
    "SARIMA(1,1,0)(0,1,1)12": dict(order=(1, 1, 0), seasonal_order=(0, 1, 1, 12), trend="n"),
    "SARIMA(1,1,1)(0,1,1)12": dict(order=(1, 1, 1), seasonal_order=(0, 1, 1, 12), trend="n"),
    "SARIMA(1,0,0)(0,1,1)12 + constante": dict(order=(1, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="c"),
    "SARIMA(2,0,0)(0,1,1)12 + constante": dict(order=(2, 0, 0), seasonal_order=(0, 1, 1, 12),
    ↪ trend="c"),
}
lignes, ajustes = [], {}
for nom, spec in candidats.items():
    res = sm.tsa.SARIMAX(y[train], exog=X[train], **spec).fit(dispatch=False)
    prevu = np.exp(res.get_forecast(24, exog=X[test]).predicted_mean)
    ajustes[nom] = res
    lignes.append({"modèle": nom, "AIC": round(res.aic, 1), "MAPE test (%)":
    ↪ round(mape(ventes["ca"][test], prevu), 2)})
comparaison = pd.DataFrame(lignes)
print(comparaison.to_string(index=False))
```

	modèle	AIC	MAPE test (%)
	SARIMA(1,1,0)(0,1,1)12	-164.5	6.93
	SARIMA(1,1,1)(0,1,1)12	-176.7	8.23
	SARIMA(1,0,0)(0,1,1)12 + constante	-185.0	8.36
	SARIMA(2,0,0)(0,1,1)12 + constante	-184.0	8.29

L'AIC n'est comparable qu'entre modèles de même ordre de différenciation. Les deux modèles « avec constante » n'ont pas été différenciés ($d = 0$) : leur AIC est calculé sur la série elle-même, alors que celui des deux premiers l'est sur la série différenciée. Comparer ces AIC n'a pas de sens, même si le tableau les met côte à côte. C'est le **MAPE sur la période de test**, mesuré sur des données que le modèle n'a pas vues, qui arbitre ici (chapitre 4, section 4.3).

On retient le meilleur modèle sur le test (son MAPE, 6,93 %, bat nettement le repère naïf saisonnier, 9,52 %), puis on regarde si ses résidus ressemblent à du bruit blanc : si ce n'est pas le cas, le modèle a laissé du signal derrière lui.

```
meilleur = comparaison.sort_values("MAPE test (%)").iloc[0]["modèle"]
res = ajustes[meilleur]
print("modèle retenu :", meilleur)
from statsmodels.stats.diagnostic import acorr_ljungbox
residus = res.resid[13:] # on écarte les premiers résidus, dominés par l'initialisation
lb = acorr_ljungbox(residus, lags=[12, 24], return_df=True)
print(lb.round(3).to_string())
```

```
modèle retenu : SARIMA(1,1,0)(0,1,1)12
      lb_stat  lb_pvalue
12   13.760    0.316
24   25.206    0.395
```

Les p-valeurs de Ljung-Box (chapitre 4, section 4.1) sont élevées : on ne détecte pas d'autocorrélation résiduelle. Le modèle a capté la structure.

P.3.3 Prévoir 2026

On réajuste le modèle retenu sur **les 120 mois**, puis on prévoit les 12 mois de 2026. Il faut fournir les valeurs futures des variables exogènes : pas de nouvel arrêt (covid = 0), et une hypothèse sur les promotions. La gérante prévoit une promotion en décembre, comme en 2025 :

```
spec = candidats[meilleur]
final = sm.tsa.SARIMAX(y, exog=X, **spec).fit(dispatch=False)
futur = pd.date_range("2026-01-01", periods=12, freq="MS")
X_futur = pd.DataFrame({"promo": 0.0, "covid": 0.0}, index=futur)
X_futur.loc["2026-12-01", "promo"] = 1.0
pred = final.get_forecast(12, exog=X_futur)
ic = pred.conf_int(alpha=0.05)
prevision = pd.DataFrame({
    "prévision": np.exp(pred.predicted_mean),
    "bas_95": np.exp(ic.iloc[:, 0]),
    "haut_95": np.exp(ic.iloc[:, 1]),
}).round(0)
prevision.index = prevision.index.strftime("%Y-%m")
print(prevision.to_string())
print()
total_2025 = ventes.loc["2025", "ca"].sum()
total_2026 = prevision["prévision"].sum()
print(f"total 2025 : {total_2025:,.0f} €   total 2026 prévu : {total_2026:,.0f} €   ({100 *
↳ (total_2026 / total_2025 - 1):+.1f} %)")
```

	prévision	bas_95	haut_95
2026-01	1517.0	1306.0	1762.0
2026-02	1835.0	1504.0	2241.0
2026-03	2407.0	1893.0	3060.0
2026-04	2531.0	1923.0	3331.0
2026-05	2838.0	2091.0	3852.0
2026-06	3087.0	2212.0	4309.0
2026-07	3215.0	2245.0	4603.0
2026-08	2972.0	2026.0	4360.0
2026-09	2324.0	1549.0	3487.0
2026-10	2023.0	1320.0	3101.0
2026-11	2681.0	1714.0	4194.0
2026-12	4599.0	2883.0	7335.0

```
total 2025 : 27,630 €   total 2026 prévu : 32,029 €   (+15.9 %)
```

Les bornes à 95 % s'écartent à mesure que l'on s'éloigne dans le futur : l'incertitude s'accumule. Voici la prévision en image :

```
fig, ax = plt.subplots(figsize=(10, 3.9))
recent = ventes.loc["2023":]
ax.plot(recent.index, recent["ca"], color="#2a78d6", lw=1.8, label="observé")
ax.plot(futur, np.exp(pred.predicted_mean), color="#eb6834", lw=1.8, label="prévision 2026")
ax.fill_between(futur, np.exp(ic.iloc[:, 0]), np.exp(ic.iloc[:, 1]), color="#eb6834", alpha=0.18,
↳ label="intervalle à 95 %")
ax.set_ylabel("chiffre d'affaires (€)")
ax.set_title("Prévision du chiffre d'affaires mensuel de 2026")
ax.legend(frameon=False, loc="upper left")
plt.tight_layout()
```

```
plt.savefig("figures/ch10-prevision-2026.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

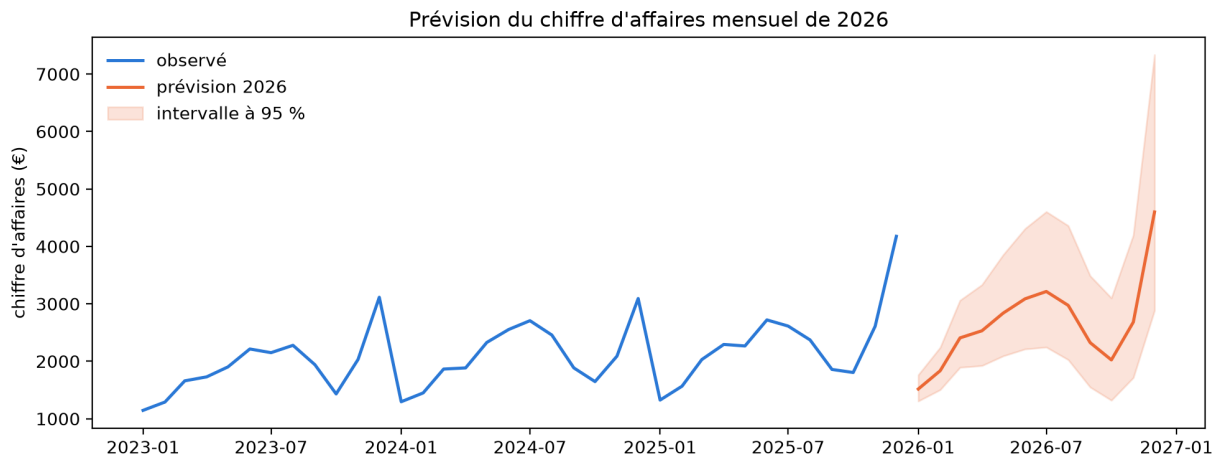


Figure 15: Chiffre d'affaires observé depuis 2023 et prévision mois par mois pour 2026, avec intervalle de prévision à 95 %.

Ce que l'intervalle ne couvre pas. L'intervalle à 95 % suppose que le **mécanisme reste le même** en 2026. Il ne couvre ni une nouvelle crise, ni une concurrence nouvelle, ni un changement de stratégie. De plus, la prévision est faite sur le logarithme puis ramenée à l'échelle d'origine : c'est une prévision de la **médiane** du mois, un peu inférieure à sa moyenne (volume I, section 2.3 : l'espérance d'une exponentielle dépasse l'exponentielle de l'espérance). Pour la trésorerie, on regardera donc plutôt la borne basse que la valeur centrale.

Deux prévisions pour la même année : laquelle croire ? Au chapitre 4 (section 4.3.7), le modèle retenu combine une différence saisonnière et une tendance déterministe ; il prévoit **29 169 €** pour 2026 (+5,6 %), avec un intervalle à 95 % de 27 073 à 31 397 €. Le modèle retenu ici (différences ordinaire et saisonnière, sans tendance) prévoit **32 029 €** (+15,9 %), **au-dessus** de la borne haute de cet intervalle. Les deux modèles décrivent la même série et sont défendables, mais leurs hypothèses sur la tendance diffèrent. L'écart rappelle qu'en plus de l'incertitude statistique que chaque intervalle mesure, il existe une **incertitude de modèle** qu'aucun intervalle ne mesure. Dans la pratique, on la traite en comparant plusieurs modèles, en regardant leurs écarts sur un rétro-test (ce que fait cette étape), et en présentant une **fourchette** plutôt qu'un chiffre unique.

P.4 Étape 3 : l'offre de bienvenue fait-elle revenir les clients ? (question 2)

La variable à expliquer est binaire (rachat_12m) : une **régression logistique** (chapitre 2, section 2.2). Surtout, l'offre a été **tirée au sort** : c'est la situation idéale où l'on peut lire l'effet comme un effet **causal** (chapitre 7 explique pourquoi). Les autres variables (âge, canal) servent à affiner, pas à identifier.

D'abord la comparaison brute, puis le modèle :

```
print(clients.groupby("offre_bienvenue")["rachat_12m"].agg(clients="count",
↳   taux_rachat="mean").round(3).to_string())
print()
```

```
logit = smf.logit("rachat_12m ~ offre_bienvenue + I(age - 36) + C(canal_acquisition)",
↳ clients).fit(dis=0)
tab = pd.DataFrame({"coef": logit.params, "rapport_de_cotes": np.exp(logit.params),
                    "IC95_bas": np.exp(logit.conf_int()[0]), "IC95_haut": np.exp(logit.conf_int()[1]),
                    "p": logit.pvalues}).round(3)
print(tab.to_string())
```

	clients	taux_rachat
offre_bienvenue		
0	985	0.448
1	1015	0.569

	coef	rapport_de_cotes	IC95_bas	IC95_haut	p
Intercept	0.031	1.031	0.847	1.255	0.761
C(canal_acquisition)[T.Réseaux]	-0.463	0.629	0.502	0.789	0.000
C(canal_acquisition)[T.Site]	-0.168	0.846	0.669	1.069	0.161
offre_bienvenue	0.493	1.638	1.370	1.957	0.000
I(age - 36)	-0.015	0.985	0.976	0.993	0.000

Un coefficient s'interprète sur l'échelle **logit** ; on le rend lisible en l'exponentiant, ce qui donne un **rapport de cotes** (*odds ratio*). Mais un rapport de cotes n'est pas une différence de probabilité, et c'est la seconde que la gérante veut : « de combien l'offre augmente-t-elle la probabilité de racheter ? ». Nous la calculons en **prédisant deux mondes** : le monde où chaque client a reçu l'offre, et celui où aucun ne l'a reçue (l'« effet marginal moyen »), puis nous en donnons un intervalle de confiance par **bootstrap** (volume I, section 3.3.5).

```
def effet_offre(df):
    m = smf.logit("rachat_12m ~ offre_bienvenue + I(age - 36) + C(canal_acquisition)", df).fit(dis=0)
    return float((m.predict(df.assign(offre_bienvenue=1)) -
↳ m.predict(df.assign(offre_bienvenue=0))).mean())

ame = effet_offre(clients)
rng = np.random.default_rng(2026)
boot = np.array([effet_offre(clients.sample(len(clients), replace=True, random_state=int(s)))
↳ for s in rng.integers(0, 2**31 - 1, 300)])
ame_bas, ame_haut = np.percentile(boot, [2.5, 97.5])
print(f"effet moyen de l'offre sur la probabilité de rachat : {100 * ame:+.1f} points")
print(f"intervalle de confiance à 95 % (bootstrap, 300 rééchantillons) : [{100 * ame_bas:+.1f} ; {100
↳ * ame_haut:+.1f}] points")
```

```
effet moyen de l'offre sur la probabilité de rachat : +12.1 points
intervalle de confiance à 95 % (bootstrap, 300 rééchantillons) : [+7.6 ; +16.3] points
```

Il reste à vérifier que le modèle est **utilisable**. Deux contrôles (chapitre 2, section 2.4) : son pouvoir de discrimination (l'aire sous la courbe ROC) et sa **calibration** (quand le modèle dit « 60 % », voit-on environ 60 % de rachats ?).

```
from sklearn.metrics import roc_auc_score
p = logit.predict(clients)
print(f"AUC : {roc_auc_score(clients['rachat_12m'], p):.3f}")
decile = pd.qcut(p, 5, labels=False)
calib = clients.assign(p=p, groupe=decile).groupby("groupe").agg(
    clients=("p", "size"), proba_moyenne_prevue=("p", "mean"), taux_observed=("rachat_12m",
↳ "mean")).round(3)
print(calib.to_string())
```

```
AUC : 0.597
```

	clients	proba_moyenne_prevue	taux_observed
groupe			
0	410	0.386	0.371
1	394	0.462	0.497
2	398	0.512	0.515

3	400	0.560	0.538
4	398	0.630	0.631

Lire une AUC modeste. Une AUC proche de 0,6 signifie que le modèle ne trie pas très bien les clients individuellement : savoir qu'un client est jeune, venu des réseaux sociaux, et qu'il a reçu l'offre aide peu à prédire **son** rachat, parce que l'essentiel de la variabilité tient à des facteurs que nous n'observons pas. Cela ne contredit pas la qualité de l'estimation de l'**effet moyen** de l'offre : on peut estimer très précisément une différence moyenne tout en prédisant mal chaque individu. Prédire et expliquer sont deux tâches distinctes.

P.5 Étape 4 : combien rapporte un client ? (question 3)

La dépense annuelle est positive, asymétrique, avec **13 % de zéros** (les clients sans commande) : ni la normale, ni le modèle Gamma seul (qui exige des valeurs strictement positives) ne conviennent. Deux solutions du chapitre 2 :

1. le modèle de **Tweedie** (section 2.6), qui gère les zéros et la partie positive d'un seul tenant ;
2. un modèle **en deux parties** (section 2.3 et 2.6) : une logistique pour « le client passe-t-il au moins une commande ? », puis un Gamma sur la dépense des acheteurs.

```
clients["achete"] = (clients["nb_commandes_an"] > 0).astype(int)
formule = "~ I(age - 36) + C(canal_acquisition) + offre_bienvenue"

tweedie = smf.glm("depense_annuelle " + formule, clients,
                  family=sm.families.Tweedie(var_power=1.5, link=sm.families.links.Log())).fit()
partiel1 = smf.logit("achete " + formule, clients).fit(dis=0)
partiel2 = smf.glm("depense_annuelle " + formule, clients[clients["achete"] == 1],
                  family=sm.families.Gamma(sm.families.links.Log())).fit()

deux_parties = partiel1.predict(clients) * partiel2.predict(clients)
print(f"dépense annuelle moyenne observée      : {clients['depense_annuelle'].mean():.1f} €")
print(f"prévue par le modèle de Tweedie          : {tweedie.predict(clients).mean():.1f} €")
print(f"prévue par le modèle en deux parties      : {deux_parties.mean():.1f} €")
print(f"corrélation entre les deux prévisions      : {np.corrcoef(tweedie.predict(clients),
↪  deux_parties)[0, 1]:.3f}")
print()
coefs = pd.DataFrame({"Tweedie": tweedie.params, "Gamma (acheteurs)": partiel2.params}).round(3)
coefs["effet Tweedie (%)"] = ((np.exp(coefs["Tweedie"]) - 1) * 100).round(1)
coefs.loc["Intercept", "effet Tweedie (%)"] = np.nan      # l'ordonnée à l'origine ne se lit pas en
↪  pourcentage
print(coefs.to_string())
print()
print(f"effet de l'offre sur la probabilité d'au moins une commande : coef logit =
↪  {partiel1.params['offre_bienvenue']:+.3f}, p = {partiel1.pvalues['offre_bienvenue']:.2f}")
```

```
dépense annuelle moyenne observée      : 247.0 €
prévue par le modèle de Tweedie          : 247.0 €
prévue par le modèle en deux parties      : 247.0 €
corrélation entre les deux prévisions      : 1.000
```

	Tweedie	Gamma (acheteurs)	effet Tweedie (%)
Intercept	5.766	5.880	NaN
C(canal_acquisition)[T.Réseaux]	-0.555	-0.499	-42.6
C(canal_acquisition)[T.Site]	-0.151	-0.152	-14.0
I(age - 36)	0.008	0.008	0.8
offre_bienvenue	-0.014	-0.010	-1.4

effet de l'offre sur la probabilité d'au moins une commande : coef logit = -0.027, p = 0.84

Les deux approches donnent des prévisions pratiquement identiques. Les coefficients d'un modèle à lien log se lisent en **pourcentages** : $\exp(\text{coef}) - 1$ est la variation relative de la dépense moyenne pour une unité de la variable. Le tableau montre un résultat **nuancé et important pour la suite** : l'offre de bienvenue n'a pratiquement **aucun effet** sur la dépense annuelle (ni sur la probabilité de passer commande), alors que le canal d'acquisition en a un net.

```
profils = pd.DataFrame({"age": [25, 25, 40, 40], "canal_acquisition": ["Réseaux", "Boutique",
↪ "Réseaux", "Boutique"],
                        "offre_bienvenue": [0, 0, 0, 0]})
profils["valeur_annuelle_DT"] = tweedie.predict(profils).round(1)
print(profils.to_string(index=False))
```

age	canal_acquisition	offre_bienvenue	valeur_annuelle_DT
25	Réseaux	0	167.5
25	Boutique	0	291.8
40	Réseaux	0	189.2
40	Boutique	0	329.8

P.6 Étape 5 : combien de temps un client reste-t-il ? (question 4)

Ici, la durée est **censurée** : 1 023 clients sur 2 000 sont encore là en décembre 2025. Les ignorer, ou les traiter comme des départs, biaiserait tout (chapitre 5, section 5.1). On commence par l'**estimateur de Kaplan-Meier** de la fonction de survie, écrit à la main (section 5.2) : à chaque date de départ, on multiplie la survie par la proportion de clients qui ont survécu parmi ceux qui étaient encore là.

```
def kaplan_meier(durees, evenements):
    """Retourne les instants de départ et la survie S(t) correspondante."""
    durees, evenements = np.asarray(durees, float), np.asarray(evenements, int)
    instants = np.unique(durees[evenements == 1])
    s, courbe = 1.0, []
    for t in instants:
        a_risque = np.sum(durees >= t)
        departes = np.sum((durees == t) & (evenements == 1))
        s *= 1 - departes / a_risque
        courbe.append(s)
    return instants, np.array(courbe)

def survie_en(t, instants, courbe):
    """S(t) : valeur de l'escalier en t (1 avant le premier départ)."""
    i = np.searchsorted(instants, t, side="right") - 1
    return 1.0 if i < 0 else courbe[i]

def survie_mediane(instants, courbe):
    sous = np.where(courbe <= 0.5)[0]
    return float(instants[sous[0]]) if len(sous) else np.nan

for nom, g in clients.groupby("offre_bienvenue"):
    t, s = kaplan_meier(g["duree_mois"], g["churn"])
    print(f"offre = {nom} : médiane de survie = {survie_mediane(t, s):.1f} mois ; "
          f"survie à 12 mois = {survie_en(12, t, s):.3f} ; à 24 mois = {survie_en(24, t, s):.3f} ; à
↪ 36 mois = {survie_en(36, t, s):.3f}")
```

```
offre = 0 : médiane de survie = 28.1 mois ; survie à 12 mois = 0.790 ; à 24 mois = 0.573 ; à 36
↪ mois = 0.393
offre = 1 : médiane de survie = 36.6 mois ; survie à 12 mois = 0.858 ; à 24 mois = 0.691 ; à 36
↪ mois = 0.509
```

Vérifions notre implémentation contre celle de `statsmodels`, puis ajustons le **modèle de Cox** (section 5.3), qui estime l'effet de plusieurs variables à la fois sur le risque de départ :

```

from statsmodels.duration.survfunc import SurvfuncRight
g1 = clients[clients["offre_bienvenue"] == 1]
sf = SurvfuncRight(g1["duree_mois"], g1["churn"])
t_h, s_h = kaplan_meier(g1["duree_mois"], g1["churn"])
ecarts = [abs(survie_en(t, t_h, s_h) - float(sf.surv_prob[max(np.searchsorted(sf.surv_times, t,
↪ side="right") - 1, 0]))
    for t in range(6, 61, 6)]
print(f"écart maximal avec statsmodels sur la grille 6, 12, ..., 60 mois : {max(ecarts):.1e}")

from statsmodels.duration.hazard_regression import PHReg
exog = pd.get_dummies(clients[["offre_bienvenue", "age", "canal_acquisition"]],
↪ columns=["canal_acquisition"], drop_first=True, dtype=float)
cox = PHReg(clients["duree_mois"], exog, status=clients["churn"]).fit()
resume = pd.DataFrame({"coef": cox.params, "rapport_de_risques": np.exp(cox.params),
    "IC95_bas": np.exp(cox.params - 1.96 * cox.bse), "IC95_haut": np.exp(cox.params
↪ + 1.96 * cox.bse),
    "p": cox.pvalues}, index=exog.columns).round(3)
print(resume.to_string())

```

```

écart maximal avec statsmodels sur la grille 6, 12, ..., 60 mois : 2.2e-16

```

	coef	rapport_de_risques	IC95_bas	IC95_haut	p
offre_bienvenue	-0.406	0.666	0.587	0.756	0.0
age	-0.014	0.986	0.981	0.992	0.0
canal_acquisition_Réseaux	0.635	1.887	1.600	2.226	0.0
canal_acquisition_Site	0.326	1.385	1.164	1.648	0.0

Un **rapport de risques** inférieur à 1 signifie que la variable *réduit* le risque instantané de départ. L'offre de bienvenue réduit le risque de départ d'environ un tiers ; les clients arrivés par les réseaux sociaux ou par le site partent plus vite que ceux de la boutique (référence).

P.7 Étape 6 : que vaut un client, et l'offre est-elle rentable ?

Assemblons les pièces. La **valeur d'un client** sur un horizon donné est la somme de ce qu'il dépensera tant qu'il reste, **actualisée** (un euro dans cinq ans vaut moins qu'un euro aujourd'hui). Si A est la **marge** annuelle (la part de la dépense qui reste après le coût des produits : c'est elle qui paie l'offre, pas le chiffre d'affaires) et $S(t)$ la probabilité de rester au moins t ans, alors, avec un taux d'actualisation δ :

$$\text{valeur} = A \int_0^{\tau} S(t) e^{-\delta t} dt$$

L'intégrale est la **durée de vie moyenne restreinte actualisée** : le nombre d'années pondérées qu'un client passe chez nous dans l'horizon τ . Nous prenons $\tau = 5$ ans et $\delta = 8\%$ (hypothèses de calcul, à discuter avec la gérante), un taux de marge brute de 40 % (hypothèse de calcul, à vérifier avec la comptabilité), appliqué à la dépense annuelle moyenne prévue par le modèle de Tweedie.

```

def duree_actualisee(instants, courbe, horizon_mois=60, taux=0.08):
    """Intègre S(t) e^(-taux t) sur [0, horizon] (t en années), par la méthode des rectangles sur une
    ↪ grille fine."""
    grille = np.linspace(0, horizon_mois, 6001)
    S = np.array([survie_en(t, instants, courbe) for t in grille])
    f = S * np.exp(-taux * grille / 12)
    return float(np.sum((f[:-1] + f[1:]) / 2 * np.diff(grille)) / 12)

depense_annuelle = tweedie.predict(clients).mean()
taux_marge = 0.40
marge_annuelle = taux_marge * depense_annuelle
cout_offre = 10.0

```

```

resultats = {}
for nom, g in clients.groupby("offre_bienvenue"):
    t, s = kaplan_meier(g["duree_mois"], g["churn"])
    d = duree_actualisee(t, s)
    resultats[nom] = d
    print(f"offre = {nom} : durée de vie actualisée sur 5 ans = {d:.2f} années ; valeur du client =
    ↳ {marge_annuelle * d:.0f} €")
gain = marge_annuelle * (resultats[1] - resultats[0])
print(f"\ngain brut de l'offre : {gain:.0f} € par client ; coût : {cout_offre:.0f} € ; gain net :
    ↳ {gain - cout_offre:.0f} € par client")

```

```

offre = 0 : durée de vie actualisée sur 5 ans = 2.23 années ; valeur du client = 220 €
offre = 1 : durée de vie actualisée sur 5 ans = 2.65 années ; valeur du client = 262 €

```

```

gain brut de l'offre : 42 € par client ; coût : 10 € ; gain net : 32 € par client

```

Le gain net est-il vraiment positif, ou le hasard de l'échantillon pourrait-il l'expliquer ? Le **bootstrap** donne une réponse : on rééchantillonne les clients, on recalcule tout, et on regarde la dispersion du gain net.

```

def gain_net(df):
    d = {}
    for nom, g in df.groupby("offre_bienvenue"):
        t, s = kaplan_meier(g["duree_mois"], g["churn"])
        d[nom] = duree_actualisee(t, s)
    return marge_annuelle * (d[1] - d[0]) - cout_offre

rng = np.random.default_rng(99)
sim = np.array([gain_net(clients.sample(len(clients), replace=True, random_state=int(s))) for s in
    ↳ rng.integers(0, 2**31 - 1, 200)])
gain_bas, gain_haut = np.percentile(sim, [2.5, 97.5])
print(f"gain net par client : {gain_net(clients):.0f} € IC95 bootstrap (200 rééchantillons) :
    ↳ [{gain_bas:.0f} ; {gain_haut:.0f}] €")
print(f"part des rééchantillons où l'offre est rentable : {np.mean(sim > 0):.2f}")

```

```

gain net par client : 32 € IC95 bootstrap (200 rééchantillons) : [18 ; 44] €
part des rééchantillons où l'offre est rentable : 1.00

```

P.8 Le rapport pour la gérante, et ce qui avait été programmé

Comme dans le projet du volume I, le rapport est **généré** à partir des résultats déjà calculés, sans aucun nombre recopié à la main.

```

def fr(x, d=0):
    return f"{x:,.{d}f}".replace(",", " ").replace(".", ",")

t_unique = ventes.loc["2025", "ca"].sum()
rapport = f"""\nPLAN 2026 : LA BOUTIQUE
{"=" * 60}

1. Ventes 2026
- Chiffre d'affaires prévu : {fr(total_2026)} € (2025 : {fr(t_unique)} €, soit {100 * (total_2026 /
    ↳ t_unique - 1):+.1f} %).
- Mois le plus fort : décembre ({fr(prevision['prévision'].iloc[-1])} €) ; le plus faible :
    ↳ {prevision['prévision'].idxmin()[-2:]/2026}.
- Précision attendue : erreur moyenne d'environ {mape(ventes['ca'][test],
    ↳ np.exp(ajustes[meilleur].get_forecast(24, exog=X[test]).predicted_mean):.0f} % mois par mois
    ↳ lors du rétro-test.

2. L'offre de bienvenue
- Elle augmente la probabilité de racheter de {100 * ame:.0f} points (intervalle à 95 % : de {100 *
    ↳ ame_bas:.0f} à {100 * ame_haut:.0f} points).

```

```

- Elle réduit le risque de départ d'environ {100 * (1 - np.exp(cox.params[0])):.0f} %.
- Elle ne change pas la dépense annuelle d'un client.

3. Valeur d'un client (horizon 5 ans, actualisé à 8 %)
- Dépense annuelle moyenne : {fr(depense_annuelle)} € (marge supposée de 40 %) ; clients du canal
↳ Boutique : plus rentables que ceux du canal Réseaux.
- Gain net de l'offre : {fr(gain_net(clients))} € par client (intervalle à 95 % : de {fr(gain_bas)}
↳ à {fr(gain_haut)} €).

Hypothèses et limites : dépense annuelle supposée constante tant que le client reste ; marge brute de
↳ 40 % ; coût de l'offre de 10 € ;
pas de nouvel arrêt d'activité en 2026 ; données d'une seule boutique.
"""
print(rapport)

```

PLAN 2026 : LA BOUTIQUE

```

=====

1. Ventes 2026
- Chiffre d'affaires prévu : 32 029 € (2025 : 27 630 €, soit +15.9 %).
- Mois le plus fort : décembre (4 599 €) ; le plus faible : 01/2026.
- Précision attendue : erreur moyenne d'environ 7 % mois par mois lors du rétro-test.

2. L'offre de bienvenue
- Elle augmente la probabilité de racheter de 12 points (intervalle à 95 % : de 8 à 16 points).
- Elle réduit le risque de départ d'environ 33 %.
- Elle ne change pas la dépense annuelle d'un client.

3. Valeur d'un client (horizon 5 ans, actualisé à 8 %)
- Dépense annuelle moyenne : 247 € (marge supposée de 40 %) ; clients du canal Boutique : plus
↳ rentables que ceux du canal Réseaux.
- Gain net de l'offre : 32 € par client (intervalle à 95 % : de 18 à 44 €).

Hypothèses et limites : dépense annuelle supposée constante tant que le client reste ; marge brute de
↳ 40 % ; coût de l'offre de 10 € ;
pas de nouvel arrêt d'activité en 2026 ; données d'une seule boutique.

```

Relisez ce rapport comme la gérante : aucun mot technique ne devrait la gêner. L'annexe technique, c'est le reste de ce projet.

Ce qui avait été programmé

Les données étant simulées, nous pouvons maintenant comparer ce que nos modèles ont **retrouvé** à ce que le générateur contenait (script `build/donnees2.py`) :

Quantité	Valeur programmée	Estimée ici
Effet de l'offre sur le logit du rachat	+0,55	voir P.4 ($\approx +0,49$, intervalle compatible)
Effet du canal Réseaux sur le logit du rachat (réf. Boutique)	-0,30	voir P.4 ($\approx -0,46$, intervalle compatible)
Effet de l'offre sur la dépense (log)	0	voir P.5 ($\approx -0,01$)
Forme de Weibull de la durée de relation	1,35	non estimée ici (chapitre 5, section 5.4)
Tendance de la série (log, par mois)	+0,0075	cohérente avec la croissance de 2016 à 2025

Pourquoi les estimations ne sont-elles pas exactement les valeurs programmées ? Deux raisons, qui valent bien au-delà de ce projet. D'abord le **hasard d'échantillonnage** : les intervalles de confiance de P.4 contiennent les valeurs programmées (par exemple, celui de l'effet d'Réseaux contient $-0,30$), ce qui est exactement ce que la théorie promet. Ensuite, la **variabilité non observée** : le générateur fait dépendre le rachat de deux « goûts » latents (produits et service, ceux que révélera l'analyse factorielle du chapitre 3) que notre modèle ne contient pas. Omettre des variables qui influencent la réponse **atténue** les coefficients d'une régression logistique, même si ces variables n'ont aucun lien avec l'offre : l'effet estimé de l'offre ($+0,49$) est un peu inférieur au $+0,55$ programmé. Ce phénomène, la *non-collapsibilité* du rapport de cotes (chapitre 2, section 2.2), n'existe pas en régression linéaire. Dans la vie réelle, on n'a jamais la valeur programmée pour comparer : il faut connaître le piège.

P.9 Un détour par de vraies données

Les modèles ci-dessus ont été testés sur des données simulées. Une dernière vérification : ces méthodes fonctionnent-elles sur des **données réelles** ? Deux jeux sont embarqués dans les bibliothèques, donc disponibles hors ligne : la concentration de CO_2 dans l'atmosphère mesurée à Mauna Loa (Hawaï), et une étude de récidive de détenus libérés (le jeu « Rossi », étudié en analyse de survie).

```
co2 = sm.datasets.co2.load_pandas().data["co2"].resample("MS").mean().interpolate()
yc = np.log(co2)
m = sm.tsa.SARIMAX(yc[:"1999-12"], order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(dispatch=False)
f = np.exp(m.get_forecast(len(yc["2000-01":])).predicted_mean)
reel = co2["2000-01":]
print(f"C02 : {len(co2)} mois ({co2.index[0]:%Y-%m} à {co2.index[-1]:%Y-%m}), prévision hors
↳ échantillon de {len(reel)} mois")
print(f"erreur moyenne en pourcentage : {mape(reel.values, f.values):.2f} %")
print(f"naïf saisonnier : {mape(reel.values[12:], co2.shift(12)['2000-01':].values[12:]):.2f} %")
```

```
C02 : 526 mois (1958-03 à 2001-12), prévision hors échantillon de 24 mois
erreur moyenne en pourcentage : 0.13 %
naïf saisonnier : 0.40 %
```

Même méthode, même code qu'en P.3 : un SARIMA saisonnier $(1,1,1)(0,1,1)_{12}$ sur la série en logarithme, entraîné sur les données jusqu'en 1999 et évalué sur la suite.

Le jeu Rossi : 432 détenus libérés ont été suivis un an ; l'événement est une nouvelle arrestation (arrest), les durées sont en semaines (week), censurées à 52 semaines pour ceux qui n'ont pas été réarrêtés.

```
from lifelines.datasets import load_rossi
rossi = load_rossi()
X_r = rossi[["fin", "age", "race", "wexp", "mar", "paro", "prio"]].astype(float)
cox_r = PHReg(rossi["week"], X_r, status=rossi["arrest"]).fit()
res_r = pd.DataFrame({"rapport_de_risques": np.exp(cox_r.params), "p": cox_r.pvalues},
↳ index=X_r.columns).round(3)
print(f"{len(rossi)} détenus, {int(rossi['arrest'].sum())} réarrestations observées ({100 * (1 -
↳ rossi['arrest'].mean()):.0f} % de censure)")
print(res_r.to_string())
```

```
432 détenus, 114 réarrestations observées (74 % de censure)
   rapport_de_risques      p
fin                0.685  0.048
age                0.944  0.009
race              1.369  0.308
wexp              0.860  0.476
mar               0.649  0.257
```

paro	0.919	0.664
prio	1.095	0.001

Les rapports de risques se lisent comme en P.6 : l'aide financière (fin) est associée à un risque de nouvelle arrestation inférieur d'environ 31 % (rapport de risques de 0,685, $p = 0,048$, à la limite de la significativité), chaque année d'âge supplémentaire à un risque inférieur de 5,6 %, et chaque condamnation antérieure (prio) à un risque supérieur d'environ 9,5 %.

Lire avec prudence. Dans cette étude, seule l'aide financière (fin) a été attribuée **au hasard** ; les autres variables (âge, antécédents, mariage...) sont seulement observées. Seul l'effet de l'aide se lit donc causalement ; les autres coefficients décrivent des associations. Cet exemple est ici pour montrer que le code vu dans ce volume fonctionne tel quel sur des données réelles, pas pour tirer des conclusions de politique pénale.

P.10 Limites, et la suite

- **Une boutique, un jeu simulé.** La vraie vie a des variables oubliées, des erreurs de saisie, et des phénomènes que nous n'avons pas programmés.
- **Des hypothèses fortes dans la valeur client :** dépense constante dans le temps, taux de marge (40 %) et taux d'actualisation (8 %) fixés, coût de l'offre (10 €) connu. La rentabilité de l'offre dépend de ces choix : une étude sérieuse ferait varier ces hypothèses (analyse de sensibilité).
- **La prévision est conditionnelle.** Elle suppose que le futur ressemble au passé, y compris pour les variables exogènes que nous avons dû fixer (promo, covid).
- **Prédire n'est pas expliquer.** L'AUC modeste de P.4 le rappelle : un modèle peut estimer correctement un effet moyen sans bien prédire chaque cas.

À retenir. Une étude de modélisation complète enchaîne : *contrôler* → *regarder* → *modéliser* → *vérifier le modèle hors échantillon* → *quantifier l'incertitude* → *traduire en décision* → *reconnaître les limites*. Le volume III, consacré à l'apprentissage automatique, reprend ce cycle avec d'autres modèles, plus flexibles, pour la **prédiction**, et la même exigence : savoir de combien l'on se trompe.

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections du livre à relire sont indiquées dans les réponses. Trente bonnes réponses sur les trente-six premières signalent un volume bien assimilé.

Régression linéaire (chapitre 1)

1. Que sont les équations normales, et que représente géométriquement la solution des moindres carrés ?
2. Dans une régression de $\ln(\text{panier})$ sur le canal, le coefficient de « Boutique » (par rapport à « Réseaux ») vaut 0,22. De combien de pourcents le panier est-il plus élevé en boutique ?
3. Quelle est la différence entre un intervalle de confiance pour la **réponse moyenne** et un intervalle de **prédiction** ? Lequel est le plus large, et pourquoi ?

4. Deux variables explicatives ont une corrélation de 0,95. Quel est le facteur d'inflation de la variance (VIF) de chacune, et que cela signifie-t-il ?
5. Pourquoi ne faut-il pas choisir le modèle qui a le plus grand R^2 ?
6. Lequel de Ridge et de Lasso peut annuler exactement des coefficients, et pourquoi ?

Modèles linéaires généralisés (chapitre 2)

7. Quels sont les trois ingrédients d'un GLM ?
8. Une régression logistique donne, pour l'offre de bienvenue, un coefficient de 0,49. Le taux de rachat sans offre est de 44,8 %. Quel est le taux avec offre, selon le modèle ?
9. Qu'est-ce que la surdispersion d'un comptage, et que faire ?
10. Pourquoi une régression Gamma à lien logarithmique convient-elle à des montants positifs ?
11. Que mesure la déviance, et comment compare-t-on deux modèles emboîtés ?
12. Vos dépenses annuelles contiennent 13 % de zéros exacts et une partie positive asymétrique. Citez deux modèles adaptés.

Analyse multivariée (chapitre 3)

13. Les valeurs propres de la matrice de corrélation de quatre variables sont 2,4, 1, 0,4 et 0,2. Quelle part de la variance la première composante résume-t-elle ?
14. Quand faut-il standardiser les variables avant une ACP ?
15. Quelle différence de nature entre l'ACP et l'analyse factorielle ?
16. Comment choisir le nombre de composantes ou de facteurs ?
17. Que minimise l'algorithme des k-means, et pourquoi le lance-t-on plusieurs fois ?
18. Pourquoi une silhouette élevée ne suffit-elle pas à prouver qu'il y a des groupes ?

Séries temporelles (chapitre 4)

19. Qu'est-ce qu'une série faiblement stationnaire ?
20. Quelle est l'allure de l'ACF d'un AR(1) avec $\varphi = 0,8$? Quelle est sa valeur au retard 3 ?
21. Dans un test de Dickey-Fuller augmenté, quelle est l'hypothèse nulle, et que conclut-on d'une p-valeur de 0,40 ?
22. Pourquoi ne peut-on pas comparer par l'AIC un modèle différencié et un modèle qui ne l'est pas ?
23. Qu'est-ce qu'un rétro-test (*rolling origin*) et pourquoi compare-t-on toujours à un repère naïf ?
24. Un modèle SARIMA a des résidus dont la statistique de Ljung-Box donne $p = 0,002$. Que faire ?

Analyse de survie (chapitre 5)

25. Pourquoi la durée moyenne calculée sur les seuls clients partis est-elle biaisée ?
26. Un client part à taux constant de 0,05 par mois. Quelle est la durée médiane de la relation ?
27. Cinq clients ont les durées observées 2, 3⁺, 5, 7, 8⁺ mois (⁺ : censuré). Calculez à la main l'estimateur de Kaplan-Meier à 2, 5 et 7 mois.
28. Un rapport de risques de 0,67 pour l'offre de bienvenue : que signifie-t-il, et quelle hypothèse suppose le modèle de Cox ?
29. Quelle est l'hypothèse nulle du test du log-rank ?
30. Pourquoi « 1 – Kaplan-Meier » surestime-t-il l'incidence d'une cause en présence de risques concurrents ?

Statistique bayésienne et simulation (chapitre 6)

31. Prior Beta(1, 1), puis 12 rachats sur 20 clients : quelle est la loi a posteriori, et sa moyenne ?
32. Différence entre un intervalle de crédibilité à 95 % et un intervalle de confiance à 95 % ?
33. Un écart-type de simulation de 0,5 : combien de tirages Monte-Carlo pour que l'erreur-type de la moyenne soit de 0,001 ?
34. Dans Metropolis-Hastings, avec une proposition symétrique, quelle est la probabilité d'accepter un candidat x' depuis x ? Pourquoi la constante de normalisation de la loi cible n'est-elle pas nécessaire ?
35. Quatre chaînes MCMC donnent un $\hat{R}$ de 1,4. Que faire ?
36. Qu'est-ce qu'une vérification prédictive a posteriori ?

Chapitres facultatifs (7, 8, 9)

37. Faut-il ajuster sur une cause commune ? Sur un effet commun (collision) ? Pourquoi ?
38. Pourquoi la randomisation permet-elle une lecture causale de l'écart de moyennes ?
39. Une variable instrumentale (un rappel envoyé au hasard) augmente la dépense moyenne de 3 € et la probabilité d'ouvrir le courriel de 0,6. Quel est l'estimateur de Wald ?
40. Trois groupes de 10 observations : $SC_{\text{inter}} = 24$ et $SC_{\text{intra}} = 60$. Quelle est la statistique F de l'ANOVA ?
41. Combien d'essais faut-il pour un plan factoriel complet à trois facteurs à deux niveaux, et que calcule-t-on pour l'effet principal d'un facteur ?
42. Un indice de Moran de +0,4 avec $n = 50$: que cela indique-t-il, et quelle est son espérance sous l'indépendance spatiale ?

Corrigés des questions

Vérification des réponses chiffrées

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

Chapitres 1 et 2 :

```
import numpy as np
from scipy import stats

# Q2 : coefficient d'un modèle en logarithme
print(f"Q2  exp(0,22) - 1 = {100 * (np.exp(0.22) - 1):.1f} %")

# Q4 : VIF pour deux variables de corrélation 0,95
r = 0.95
print(f"Q4  VIF = 1 / (1 - r^2) = {1 / (1 - r**2):.2f}")

# Q8 : rapport de cotes -> probabilité
p0, coef = 0.448, 0.49
cotes = p0 / (1 - p0) * np.exp(coef)
print(f"Q8  rapport de cotes = {np.exp(coef):.3f} ; probabilité avec offre = {cotes / (1 + cotes):.3f}")
```

```
Q2  exp(0,22) - 1 = 24.6 %
Q4  VIF = 1 / (1 - r^2) = 10.26
Q8  rapport de cotes = 1.632 ; probabilité avec offre = 0.570
```

Chapitres 3 et 4 :

```
# Q13 : part de variance de la première composante (matrice de corrélation : somme des valeurs propres
↪ = nombre de variables)
vp = np.array([2.4, 1.0, 0.4, 0.2])
print(f"Q13 {vp[0] / vp.sum():.0%} de la variance")
```

```
# Q20 : ACF de l'AR(1)
print("Q20 ACF aux retards 1, 2, 3 :", [round(0.8**k, 3) for k in (1, 2, 3)])
```

Q13 60% de la variance
Q20 ACF aux retards 1, 2, 3 : [0.8, 0.64, 0.512]

Chapitre 5 :

```
# Q26 : médiane d'une durée exponentielle
print(f"Q26 ln(2) / 0,05 = {np.log(2) / 0.05:.2f} mois")

# Q27 : Kaplan-Meier à la main
durees = np.array([2, 3, 5, 7, 8]); evenements = np.array([1, 0, 1, 1, 0])
S = 1.0
for t in sorted(durees[evenements == 1]):
    a_risque = np.sum(durees >= t)
    S *= 1 - 1 / a_risque
    print(f"Q27 t = {t} : {a_risque} à risque, S = {S:.4f}")
```

Q26 $\ln(2) / 0,05 = 13.86$ mois
Q27 t = 2 : 5 à risque, S = 0.8000
Q27 t = 5 : 3 à risque, S = 0.5333
Q27 t = 7 : 2 à risque, S = 0.2667

Chapitres 6 à 9 :

```
# Q31 : bêta-binomiale
a, b = 1 + 12, 1 + 8
print(f"Q31 posteriori Beta({a}, {b}), moyenne = {a / (a + b):.3f}, IC crédible 95 % =
↪ [{stats.beta.ppf(0.025, a, b):.3f} ; {stats.beta.ppf(0.975, a, b):.3f}]" )

# Q33 : taille de simulation
print(f"Q33 n = (0,5 / 0,001)^2 = {(0.5 / 0.001) ** 2:,.0f}")

# Q39 : Wald
print(f"Q39 3 / 0,6 = {3 / 0.6:.1f} €")

# Q40 : F de l'ANOVA
k, n = 3, 30
F = (24 / (k - 1)) / (60 / (n - k))
print(f"Q40 F = {F:.2f}, p = {stats.f.sf(F, k - 1, n - k):.4f}")

# Q42 : espérance de l'indice de Moran
print(f"Q42 E[I] = -1/(n-1) = {-1 / 49:.4f}")
```

Q31 posteriori Beta(13, 9), moyenne = 0.591, IC crédible 95 % = [0.384 ; 0.782]
Q33 n = $(0,5 / 0,001)^2 = 250,000$
Q39 $3 / 0,6 = 5.0$ €
Q40 F = 5.40, p = 0.0106
Q42 E[I] = $-1/(n-1) = -0.0204$

Réponses

1. Les **équations normales** $\mathbf{X}^\top \mathbf{X} \beta = \mathbf{X}^\top \mathbf{y}$ expriment que le résidu est **orthogonal** à toutes les colonnes de $\mathbf{X}$. Géométriquement, $\mathbf{X} \hat{\beta}$ est la **projection orthogonale** de $\mathbf{y}$ sur le sous-espace engendré par les colonnes de $\mathbf{X}$. (1.1)
2. $e^{0,22} - 1 \approx 24,6\%$: en boutique, le panier est environ **un quart plus élevé**, toutes choses égales par ailleurs. Dans un modèle en logarithme, un coefficient β se lit comme un effet **multiplicatif** e^β , et non comme une différence en euros. (1.1)
3. L'intervalle sur la **réponse moyenne** encadre la valeur moyenne de y pour des valeurs données de x ; l'intervalle de **prédiction** encadre une **nouvelle observation** individuelle. Le second est plus large : il ajoute la variance du bruit individuel σ^2 à l'incertitude sur la moyenne. (1.2)
4. $\text{VIF} = 1/(1 - r^2) \approx 10,26$: la variance du coefficient est multipliée par plus de dix à cause de la colinéarité. On interprète mal chaque coefficient séparément, même si les prédictions restent correctes. (1.3)
5. Le R^2 **ne peut qu'augmenter** quand on ajoute des variables, même du bruit pur : il récompense le sur-ajustement. On choisit avec l'AIC, le BIC, le R^2 ajusté, ou, mieux, une **validation croisée** sur des données non utilisées pour l'ajustement. (1.4)
6. Le **Lasso** (pénalité ℓ_1), parce que la géométrie de la contrainte $\sum |\beta_j| \leq t$ a des **coins** sur les axes : la solution tombe souvent dessus. Ridge (pénalité ℓ_2 , contrainte sphérique) rétrécit tous les coefficients sans jamais les annuler exactement. (1.5)
7. Une **loi** de la famille exponentielle pour la réponse, un **prédicteur linéaire** $\eta = \mathbf{x}^\top \beta$, et une **fonction de lien** g qui relie la moyenne au prédicteur : $g(\mu) = \eta$. (2.1)
8. Environ **57 %** : les cotes sans offre valent $0,448/0,552 \approx 0,81$, multipliées par $e^{0,49} \approx 1,63$, puis reconverties en probabilité. Le rapport de cotes de 1,63 n'est **pas** un rapport de probabilités : une cote multipliée par 1,63 ne multiplie pas la probabilité par 1,63. (2.2)
9. Il y a **surdispersion** quand la variance observée dépasse la moyenne, alors que la loi de Poisson impose variance = moyenne. Les erreurs-types sont alors trop optimistes. On passe à une loi **binomiale négative**, ou à un Poisson avec erreurs-types robustes (quasi-vraisemblance). (2.3)
10. Les montants sont **strictement positifs** et leur variabilité **croît avec le niveau** (écart-type proportionnel à la moyenne), ce que fait la loi Gamma ($\text{Var} = \phi \mu^2$). Le lien logarithmique garantit des moyennes positives et donne des effets **multiplicatifs**. (2.3)
11. La **déviance** est $2(\ell_{\text{saturé}} - \ell_{\text{modèle}})$: l'écart de vraisemblance au modèle parfait. Pour deux modèles **emboîtés**, la différence de déviances suit approximativement une loi du χ^2 dont les degrés de liberté valent le nombre de paramètres en plus (test du rapport de vraisemblance). (2.4)
12. Un modèle de **Tweedie** (avec $1 < p < 2$), qui mêle masse en zéro et partie positive continue ; ou un **modèle en deux parties** (logistique pour « dépense nulle ou non », puis Gamma sur les dépenses positives). (2.6)
13. $2,4/4 = 60\%$ (la somme des valeurs propres d'une matrice de corrélation vaut le nombre de variables). (3.1)
14. Quand les variables ont des **unités ou des échelles différentes** (euros, âges, notes) : sans standardisation, la variable de plus grande variance domine l'ACP. Avec des variables de même nature et de même échelle, on peut travailler sur la matrice de covariance. (3.1)
15. L'ACP **résume** : elle cherche les combinaisons de variables de variance maximale, sans modèle. L'analyse factorielle **modélise** : elle suppose que les corrélations viennent de facteurs cachés, avec

une part de bruit propre à chaque variable ($\Sigma = \Lambda\Lambda^T + \Psi$). (3.2)

16. Éboulis des valeurs propres (le coude), critère de Kaiser (valeurs propres > 1 , à manier avec prudence), et surtout **analyse parallèle** (comparaison à des données sans structure), ajoutés à l'interprétabilité. En analyse factorielle, on dispose en plus d'un **test d'ajustement**. (3.1 et 3.2)

17. La **somme des carrés intra-classes** (inertie intra). L'algorithme converge vers un **minimum local** qui dépend de l'initialisation : on le lance plusieurs fois (avec k-means++) et on garde le meilleur. (3.3)

18. Parce qu'un nuage **sans structure** mais asymétrique peut aussi obtenir une silhouette élevée : une méthode de classification rend toujours des groupes. Il faut comparer à une référence sans groupes et tester la **stabilité** (rééchantillonnage, indice de Rand ajusté). (3.3)

19. Une série dont l'**espérance** est constante, la **variance** constante et dont l'**autocovariance** ne dépend que du décalage entre les dates, pas des dates elles-mêmes. (4.1)

20. Une décroissance **géométrique** : $\rho(k) = \varphi^k$, soit 0,8, 0,64 et 0,512 aux retards 1, 2 et 3. (4.1 et 4.2)

21. L'hypothèse nulle est la **présence d'une racine unitaire** (non-stationnarité). Une p-valeur de 0,40 ne permet pas de la rejeter : la série est compatible avec une marche aléatoire ; on la différencie. (« Ne pas rejeter » n'est pas « prouver ».) (4.1)

22. Parce que la vraisemblance porte sur **des données différentes** : la série différenciée a moins d'observations et une autre échelle. L'AIC ne se compare qu'entre modèles ajustés **à la même série**. Pour arbitrer entre ordres de différenciation, on compare des **prévisions hors échantillon**. (4.2 et 4.3)

23. On prévoit à plusieurs **origines successives** : on ajuste sur le passé jusqu'à t , on prévoit $t + 1, \dots, t + h$, on avance t et on recommence, pour mesurer des erreurs sur des données jamais vues. Le **repère naïf** (la dernière valeur, ou la même saison l'an passé) fixe le seuil à battre : un modèle sophistiqué qui ne le bat pas ne sert à rien. (4.3)

24. Une p-valeur de 0,002 signale une **autocorrélation résiduelle** : le modèle a laissé du signal. On ajoute des termes AR/MA ou saisonniers, on traite une rupture ou un choc non modélisé, puis on relance les diagnostics. (4.2)

25. Parce qu'on ne regarde que les clients **partis tôt** : les clients fidèles, qui restent encore au moment de l'analyse, sont exclus alors que ce sont eux qui ont les durées les plus longues. La moyenne est donc sous-estimée. (5.1)

26. Pour un risque constant λ , $S(t) = e^{-\lambda t}$, et la médiane vaut $\ln 2/\lambda \approx 13,86$ mois. (5.1)

27. À 2 mois, 5 clients à risque, 1 départ : $S = 0,8$. À 5 mois, il reste 3 clients à risque (5, 7 et 8+), 1 départ : $S = 0,8 \times \frac{2}{3} \approx 0,5333$. À 7 mois, il en reste 2 à risque, 1 départ : $S = 0,5333 \times \frac{1}{2} \approx 0,2667$. Le client censuré à 3 mois n'est plus à risque après, mais **il a compté** dans le dénominateur jusqu'à sa sortie. (5.2)

28. Le risque instantané de départ des clients avec offre vaut **67 % de celui** des clients sans offre, à chaque instant, soit **33 % de moins**. Le modèle de Cox suppose les **risques proportionnels** : ce rapport est le même à toutes les dates. On le vérifie (graphique log-log, test de Grambsch-Therneau). (5.3)

29. Que les **fonctions de survie des groupes sont égales** à toutes les dates. (5.2)

30. Parce que « 1 – Kaplan-Meier » traite les départs pour les **autres causes** comme des censures, comme si ces clients allaient encore pouvoir partir pour la cause étudiée. Or ils ne le peuvent

plus : l'incidence est donc surestimée. On utilise l'estimateur d'**Aalen-Johansen** de l'incidence cumulée. (5.5)

31. Beta(13, 9) : on ajoute les succès à a et les échecs à b . La moyenne vaut $13/22 \approx 0,591$; l'intervalle de crédibilité à 95 % est donné par le code ci-dessus. (6.1)

32. L'intervalle de **crédibilité** dit : « étant donné les données et l'a priori, le paramètre a 95 % de chances d'être dans cet intervalle ». L'intervalle de **confiance** dit : « la méthode encadre la vraie valeur dans 95 % des échantillons possibles ». Le premier est une probabilité sur le paramètre, le second une propriété de la procédure. (6.1)

33. $(0,5/0,001)^2 = 250\,000$ tirages : l'erreur décroît en $1/\sqrt{n}$, donc pour la diviser par dix il faut cent fois plus de tirages. (6.2)

34. On accepte avec la probabilité $\min(1, \pi(x')/\pi(x))$. Le rapport $\pi(x')/\pi(x)$ **simplifie la constante de normalisation**, inconnue en général (c'est l'intégrale du produit vraisemblance $\times$ a priori) : elle apparaît au numérateur et au dénominateur. (6.3)

35. Un $\hat{R}$ de 1,4 (bien supérieur à 1,01) indique que les chaînes **n'ont pas convergé vers la même loi**. Ne pas utiliser les résultats : allonger les chaînes, revoir la paramétrisation, le pas de la proposition, les valeurs initiales, ou le modèle lui-même (multimodalité). (6.4)

36. On **simule des jeux de données** à partir du modèle ajusté (en tirant les paramètres dans leur loi a posteriori), et on regarde si les données observées ressemblent à ces répliques sur une statistique bien choisie (variance, nombre de zéros, maximum). Si les données réelles sortent de la distribution des répliques, le modèle ne reproduit pas un aspect important. (6.4)

37. Sur une **cause commune** (variable de confusion) : oui, on ajuste, pour bloquer le chemin de confusion. Sur un **effet commun** (collision) : **non**, car conditionner sur un effet commun **ouvre** un chemin artificiel entre ses deux causes et crée une association qui n'existe pas. (7.1)

38. Parce que, l'affectation étant faite au hasard, les groupes sont **comparables en moyenne sur tout**, y compris sur ce qu'on n'observe pas. L'écart de moyennes mesure alors uniquement l'effet du traitement : le **biais de sélection** est nul en espérance. (7.1)

39. $3/0,6 = 5$ € : l'effet du rappel sur la dépense (forme réduite) divisé par son effet sur l'ouverture du courriel (première étape). C'est l'effet moyen **pour les « complaisants »**, c'est-à-dire ceux dont le comportement change à cause du rappel. (7.4)

40. $F = \frac{24/2}{60/27} = 5,40$, avec 2 et 27 degrés de liberté ; la p-valeur est donnée par le code. (8.2)

41. $2^3 = 8$ essais. L'effet principal d'un facteur est la **différence entre la moyenne des réponses au niveau haut et la moyenne au niveau bas**, calculée sur les 4 essais de chaque niveau. (8.3)

42. Un indice **positif** indique une **autocorrélation spatiale positive** : des zones voisines se ressemblent plus que ne le voudrait le hasard. Son espérance sous indépendance est $-1/(n-1) = -1/49 \approx -0,0204$. On teste ensuite l'écart par **permutations**. (9.2)

Grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Écrire un modèle linéaire, l'estimer et démontrer ses propriétés	1.1
Interpréter les coefficients (indicatrices, logarithmes)	1.1

Compétence	Où la retravailler
Tester, calculer intervalles de confiance et de prédiction	1.2
Diagnostiquer un modèle (résidus, levier, colinéarité)	1.3
Choisir un modèle sans tricher	1.4
Régulariser, résister aux aberrations, modéliser des groupes	1.5, 1.6, 1.7
Choisir loi et lien d'un GLM, estimer et vérifier	2.1 à 2.4
Modéliser des zéros en excès, assouplir un effet	2.5, 2.6
Réduire la dimension (ACP, analyse factorielle)	3.1, 3.2
Classer sans étiquettes et vérifier qu'il y a des groupes	3.3
Diagnostiquer la stationnarité, lire une ACF	4.1
Ajuster un SARIMA et évaluer des prévisions	4.2, 4.3
Traiter des durées censurées (Kaplan-Meier, Cox)	5.1, 5.2, 5.3
Ajuster des modèles de durée paramétriques	5.4
Raisonner à la Bayes, choisir un a priori	6.1
Simuler (Monte-Carlo, MCMC) et diagnostiquer	6.2, 6.3, 6.4
Formuler une question causale et choisir une méthode	7.1 à 7.4
Concevoir une expérience, analyser un plan	8.1 à 8.4
Mesurer l'autocorrélation spatiale, krigeage	9.2, 9.3
Mener une étude de modélisation de bout en bout	Projet du volume